

DISCOVERING SUBGROUPS IN PREECLAMPSIA DATA THROUGH UNSUPERVISED LEARNING TECHNIQUES

- **GRACE PETER UDO**
- **SUPERVISED BY: EVA SOUSA**

Imports and set up

```
In [ ]: from google.colab import drive
import os

# Mount Google Drive
drive.mount('/content/drive')

# Path to your notebook in Drive
notebook_path = "/content/drive/MyDrive/Colab Notebooks/Copy of preeclampsia-1.i

# Check if the notebook exists
if os.path.exists(notebook_path):
    print(f"Notebook found: {notebook_path}")

    # Convert to HTML (keeps plots)
    os.system(f'jupyter nbconvert --to html "{notebook_path}"')

    # HTML file will be saved in the same folder as notebook
    html_path = notebook_path.replace('.ipynb', '.html')

    if os.path.exists(html_path):
        print(f"HTML successfully created: {html_path}")
        from google.colab import files
        files.download(html_path)
    else:
        print("Conversion failed: HTML file not found.")
else:
    print("Notebook not found! Check the path inside Drive.")
```

```
In [ ]: # All libraries used for the study
from pathlib import Path
import zipfile, json, re, warnings
import pandas as pd, shutil
import numpy as np
import seaborn as sns
import os, glob
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, silhouette_samples
from sklearn.metrics import adjusted_rand_score, adjusted_mutual_info_score
import hdbscan
from pathlib import Path
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_har
from scipy.stats import mannwhitneyu
import umap
import joblib
```

```
/usr/local/lib/python3.12/dist-packages/hdbscan/robust_single_linkage_.py:175: SyntaxWarning: invalid escape sequence '\{'
    $max \{ core_k(a), core_k(b), 1/\alpha d(a,b) \}$.
```

Mounting google drive

```
In [ ]: from google.colab import drive
        drive.mount('/content/drive')
```

Mounted at /content/drive

Installing and importing phenograph

```
In [ ]: !pip install phenograph --quiet
```

```
===== 0.0/159.6 kB ? eta -:--:--
===== 159.6/159.6 kB 4.8 MB/s eta 0:00:00
===== 2.7/2.7 MB 26.1 MB/s eta 0:00:00
===== 5.7/5.7 MB 35.8 MB/s eta 0:00:00
```

```
In [ ]: import phenograph
```

Importing fcsparser

```
In [ ]: !pip install fcsparser
```

```
Requirement already satisfied: fcsparser in /usr/local/lib/python3.12/dist-packages (0.2.8)
Requirement already satisfied: numpy<2,>=1 in /usr/local/lib/python3.12/dist-packages (from fcsparser) (1.26.4)
Requirement already satisfied: pandas>=1.5.3 in /usr/local/lib/python3.12/dist-packages (from fcsparser) (2.2.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.5.3->fcsparser) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.5.3->fcsparser) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.5.3->fcsparser) (2025.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=1.5.3->fcsparser) (1.17.0)
```

```
In [ ]: # fcs parser
        try:
            from fcsparser import parse as fcs_parse
        except Exception as e:
            raise RuntimeError("Please install fcsparser: pip install fcsparser") from e
```

DATA CLEANING AND EXPLORATION

Configuring paths

```
In [ ]: ZIP_PATH = Path("/content/drive/MyDrive/pre eclampsia/Copy of 10659650.zip")
        RAW_DIR = Path("data_raw")
        CLEAN_DIR = Path("data_clean") # will hold full cleaned dataset
        QC_DIR = Path("results_qc") # per-file QC summary tables
        for d in [RAW_DIR, CLEAN_DIR, QC_DIR]: d.mkdir(exist_ok=True, parents=True)

        # Arsinh cofactor for intensity reduction and seed setting for reproducibility
```

```
COFACTOR = 5.0
np.random.seed(42)
```

```
In [ ]: # Creating file paths for raw dataset/ using existing ones
if "ZIP_PATH" not in globals(): ZIP_PATH = Path("10659650.zip")
if "RAW_DIR" not in globals(): RAW_DIR = Path("data_raw")

RAW_DIR.mkdir(exist_ok=True, parents=True)
```

```
In [ ]: # Accessing and viewing the contents of the raw dataset in the zip folder
if not (RAW_DIR/".extracted.ok").exists():
    with zipfile.ZipFile(ZIP_PATH, "r") as zf:
        zf.extractall(RAW_DIR)
        (RAW_DIR/".extracted.ok").touch()

rows = []
for p in RAW_DIR.rglob("*"):
    if p.is_file():
        rows.append({
            "path": str(p.relative_to(RAW_DIR)),
            "ext": p.suffix.lower(),
            "size_MB": round(p.stat().st_size/1_000_000, 3)
        })
inventory_df = pd.DataFrame(rows).sort_values(["ext", "path"])
display(inventory_df.head(20))
print("Total files:", len(inventory_df))
inventory_df["ext"].value_counts()
```

	path	ext	size_MB
57	fcs_info.csv	.csv	0.002
31	marker.csv	.csv	0.000
56	test_labels_4.csv	.csv	0.001
19	test_labels_44.csv	.csv	0.001
65	test_labels_444.csv	.csv	0.001
37	test_labels_4444.csv	.csv	0.001
25	test_labels_44444.csv	.csv	0.001
59	train_labels_4.csv	.csv	0.001
51	train_labels_44.csv	.csv	0.001
27	train_labels_444.csv	.csv	0.001
13	train_labels_4444.csv	.csv	0.001
41	train_labels_44444.csv	.csv	0.001
18	PBMC_1052311088_Ctrl_unstim.fcs	.fcs	24.536
23	PBMC_1052311288_Ctrl_unstim.fcs	.fcs	29.597
58	PBMC_1052411118_Ctrl_unstim.fcs	.fcs	25.821
26	PBMC_1052411268_Ctrl_unstim.fcs	.fcs	60.607
24	PBMC_1054111098_Ctrl_unstim.fcs	.fcs	19.169
5	PBMC_1054111158_Ctrl_unstim.fcs	.fcs	43.133
1	PBMC_1054711108_Ctrl_unstim.fcs	.fcs	32.023
62	PBMC_1054711248_Ctrl_unstim.fcs	.fcs	25.596

Total files: 66

Out[]: **count**

ext	
.fcs	46
.csv	12
.json	5
.txt	2
.ok	1

dtype: int64

Sorting, cleaning, and viewing data contained in each file type

```
In [ ]: # Sorting and cleaning all CSV files
        csv_paths = sorted(RAW_DIR,rglob("*.csv"))
```



```

csv_dfs = {}

for p in csv_paths:
    print(f"{p.name}")
    try:
        df = pd.read_csv(p)
    except Exception:
        # let pandas sniff
        df = pd.read_csv(p, engine="python", sep=None)
    csv_dfs[p.name] = df
    print("shape:", df.shape)
    print("columns:", list(df.columns))
    print("NaNs per column:\n", df.isna().sum())
    display(df.head(5))

```

```

fcs_info.csv
shape: (46, 6)
columns: ['Unnamed: 0', 'file_id', 'condition', 'age', 'age_label', 'random1']
NaNs per column:
Unnamed: 0    0
file_id       0
condition     0
age           0
age_label     0
random1       0
dtype: int64

```

	Unnamed: 0	file_id	condition	age	age_label	random1
0	0	PBMC_1072111118_Ctrl_unstim.fcs	0	11	0	1
1	1	PBMC_1072111278_Ctrl_unstim.fcs	0	27	1	0
2	2	PBMC_1072311258_PreE_unstim.fcs	1	25	1	0
3	3	PBMC_1072311138_PreE_unstim.fcs	1	13	0	1
4	4	PBMC_1073611138_PreE_unstim.fcs	1	13	0	1

```

marker.csv
shape: (0, 33)
columns: ['115In_CD45', '139La_CD66', '141Pr_CD7', '142Nd_CD19', '143Nd_CD45RA',
'144Nd_CD11b', '145Nd_CD4', '146Nd_CD8a', '147Sm_CD11c', '148Nd_CD123', '149Sm_CR
EB', '150Nd_STAT5', '151Eu_p38', '152Sm_TCRgd', '153Eu_STAT1', '154Sm_STAT3', '15
5Gd_S6', '158Gd_CD33', '159Tb_MAPKAPK2', '160Gd_Tbet', '161Dy_cPARP', '164Dy_Ik
B', '165Ho_CD16', '166Er_NFkB', '167Er_GATA3', '168Er_pSTAT6', '169Tm_CD25', '170
Er_CD3', '171Yb_pERK', '172Yb_RORgd', '174Yb_HLADR', '175Lu_CD14', '176Yb_CD56']
NaNs per column:
  115In_CD45      0
  139La_CD66      0
  141Pr_CD7       0
  142Nd_CD19      0
  143Nd_CD45RA    0
  144Nd_CD11b     0
  145Nd_CD4       0
  146Nd_CD8a      0
  147Sm_CD11c     0
  148Nd_CD123     0
  149Sm_CREB      0
  150Nd_STAT5     0
  151Eu_p38       0
  152Sm_TCRgd     0
  153Eu_STAT1     0
  154Sm_STAT3     0
  155Gd_S6        0
  158Gd_CD33      0
  159Tb_MAPKAPK2  0
  160Gd_Tbet      0
  161Dy_cPARP     0
  164Dy_IkB       0
  165Ho_CD16      0
  166Er_NFkB      0
  167Er_GATA3     0
  168Er_pSTAT6    0
  169Tm_CD25      0
  170Er_CD3       0
  171Yb_pERK      0
  172Yb_RORgd     0
  174Yb_HLADR     0
  175Lu_CD14      0
  176Yb_CD56      0
dtype: int64

```

115In_CD45	139La_CD66	141Pr_CD7	142Nd_CD19	143Nd_CD45RA	144Nd_CD11b	14
------------	------------	-----------	------------	--------------	-------------	----

0 rows × 33 columns

```

test_labels_4.csv
shape: (20, 5)
columns: ['fcs_filename', 'condition', 'age', 'age_label', 'random1']
NaNs per column:
  fcs_filename      0
  condition         0
  age               0
  age_label         0
  random1           0
dtype: int64

```

	fcs_filename	condition	age	age_label	random1
0	PBMC_1072311258_PreE_unstim.fcs	1	25	1	0
1	PBMC_1072311138_PreE_unstim.fcs	1	13	0	1
2	PBMC_1970711248_Ctrl_unstim.fcs	0	24	1	0
3	PBMC_1970711128_Ctrl_unstim.fcs	0	12	0	1
4	PBMC_1063911168_PreE_unstim.fcs	1	16	1	0

test_labels_44.csv

shape: (20, 5)

columns: ['fcs_filename', 'condition', 'age', 'age_label', 'random1']

NaNs per column:

fcs_filename 0

condition 0

age 0

age_label 0

random1 0

dtype: int64

	fcs_filename	condition	age	age_label	random1
0	PBMC_1970711248_Ctrl_unstim.fcs	0	24	1	0
1	PBMC_1970711128_Ctrl_unstim.fcs	0	12	0	1
2	PBMC_1063911168_PreE_unstim.fcs	1	16	1	0
3	PBMC_1063911118_PreE_unstim.fcs	1	11	0	1
4	PBMC_1080411158_Ctrl_unstim.fcs	0	15	0	1

test_labels_444.csv

shape: (20, 5)

columns: ['fcs_filename', 'condition', 'age', 'age_label', 'random1']

NaNs per column:

fcs_filename 0

condition 0

age 0

age_label 0

random1 0

dtype: int64

	fcs_filename	condition	age	age_label	random1
0	PBMC_1063411248_PreE_unstim.fcs	1	24	1	0
1	PBMC_1063411118_PreE_unstim.fcs	1	11	0	1
2	PBMC_1970711248_Ctrl_unstim.fcs	0	24	1	0
3	PBMC_1970711128_Ctrl_unstim.fcs	0	12	0	1
4	PBMC_1063911168_PreE_unstim.fcs	1	16	1	0

test_labels_4444.csv

shape: (20, 5)

columns: ['fcs_filename', 'condition', 'age', 'age_label', 'random1']

NaNs per column:

```
fcs_filename    0
condition       0
age             0
age_label       0
random1         0
```

dtype: int64

	fcs_filename	condition	age	age_label	random1
0	PBMC_1063411248_PreE_unstim.fcs	1	24	1	0
1	PBMC_1063411118_PreE_unstim.fcs	1	11	0	1
2	PBMC_1970711248_Ctrl_unstim.fcs	0	24	1	0
3	PBMC_1970711128_Ctrl_unstim.fcs	0	12	0	1
4	PBMC_1054711108_Ctrl_unstim.fcs	0	10	0	0

test_labels_44444.csv

shape: (20, 5)

columns: ['fcs_filename', 'condition', 'age', 'age_label', 'random1']

NaNs per column:

```
fcs_filename    0
condition       0
age             0
age_label       0
random1         0
```

dtype: int64

	fcs_filename	condition	age	age_label	random1
0	PBMC_1072111118_Ctrl_unstim.fcs	0	11	0	1
1	PBMC_1072111278_Ctrl_unstim.fcs	0	27	1	0
2	PBMC_1072311258_PreE_unstim.fcs	1	25	1	0
3	PBMC_1072311138_PreE_unstim.fcs	1	13	0	1
4	PBMC_1054711108_Ctrl_unstim.fcs	0	10	0	0

train_labels_4.csv

shape: (20, 5)

columns: ['fcs_filename', 'condition', 'age', 'age_label', 'random1']

NaNs per column:

```
fcs_filename    0
condition       0
age             0
age_label       0
random1         0
```

dtype: int64

	fcs_filename	condition	age	age_label	random1
0	PBMC_1072111118_Ctrl_unstim.fcs	0	11	0	1
1	PBMC_1072111278_Ctrl_unstim.fcs	0	27	1	0
2	PBMC_1073611138_PreE_unstim.fcs	1	13	0	1
3	PBMC_1073611258_PreE_unstim.fcs	1	25	1	1
4	PBMC_1063411248_PreE_unstim.fcs	1	24	1	0

train_labels_44.csv

shape: (20, 5)

columns: ['fcs_filename', 'condition', 'age', 'age_label', 'random1']

NaNs per column:

fcs_filename 0

condition 0

age 0

age_label 0

random1 0

dtype: int64

	fcs_filename	condition	age	age_label	random1
0	PBMC_1072311258_PreE_unstim.fcs	1	25	1	0
1	PBMC_1072311138_PreE_unstim.fcs	1	13	0	1
2	PBMC_1073611138_PreE_unstim.fcs	1	13	0	1
3	PBMC_1073611258_PreE_unstim.fcs	1	25	1	1
4	PBMC_1063411248_PreE_unstim.fcs	1	24	1	0

train_labels_444.csv

shape: (20, 5)

columns: ['fcs_filename', 'condition', 'age', 'age_label', 'random1']

NaNs per column:

fcs_filename 0

condition 0

age 0

age_label 0

random1 0

dtype: int64

	fcs_filename	condition	age	age_label	random1
0	PBMC_1072111118_Ctrl_unstim.fcs	0	11	0	1
1	PBMC_1072111278_Ctrl_unstim.fcs	0	27	1	0
2	PBMC_1072311258_PreE_unstim.fcs	1	25	1	0
3	PBMC_1072311138_PreE_unstim.fcs	1	13	0	1
4	PBMC_1073611138_PreE_unstim.fcs	1	13	0	1

train_labels_4444.csv

shape: (20, 5)

columns: ['fcs_filename', 'condition', 'age', 'age_label', 'random1']

NaNs per column:

```
fcs_filename    0
condition       0
age             0
age_label       0
random1         0
```

dtype: int64

	fcs_filename	condition	age	age_label	random1
0	PBMC_1072111118_Ctrl_unstim.fcs	0	11	0	1
1	PBMC_1072111278_Ctrl_unstim.fcs	0	27	1	0
2	PBMC_1072311258_PreE_unstim.fcs	1	25	1	0
3	PBMC_1072311138_PreE_unstim.fcs	1	13	0	1
4	PBMC_1073611138_PreE_unstim.fcs	1	13	0	1

train_labels_44444.csv

shape: (20, 5)

columns: ['fcs_filename', 'condition', 'age', 'age_label', 'random1']

NaNs per column:

```
fcs_filename    0
condition       0
age             0
age_label       0
random1         0
```

dtype: int64

	fcs_filename	condition	age	age_label	random1
0	PBMC_1063411248_PreE_unstim.fcs	1	24	1	0
1	PBMC_1063411118_PreE_unstim.fcs	1	11	0	1
2	PBMC_1063911168_PreE_unstim.fcs	1	16	1	0
3	PBMC_1063911118_PreE_unstim.fcs	1	11	0	1
4	PBMC_1901711258_PreE_unstim.fcs	1	25	1	0

```
In [ ]: # Putting fcs csv content in a dataframe
fcs_info = csv_dfs["fcs_info.csv"]
fcs_info
```

Out[]:

	Unnamed: 0	file_id	condition	age	age_label	random1
0	0	PBMC_1072111118_Ctrl_unstim.fcs	0	11	0	1
1	1	PBMC_1072111278_Ctrl_unstim.fcs	0	27	1	0
2	2	PBMC_1072311258_PreE_unstim.fcs	1	25	1	0
3	3	PBMC_1072311138_PreE_unstim.fcs	1	13	0	1
4	4	PBMC_1073611138_PreE_unstim.fcs	1	13	0	1
5	5	PBMC_1073611258_PreE_unstim.fcs	1	25	1	1
6	6	PBMC_1063411248_PreE_unstim.fcs	1	24	1	0
7	7	PBMC_1063411118_PreE_unstim.fcs	1	11	0	1
8	8	PBMC_1970711248_Ctrl_unstim.fcs	0	24	1	0
9	9	PBMC_1970711128_Ctrl_unstim.fcs	0	12	0	1
10	10	PBMC_1054711108_Ctrl_unstim.fcs	0	10	0	0
11	11	PBMC_1054711248_Ctrl_unstim.fcs	0	24	1	1
12	12	PBMC_1063911168_PreE_unstim.fcs	1	16	1	0
13	13	PBMC_1063911118_PreE_unstim.fcs	1	11	0	1
14	14	PBMC_1080411158_Ctrl_unstim.fcs	0	15	0	1
15	15	PBMC_1080411288_Ctrl_unstim.fcs	0	28	1	1
16	16	PBMC_1901711258_PreE_unstim.fcs	1	25	1	0
17	17	PBMC_1901711168_PreE_unstim.fcs	1	16	1	1
18	18	PBMC_1960011258_PreE_unstim.fcs	1	25	1	0
19	19	PBMC_1960011088_PreE_unstim.fcs	1	8	0	0
20	20	PBMC_1072711118_PreE_unstim.fcs	1	11	0	0
21	21	PBMC_1072711278_PreE_unstim.fcs	1	27	1	0
22	22	PBMC_1052411118_Ctrl_unstim.fcs	0	11	0	1
23	23	PBMC_1052411268_Ctrl_unstim.fcs	0	26	1	1
24	24	PBMC_1080311261_PreE_unstim.fcs	1	26	1	1
25	25	PBMC_1080311138_PreE_unstim.fcs	1	13	0	0
26	26	PBMC_1073311128_Ctrl_unstim.fcs	0	12	0	1
27	27	PBMC_1073311268_Ctrl_unstim.fcs	0	26	1	0
28	28	PBMC_1072211178_PreE_unstim.fcs	1	17	1	1
29	29	PBMC_1072211128_PreE_unstim.fcs	1	12	0	0
30	30	PBMC_1081611118_PreE_unstim.fcs	1	11	0	0
31	31	PBMC_1081611168_PreE_unstim.fcs	1	16	1	0

	Unnamed: 0	file_id	condition	age	age_label	random1
32	32	PBMC_1054111158_Ctrl_unstim.fcs	0	15	0	1
33	33	PBMC_1054111098_Ctrl_unstim.fcs	0	9	0	1
34	34	PBMC_1071211258_Ctrl_unstim.fcs	0	25	1	0
35	35	PBMC_1071211138_Ctrl_unstim.fcs	0	13	0	1
36	36	PBMC_1080911248_PreE_unstim.fcs	1	24	1	0
37	37	PBMC_1080911118_PreE_unstim.fcs	1	11	0	1
38	38	PBMC_1052311088_Ctrl_unstim.fcs	0	8	0	0
39	39	PBMC_1052311288_Ctrl_unstim.fcs	0	28	1	0
40	40	PBMC_1073211248_Ctrl_unstim.fcs	0	24	1	0
41	41	PBMC_1073211118_Ctrl_unstim.fcs	0	11	0	0
42	42	PBMC_1082111121_Ctrl_unstim.fcs	0	12	0	1
43	43	PBMC_1082111261_Ctrl_unstim.fcs	0	26	1	1
44	44	PBMC_1083111111_Ctrl_unstim.fcs	0	11	0	1
45	45	PBMC_1083111161_Ctrl_unstim.fcs	0	16	1	0

```
In [ ]: # Separating controls and pre-eclampsia samples in dataframes
ctrl_df = fcs_info[fcs_info["condition"] == 0]
pre_df = fcs_info[fcs_info["condition"] == 1]
```

```
In [ ]: # Viewing the number of rows and structure of the fcs files
print("Control samples:", len(ctrl_df))
print("Pre-eclampsia samples:", len(pre_df))

display(ctrl_df.head())
display(pre_df.head())
```

Control samples: 24

Pre-eclampsia samples: 22

	Unnamed: 0	file_id	condition	age	age_label	random1
0	0	PBMC_1072111118_Ctrl_unstim.fcs	0	11	0	1
1	1	PBMC_1072111278_Ctrl_unstim.fcs	0	27	1	0
8	8	PBMC_1970711248_Ctrl_unstim.fcs	0	24	1	0
9	9	PBMC_1970711128_Ctrl_unstim.fcs	0	12	0	1
10	10	PBMC_1054711108_Ctrl_unstim.fcs	0	10	0	0

	Unnamed: 0		file_id	condition	age	age_label	random1
2	2	PBMC_1072311258_PreE_unstim.fcs		1	25	1	0
3	3	PBMC_1072311138_PreE_unstim.fcs		1	13	0	1
4	4	PBMC_1073611138_PreE_unstim.fcs		1	13	0	1
5	5	PBMC_1073611258_PreE_unstim.fcs		1	25	1	1
6	6	PBMC_1063411248_PreE_unstim.fcs		1	24	1	0

JSON FILES

```
In [ ]: # Accessing, viewing and cleaning all JSON files
json_paths = sorted(RAW_DIR.rglob("*.json"))
json_dfs = {}

if not json_paths:
    print(f"No JSON files found under {RAW_DIR.resolve()}")

for p in json_paths:
    print(f"\n=== {p.name} ===")
    txt = p.read_text(encoding="utf-8", errors="ignore").strip()
    if not txt:
        print("empty file")
        continue

    # Try real JSON; otherwise treat as newline-delimited text
    try:
        obj = json.loads(txt)
        if isinstance(obj, dict):
            df = pd.json_normalize(obj)
        elif isinstance(obj, list):
            try:
                df = pd.json_normalize(obj)
            except Exception:
                df = pd.DataFrame({"item": obj})
        else:
            df = pd.DataFrame({"value": [obj]})
        print(f"Parsed as JSON. shape:", df.shape)
    except Exception:
        lines = [ln.strip() for ln in txt.splitlines() if ln.strip()]
        df = pd.DataFrame({"line": lines})
        print(f"Not JSON; treated as text lines. lines:", len(df))

    json_dfs[p.name] = df
    print(f"NaNs per column:\n", df.isna().sum())
    display(df.head(5))
```

```
=== filenames_4.json ===
Not JSON; treated as text lines. lines: 40
NaNs per column:
line    0
dtype: int64
```

line

```
0 train/PBMC_1072111118_Ctrl_unstim.fcs
1 train/PBMC_1072111278_Ctrl_unstim.fcs
2 train/PBMC_1073611138_PreE_unstim.fcs
3 train/PBMC_1073611258_PreE_unstim.fcs
4 train/PBMC_1063411248_PreE_unstim.fcs
```

```
=== filenames_44.json ===
Not JSON; treated as text lines. lines: 40
NaNs per column:
  line    0
dtype: int64
```

line

```
0 train/PBMC_1072311258_PreE_unstim.fcs
1 train/PBMC_1072311138_PreE_unstim.fcs
2 train/PBMC_1073611138_PreE_unstim.fcs
3 train/PBMC_1073611258_PreE_unstim.fcs
4 train/PBMC_1063411248_PreE_unstim.fcs
```

```
=== filenames_444.json ===
Not JSON; treated as text lines. lines: 40
NaNs per column:
  line    0
dtype: int64
```

line

```
0 train/PBMC_1072111118_Ctrl_unstim.fcs
1 train/PBMC_1072111278_Ctrl_unstim.fcs
2 train/PBMC_1072311258_PreE_unstim.fcs
3 train/PBMC_1072311138_PreE_unstim.fcs
4 train/PBMC_1073611138_PreE_unstim.fcs
```

```
=== filenames_4444.json ===
Not JSON; treated as text lines. lines: 40
NaNs per column:
  line    0
dtype: int64
```

line

```

0 train/PBMC_1072111118_Ctrl_unstim.fcs
1 train/PBMC_1072111278_Ctrl_unstim.fcs
2 train/PBMC_1072311258_PreE_unstim.fcs
3 train/PBMC_1072311138_PreE_unstim.fcs
4 train/PBMC_1073611138_PreE_unstim.fcs

```

```

=== filenames_44444.json ===
Not JSON; treated as text lines. lines: 40
NaNs per column:
  line    0
dtype: int64

```

line

```

0 train/PBMC_1063411248_PreE_unstim.fcs
1 train/PBMC_1063411118_PreE_unstim.fcs
2 train/PBMC_1063911168_PreE_unstim.fcs
3 train/PBMC_1063911118_PreE_unstim.fcs
4 train/PBMC_1901711258_PreE_unstim.fcs

```

TxT files

```

In [ ]: # Use your RAW_DIR folder or existing folder
root = RAW_DIR if 'RAW_DIR' in globals() else Path('.')

txt_paths = sorted(root.rglob("*.txt"))
if not txt_paths:
    print(f"No .txt files found under: {root.resolve()}")

MISSING_TOKENS = {"na", "nan", "null", "none", "n/a"} # case-insensitive cleaning
n_show = 10 # Number of lines to display

for p in txt_paths:
    print(f"FILE: {p.name}")
    print(f"PATH: {p}")
    try:
        size_mb = p.stat().st_size / 1_000_000
        print(f"SIZE: {size_mb:.3f} MB")
    except Exception:
        pass

    try:
        text = p.read_text(encoding="utf-8", errors="ignore")
    except Exception as e:
        print(f"Could not read file: {e}")
        continue

    # Keep raw lines (don't drop blanks)
    raw_lines = text.splitlines()
    total_lines = len(raw_lines)

```

```

print(f"LINES: {total_lines}")

# Display samples
print(f"\n--- First {min(n_show, total_lines)} lines ---")
for i, ln in enumerate(raw_lines[:n_show], 1):
    print(f"{i:>3}: {ln}")

if total_lines > n_show:
    print(f"\n--- Last {min(n_show, total_lines)} lines ---")
    for i, ln in enumerate(raw_lines[-n_show:], total_lines - n_show + 1):
        print(f"{i:>3}: {ln}")

# Missing/NaN-like checks (very conservative)
blank_lines = sum(1 for ln in raw_lines if ln.strip() == "")
lines_with_missing_tokens = 0
missing_token_count = 0

for ln in raw_lines:
    # split on whitespace; count tokens that look like missing values
    tokens = [t for t in ln.split() if t != ""]
    # case-insensitive membership in the small set above
    miss = [t for t in tokens if t.lower() in MISSING_TOKENS]
    if miss:
        lines_with_missing_tokens += 1
        missing_token_count += len(miss)

print("\n--- Missing-like summary ---")
print(f"Blank lines: {blank_lines} / {total_lines}")
print(f"Lines containing missing-like tokens (NA/NAN/NULL/NONE/N/A): {lines_")
print(f"Total missing-like tokens found: {missing_token_count}")

```

FILE: age_tripletlist_subpick_test_rffmax_same0.2_diff0.2.txt
PATH: data_raw/age_tripletlist_subpick_test_rffmax_same0.2_diff0.2.txt
SIZE: 0.002 MB
LINES: 212

--- First 10 lines ---

1: 20 23 24
2: 20 25 33
3: 20 29 24
4: 20 29 33
5: 20 30 24
6: 20 32 33
7: 20 34 24
8: 20 35 24
9: 20 35 33
10: 20 37 24

--- Last 10 lines ---

203: 39 29 24
204: 39 30 24
205: 39 32 33
206: 39 34 24
207: 39 34 33
208: 39 35 24
209: 39 35 33
210: 39 37 24
211: 39 38 24
212: 39 38 33

--- Missing-like summary ---

Blank lines: 0 / 212

Lines containing missing-like tokens (NA/NAN/NULL/NONE/N/A): 0

Total missing-like tokens found: 0

FILE: age_tripletlist_subpick_trainval_rffmax_same0.2_diff0.2.txt

PATH: data_raw/age_tripletlist_subpick_trainval_rffmax_same0.2_diff0.2.txt

SIZE: 0.001 MB

LINES: 184

--- First 10 lines ---

1: 1 0 19
2: 1 0 9
3: 1 10 19
4: 1 10 9
5: 1 13 19
6: 1 17 19
7: 1 17 9
8: 1 6 19
9: 1 6 9
10: 11 0 19

--- Last 10 lines ---

175: 9 5 11
176: 9 5 12
177: 9 5 3
178: 9 5 7
179: 9 5 8
180: 9 6 1
181: 9 6 12
182: 9 6 14
183: 9 6 16

184: 9 6 4

--- Missing-like summary ---

Blank lines: 0 / 184

Lines containing missing-like tokens (NA/NAN/NULL/NONE/N/A): 0

Total missing-like tokens found: 0

Finally doing thorough cleaning for .fcs files as well. Cell is opened with fcs parser initially imported

```
In [ ]: # Finding all FCS files
fcs_paths = sorted(RAW_DIR.rglob("*.fcs"))
print("FCS files found:", len(fcs_paths))

reports = []
for fp in fcs_paths:
    # Read; prefer $PnN naming (isotope label), fall back if fcsparser doesn't s
    try:
        meta, df = fcs_parse(str(fp), reformat_meta=True, channel_naming="$PnN")
    except TypeError:
        meta, df = fcs_parse(str(fp), reformat_meta=True)
    except Exception as e:
        print(f"Read failed: {fp.name} -> {e}")
        continue

    # Keep numeric columns only (no assumptions about suffixes)
    num_cols = [c for c in df.columns if pd.api.types.is_numeric_dtype(df[c])]
    X = df[num_cols]

    # NaN stats
    nan_mask = X.isna()
    rows_with_any_nan = int(nan_mask.any(axis=1).sum())
    total_nans = int(nan_mask.sum().sum())
    top_nan_cols = nan_mask.sum().sort_values(ascending=False).head(5)
    top_nan_cols_str = ", ".join([f"{k}:{int(v)}" for k, v in top_nan_cols.items])

    reports.append({
        "filename": fp.name,
        "n_rows": len(X),
        "n_numeric_cols": len(num_cols),
        "rows_with_any_nan": rows_with_any_nan,
        "total_nans": total_nans,
        "top_nan_columns": top_nan_cols_str
    })

    # Quick glance at the structure
    print(f"{fp.name} - first numeric columns: {num_cols[:10]}")

fcs_nan_df = pd.DataFrame(reports).sort_values("rows_with_any_nan", ascending=False)
display(fcs_nan_df.head(10))

# Save a full summary for your records
(QC_DIR/"fcs_nan_summary.csv").parent.mkdir(parents=True, exist_ok=True)
fcs_nan_df.to_csv(QC_DIR/"fcs_nan_summary.csv", index=False)
print("Saved:", (QC_DIR/"fcs_nan_summary.csv").resolve())
```

[illegible]

20/662


```

15Di']
PBM_1901711168_PreE_unstim.fcs – first numeric columns: ['Time', 'Event_length',
'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In1
15Di']
PBM_1901711258_PreE_unstim.fcs – first numeric columns: ['Time', 'Event_length',
'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In1
15Di']
PBM_1960011088_PreE_unstim.fcs – first numeric columns: ['Time', 'Event_length',
'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In1
15Di']
PBM_1960011258_PreE_unstim.fcs – first numeric columns: ['Time', 'Event_length',
'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In1
15Di']
PBM_1970711128_Ctrl_unstim.fcs – first numeric columns: ['Time', 'Event_length',
'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In1
15Di']
PBM_1970711248_Ctrl_unstim.fcs – first numeric columns: ['Time', 'Event_length',
'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In1
15Di']

```

	filename	n_rows	n_numeric_cols	rows_with_any_nan	total_na
0	PBM_1052311088_Ctrl_unstim.fcs	122650	50	0	
34	PBM_1081611118_PreE_unstim.fcs	265590	50	0	
25	PBM_1073311268_Ctrl_unstim.fcs	555448	50	0	
26	PBM_1073611138_PreE_unstim.fcs	343023	50	0	
27	PBM_1073611258_PreE_unstim.fcs	245812	50	0	
28	PBM_1080311138_PreE_unstim.fcs	180824	50	0	
29	PBM_1080311261_PreE_unstim.fcs	215422	50	0	
30	PBM_1080411158_Ctrl_unstim.fcs	213955	50	0	
31	PBM_1080411288_Ctrl_unstim.fcs	271652	50	0	
32	PBM_1080911118_PreE_unstim.fcs	373463	50	0	



Saved: /content/results_qc/fcs_nan_summary.csv

viewing each .fcs as a DataFrame

```

In [ ]: # Pick a root folder safely: prefer ./data_raw if it exists, else current dir
root_dir = "data_raw" if os.path.isdir("data_raw") else "."
print("Root:", os.path.abspath(root_dir))

# Find .fcs files under root_dir (recursively)
fcs_paths = sorted(glob.glob(os.path.join(root_dir, "**", "*.fcs"), recursive=True)
print("FCS files found:", len(fcs_paths))
if not fcs_paths:
    print("No .fcs files found under the root above.")

for fp in fcs_paths:
    print("\n" + "="*80)
    print("FILE:", os.path.basename(fp))
    print("PATH:", fp)

```

```

# Read with clean channel names if supported; otherwise default
try:
    try:
        meta, df = fcs_parse(fp, reformat_meta=True, channel_naming="$PnN")
    except TypeError:
        meta, df = fcs_parse(fp, reformat_meta=True)
except Exception as e:
    print("Read failed:", e)
    continue

# Show a small preview
print("shape:", df.shape)
print("first columns:", list(df.columns)[:15])
display(df.head(5)) # small preview to avoid huge output

# NaN check on numeric columns (no assumptions about names)
num_cols = [c for c in df.columns if pd.api.types.is_numeric_dtype(df[c])]
if num_cols:
    X = df[num_cols]
    rows_with_any_nan = int(X.isna().any(axis=1).sum())
    total_nans = int(X.isna().sum().sum())
    print(f"Numeric cols: {len(num_cols)} | NaN rows: {rows_with_any_nan} / {total_nans}")
    if total_nans:
        top = X.isna().sum().sort_values(ascending=False).head(10)
        print("Top NaN columns:", ", ".join(f"{k}:{int(v)}" for k, v in top.items()))
    else:
        print("No numeric columns detected.")

```

Root: /content/data_raw

FCS files found: 46

```

=====
FILE: PBMC_1052311088_Ctrl_unstim.fcs
PATH: data_raw/PBMC_1052311088_Ctrl_unstim.fcs
shape: (122650, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

```

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	3914.089111	16.0	1016.304504	36.386017	26.760574	735.663757	36.075294
1	37213.867188	16.0	1159.808960	41.838978	40.582455	814.272156	48.861229
2	43903.972656	16.0	861.785217	37.331711	47.393581	700.374023	28.453079
3	117463.726562	14.0	531.848450	16.742098	45.077072	382.798065	15.631414
4	123789.078125	16.0	1430.883545	69.076477	59.128632	1155.441528	52.065817

5 rows × 50 columns



Numeric cols: 50 | NaN rows: 0 / 122650 | Total NaNs: 0

```
=====
FILE: PBMC_1052311288_Ctrl_unstim.fcs
PATH: data_raw/PBMC_1052311288_Ctrl_unstim.fcs
shape: (147955, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106D
i', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di',
'Nd143Di', 'Nd144Di']
```

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	13573.476562	17.0	48.817158	655.592468	823.232178	97.442291	29.053242
1	117263.906250	18.0	32.865231	537.188904	661.936401	54.178905	32.161396
2	117905.406250	17.0	43.888794	591.731201	649.009827	69.097122	41.844864
3	121011.234375	16.0	39.436890	451.606354	644.021362	88.835518	30.114401
4	123849.859375	20.0	41.083504	721.326233	900.593445	96.196091	23.145218

5 rows × 50 columns

```
◀  ▶
Numeric cols: 50 | NaN rows: 0 / 147955 | Total NaNs: 0

=====
FILE: PBMC_1052411118_Ctrl_unstim.fcs
PATH: data_raw/PBMC_1052411118_Ctrl_unstim.fcs
shape: (129071, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106D
i', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di',
'Nd143Di', 'Nd144Di']
```

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	2145.781006	15.0	1295.758667	61.564724	815.993103	90.292465	61.272190
1	2257.864990	17.0	1448.245850	56.369778	1019.870850	101.462097	65.956055
2	2808.762939	15.0	1159.548950	42.301075	863.157227	90.126671	39.628033
3	3088.189941	16.0	1447.391479	78.831688	900.574219	86.848137	58.893700
4	4142.096191	15.0	1628.806519	56.919075	1012.915039	83.227951	43.628960

5 rows × 50 columns

```
◀  ▶
Numeric cols: 50 | NaN rows: 0 / 129071 | Total NaNs: 0

=====
FILE: PBMC_1052411268_Ctrl_unstim.fcs
PATH: data_raw/PBMC_1052411268_Ctrl_unstim.fcs
shape: (303004, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106D
i', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di',
'Nd143Di', 'Nd144Di']
```

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	135.246994	19.0	46.224361	380.859619	454.559235	549.143127	18.169264
1	1829.961060	16.0	56.867580	510.276550	702.362732	700.479797	23.307884
2	1865.378052	18.0	31.753508	380.447174	613.266663	575.487671	15.130569
3	2268.489990	19.0	60.949448	443.090424	457.941284	556.432251	21.017109
4	2426.718994	19.0	51.123966	359.791077	511.875641	549.843079	31.628384

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 303004 | Total NaNs: 0

=====

FILE: PBMC_1054111098_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1054111098_Ctrl_unstim.fcs

shape: (95813, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	176.328003	17.0	2134.489746	72.860893	1359.410889	121.986275	79.607147
1	435.794006	17.0	1558.799927	71.012932	1040.500854	160.265884	53.727940
2	441.601990	15.0	564.907776	27.146765	336.899261	35.407307	30.260365
3	853.802002	18.0	893.754944	48.163776	588.907288	73.135918	49.645290
4	987.252991	20.0	828.340698	55.218922	509.364227	50.421040	36.061714

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 95813 | Total NaNs: 0

=====

FILE: PBMC_1054111158_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1054111158_Ctrl_unstim.fcs

shape: (215633, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di	
0	182.878006	15.0	726.151672	51.683105	53.332905	472.447113	26.309093	24
1	183.397995	16.0	772.678650	33.509987	42.758633	534.584167	30.461277	26
2	657.005005	16.0	799.127075	25.156101	46.673817	541.828308	30.721750	32
3	891.744995	16.0	1003.527100	45.762711	68.190216	589.106995	49.093880	36
4	933.098999	17.0	737.416870	30.511505	39.846245	458.926758	25.541983	27

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 215633 | Total NaNs: 0

=====

FILE: PBMC_1054711108_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1054711108_Ctrl_unstim.fcs

shape: (160081, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di	
0	23.150999	17.0	1302.840698	108.941589	953.792419	134.777969	90.795868	
1	119.688004	16.0	658.654724	19.218103	422.592041	41.332172	8.262961	
2	147.070007	17.0	1024.061035	74.064949	707.637390	132.300842	32.961296	
3	213.307007	18.0	549.039062	23.965666	380.684204	62.364449	20.635025	
4	221.223999	18.0	883.557251	68.554596	621.316345	91.003914	58.388203	

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 160081 | Total NaNs: 0

=====

FILE: PBMC_1054711248_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1054711248_Ctrl_unstim.fcs

shape: (127947, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	10.078000	19.0	114.548073	646.754333	857.885864	930.120789	21.026897
1	69.023003	20.0	95.939697	764.948242	996.825562	1082.273193	30.136658
2	201.757996	17.0	57.561424	780.956848	967.669861	1054.272095	8.267681
3	267.617004	18.0	65.684959	637.569519	956.184937	1000.274170	37.725849
4	281.289001	18.0	76.003593	656.312439	845.150940	981.624146	25.074995

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 127947 | Total NaNs: 0

=====

FILE: PBMC_1063411118_PreE_unstim.fcs

PATH: data_raw/PBMC_1063411118_PreE_unstim.fcs

shape: (230373, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	217.513000	18.0	1029.236816	602.095825	622.345581	76.283081	47.382778
1	1269.401001	17.0	342.582092	173.649307	231.550766	22.495617	12.930871
2	1458.840942	16.0	621.325623	356.673767	475.134430	54.473301	35.590435
3	1588.151001	16.0	476.549103	229.767731	295.278687	24.361712	6.984618
4	1857.616943	15.0	812.131531	407.959137	480.196503	82.338287	24.930758

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 230373 | Total NaNs: 0

=====

FILE: PBMC_1063411248_PreE_unstim.fcs

PATH: data_raw/PBMC_1063411248_PreE_unstim.fcs

shape: (171828, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	382.565002	16.0	2149.362793	75.481041	1293.807007	1478.478638	54.61978
1	705.247009	17.0	1076.294189	53.911240	757.740051	819.936890	28.68096
2	1441.991943	15.0	1280.074585	71.120003	864.417664	1052.155029	47.81940
3	1561.093994	19.0	1503.423340	67.961807	997.169922	1085.591187	39.55261
4	1575.534058	16.0	2201.324951	121.701523	1437.466919	1695.006958	73.58241

5 rows × 50 columns

 
Numeric cols: 50 | NaN rows: 0 / 171828 | Total NaNs: 0

=====

FILE: PBMC_1063911118_PreE_unstim.fcs
PATH: data_raw/PBMC_1063911118_PreE_unstim.fcs
shape: (87847, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	49.570000	20.0	3559.202393	2004.948853	2347.869629	277.354736	96.52670
1	61.094002	20.0	5354.055664	2953.308350	3658.191406	363.192169	137.15667
2	115.116997	29.0	4642.281250	2264.455811	3080.928955	339.121796	139.34779
3	253.294006	18.0	3020.992188	1528.083862	1945.528198	181.815216	65.95319
4	330.325989	18.0	3681.370117	1852.642578	2467.879883	216.845627	68.25830

5 rows × 50 columns

 
Numeric cols: 50 | NaN rows: 0 / 87847 | Total NaNs: 0

=====

FILE: PBMC_1063911168_PreE_unstim.fcs
PATH: data_raw/PBMC_1063911168_PreE_unstim.fcs
shape: (210758, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di	
0	76.458000	16.0	841.090210	448.532410	103.437744	64.962326	551.408380	
1	327.487000	19.0	1635.577271	894.612244	205.930801	146.546997	1125.698850	
2	334.088989	12.0	222.866852	147.583069	16.864908	5.301559	128.979700	
3	414.882996	18.0	894.290100	538.987732	57.943859	42.386211	543.340140	
4	791.614990	17.0	1206.573120	670.141418	125.362526	95.099960	814.127010	

5 rows × 50 columns

 
Numeric cols: 50 | NaN rows: 0 / 210758 | Total NaNs: 0

=====

FILE: PBMC_1071211138_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1071211138_Ctrl_unstim.fcs

shape: (419065, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di	
0	9.492000	17.0	795.373596	53.758995	70.681656	677.929810	45.680965	360
1	10.130000	18.0	619.981995	32.528042	49.717258	463.468719	51.242325	260
2	20.677000	16.0	935.738892	46.424053	67.547318	720.095154	34.602600	390
3	54.206001	18.0	823.704773	30.104240	33.384762	514.386108	32.473839	320
4	66.991997	17.0	1009.248840	37.400791	61.086121	663.033020	34.366402	360

5 rows × 50 columns

 
Numeric cols: 50 | NaN rows: 0 / 419065 | Total NaNs: 0

=====

FILE: PBMC_1071211258_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1071211258_Ctrl_unstim.fcs

shape: (262874, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	0.911000	17.0	65.735565	778.830627	994.163269	86.677612	6.609682
1	16.823000	17.0	84.385780	1157.372437	1548.380249	103.950142	37.257927
2	35.924000	16.0	52.028553	560.842407	761.579468	59.193378	25.004383
3	61.731998	16.0	36.680443	780.251038	990.044922	96.789108	34.027763
4	152.487000	17.0	95.891975	862.640503	1241.650879	129.390289	66.112236

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 262874 | Total NaNs: 0

=====

FILE: PBMC_1072111118_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1072111118_Ctrl_unstim.fcs

shape: (247492, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	56.289001	20.0	1145.043335	40.026890	68.095131	860.613098	44.496658
1	70.598999	20.0	1815.408447	81.019188	95.981430	1330.903931	81.264677
2	735.781006	17.0	2587.154053	104.199448	133.610779	1983.014771	86.813076
3	1026.732056	15.0	1435.218384	47.495392	62.066380	1048.711914	44.526886
4	1336.041992	18.0	2482.503418	81.801170	126.100510	1787.686890	82.796417

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 247492 | Total NaNs: 0

=====

FILE: PBMC_1072111278_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1072111278_Ctrl_unstim.fcs

shape: (88725, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	250.052002	17.0	30.410957	1153.192505	1523.049316	74.162262	29.785612
1	686.575989	18.0	44.007477	1765.053223	2301.425781	156.954880	26.212555
2	2471.627930	18.0	72.393356	2393.143311	3031.377197	202.758194	49.425003
3	2631.666992	20.0	18.432238	1162.467041	1559.539551	98.269913	19.777422
4	3915.272949	17.0	23.801508	1321.879150	1712.284058	98.627846	21.813786

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 88725 | Total NaNs: 0

=====

FILE: PBMC_1072211128_PreE_unstim.fcs

PATH: data_raw/PBMC_1072211128_PreE_unstim.fcs

shape: (152375, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	115336.742188	15.0	884.493896	464.401428	584.256042	80.175484	37.672329
1	115596.585938	14.0	549.034973	279.235413	366.002716	55.104420	20.931004
2	117471.312500	16.0	712.957214	377.209778	479.952179	46.996666	23.436577
3	121238.804688	19.0	739.666016	448.744446	524.932617	99.279114	55.267071
4	121959.234375	17.0	962.481140	541.278137	667.708313	84.313309	50.376400

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 152375 | Total NaNs: 0

=====

FILE: PBMC_1072211178_PreE_unstim.fcs

PATH: data_raw/PBMC_1072211178_PreE_unstim.fcs

shape: (94591, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	27150.794922	16.0	631.102844	360.988495	90.176476	43.145588	356.099495
1	44874.207031	19.0	695.592163	359.869385	37.849293	46.200535	441.756371
2	113960.367188	12.0	198.902496	102.553543	18.407846	10.222953	135.606043
3	126675.140625	15.0	850.141724	436.137421	72.388100	69.584213	557.200071
4	127566.937500	17.0	1301.900269	784.824829	109.225807	96.384964	877.630921

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 94591 | Total NaNs: 0

=====

FILE: PBMC_1072311138_PreE_unstim.fcs
PATH: data_raw/PBMC_1072311138_PreE_unstim.fcs
shape: (254287, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	204.804993	17.0	1130.229736	697.619629	938.637146	105.635971	47.664931
1	230.207993	20.0	2565.268311	1369.440430	1681.489990	164.187622	48.113171
2	277.955994	22.0	2070.763184	1050.395386	1357.726440	135.792786	49.287431
3	1324.284058	21.0	2264.791016	1205.175659	1468.713989	149.683777	66.684091
4	1831.197998	21.0	540.913757	285.274048	362.006927	49.456745	25.117771

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 254287 | Total NaNs: 0

=====

FILE: PBMC_1072311258_PreE_unstim.fcs
PATH: data_raw/PBMC_1072311258_PreE_unstim.fcs
shape: (164143, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di	I
0	1037.760010	20.0	2174.171143	78.559868	1470.957397	1691.804443	40.01978	
1	4096.067871	17.0	2083.606201	86.142578	1442.571411	1580.637451	49.42404	
2	6654.596191	19.0	2977.077881	115.201012	1937.411499	2115.302734	78.24616	
3	6898.763184	20.0	2003.621582	70.641075	1259.879272	1382.771973	32.31044	
4	7046.576172	18.0	2201.270264	93.166817	1451.917358	1711.638428	56.09850	

5 rows × 50 columns

 
Numeric cols: 50 | NaN rows: 0 / 164143 | Total NaNs: 0

=====

FILE: PBMC_1072711118_PreE_unstim.fcs
PATH: data_raw/PBMC_1072711118_PreE_unstim.fcs
shape: (306914, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di	I
0	36.185001	15.0	712.339233	394.365845	542.199036	70.420204	27.379215	17
1	82.903999	17.0	848.848267	508.332703	589.010193	67.248779	28.586428	20
2	111.197998	21.0	854.548157	411.051819	543.079041	86.704750	45.551308	9
3	305.507996	17.0	574.836304	240.334763	356.203339	39.406109	30.265533	16
4	456.692993	20.0	460.325531	277.107544	332.723572	33.534931	24.010422	12

5 rows × 50 columns

 
Numeric cols: 50 | NaN rows: 0 / 306914 | Total NaNs: 0

=====

FILE: PBMC_1072711278_PreE_unstim.fcs
PATH: data_raw/PBMC_1072711278_PreE_unstim.fcs
shape: (185950, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	254.934998	18.0	1924.532471	88.592957	1316.787109	115.852791	1106.85190
1	866.809998	19.0	1095.695190	68.338226	596.575439	97.178345	639.50317
2	905.091003	18.0	2030.979370	94.365448	1350.273682	146.906006	1269.83050
3	927.278992	19.0	1629.230347	82.325279	939.586792	76.386162	920.41870
4	1107.395996	20.0	1780.604614	63.917973	1100.651489	121.017662	1081.44870

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 185950 | Total NaNs: 0

=====

FILE: PBMC_1073211118_Ctrl_unstim.fcs
PATH: data_raw/PBMC_1073211118_Ctrl_unstim.fcs
shape: (127879, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	4707.903809	15.0	724.200684	35.268703	42.110043	573.990295	27.488740
1	6146.861816	19.0	940.031738	40.908226	47.377926	771.078247	49.117110
2	18669.453125	17.0	792.265991	25.297506	35.444702	577.546875	26.818070
3	21521.457031	18.0	2474.248047	99.924553	156.289429	2031.646973	90.303900
4	21581.171875	23.0	1599.306274	60.793552	69.875885	1322.331787	62.601800

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 127879 | Total NaNs: 0

=====

FILE: PBMC_1073211248_Ctrl_unstim.fcs
PATH: data_raw/PBMC_1073211248_Ctrl_unstim.fcs
shape: (201344, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di	
0	1502.864990	15.0	21.617542	185.853058	240.682480	24.317186	10.158498	18
1	2742.095947	14.0	37.031063	241.139496	281.794189	30.825291	8.946255	17
2	3365.352051	16.0	29.716709	239.405182	318.872314	24.566605	22.844122	20
3	6557.812012	14.0	36.299999	223.646729	294.163483	40.460327	11.122639	20
4	7814.856934	14.0	33.898457	378.580383	522.915222	51.985004	35.105068	35

5 rows × 50 columns



Numeric cols: 50 | NaN rows: 0 / 201344 | Total NaNs: 0

=====

FILE: PBMC_1073311128_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1073311128_Ctrl_unstim.fcs

shape: (282767, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di	
0	68.450996	15.0	1072.814575	53.565224	645.835144	67.974213	31.565294	4
1	1755.793945	15.0	976.088135	52.404469	629.972778	85.190437	51.522724	4
2	2083.464111	18.0	900.269226	39.206726	557.483521	69.880379	29.286598	3
3	2554.102051	14.0	719.625854	42.423363	448.709167	49.371208	17.567259	2
4	2594.766113	15.0	1112.658691	54.796082	856.849426	86.953865	35.207993	4

5 rows × 50 columns



Numeric cols: 50 | NaN rows: 0 / 282767 | Total NaNs: 0

=====

FILE: PBMC_1073311268_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1073311268_Ctrl_unstim.fcs

shape: (555448, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	688.072998	15.0	75.423721	421.299713	485.029083	549.295471	18.702778
1	738.125000	18.0	37.303551	244.562668	339.424652	382.613098	12.611608
2	893.424011	18.0	81.276146	425.836243	476.925323	665.957886	34.089664
3	1235.819946	19.0	86.904396	556.298950	720.748291	725.344421	31.946157
4	1623.762939	14.0	22.148123	260.169647	324.934082	390.466400	20.891378

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 555448 | Total NaNs: 0

=====

FILE: PBMC_1073611138_PreE_unstim.fcs

PATH: data_raw/PBMC_1073611138_PreE_unstim.fcs

shape: (343023, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	85.482002	18.0	701.295349	384.559174	457.644257	51.787285	20.835894
1	122.591003	16.0	1172.646851	615.039917	803.331604	110.291901	44.004314
2	252.121994	16.0	862.898621	469.633423	628.805237	85.586014	26.183077
3	421.015991	18.0	768.361084	471.798279	496.835663	82.414543	17.842560
4	569.830994	19.0	850.480042	420.830200	546.032104	58.286385	14.296757

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 343023 | Total NaNs: 0

=====

FILE: PBMC_1073611258_PreE_unstim.fcs

PATH: data_raw/PBMC_1073611258_PreE_unstim.fcs

shape: (245812, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	252.981995	18.0	2435.060303	114.176231	1654.640991	1968.373901	80.21690
1	544.987000	14.0	1223.539185	64.590736	887.267090	984.330383	61.5887
2	610.598999	16.0	1367.083008	78.356705	903.213135	1162.558838	44.83530
3	1197.435059	17.0	2052.856934	107.122803	1354.994873	1774.061646	64.5363
4	1368.255005	16.0	1273.494141	73.030472	801.202637	994.101746	45.2563

5 rows × 50 columns

◀  ▶

Numeric cols: 50 | NaN rows: 0 / 245812 | Total NaNs: 0

=====

FILE: PBMC_1080311138_PreE_unstim.fcs

PATH: data_raw/PBMC_1080311138_PreE_unstim.fcs

shape: (180824, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	981.523010	16.0	706.103271	344.917145	455.078857	47.313366	28.177275
1	2381.952881	15.0	698.295837	343.453461	495.988312	55.931850	20.790291
2	8545.351562	16.0	779.708557	530.332947	551.568604	69.981247	36.542427
3	10511.627930	15.0	833.174438	476.912903	601.718689	95.997299	32.656406
4	10590.833008	15.0	635.911865	364.185516	445.889862	61.781799	28.745634

5 rows × 50 columns

◀  ▶

Numeric cols: 50 | NaN rows: 0 / 180824 | Total NaNs: 0

=====

FILE: PBMC_1080311261_PreE_unstim.fcs

PATH: data_raw/PBMC_1080311261_PreE_unstim.fcs

shape: (215422, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	314.466003	20.0	864.974365	49.845642	643.876953	682.156677	23.711617
1	1108.241943	19.0	1172.008789	55.035873	788.869934	900.607605	21.404461
2	4925.391113	15.0	1014.874390	35.023556	640.388184	711.021606	20.401144
3	7562.409180	16.0	1023.725098	48.777950	729.205688	863.625549	30.898485
4	9473.918945	20.0	1146.289917	44.276955	698.876038	792.123535	23.797216

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 215422 | Total NaNs: 0

=====

FILE: PBMC_1080411158_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1080411158_Ctrl_unstim.fcs

shape: (213955, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	3477.435059	17.0	736.001770	48.436699	45.684753	486.634521	44.090702
1	9375.208008	16.0	621.757263	22.735176	32.370991	524.783447	35.208717
2	9782.473633	16.0	657.532532	59.081699	72.698486	487.462189	35.755329
3	12041.250000	14.0	921.300964	71.660179	70.005707	660.714172	61.450630
4	12304.570312	16.0	913.399170	40.987495	51.304970	620.919250	33.537121

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 213955 | Total NaNs: 0

=====

FILE: PBMC_1080411288_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1080411288_Ctrl_unstim.fcs

shape: (271652, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	935.559998	16.0	38.945667	112.531647	154.062897	155.799713	15.612704
1	3167.929932	15.0	50.192852	158.332550	196.037872	217.328964	19.450647
2	6657.161133	18.0	73.367035	214.058929	271.669922	325.412811	37.160301
3	7384.504883	15.0	95.296494	231.035324	264.210754	319.393463	25.931509
4	9944.870117	18.0	46.809452	145.043793	135.443466	217.118744	16.162991

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 271652 | Total NaNs: 0

=====

FILE: PBMC_1080911118_PreE_unstim.fcs

PATH: data_raw/PBMC_1080911118_PreE_unstim.fcs

shape: (373463, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	1090.026001	17.0	1149.327393	619.270813	701.975281	91.578659	44.795639
1	1791.159058	19.0	818.270081	419.761200	446.229828	78.710793	16.703663
2	1879.465942	15.0	1184.372925	660.085022	732.864075	79.458076	21.746050
3	2218.125000	15.0	743.478271	331.218903	464.819580	35.868427	19.628897
4	2989.335938	13.0	644.603271	368.456573	432.954346	73.132225	5.326975

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 373463 | Total NaNs: 0

=====

FILE: PBMC_1080911248_PreE_unstim.fcs

PATH: data_raw/PBMC_1080911248_PreE_unstim.fcs

shape: (333504, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	54.647999	19.0	1237.174194	42.171101	781.956421	919.285034	24.704676
1	340.299011	16.0	1705.434814	94.050102	1110.894409	1307.502441	65.806343
2	978.359009	17.0	1516.221680	89.243156	895.907593	1088.944702	47.375710
3	2182.004883	16.0	1251.651733	52.455051	756.914734	958.543335	9.183617
4	2242.214111	15.0	1325.312866	64.526001	792.052063	1013.482422	36.480491

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 333504 | Total NaNs: 0

=====

FILE: PBMC_1081611118_PreE_unstim.fcs
PATH: data_raw/PBMC_1081611118_PreE_unstim.fcs
shape: (265590, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	4375.182129	20.0	1077.901978	535.961792	665.392029	80.628098	42.182229
1	5005.168945	18.0	1020.346985	637.136536	699.334656	112.051125	31.879429
2	8987.017578	16.0	1008.119263	465.141815	577.830750	81.423592	15.534409
3	20108.619141	17.0	1570.184570	868.412720	1016.366699	130.112488	61.412499
4	29875.859375	20.0	1149.078003	612.672424	649.642273	95.264809	35.727739

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 265590 | Total NaNs: 0

=====

FILE: PBMC_1081611168_PreE_unstim.fcs
PATH: data_raw/PBMC_1081611168_PreE_unstim.fcs
shape: (270965, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	199.727005	18.0	1804.315552	961.636902	230.210983	107.002403	1066.94
1	3714.660889	19.0	1510.897949	782.164124	151.921310	55.511196	876.24
2	4295.391113	20.0	1254.551758	564.310669	121.532394	62.021832	767.70
3	7664.102051	17.0	824.285583	516.356201	78.795586	48.984646	487.93
4	14658.750000	17.0	2213.473389	1173.125854	221.173569	114.962837	1333.31

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 270965 | Total NaNs: 0

=====

FILE: PBMC_1082111121_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1082111121_Ctrl_unstim.fcs

shape: (201310, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	600.026001	15.0	229.191452	113.180153	172.893204	36.892132	11.179440
1	761.328003	13.0	484.445831	209.120560	254.096939	61.165001	24.133085
2	908.320007	14.0	330.840515	198.274353	239.314911	51.053204	15.981511
3	971.054993	15.0	321.437164	168.493042	220.917679	31.151960	32.331455
4	1067.201050	16.0	235.044571	109.434036	127.998566	27.197321	18.084425

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 201310 | Total NaNs: 0

=====

FILE: PBMC_1082111261_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1082111261_Ctrl_unstim.fcs

shape: (140771, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	420.807007	16.0	2376.726807	114.089798	1435.311157	1718.473267	72.24484
1	1270.845947	16.0	2306.668457	96.141777	1393.242432	1607.552490	52.88156
2	3504.141113	13.0	1436.997437	43.211082	980.648621	1164.812134	55.29490
3	3700.168945	19.0	2164.585449	55.119873	1196.826538	1498.847656	66.75551
4	4659.088867	13.0	2179.272949	84.841743	1365.029419	1576.934814	42.34328

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 140771 | Total NaNs: 0

=====

FILE: PBMC_1083111111_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1083111111_Ctrl_unstim.fcs

shape: (153842, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	1221.276001	13.0	1413.835571	56.540188	70.879448	1047.174805	46.200455
1	1719.609009	18.0	1391.333740	58.666981	52.062267	971.447144	40.180000
2	5078.333008	16.0	1811.576294	57.599850	94.485512	1253.147095	64.297234
3	5098.085938	14.0	761.661865	38.966667	29.697571	608.575867	15.685079
4	5152.252930	15.0	1853.336426	51.955429	62.028419	1091.798218	52.405602

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 153842 | Total NaNs: 0

=====

FILE: PBMC_1083111161_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1083111161_Ctrl_unstim.fcs

shape: (225102, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	1101.810059	13.0	24.101130	763.050476	1076.971191	1052.208008	22.273348
1	1141.953003	17.0	32.761124	752.826477	982.896606	1051.014160	38.911121
2	1377.409058	17.0	27.234814	893.714233	1179.390015	1103.402344	10.357601
3	1412.694946	17.0	24.592291	667.879028	839.011353	960.229065	12.419271
4	1468.359009	14.0	28.728035	552.281921	721.102905	844.035339	16.286591

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 225102 | Total NaNs: 0

=====

FILE: PBMC_1901711168_PreE_unstim.fcs

PATH: data_raw/PBMC_1901711168_PreE_unstim.fcs

shape: (222671, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	1.107000	19.0	747.684509	426.701111	105.181175	44.404560	430.547119
1	2.630000	17.0	826.447754	431.721008	93.609749	102.688850	511.289551
2	4.219000	16.0	455.489258	243.839798	60.474365	30.594135	346.731995
3	5.742000	17.0	796.617249	434.600891	108.037178	68.838318	462.969940
4	95.559998	18.0	1214.645630	552.662598	169.911743	125.092949	650.708557

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 222671 | Total NaNs: 0

=====

FILE: PBMC_1901711258_PreE_unstim.fcs

PATH: data_raw/PBMC_1901711258_PreE_unstim.fcs

shape: (129111, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di	I
0	11.966000	21.0	465.519867	14.738660	290.578888	332.997162	10.336295	4
1	39.740002	20.0	1006.838013	49.268490	553.503540	673.351257	34.051067	18
2	49.583000	17.0	955.934570	50.973366	617.655273	682.403625	20.137224	8
3	50.702999	18.0	919.326599	80.021812	568.205811	617.949768	40.837196	3
4	88.945000	17.0	1324.757935	70.394760	809.241089	909.074097	68.856766	8

5 rows × 50 columns

 
Numeric cols: 50 | NaN rows: 0 / 129111 | Total NaNs: 0

=====

FILE: PBMC_1960011088_PreE_unstim.fcs
PATH: data_raw/PBMC_1960011088_PreE_unstim.fcs
shape: (211370, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di	
0	41.602001	17.0	884.360229	453.682190	562.399841	66.904602	35.277832	4
1	116.015999	17.0	664.167786	309.334442	480.641083	71.093224	45.437302	18
2	297.018005	17.0	629.105652	360.036133	503.215149	94.352936	26.231319	8
3	539.062012	15.0	490.246490	235.222549	282.469879	34.229771	10.889809	3
4	1157.121948	22.0	949.931519	561.907227	605.012512	94.599800	31.425571	8

5 rows × 50 columns

 
Numeric cols: 50 | NaN rows: 0 / 211370 | Total NaNs: 0

=====

FILE: PBMC_1960011258_PreE_unstim.fcs
PATH: data_raw/PBMC_1960011258_PreE_unstim.fcs
shape: (98739, 50)
first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	562.018005	15.0	2320.109375	114.381912	1541.297119	1823.668701	59.62614
1	2580.208008	18.0	1880.177612	73.145706	1230.813232	1475.425781	49.65824
2	2847.460938	17.0	2287.603027	92.068695	1546.274292	1589.080566	72.39670
3	4228.215820	17.0	2312.325684	87.977760	1426.131348	1618.280762	44.96613
4	4330.025879	18.0	2463.726318	88.871864	1538.275391	1845.154297	70.28204

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 98739 | Total NaNs: 0

=====

FILE: PBMC_1970711128_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1970711128_Ctrl_unstim.fcs

shape: (212242, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	514.257996	15.0	490.267944	15.203754	20.891817	304.407166	9.976910
1	692.044006	16.0	1164.141479	41.772728	68.168228	921.208984	41.514664
2	1475.637939	15.0	881.051758	46.022148	29.254143	637.033691	12.698973
3	2518.931885	15.0	1193.219727	43.557930	38.803516	845.153198	32.541103
4	3379.452881	17.0	1302.153564	54.799538	60.814884	1085.086304	47.843090

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 212242 | Total NaNs: 0

=====

FILE: PBMC_1970711248_Ctrl_unstim.fcs

PATH: data_raw/PBMC_1970711248_Ctrl_unstim.fcs

shape: (185612, 50)

first columns: ['Time', 'Event_length', 'Pd102Di', 'Pd104Di', 'Pd105Di', 'Pd106Di', 'Pd108Di', 'Pd110Di', 'In113Di', 'In115Di', 'La139Di', 'Pr141Di', 'Nd142Di', 'Nd143Di', 'Nd144Di']

	Time	Event_length	Pd102Di	Pd104Di	Pd105Di	Pd106Di	Pd108Di
0	1781.029053	16.0	110.119072	943.780090	1220.610107	153.985855	30.043285
1	2056.602051	14.0	16.936028	659.199036	727.774353	52.936874	18.944862
2	2847.891113	15.0	15.028853	714.860718	975.116638	75.150375	25.280613
3	4026.406006	15.0	12.549950	382.863129	497.290802	55.741615	3.758124
4	4242.564941	17.0	20.344397	915.178589	1261.524902	77.875931	13.362976

5 rows × 50 columns

Numeric cols: 50 | NaN rows: 0 / 185612 | Total NaNs: 0

Harmonizing isotope labels in .fcs files with markers from marker.csv

```
In [ ]: # Isotope to protein mapping
marker_path = os.path.join(str(RAW_DIR), "marker.csv")
marker_df = pd.read_csv(marker_path)
marker_cols = list(marker_df.columns)

iso_to_protein = {}
for col in marker_cols:
    if "_" not in col:
        continue
    left, protein = col.split("_", 1)
    # accept either "115In" or "In115" in left and normalize to "115In"
    m1 = re.match(r"^(\d+)([A-Za-z]+$)", left)      # "115In"
    m2 = re.match(r"^([A-Za-z]+)(\d+)$", left)      # "In115"
    if m1:
        mass, elem = m1.groups()
        iso_to_protein[f"{mass}{elem}"] = protein
    elif m2:
        elem, mass = m2.groups()
        iso_to_protein[f"{mass}{elem}"] = protein

print("Example iso mappings:", list(iso_to_protein.items())[:10])
```

Example iso mappings: [('115In', 'CD45'), ('139La', 'CD66'), ('141Pr', 'CD7'), ('142Nd', 'CD19'), ('143Nd', 'CD45RA'), ('144Nd', 'CD11b'), ('145Nd', 'CD4'), ('146Nd', 'CD8a'), ('147Sm', 'CD11c'), ('148Nd', 'CD123')]

```
In [ ]: def _list_columns_fast(fp):
    # try meta-only; if not supported, do a full read
    try:
        res = fcs_parse(fp, meta_data_only=True, reformat_meta=True, channel_nam
        meta = res[0] if isinstance(res, (tuple, list)) else res
        ch_tbl = meta.get("_channels_", None)
        if ch_tbl is not None:
            col = "$PnN" if "$PnN" in ch_tbl.columns else ("PnS" if "PnS" in c
            if col is not None:
                return list(ch_tbl[col].astype(str))
        except TypeError:
            pass
    # fallback tiny full read
    try:
        _, df = fcs_parse(fp, reformat_meta=True, channel_naming="$PnN")
```

```

except TypeError:
    _, df = fcs_parse(fp, reformat_meta=True)
return list(df.columns)

def _iso_tokens_from_col(col):
    # Pull tokens like "115In" from names like "In115Di", "Nd142Di", "115In"
    toks = set()
    m1 = re.search(r"([A-Za-z]+)(\d+)", col) # In + 115
    if m1:
        elem, mass = m1.groups()
        toks.add(f"{mass}{elem}")
    m2 = re.search(r"(\d+)([A-Za-z]+)", col) # 115 + In
    if m2:
        mass, elem = m2.groups()
        toks.add(f"{mass}{elem}")
    return list(toks)

fcs_paths = sorted(glob.glob(os.path.join(str(RAW_DIR), "**", "*.fcs"), recursive=True))
print("FCS files:", len(fcs_paths))

```

FCS files: 46

```

In [ ]: # Count how many files each protein appears in
from collections import Counter
prot_coverage = Counter()

per_file_map = {} # filename -> {channel_name: protein}
for fp in fcs_paths:
    cols = _list_columns_fast(fp)
    ch2prot = {}
    for c in cols:
        for tok in _iso_tokens_from_col(c):
            if tok in iso_to_protein:
                ch2prot[c] = iso_to_protein[tok]
                break
    per_file_map[os.path.basename(fp)] = ch2prot
    prot_coverage.update(ch2prot.values())

coverage_df = (pd.Series(prot_coverage, name="files_present_in")
               .sort_values(ascending=False).to_frame())
coverage_df["total_files"] = len(fcs_paths)
coverage_df["coverage_pct"] = 100 * coverage_df["files_present_in"] / coverage_df["total_files"]
coverage_df.head(46)

```

Out[]:

	files_present_in	total_files	coverage_pct
CD45	46	46	100.0
CD33	46	46	100.0
CD14	46	46	100.0
HLADR	46	46	100.0
RORgd	46	46	100.0
pERK	46	46	100.0
CD3	46	46	100.0
CD25	46	46	100.0
pSTAT6	46	46	100.0
GATA3	46	46	100.0
NFkB	46	46	100.0
CD16	46	46	100.0
IkB	46	46	100.0
cPARP	46	46	100.0
Tbet	46	46	100.0
MAPKAPK2	46	46	100.0
S6	46	46	100.0
CD66	46	46	100.0
STAT3	46	46	100.0
STAT1	46	46	100.0
TCRgd	46	46	100.0
p38	46	46	100.0
STAT5	46	46	100.0
CREB	46	46	100.0
CD123	46	46	100.0
CD11c	46	46	100.0
CD8a	46	46	100.0
CD4	46	46	100.0
CD11b	46	46	100.0
CD45RA	46	46	100.0
CD19	46	46	100.0
CD7	46	46	100.0
CD56	46	46	100.0

```
In [ ]: try:
        os.makedirs(str(CLEAN_DIR), exist_ok=True)
    except NameError:
        CLEAN_DIR = None

    try:
        os.makedirs(str(QC_DIR), exist_ok=True)
    except NameError:
        QC_DIR = None
```

```
In [ ]: # Keep proteins present in most files (adjust threshold if you like)
KEEP_PROTEINS = set(coverage_df.query("coverage_pct >= 90").index.tolist())
print("Keep proteins (n):", len(KEEP_PROTEINS))

# Load sample-level metadata
fcs_info_path = os.path.join(str(RAW_DIR), "fcs_info.csv")
fcs_info = pd.read_csv(fcs_info_path)[["file_id", "condition", "age", "age_label"]]

def _read_full(fp):
    try:
        meta, df = fcs_parse(fp, reformat_meta=True, channel_naming="$PnN")
    except TypeError:
        meta, df = fcs_parse(fp, reformat_meta=True)
    return meta, df

def _arcsinh_frame(df, cols, c):
    X = df[cols].astype("float32")
    return pd.DataFrame(np.arcsinh(X / c), columns=cols)

qc_rows = []
os.makedirs(str(CLEAN_DIR), exist_ok=True)
os.makedirs(str(QC_DIR), exist_ok=True)
```

Keep proteins (n): 33

```
In [ ]: # ROOT FOLDERS (use existing if you already set them; otherwise default)
if 'RAW_DIR' not in globals(): RAW_DIR = "data_raw"
if 'CLEAN_DIR' not in globals(): CLEAN_DIR = "data_clean"
if 'QC_DIR' not in globals(): QC_DIR = "results_qc"

os.makedirs(CLEAN_DIR, exist_ok=True)
os.makedirs(QC_DIR, exist_ok=True)

# Transform cofactor (CyTOF convention)
if 'COFACTOR' not in globals(): COFACTOR = 5.0
np.random.seed(42)

# Helper: safe FCS read with $PnN first, fallback if needed
def read_fcs(fp):
    try:
        meta, df = fcs_parse(fp, reformat_meta=True, channel_naming="$PnN")
    except TypeError:
        meta, df = fcs_parse(fp, reformat_meta=True) # older fcsparser
    return meta, df

# Helper: list channel names *without* loading full data when possible
def list_columns_fast(fp):
    # Try meta-only
```

```

try:
    res = fcs_parse(fp, meta_data_only=True, reformat_meta=True, channel_name=
    meta = res[0] if isinstance(res, (tuple, list)) else res
    ch_tbl = meta.get("_channels_", None)
    if ch_tbl is not None:
        if "$PnN" in ch_tbl.columns:
            return list(ch_tbl["$PnN"].astype(str))
        if "$PnS" in ch_tbl.columns:
            return list(ch_tbl["$PnS"].astype(str))
except TypeError:
    pass
# Fallback: tiny full read
try:
    _, df = read_fcs(fp)
    return list(df.columns)
except Exception:
    return []

# Helper: pull isotope tokens from a column name (supports 115In, In115, Nd142Di
def isotope_tokens(colname):
    toks = set()
    m1 = re.search(r"([A-Za-z]+)(\d+)", colname) # In + 115
    if m1:
        elem, mass = m1.groups()
        toks.add(f"{mass}{elem}")
    m2 = re.search(r"(\d+)([A-Za-z]+)", colname) # 115 + In
    if m2:
        mass, elem = m2.groups()
        toks.add(f"{mass}{elem}")
    return list(toks)

```

```

In [ ]: # Clean and Save (all cells, arcsinh, protein names, with metadata)
# Load sample-level metadata
fcs_info_path = os.path.join(RAW_DIR, "fcs_info.csv")
fcs_info = pd.read_csv(fcs_info_path)[["file_id", "condition", "age", "age_label"]]

def arcsinh_df(df, cols, cofactor):
    X = df[cols].astype("float32")
    return pd.DataFrame(np.arcsinh(X / cofactor), columns=cols)

qc_rows = []

for fp in fcs_paths:
    fname = os.path.basename(fp)

    # find metadata row
    mrow = fcs_info.loc[fcs_info["file_id"] == fname]
    if mrow.empty:
        warnings.warn(f"{fname}: missing in fcs_info.csv; skipping")
        continue
    mrow = mrow.iloc[0]

    # read full FCS
    try:
        _, df = read_fcs(fp)
    except Exception as e:
        warnings.warn(f"{fname}: read failed ({e})")
        continue

    # pick channels in this file that map to KEPT proteins

```

```

ch2prot = per_file_map.get(fname, {})
keep_channels = [ch for ch, prot in ch2prot.items() if prot in KEEP_PROTEINS
if not keep_channels:
    warnings.warn(f"{fname}: no usable channels; skipping")
    continue

# arcsinh transform and rename columns to protein names
X = arcsinh_df(df, keep_channels, COFACTOR)
X.columns = [ch2prot[ch] for ch in keep_channels]

# drop any NaN rows (usually rare)
n_before = len(X)
n_nan = int(X.isna().any(axis=1).sum())
if n_nan:
    X = X.dropna(axis=0, how="any")

# attach sample-level metadata
X["condition"] = int(mrow["condition"])
X["age"] = float(mrow["age"])
X["age_label"] = int(mrow["age_label"])
X["filename"] = fname

# save: try Parquet first; if it fails, auto-fallback to CSV (same basename)
out_parquet = os.path.join(CLEAN_DIR, f"{fname}.parquet")
try:
    X.to_parquet(out_parquet, index=False)
    saved_as = out_parquet
except Exception as e:
    out_csv = out_parquet.replace(".parquet", ".csv")
    print(f"Parquet failed for {fname} ({type(e).__name__}) - saving CSV ins
    X.to_csv(out_csv, index=False)
    saved_as = out_csv

# record QC
qc_rows.append({
    "filename": fname,
    "n_cells_before": n_before,
    "n_nan_rows": n_nan,
    "n_cells_after": len(X),
    "n_markers": len(KEEP_PROTEINS),
    "saved_as": saved_as
})

# QC summary
qc_df = pd.DataFrame(qc_rows).sort_values("n_cells_after", ascending=False)
qc_csv = os.path.join(QC_DIR, "per_file_qc.csv")
qc_df.to_csv(qc_csv, index=False)
display(qc_df.head(10))
print("QC summary saved to:", qc_csv)

```

	filename	n_cells_before	n_nan_rows	n_cells_after	n_markers
25	PBMC_1073311268_Ctrl_unstim.fcs	555448	0	555448	33
12	PBMC_1071211138_Ctrl_unstim.fcs	419065	0	419065	33
32	PBMC_1080911118_PreE_unstim.fcs	373463	0	373463	33
26	PBMC_1073611138_PreE_unstim.fcs	343023	0	343023	33
33	PBMC_1080911248_PreE_unstim.fcs	333504	0	333504	33
20	PBMC_1072711118_PreE_unstim.fcs	306914	0	306914	33
3	PBMC_1052411268_Ctrl_unstim.fcs	303004	0	303004	33
24	PBMC_1073311128_Ctrl_unstim.fcs	282767	0	282767	33
31	PBMC_1080411288_Ctrl_unstim.fcs	271652	0	271652	33
35	PBMC_1081611168_PreE_unstim.fcs	270965	0	270965	33



QC summary saved to: results_qc/per_file_qc.csv

Visualization

```
In [ ]: # Viewing marker intensity distribution in each file
CLEAN_DIR = "data_clean" # adjust if different

def load_clean_any(path_parquet):
    csv = path_parquet.replace(".parquet", ".csv")
    if os.path.exists(path_parquet):
        return pd.read_parquet(path_parquet)
    elif os.path.exists(csv):
        return pd.read_csv(csv)
    else:
        raise FileNotFoundError(path_parquet + " (or CSV) not found.")

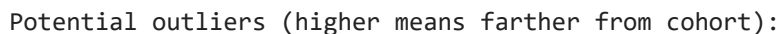
parqs = sorted(glob.glob(os.path.join(CLEAN_DIR, "*.parquet")))
csvs = sorted(glob.glob(os.path.join(CLEAN_DIR, "*.csv")))
files = sorted(set([p.replace(".csv", ".parquet") for p in csvs] + parqs))
print("Cleaned files:", len(files))

rows = []
for p in files:
    df = load_clean_any(p)
    markers = [c for c in df.columns if c not in ("condition", "age", "age_label",
    med = df[markers].median().rename("median")
    iqr = (df[markers].quantile(0.75) - df[markers].quantile(0.25)).rename("IQR")
    tmp = pd.concat([med, iqr], axis=1)
    tmp["filename"] = os.path.basename(p).replace(".parquet", "").replace(".csv")
    tmp["condition"] = int(df["condition"].iat[0])
    tmp["age"] = float(df["age"].iat[0])
    rows.append(tmp.reset_index().rename(columns={"index": "marker"}))

file_stats = pd.concat(rows, ignore_index=True)

# pivot medians for a simple heatmap
med_piv = file_stats.pivot(index="filename", columns="marker", values="median")
plt.figure(figsize=(0.45*med_piv.shape[1]+3, 0.25*med_piv.shape[0]+3))
```

Cleaned files: 46



	mean_abs_z
filename	
PBMC_1080411288_Ctrl_unstim.fcs	2.287053
PBMC_1083111161_Ctrl_unstim.fcs	1.433258
PBMC_1080411158_Ctrl_unstim.fcs	1.242168
PBMC_1072311138_PreE_unstim.fcs	1.041498
PBMC_1073311268_Ctrl_unstim.fcs	0.976113
PBMC_1072711118_PreE_unstim.fcs	0.949045
PBMC_1082111261_Ctrl_unstim.fcs	0.900553
PBMC_1970711248_Ctrl_unstim.fcs	0.890136
PBMC_1052411118_Ctrl_unstim.fcs	0.878330
PBMC_1083111111_Ctrl_unstim.fcs	0.864636

Selecting only 5k cells per sample for EDA and then clustering later

```
In [ ]: N_PER_FILE_EDA = 5000 # Selecting only 5k cells per sample
parts = []
for p in files:
    df = load_clean_any(p)
    take = min(N_PER_FILE_EDA, len(df))
    parts.append(df.sample(take, random_state=42))
eda = pd.concat(parts, ignore_index=True)
print("EDA sample shape:", eda.shape)
eda.head()
```

EDA sample shape: (230000, 37)

```
Out[ ]:      CD45    CD66    CD7    CD19  CD45RA  CD11b    CD4    CD8a  CD11
0  4.381964  0.250840  2.579434  0.000000  0.226083  0.089706  0.000000  0.000000  0.
1  4.406560  0.348977  3.768505  0.095871  3.881555  0.029366  3.742608  0.993199  0.
2  4.468272  0.007222  3.753519  0.000000  2.186813  0.046937  3.928551  0.062450  0.
3  4.600840  0.479660  4.049463  0.000000  2.234242  0.277734  3.952479  0.851627  0.
4  4.363682  0.000000  4.126595  0.918950  4.239831  0.000000  0.158081  3.589533  0.
```

5 rows × 37 columns



Viewing arnicsh compressed marker intensities in histogram

```
In [ ]: # Use all marker columns (exclude metadata)
meta_cols = {"condition", "age", "age_label", "filename", "_umap1", "_umap2"}
markers = [c for c in eda.columns if c not in meta_cols]
print(f"Markers detected (n={len(markers)}):", markers)
```

```

# Histograms for all markers
for m in markers:
    plt.figure(figsize=(5,3.5))
    plt.hist(eda[m].dropna().values, bins=80)
    plt.title(f"{m} (arcsinh)")
    plt.xlabel(m); plt.ylabel("Count")
    plt.tight_layout(); plt.show()

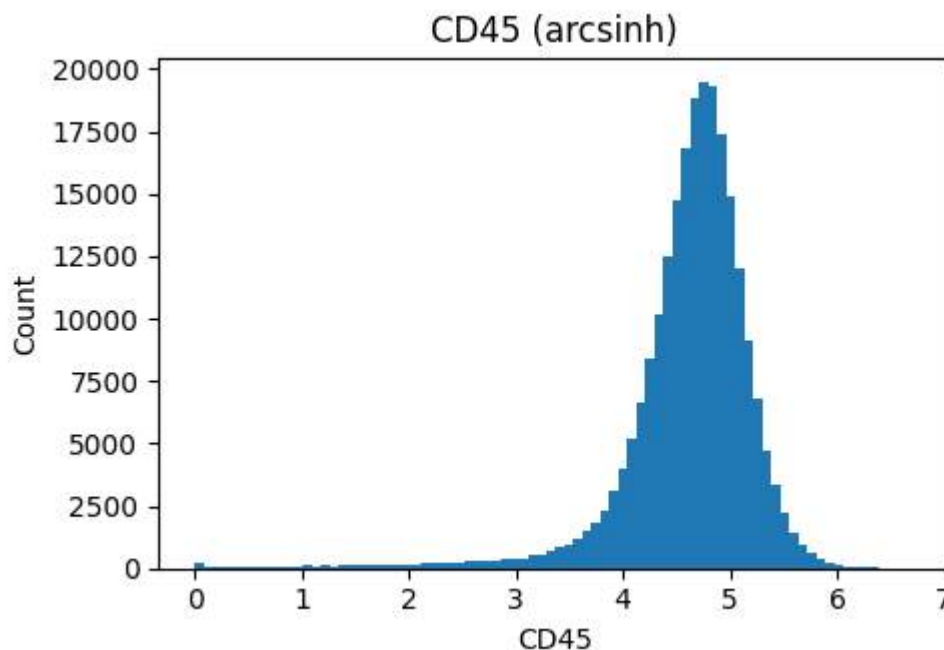
# Small helper to plot a correlation heatmap for a given subset
def plot_corr(df, cols, title):
    if len(cols) == 0 or df.empty:
        print(f"[{title}] Nothing to plot.")
        return
    corr = df[cols].corr()
    plt.figure(figsize=(0.5*len(cols)+2, 0.5*len(cols)+2))
    plt.imshow(corr.values, interpolation="nearest")
    plt.xticks(range(len(cols)), cols, rotation=45, ha="right")
    plt.yticks(range(len(cols)), cols)
    plt.title(title)
    plt.colorbar()
    plt.tight_layout(); plt.show()

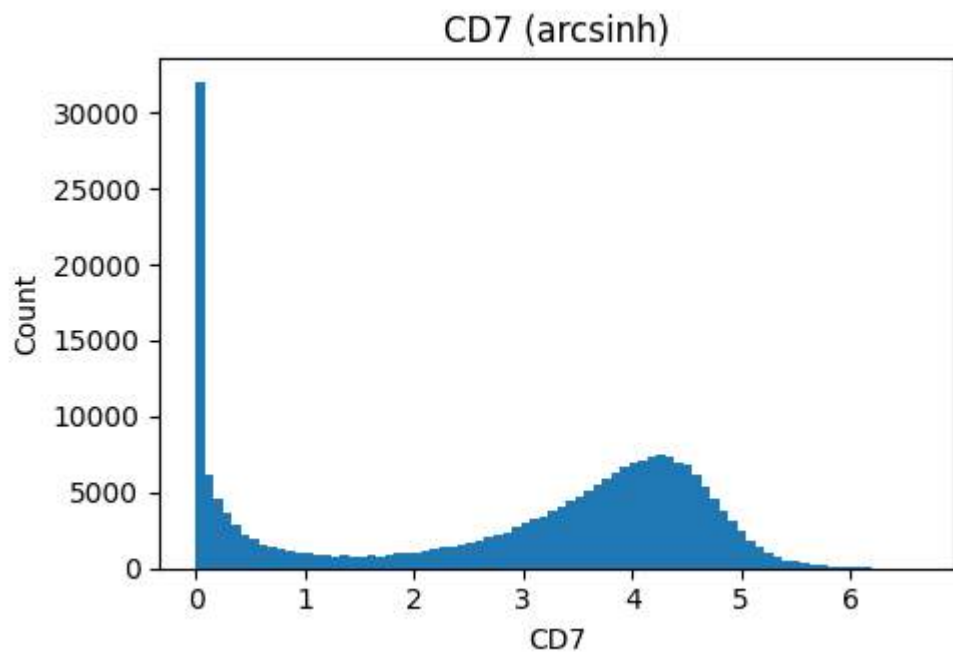
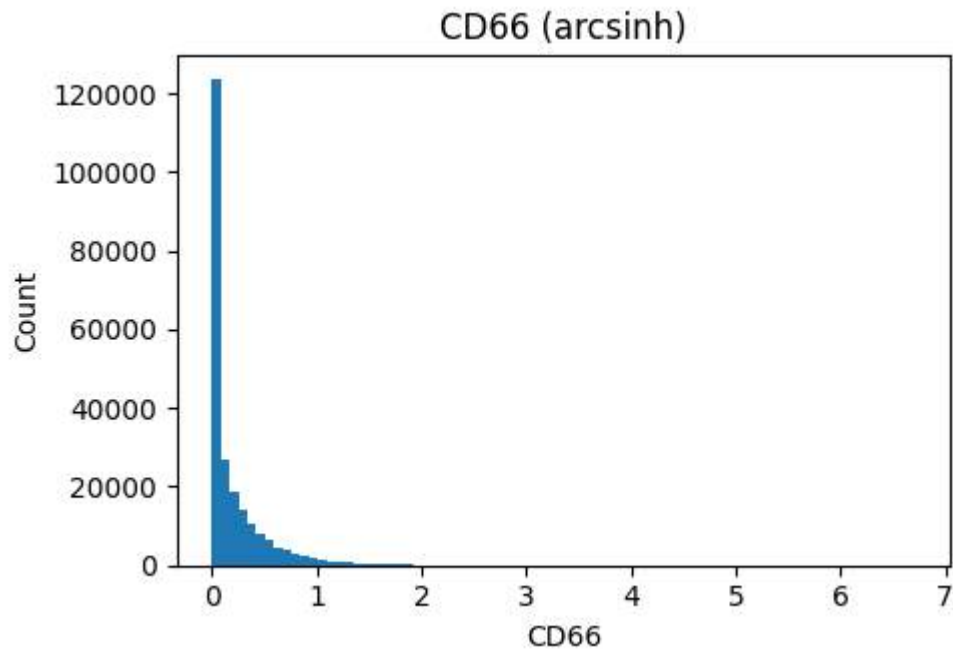
# 3) Heatmap for ALL cells
plot_corr(eda, markers, "Marker-marker correlation (ALL cells)")

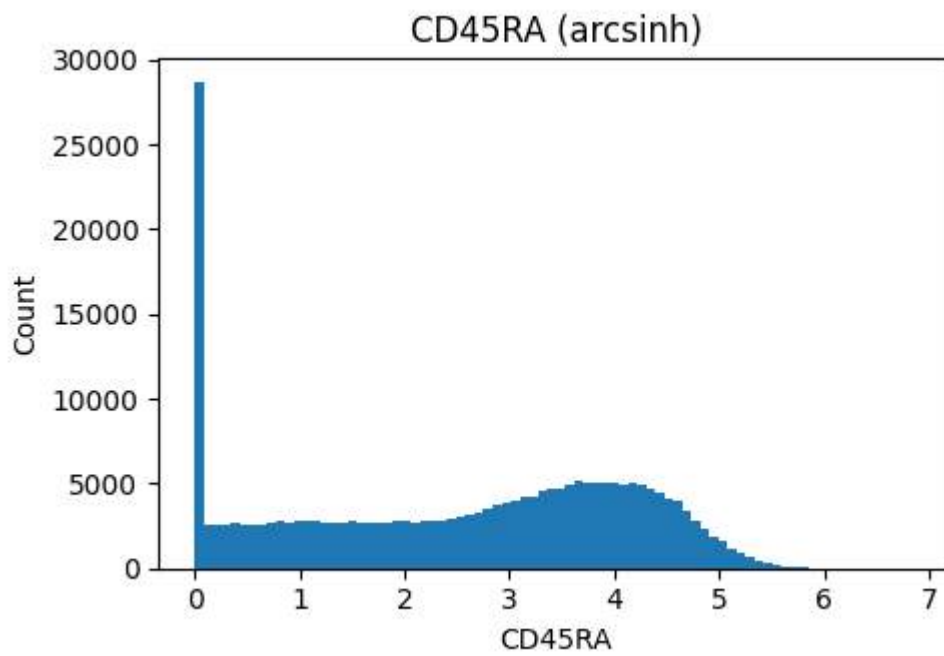
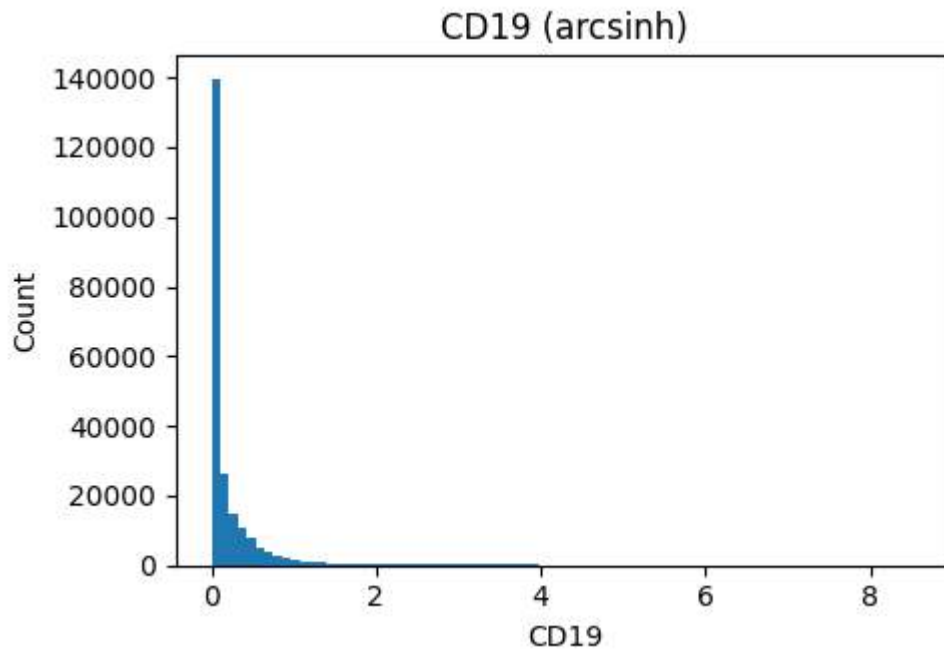
eda_ctrl = eda[eda["condition"] == 0]
eda_pe = eda[eda["condition"] == 1]
plot_corr(eda_ctrl, markers, "Marker-marker correlation (Controls)")
plot_corr(eda_pe, markers, "Marker-marker correlation (PE)")

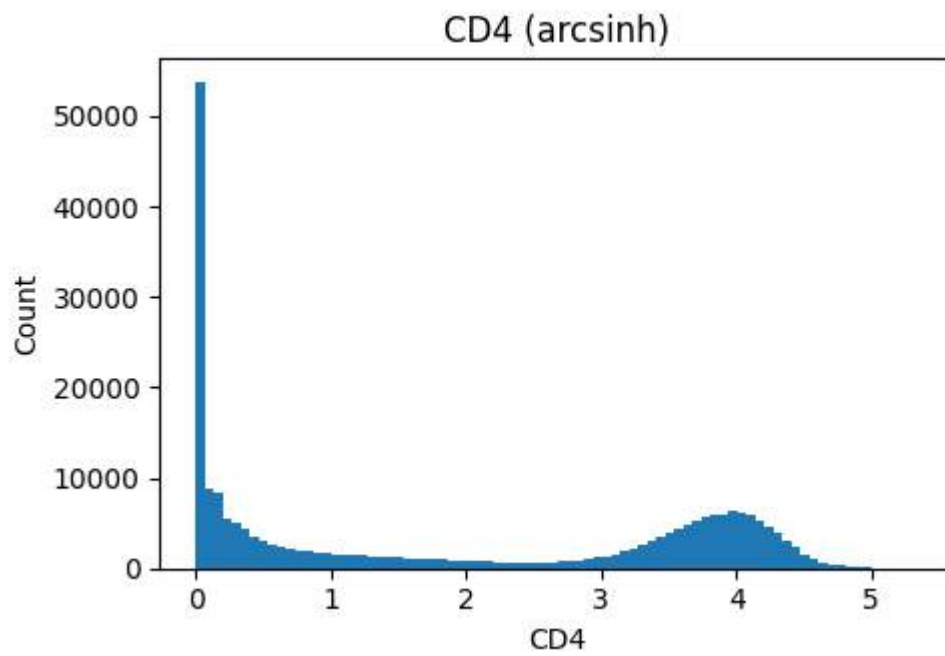
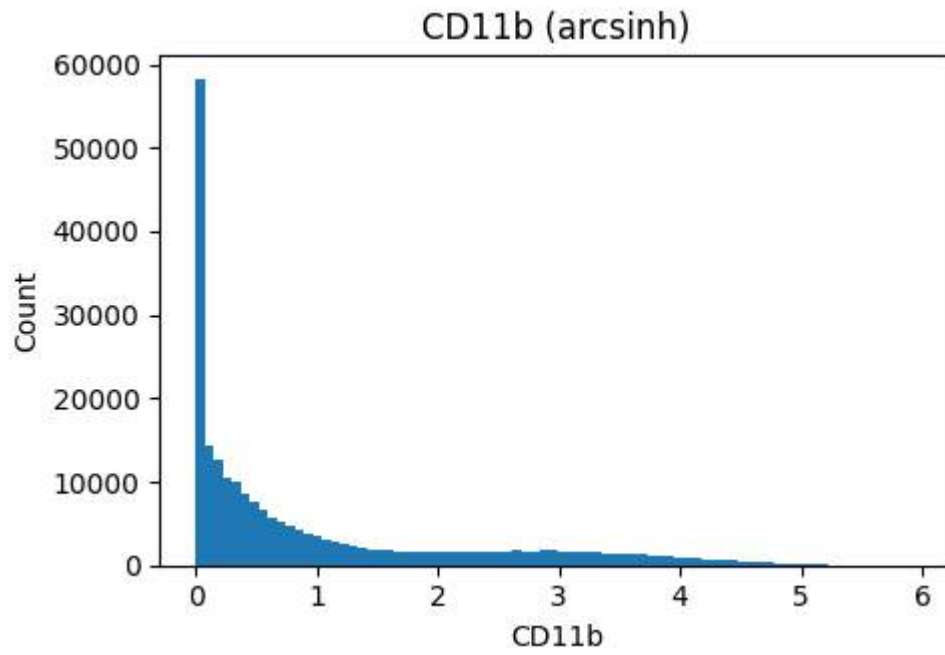
```

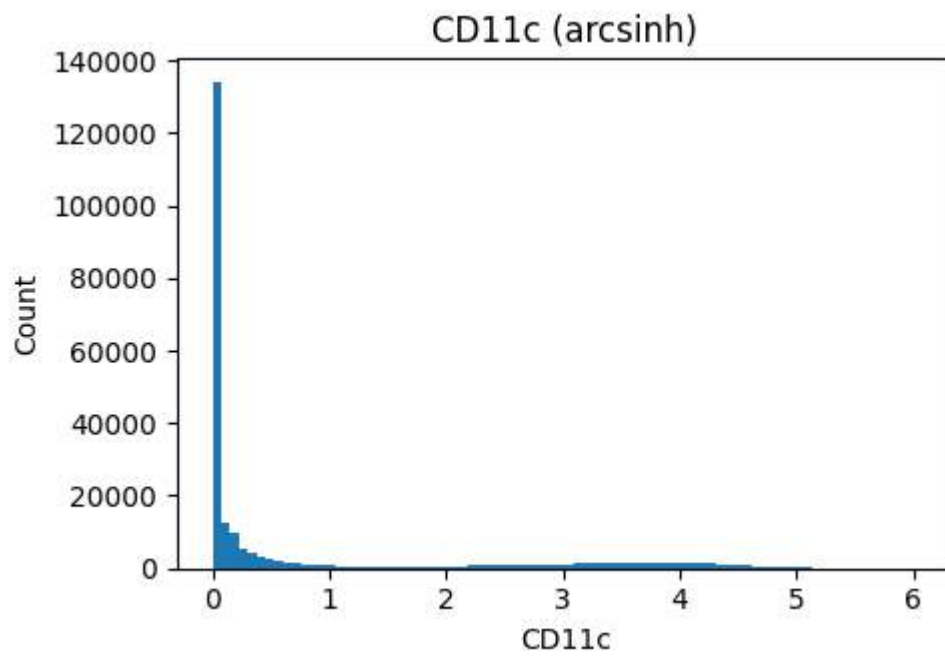
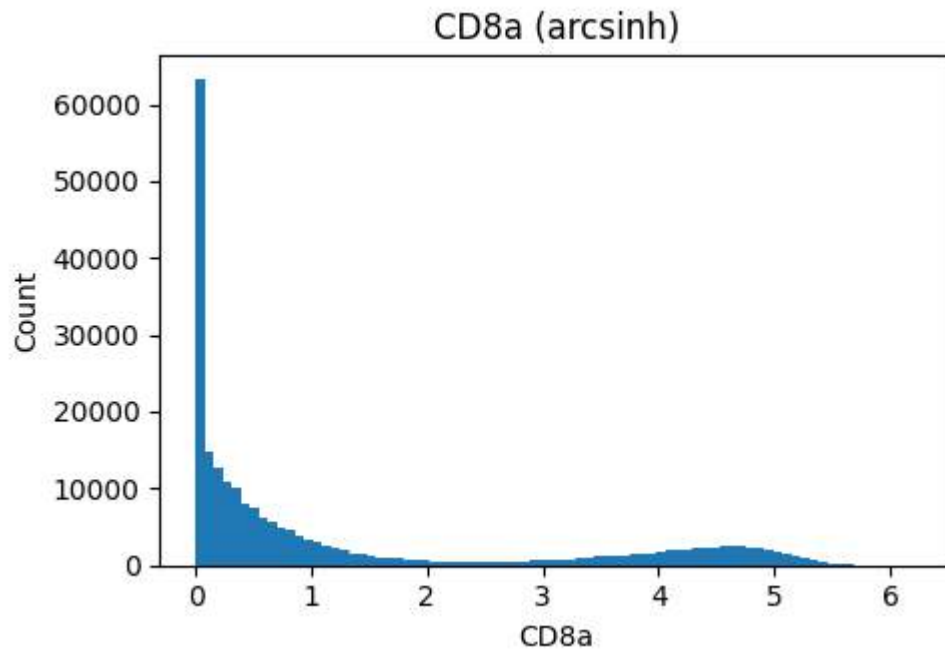
Markers detected (n=33): ['CD45', 'CD66', 'CD7', 'CD19', 'CD45RA', 'CD11b', 'CD4', 'CD8a', 'CD11c', 'CD123', 'CREB', 'STAT5', 'p38', 'TCRgd', 'STAT1', 'STAT3', 'S6', 'CD33', 'MAPKAPK2', 'Tbet', 'cPARP', 'IκB', 'CD16', 'NFκB', 'GATA3', 'pSTAT6', 'CD25', 'CD3', 'pERK', 'RORgd', 'HLADR', 'CD14', 'CD56']

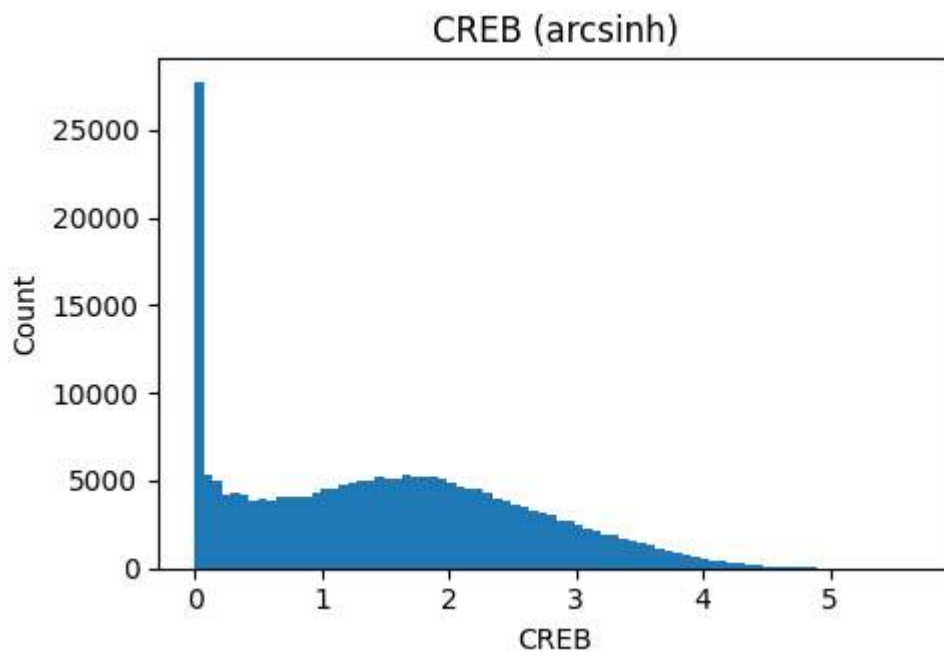
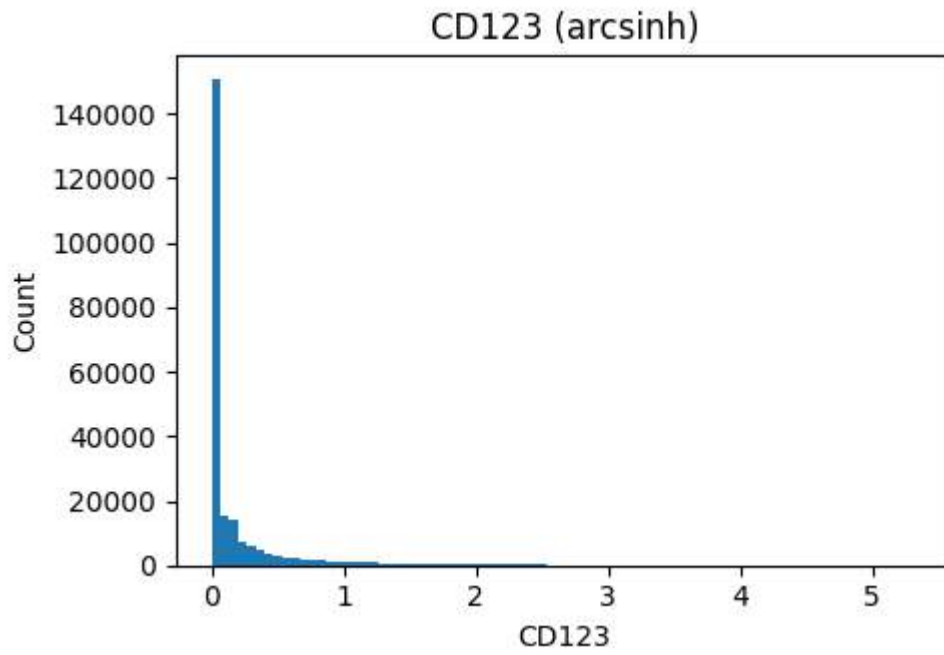


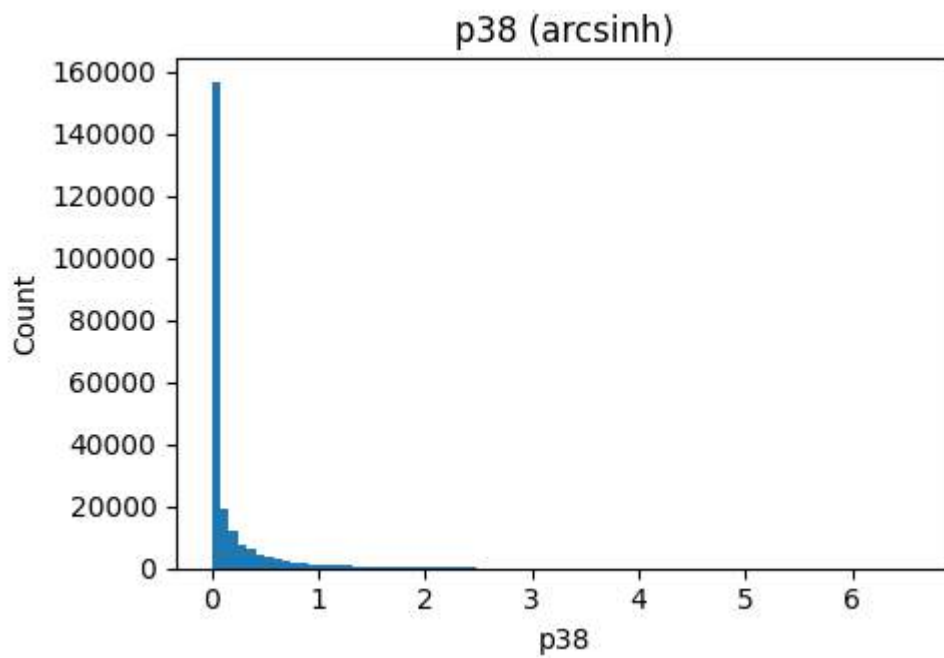
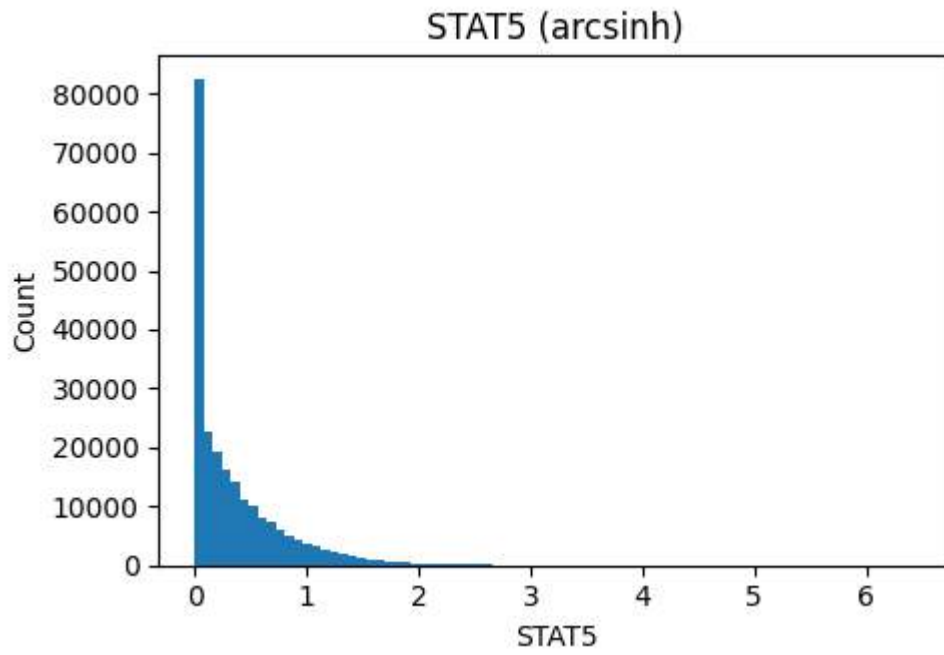


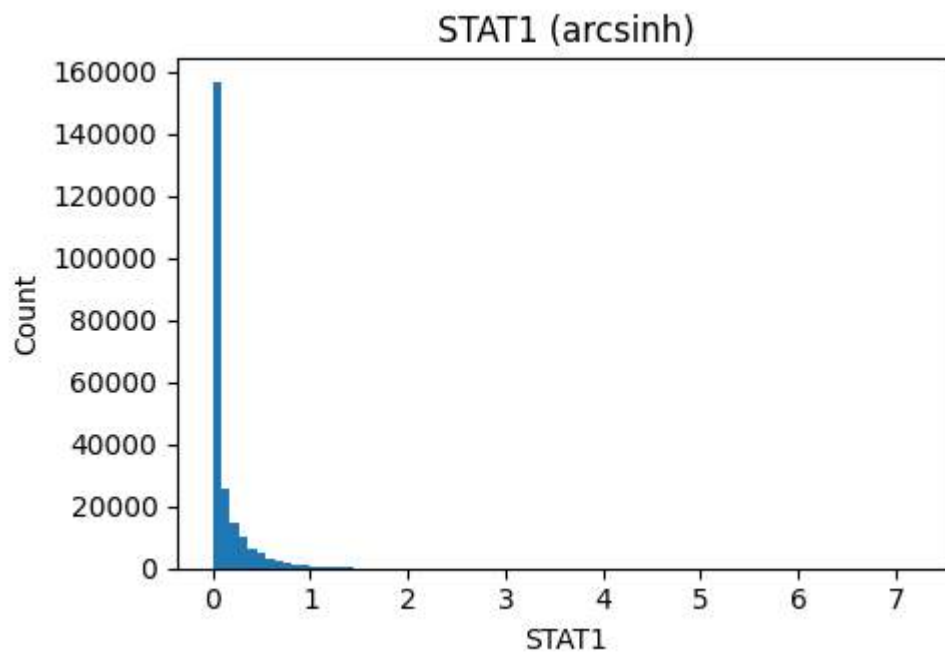
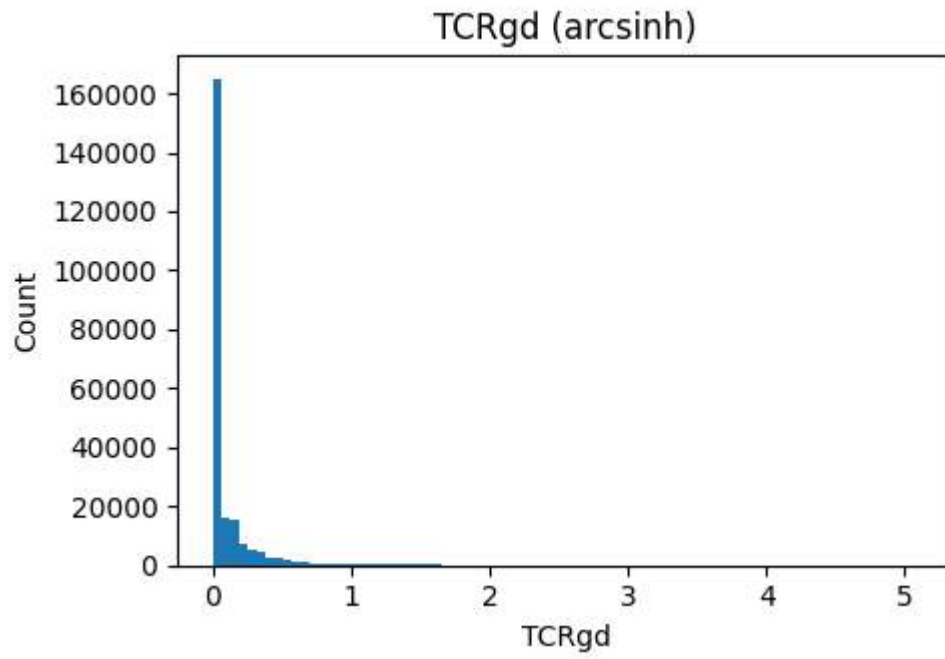


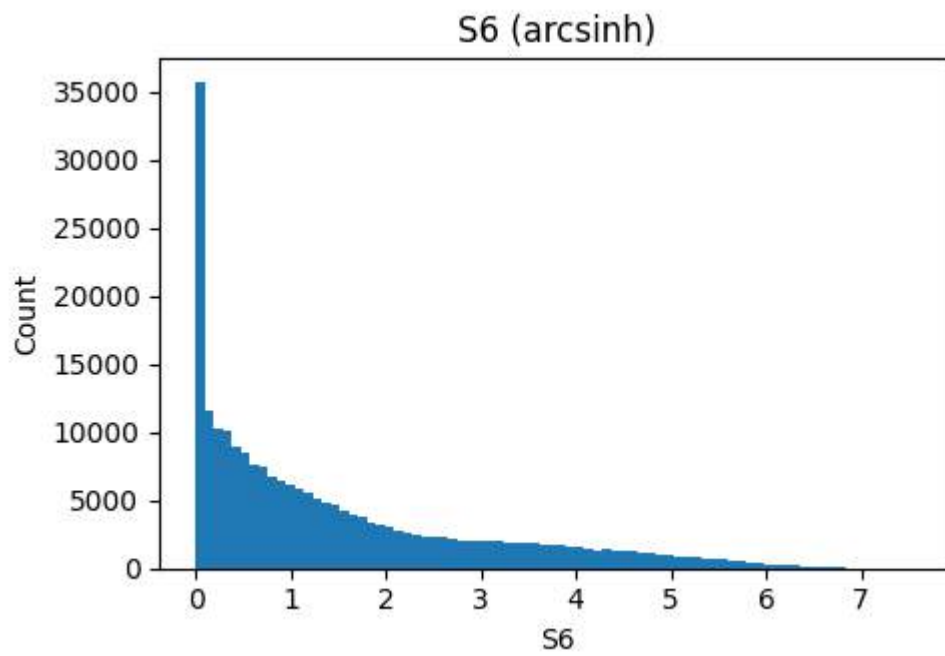
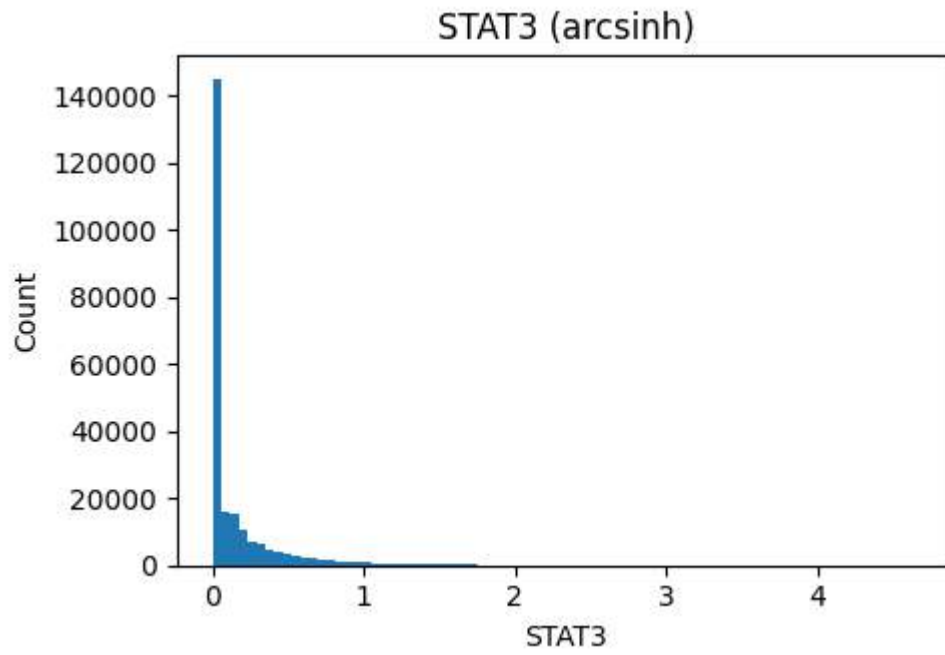


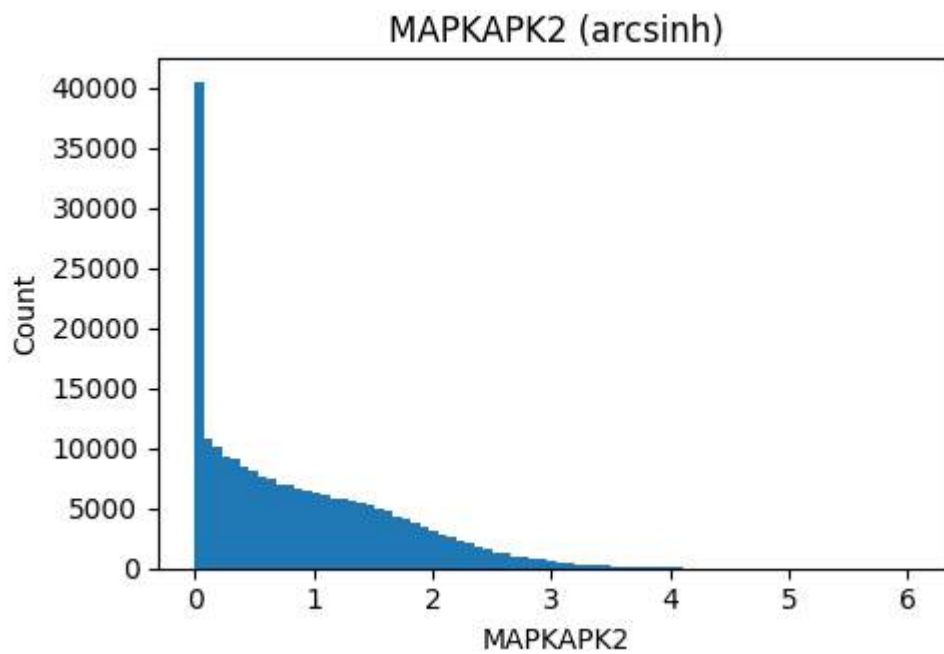
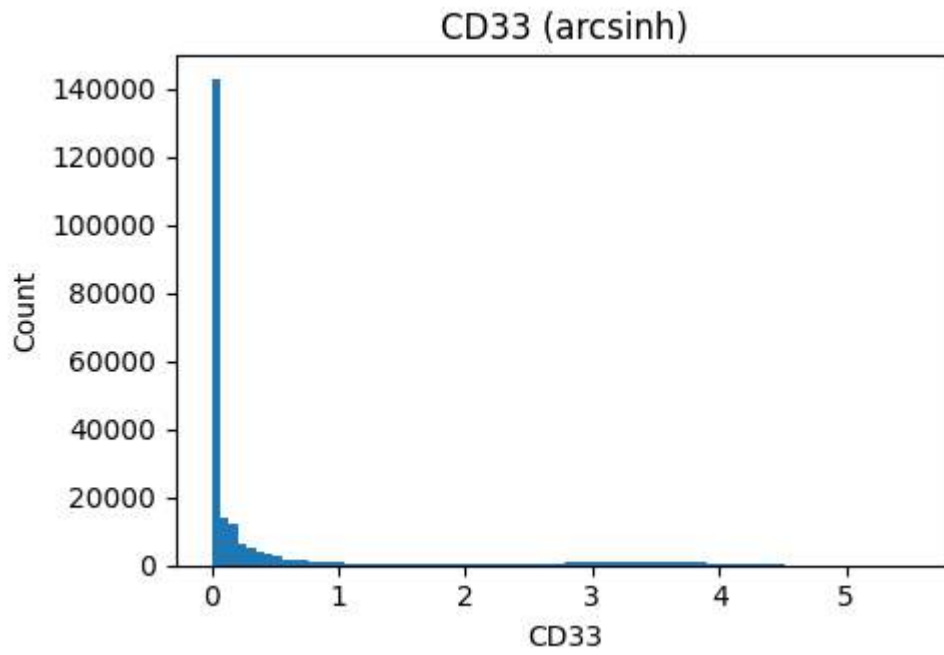


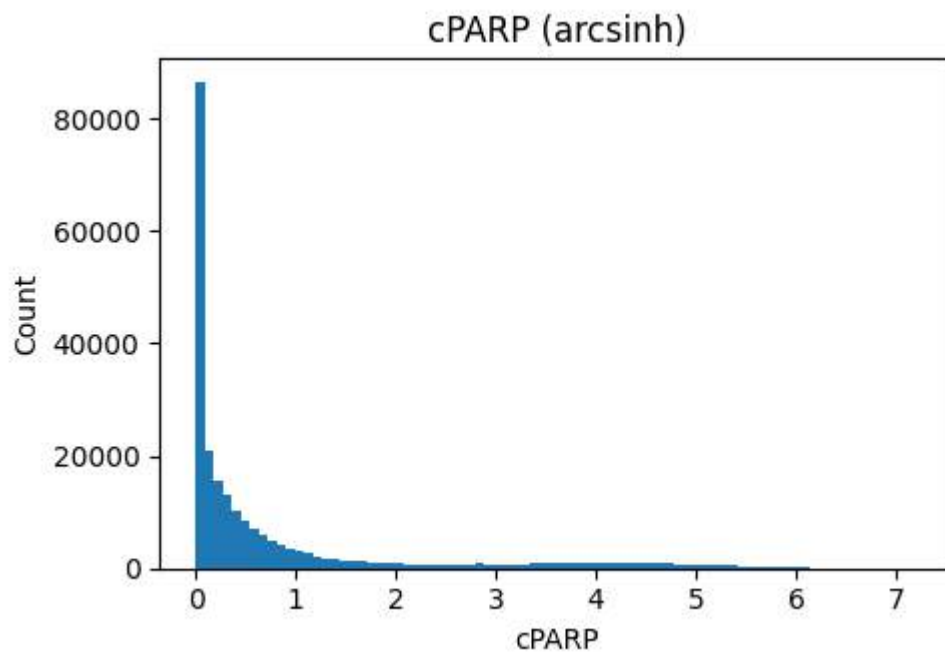
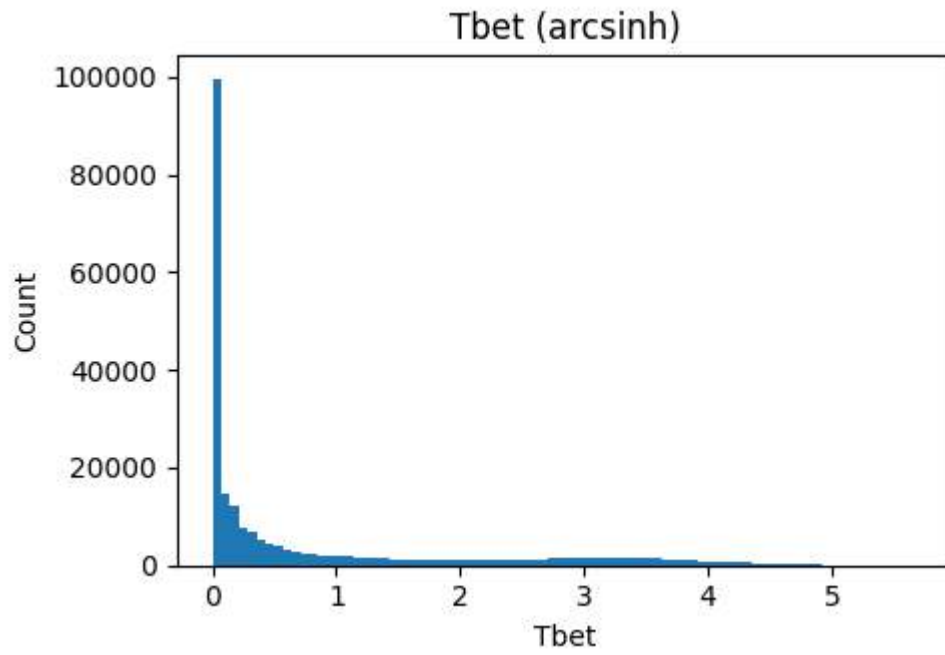


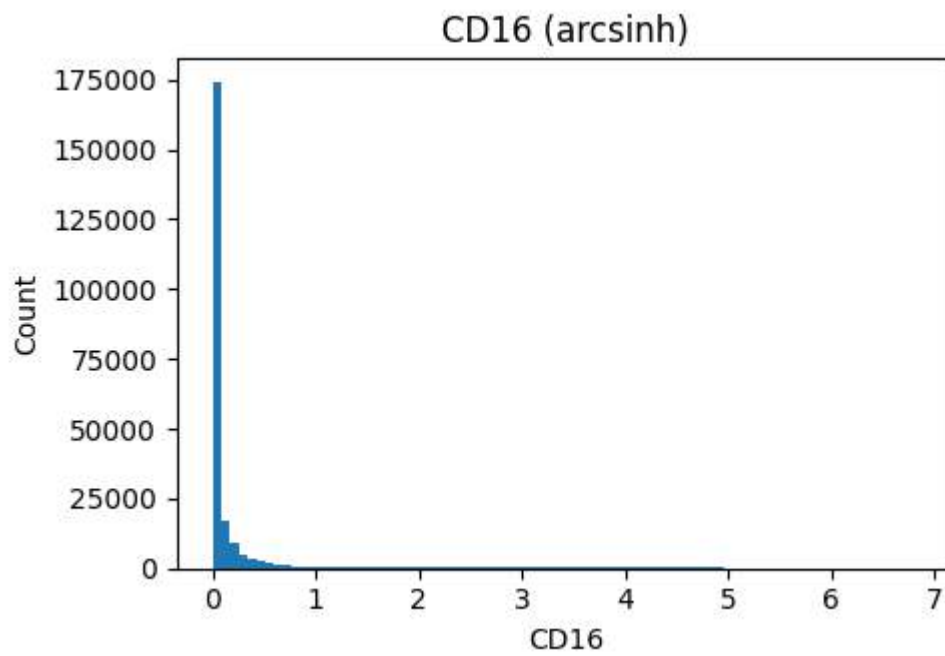
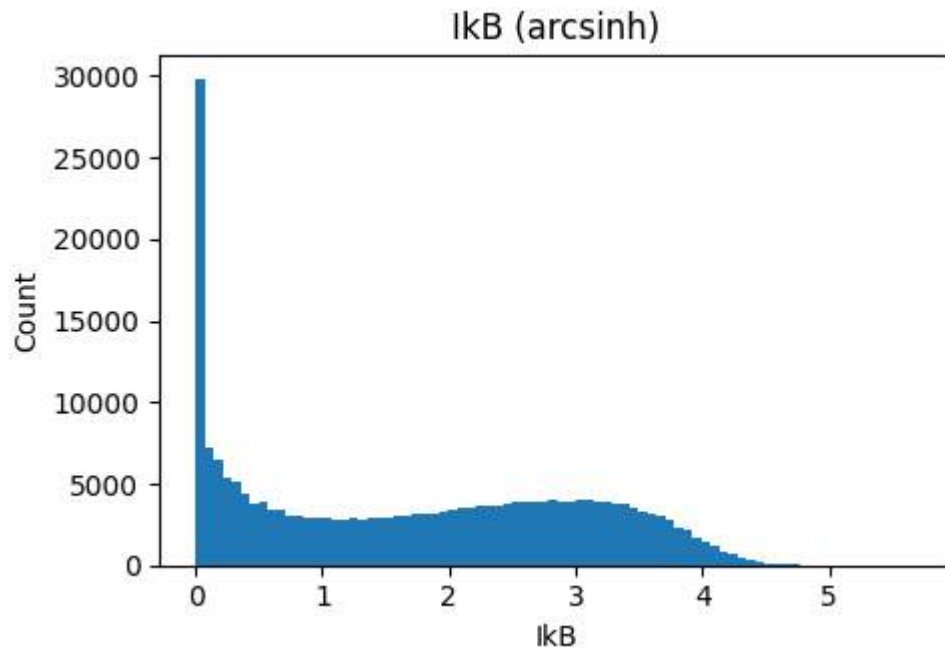


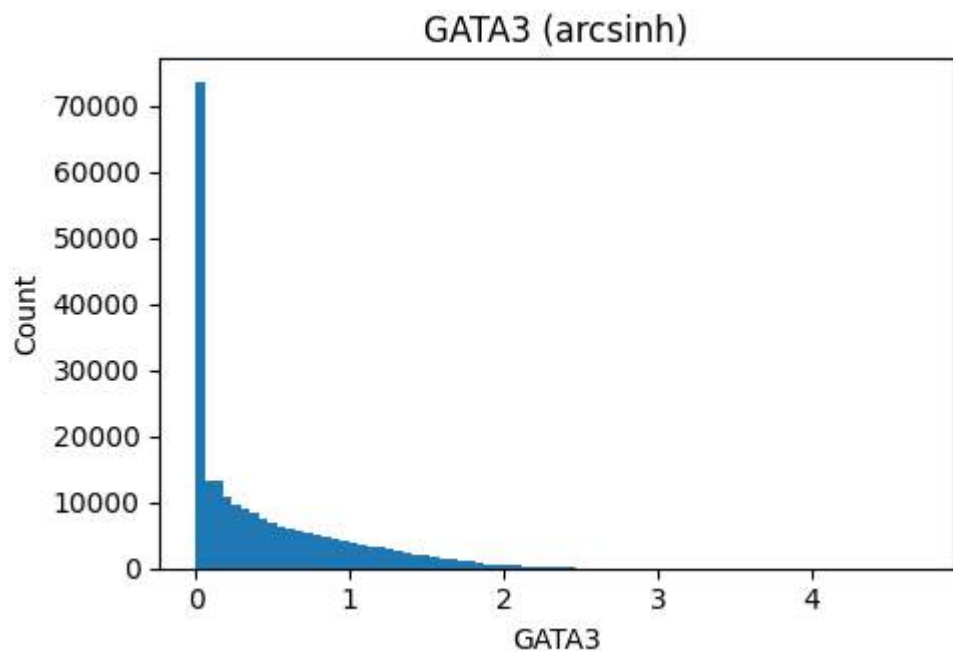
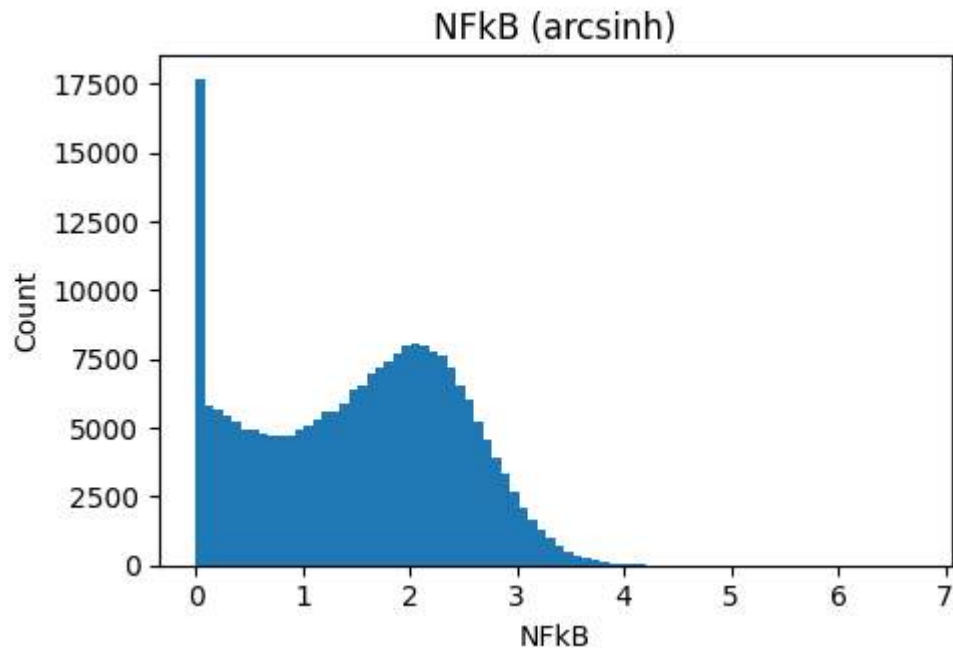


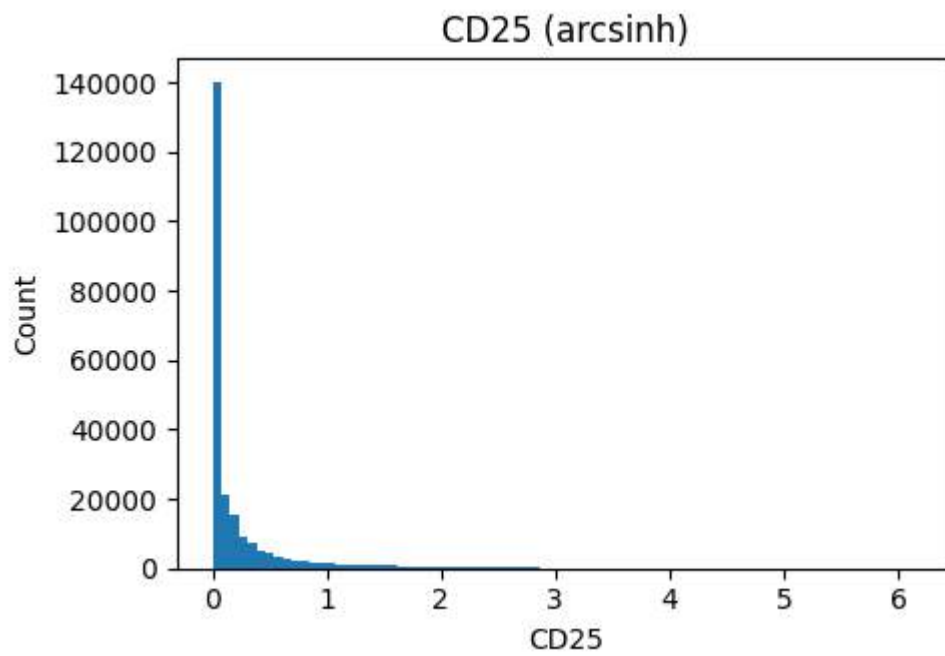
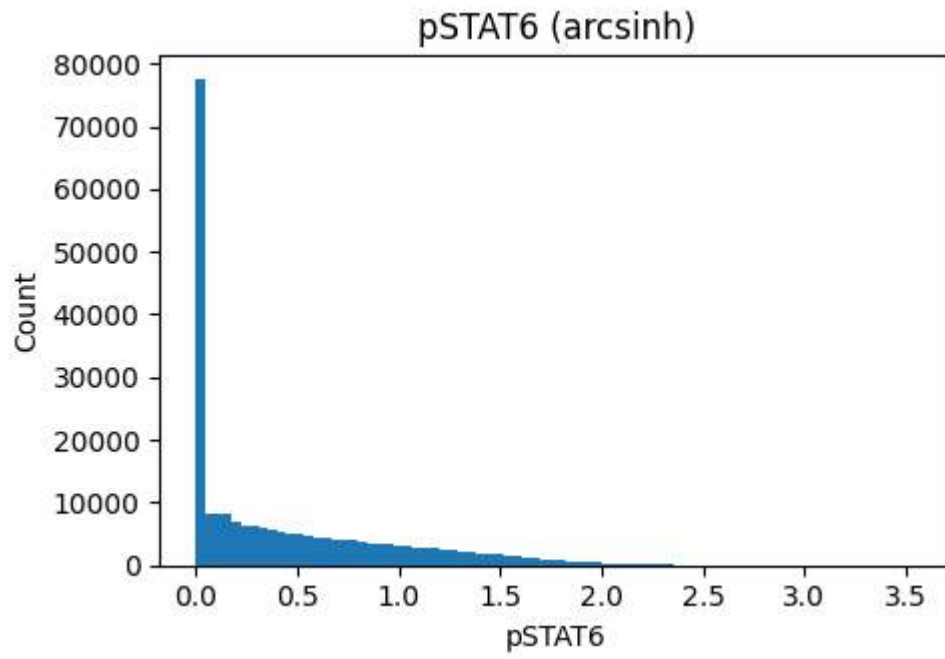


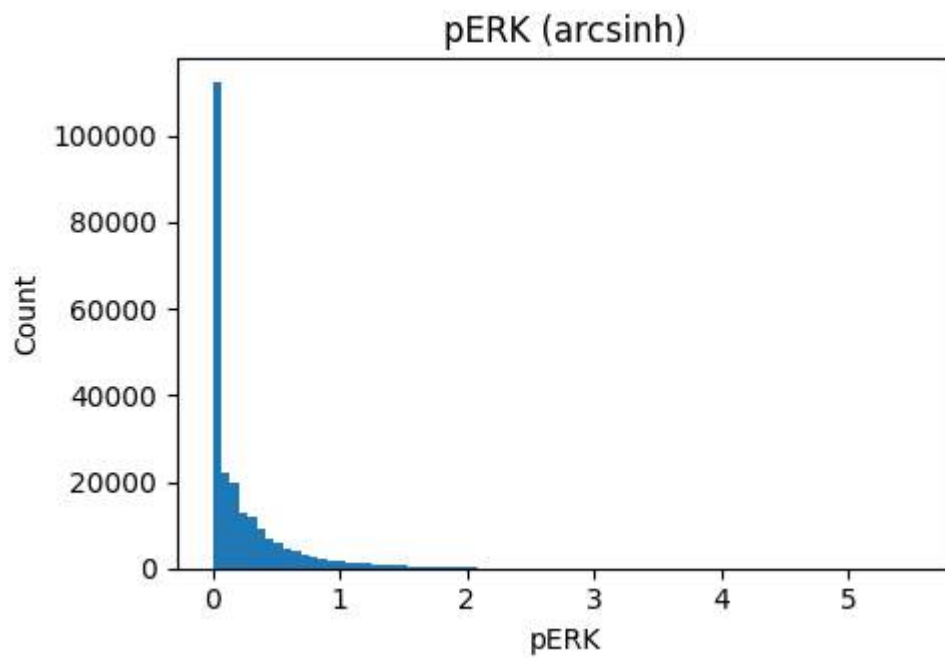
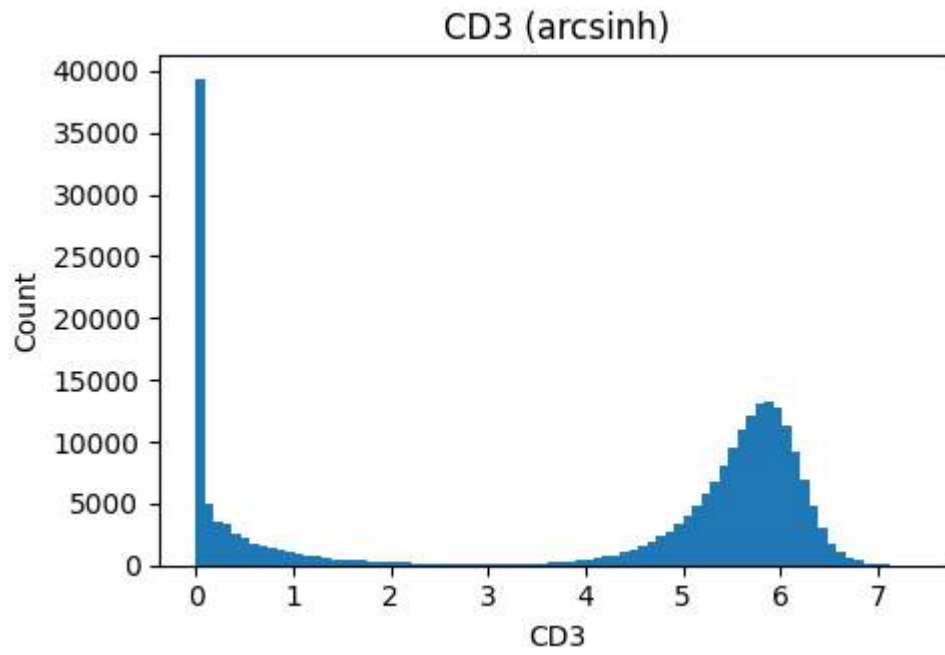


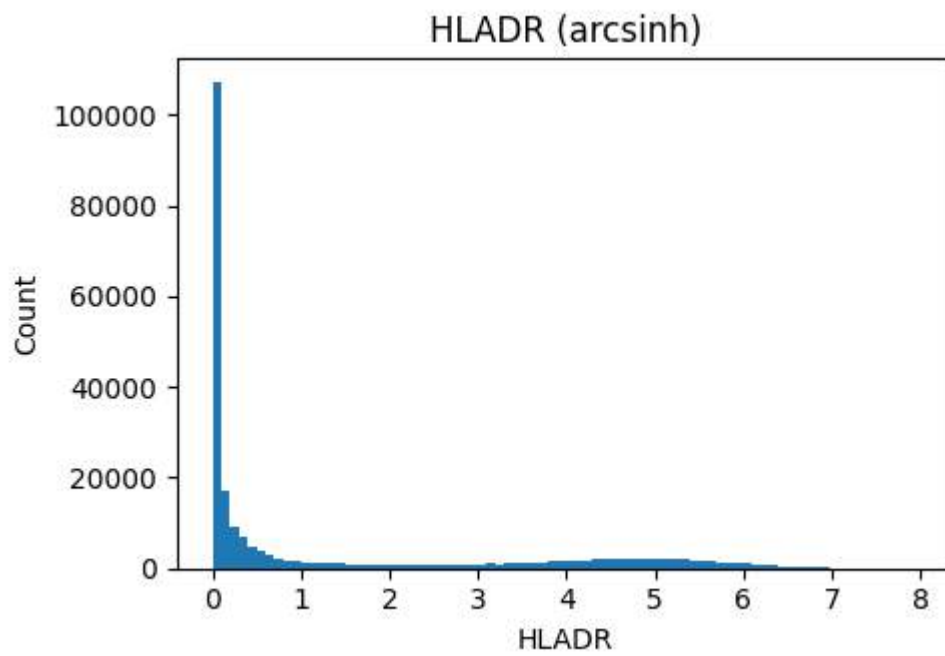
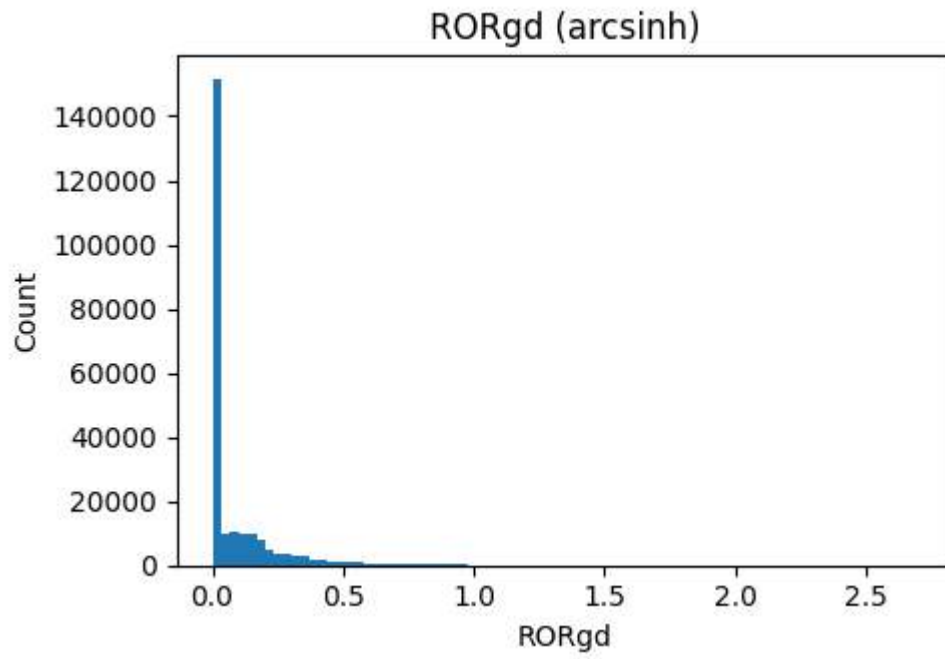


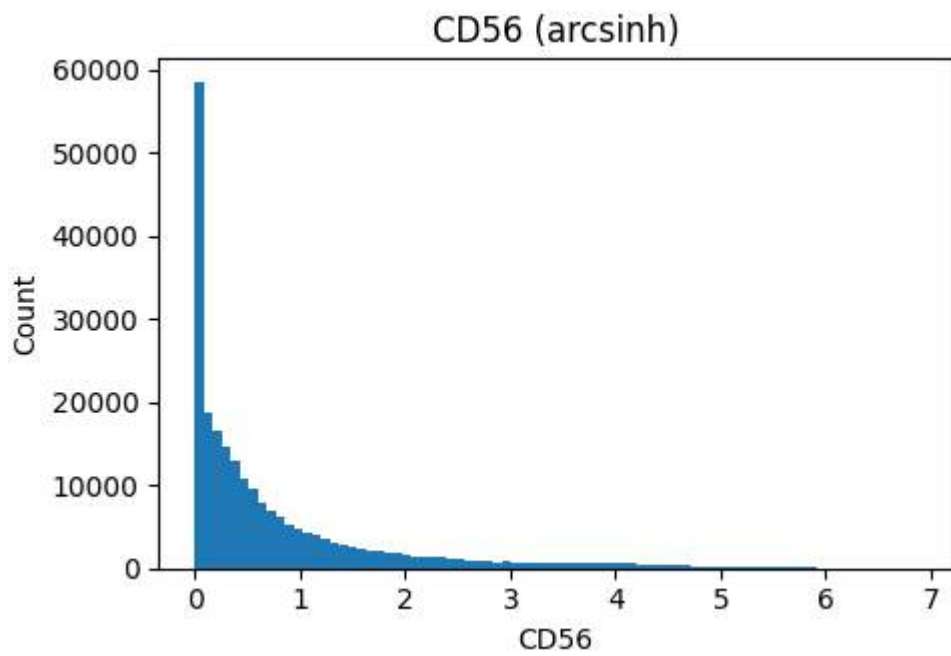
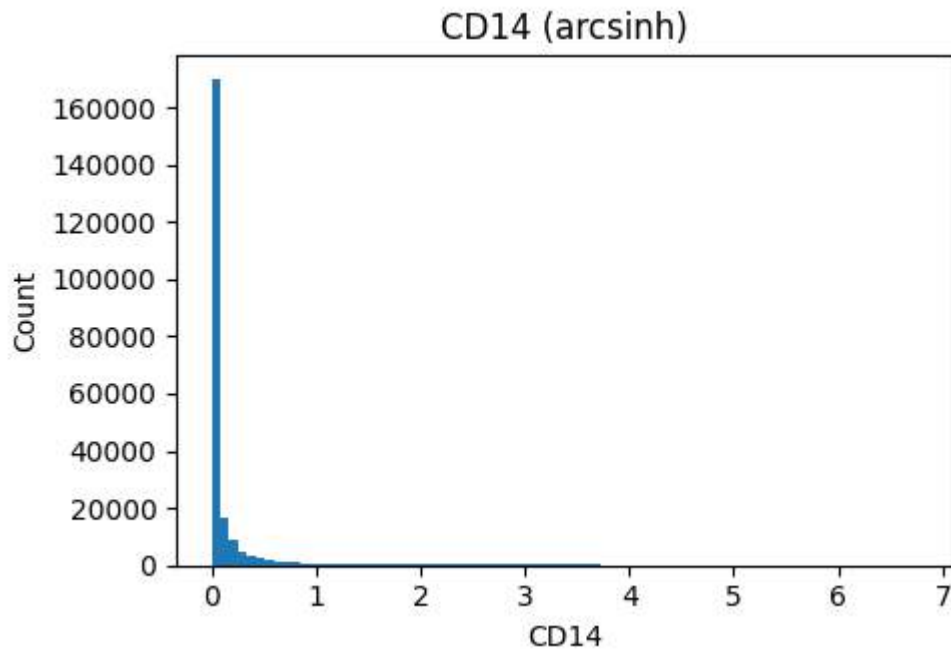


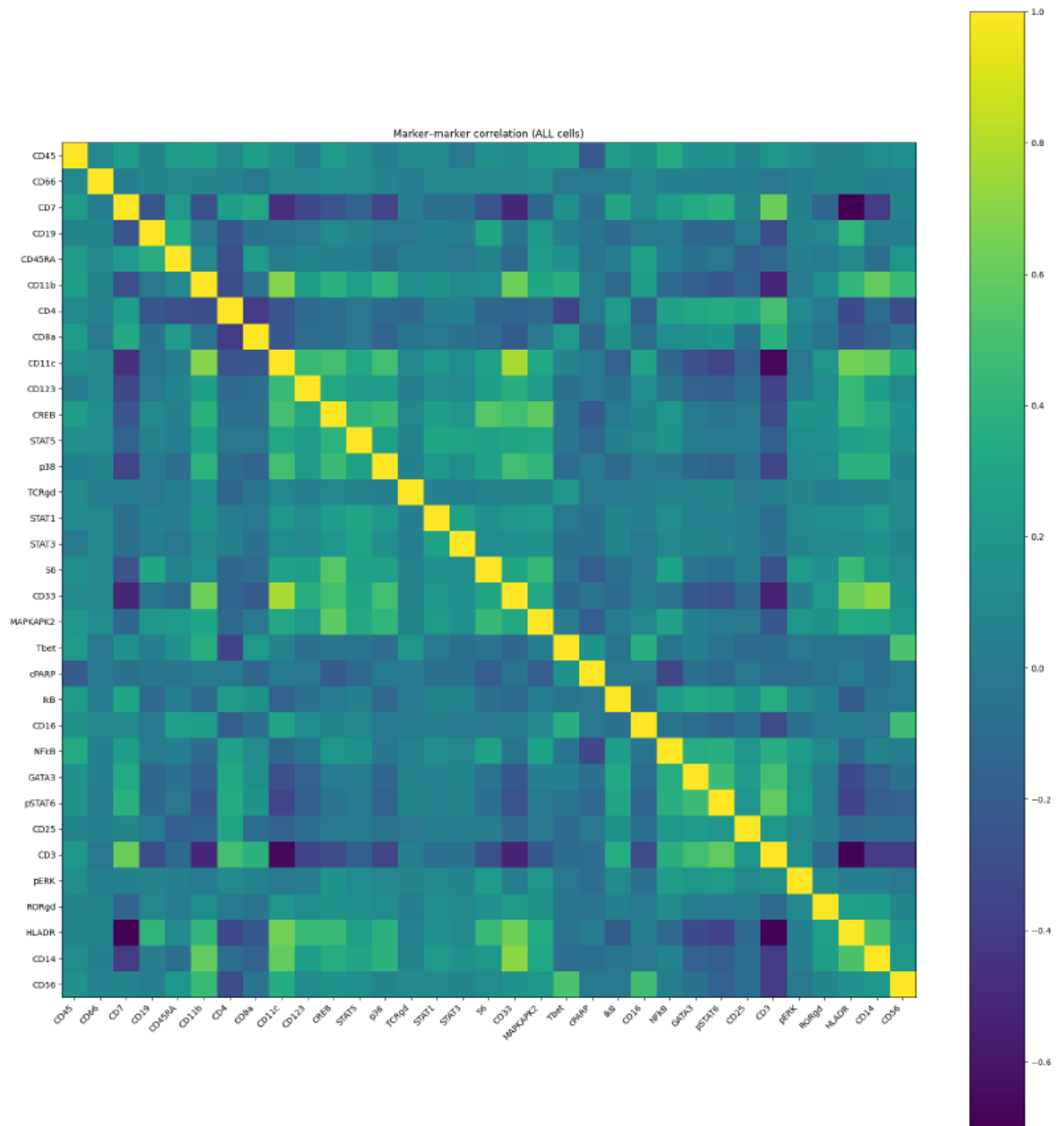


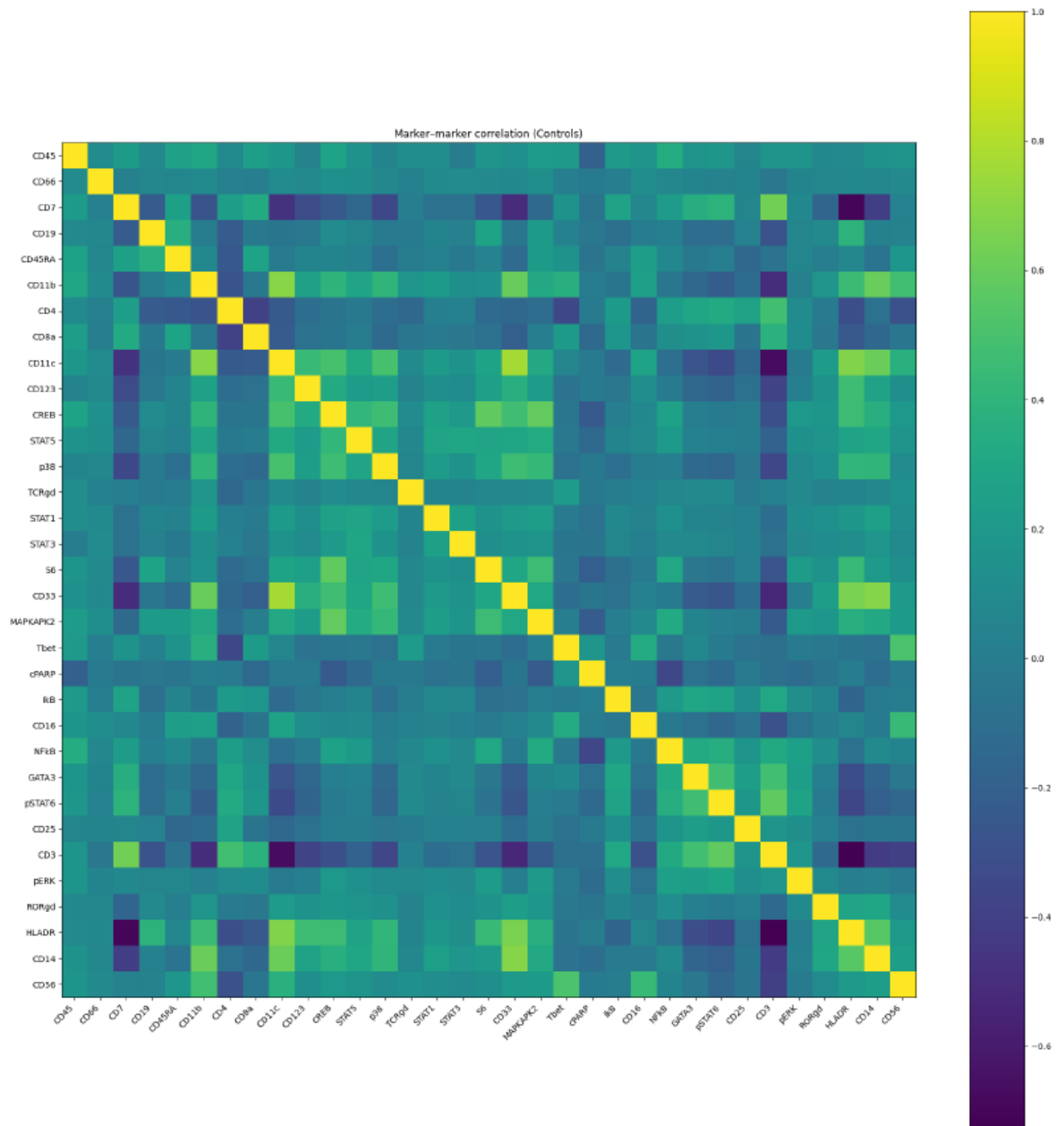


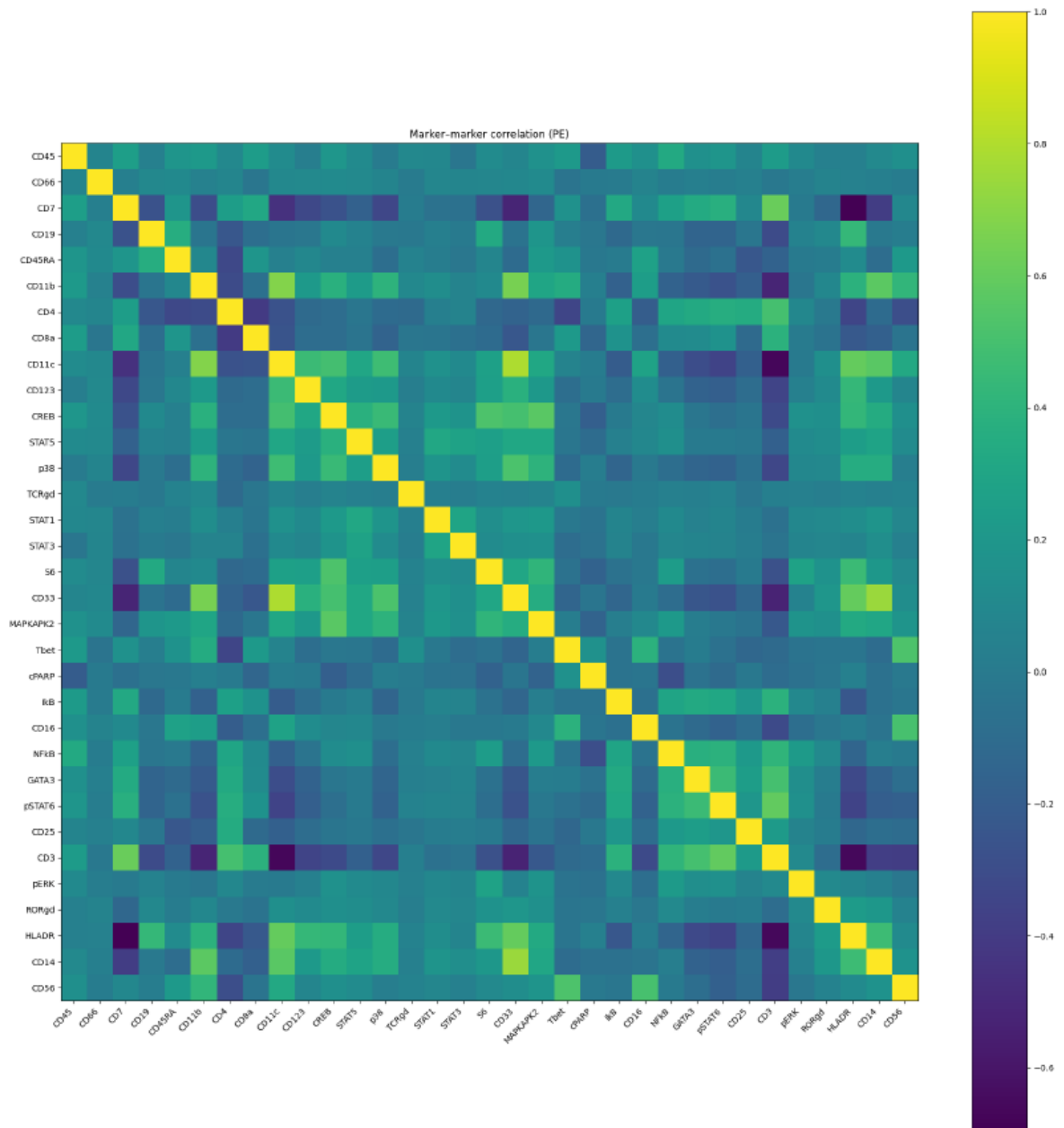












```
In [ ]: # Just save eda to your Drive (same folder as other results)
eda.to_csv("/content/drive/MyDrive/pre eclampsia/eda.csv", index=False)

print("Saved eda → /content/drive/MyDrive/pre eclampsia/eda.csv")
```

Saved eda → /content/drive/MyDrive/pre eclampsia/eda.csv

UMAP VIEW OF MARKER INTENSITY DISTRIBUTION

```
In [ ]: use_markers = [c for c in eda.columns if c not in ("condition", "age", "age_label")]
reducer = umap.UMAP(n_neighbors=30, min_dist=0.3, random_state=42)
emb = reducer.fit_transform(eda[use_markers].values.astype("float32"))

eda["_umap1"], eda["_umap2"] = emb[:,0], emb[:,1]
```

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
warn(
```

```
In [ ]: # Defining function to plot marker intensity view in UMAP view
def umap_by_marker_split_condition(df, marker, vmin=None, vmax=None):
    if marker not in df.columns:
```

```

    print(f"{marker} not found.")
    return

# Robust color range from pooled data for fair comparison (1%-99% quantiles)
vals = df[marker].dropna().values
if vmin is None or vmax is None:
    q1, q99 = np.percentile(vals, [1, 99])
    vmin, vmax = float(q1), float(q99)

for cond, title in [(0, "Control"), (1, "PE")]:
    mask = df["condition"] == cond
    if mask.sum() == 0:
        print(f"No cells for condition={cond}")
        continue

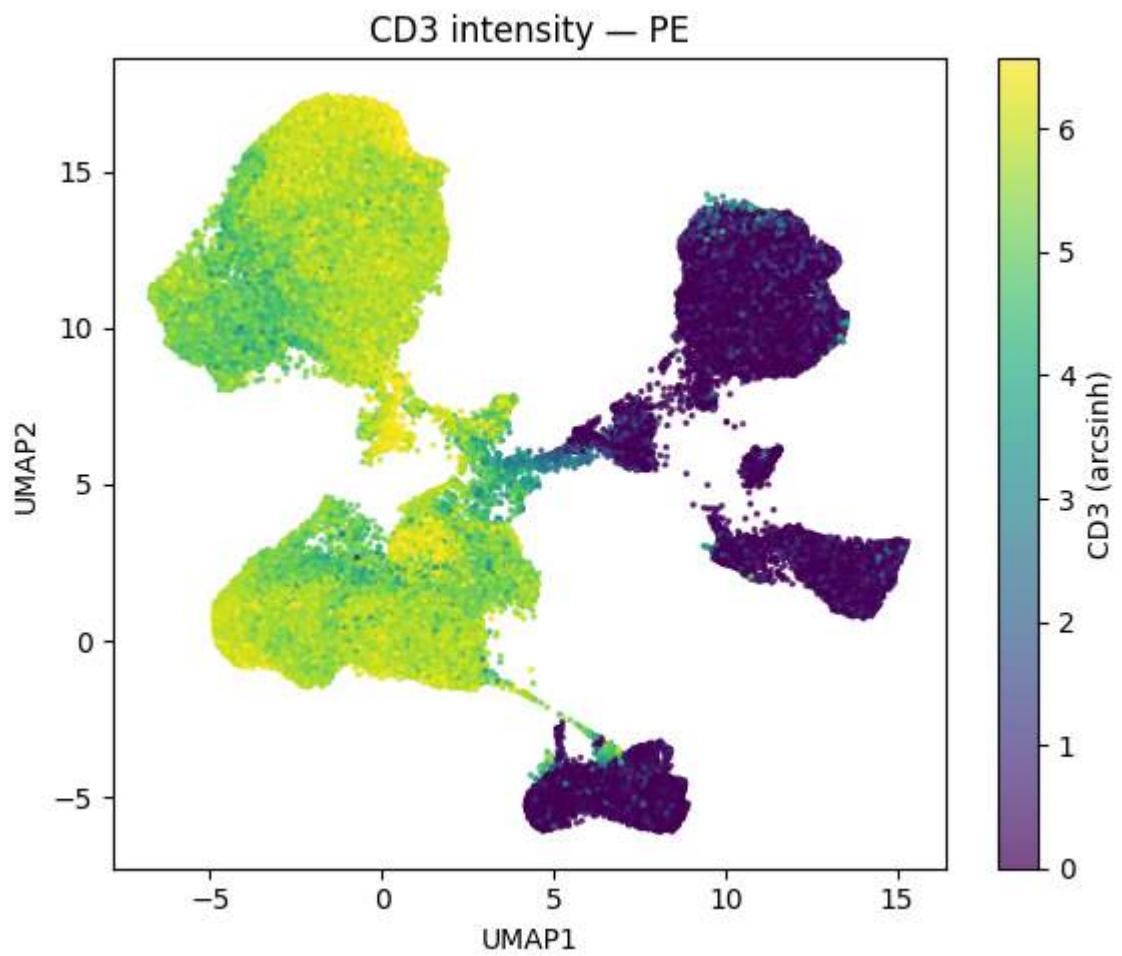
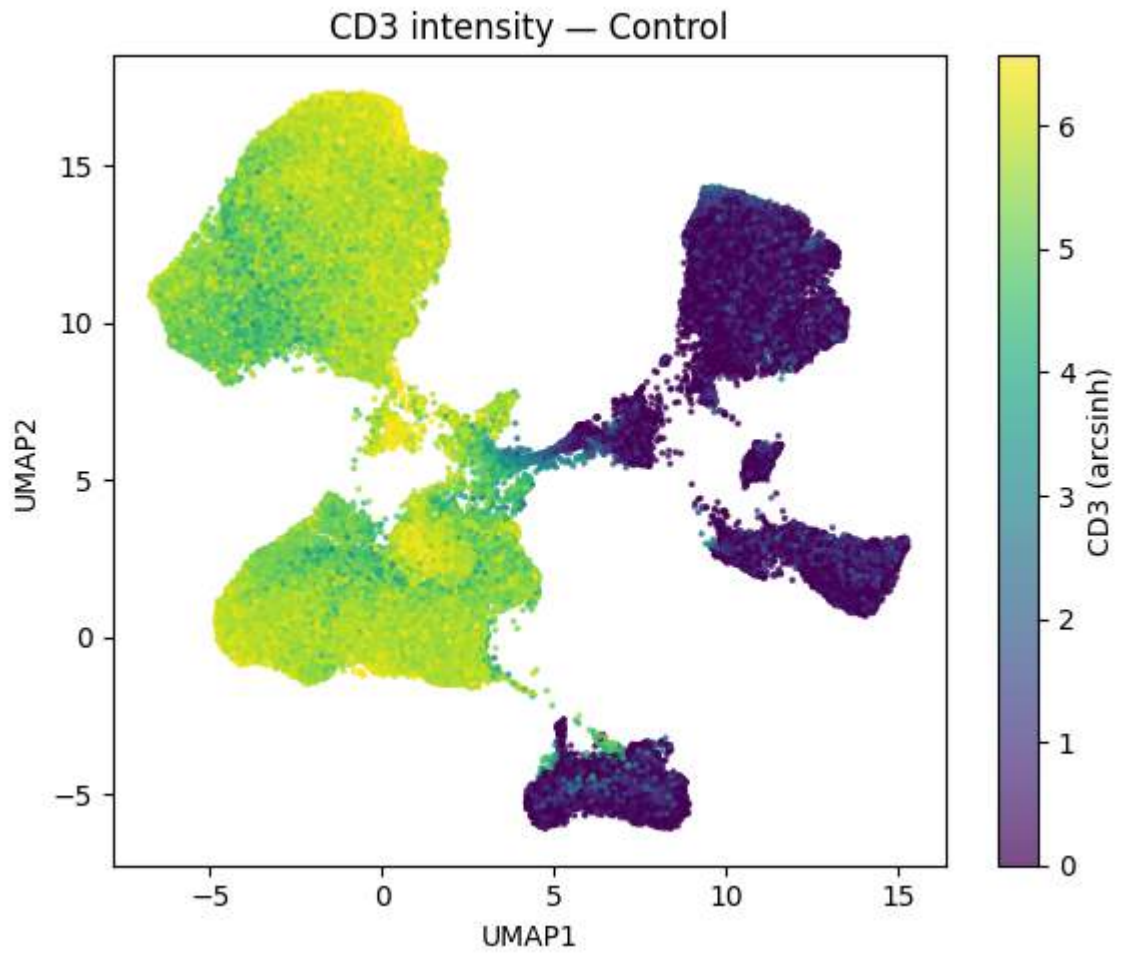
    plt.figure(figsize=(6,5))
    sc = plt.scatter(df.loc[mask, "_umap1"], df.loc[mask, "_umap2"],
                    s=2, c=df.loc[mask, marker], vmin=vmin, vmax=vmax, alpha=0.5)
    plt.xlabel("UMAP1"); plt.ylabel("UMAP2")
    plt.title(f"{marker} intensity - {title}")
    plt.colorbar(sc, label=f"{marker} (arcsinh)")
    plt.tight_layout(); plt.show()

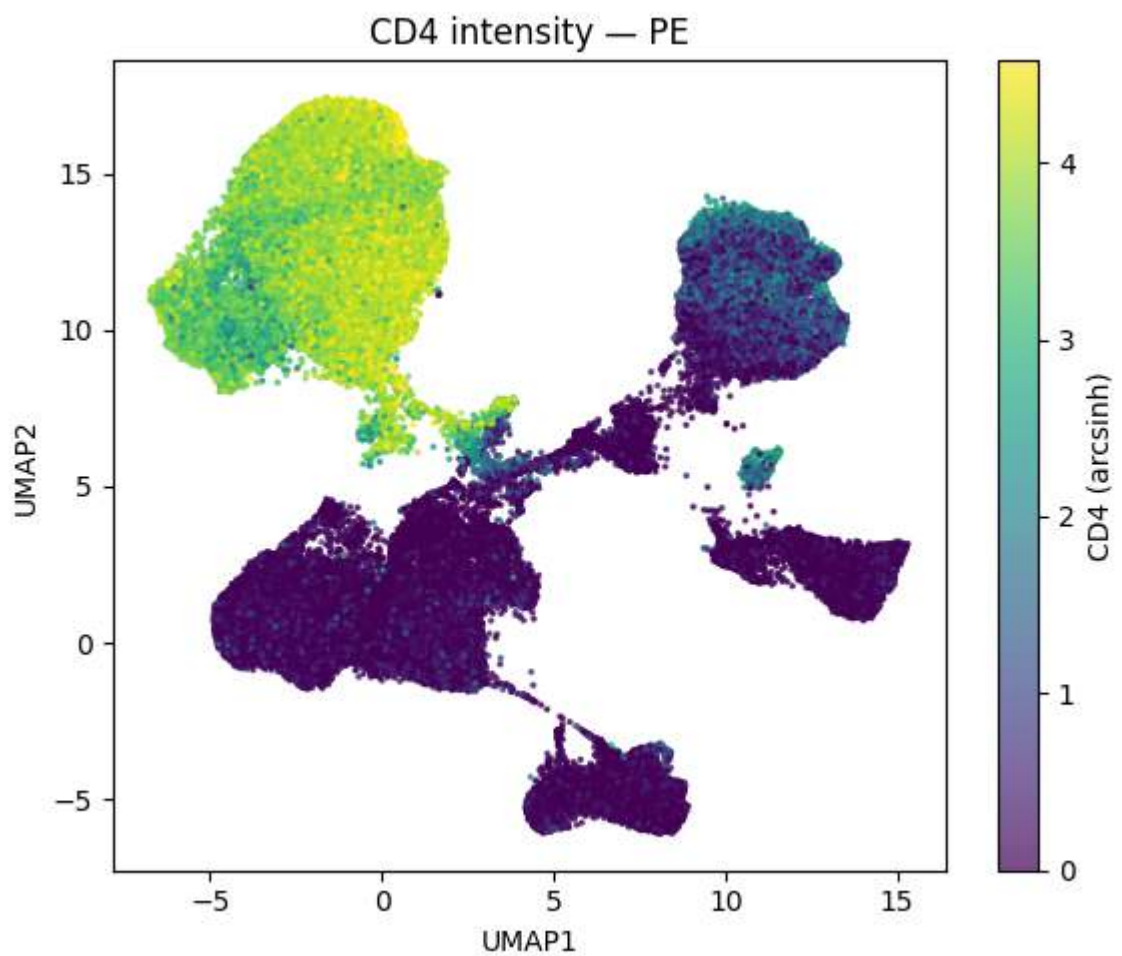
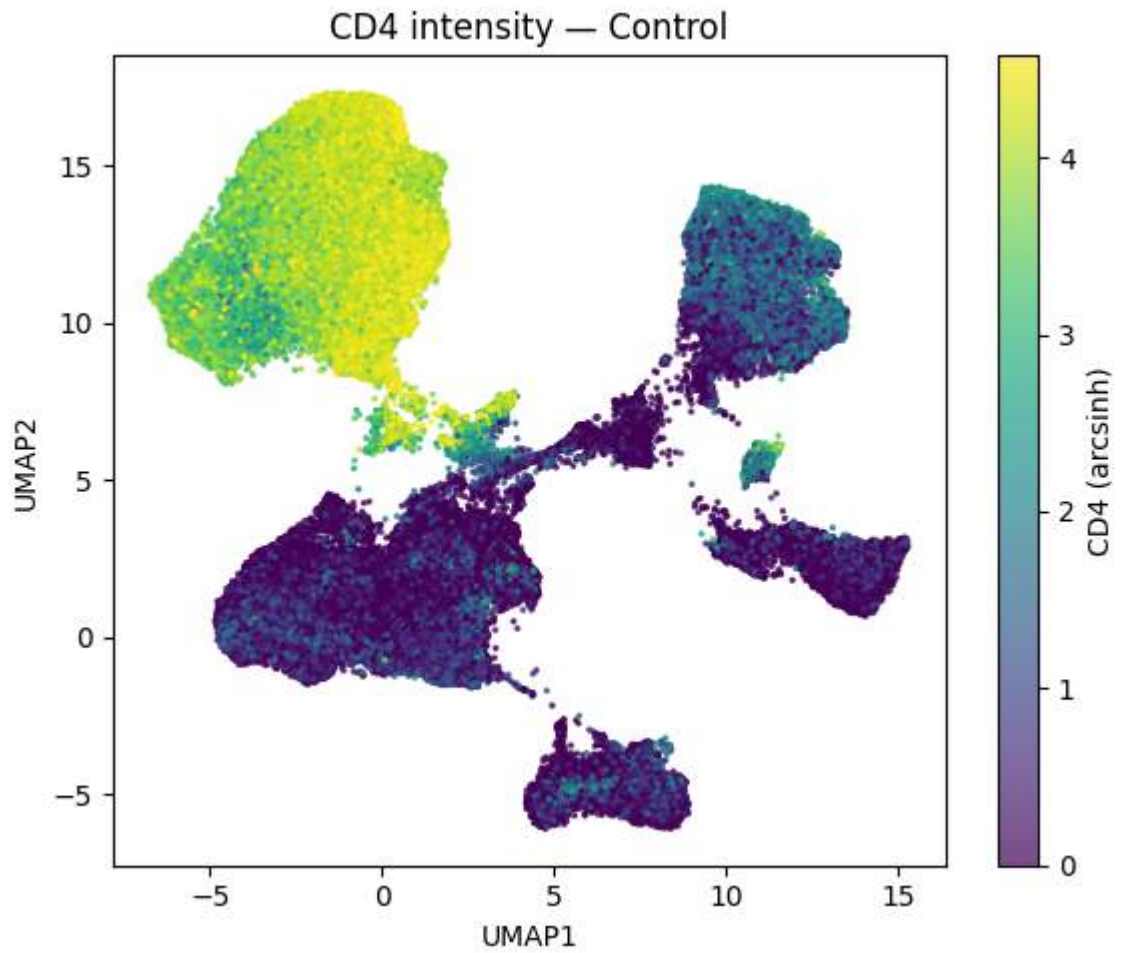
# Try a few markers you care about
markers_of_interest = [m for m in ["CD3", "CD4", "CD7", "CD8a", "CD14", "MABKAP2", "CD19", "CD56", "CD11c", "S6", "CD11b"]]
print("Markers to plot:", markers_of_interest)

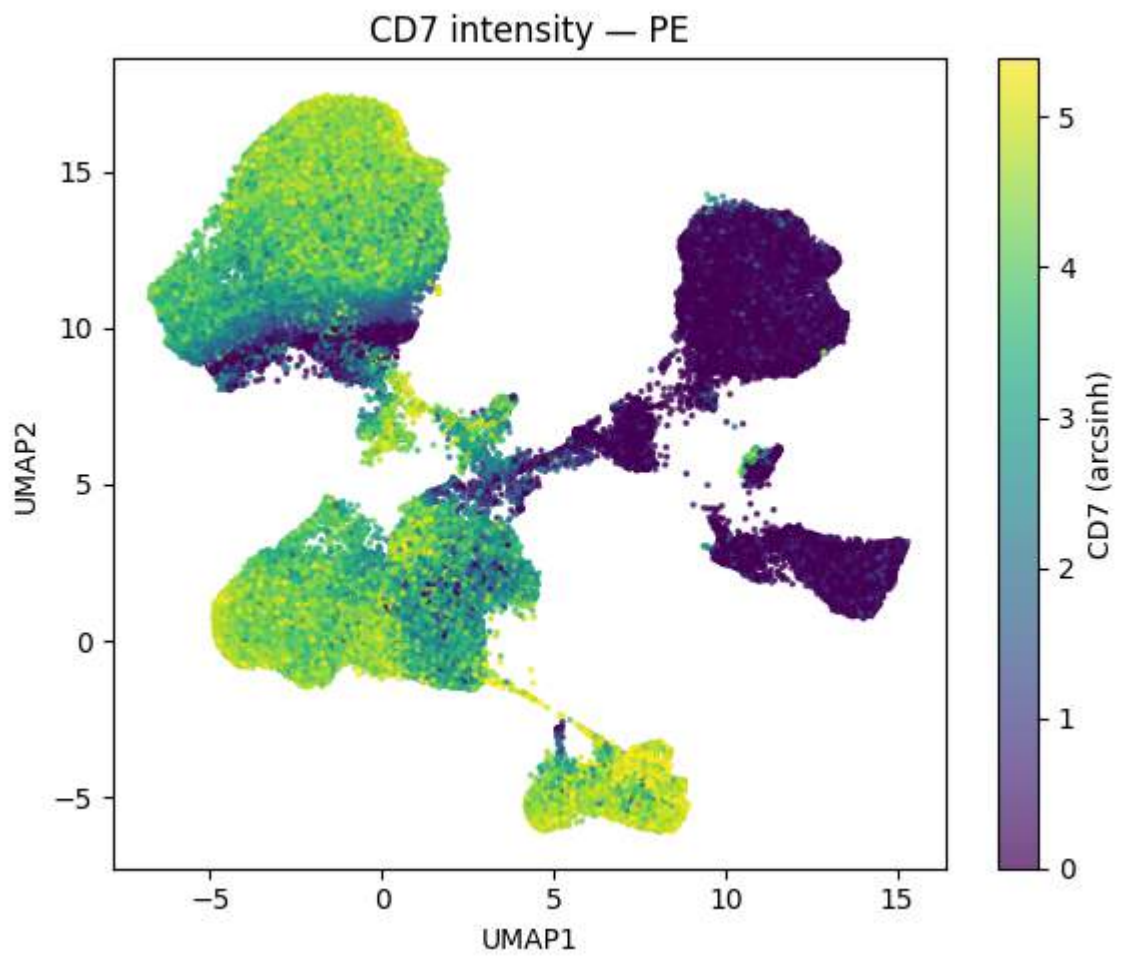
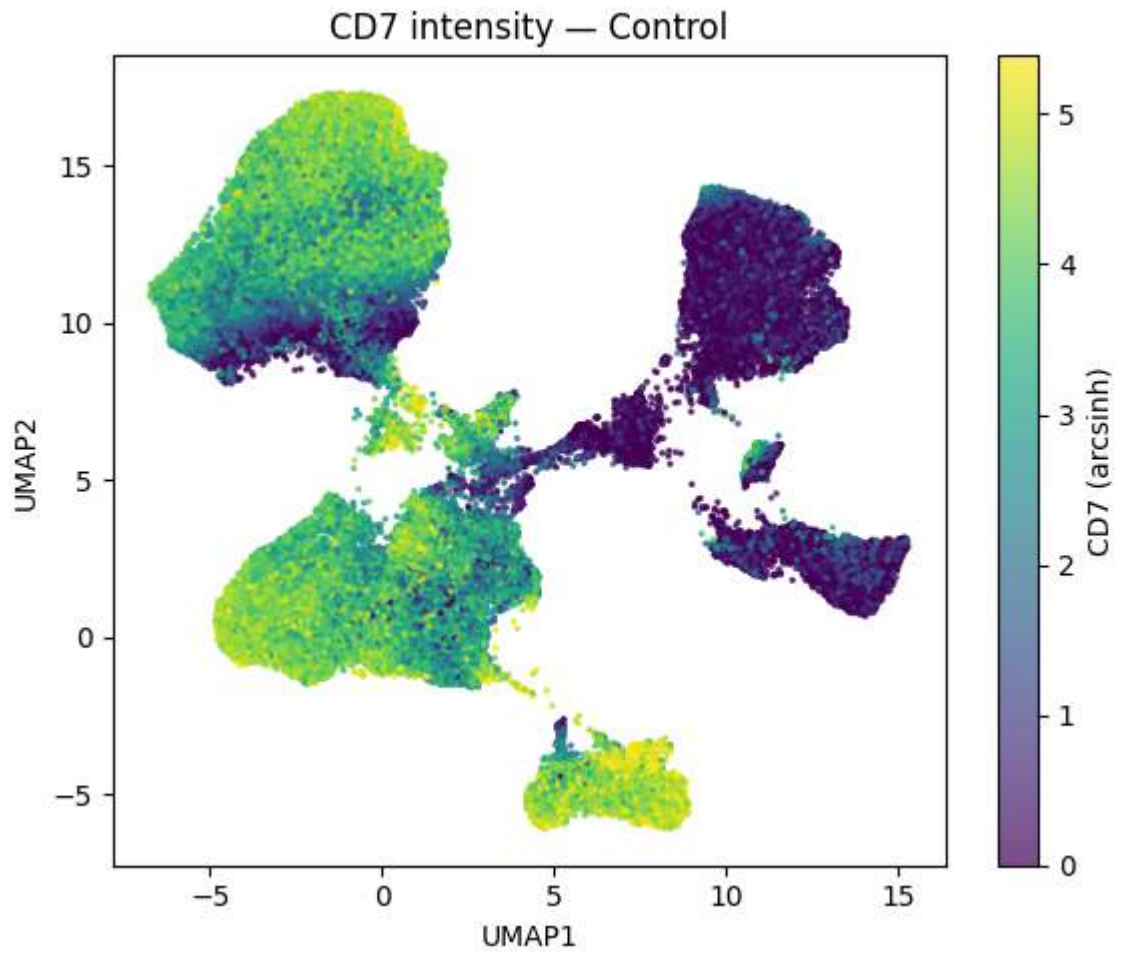
for m in markers_of_interest:
    umap_by_marker_split_condition(eda, m)

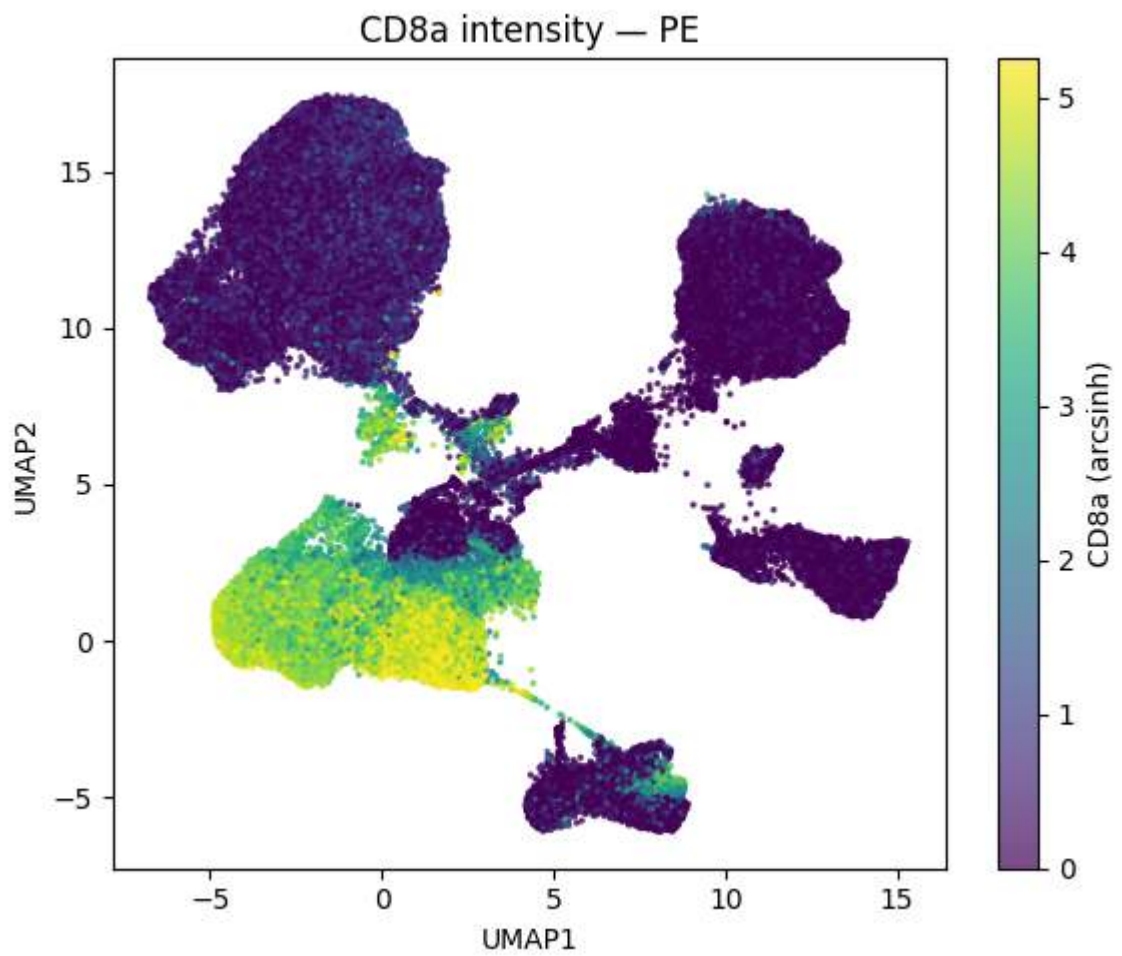
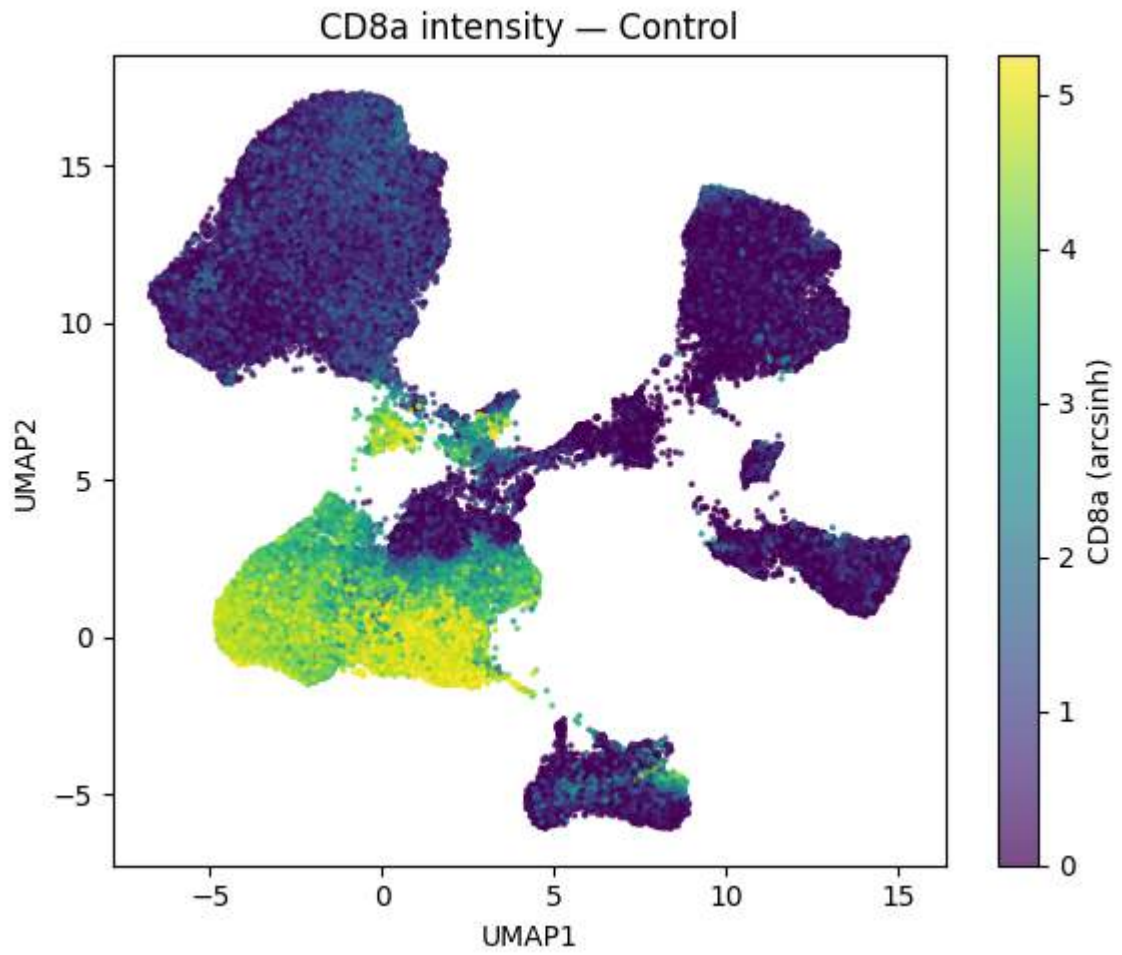
```

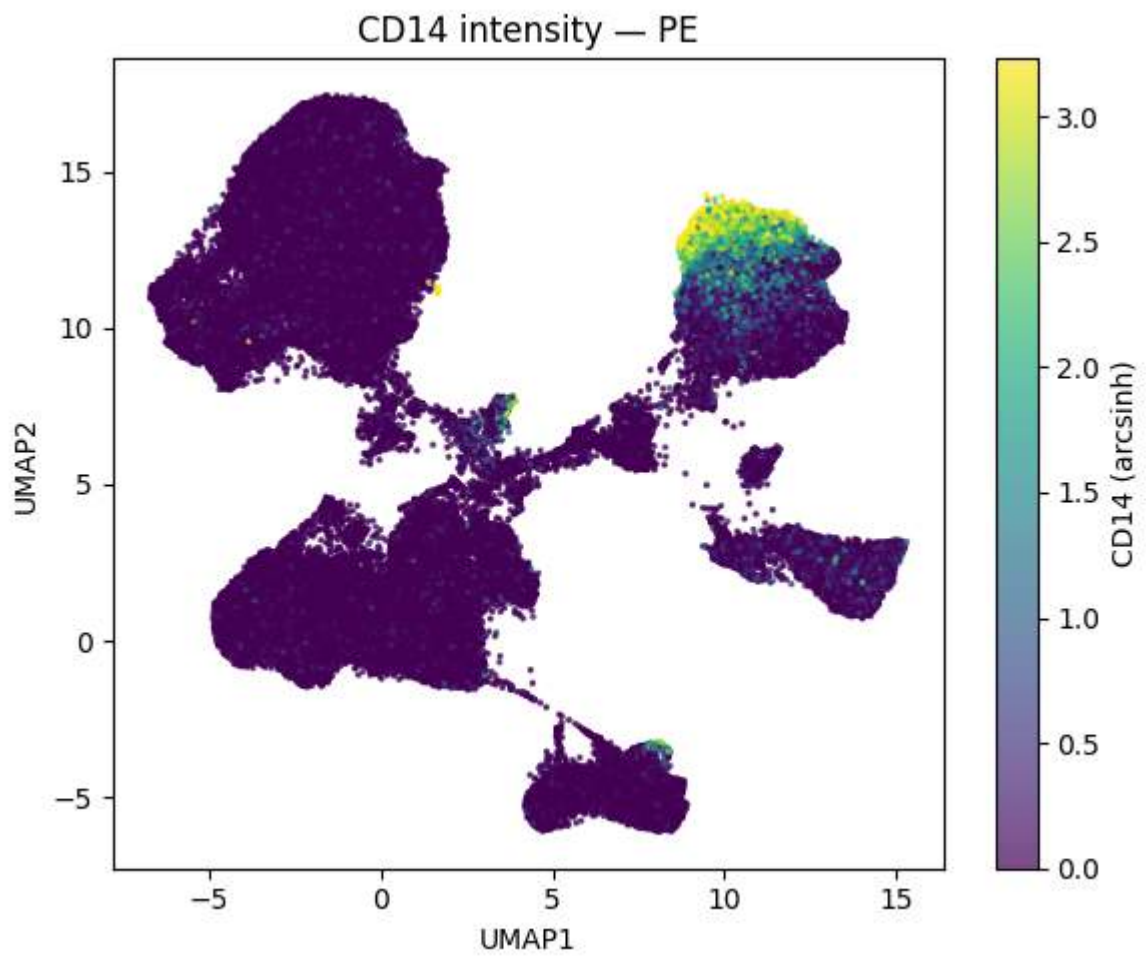
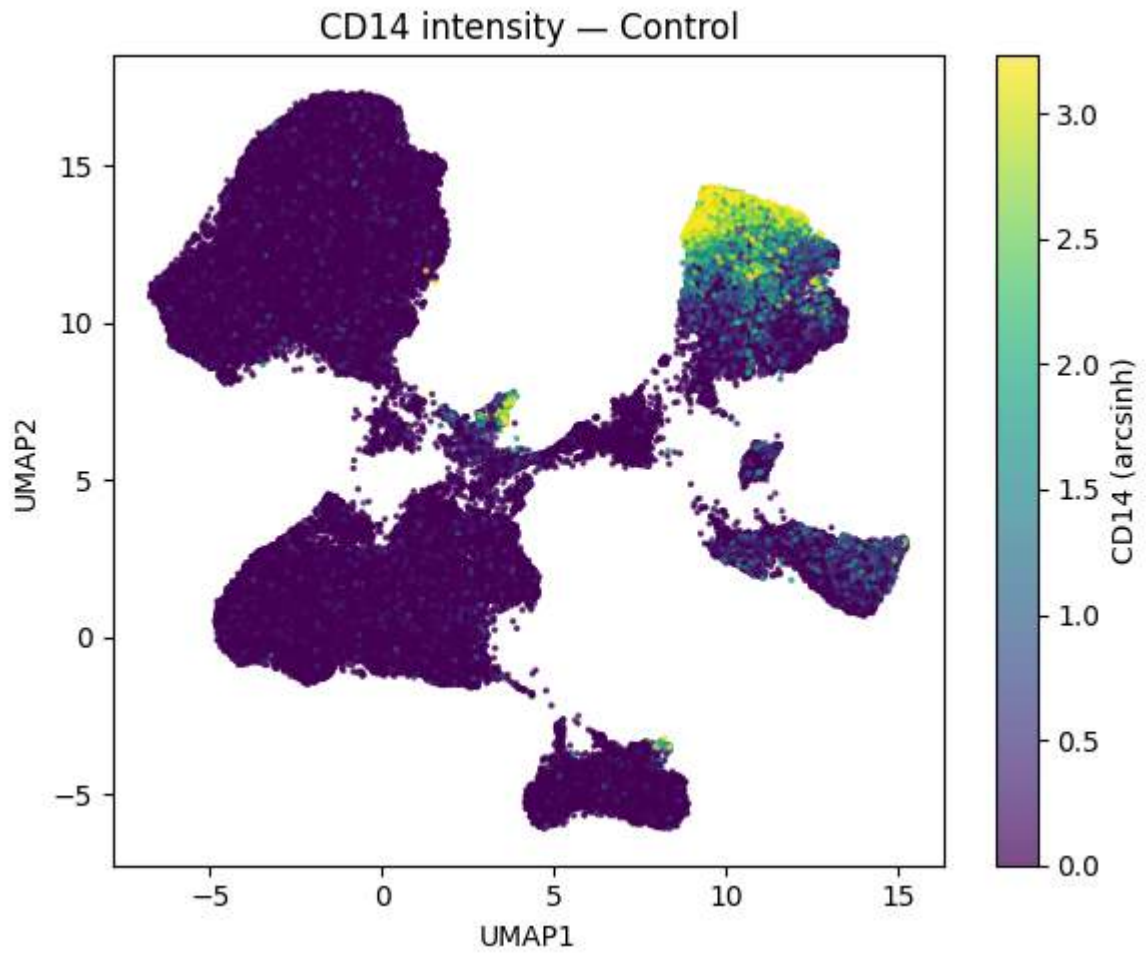
Markers to plot: ['CD3', 'CD4', 'CD7', 'CD8a', 'CD14', 'CD45', 'CD16', 'CREB', 'CD19', 'CD56', 'CD11c', 'S6', 'CD11b']

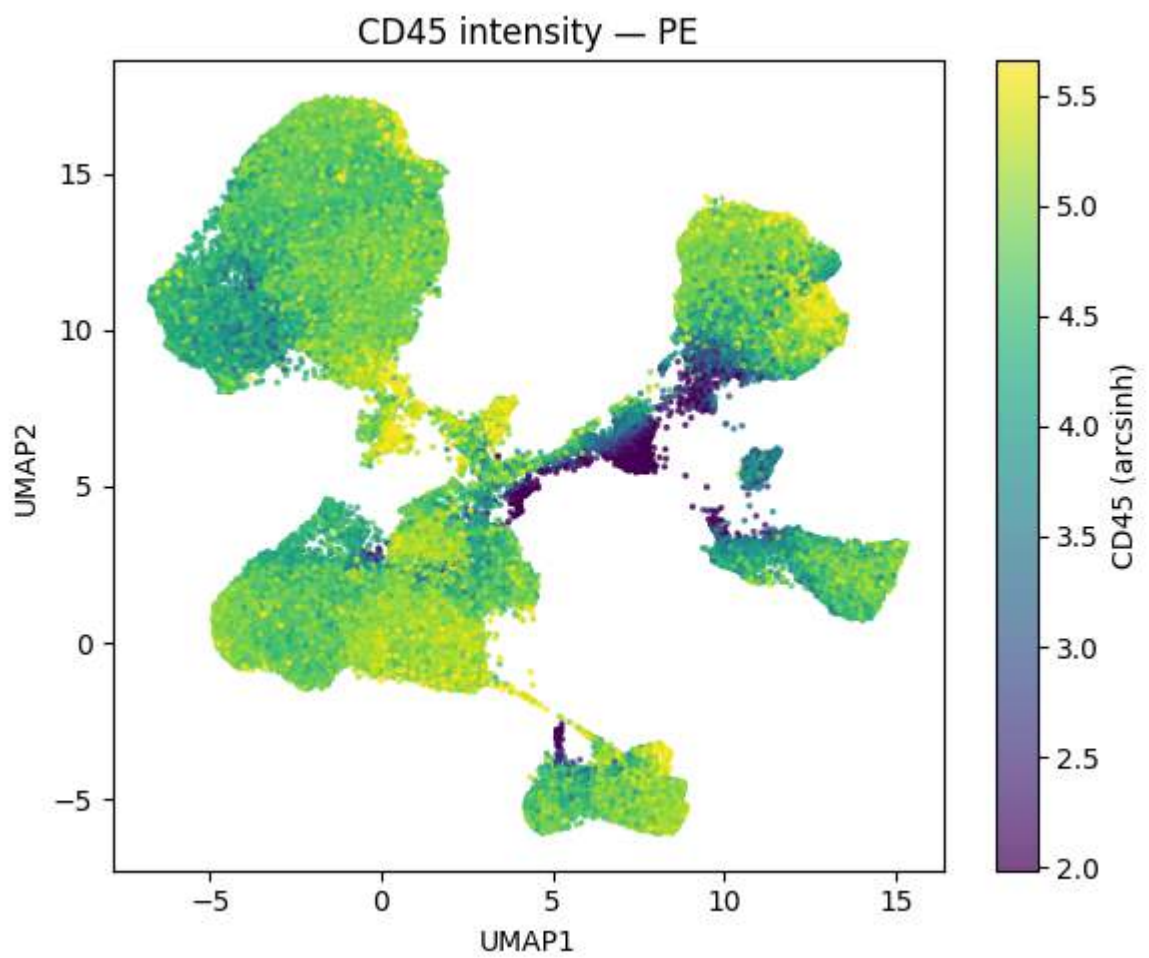
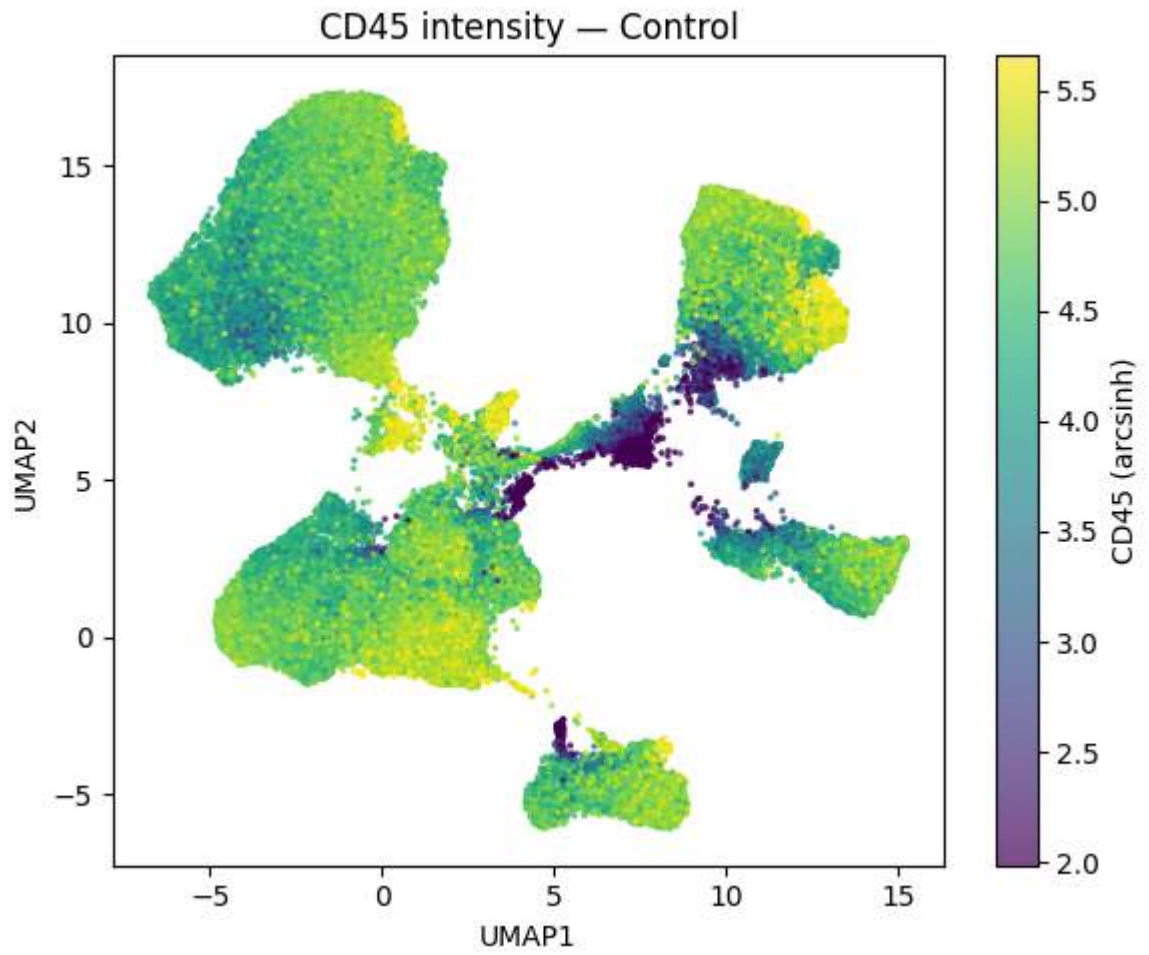


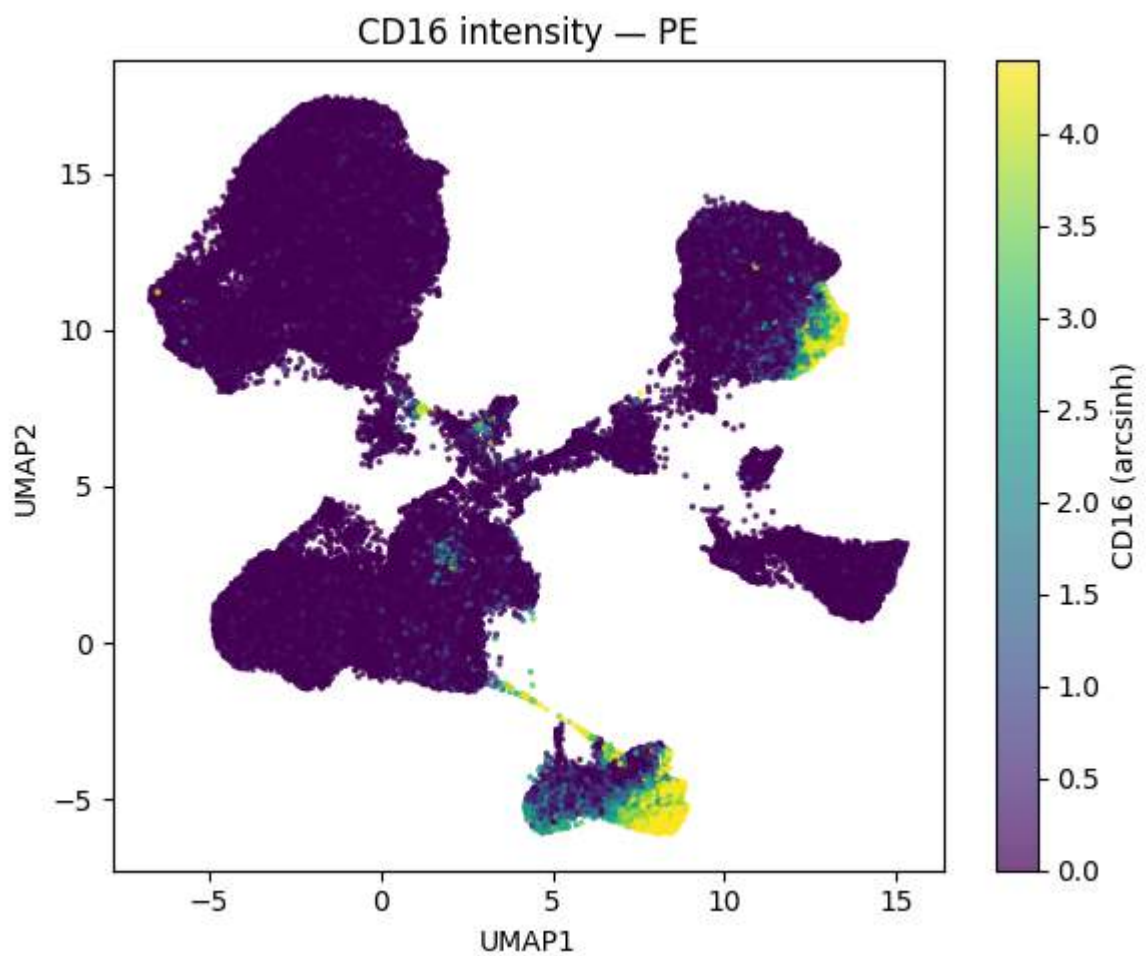
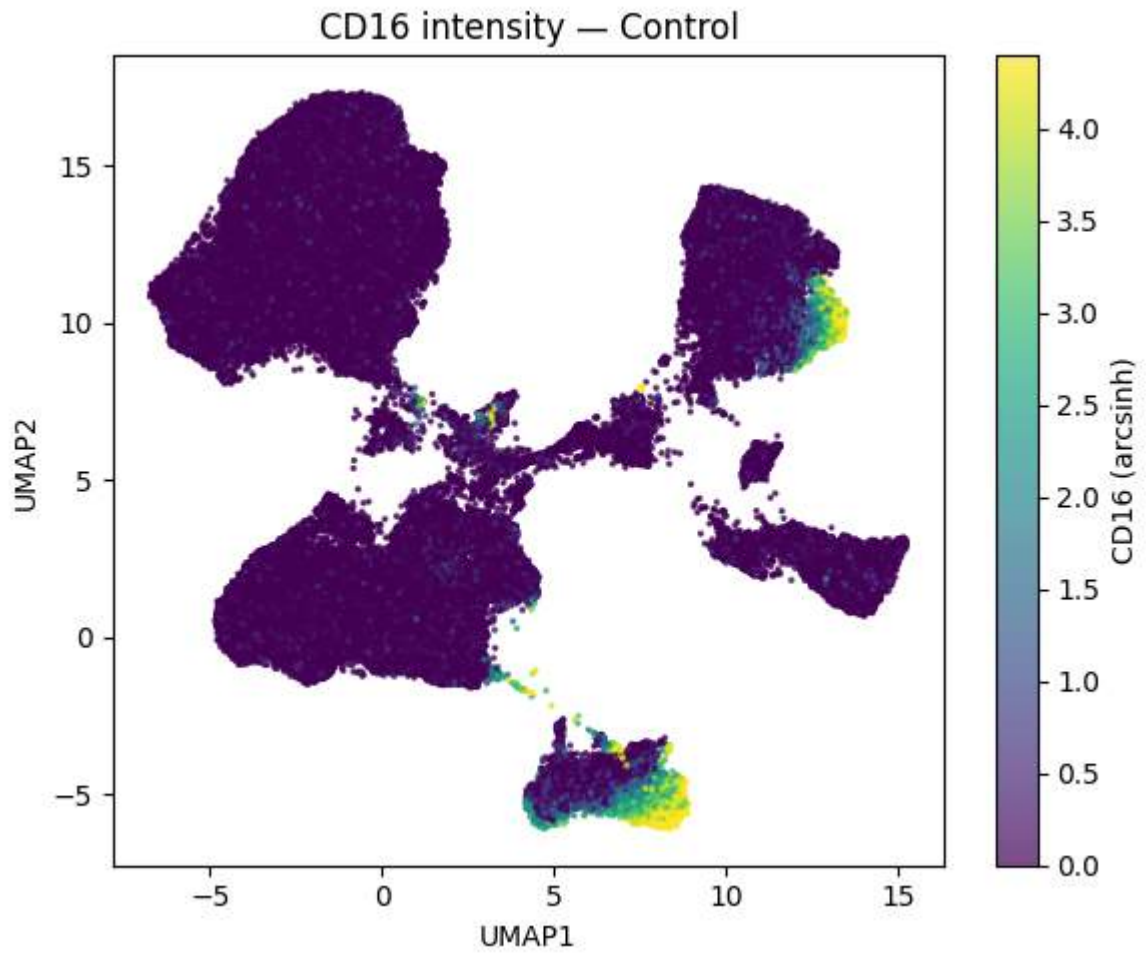


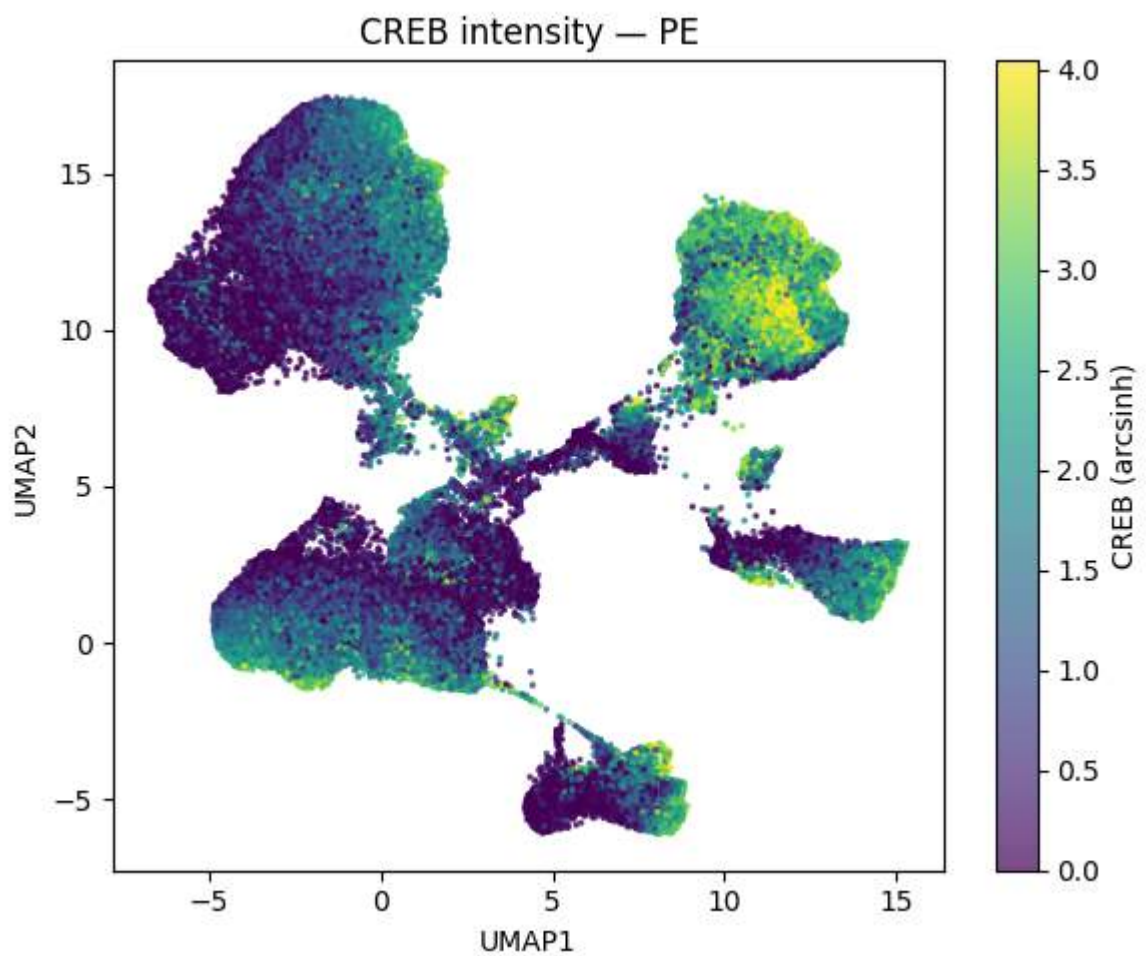
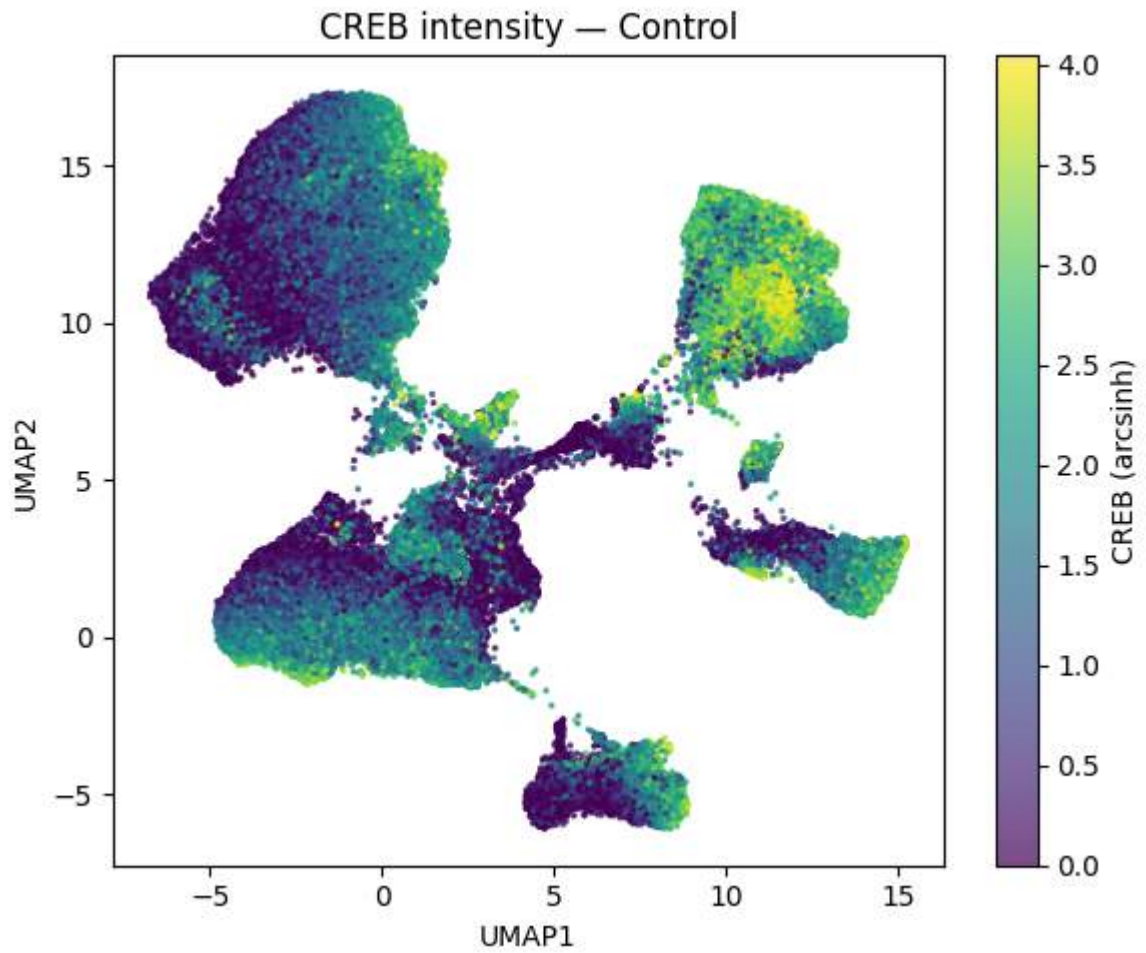


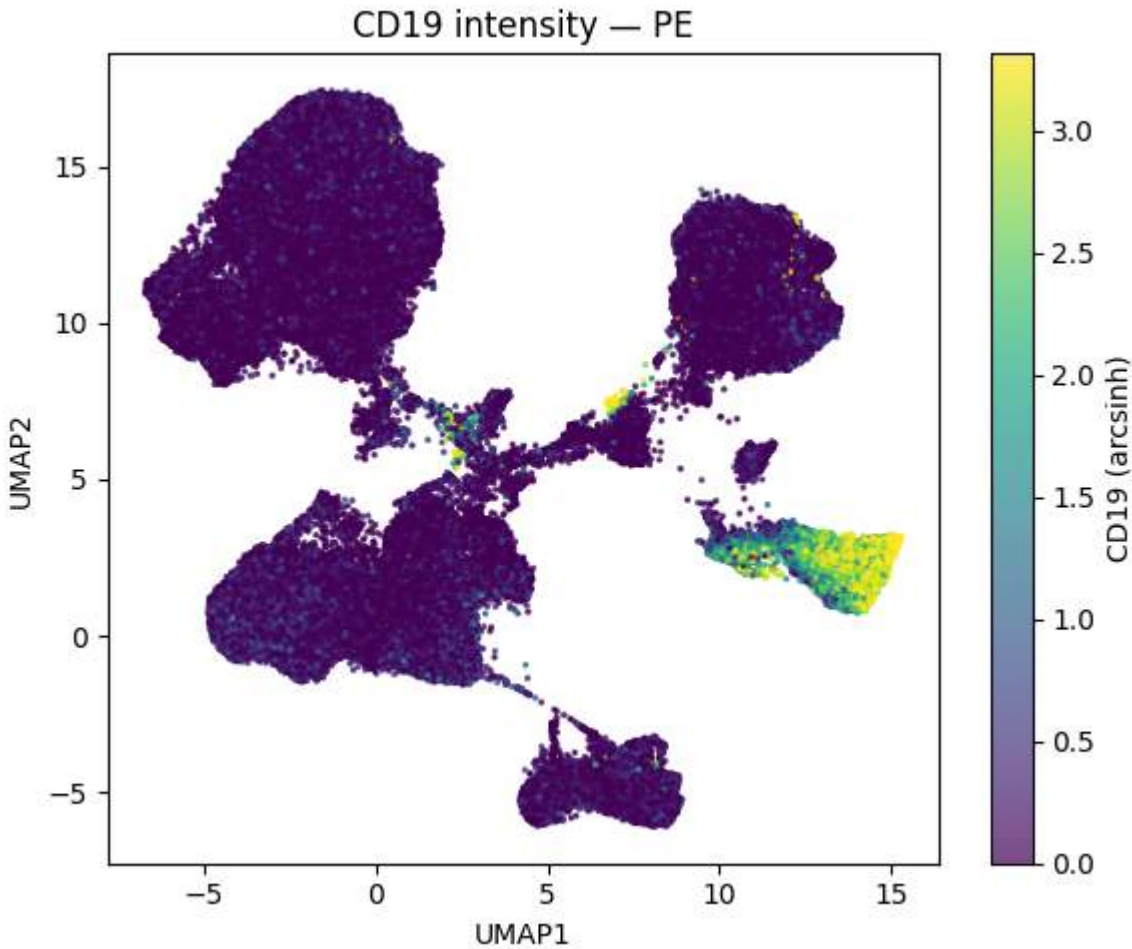
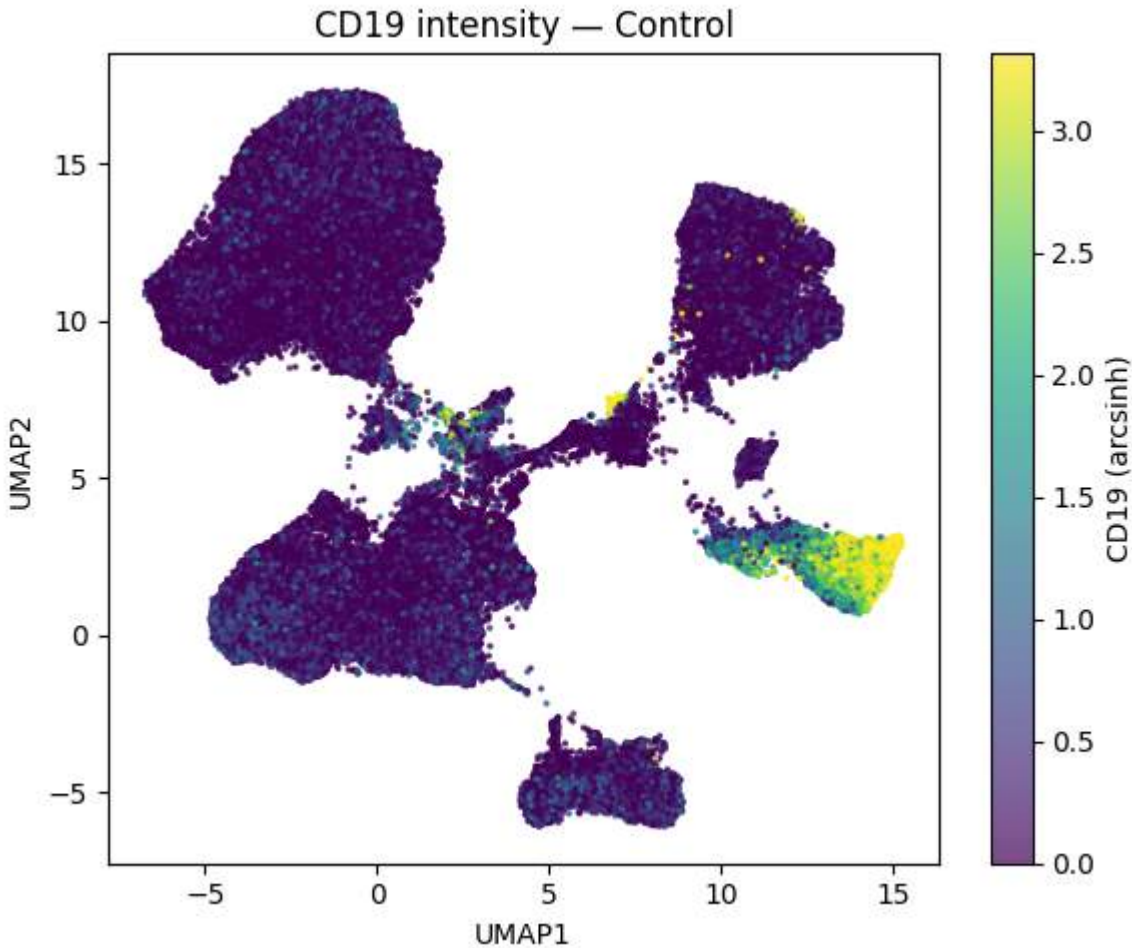


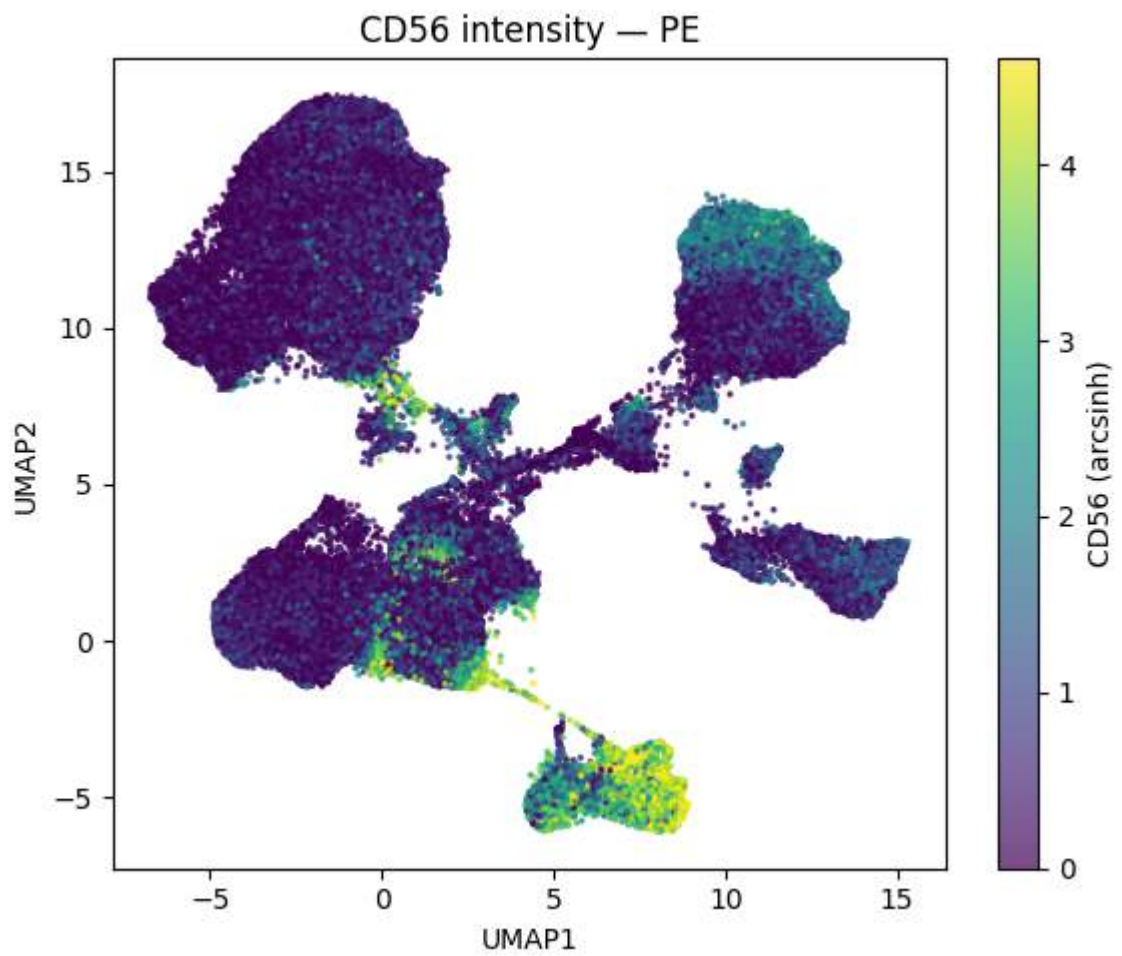
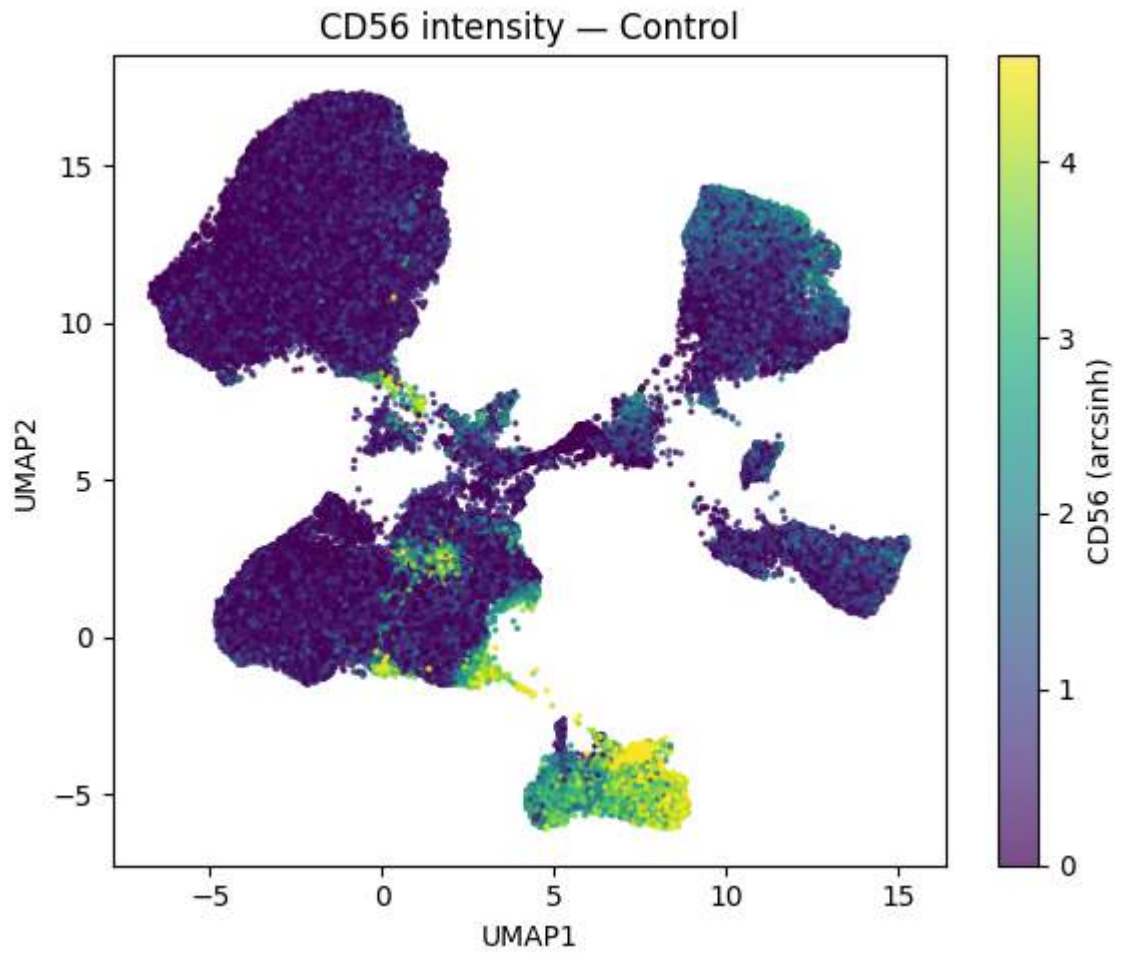


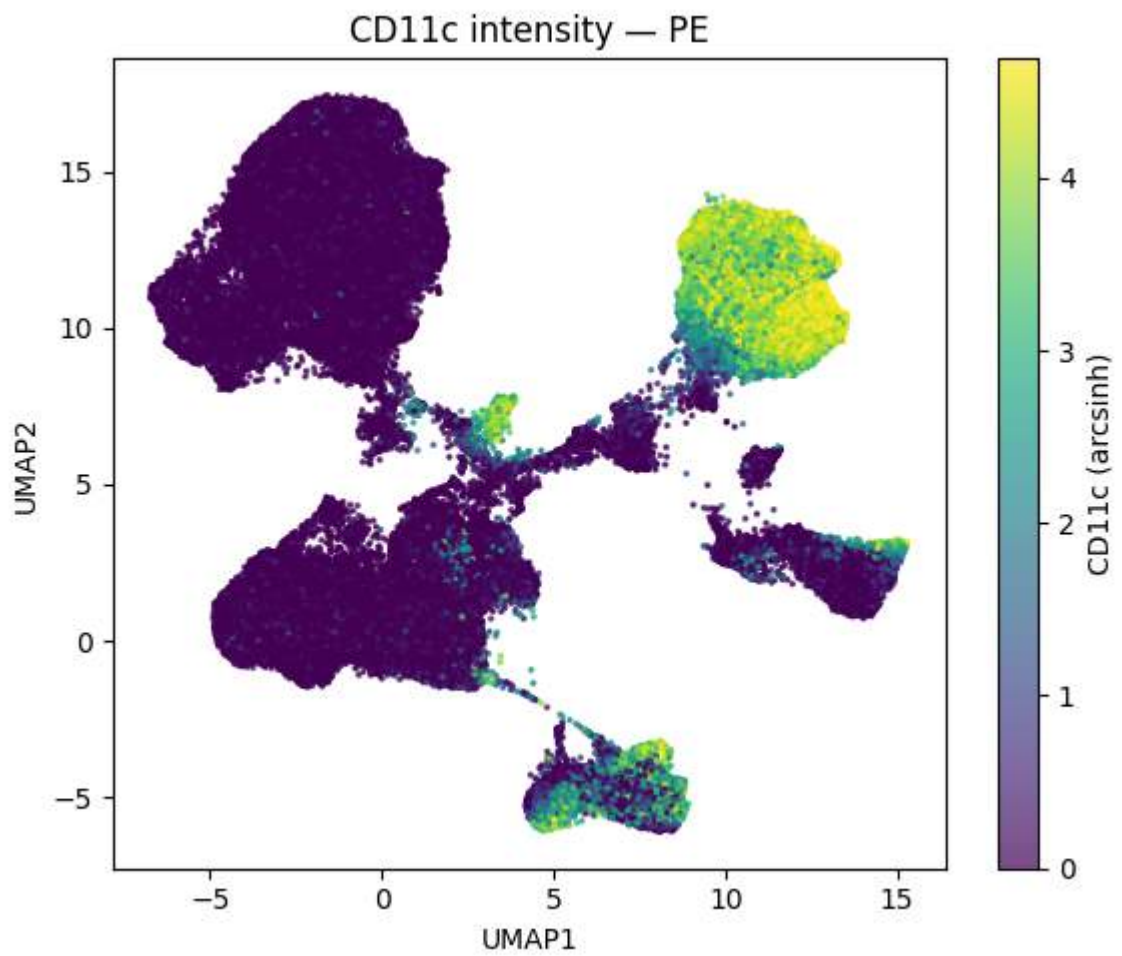
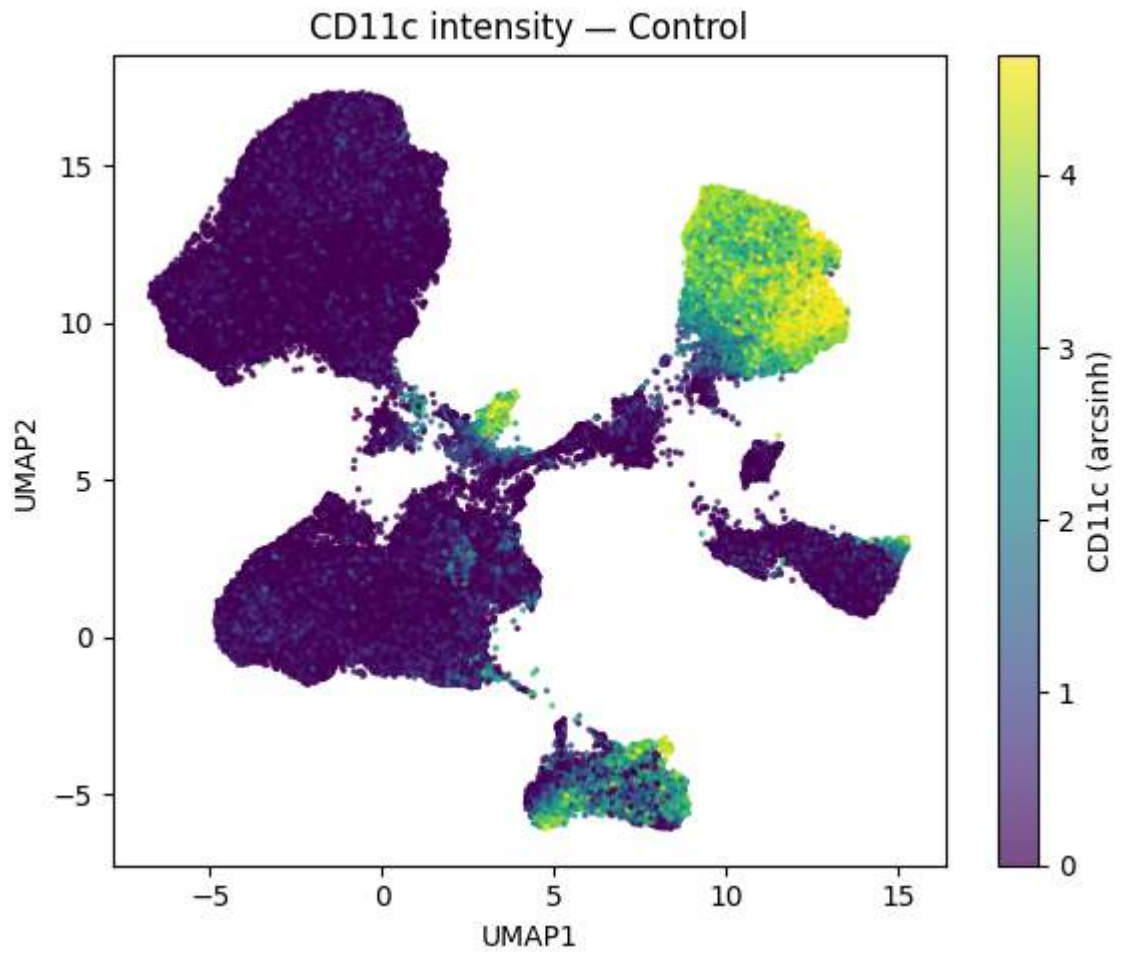


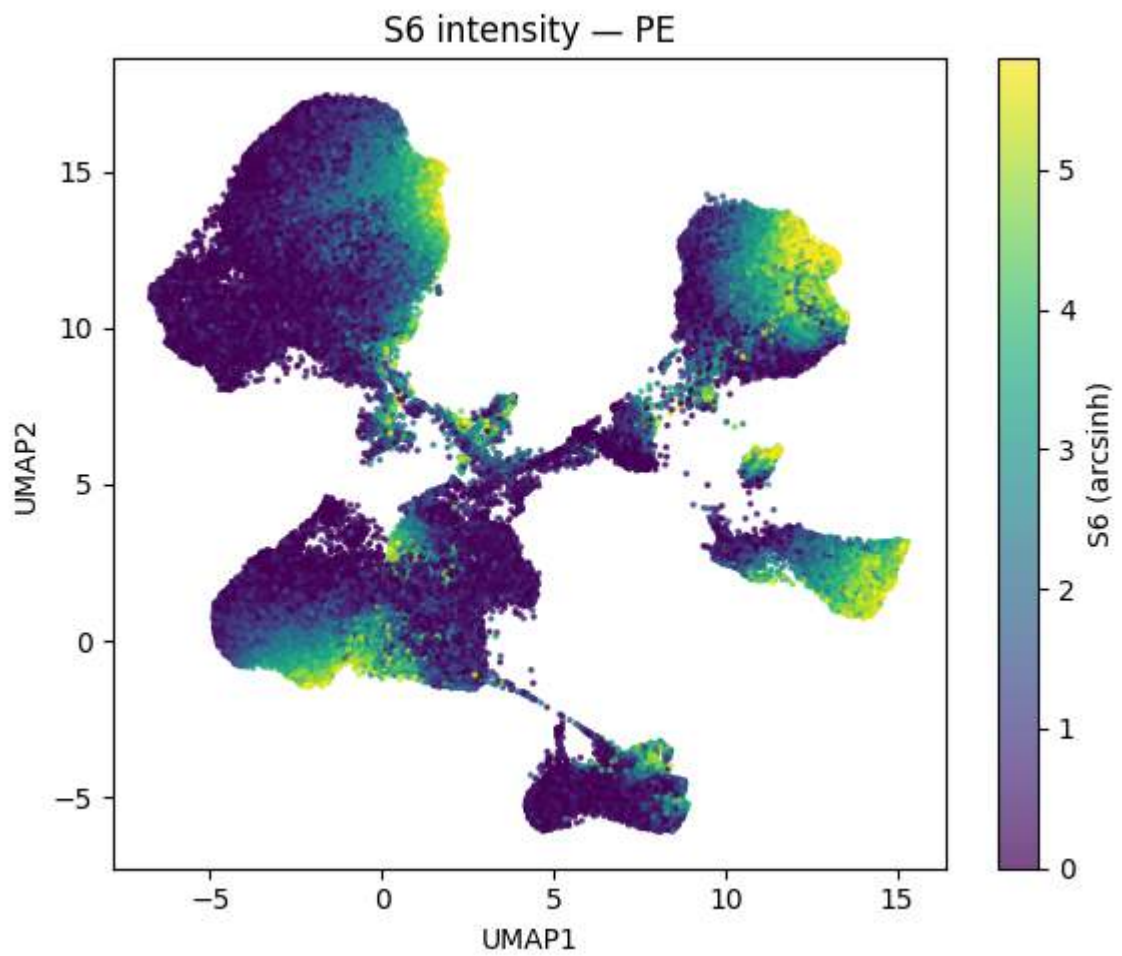
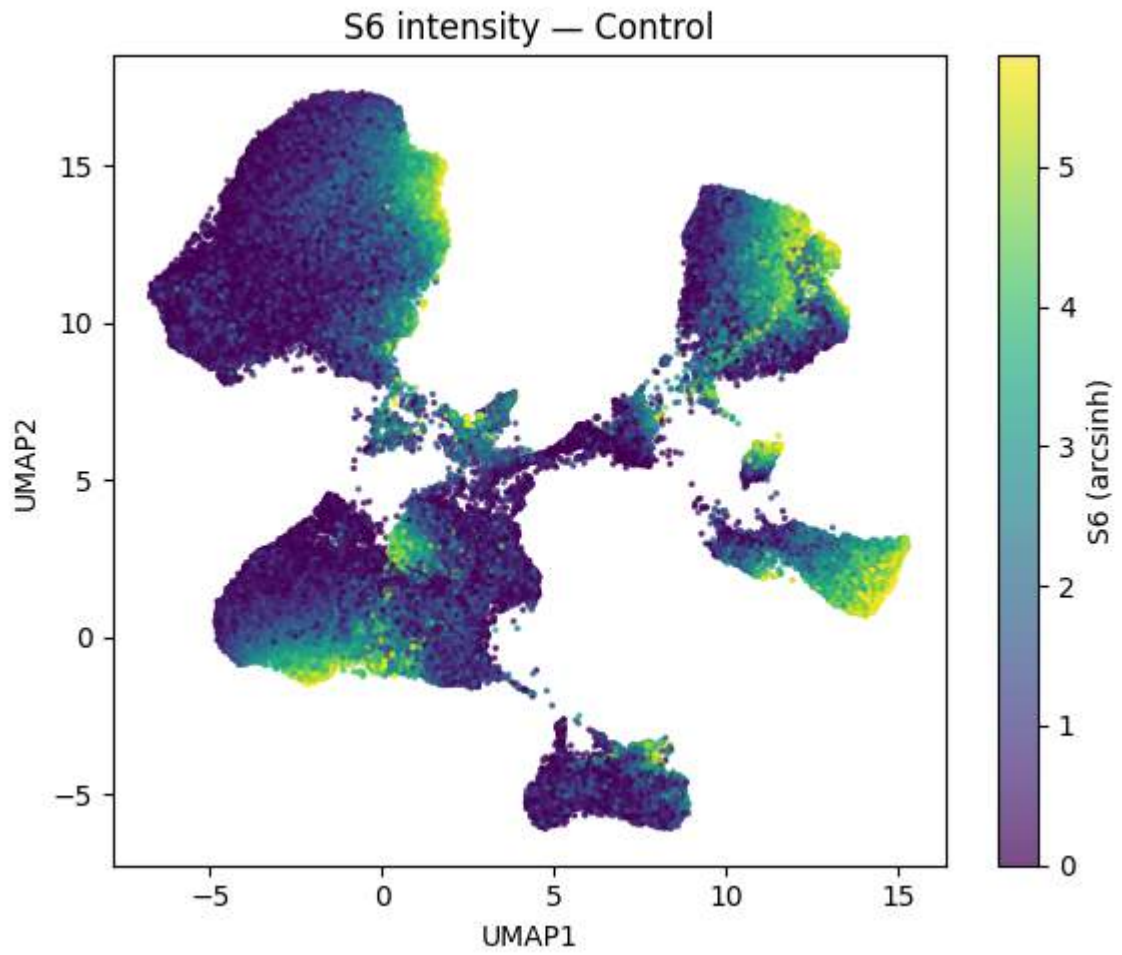


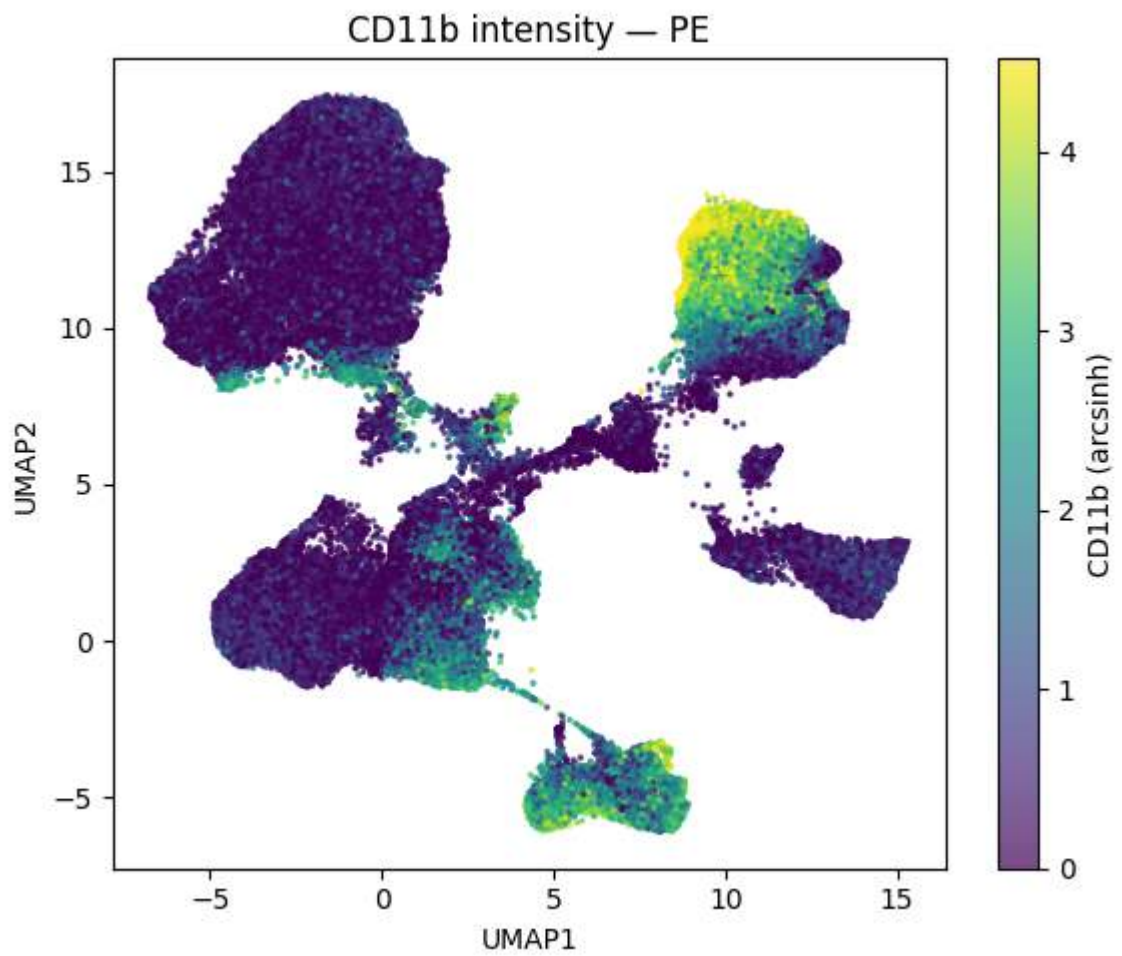
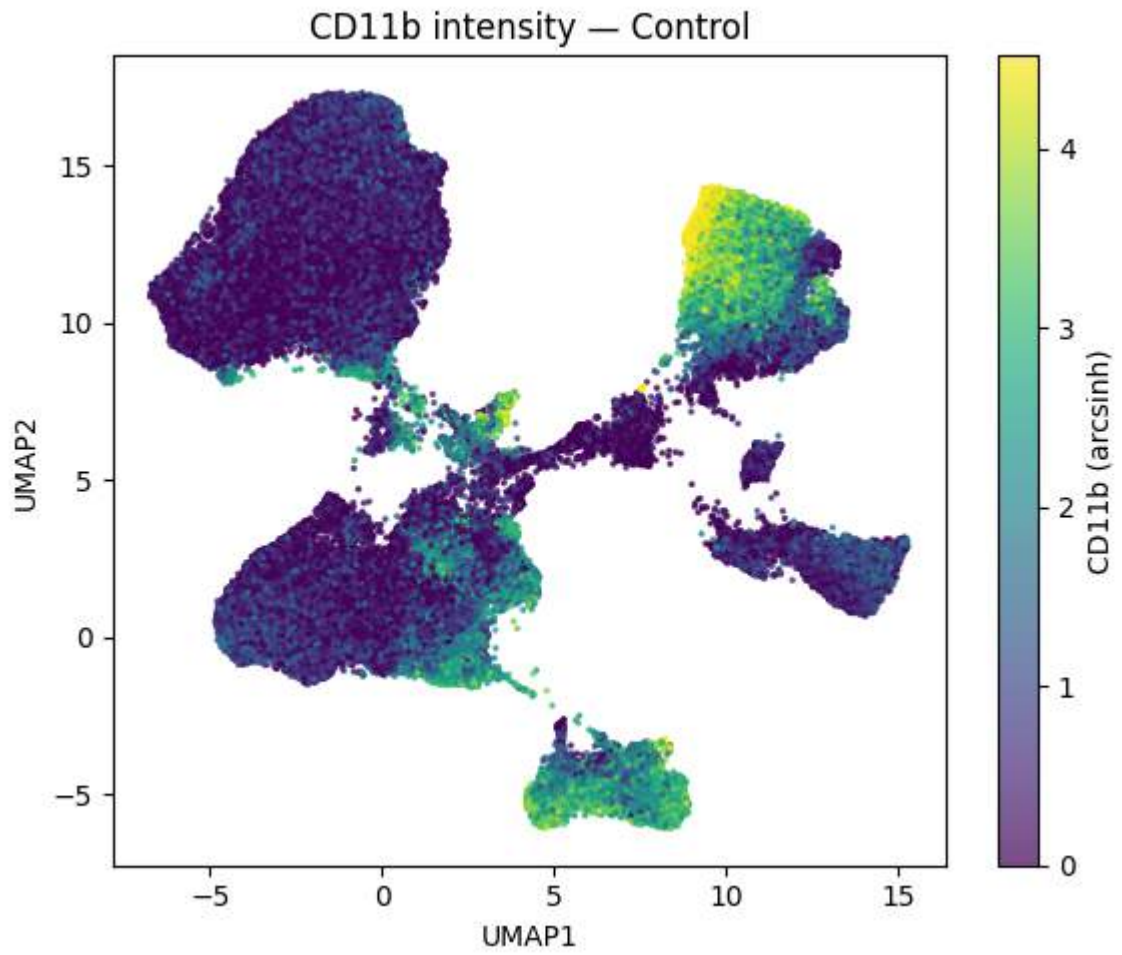












```
In [ ]: # Define bins to connote gestation: 0-10, 10-20, 20-30, up to max age
max_age = int(eda["age"].max()) + 1
bins = list(range(0, max_age + 10, 10)) # 0,10,20,30
labels = [f"{bins[i]}-{bins[i+1]-1}w" for i in range(len(bins)-1)]

eda["age_bin_10w"] = pd.cut(eda["age"], bins=bins, labels=labels, include_lowest
```

```
In [ ]: # # Plot UMAP embeddings colored by a marker's intensity, generating one compar

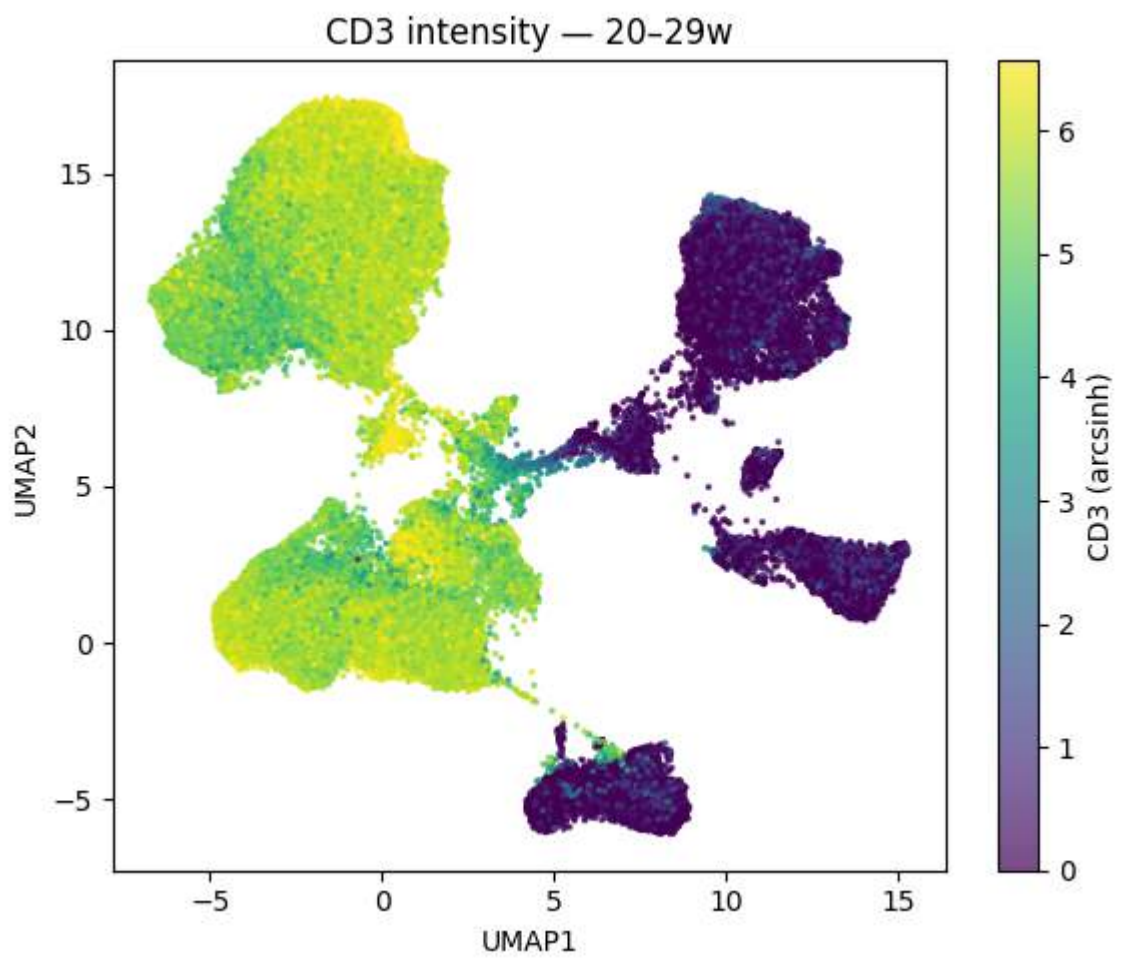
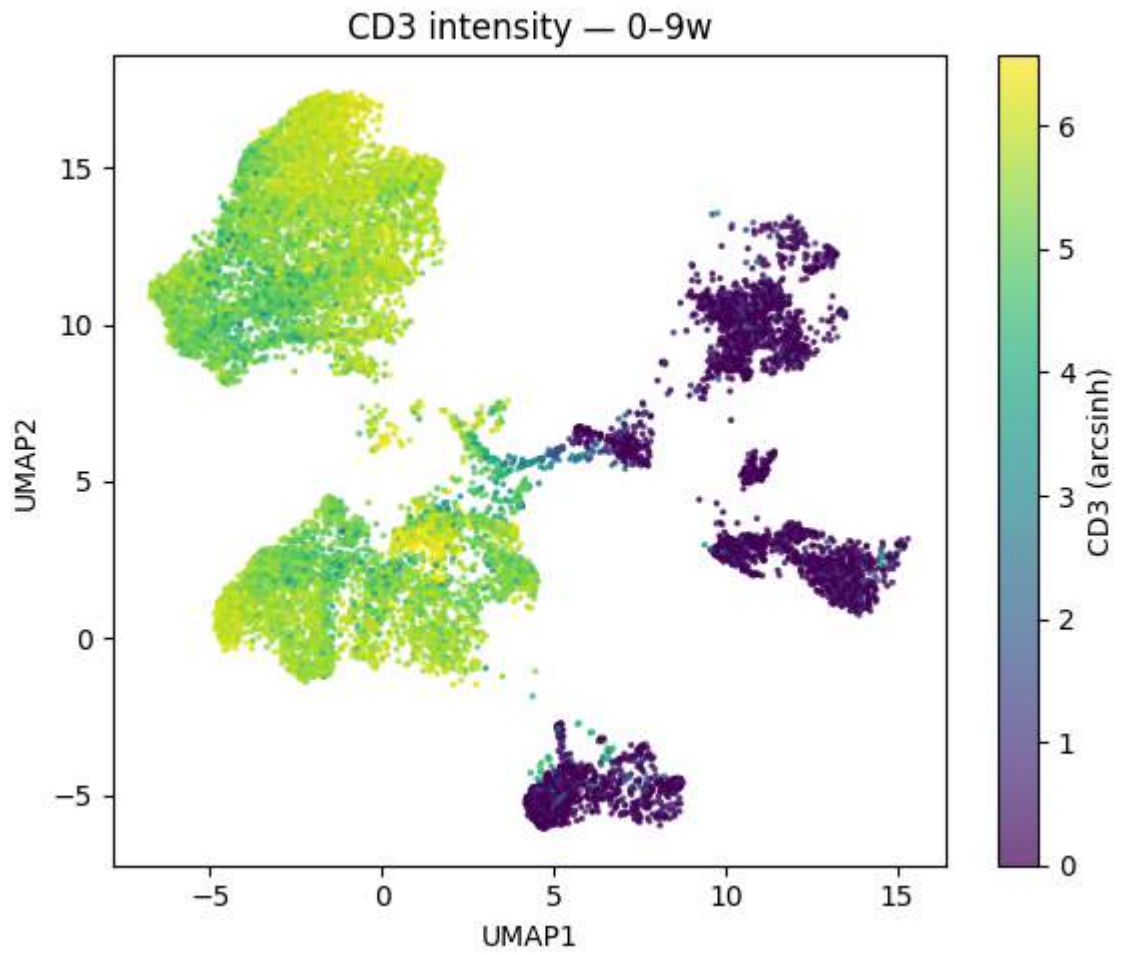
def umap_by_marker_age_bins(df, marker, age_col="age_bin_10w", vmin=None, vmax=None):
    if marker not in df.columns:
        print(f"{marker} not found.")
        return

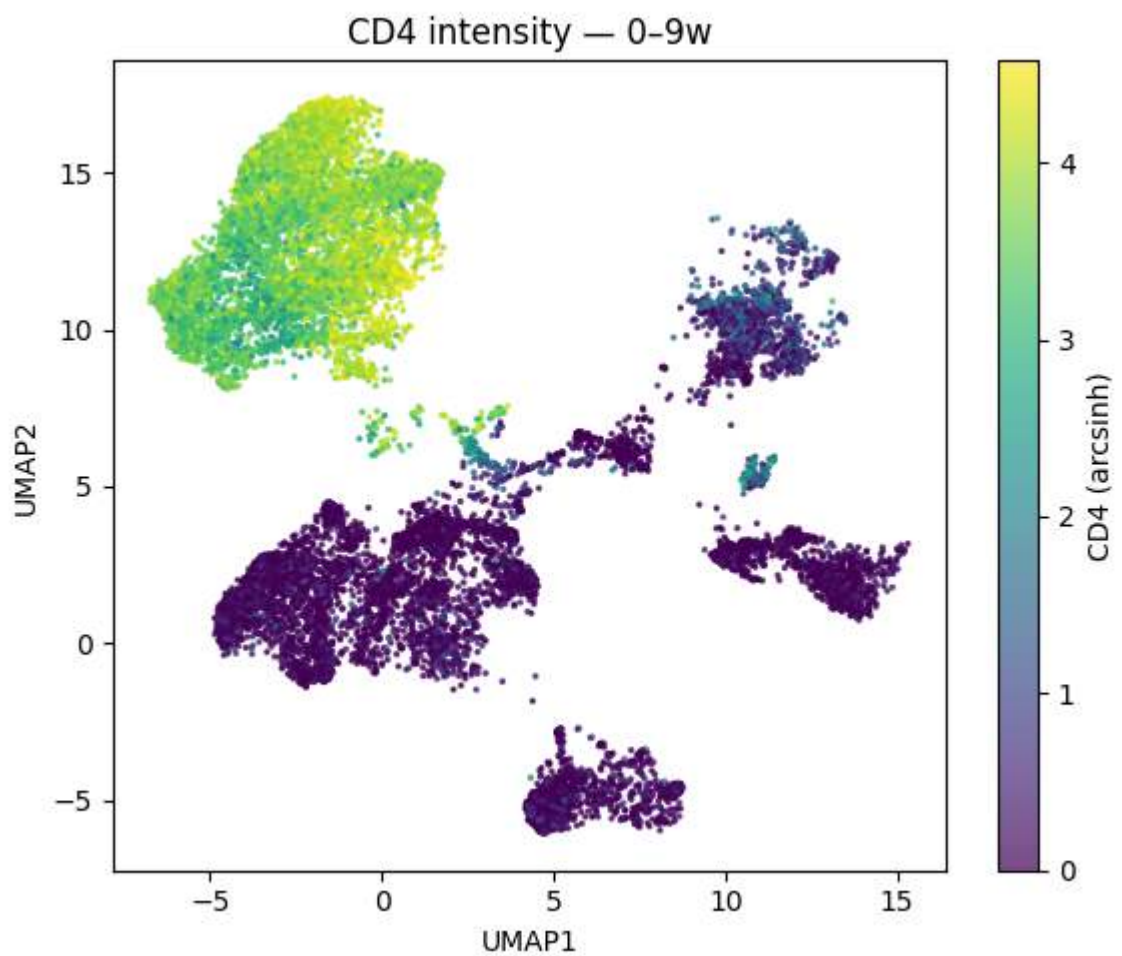
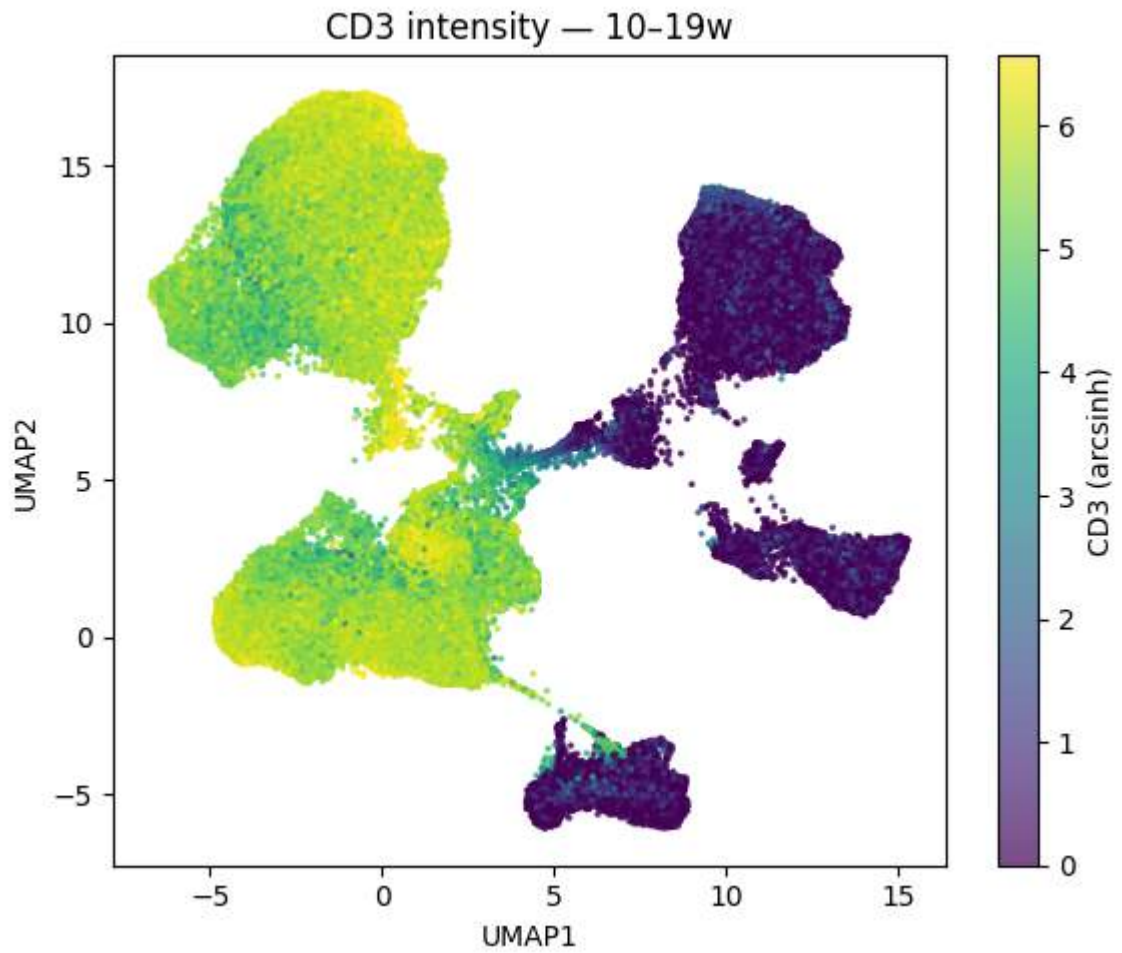
    if "_umap1" not in df.columns or "_umap2" not in df.columns:
        print("UMAP coordinates not found. Run UMAP first.")
        return

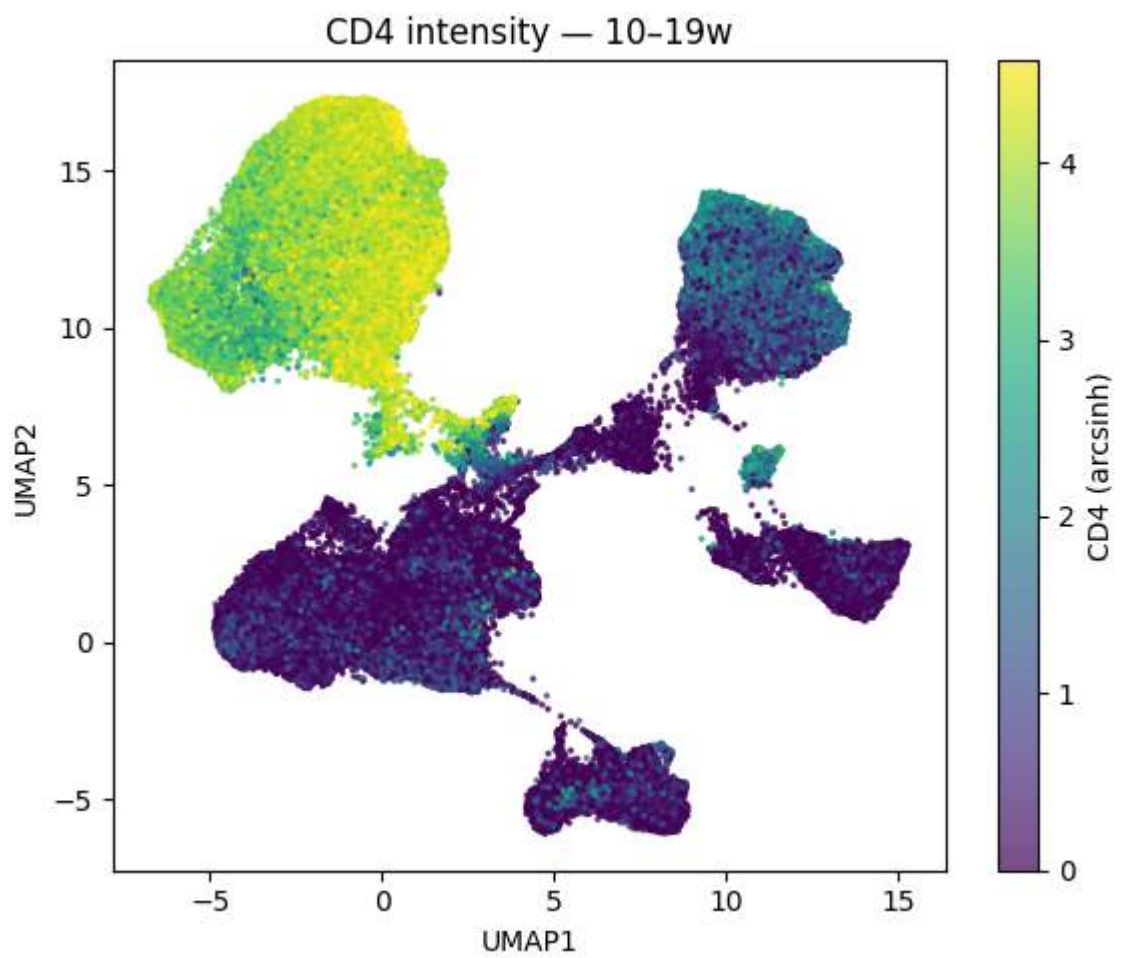
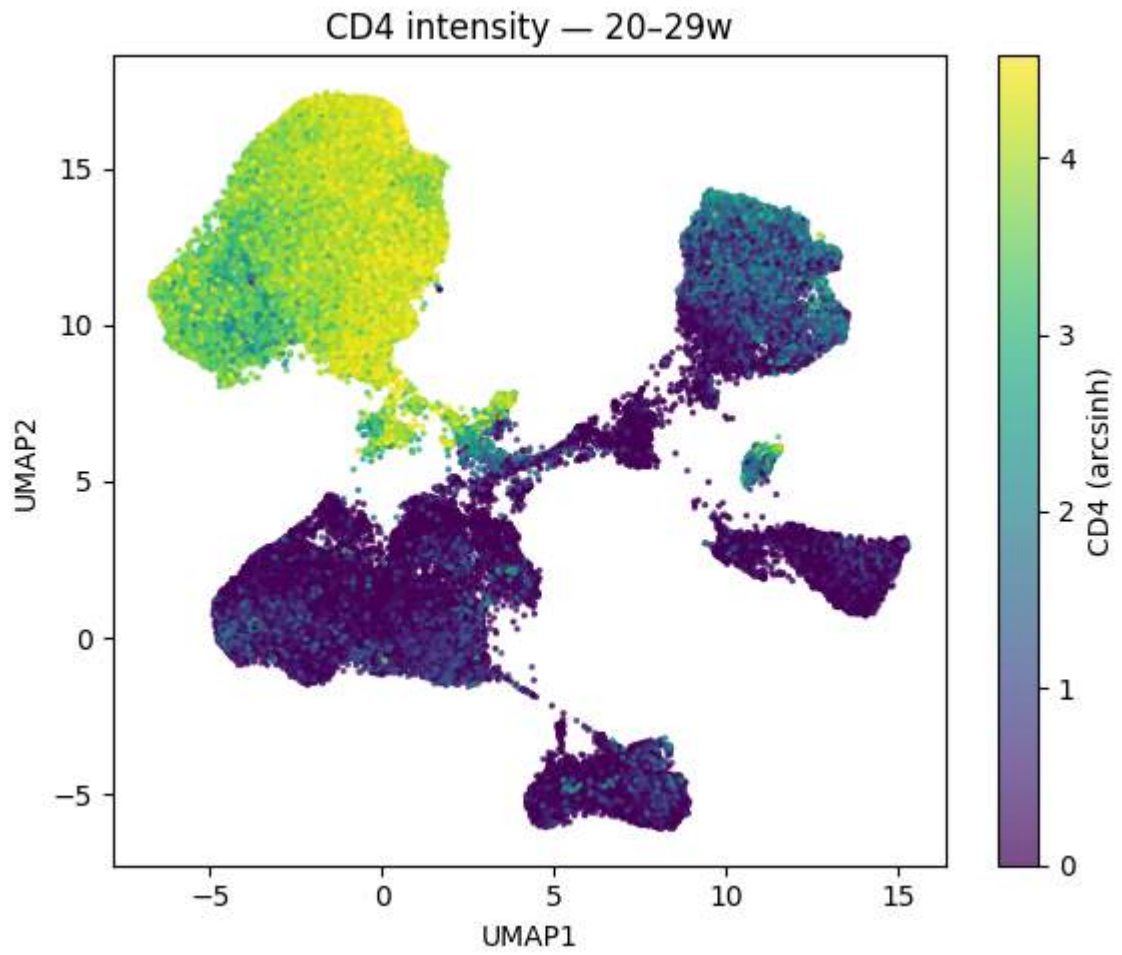
    vals = df[marker].dropna().values
    if vmin is None or vmax is None:
        q1, q99 = np.percentile(vals, [1, 99])
        vmin, vmax = float(q1), float(q99)

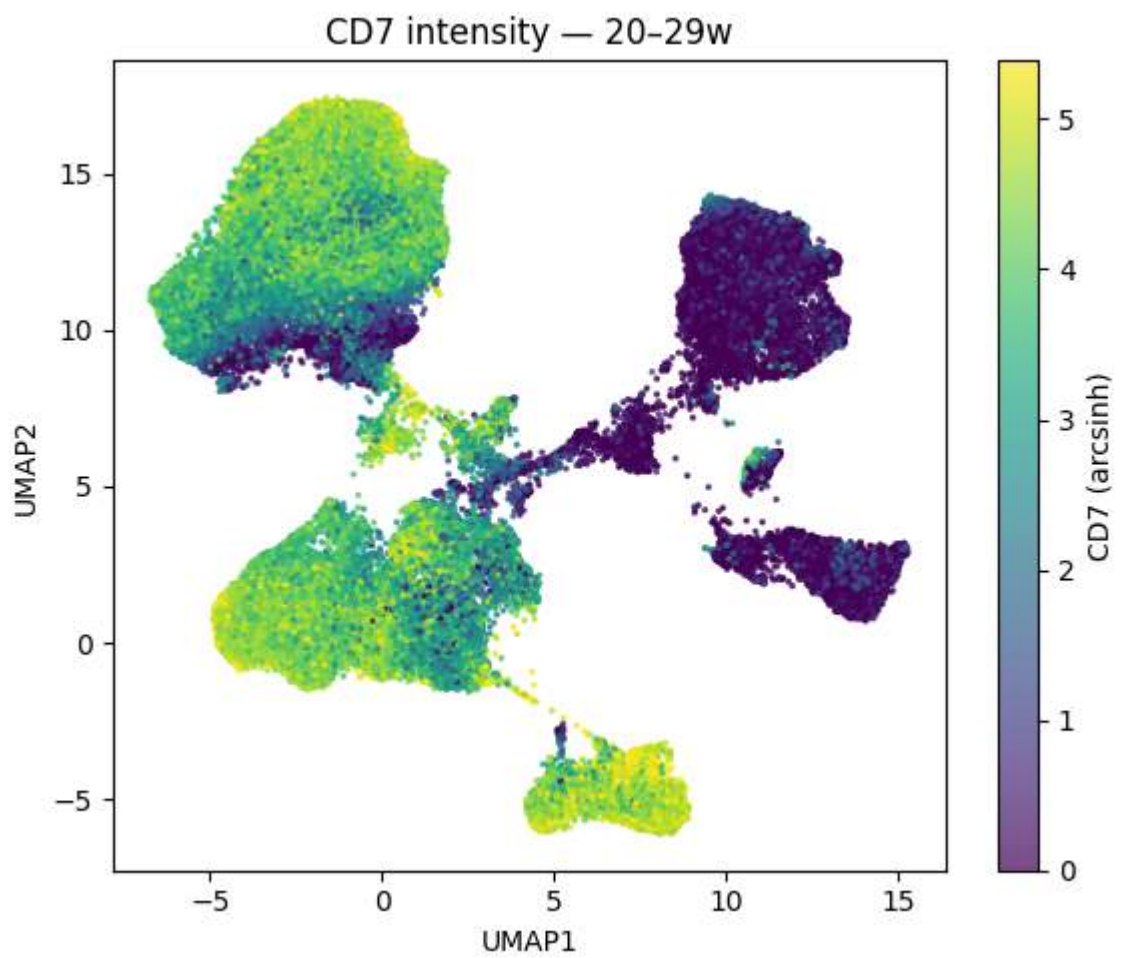
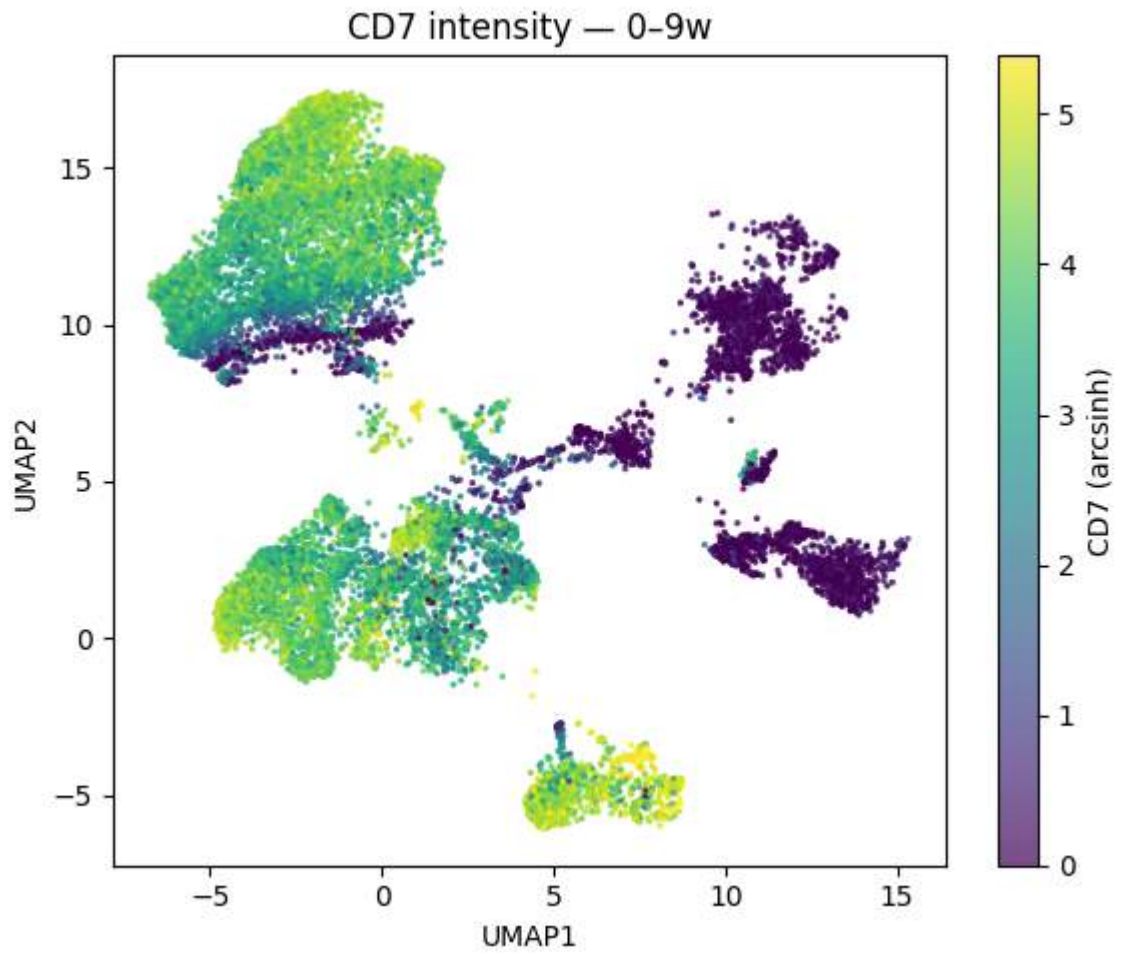
    for lab in df[age_col].dropna().unique():
        sub = df[df[age_col] == lab]
        if sub.empty:
            continue
        plt.figure(figsize=(6,5))
        sc = plt.scatter(sub["_umap1"], sub["_umap2"], s=2, c=sub[marker], vmin=vmin,
        plt.xlabel("UMAP1"); plt.ylabel("UMAP2")
        plt.title(f"{marker} intensity - {lab}")
        plt.colorbar(sc, label=f"{marker} (arcsinh)")
        plt.tight_layout(); plt.show()
```

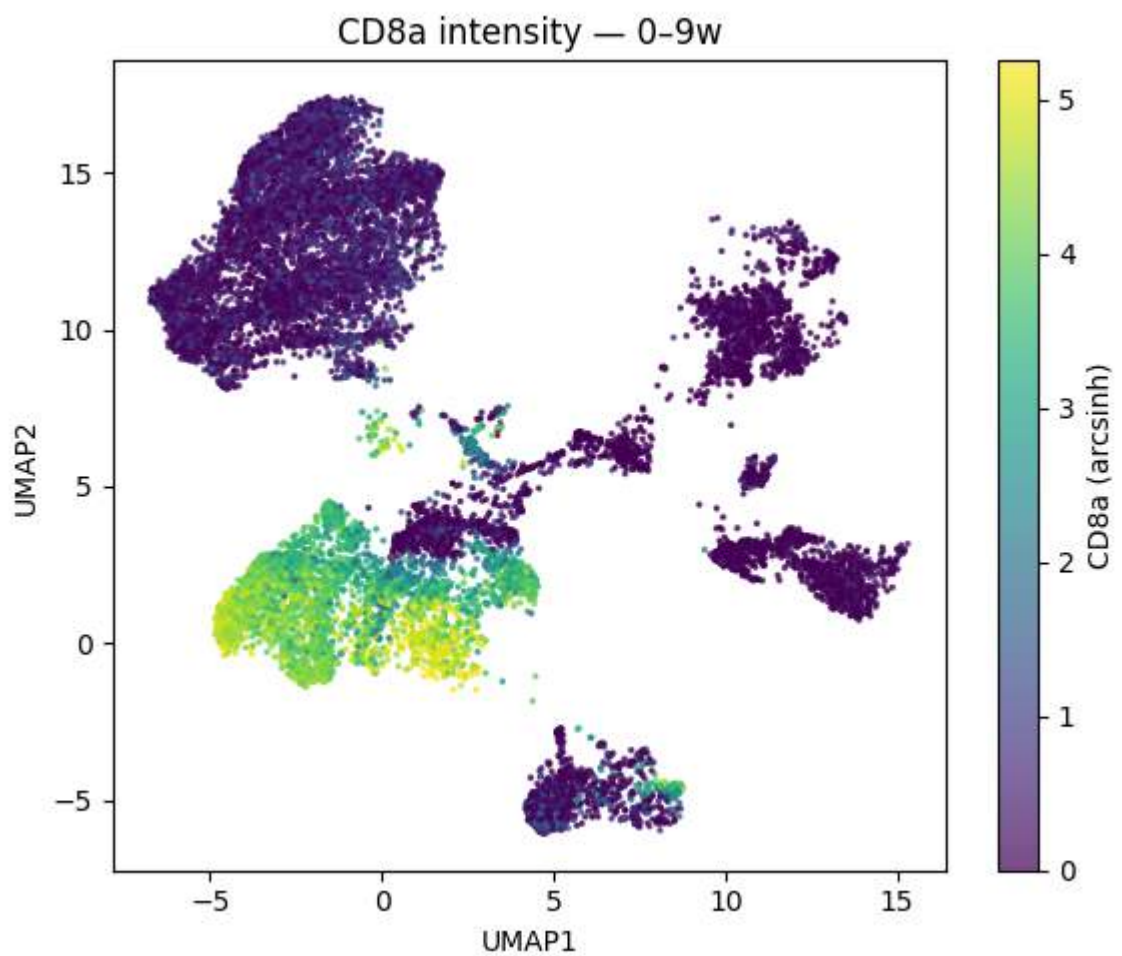
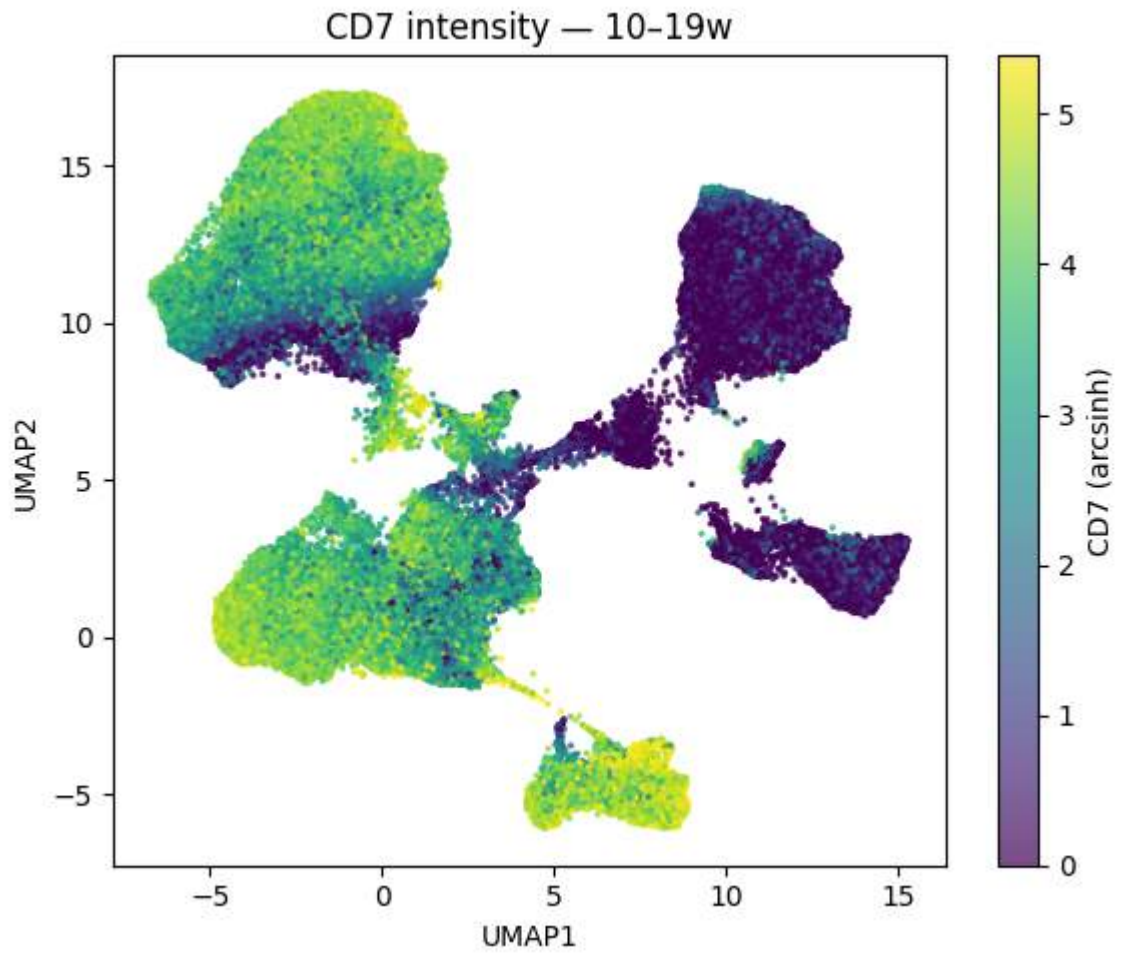
```
In [ ]: for m in markers_of_interest:
    umap_by_marker_age_bins(eda, m)
```

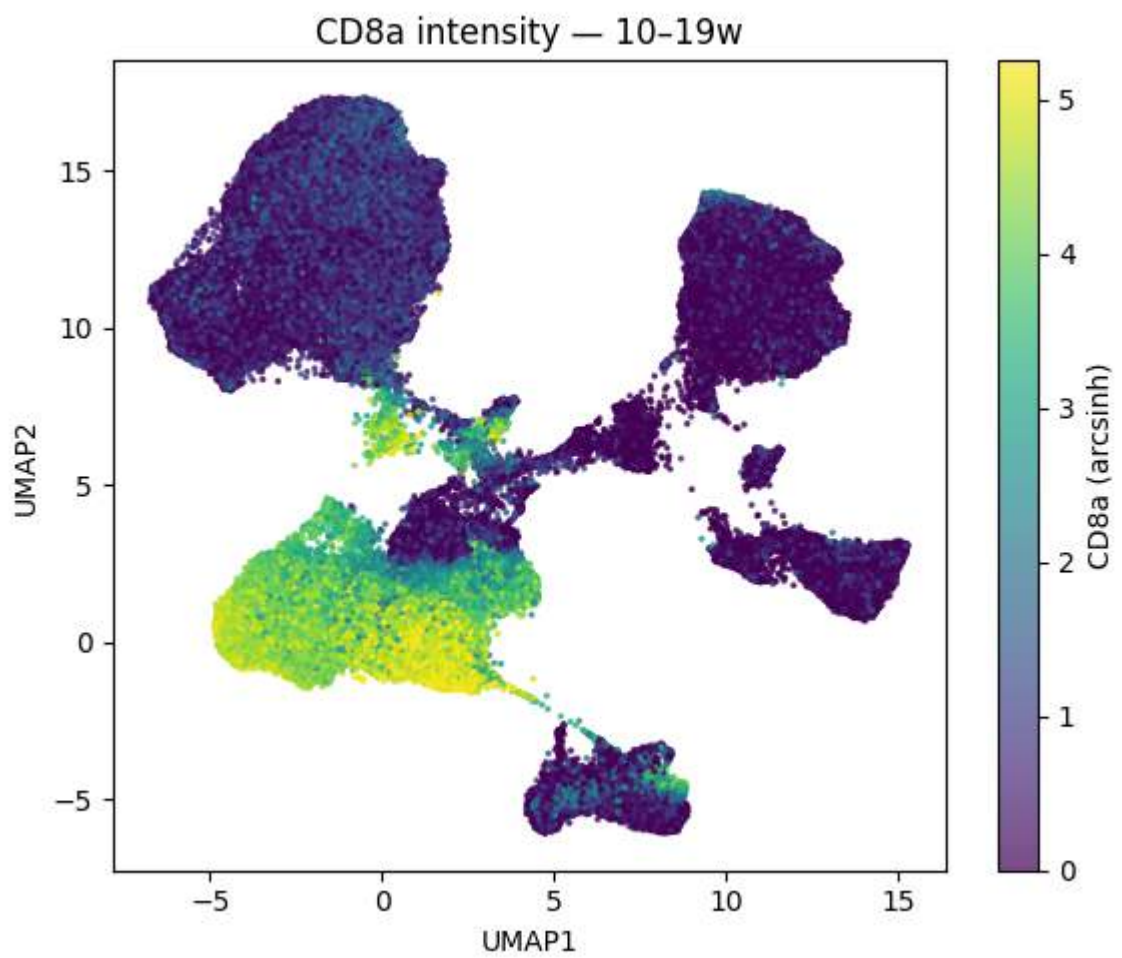
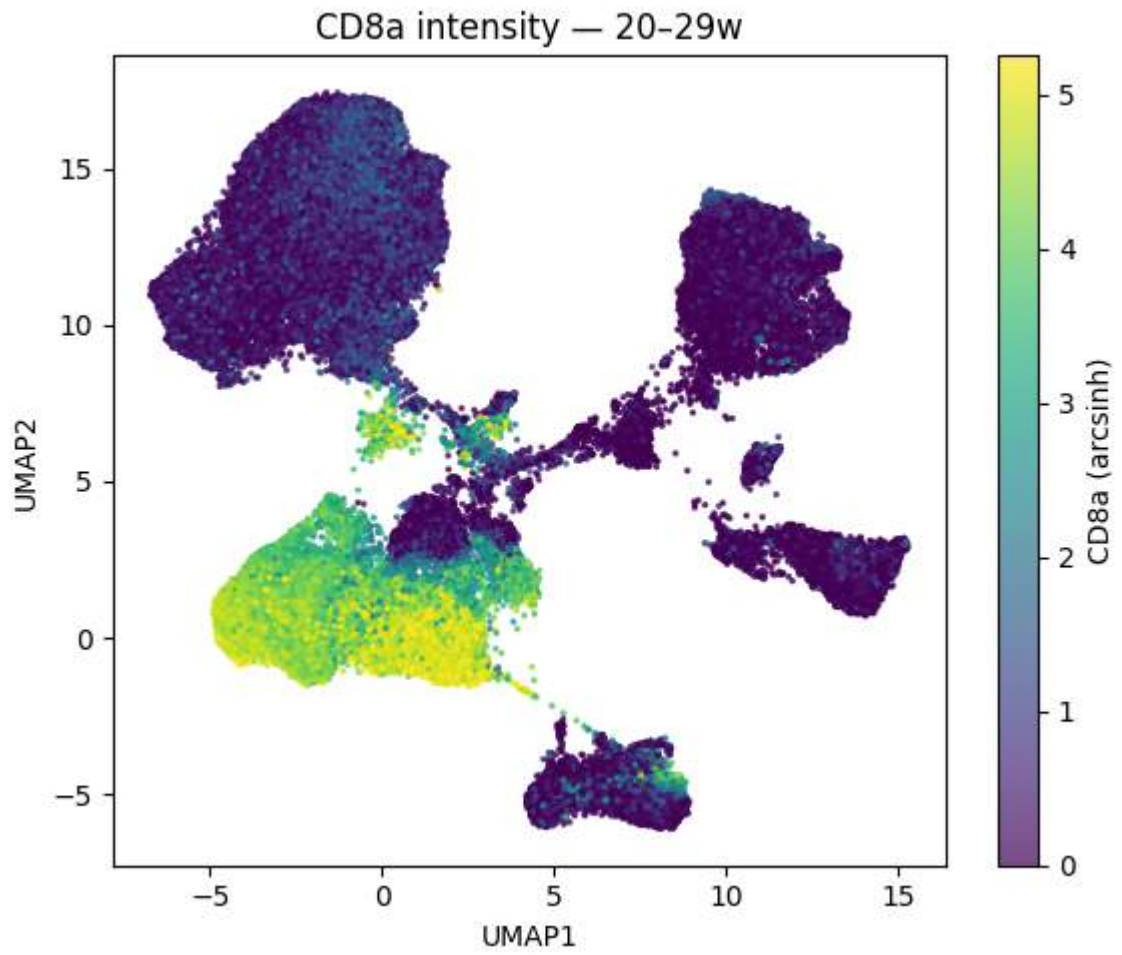



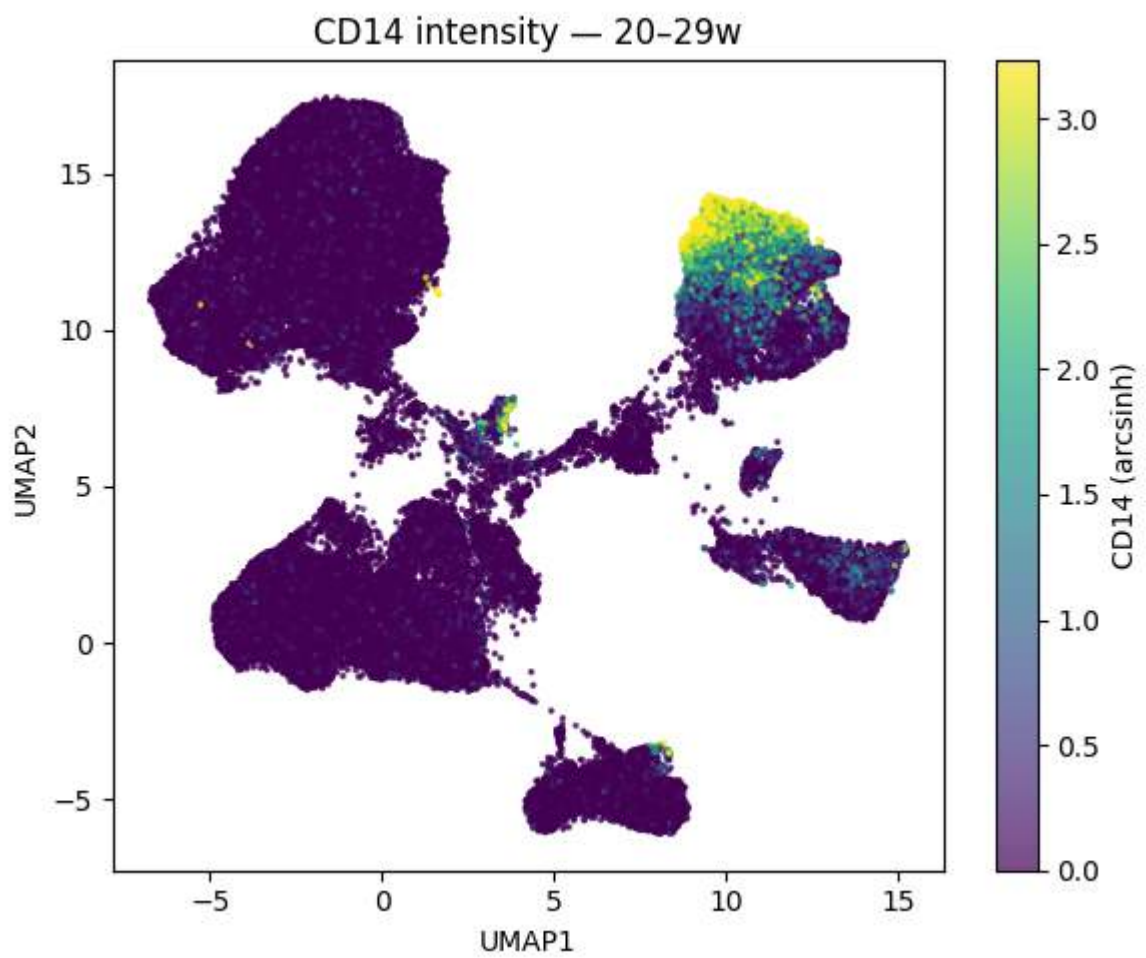
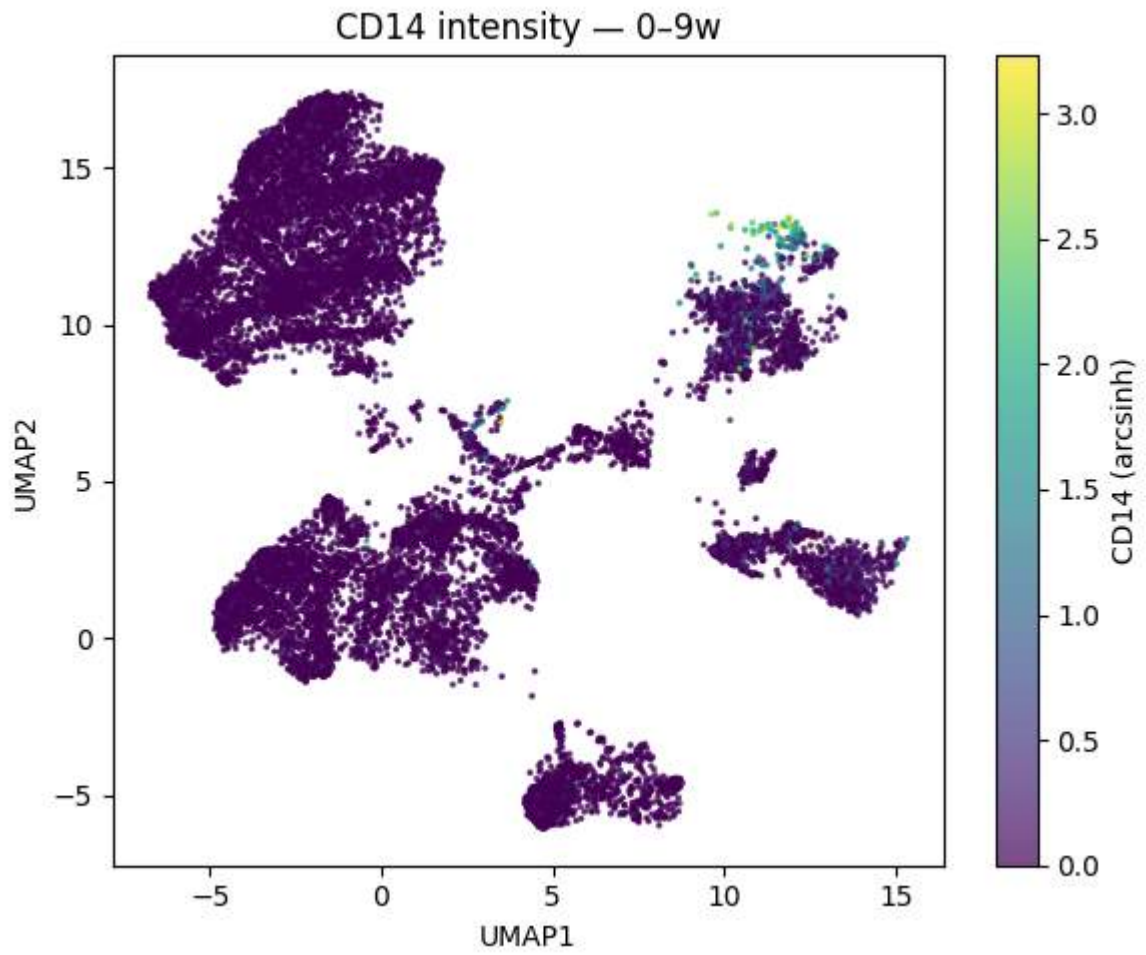


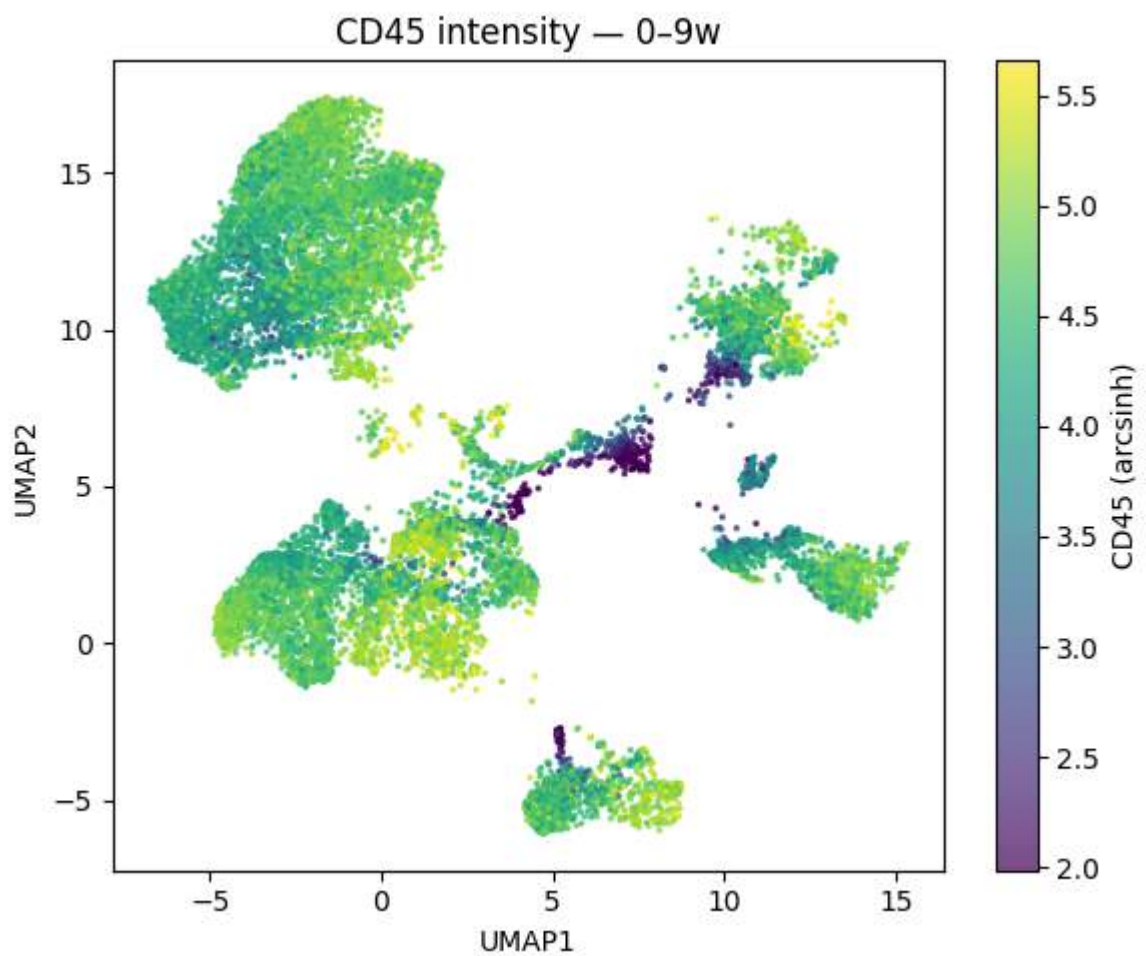
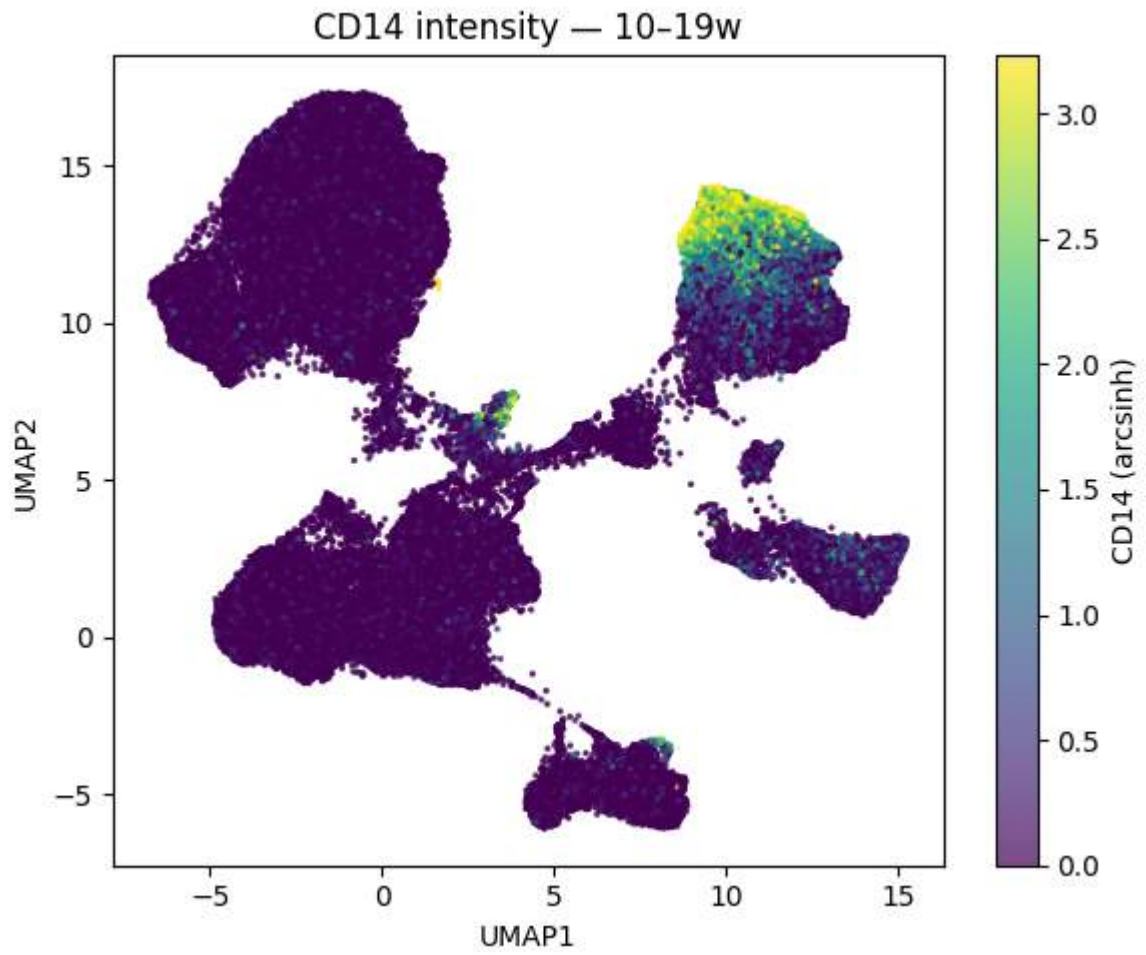


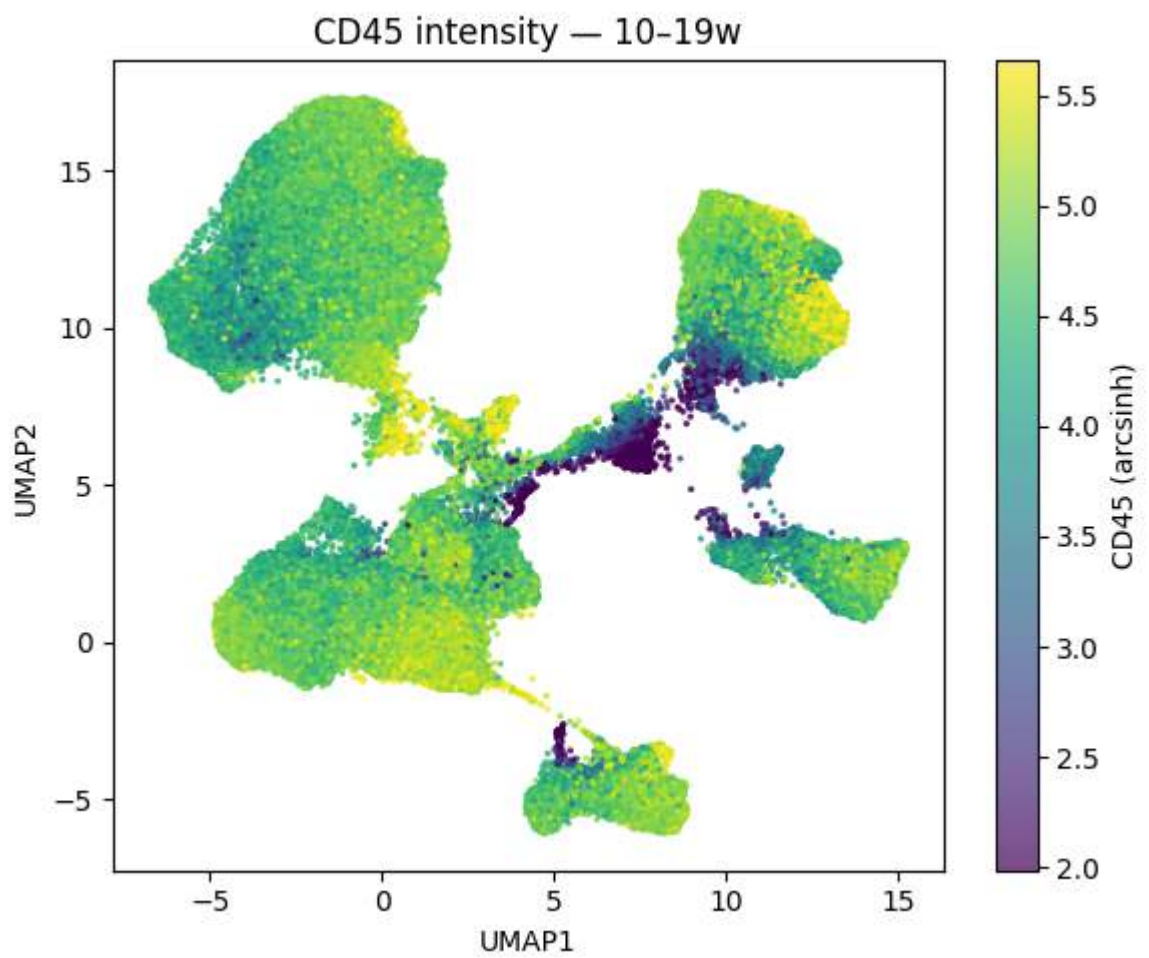
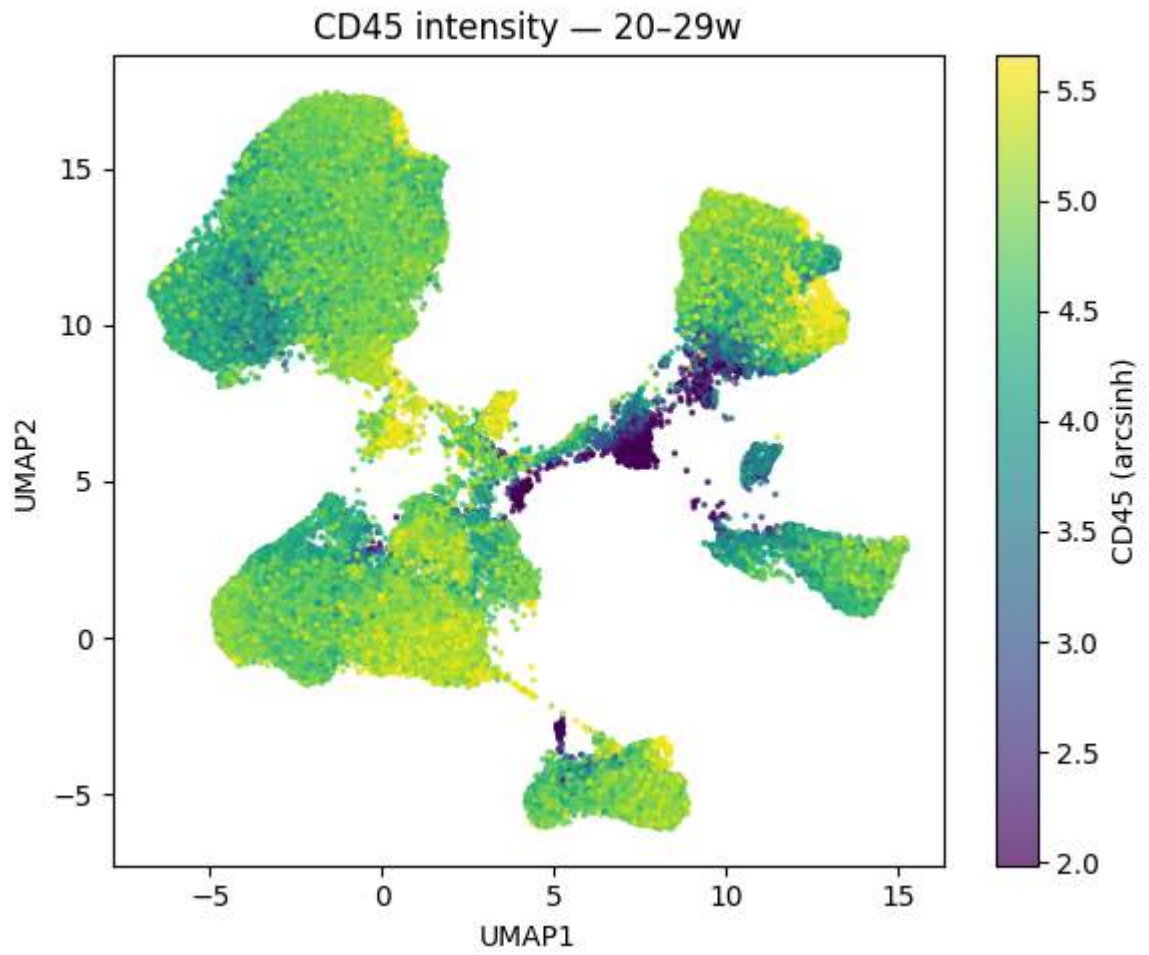


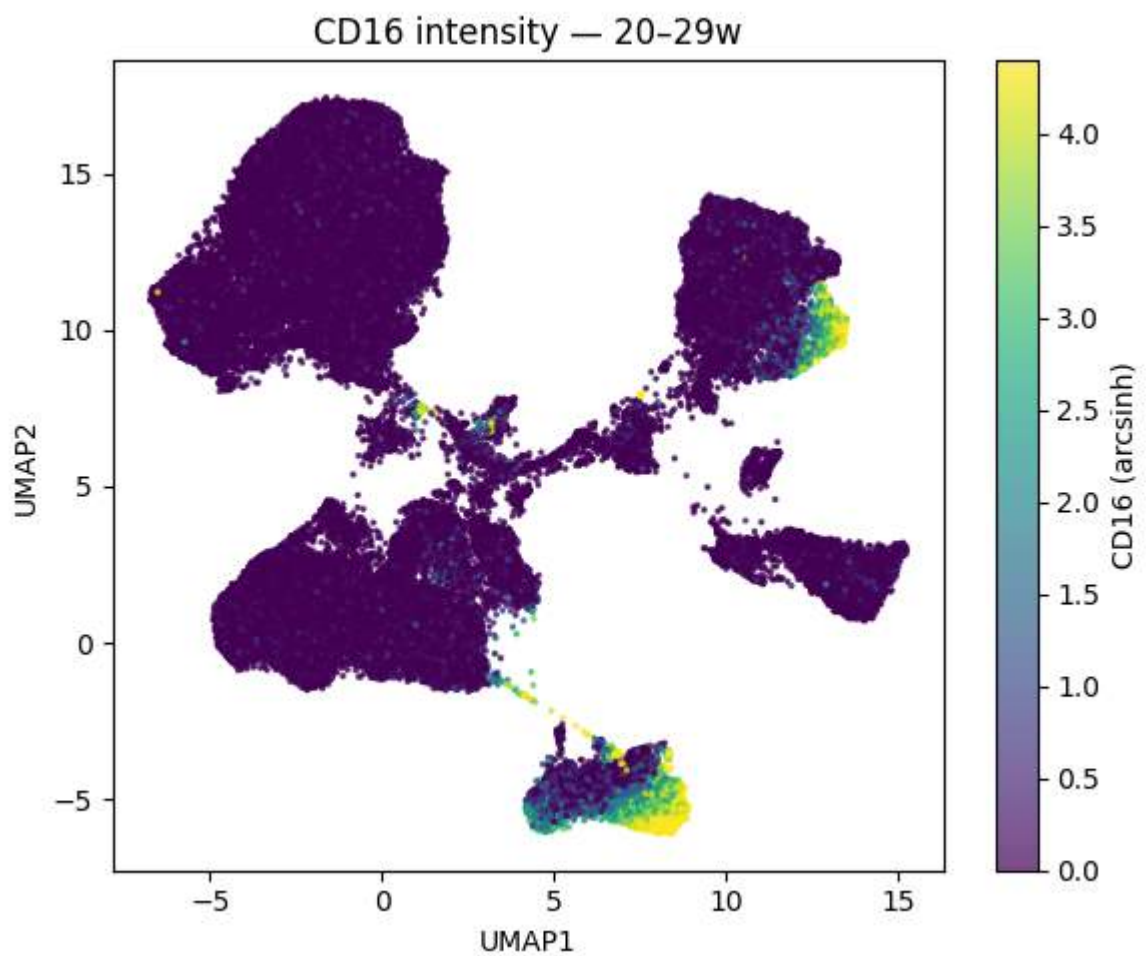
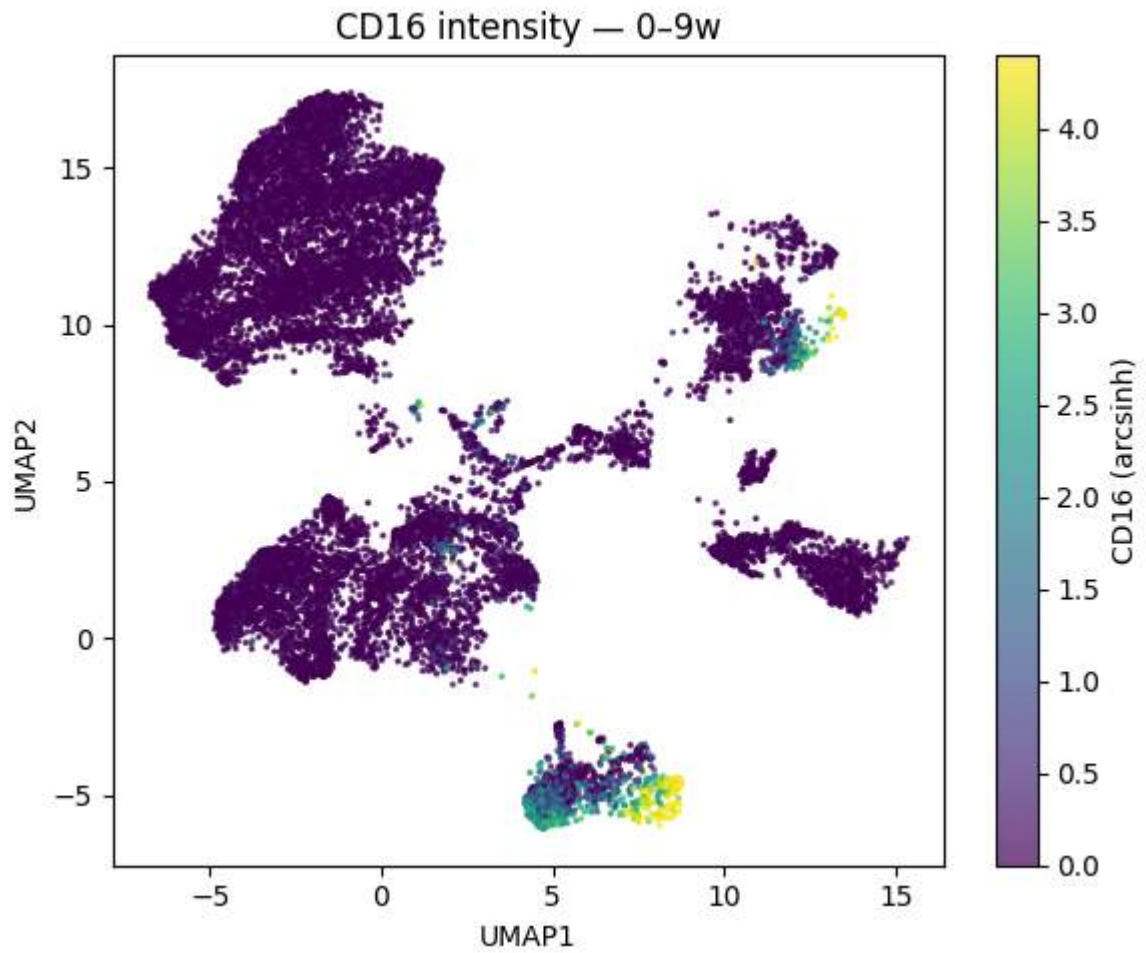


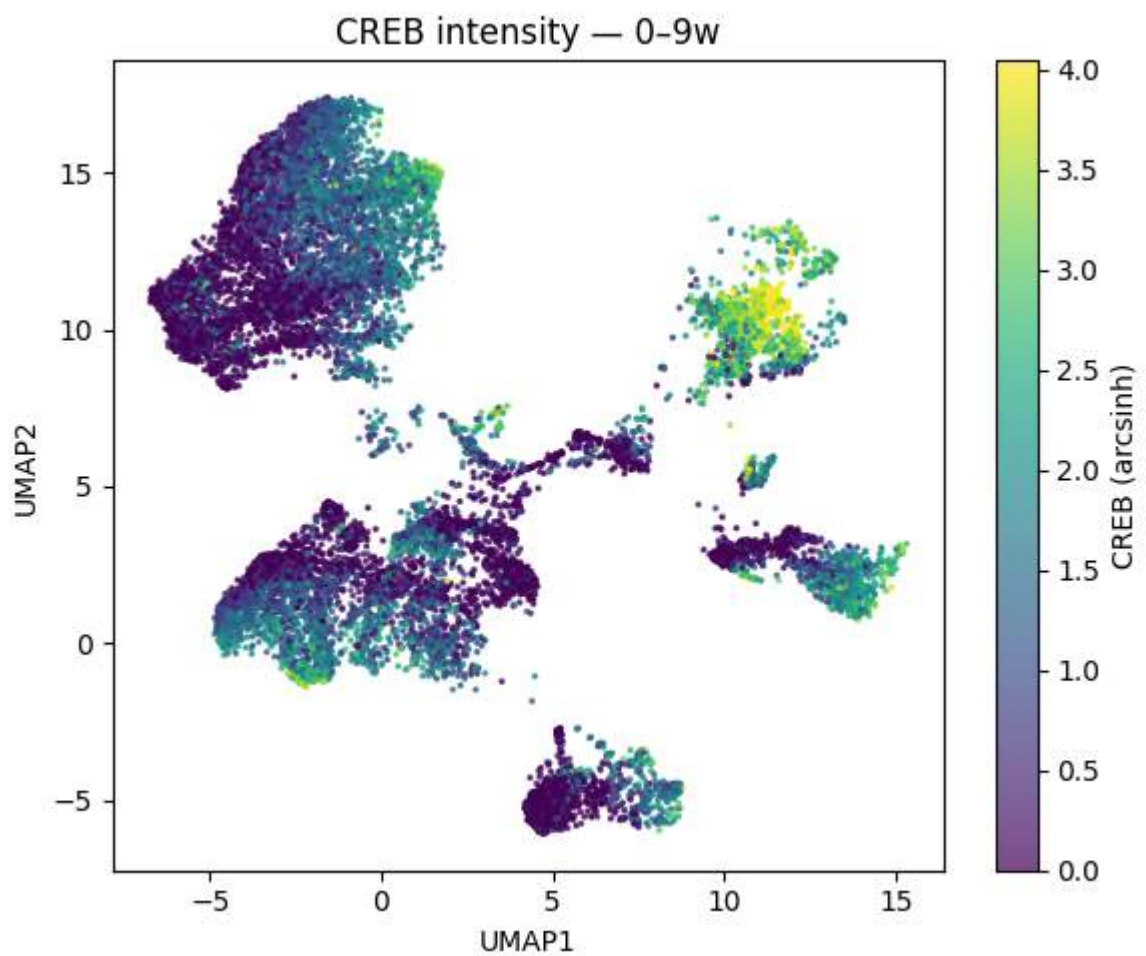
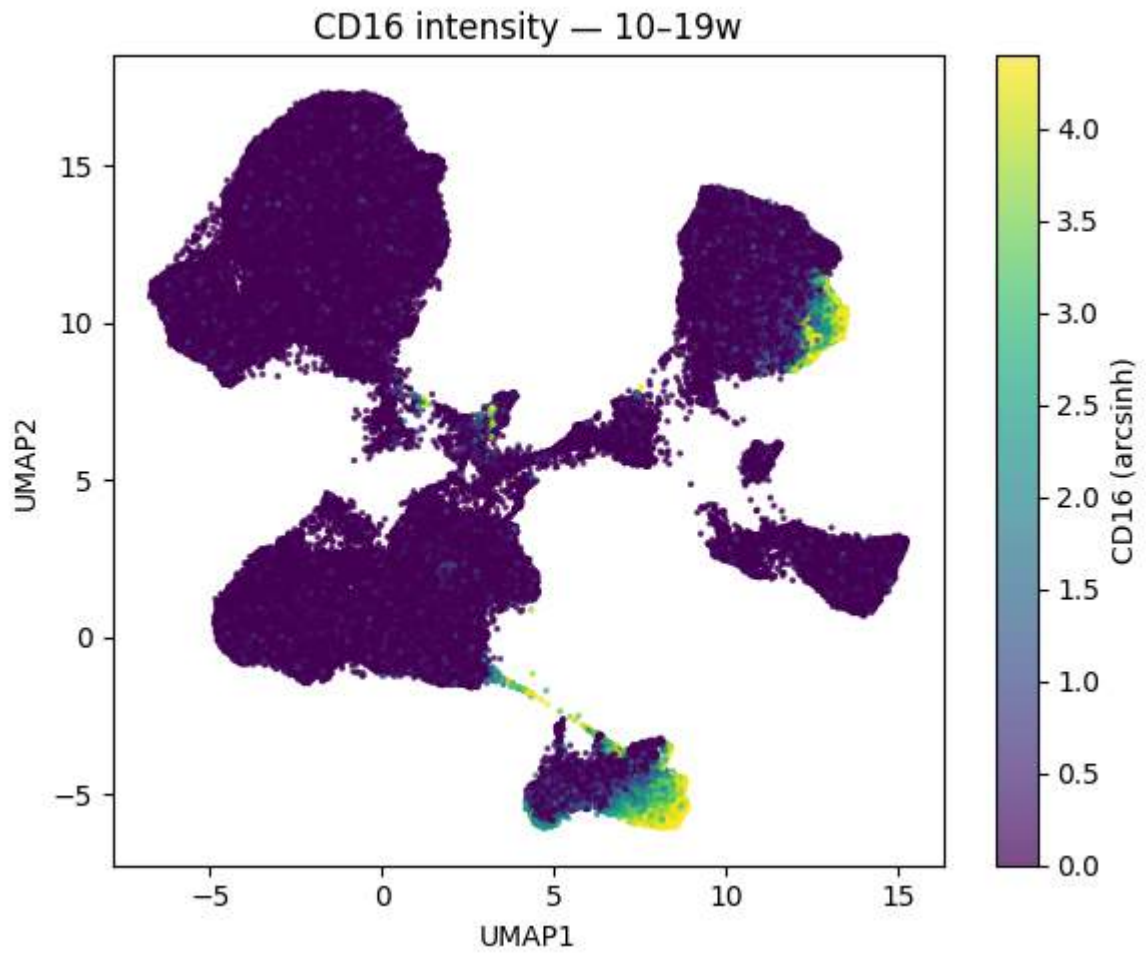


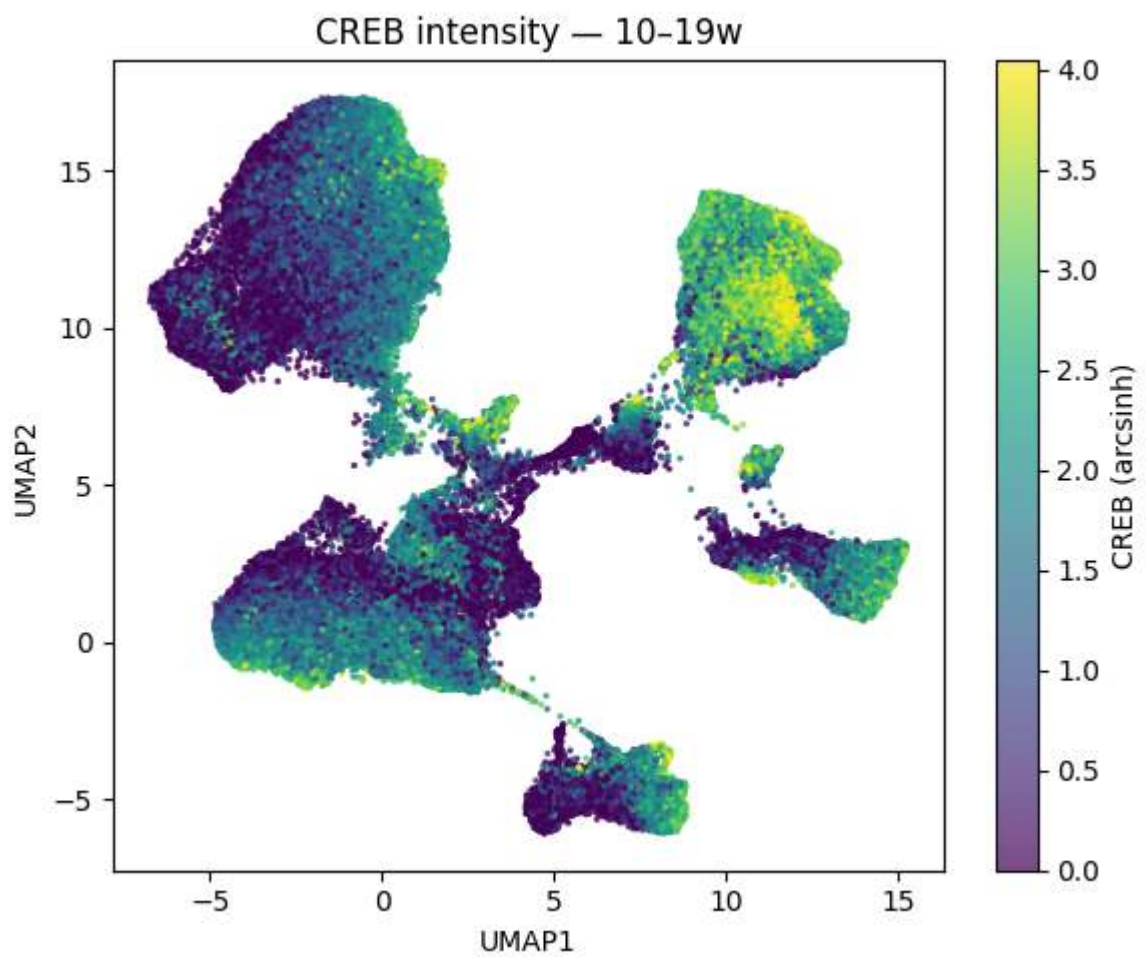
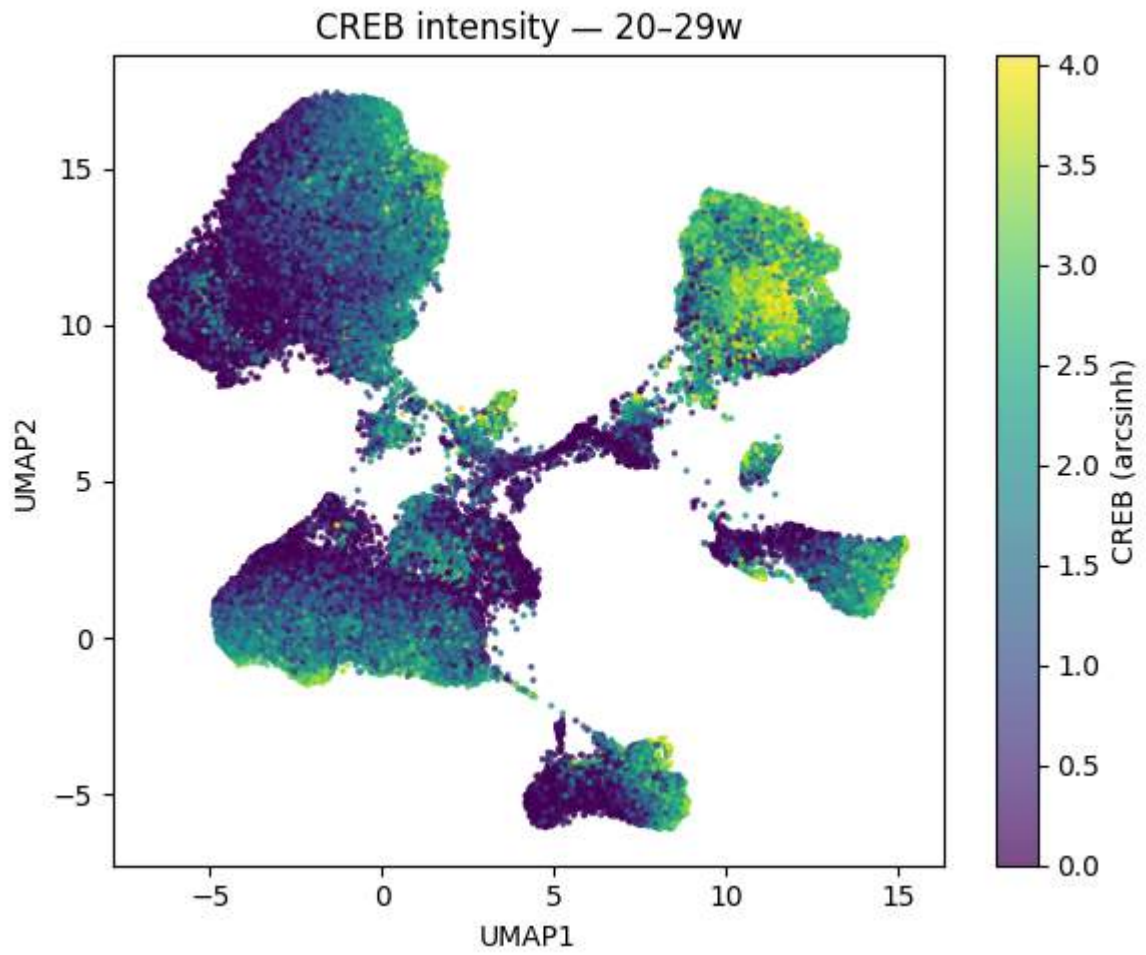


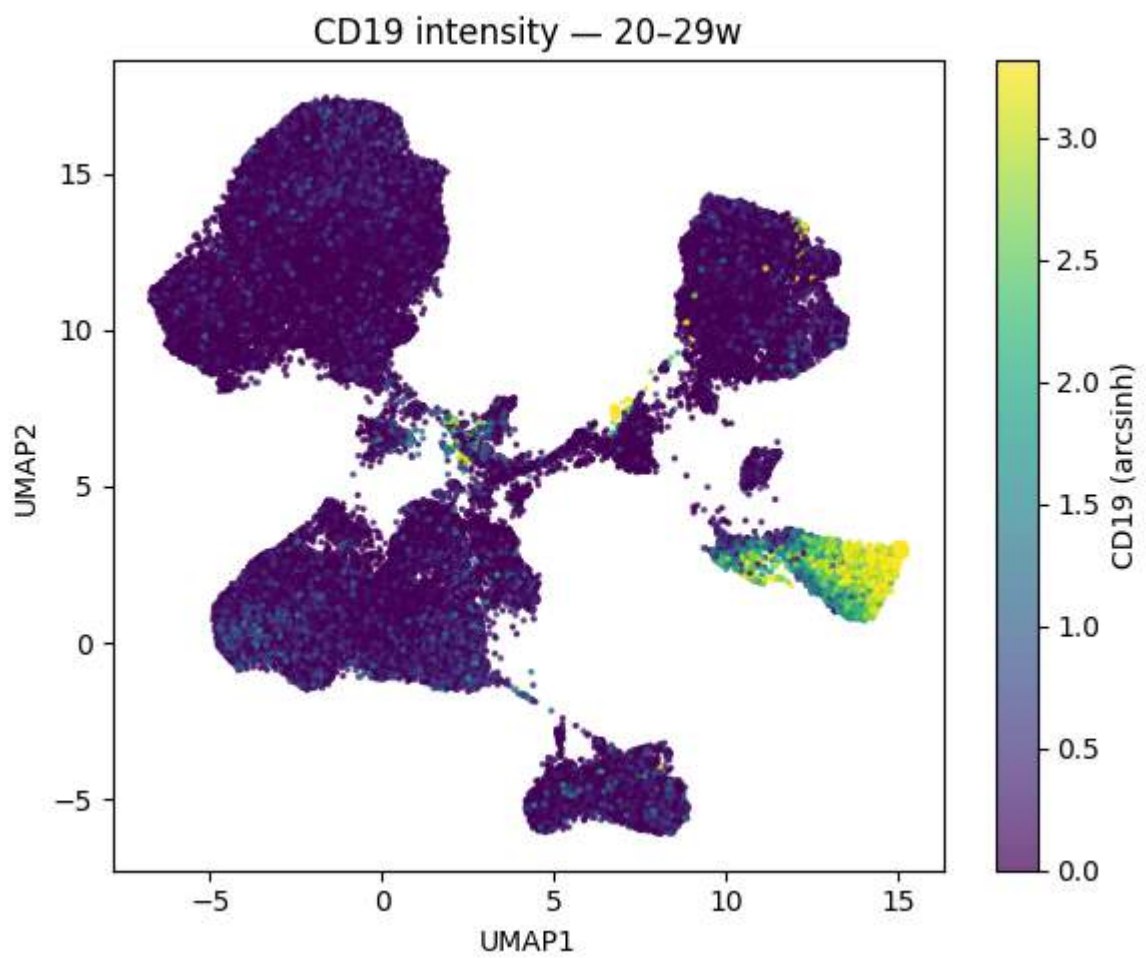
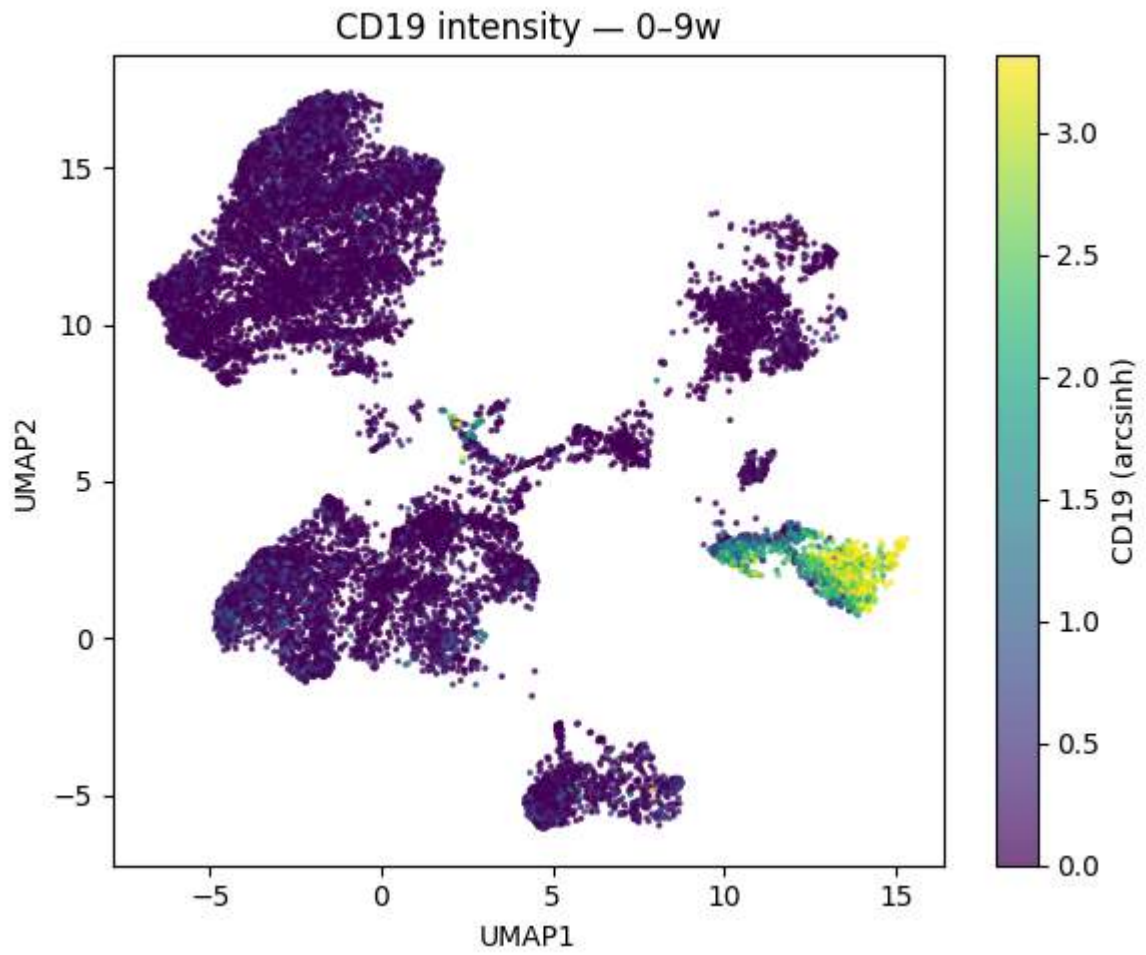


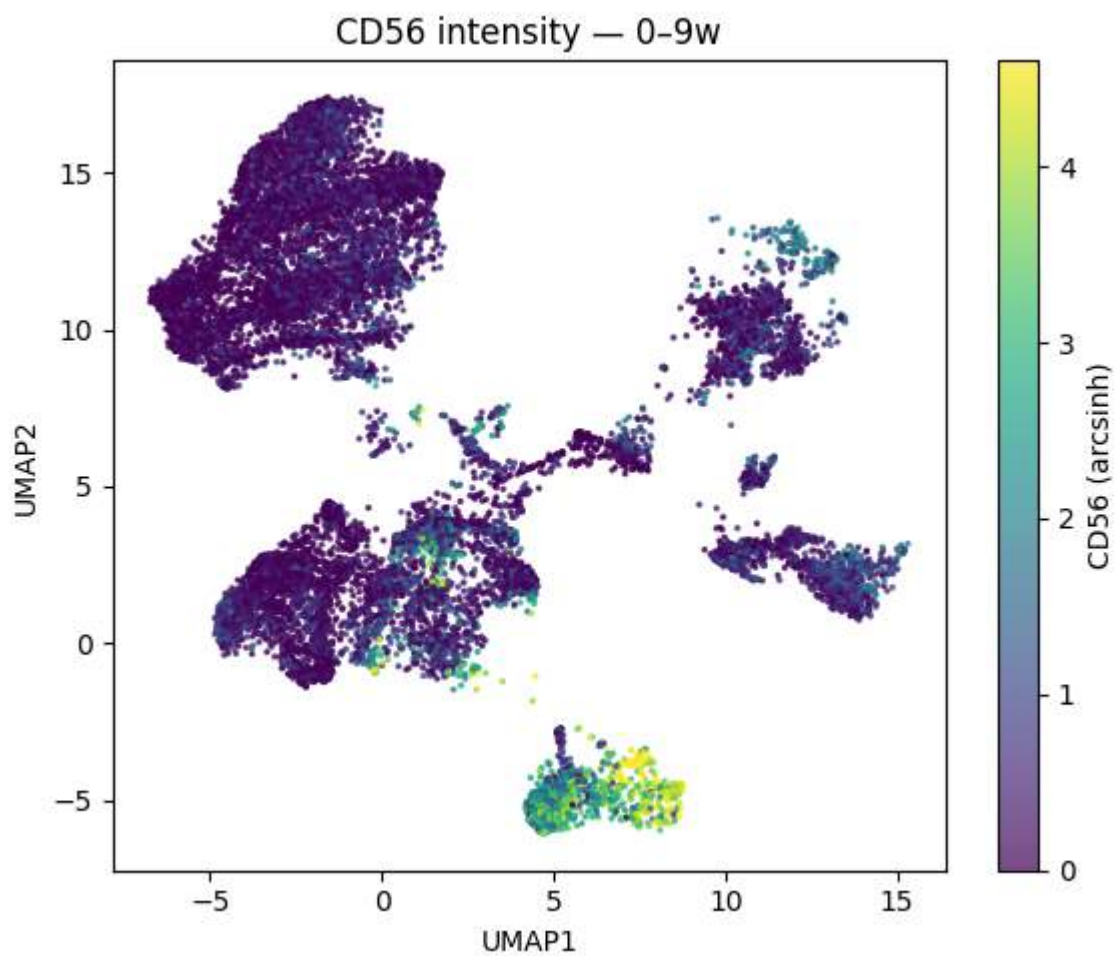
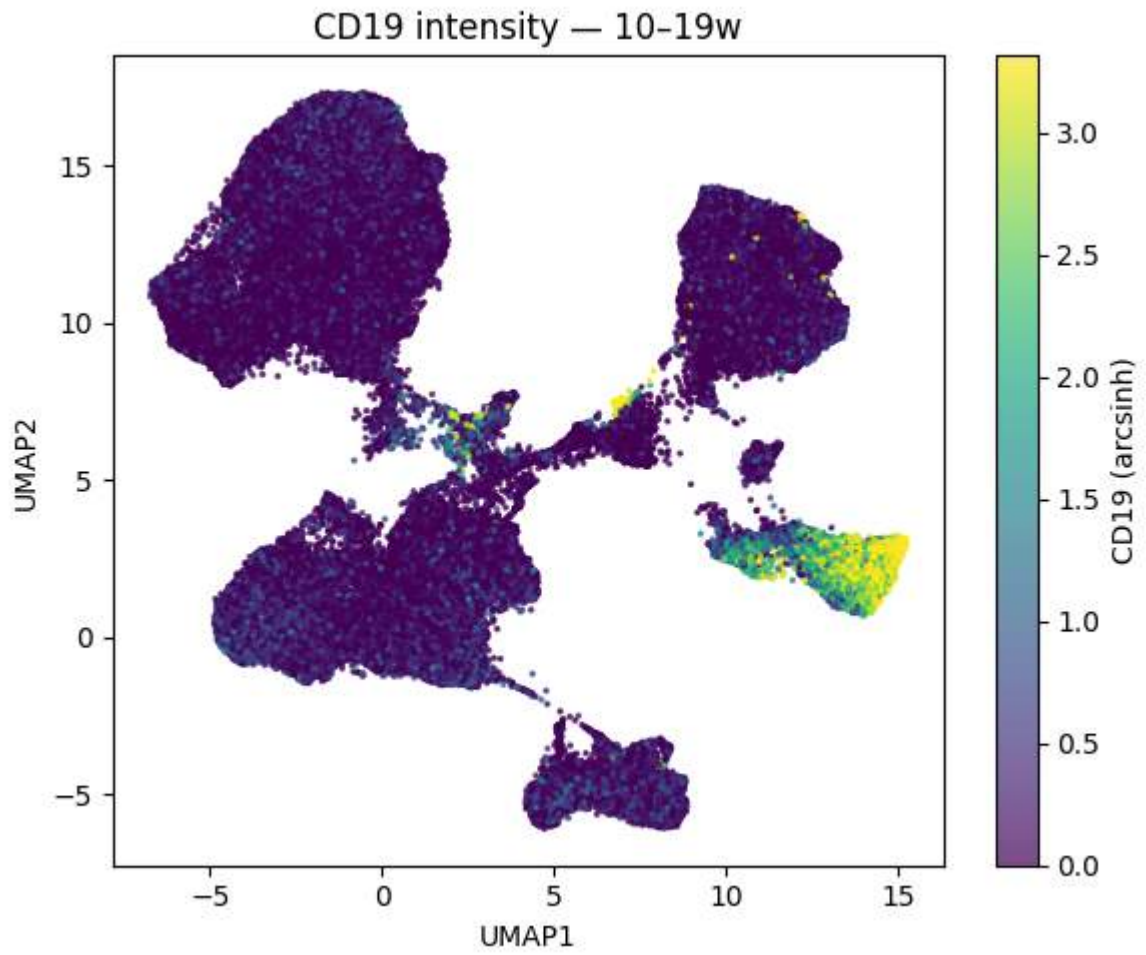


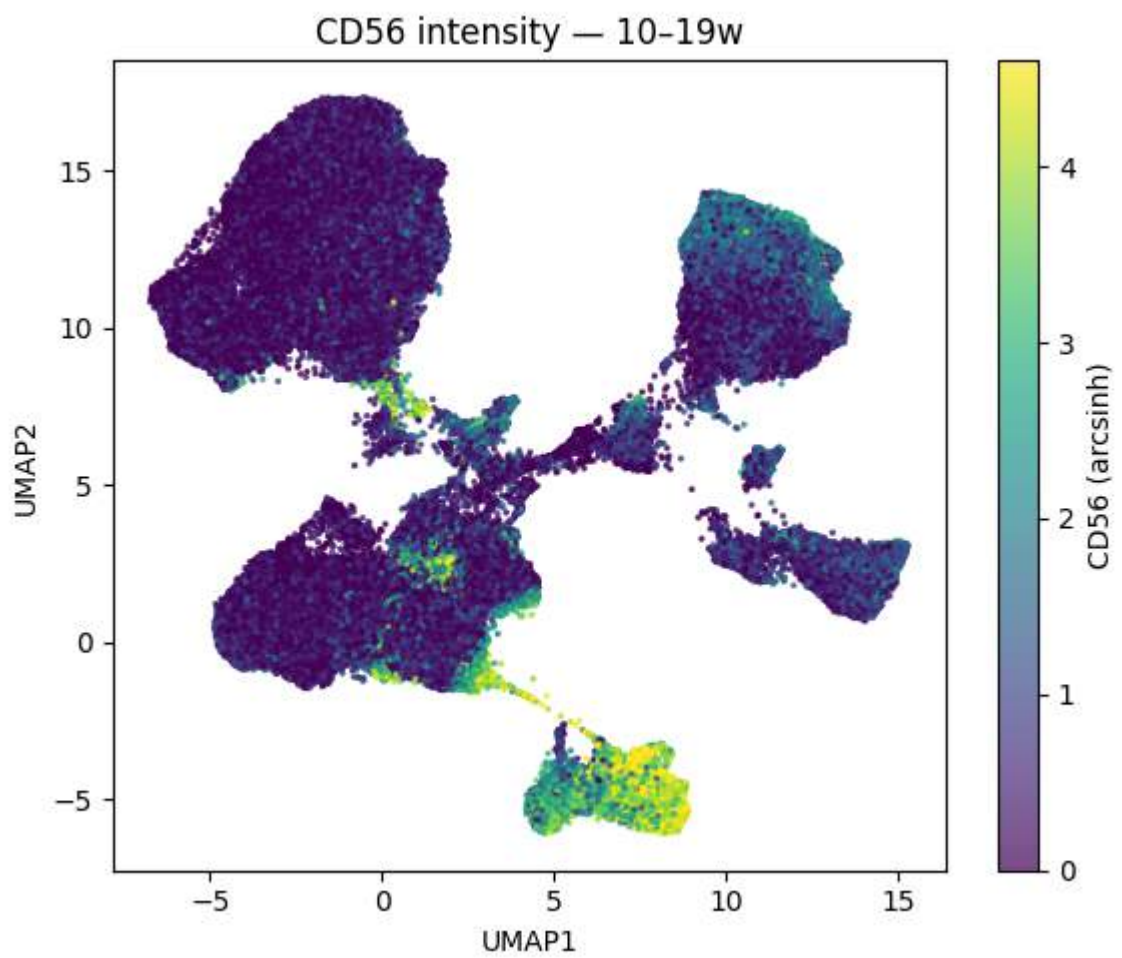
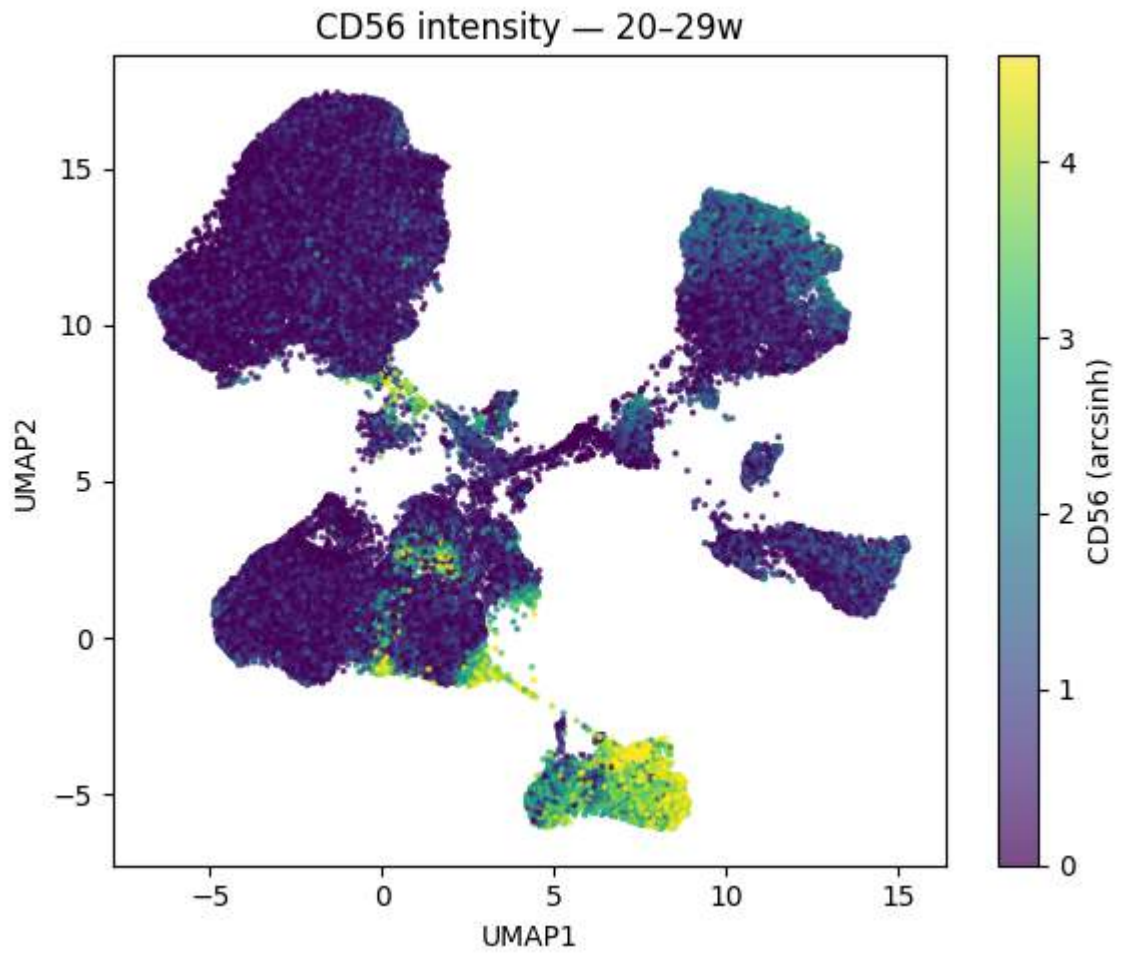


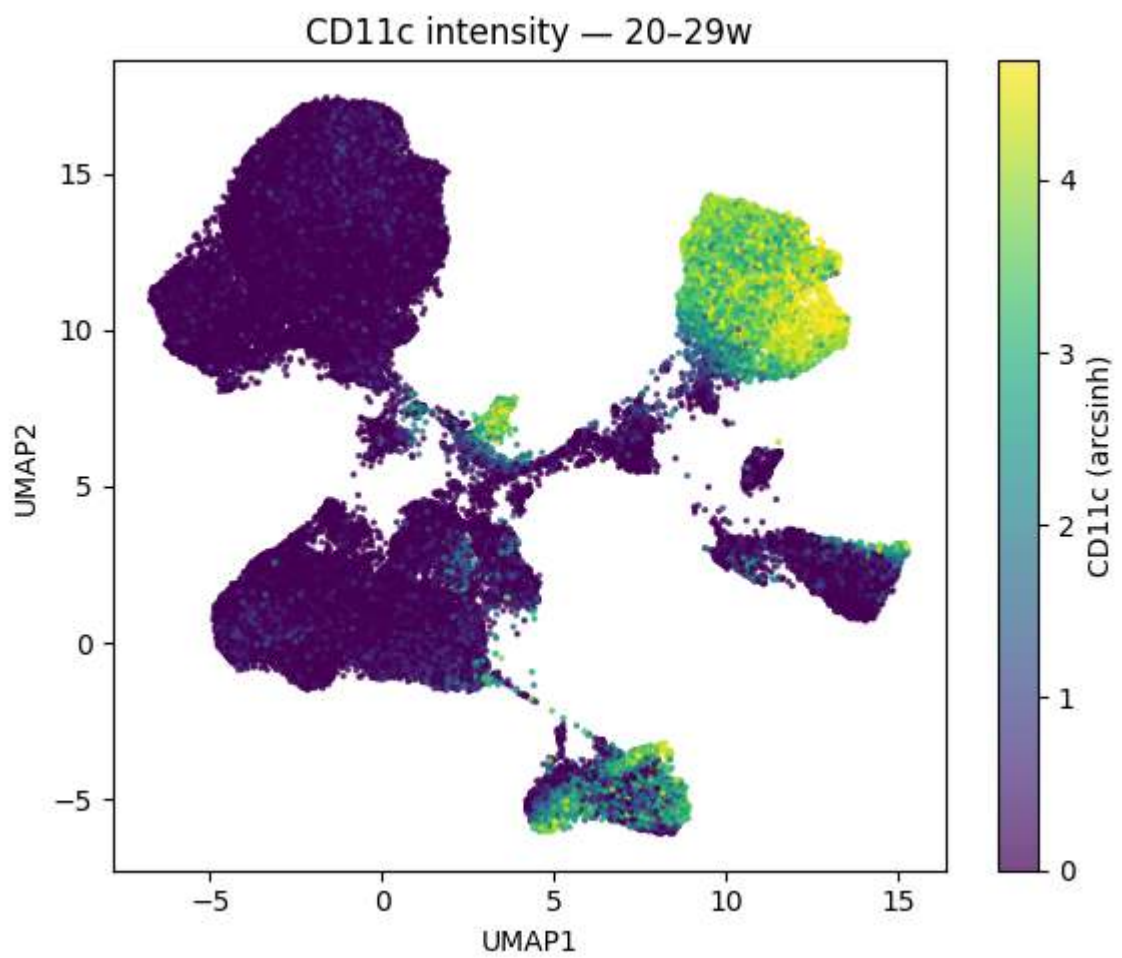
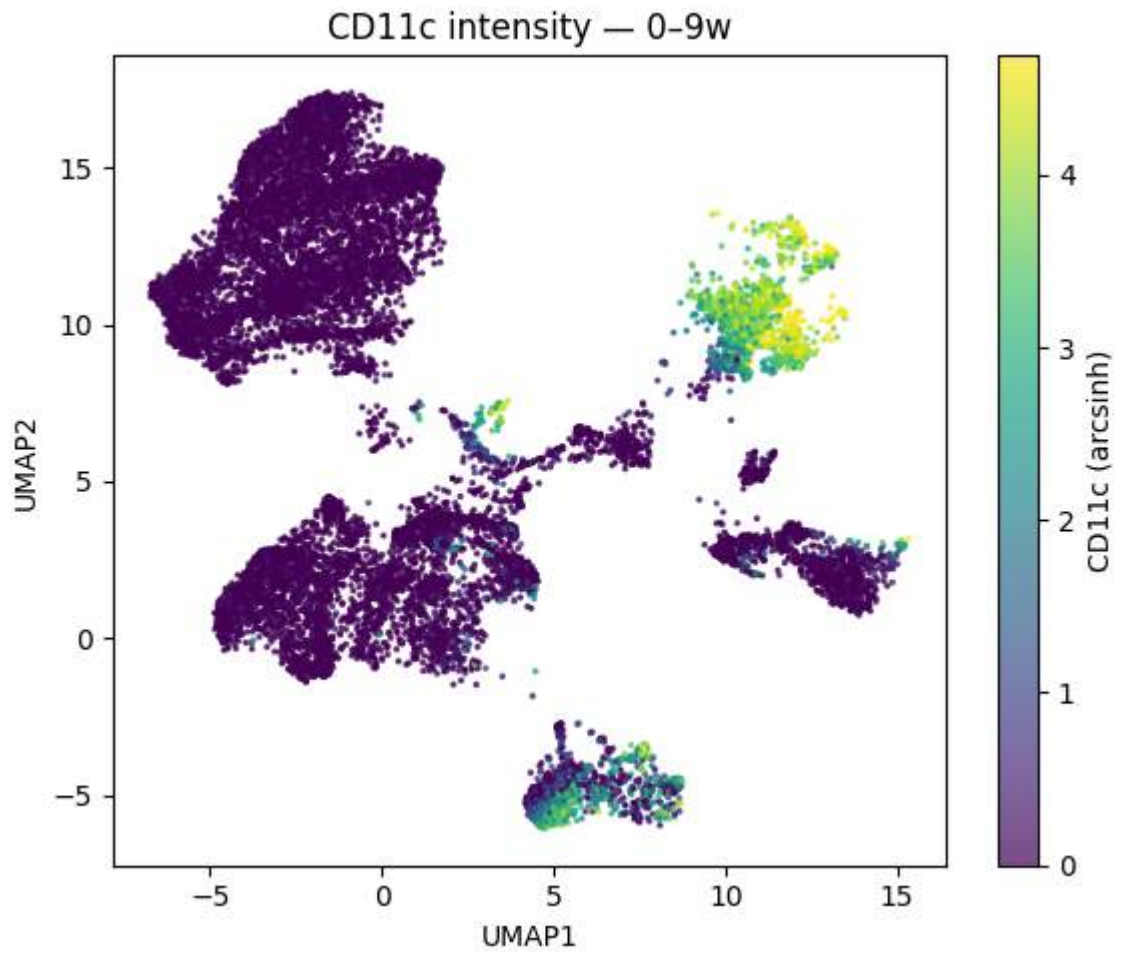


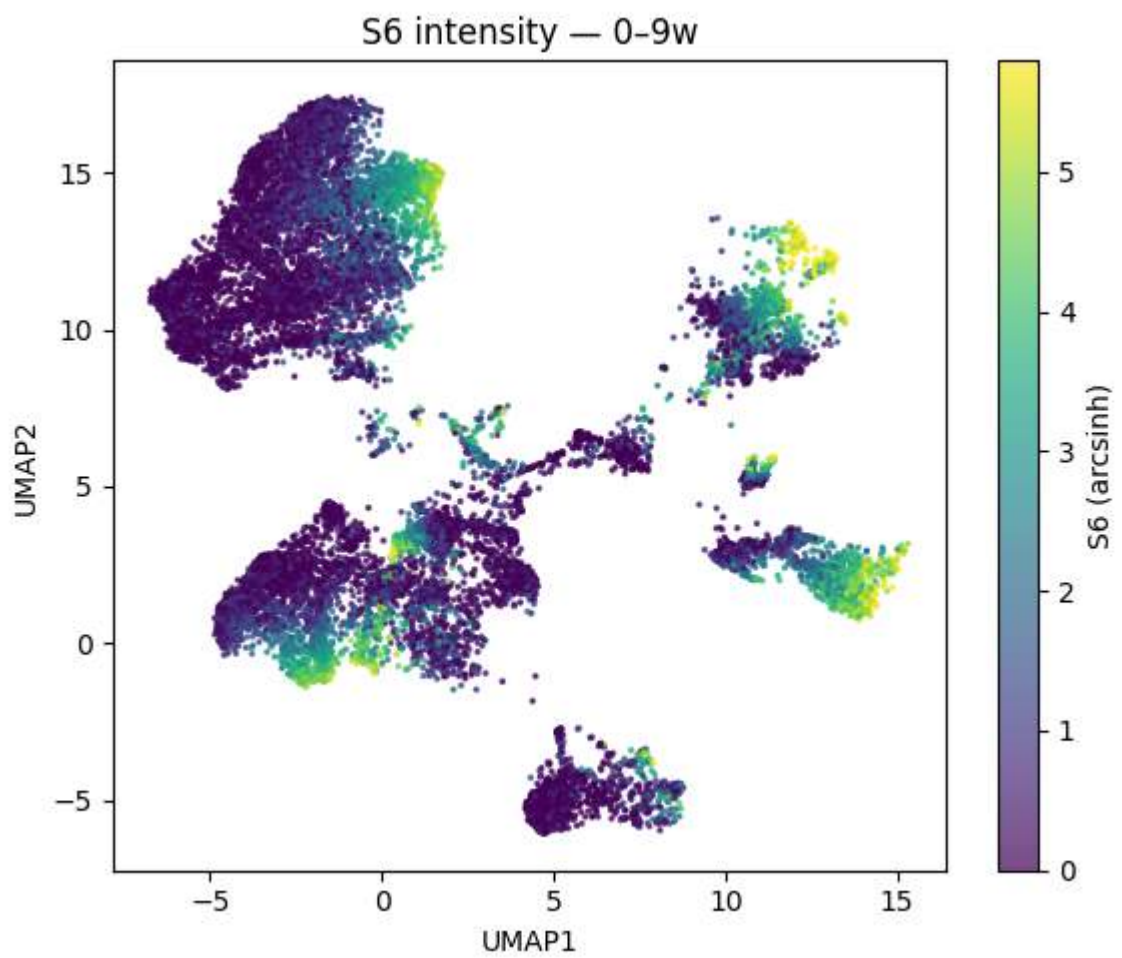
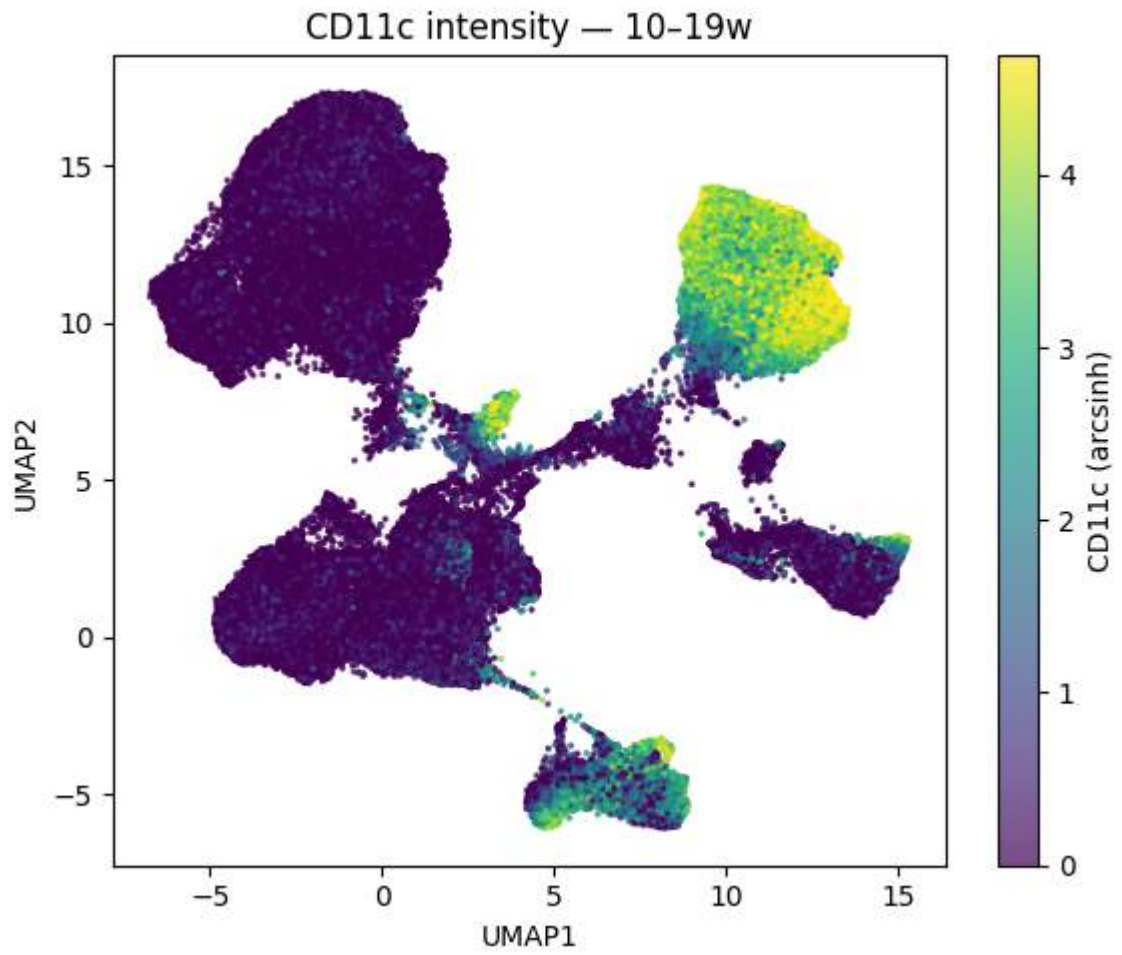


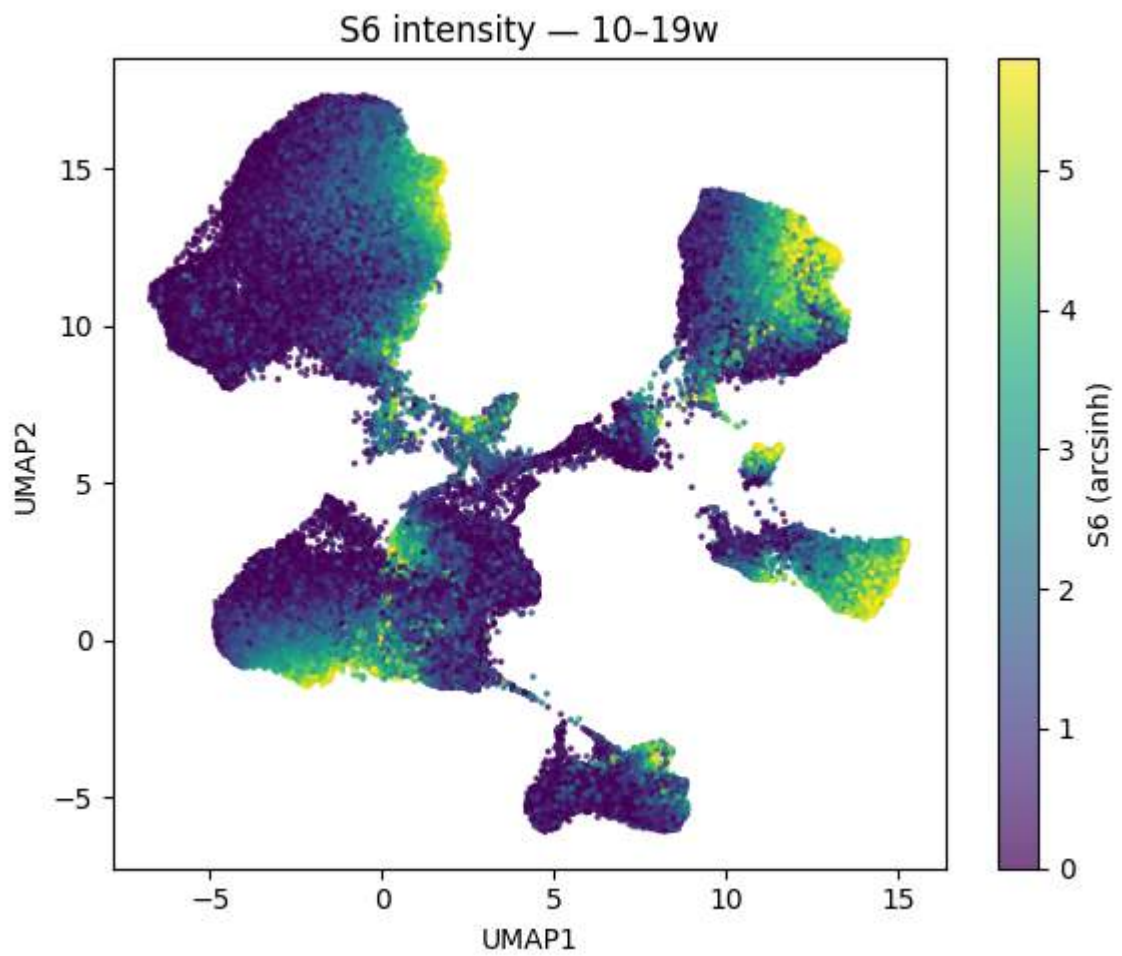
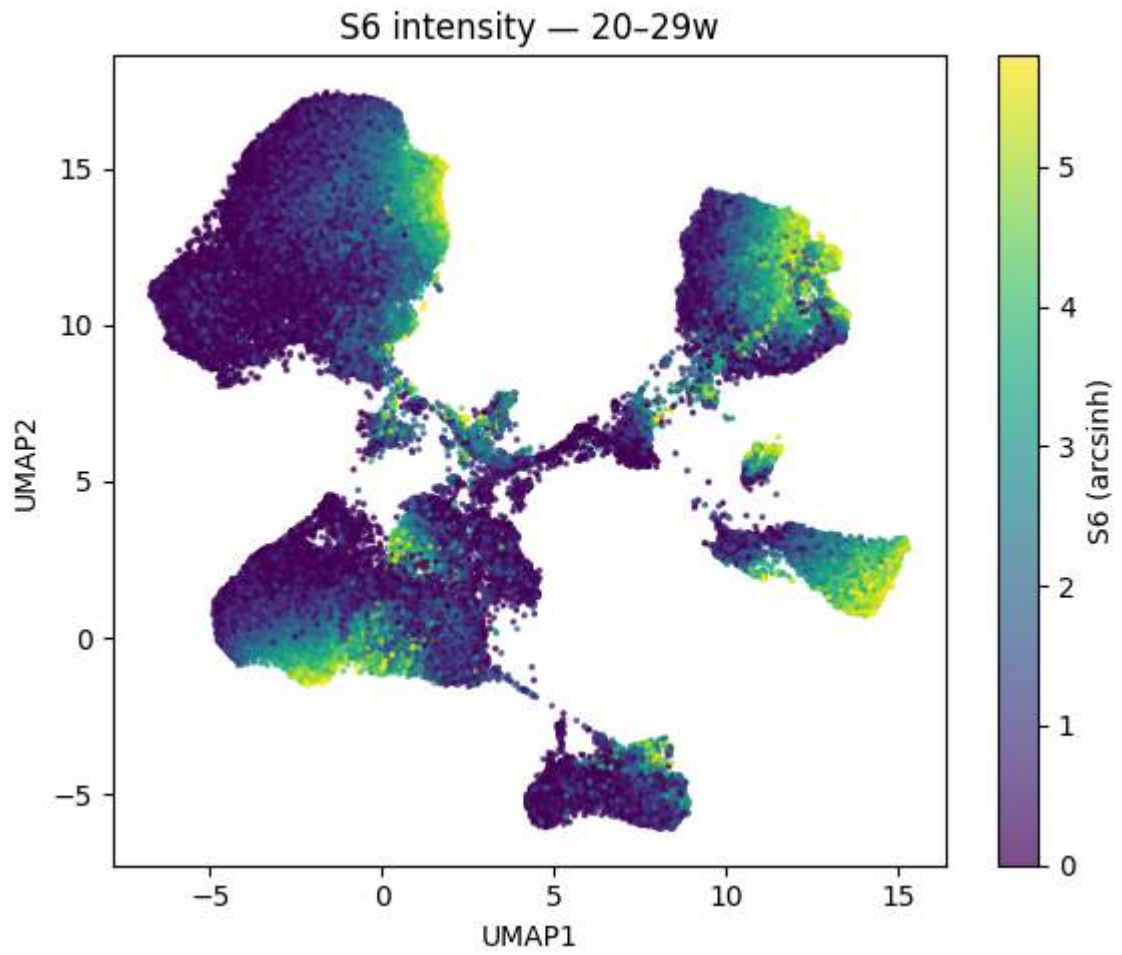


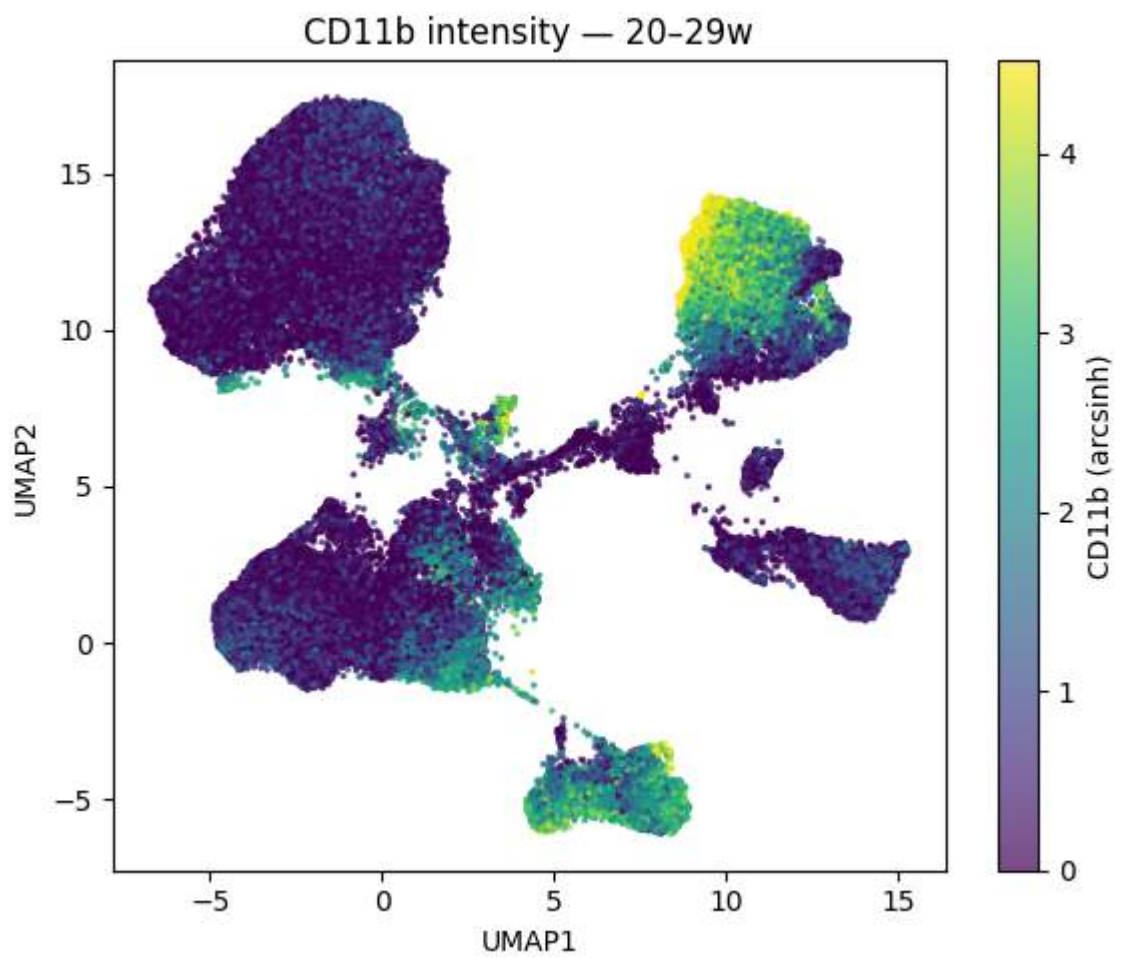
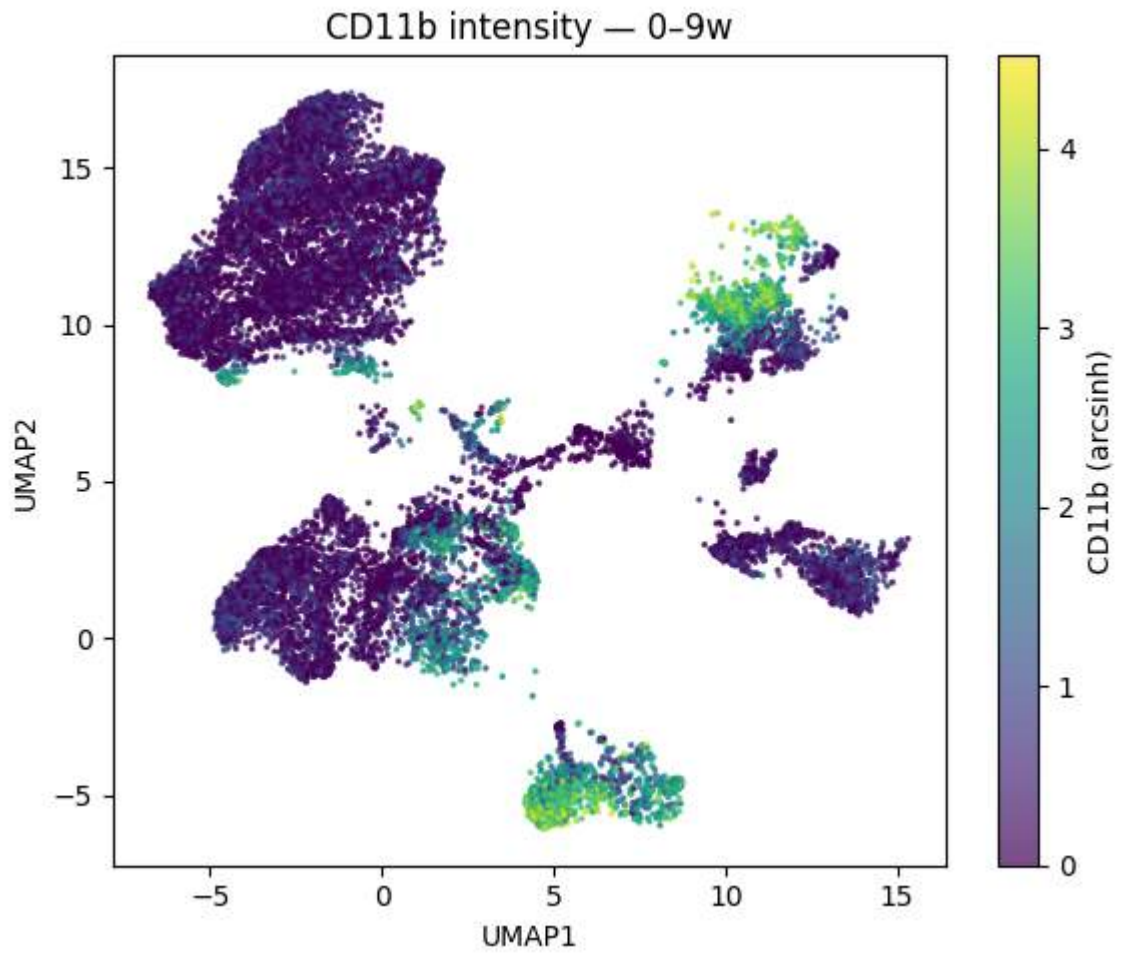


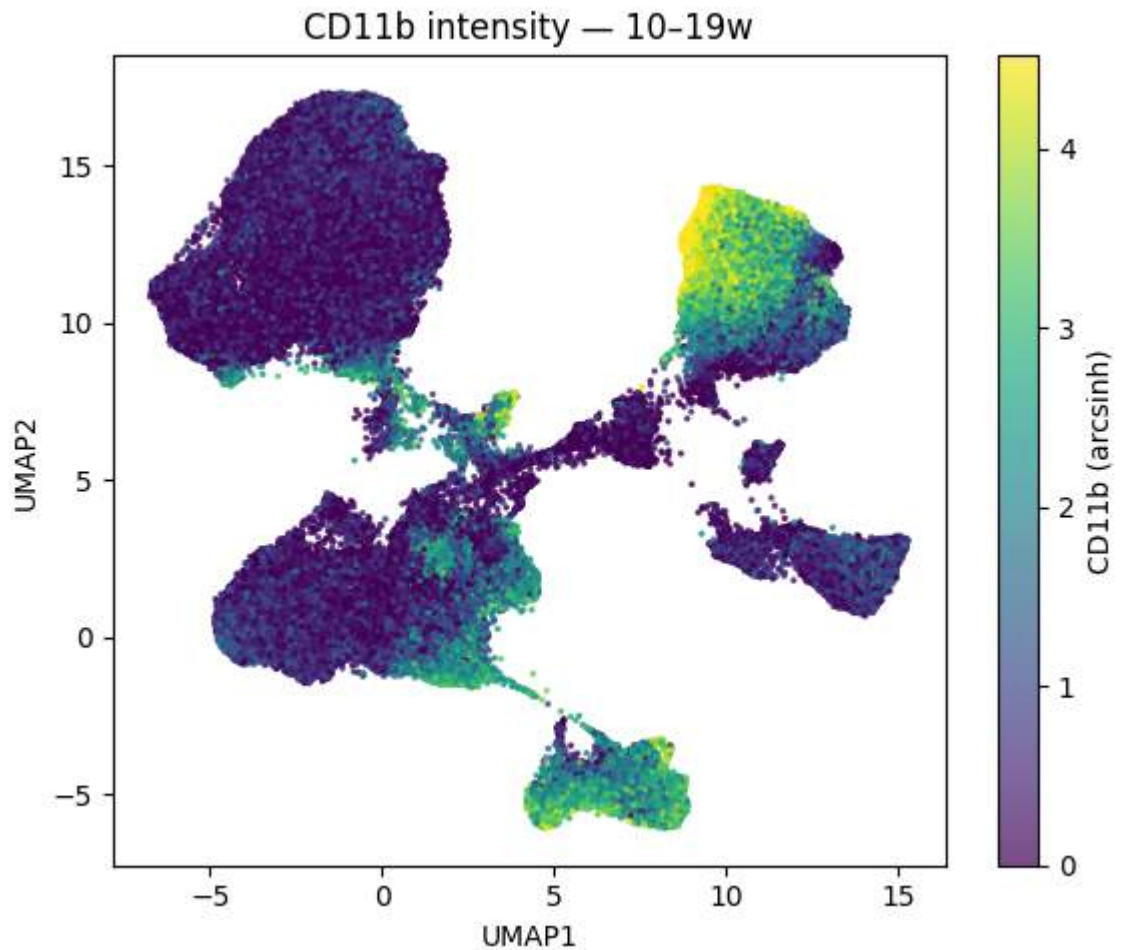












CLUSTERING

```
In [ ]: print("Shape:", eda.shape)
print("\nDtypes:\n", eda.dtypes)
print("\nMissing values per column (non-zero only):")
mis = eda.isna().sum()
print(mis[mis>0].sort_values(ascending=False))

print("\nFirst 3 rows:")
display(eda.head(3))
```


Shape: (230000, 40)

Dtypes:

CD45	float64
CD66	float64
CD7	float64
CD19	float64
CD45RA	float64
CD11b	float64
CD4	float64
CD8a	float64
CD11c	float64
CD123	float64
CREB	float64
STAT5	float64
p38	float64
TCRgd	float64
STAT1	float64
STAT3	float64
S6	float64
CD33	float64
MAPKAPK2	float64
Tbet	float64
cPARP	float64
IkB	float64
CD16	float64
NFkB	float64
GATA3	float64
pSTAT6	float64
CD25	float64
CD3	float64
pERK	float64
RORgd	float64
HLADR	float64
CD14	float64
CD56	float64
condition	int64
age	float64
age_label	int64
filename	object
_umap1	float32
_umap2	float32
age_bin_10w	category

dtype: object

Missing values per column (non-zero only):

Series([], dtype: int64)

First 3 rows:

	CD45	CD66	CD7	CD19	CD45RA	CD11b	CD4	CD8a	CD11c
0	4.381964	0.250840	2.579434	0.000000	0.226083	0.089706	0.000000	0.000000	0.0
1	4.406560	0.348977	3.768505	0.095871	3.881555	0.029366	3.742608	0.993199	0.0
2	4.468272	0.007222	3.753519	0.000000	2.186813	0.046937	3.928551	0.062450	0.0

3 rows × 40 columns

In []: *## Plot UMAP embeddings colored by a marker's intensity, generating one compar*

```
# Detect markers automatically (no hard-coding)
ALL_COLS = list(eda.columns)

# columns we consider metadata / embeddings
meta_like = {"condition", "age", "age_label", "filename", "age_bin_10w",
             "_umap1", "_umap2"}

# any PCA columns already present
pca_cols = [c for c in ALL_COLS if c.upper().startswith("PCA_")]
# any UMAP columns already present
umap_cols = [c for c in ALL_COLS if c.upper().startswith("UMAP") or c in ["_umap

# numeric columns only
num_cols = [c for c in ALL_COLS if pd.api.types.is_numeric_dtype(eda[c])]

# markers = numeric - (meta + PCA + UMAP)
markers = [c for c in num_cols if c not in meta_like and c not in pca_cols and c

print(f"Detected marker columns (n={len(markers)}):")
print(markers)

print("\nPCA columns found:", pca_cols if pca_cols else "(none)")
print("UMAP columns found:", umap_cols if umap_cols else "(none)")

# sanity: quick stats for a few markers
peek = markers[:8]
if peek:
    print("\nQuick stats (first few markers):")
    display(eda[peek].describe().T)
```

Detected marker columns (n=33):

```
['CD45', 'CD66', 'CD7', 'CD19', 'CD45RA', 'CD11b', 'CD4', 'CD8a', 'CD11c', 'CD12
3', 'CREB', 'STAT5', 'p38', 'TCRgd', 'STAT1', 'STAT3', 'S6', 'CD33', 'MAPKAPK2',
'Tbet', 'cPARP', 'IkB', 'CD16', 'NFKB', 'GATA3', 'pSTAT6', 'CD25', 'CD3', 'pERK',
'RORgd', 'HLADR', 'CD14', 'CD56']
```

PCA columns found: (none)

UMAP columns found: ['_umap1', '_umap2']

Quick stats (first few markers):

	count	mean	std	min	25%	50%	75%	max
CD45	230000.0	4.614776	0.619588	0.0	4.400033	4.702942	4.961181	6.730154
CD66	230000.0	0.192762	0.314914	0.0	0.000000	0.057072	0.271490	6.714129
CD7	230000.0	2.836035	1.772144	0.0	0.820077	3.509393	4.261615	6.619624
CD19	230000.0	0.286640	0.642180	0.0	0.000000	0.018332	0.263639	8.573904
CD45RA	230000.0	2.503099	1.609959	0.0	1.026918	2.778154	3.881788	6.883657
CD11b	230000.0	1.015058	1.240396	0.0	0.071134	0.452590	1.574878	5.963515
CD4	230000.0	1.910454	1.751032	0.0	0.096144	1.405512	3.765959	5.343919
CD8a	230000.0	1.292484	1.710771	0.0	0.047953	0.424097	1.800670	6.239594

```
In [ ]: # Setting age bin (age bins x condition)
if "age_bin_10w" in eda.columns:
    bins = eda["age_bin_10w"].astype(str).unique().tolist()
    print("Age bins present:", bins)

    ct = eda.pivot_table(index="age_bin_10w", columns="condition", values=marker)
    ct = ct.rename(columns={0:"Control",1:"PE"}).fillna(0).astype(int)
    print("\nCounts per age bin x condition:")
    display(ct)
else:
    print("Column 'age_bin_10w' not found.")
```

Age bins present: ['0-9w', '20-29w', '10-19w']

Counts per age bin x condition:

```
/tmp/ipython-input-1810127174.py:6: FutureWarning: The default value of observed=
False is deprecated and will change to observed=True in a future version of panda
s. Specify observed=False to silence this warning and retain the current behavior
ct = eda.pivot_table(index="age_bin_10w", columns="condition", values=markers
[0], aggfunc="count")
```

	condition	Control	PE
age_bin_10w			
0-9w		10000	5000
10-19w		60000	65000
20-29w		50000	40000

How can HDBSCAN, K-means, and PhenoGraph be integrated?

```
In [ ]: eda = pd.read_csv("/content/drive/MyDrive/pre eclampsia/eda.csv")
print("Reloaded eda:", eda.shape)
```

Reloaded eda: (230000, 37)

PCA Pre processing

K MEANS

Defining K through Silhouette and Gap Statistic

Silhouette and Gap Analysis to select the optimal number of clusters

```

In [ ]: ## Prepare features and reproducible helpers, then use the Gap statistic to sel
# paths
output_dir = "/content/drive/MyDrive/preeclampsia_results"
Path(output_dir).mkdir(parents=True, exist_ok=True)

# data columns
FEATURES = ['CD45', 'CD66', 'CD7', 'CD19', 'CD45RA', 'CD11b', 'CD4', 'CD8a', 'CD11c', 'CD
            'CREB', 'STAT5', 'p38', 'TCRgd', 'STAT1', 'STAT3', 'S6', 'CD33', 'MAPKAPK2',
            'cPARP', 'Ikb', 'CD16', 'NFkB', 'GATA3', 'pSTAT6', 'CD25', 'CD3', 'pERK', 'RC
            'HLADR', 'CD14', 'CD56']

COND_NAME = {0:"Control", 1:"Pre-Eclampsia"}

# global scoring params (adjustable)
K_MIN, K_MAX = 2, 20          # k sweep range
SUBSAMPLE_N = 5000            # max cells per subset for scoring (keeps runtime
GAP_B = 20                    # # of reference datasets for Gap statistic
RANDOM_STATE = 42

rng = np.random.default_rng(RANDOM_STATE)

def subsample_rows(X, nmax=SUBSAMPLE_N, seed=RANDOM_STATE):
    n = len(X)
    if n <= nmax:
        return X
    idx = rng.choice(n, size=nmax, replace=False)
    return X[idx]

def _within_ss_from_kmeans(model, X):
    # KMeans inertia_ is the within-cluster sum of squares W_k
    return float(model.inertia_)

def gap_statistic(X, k_range, B=GAP_B, random_state=RANDOM_STATE):
    """
    Tibshirani et al. (2001): Gap(k) = E_b[log(W_k^b)] - log(W_k),
    where W_k is within-cluster dispersion (use KMeans inertia).
    Reference datasets: uniform in bounding box of X.
    Returns: DataFrame with columns [k, Wk, logWk, gap, sk] and chosen_k by stan
            first k such that Gap(k) >= Gap(k+1) - s_{k+1}.
    """
    X = np.asarray(X, dtype=np.float32)
    n, d = X.shape
    mins = X.min(axis=0)
    maxs = X.max(axis=0)

    rows = []
    for k in k_range:
        # real data
        km = KMeans(n_clusters=k, random_state=random_state, n_init=20).fit(X)
        Wk = _within_ss_from_kmeans(km, X)
        logWk = np.log(Wk + 1e-12)

        # reference
        logWk_b = []
        for b in range(B):

```

```

        U = rng.uniform(mins, maxs, size=(n, d))
        kmb = KMeans(n_clusters=k, random_state=random_state + b + k, n_init
            Wkb = _within_ss_from_kmeans(kmb, U)
            logWk_b.append(np.log(Wkb + 1e-12))

    logWk_b = np.array(logWk_b, dtype=np.float64)
    gap = logWk_b.mean() - logWk
    sk = np.sqrt(1 + 1.0/B) * logWk_b.std(ddof=1)
    rows.append((k, Wk, logWk, gap, sk))

df = pd.DataFrame(rows, columns=["k", "Wk", "logWk", "gap", "sk"])
# Gap rule: first k such that gap(k) >= gap(k+1) - s(k+1)
chosen = None
for i in range(len(df)-1):
    if df.loc[i, "gap"] >= (df.loc[i+1, "gap"] - df.loc[i+1, "sk"]):
        chosen = int(df.loc[i, "k"]); break
if chosen is None:
    chosen = int(df.loc[df["gap"].idxmax(), "k"])
return df, chosen

```

Compute silhouette scores across a range of k using KMeans to identify the cluster count with the highest average separation.

```

In [ ]: def silhouette_curve(X, k_range):
        X = np.asarray(X, dtype=np.float32)
        scores = []
        for k in k_range:
            if k >= len(X):
                break
            labels = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=20).fit_
            if len(np.unique(labels)) < 2:
                scores.append(np.nan)
            else:
                scores.append(silhouette_score(X, labels))
        ks = list(k_range)[:len(scores)]
        df = pd.DataFrame({"k": ks, "silhouette": scores})
        best_k = int(df.loc[df["silhouette"].idxmax(), "k"]) if df["silhouette"].not
        return df, best_k

```

```

In [ ]: # sweep k to compute and save silhouette and gap curves (with plots), then sugge
def choose_k_by_gap_and_silhouette(X, title, plot_prefix):
    # subsample for speed
    Xs = subsample_rows(X, SUBSAMPLE_N)
    k_range = range(K_MIN, min(K_MAX, len(Xs)-1) + 1)
    if len(k_range) < 1:
        return None, None, None

    # silhouette
    sil_df, sil_k = silhouette_curve(Xs, k_range)
    sil_csv = f"{plot_prefix}_silhouette.csv"
    sil_df.to_csv(sil_csv, index=False)

    plt.figure(figsize=(7,5))
    plt.plot(sil_df["k"], sil_df["silhouette"], marker="o")
    plt.title(f"Silhouette vs k - {title}")
    plt.xlabel("k"); plt.ylabel("Silhouette"); plt.grid(True)
    sil_png = f"{plot_prefix}_silhouette.png"
    plt.tight_layout(); plt.savefig(sil_png, dpi=150); plt.show(); plt.close()

```

```

# gap
gap_df, gap_k = gap_statistic(Xs, k_range, B=GAP_B, random_state=RANDOM_STAT
gap_csv = f"{plot_prefix}_gap.csv"
gap_df.to_csv(gap_csv, index=False)

plt.figure(figsize=(7,5))
plt.plot(gap_df["k"], gap_df["gap"], marker="o")
plt.title(f"Gap statistic vs k - {title}")
plt.xlabel("k"); plt.ylabel("Gap"); plt.grid(True)
gap_png = f"{plot_prefix}_gap.png"
plt.tight_layout(); plt.savefig(gap_png, dpi=150); plt.show(); plt.close()

# final suggestion: prefer Gap rule; if None, fallback to silhouette
final_k = gap_k if gap_k is not None else sil_k
return final_k, (sil_k, sil_csv, sil_png), (gap_k, gap_csv, gap_png)

```

```

In [ ]: # Ensuring age bins fall within 10s
# Determine bin edges (1-10, 11-20, ...)
max_age = int(eda['age'].max())
bins = np.arange(0, max_age + 10, 10, dtype=int)

# Create neat labels like "1-10w", "11-20w", ...
labels = [f"{start+1}-{end}w" for start, end in zip(bins[:-1], bins[1:])]

# Create the categorical age bins
eda['age_bin_10w'] = pd.cut(
    eda['age'],
    bins=bins,
    labels=labels,
    include_lowest=True,
    right=True
)

# Ensure categorical dtype
eda['age_bin_10w'] = eda['age_bin_10w'].astype('category')

print(" Created clean integer-based age_bin_10w:")
print(eda['age_bin_10w'].value_counts().sort_index())

```

✓ Created clean integer-based age_bin_10w:

```

age_bin_10w
1-10w      20000
11-20w     120000
21-30w      90000
Name: count, dtype: int64

```

Standardizing with 20PCs (Step needed for noise and dimension reduction)

```

In [ ]: # PCA(20) cache
for interval in eda['age_bin_10w'].cat.categories:
    for cond in [0,1]:
        sub = eda[(eda['age_bin_10w']==interval) & (eda['condition']==cond)]
        if sub.empty:
            continue
        tag = f"{interval}_{cond}"
        pca_path = os.path.join(output_dir, f"pca20_{tag}.npy")
        if os.path.exists(pca_path):
            print("PCA20 cached:", tag);
            continue

```

```
X = sub[FEATURES].to_numpy(dtype="float32")
X = StandardScaler().fit_transform(X)
X_pca20 = PCA(n_components=20, random_state=RANDOM_STATE).fit_transform(
    np.save(pca_path, X_pca20)
print("PCA20 saved:", tag, X_pca20.shape)
```

```
PCA20 cached: 1-10w_0
PCA20 cached: 1-10w_1
PCA20 cached: 11-20w_0
PCA20 cached: 11-20w_1
PCA20 cached: 21-30w_0
PCA20 cached: 21-30w_1
```

```
In [ ]: print(eda.columns.tolist())
```

```
['CD45', 'CD66', 'CD7', 'CD19', 'CD45RA', 'CD11b', 'CD4', 'CD8a', 'CD11c', 'CD123', 'CREB', 'STAT5', 'p38', 'TCRgd', 'STAT1', 'STAT3', 'S6', 'CD33', 'MAPKAPK2', 'Tbet', 'cPARP', 'IkB', 'CD16', 'NFKB', 'GATA3', 'pSTAT6', 'CD25', 'CD3', 'pERK', 'RORgd', 'HLADR', 'CD14', 'CD56', 'condition', 'age', 'age_label', 'filename', 'age_bin_10w']
```

Pipeline 1: Clustering in PCA Space and Visualizing in UMAP 2D

```
In [ ]: # Pipeline 1: k selection (silhouette+gap) - KMeans clustered in PCA20 - UMAP-2D
summary_rows = []

for interval in eda['age_bin_10w'].cat.categories:
    for cond in [0,1]:
        sub = eda[(eda['age_bin_10w']==interval) & (eda['condition']==cond)]
        if sub.empty:
            continue

        tag = f"{interval}_{cond}"
        X_pca20 = np.load(os.path.join(output_dir, f"pca20_{tag}.npy"))
        if len(X_pca20) < 5:
            print(f"[P1] Too few rows for {tag}")
            continue

        title = f"{interval} - {COND_NAME[cond]} (PCA space)"
        prefix = os.path.join(output_dir, f"p1_{interval}_cond{cond}")

        k_star, sil_info, gap_info = choose_k_by_gap_and_silhouette(X_pca20, title)
        sil_k, sil_csv, sil_png = sil_info
        gap_k, gap_csv, gap_png = gap_info

        if (k_star is None) or (k_star >= len(X_pca20)) or (k_star < 2):
            print(f"[P1] No valid k for {tag}");
            continue

        # final clustering in PCA20
        kmeans = KMeans(n_clusters=k_star, random_state=RANDOM_STATE, n_init=20)
        labels = kmeans.labels_
        pd.Series(labels, index=sub.index, name="kmeans_pipeline1_pca20")\
            .to_csv(os.path.join(output_dir, f"labels_kmeans_pipeline1_{tag}.csv"))

        # UMAP-2D (viz only)
        emb2 = umap.UMAP(n_components=2, n_neighbors=30, min_dist=0.3, random_state=RANDOM_STATE)\
            .fit_transform(X_pca20)
        np.save(os.path.join(output_dir, f"umap2_pipeline1_{tag}.npy"), emb2)
```

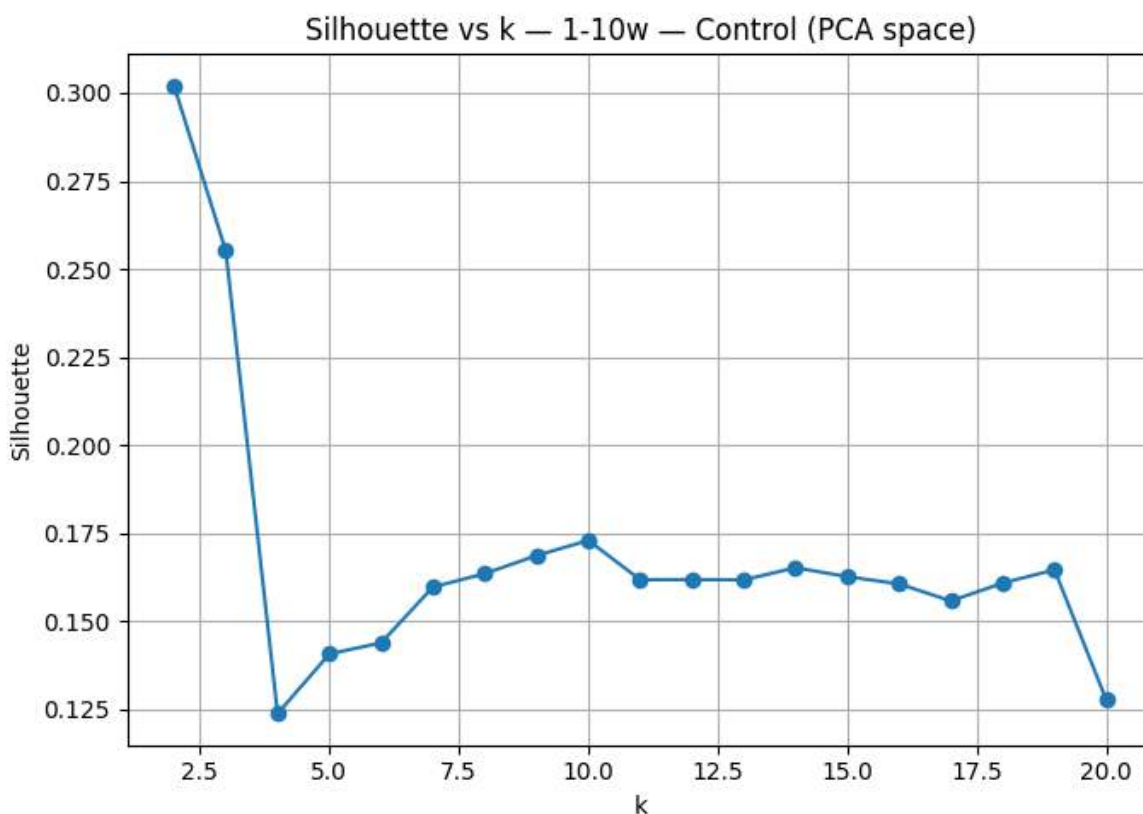
```

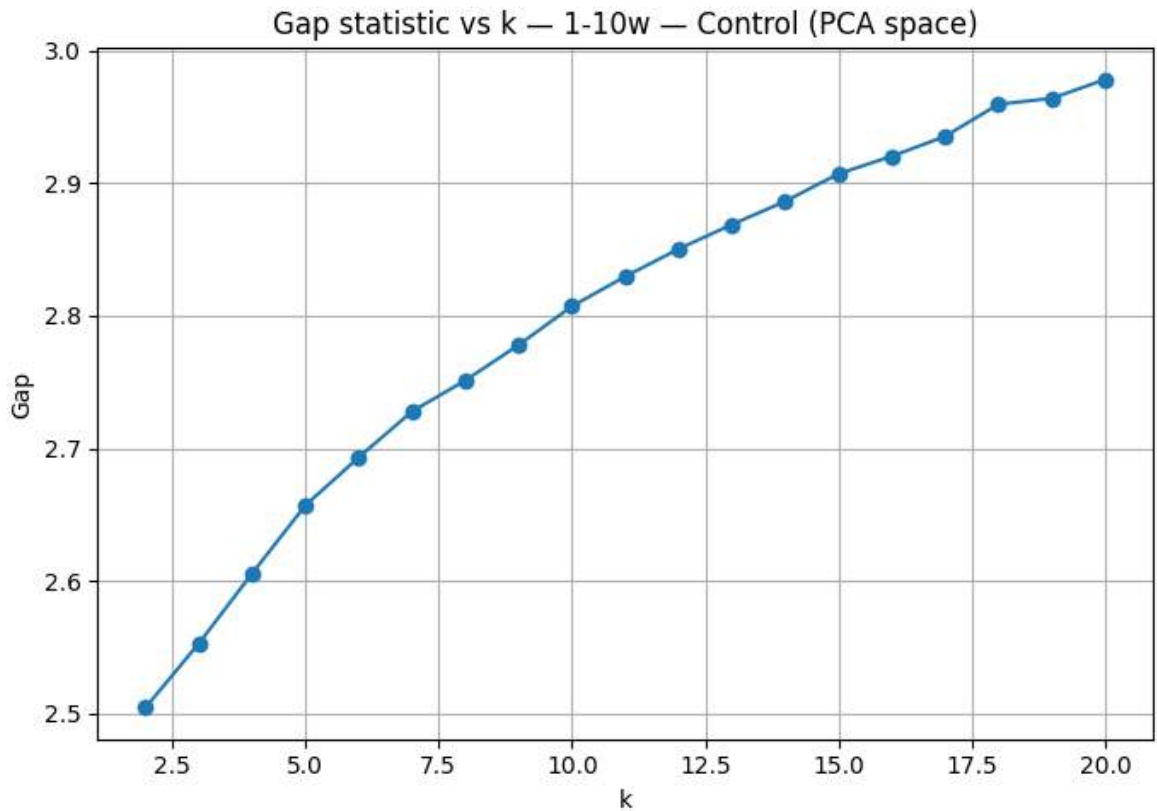
# plot
plt.figure(figsize=(8,7))
sc = plt.scatter(emb2[:,0], emb2[:,1], c=labels, s=6)
plt.title(f"P1: KMeans (PCA-20) - UMAP-2D viz\n{interval} - {COND_NAME[c]}")
plt.xlabel("UMAP1"); plt.ylabel("UMAP2")
uniq = np.unique(labels)
handles = [plt.Line2D([0],[0], marker='o', linestyle='', markersize=8,
                    markerfacecolor=sc.cmap(sc.norm(1)), markeredgcolor='none',
                    label=f"C{int(l)}") for l in uniq]
plt.legend(handles=handles, title="Clusters", loc="best", frameon=True)
out_png = os.path.join(output_dir, f"umap2_plot_kmeans_pipeline1_{tag}").
plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.show();

# record metrics
s_score = silhouette_score(subsample_rows(X_pca20), labels[:len(subsample_rows)])
summary_rows.append([str(interval), cond, "Pipeline1(PCA20)", k_star, s_score,
                    os.path.basename(sil_csv), os.path.basename(gap_csv),
                    os.path.basename(sil_png), os.path.basename(gap_png)])

p1_summary = pd.DataFrame(summary_rows, columns=[
    "interval", "condition", "pipeline", "k_final", "k_sil", "k_gap", "silhouette_at_k",
    "silhouette_csv", "gap_csv", "silhouette_png", "gap_png", "umap_plot_png"
])
p1_summary.to_csv(os.path.join(output_dir, "kmeans_p1_summary.csv"), index=False)
p1_summary

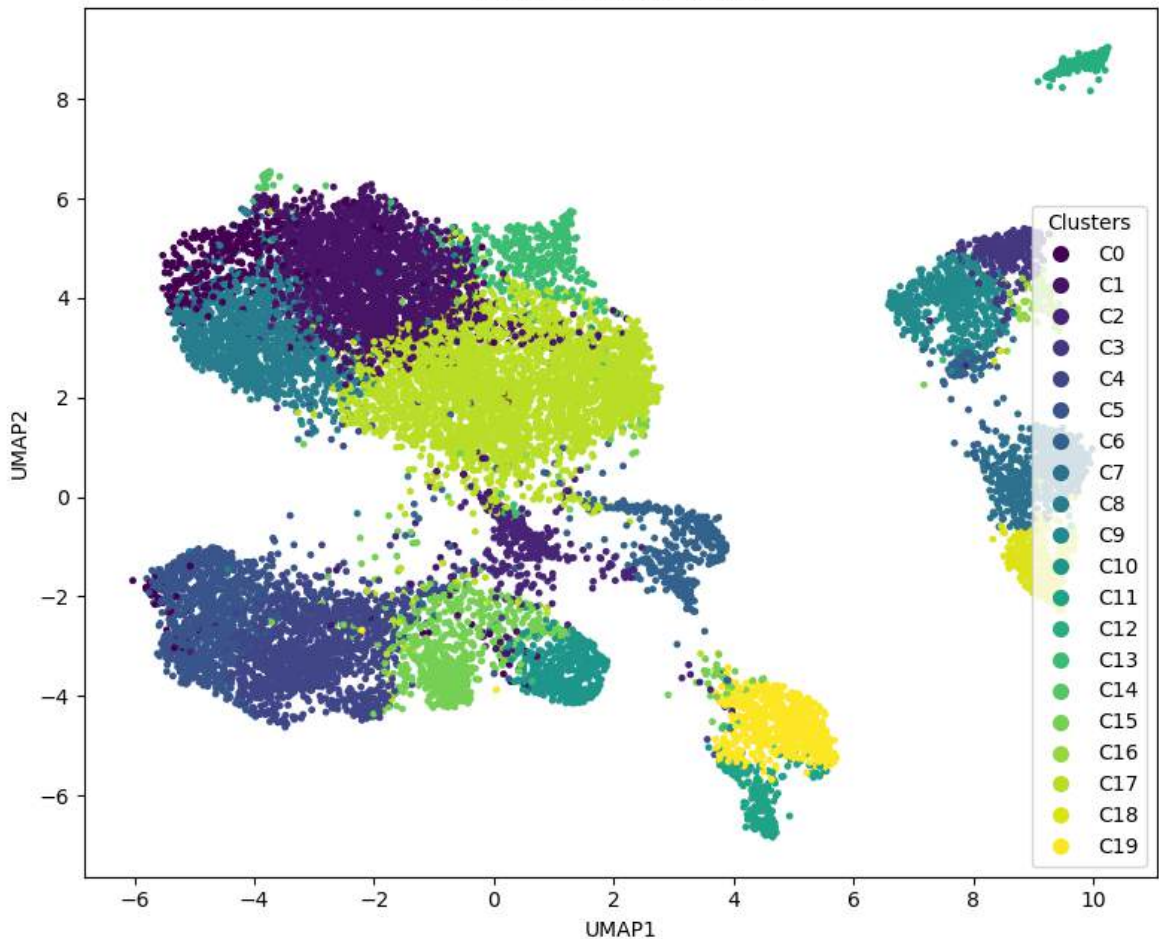
```

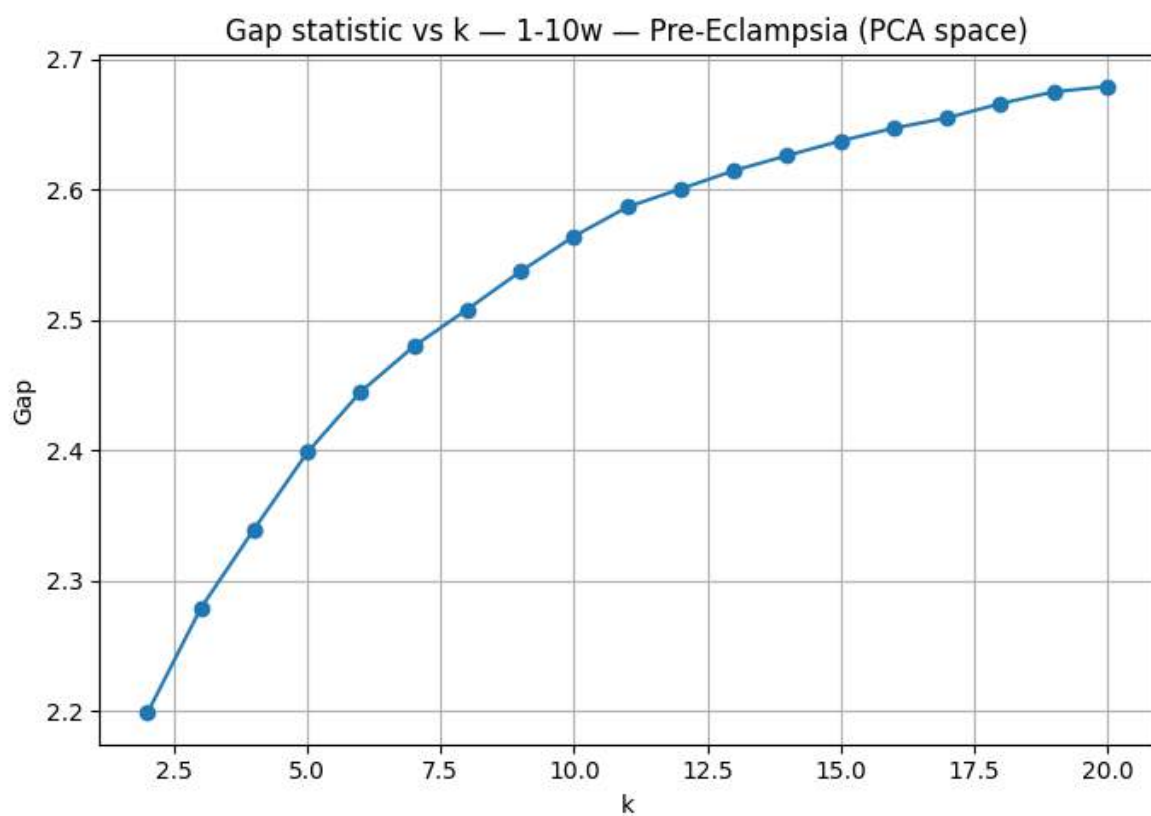
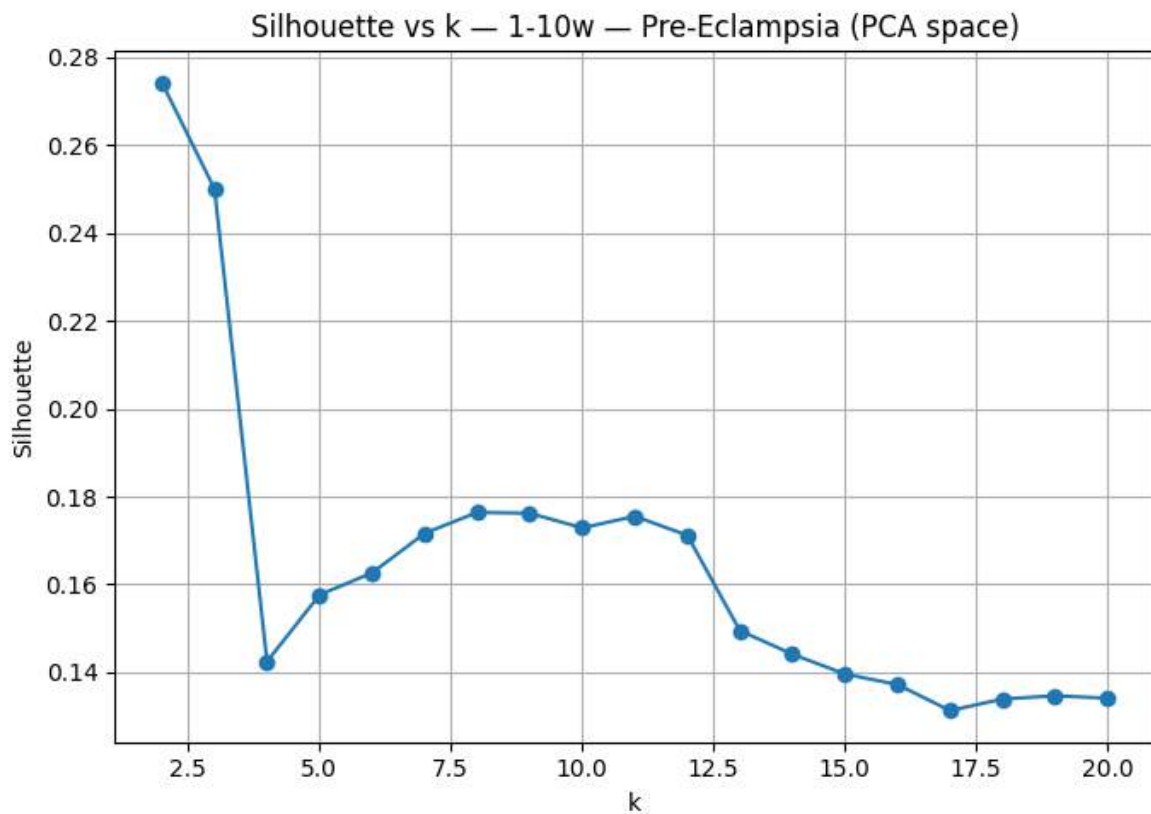




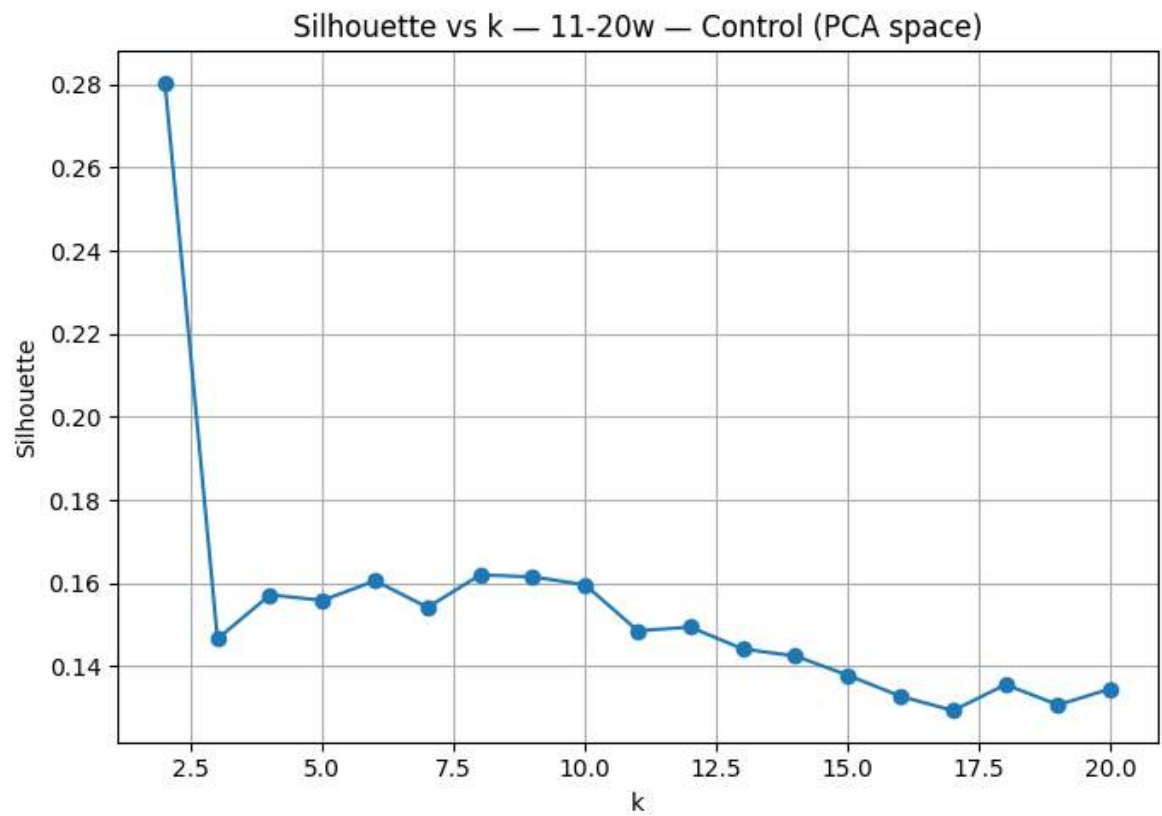
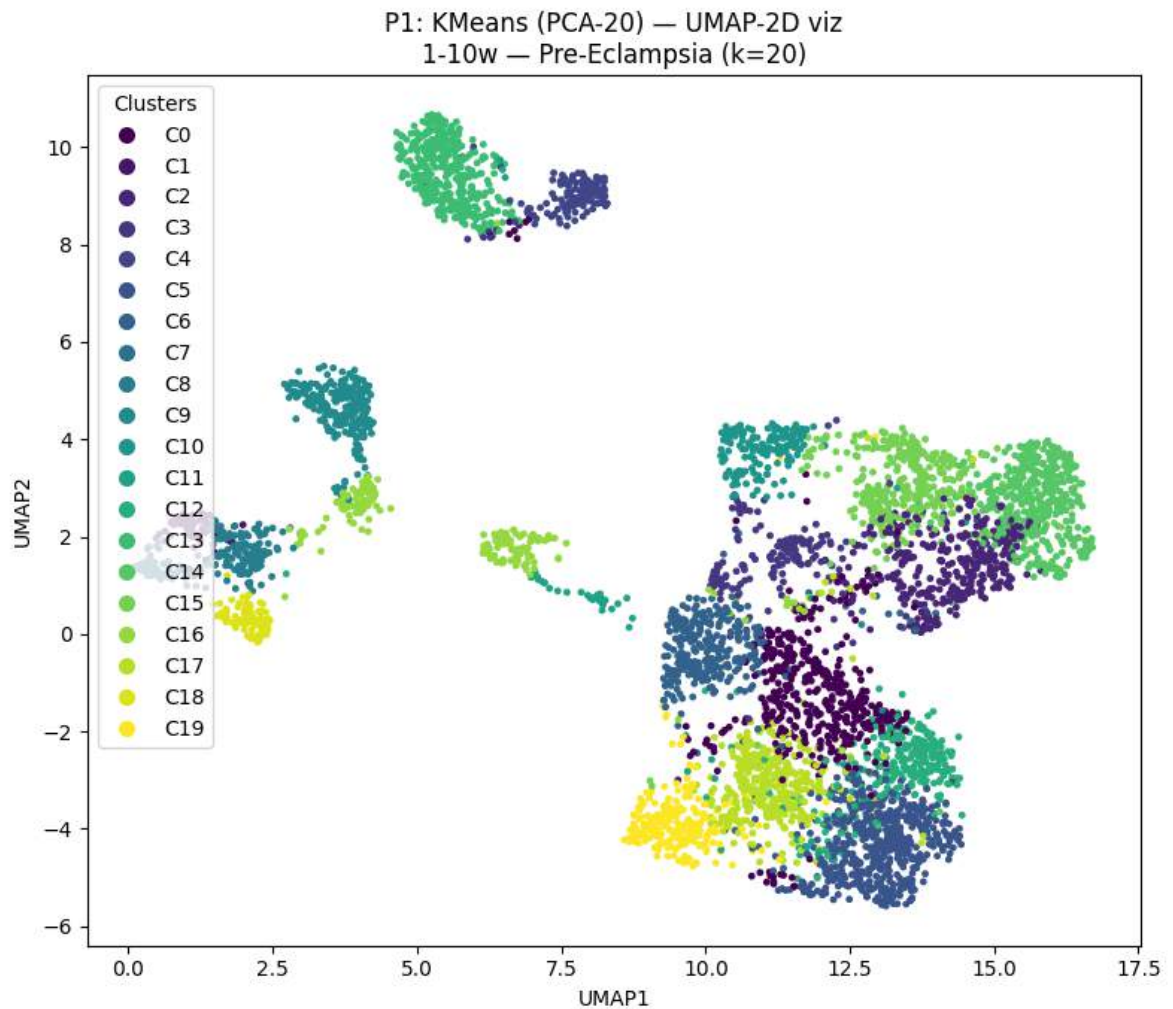
```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v  
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.  
warn(
```

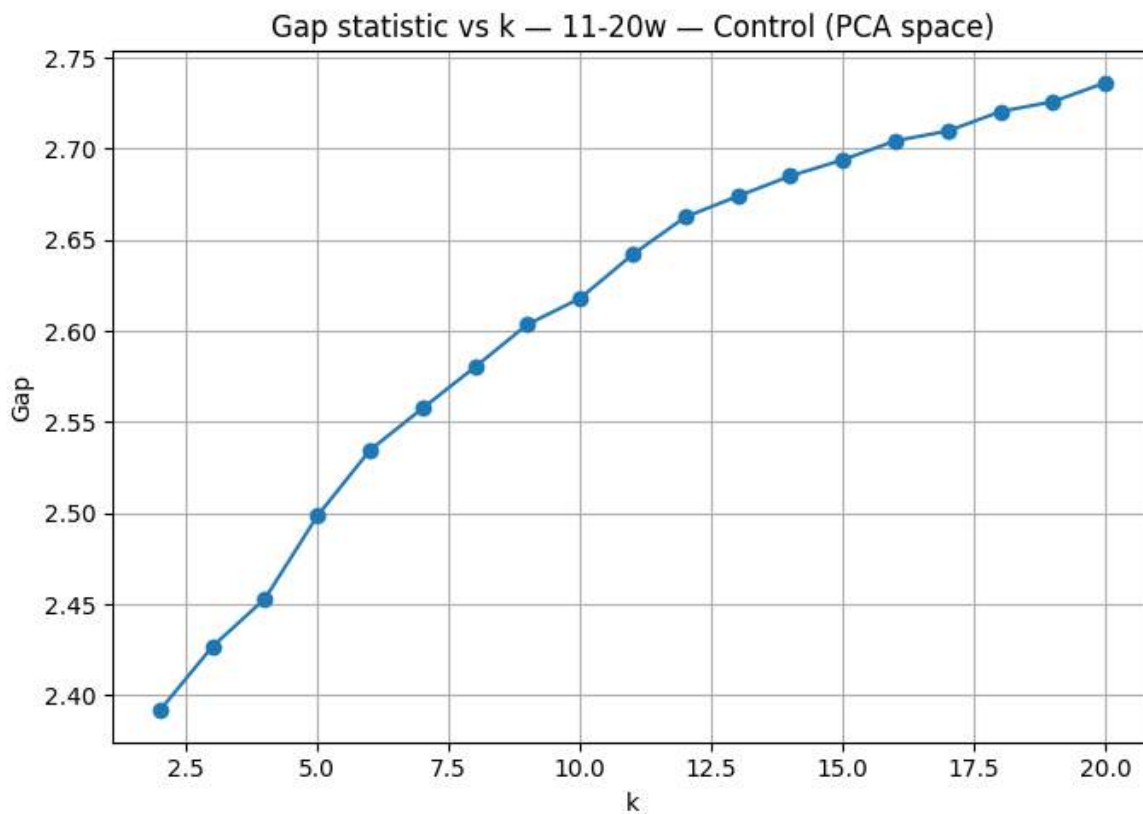
P1: KMeans (PCA-20) — UMAP-2D viz
1-10w — Control (k=20)





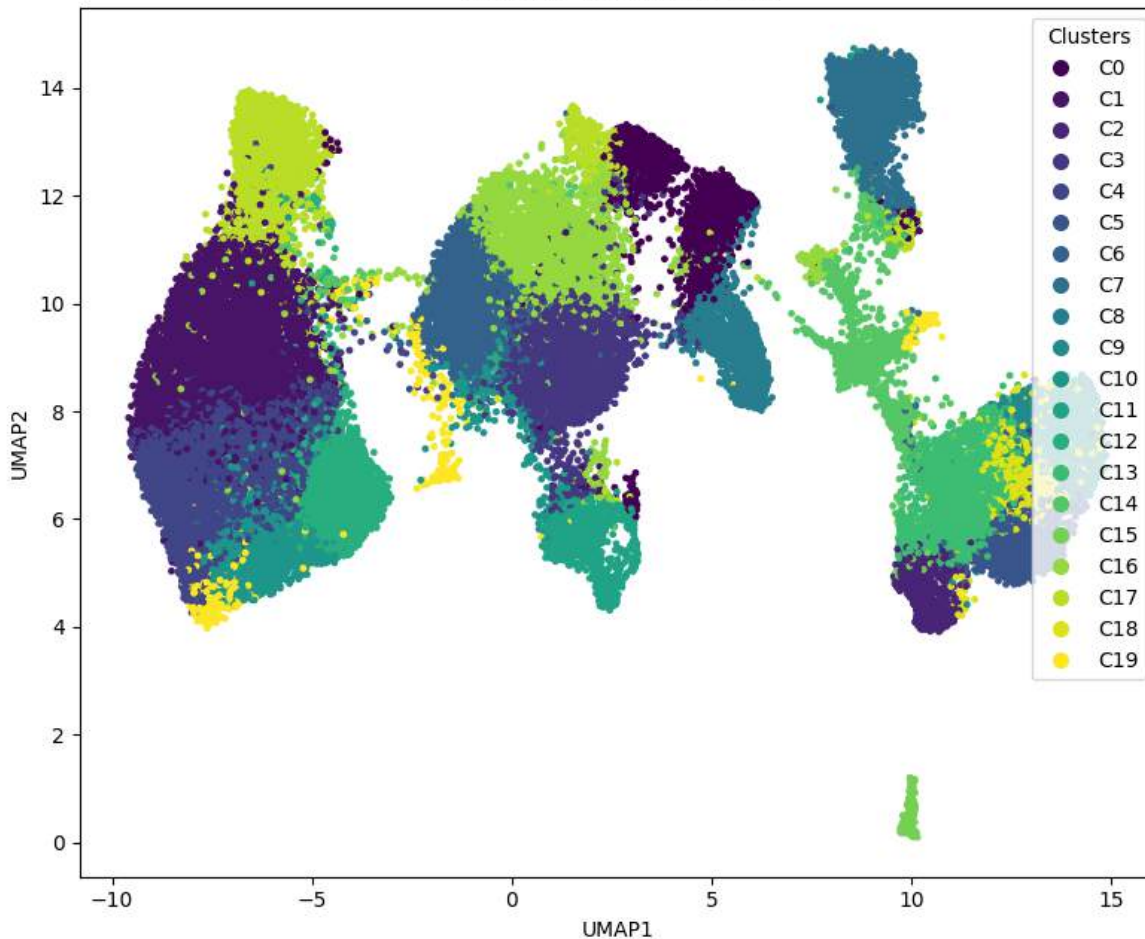
```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v  
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.  
warn(
```

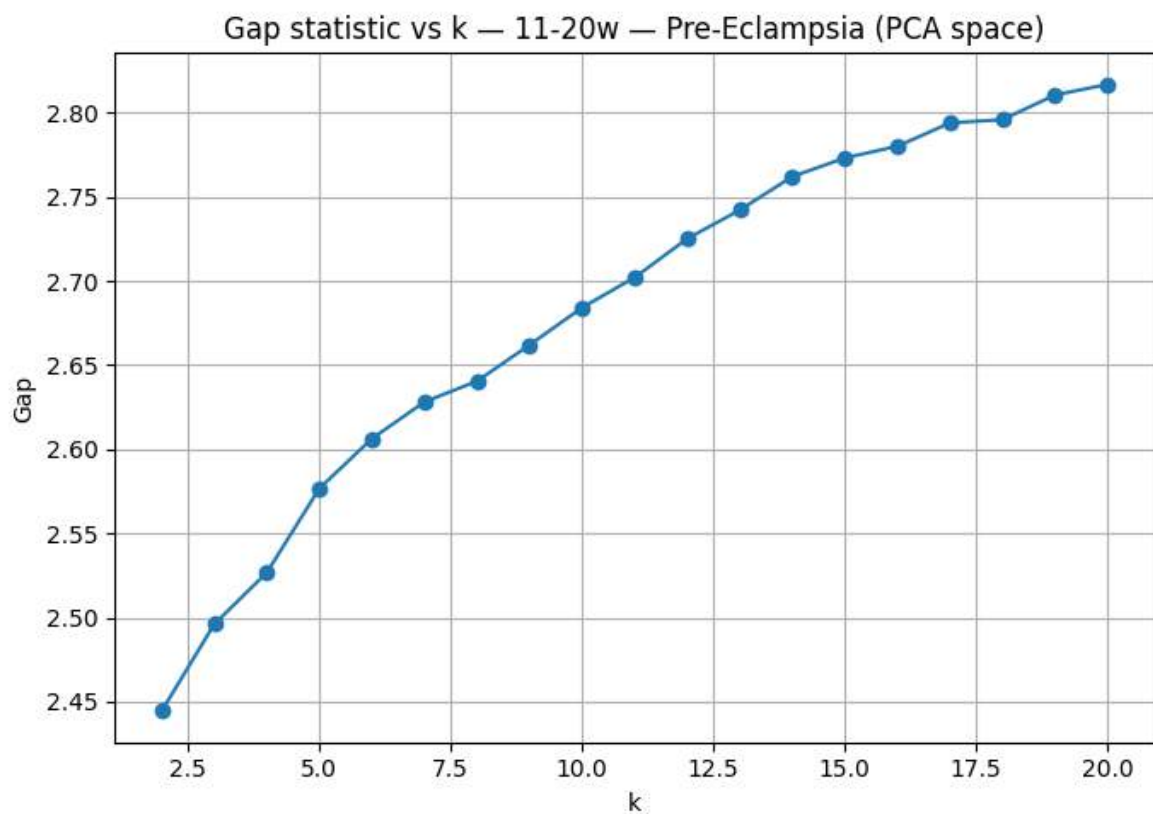
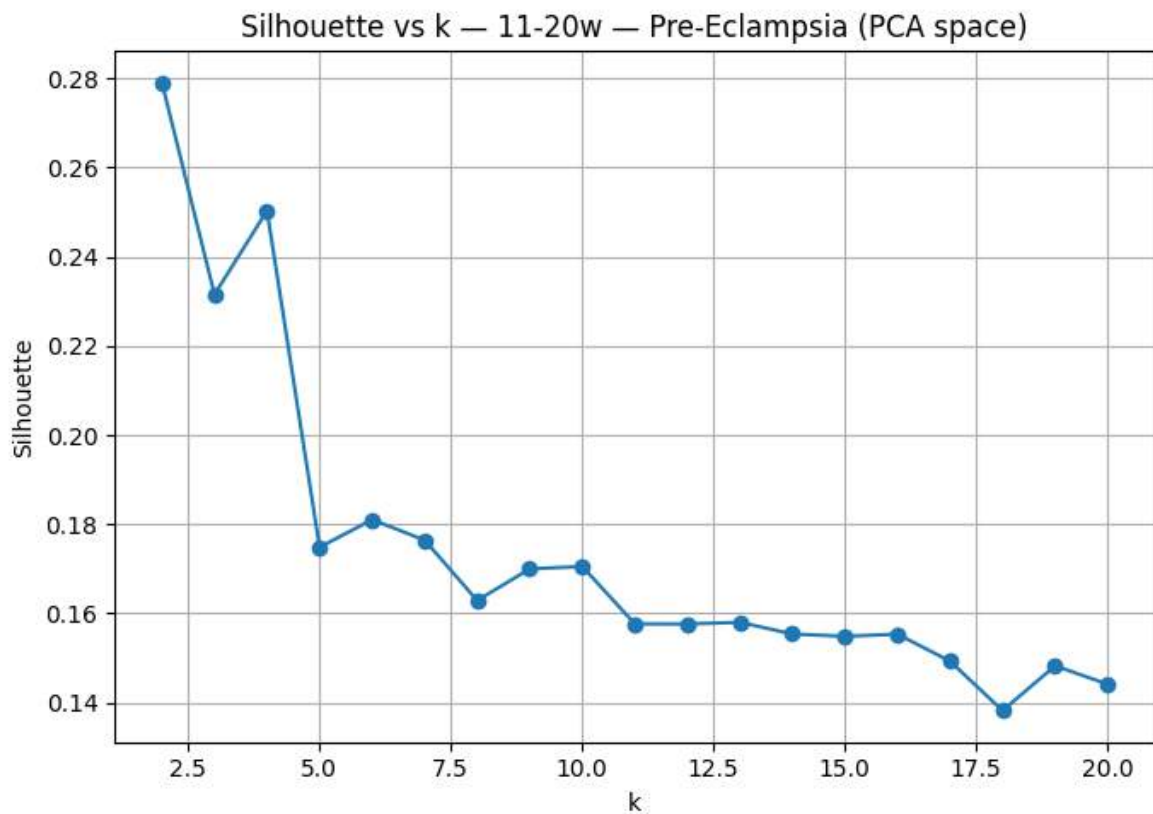




```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v  
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.  
warn(
```

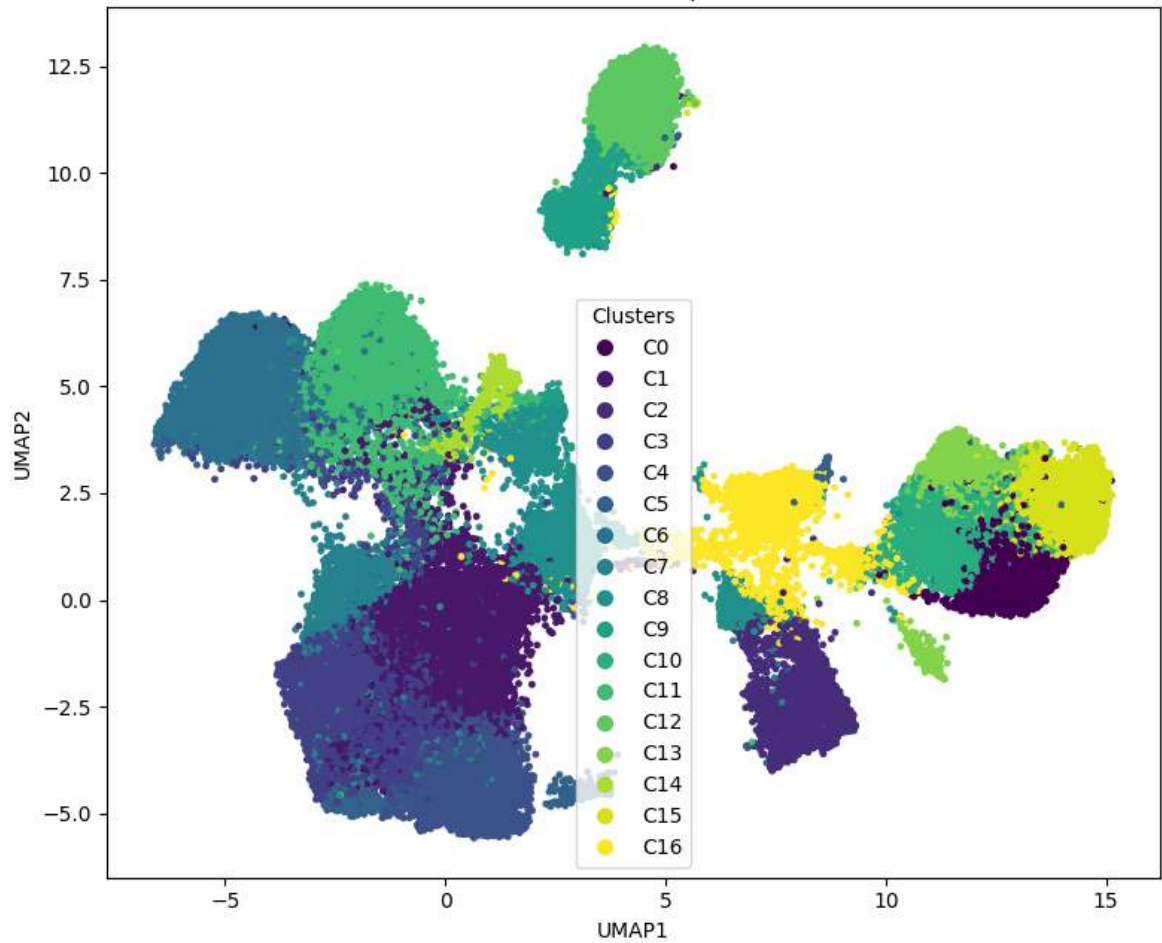
P1: KMeans (PCA-20) — UMAP-2D viz
11-20w — Control (k=20)



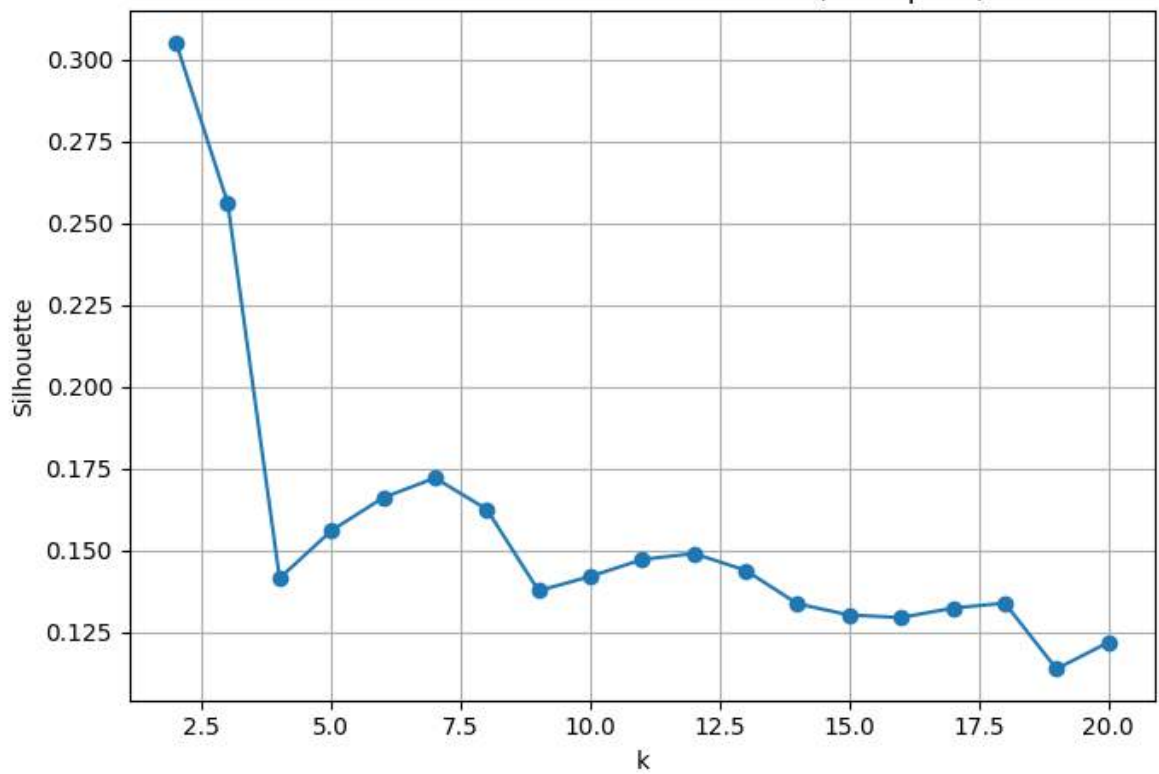


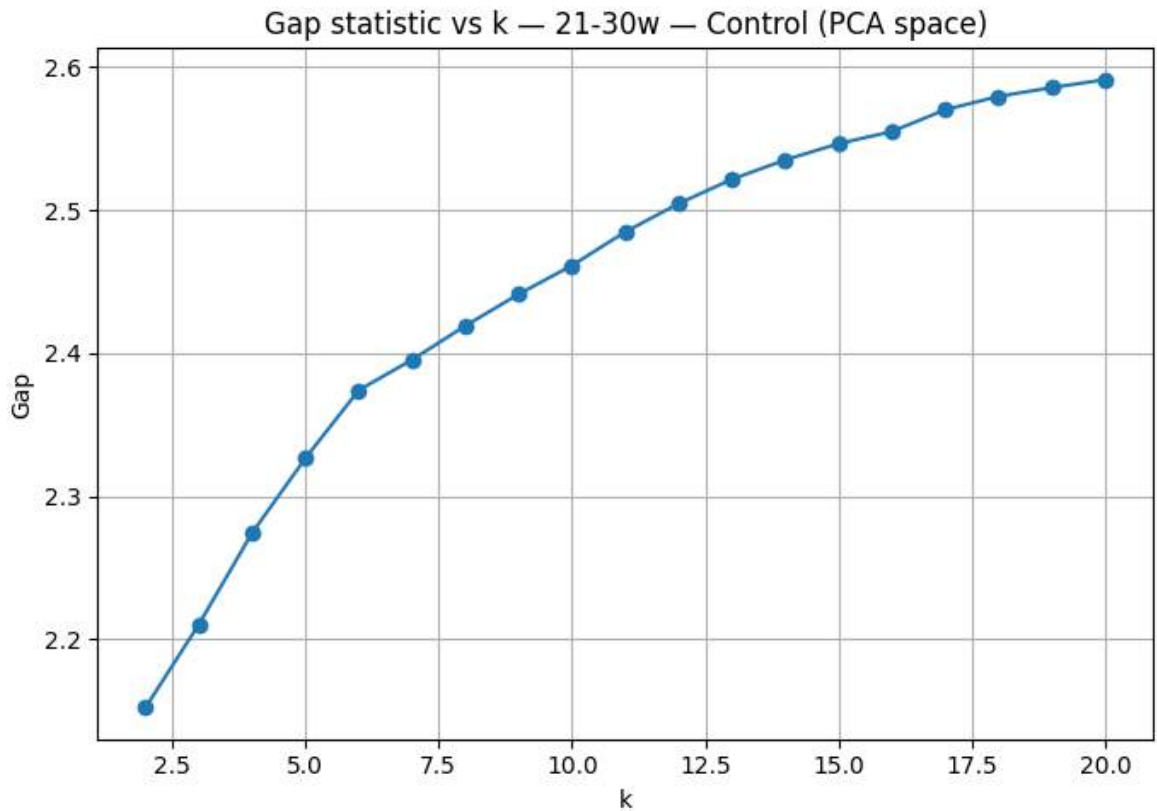
```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v  
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.  
warn(
```


P1: KMeans (PCA-20) — UMAP-2D viz
11-20w — Pre-Eclampsia (k=17)



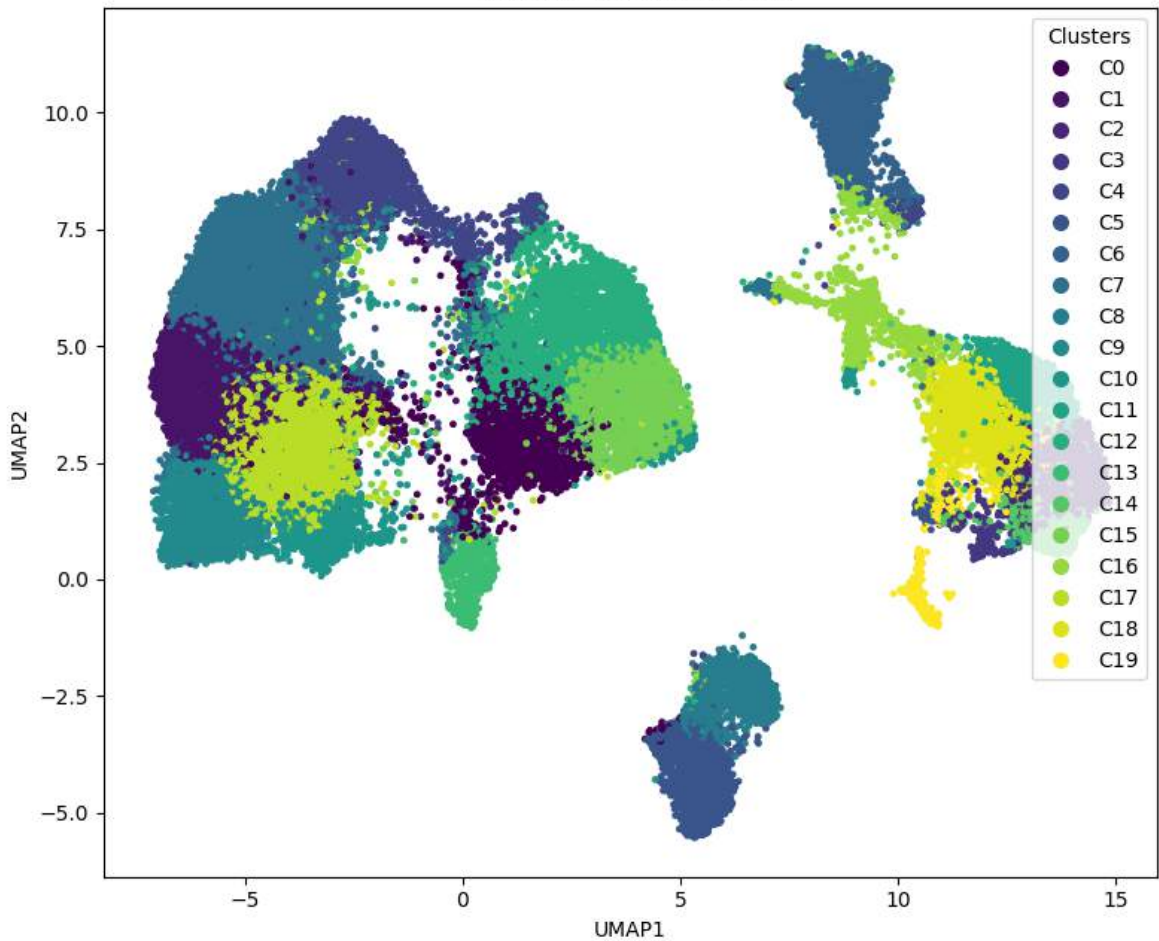
Silhouette vs k — 21-30w — Control (PCA space)

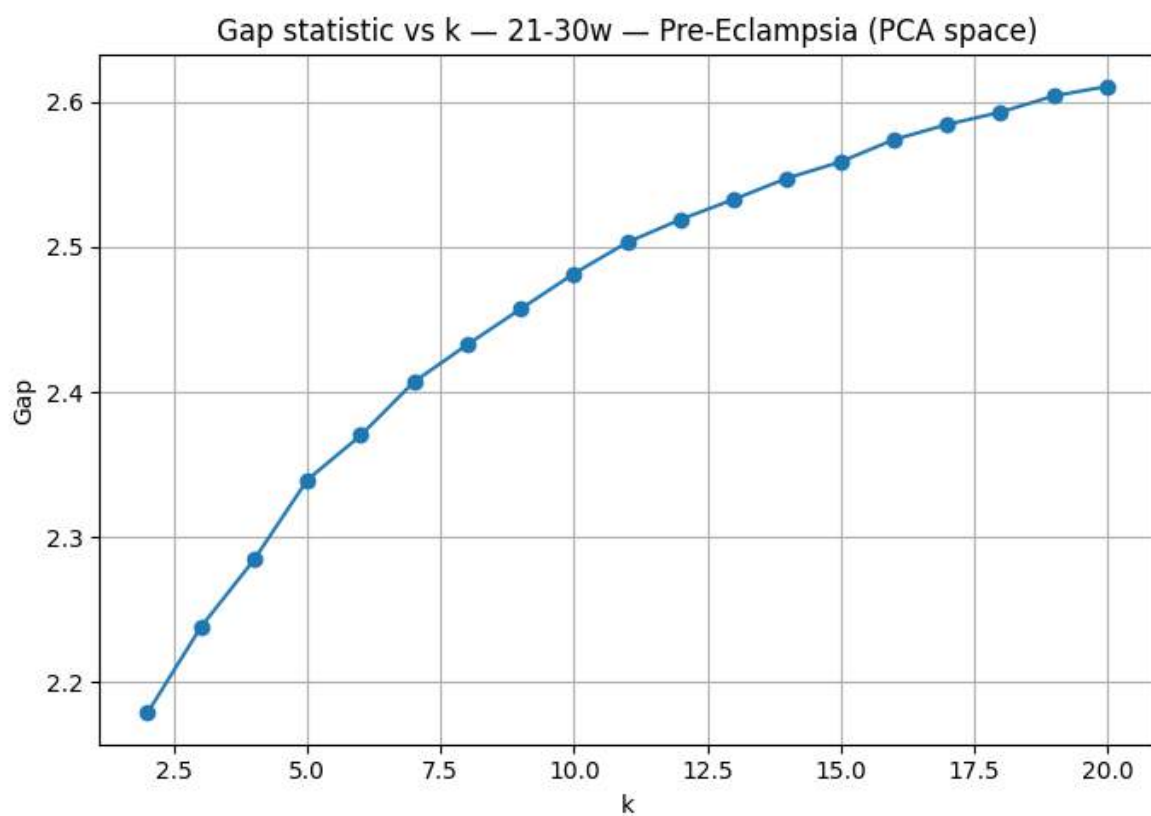
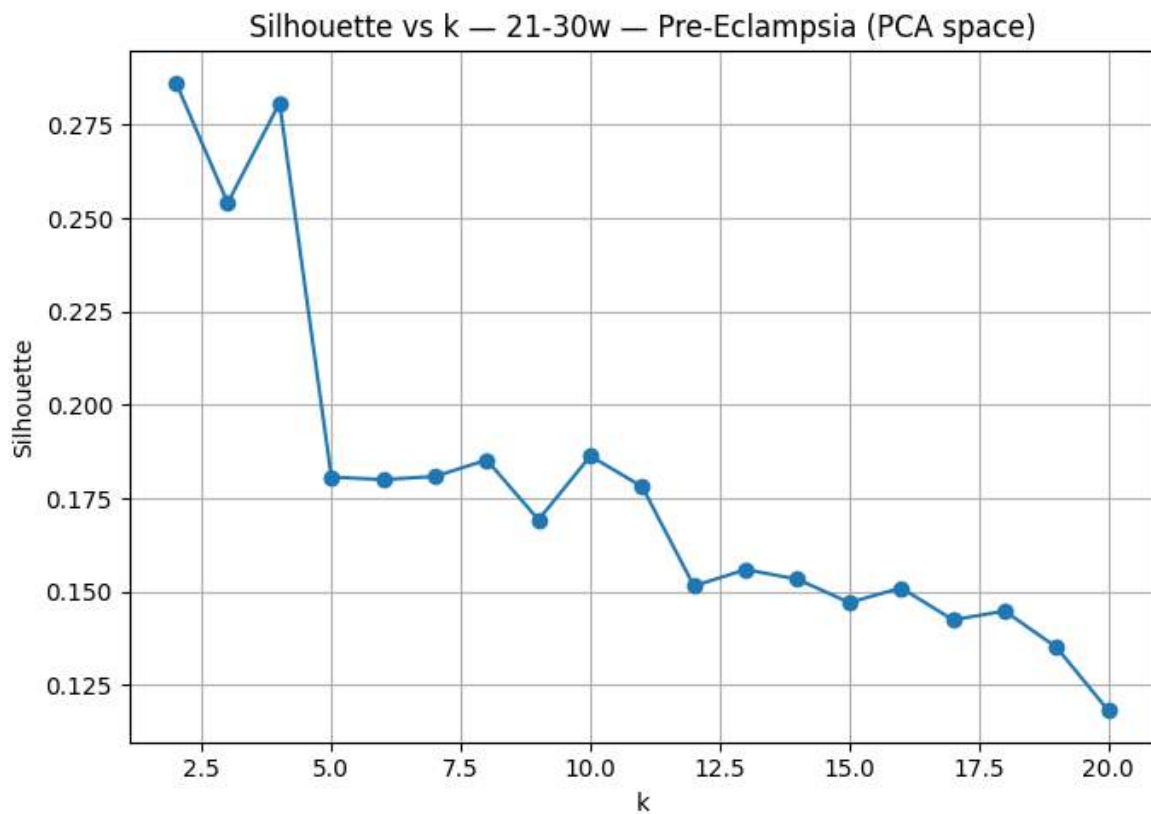




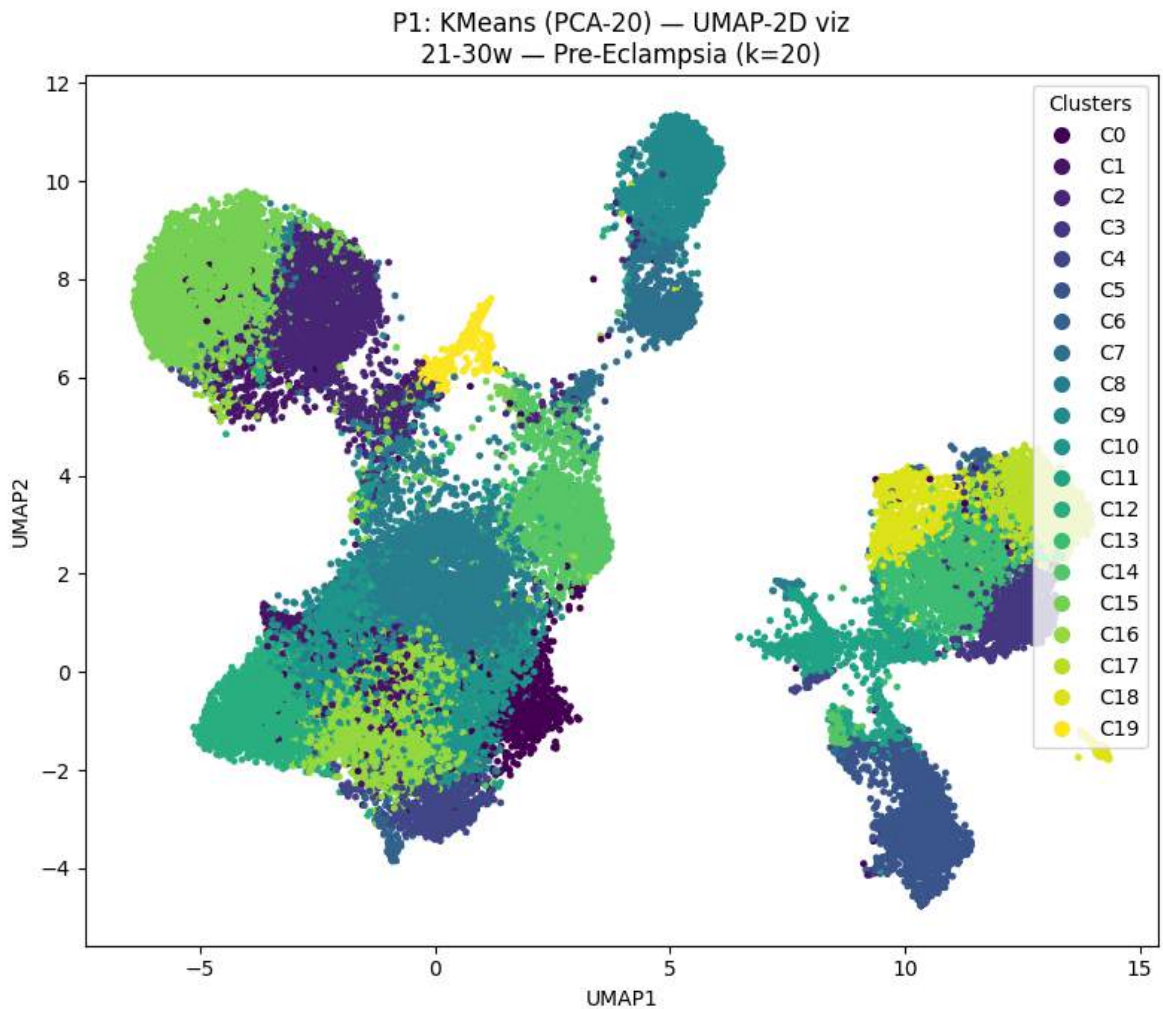
```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v  
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.  
warn(
```

P1: KMeans (PCA-20) — UMAP-2D viz
21-30w — Control (k=20)





```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v  
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.  
warn(
```

Out[]:

	interval	condition	pipeline	k_final	k_sil	k_gap	silhouette_at_k	
0	1-10w	0	Pipeline1(PCA20)	20	2	20	-0.087348	10w_cond0
1	1-10w	1	Pipeline1(PCA20)	20	2	20	0.134068	10w_cond1
2	11-20w	0	Pipeline1(PCA20)	20	2	20	-0.098687	20w_cond0
3	11-20w	1	Pipeline1(PCA20)	17	2	17	-0.056533	20w_cond1
4	21-30w	0	Pipeline1(PCA20)	20	2	20	-0.173056	30w_cond0
5	21-30w	1	Pipeline1(PCA20)	20	2	20	-0.051447	30w_cond1

Second Pipeline: Clustering 20PCAs in UMAP 2D, 3D AND 5D Spaces

```
In [ ]: # Pipeline 2: create UMAP embeddings 2D/3D/5D from PCA20 (20 PCs were just used
for interval in eda['age_bin_10w'].cat.categories:
    for cond in [0,1]:
        tag = f"{interval}_{cond}"
        pca_path = os.path.join(output_dir, f"pca20_{tag}.npy")
        if not os.path.exists(pca_path):
```

```

        print("[P2] Missing PCA20 for:", tag)
        continue

X_pca20 = np.load(pca_path)
for d in (2,3,5):
    upath = os.path.join(output_dir, f"umap{d}_pipeline2__{tag}.npy")
    if os.path.exists(upath):
        print(f"[P2] UMAP{d} cached:", tag);
        continue
    reducer = umap.UMAP(n_components=d, n_neighbors=30, min_dist=0.3, ra
    emb = reducer.fit_transform(X_pca20)
    np.save(upath, emb)
    print(f"[P2] UMAP{d} saved:", tag, emb.shape)

```

```

[P2] UMAP2 cached: 1-10w_0
[P2] UMAP3 cached: 1-10w_0
[P2] UMAP5 cached: 1-10w_0
[P2] UMAP2 cached: 1-10w_1
[P2] UMAP3 cached: 1-10w_1
[P2] UMAP5 cached: 1-10w_1
[P2] UMAP2 cached: 11-20w_0
[P2] UMAP3 cached: 11-20w_0
[P2] UMAP5 cached: 11-20w_0
[P2] UMAP2 cached: 11-20w_1
[P2] UMAP3 cached: 11-20w_1
[P2] UMAP5 cached: 11-20w_1
[P2] UMAP2 cached: 21-30w_0
[P2] UMAP3 cached: 21-30w_0
[P2] UMAP5 cached: 21-30w_0
[P2] UMAP2 cached: 21-30w_1
[P2] UMAP3 cached: 21-30w_1
[P2] UMAP5 cached: 21-30w_1

```

Selection best k from silhouette and gap, clustering in UMAP space and visualizing there

```

In [ ]: # Pipeline 2: k selection (silhouette+gap) in UMAP space - KMeans - viz

summary_rows = []

for interval in eda['age_bin_10w'].cat.categories:
    for cond in [0,1]:
        sub = eda[(eda['age_bin_10w']==interval) & (eda['condition']==cond)]
        if sub.empty:
            continue
        tag = f"{interval}_{cond}"

        # Ensure UMAP embeddings exist, otherwise regenerate
        for d in (2,3,5):
            umap_path = os.path.join(output_dir, f"umap{d}_pipeline2__{tag}.npy")

            if not os.path.exists(umap_path):
                print(f"[P2-UMAP{d}] Missing embedding: {tag} → regenerating...")
                # Try to regenerate from PCA20
                pca_path = os.path.join(output_dir, f"pca20__{tag}.npy")
                if not os.path.exists(pca_path):
                    print(f"[P2-UMAP{d}] Missing PCA20 for {tag}, creating it...")
                    X = sub[FEATURES].to_numpy(dtype="float32")
                    X = StandardScaler().fit_transform(X)
                    X_pca20 = PCA(n_components=20, random_state=RANDOM_STATE).fi
                    np.save(pca_path, X_pca20)

```

```

else:
    X_pca20 = np.load(pca_path)

    # create UMAP
    reducer = umap.UMAP(
        n_components=d, n_neighbors=30, min_dist=0.3, random_state=R
    )
    emb = reducer.fit_transform(X_pca20)
    np.save(umap_path, emb)
    print(f"[P2-UMAP{d}] Regenerated and saved:", tag, emb.shape)
else:
    emb = np.load(umap_path)

if len(emb) < 5:
    print(f"[P2-UMAP{d}] Too few rows for {tag}")
    continue

title = f"{interval} - {COND_NAME[cond]} (UMAP{d} space)"
prefix = os.path.join(output_dir, f"p2_umap{d}_{interval}_cond{cond}")

# Choose k (silhouette + gap)
k_star, sil_info, gap_info = choose_k_by_gap_and_silhouette(emb, tit
sil_k, sil_csv, sil_png = sil_info
gap_k, gap_csv, gap_png = gap_info

if (k_star is None) or (k_star >= len(emb)) or (k_star < 2):
    print(f"[P2-UMAP{d}] No valid k for {tag}")
    continue

# Perform KMeans clustering
labels = KMeans(
    n_clusters=k_star, random_state=RANDOM_STATE, n_init=20
).fit_predict(emb)

pd.Series(labels, index=sub.index, name=f"kmeans_pipeline2_umap{d}")
.to_csv(os.path.join(output_dir, f"labels_kmeans_pipeline2_umap{d}")

# Visualization
if d == 2:
    plt.figure(figsize=(8,7))
    sc = plt.scatter(emb[:,0], emb[:,1], c=labels, s=6)
    plt.title(f"P2: KMeans in UMAP-2D - {interval} - {COND_NAME[cond]}")
    plt.xlabel("UMAP1"); plt.ylabel("UMAP2")
    uniq = np.unique(labels)
    handles = [plt.Line2D([0],[0], marker='o', linestyle='', markers
        markerfacecolor=sc.cmap(sc.norm(1)), markeredgecolor
        label=f"C{int(1)}") for l in uniq]
    plt.legend(handles=handles, title="Clusters", loc="best", framed
    out_png = os.path.join(output_dir, f"umap2_plot_kmeans_pipeline2
    plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.show(); p

elif d == 3:
    fig = plt.figure(figsize=(9,7)); ax = fig.add_subplot(111, proje
    sc = ax.scatter(emb[:,0], emb[:,1], emb[:,2], c=labels, s=6)
    ax.set_title(f"P2: KMeans in UMAP-3D - {interval} - {COND_NAME[c
    ax.set_xlabel("UMAP1"); ax.set_ylabel("UMAP2"); ax.set_zlabel("U
    fig.colorbar(sc, label="cluster"); plt.tight_layout()
    out_png = os.path.join(output_dir, f"umap3_plot_kmeans_pipeline2
    plt.savefig(out_png, dpi=150); plt.show(); plt.close()

```

```

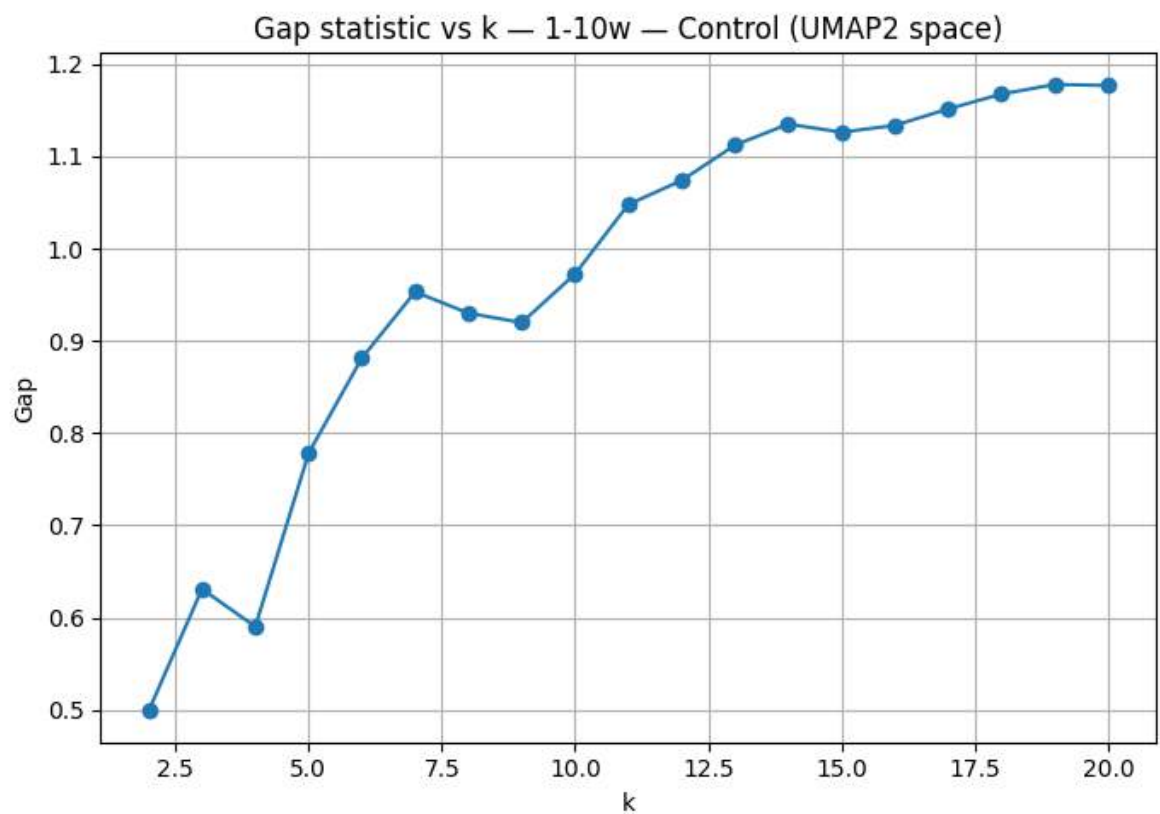
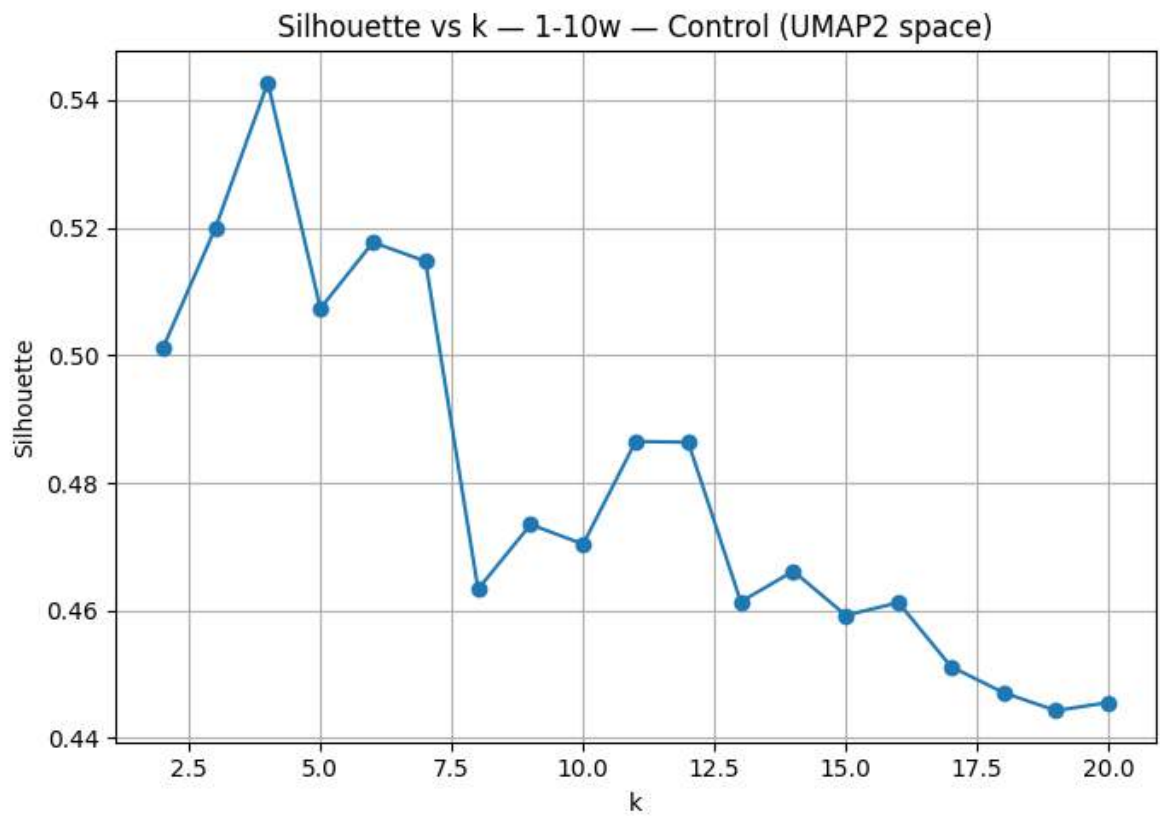
elif d == 5:
    # 3D projection from first 3 of 5 UMAP dims
    fig = plt.figure(figsize=(9,7))
    ax = fig.add_subplot(111, projection='3d')
    sc = ax.scatter(emb[:,0], emb[:,1], emb[:,2], c=labels, s=6)
    ax.set_title(f"P2: KMeans in UMAP-5D (3D projection) - {interval}")
    ax.set_xlabel("UMAP1"); ax.set_ylabel("UMAP2"); ax.set_zlabel("UMAP3")
    fig.colorbar(sc, label="cluster")
    out_png = os.path.join(output_dir, f"umap5_plot3D_kmeans_pipeline_{interval}.png")
    plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.show()

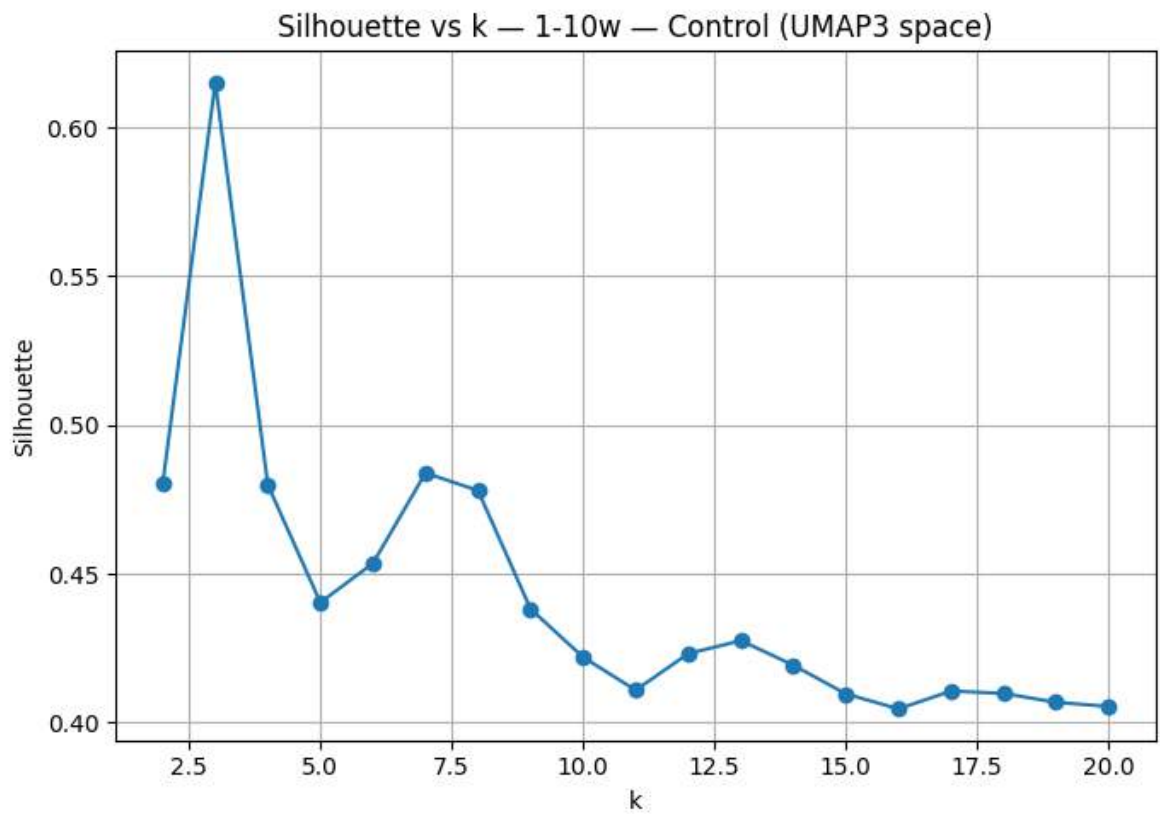
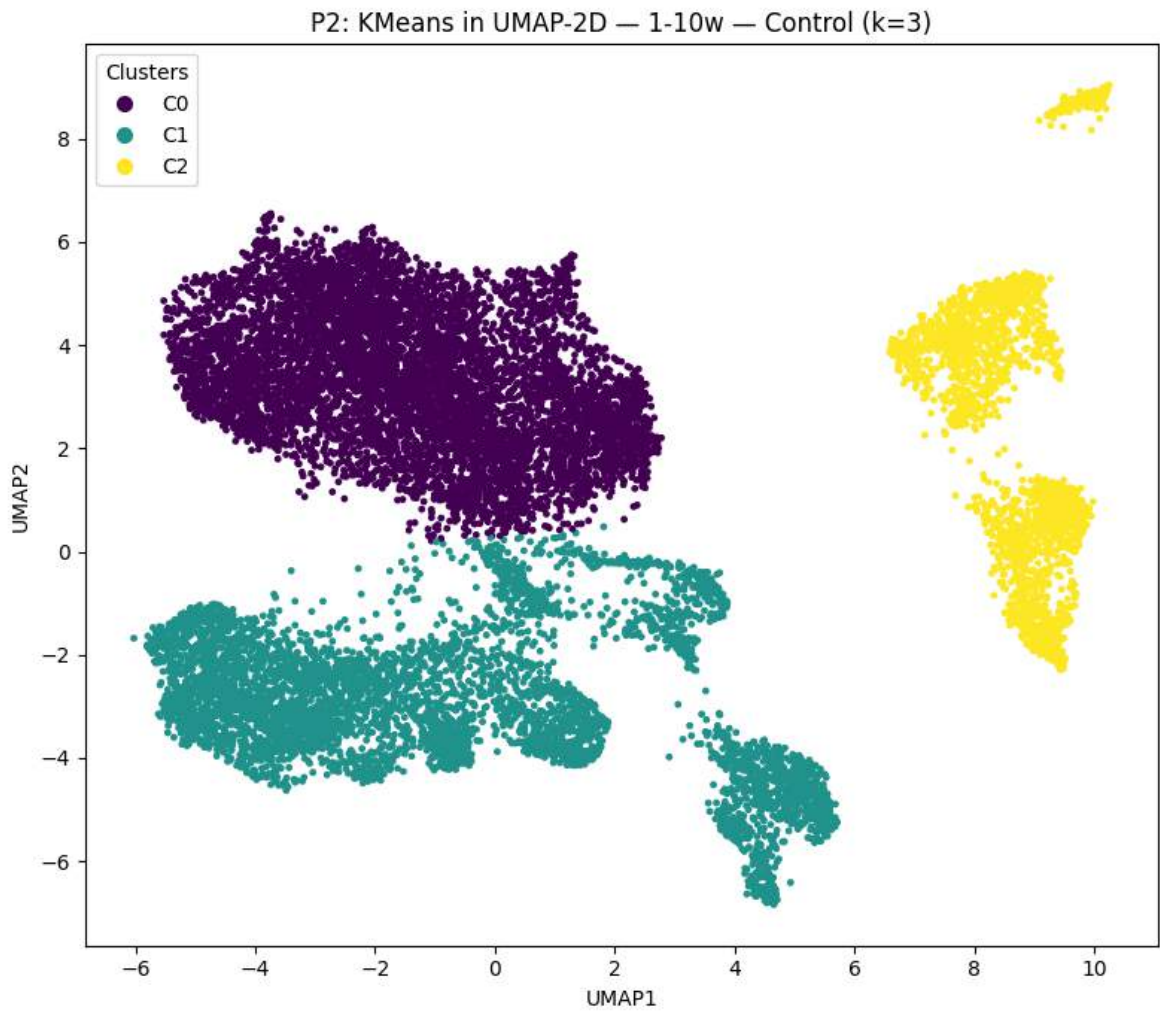
# Record metrics
Xs = subsample_rows(emb)
s_score = np.nan
if len(np.unique(labels)) > 1:
    s_idx = np.arange(len(emb))
    if len(emb) > len(Xs):
        s_idx = rng.choice(len(emb), size=len(Xs), replace=False)
        s_score = silhouette_score(emb[s_idx], labels[s_idx])
    else:
        s_score = silhouette_score(emb, labels)

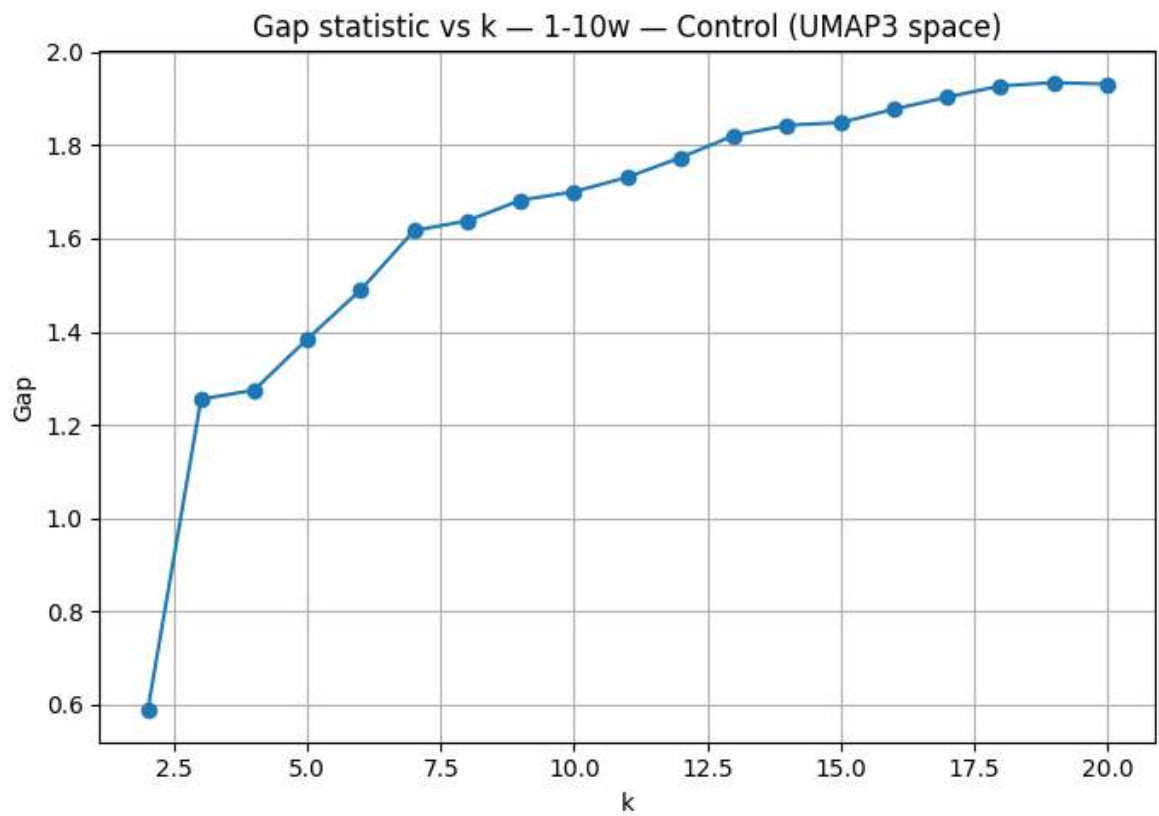
summary_rows.append([str(interval), cond, f"Pipeline2(UMAP{d})",
                    k_star, sil_k, gap_k, s_score,
                    os.path.basename(sil_csv), os.path.basename(gap_csv),
                    os.path.basename(sil_png), os.path.basename(gap_png),
                    os.path.basename(out_png)])

# Save summary
p2_summary = pd.DataFrame(summary_rows, columns=[
    "interval", "condition", "pipeline", "k_final", "k_sil", "k_gap", "silhouette_at_k",
    "silhouette_csv", "gap_csv", "silhouette_png", "gap_png", "umap_plot_png"
])
p2_summary.to_csv(os.path.join(output_dir, "kmeans_p2_summary.csv"), index=False)
print("\n Pipeline 2 complete. Summary saved to kmeans_p2_summary.csv")
p2_summary

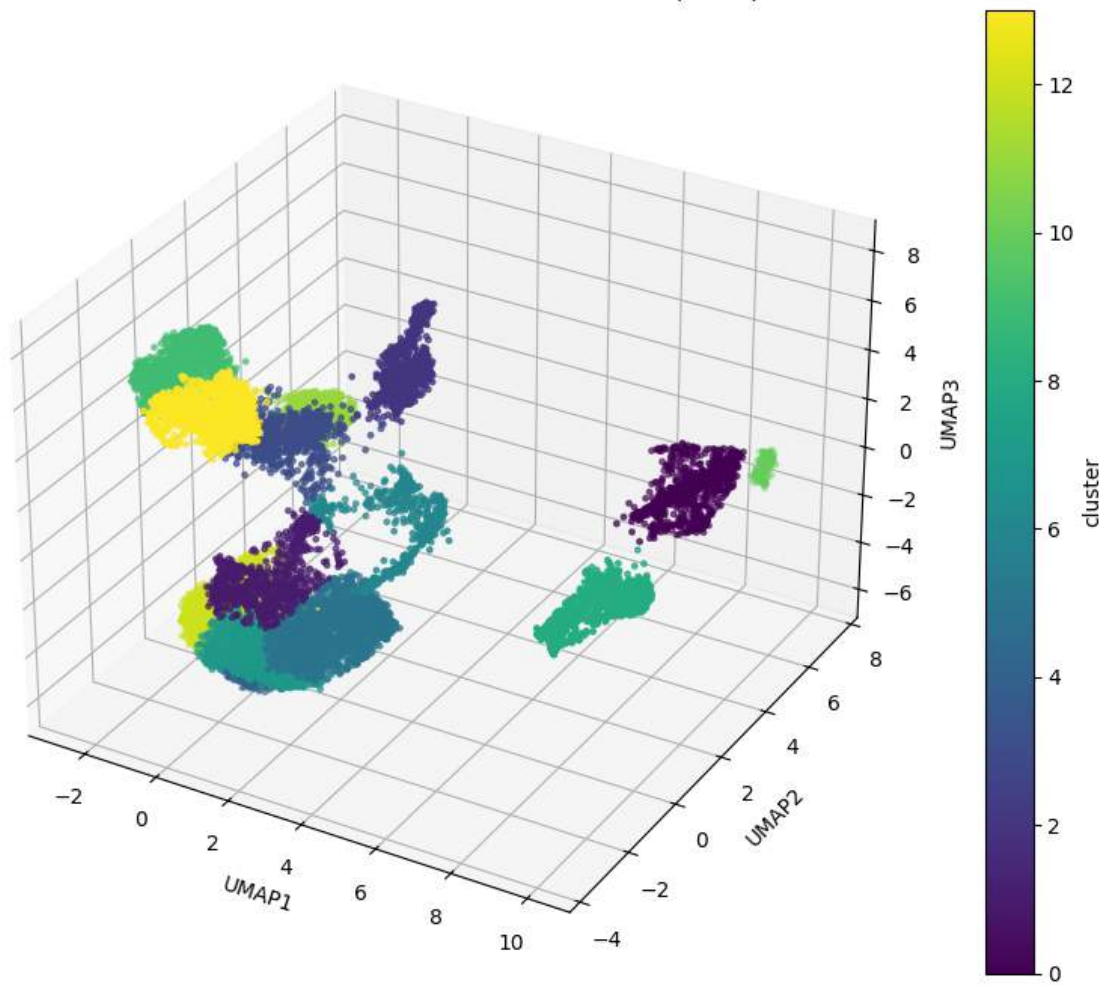
```

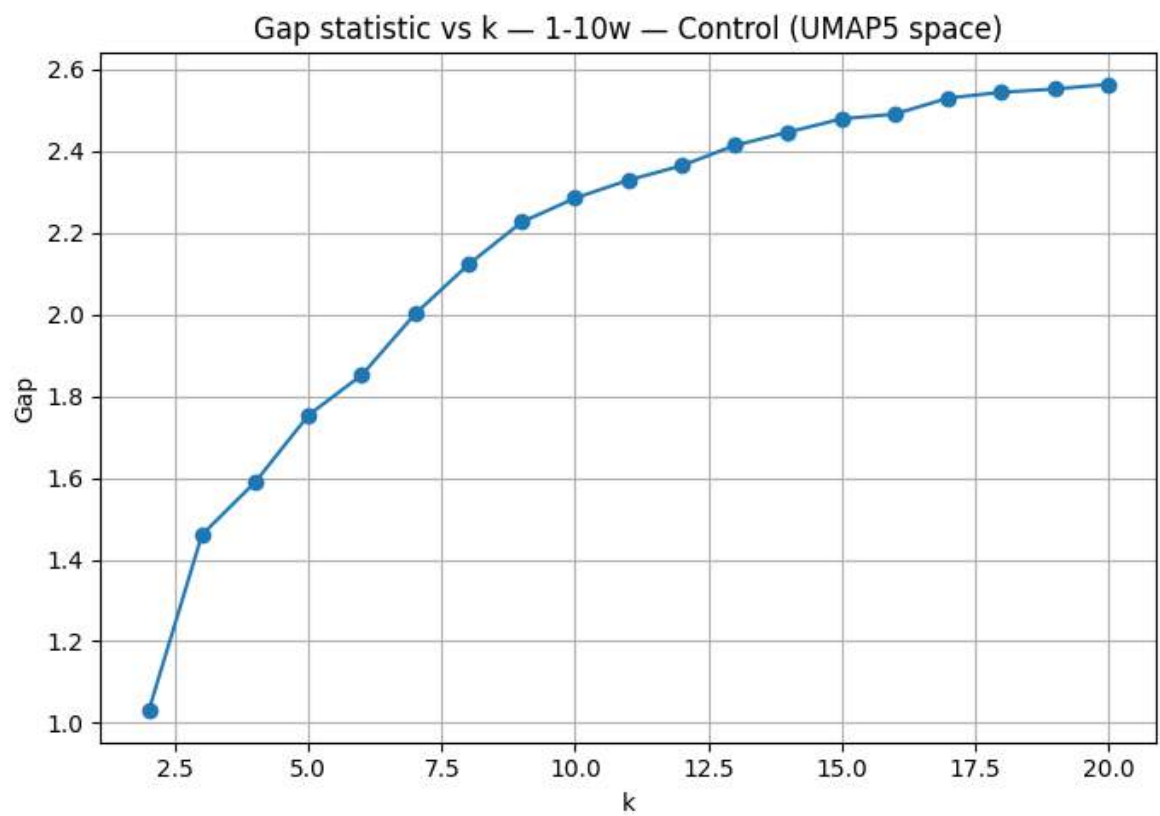
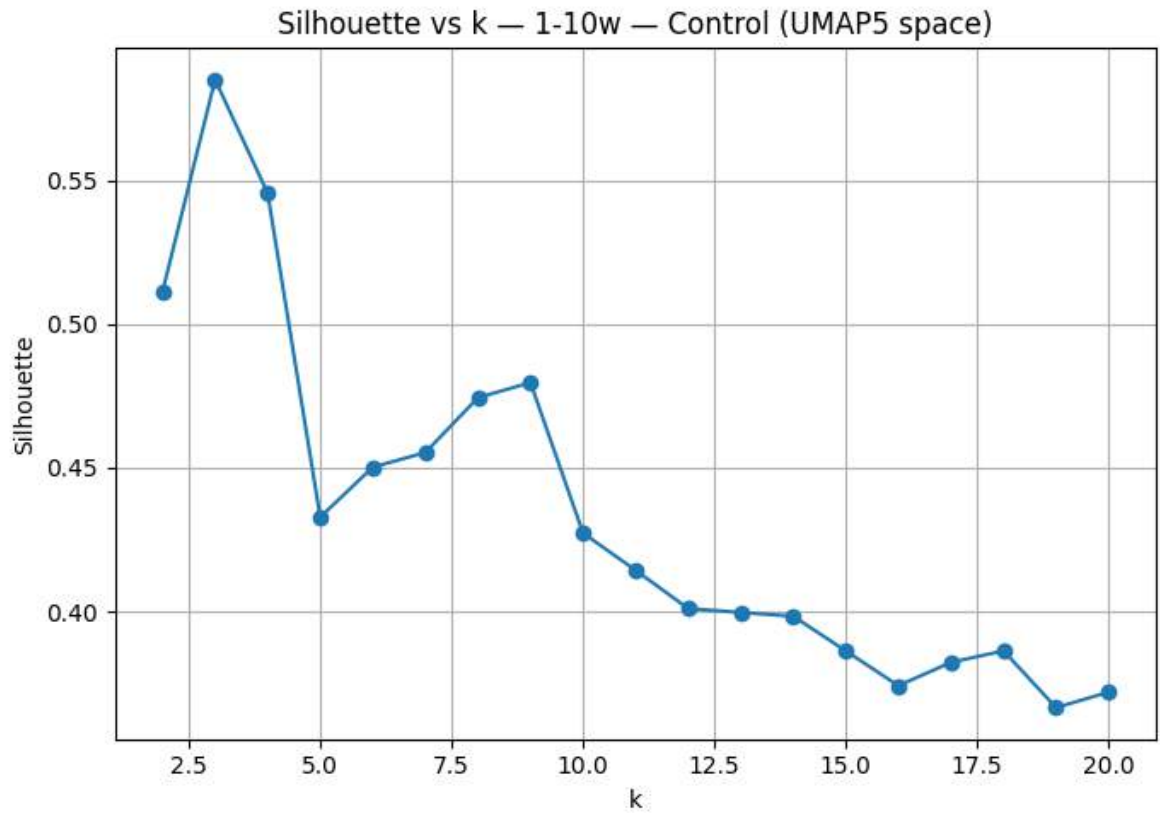




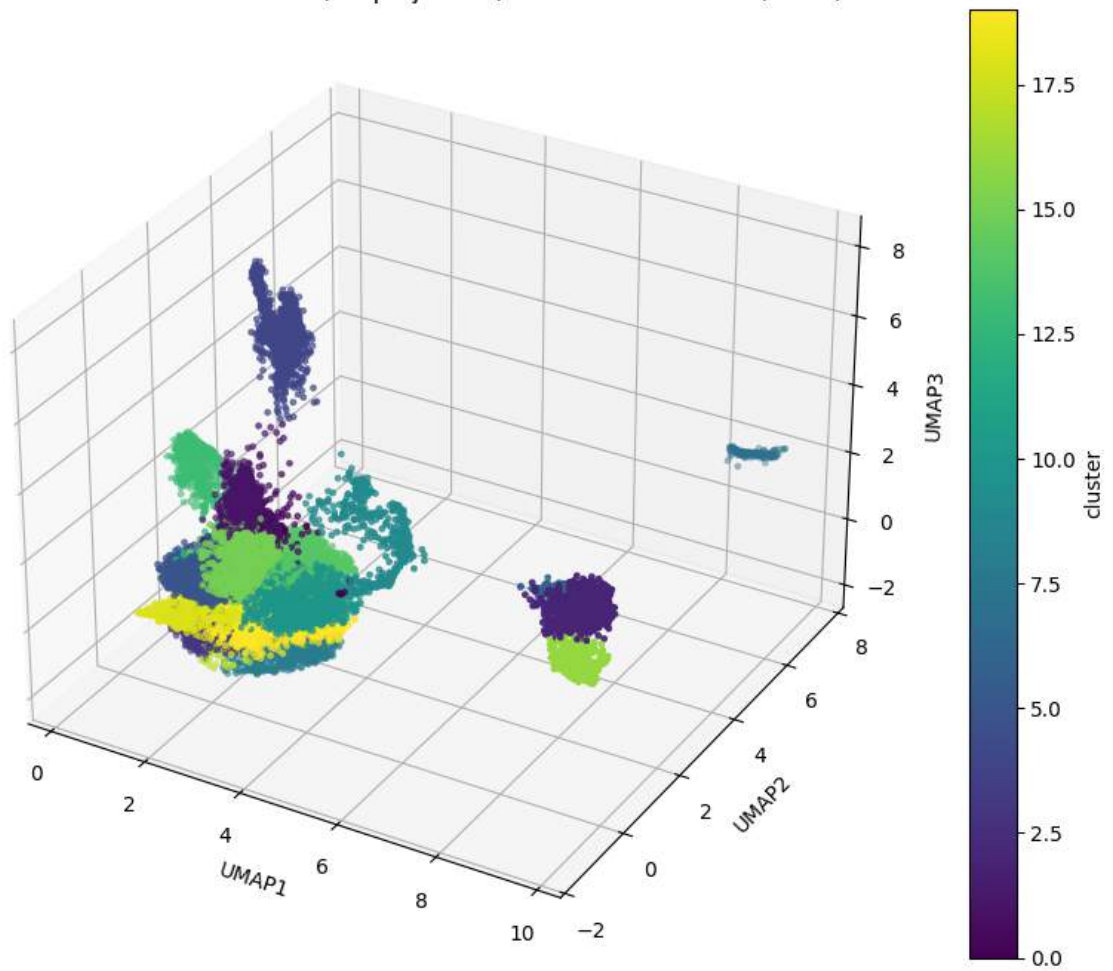


P2: KMeans in UMAP-3D — 1-10w — Control (k=14)

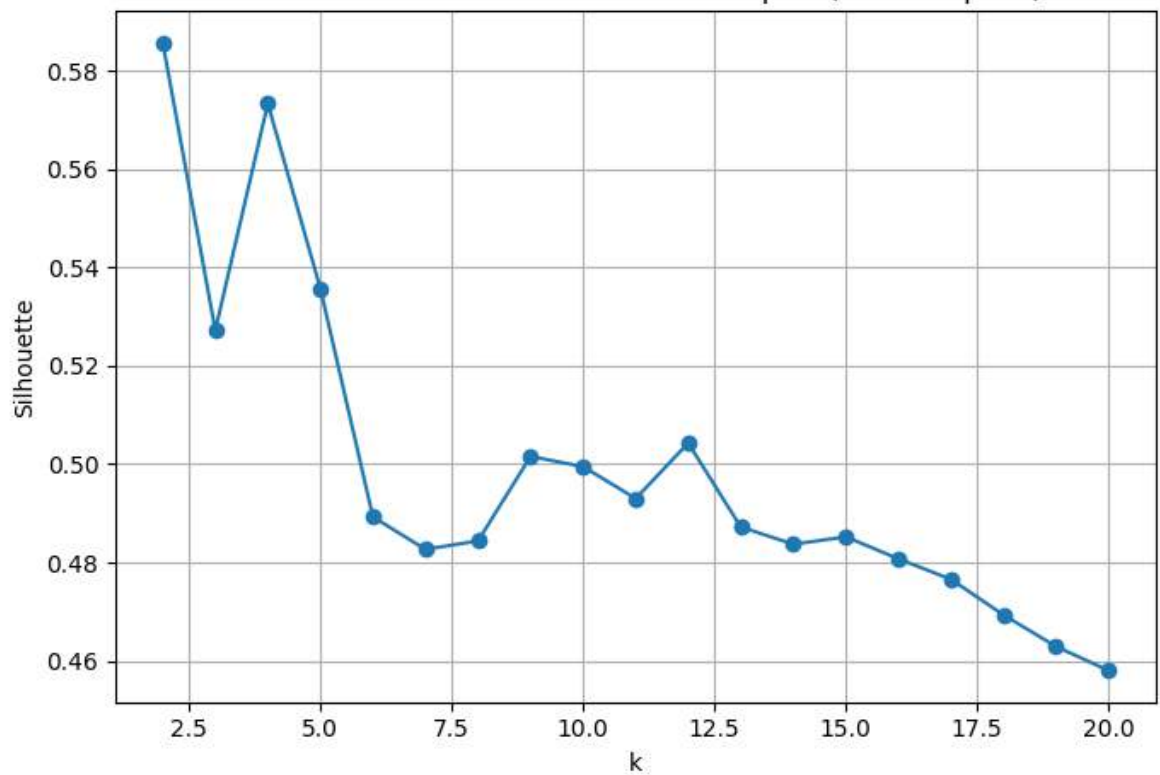


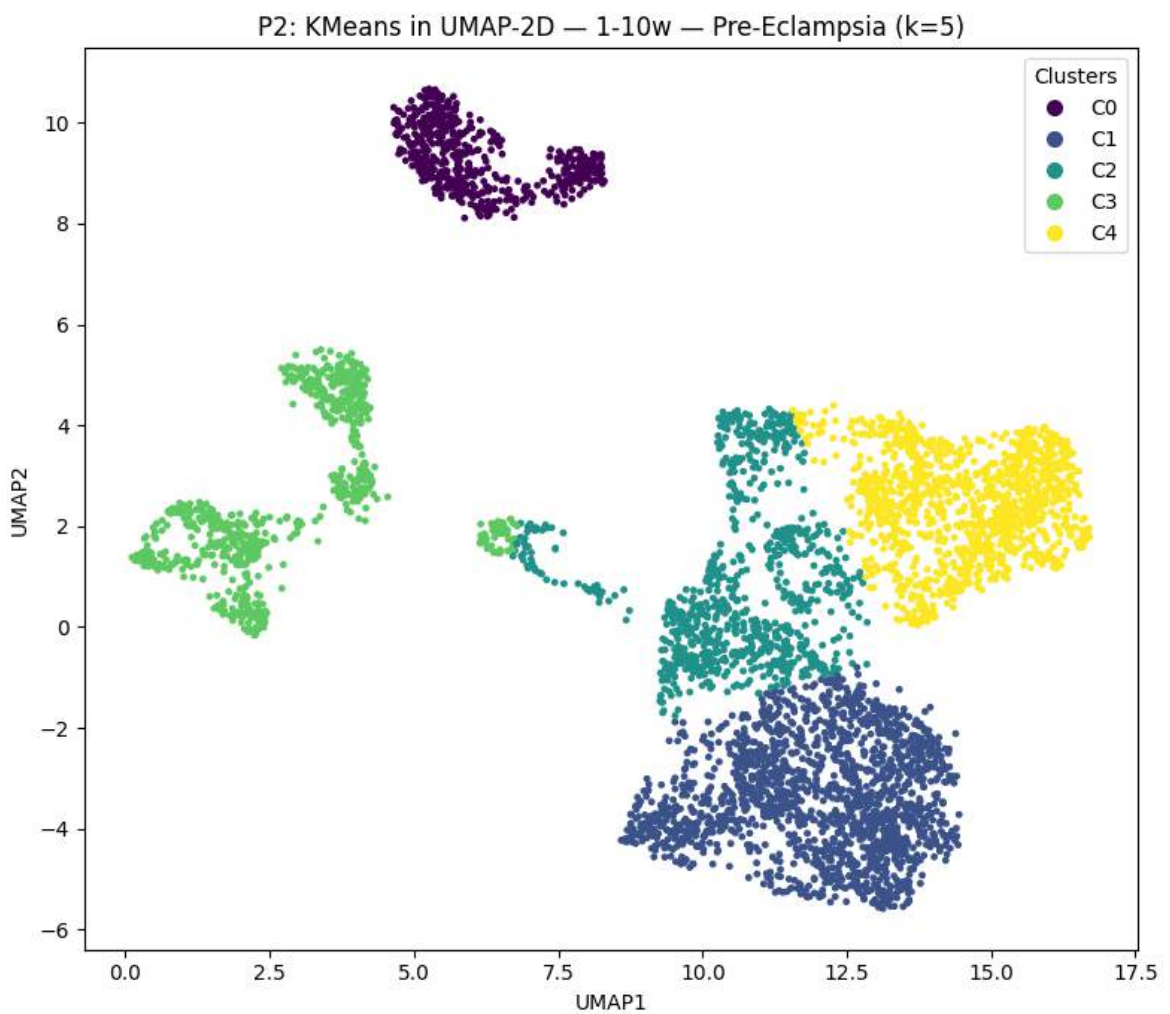
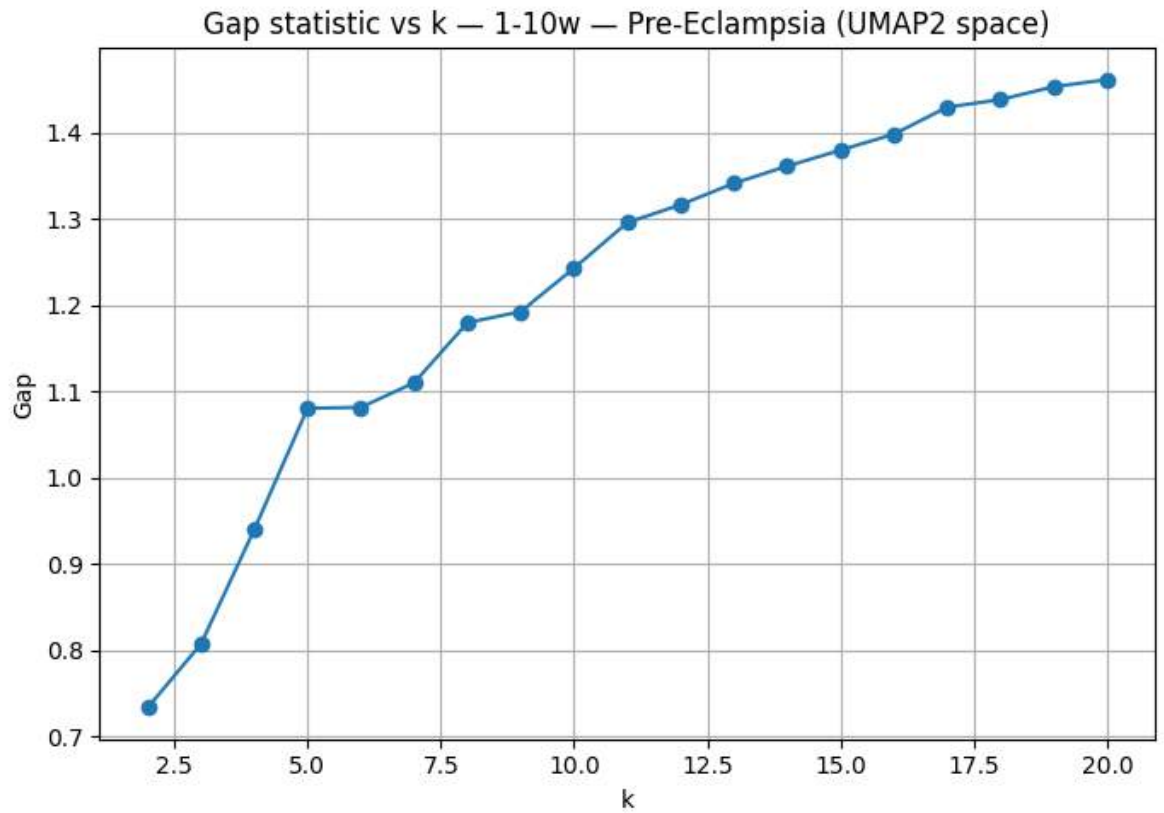


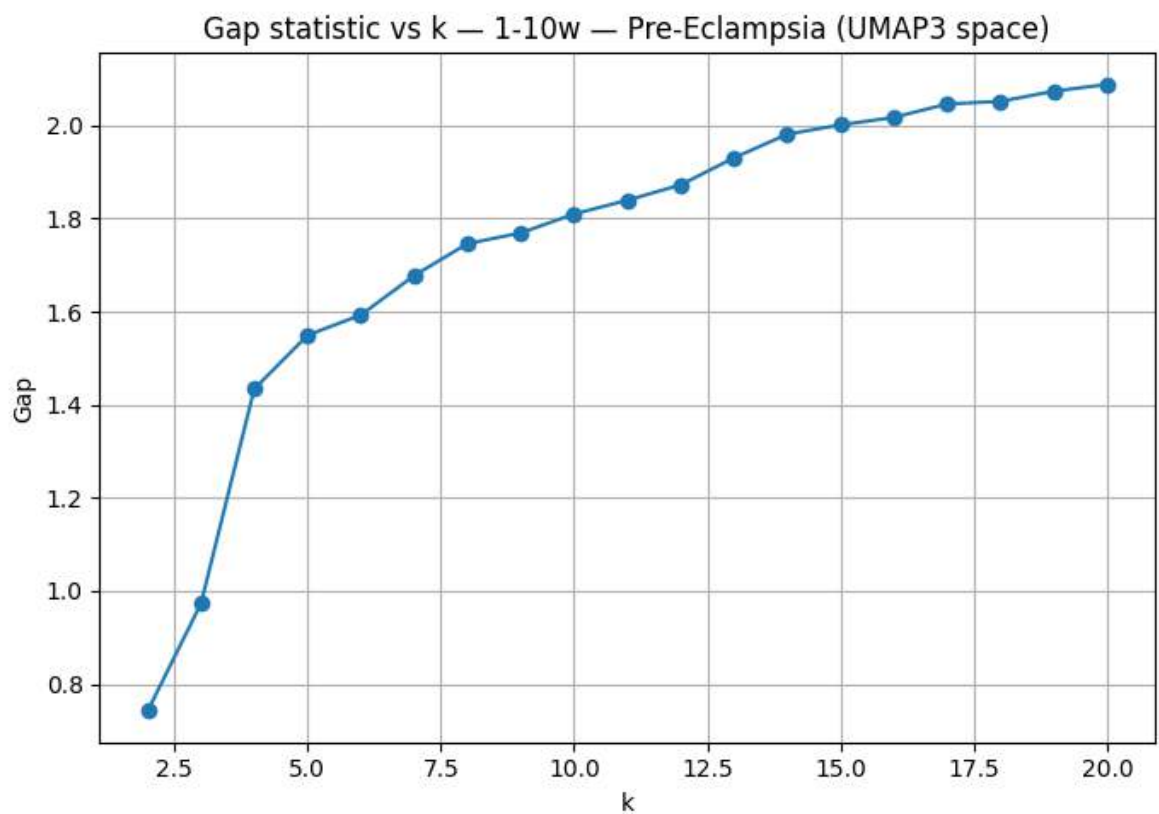
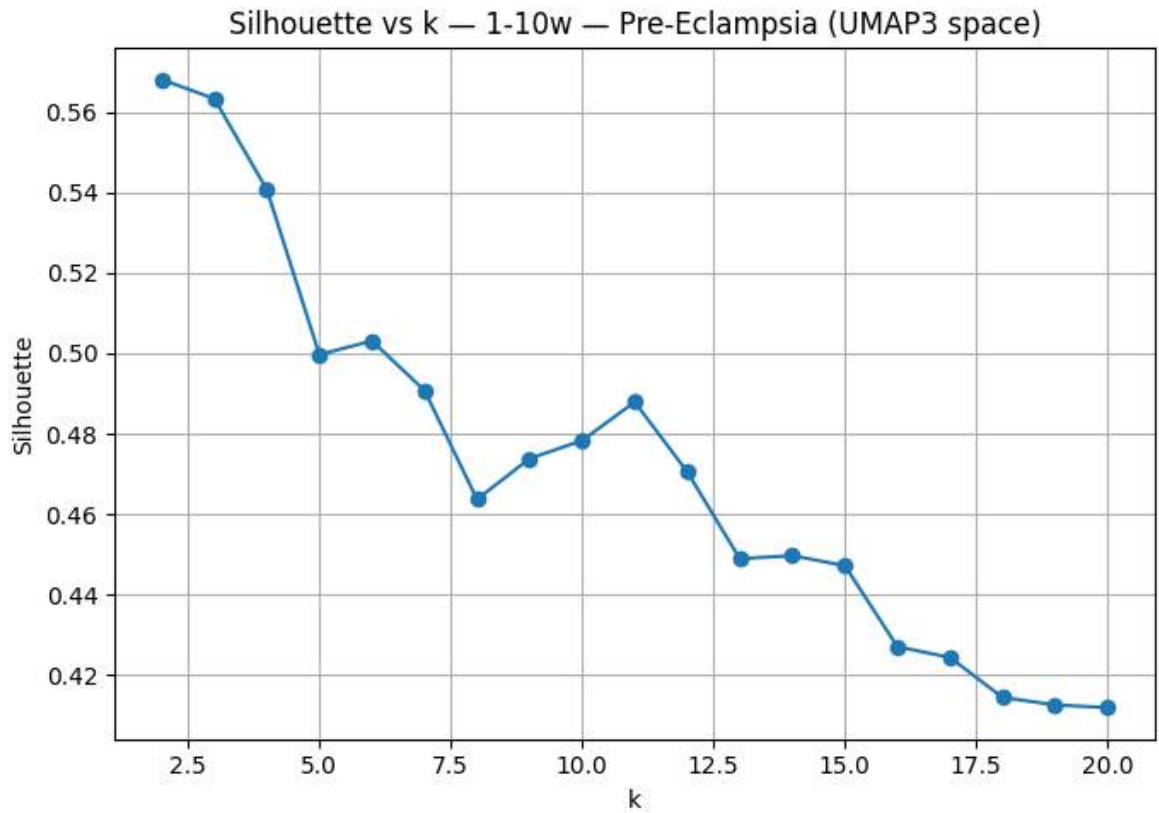
P2: KMeans in UMAP-5D (3D projection) — 1-10w — Control (k=20)



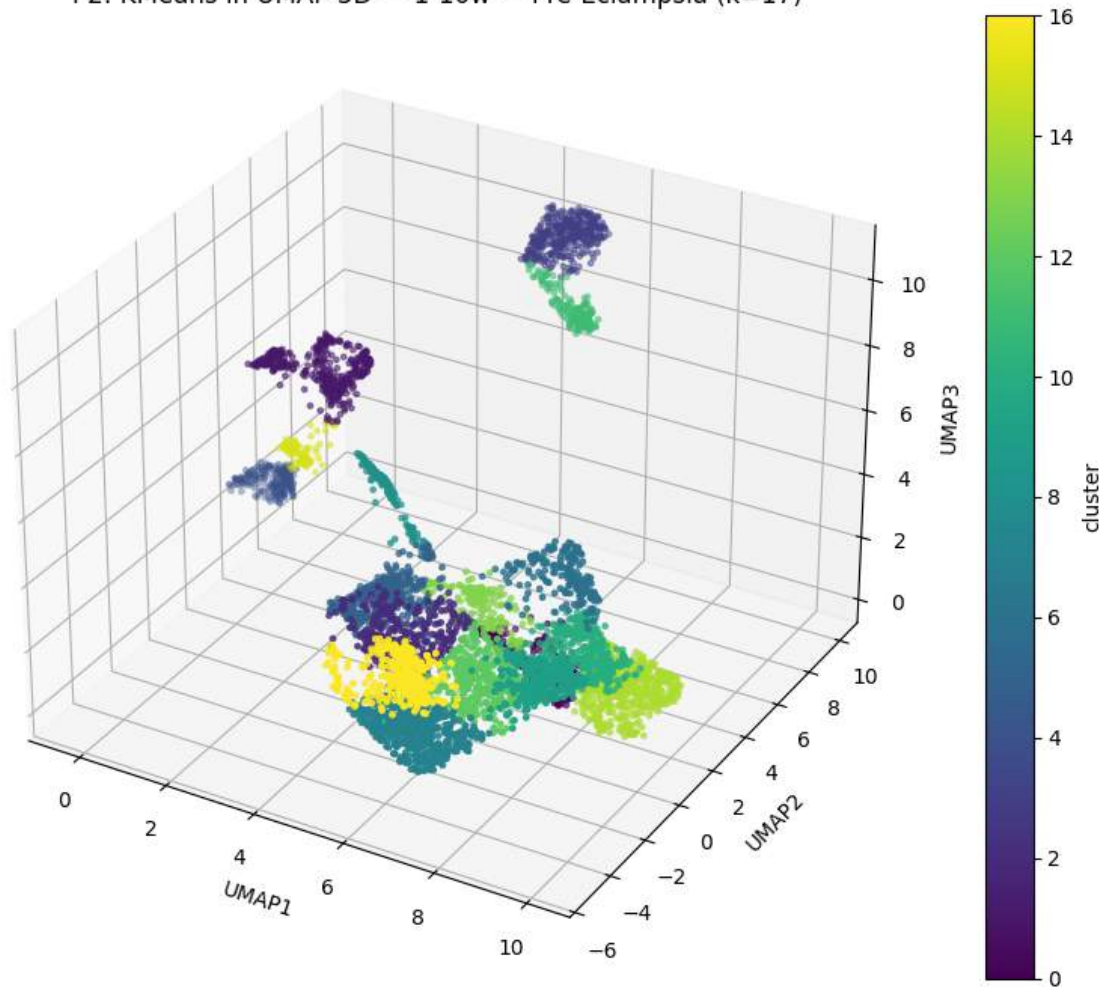
Silhouette vs k — 1-10w — Pre-Eclampsia (UMAP2 space)



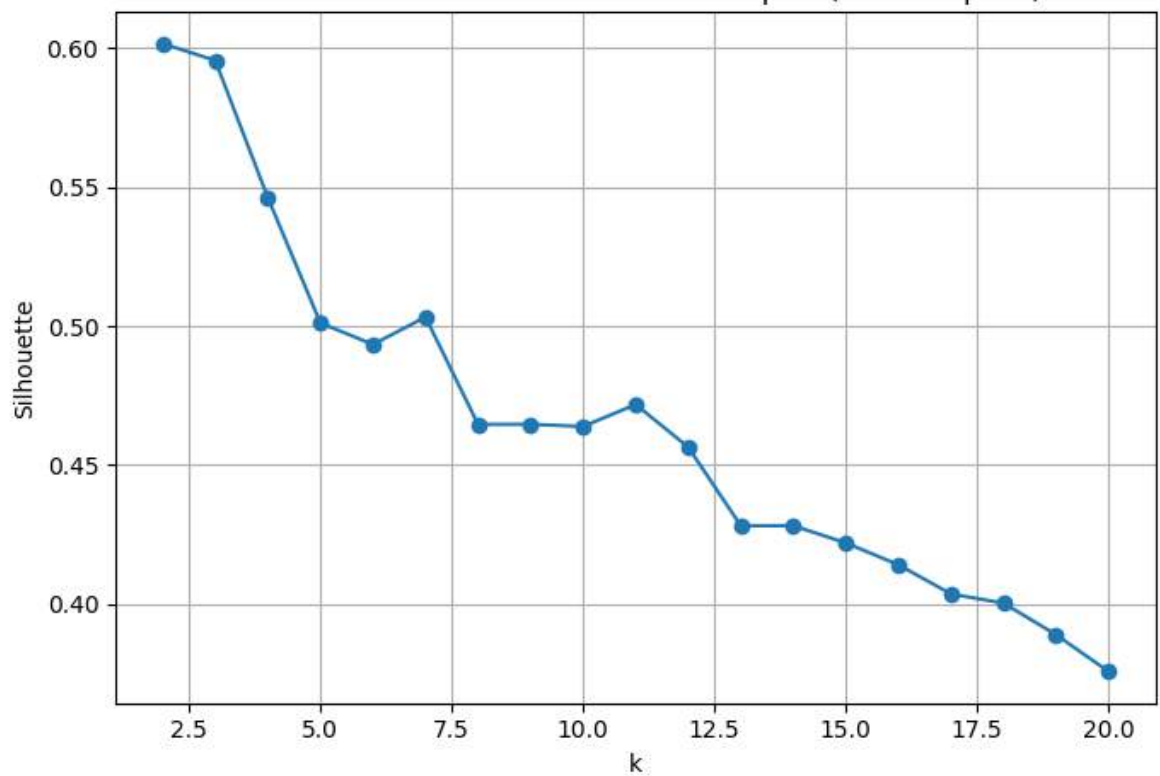


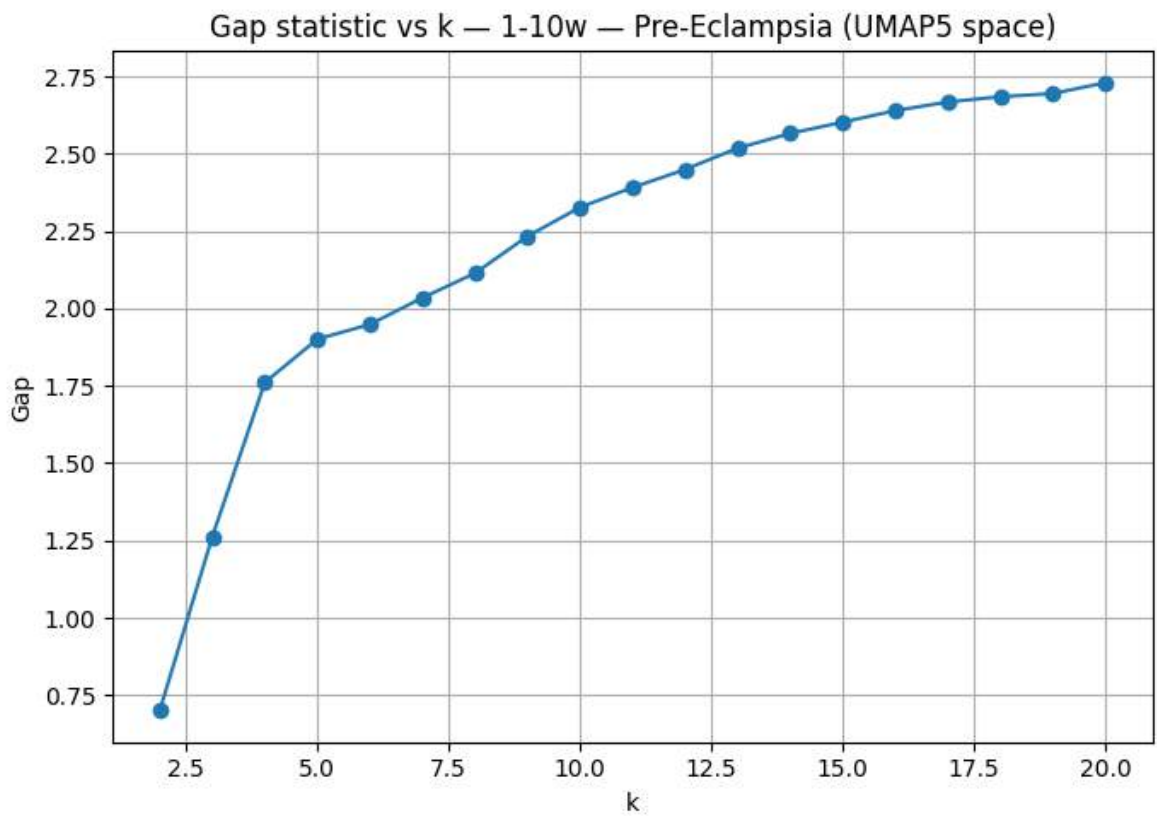


P2: KMeans in UMAP-3D — 1-10w — Pre-Eclampsia (k=17)

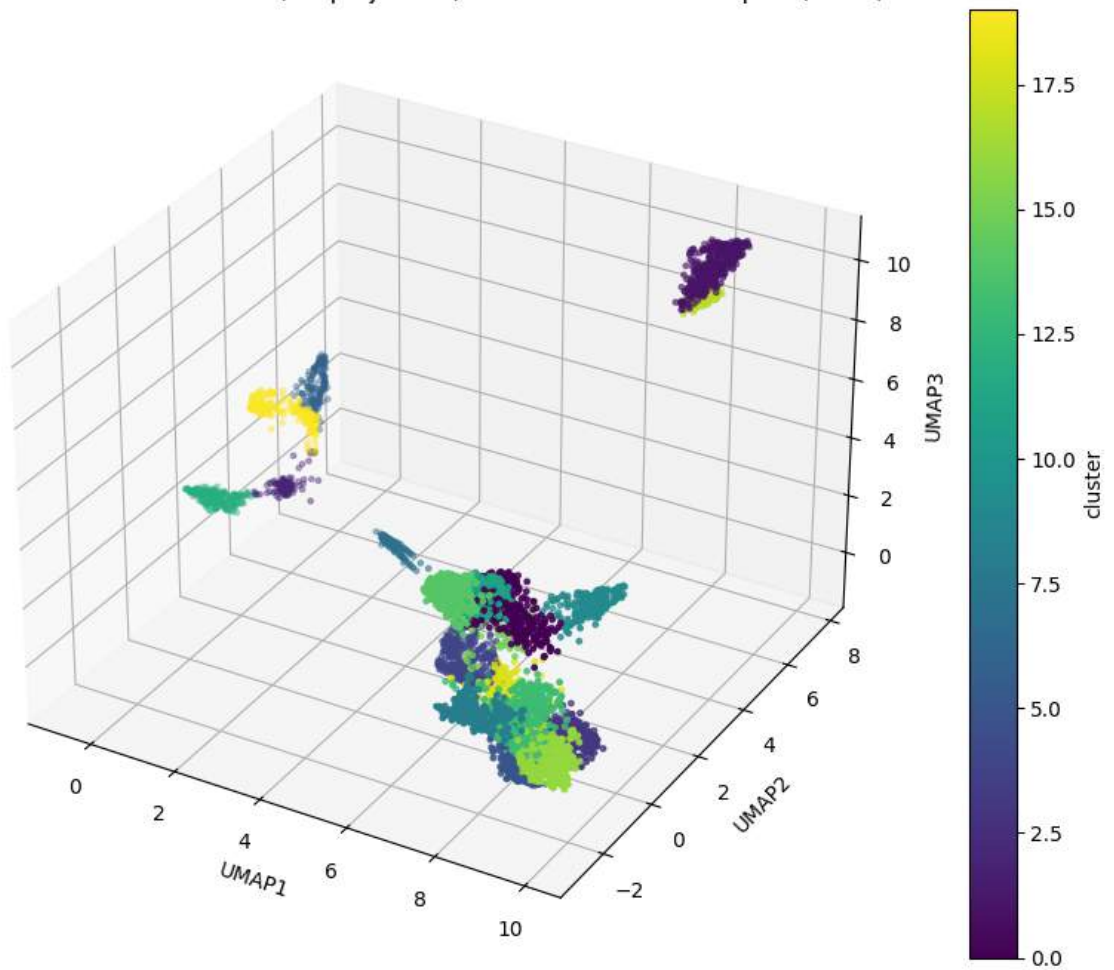


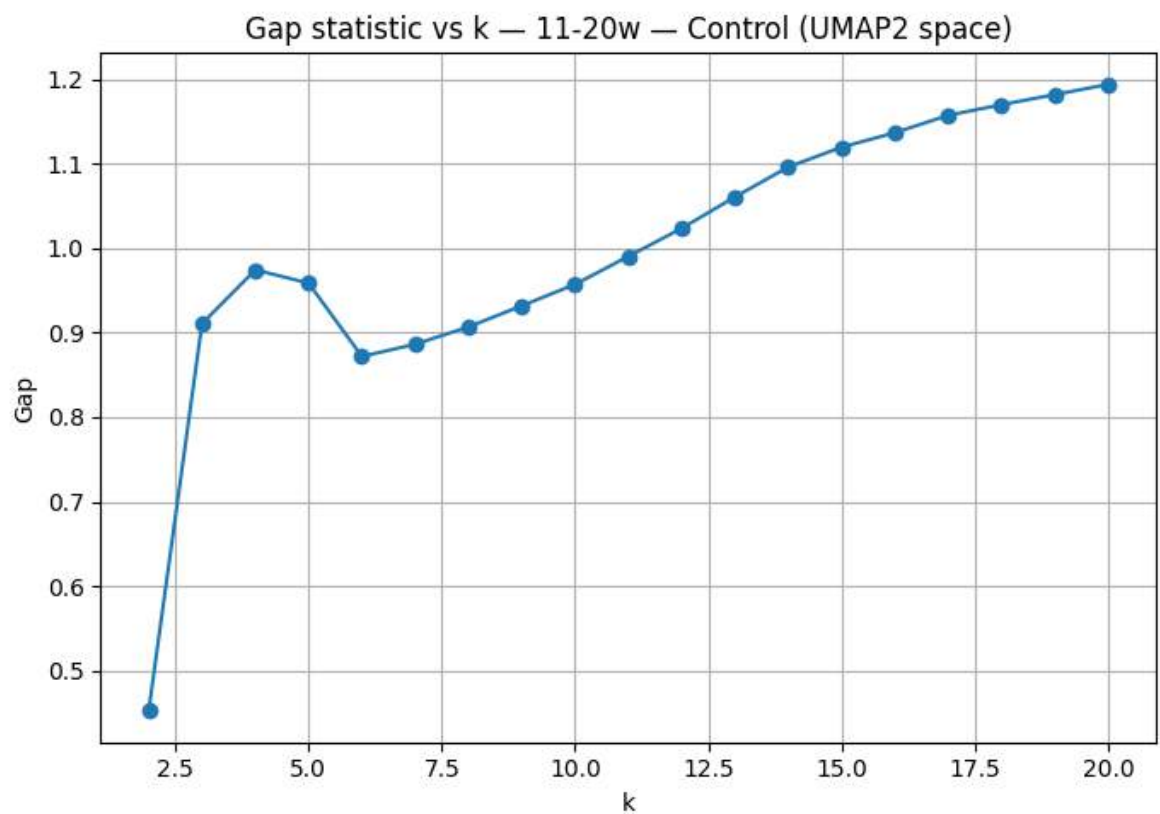
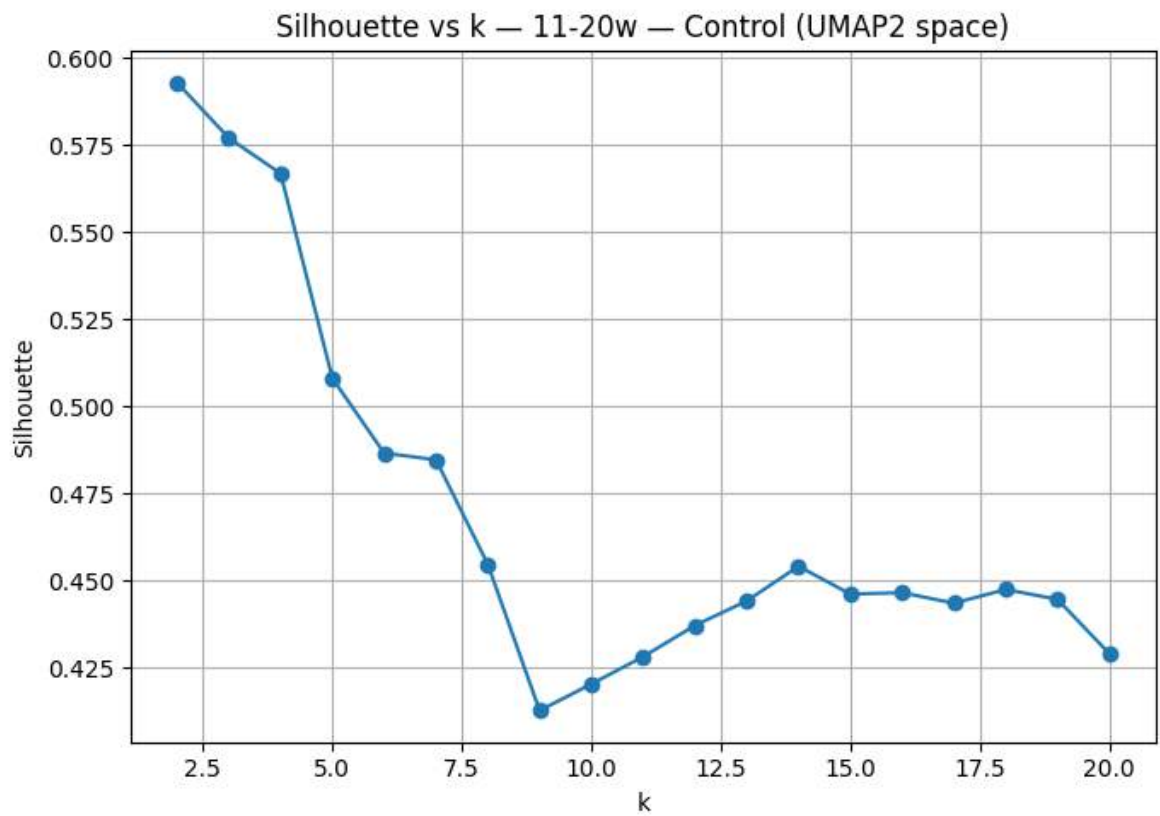
Silhouette vs k — 1-10w — Pre-Eclampsia (UMAP5 space)

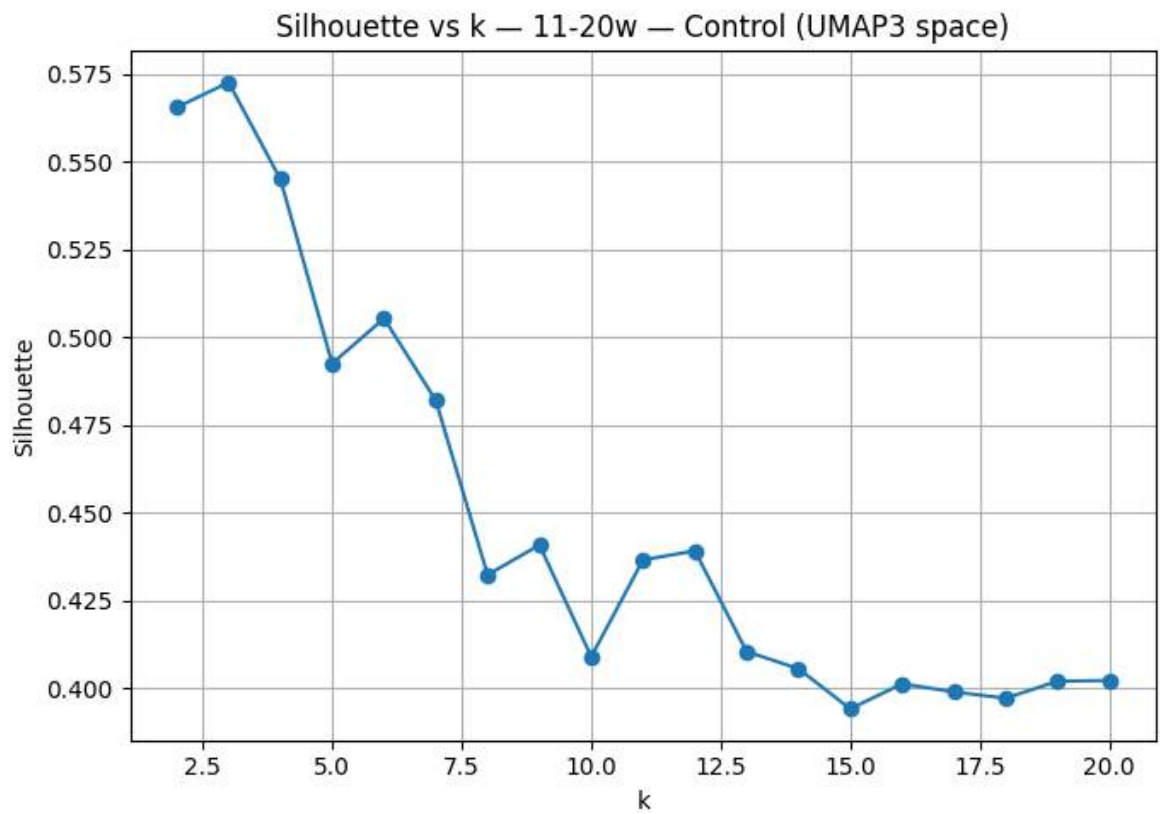
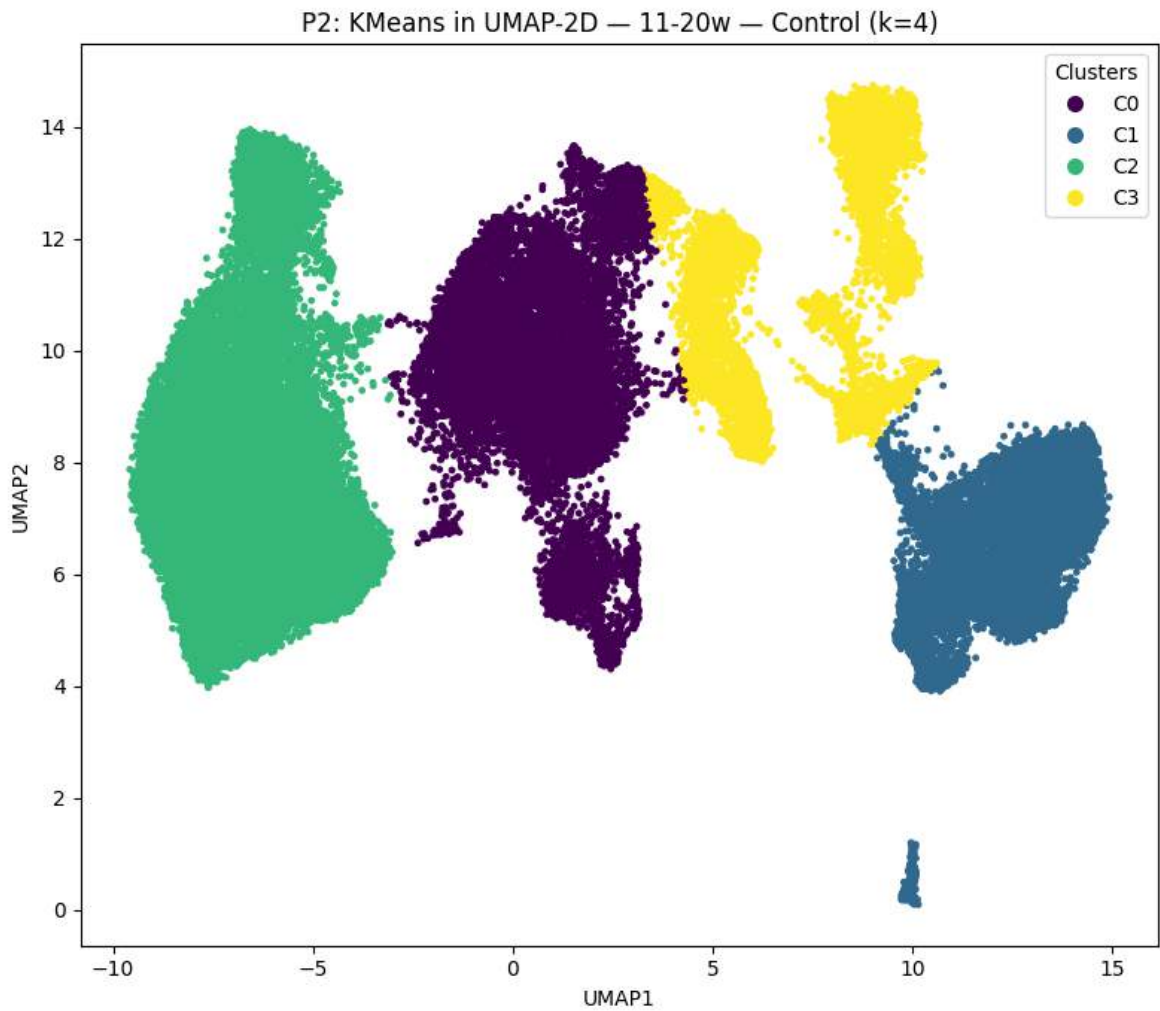


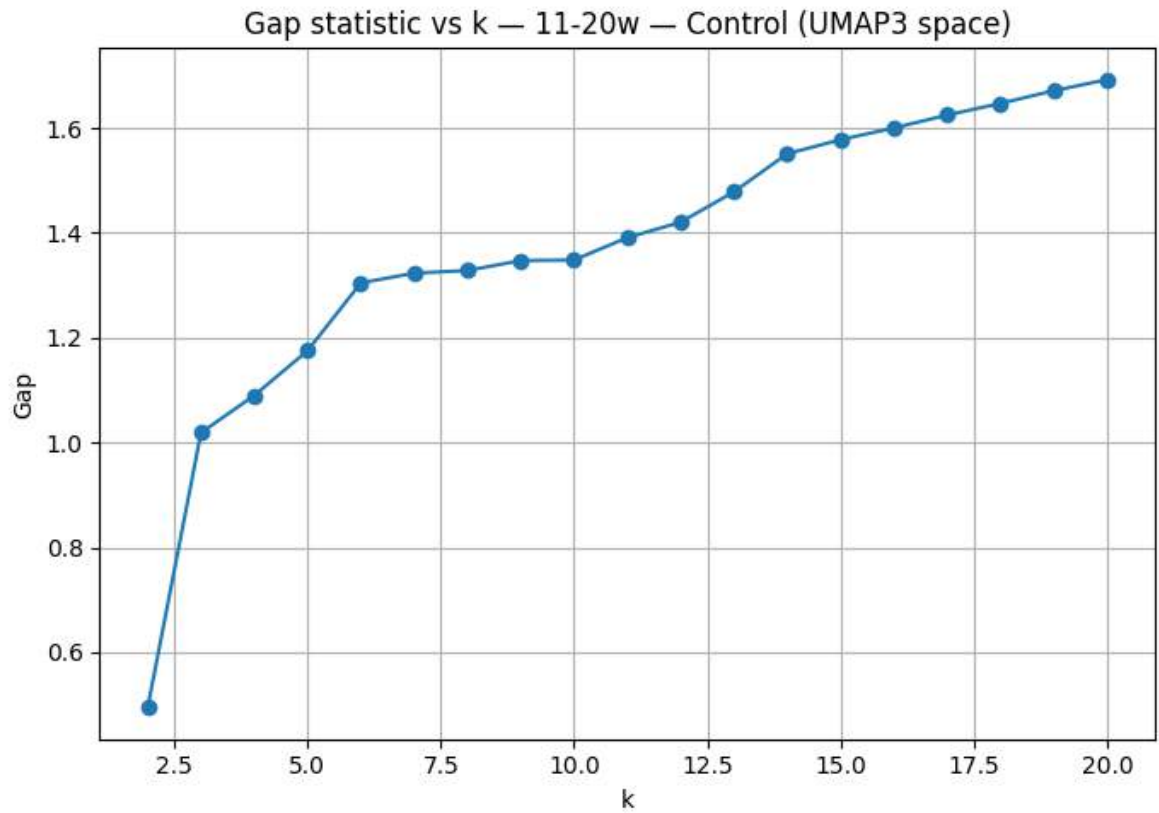


P2: KMeans in UMAP-5D (3D projection) — 1-10w — Pre-Eclampsia (k=20)

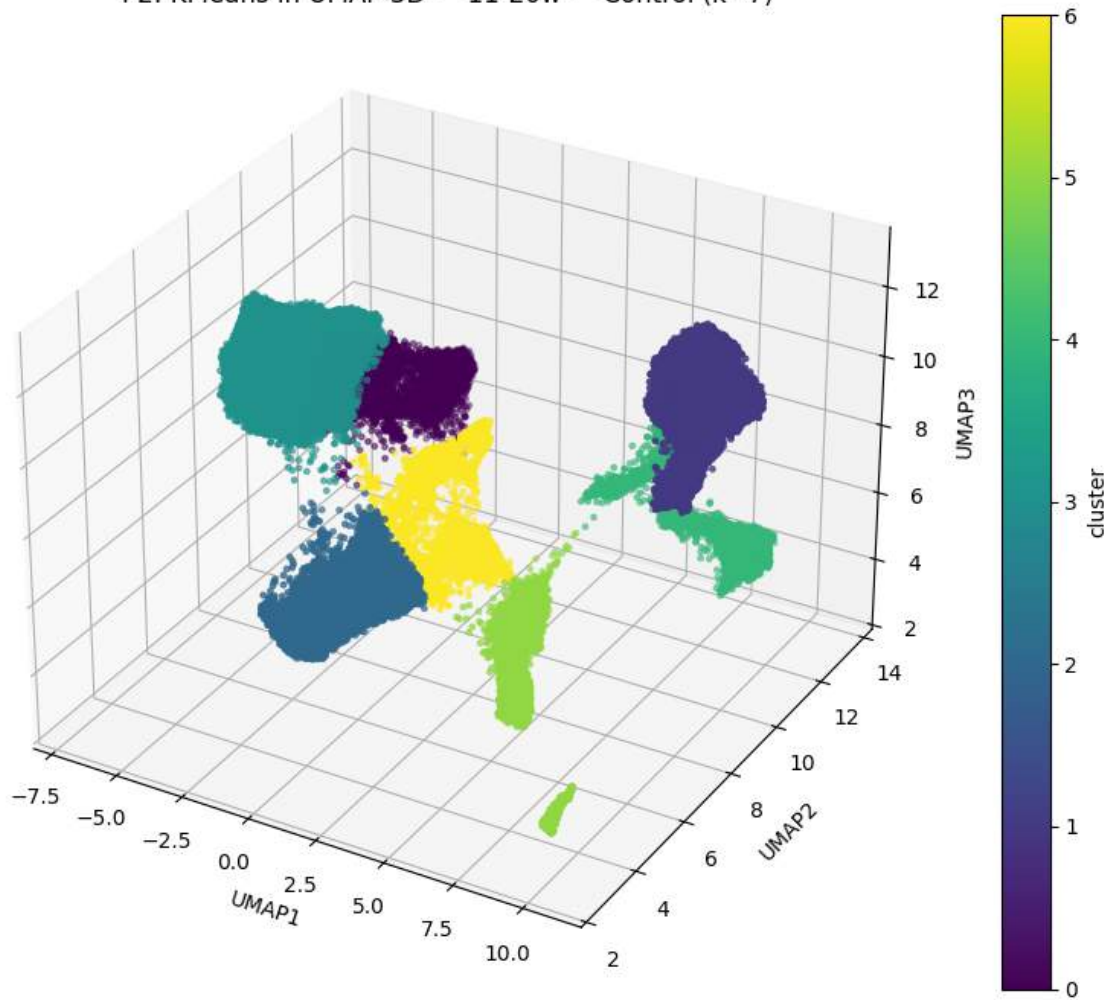


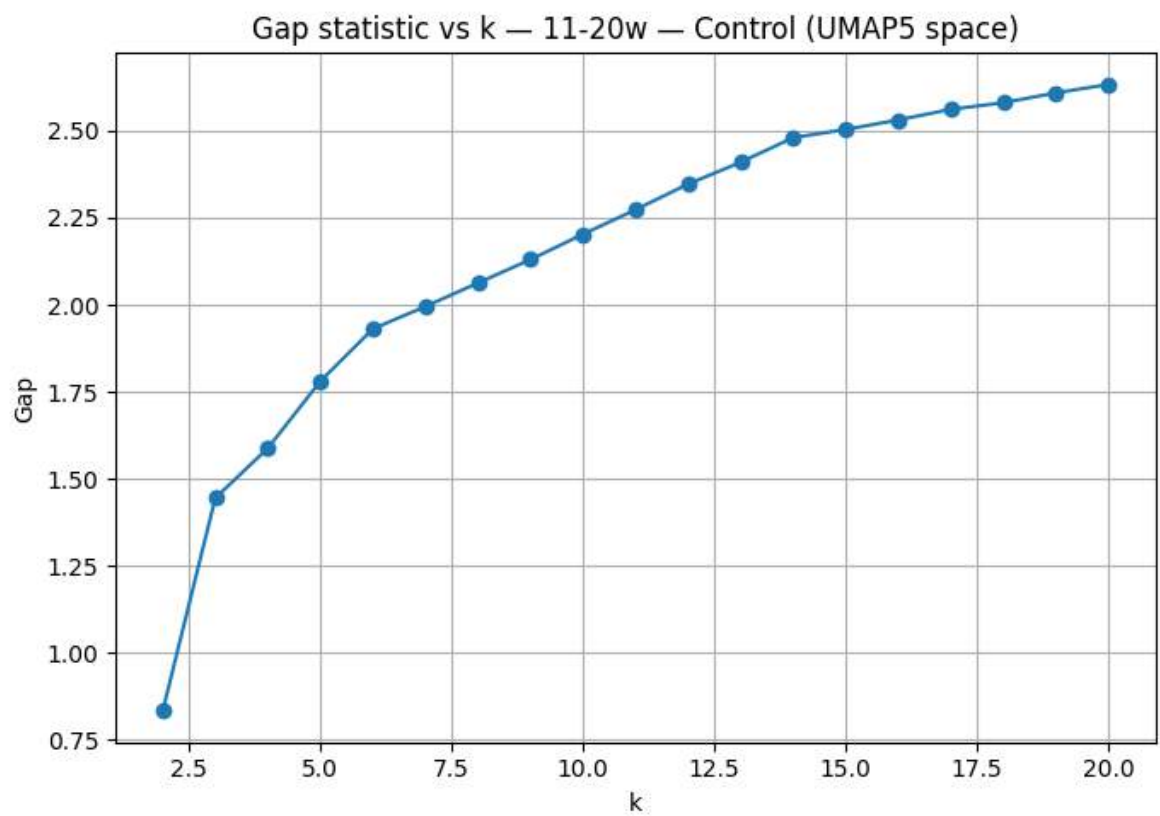
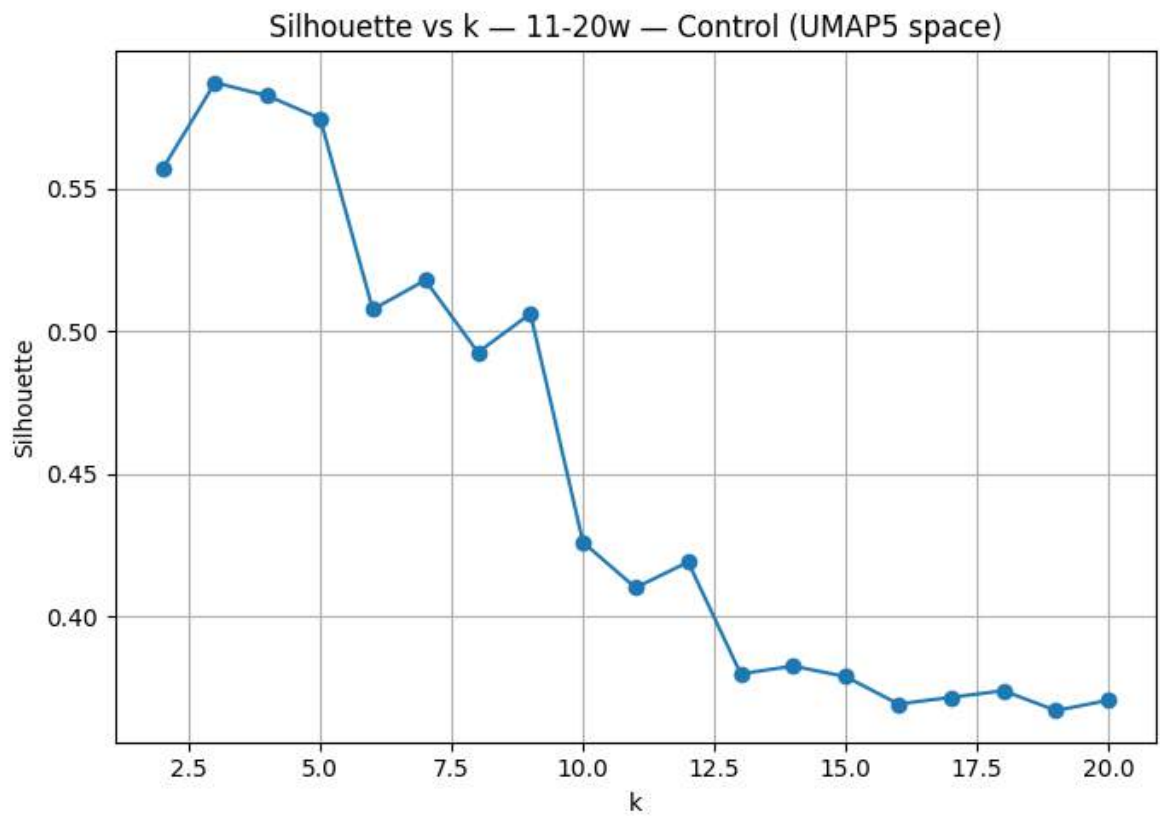




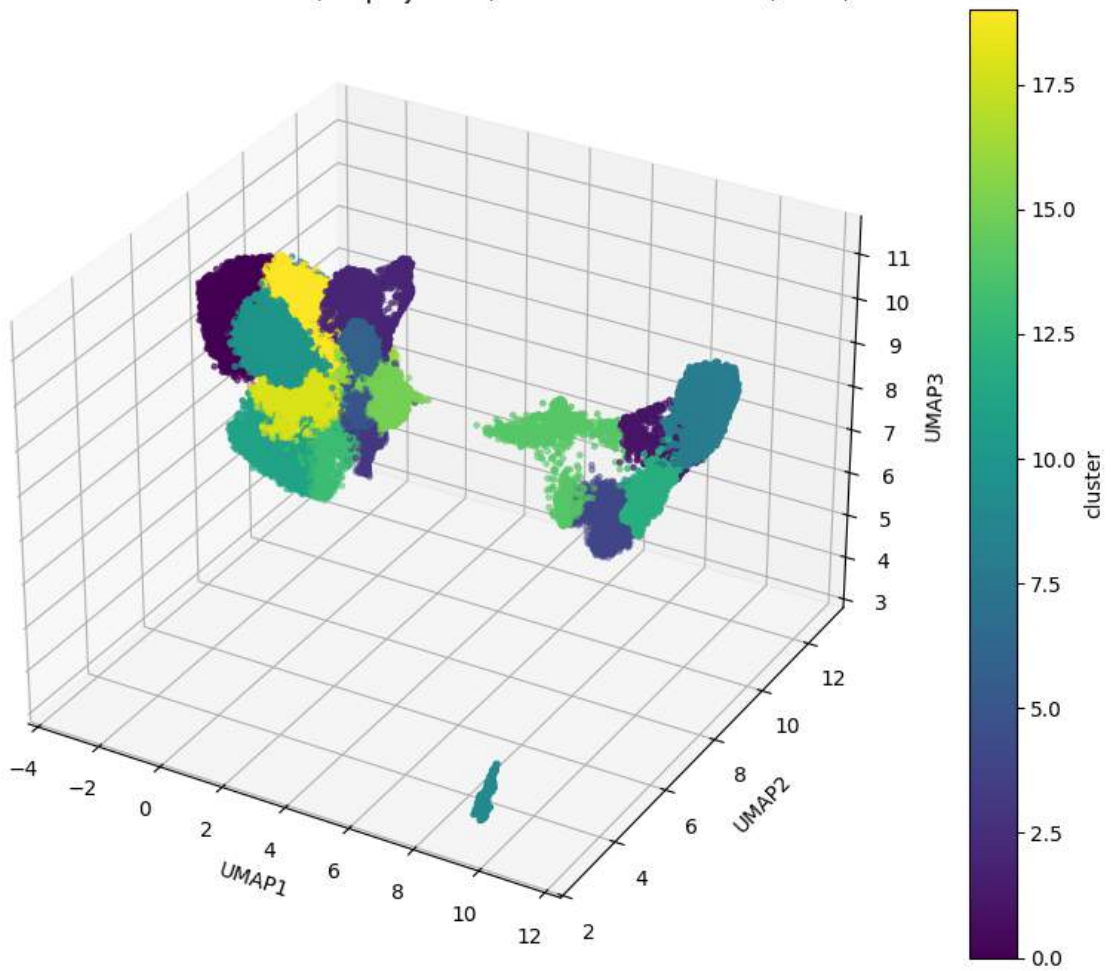


P2: KMeans in UMAP-3D — 11-20w — Control (k=7)

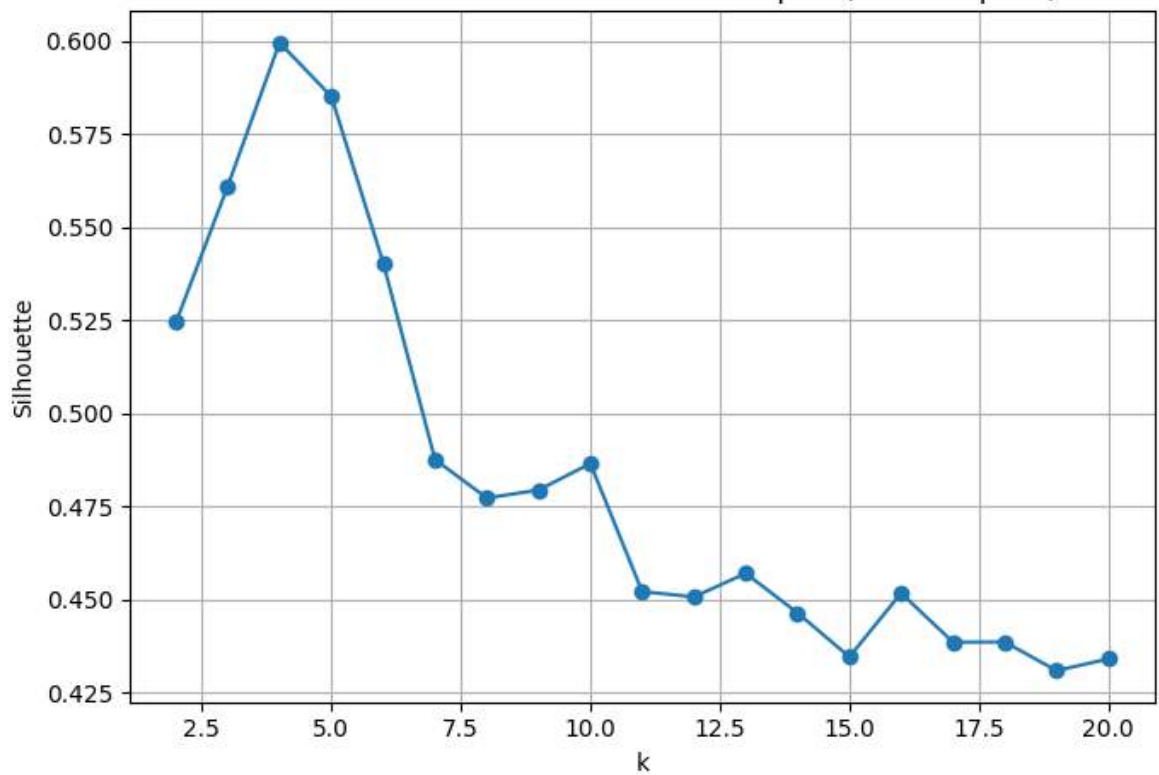


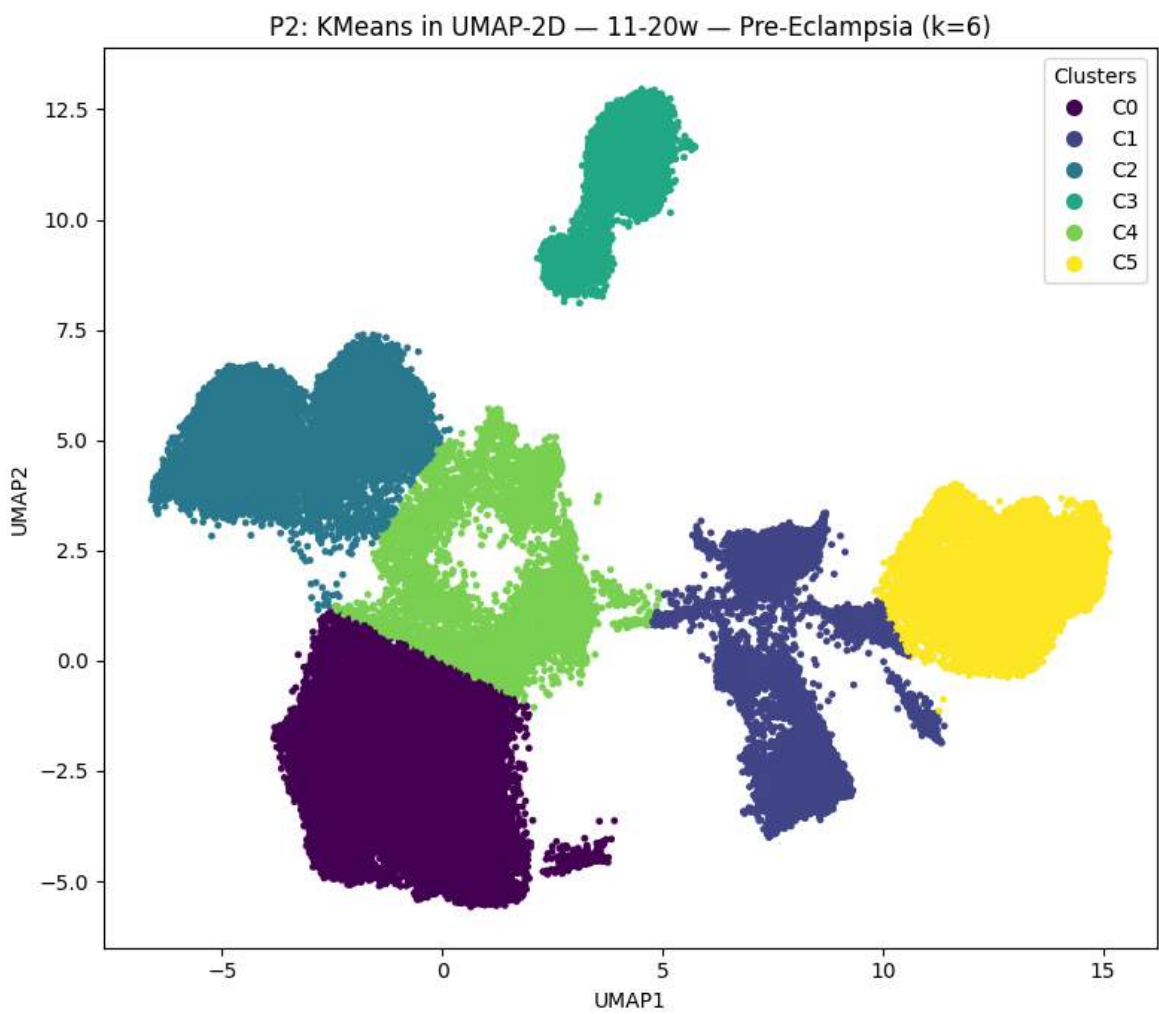
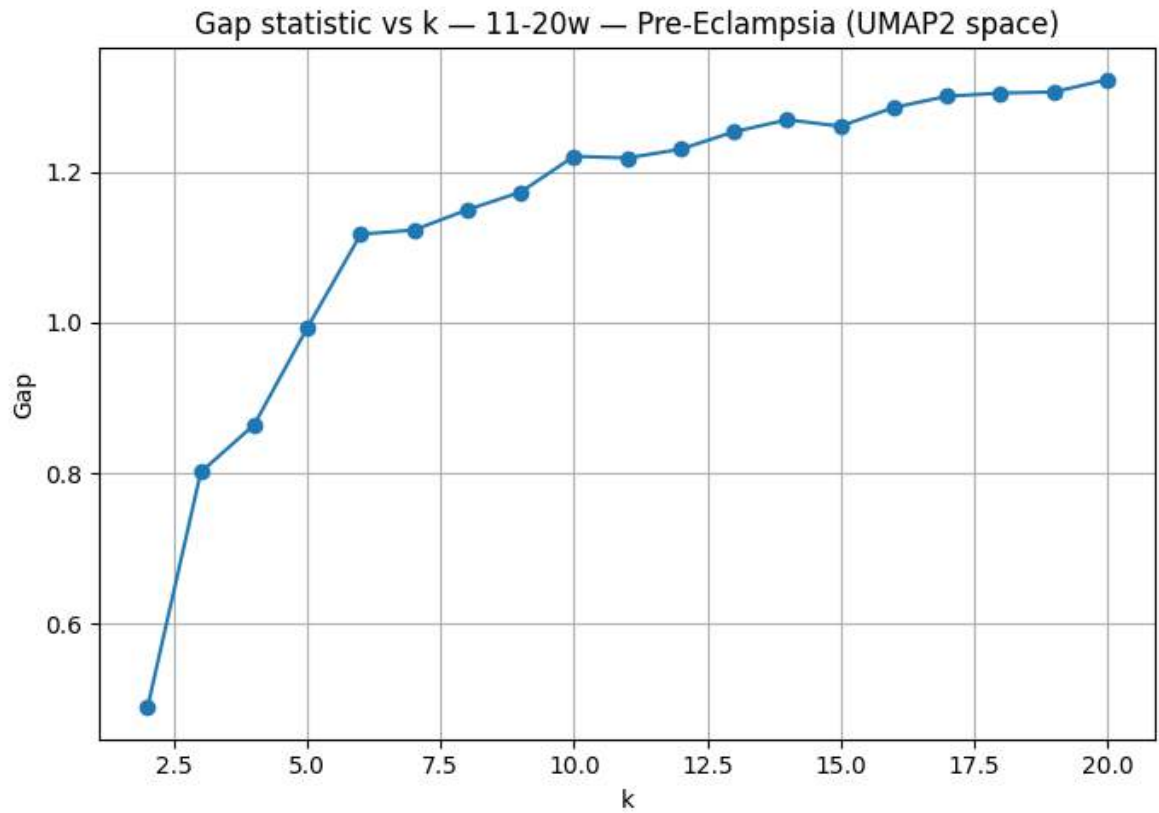


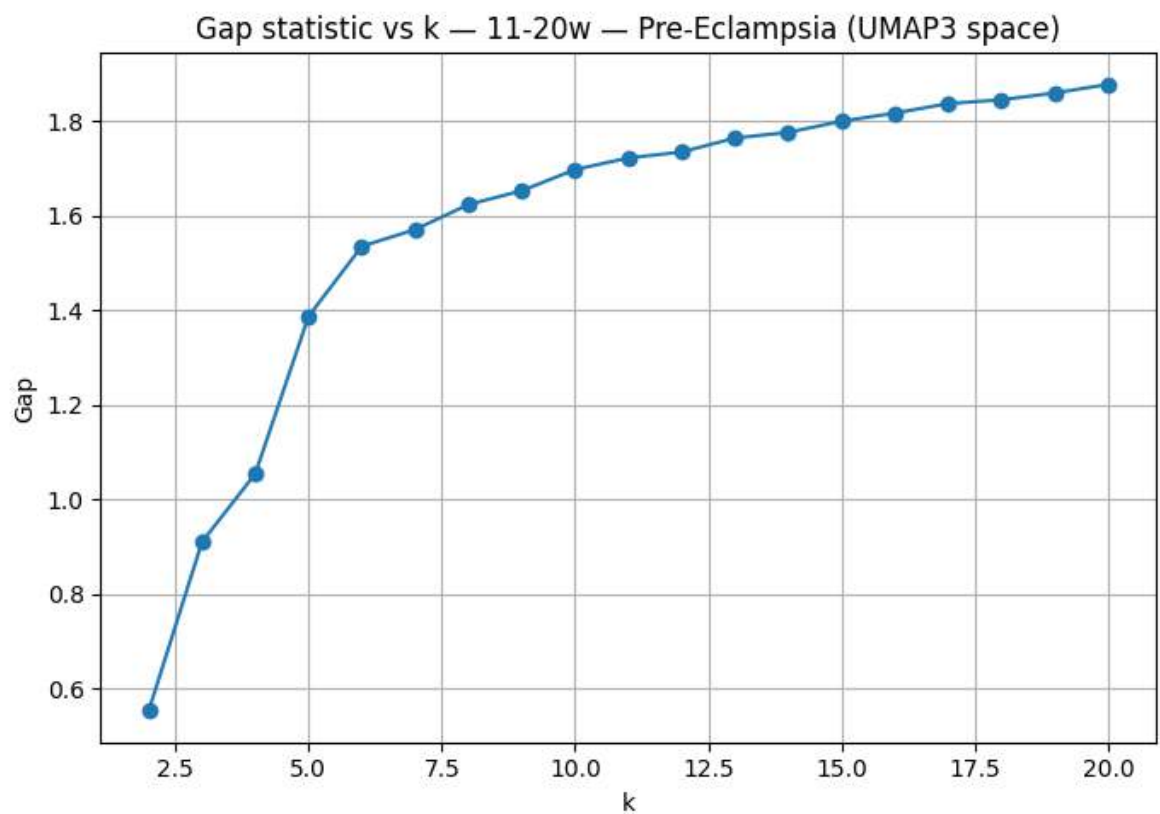
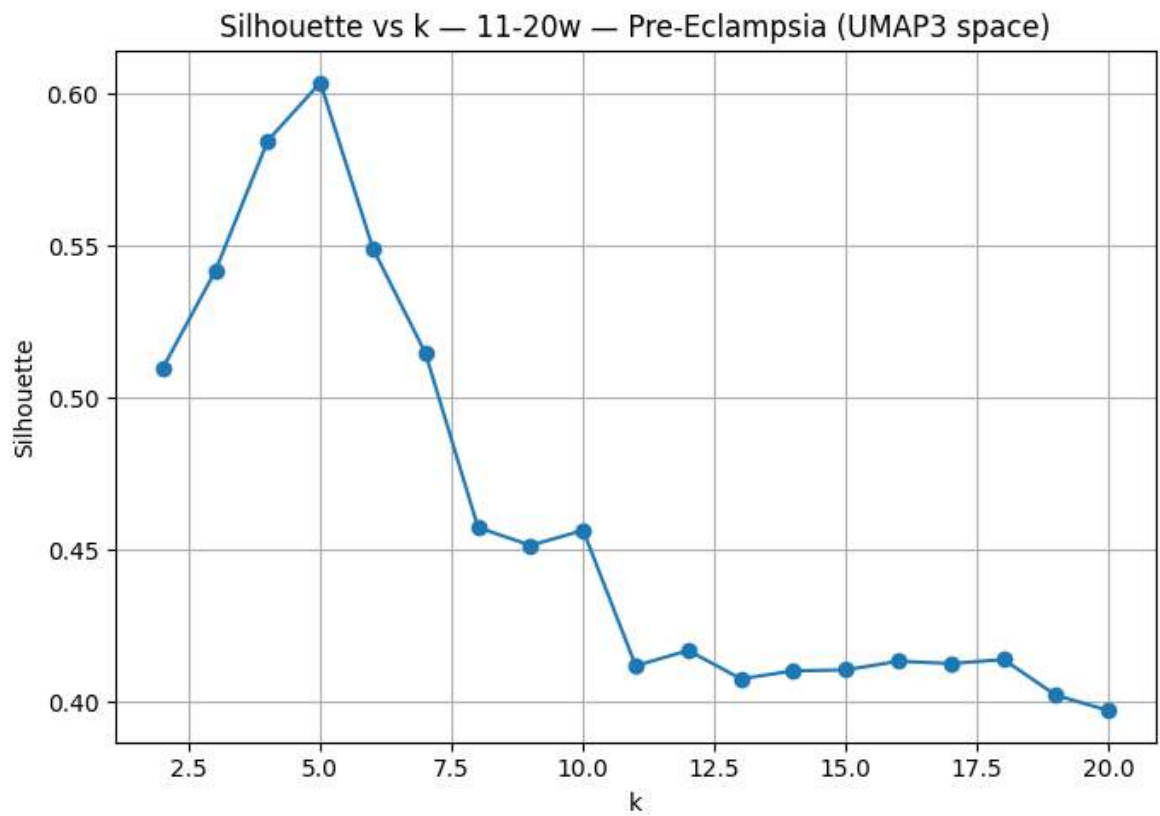
P2: KMeans in UMAP-5D (3D projection) — 11-20w — Control (k=20)



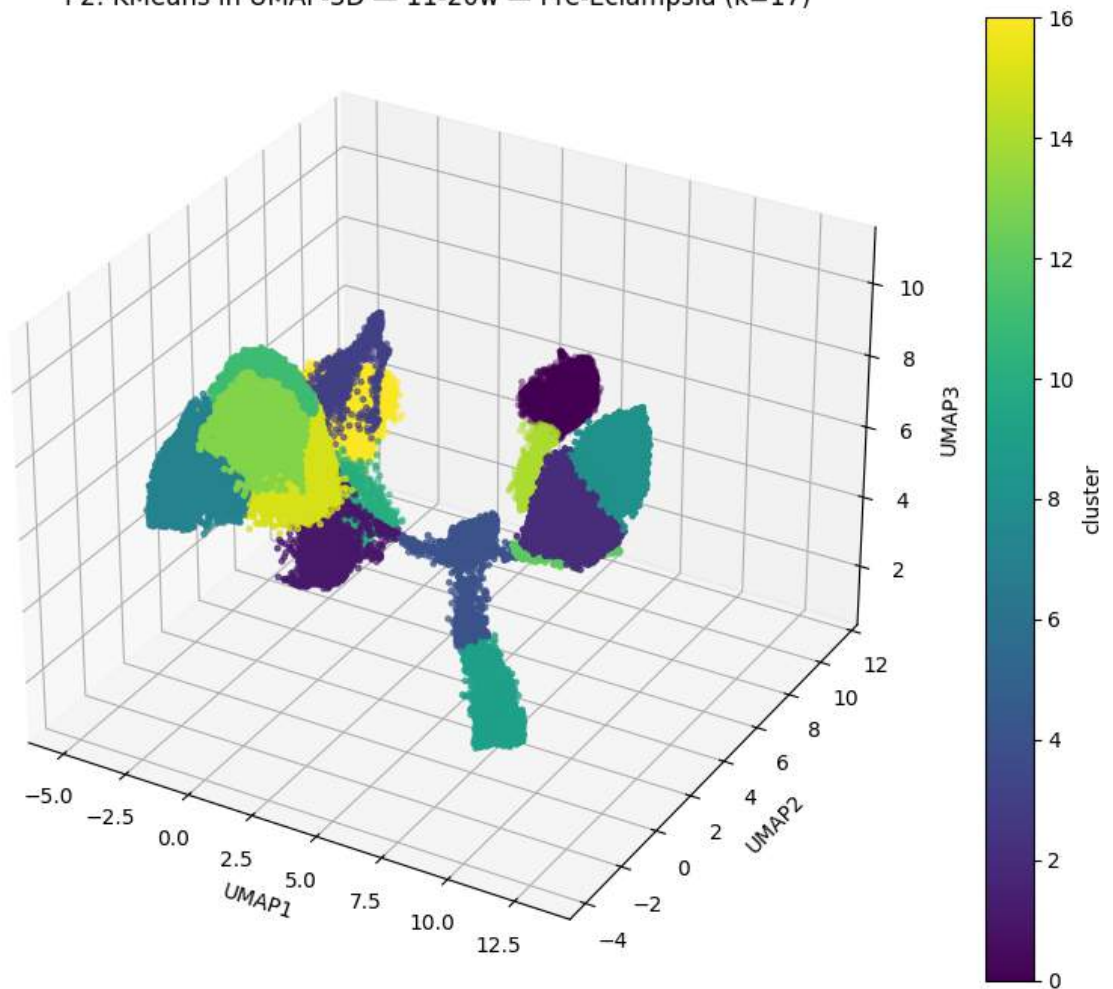
Silhouette vs k — 11-20w — Pre-Eclampsia (UMAP2 space)



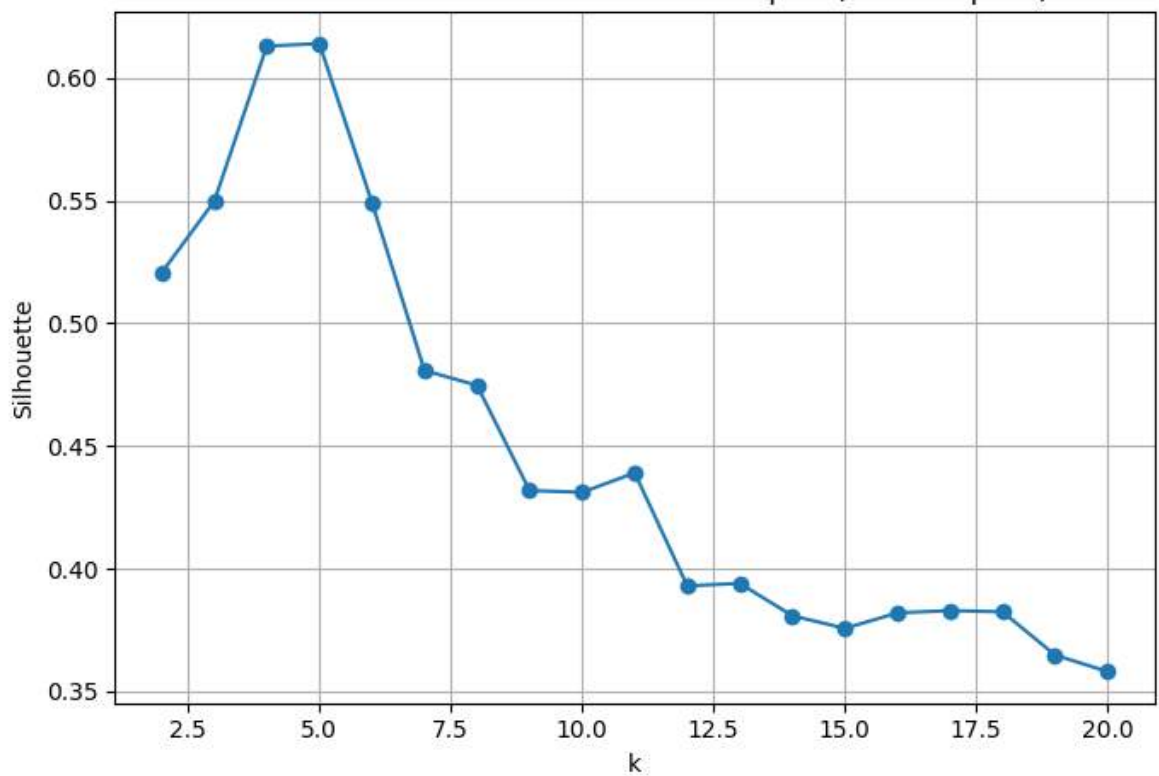


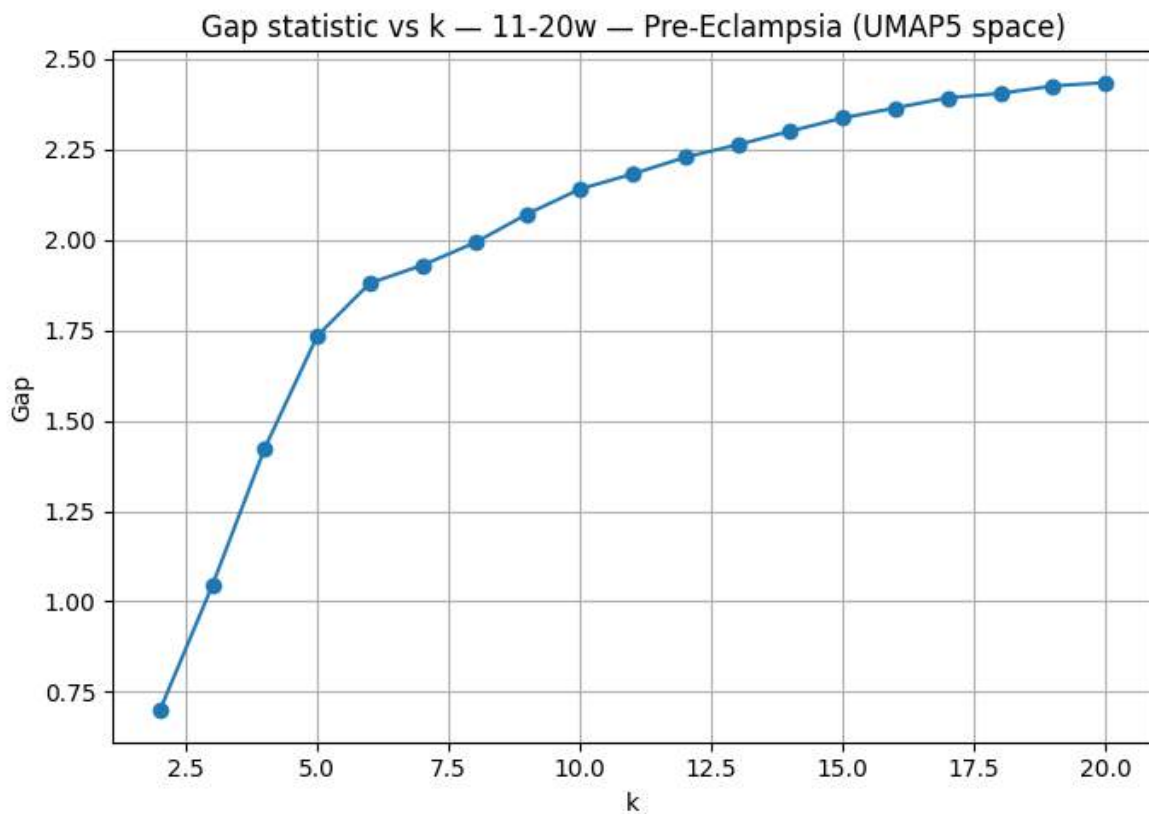


P2: KMeans in UMAP-3D — 11-20w — Pre-Eclampsia (k=17)

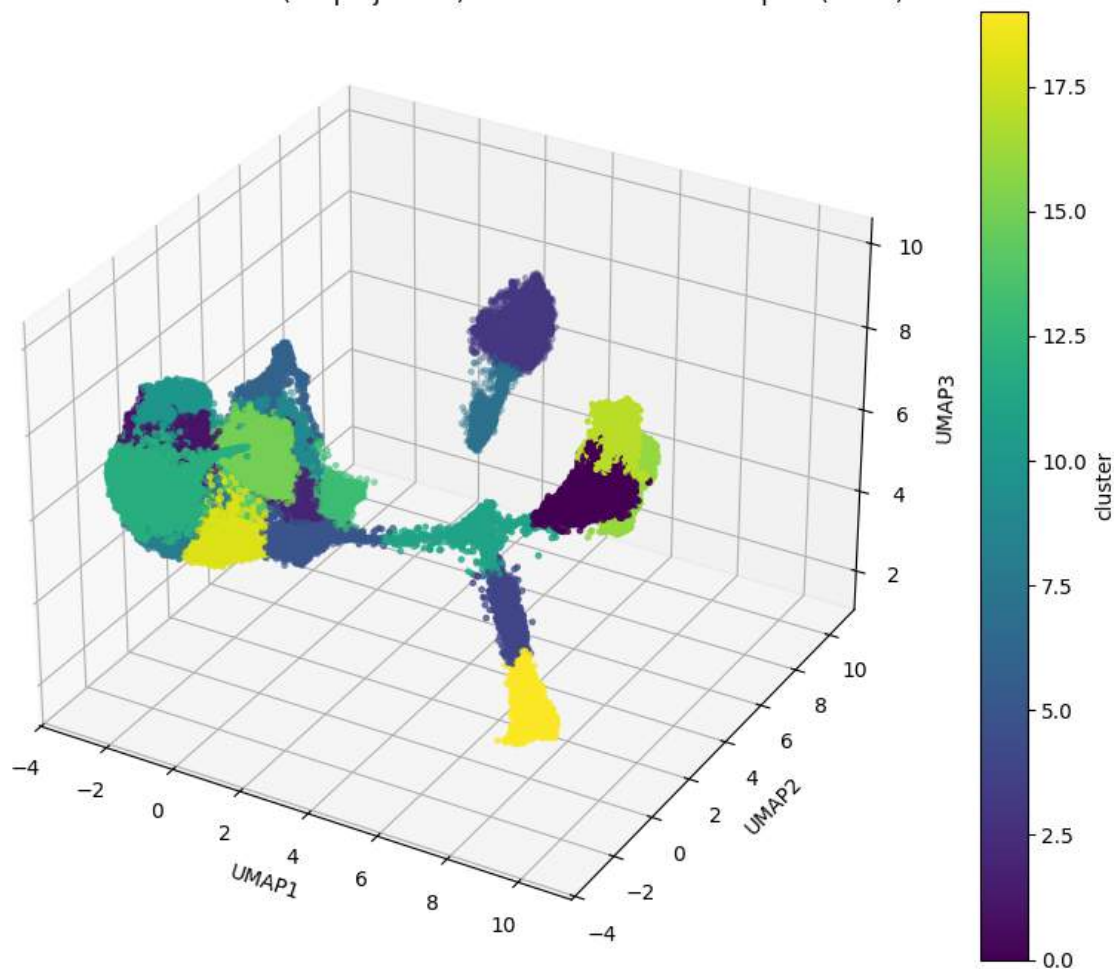


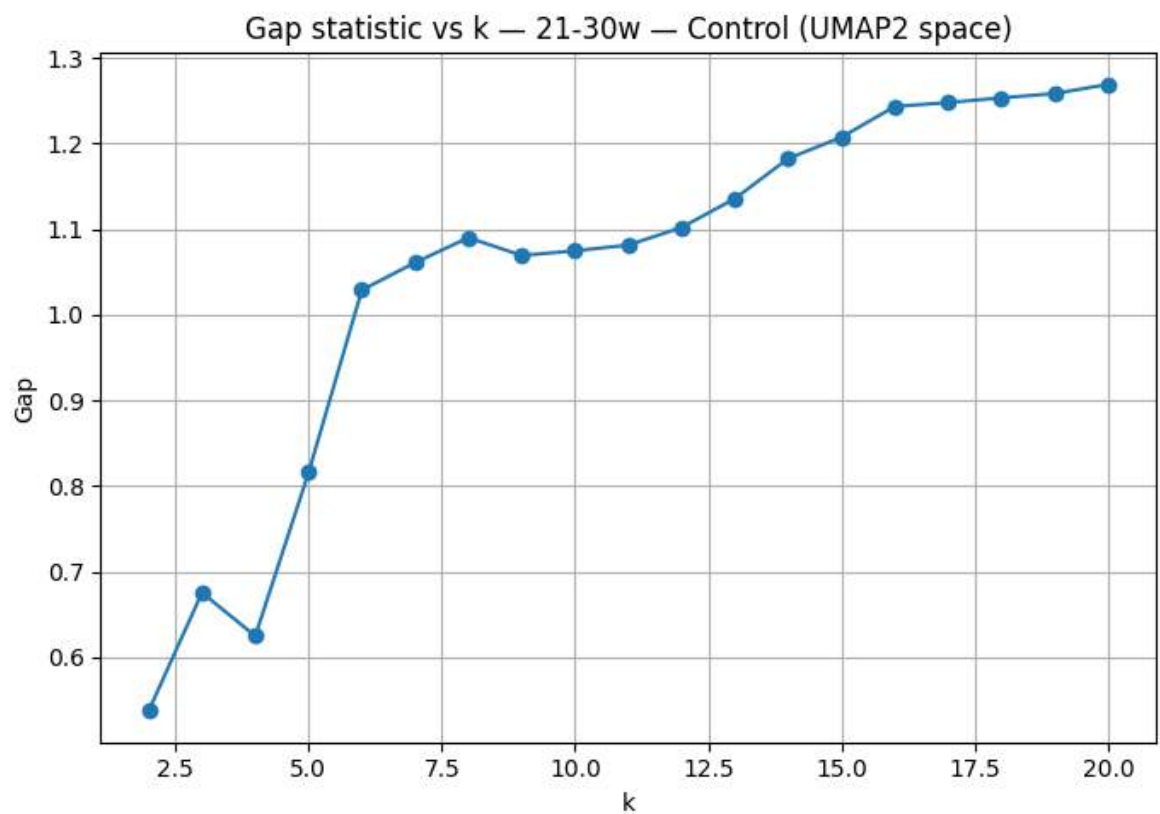
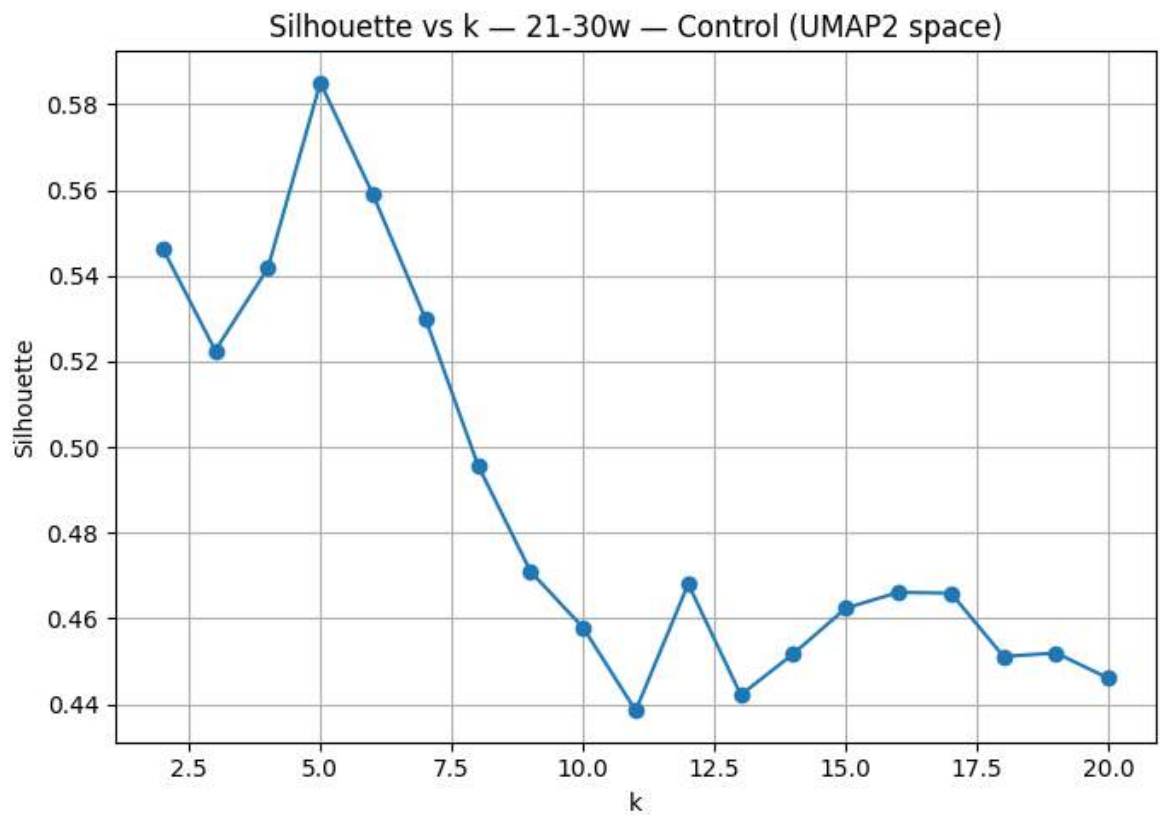
Silhouette vs k — 11-20w — Pre-Eclampsia (UMAP5 space)

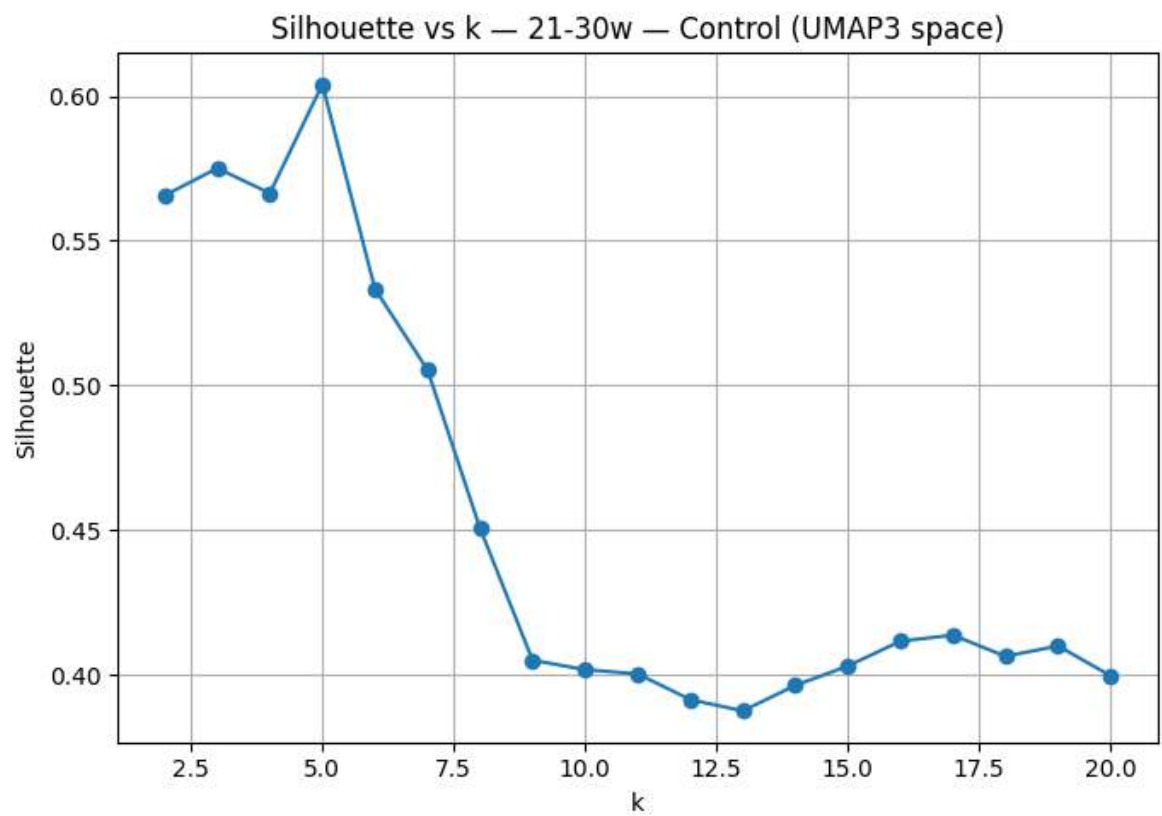
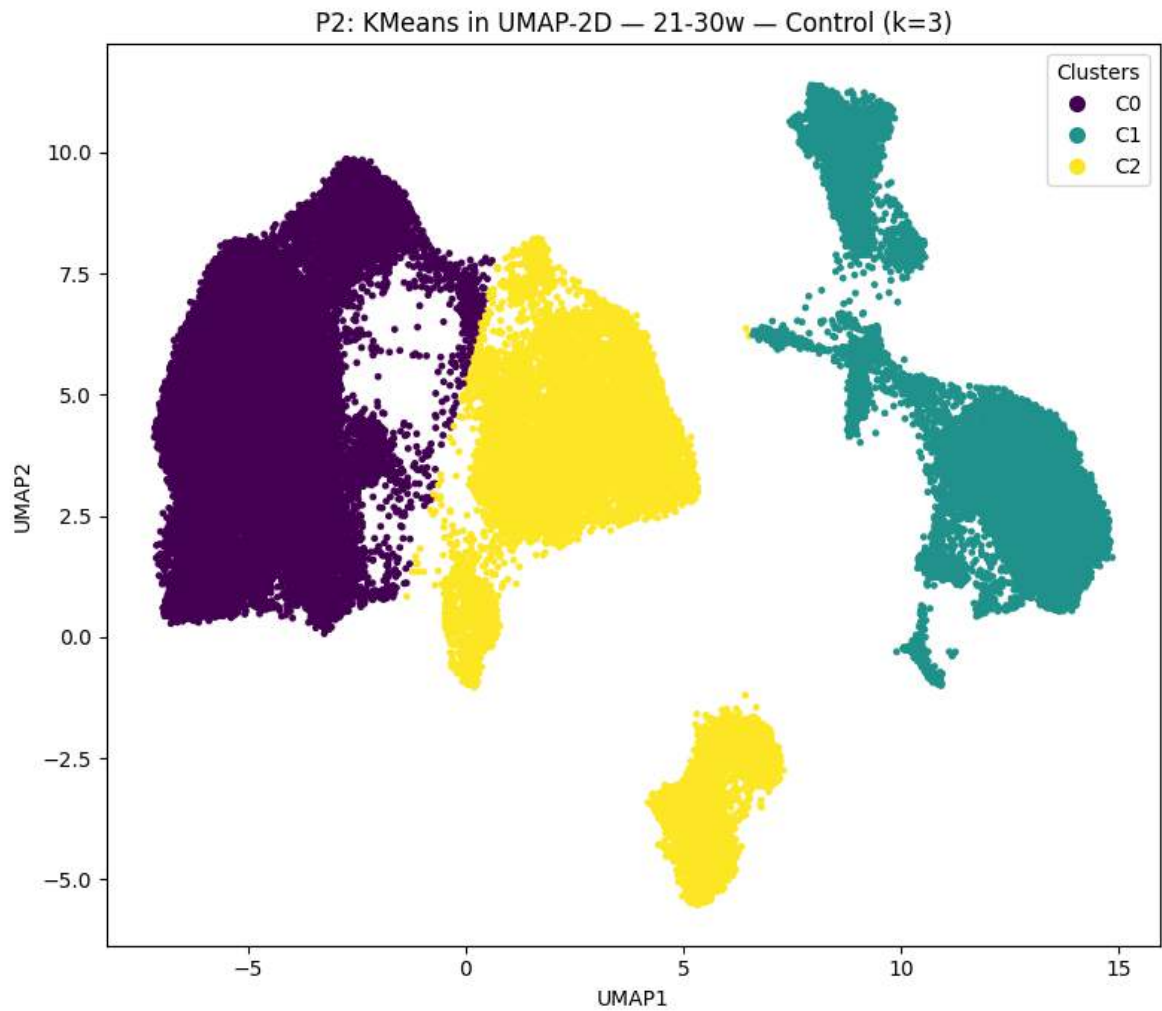


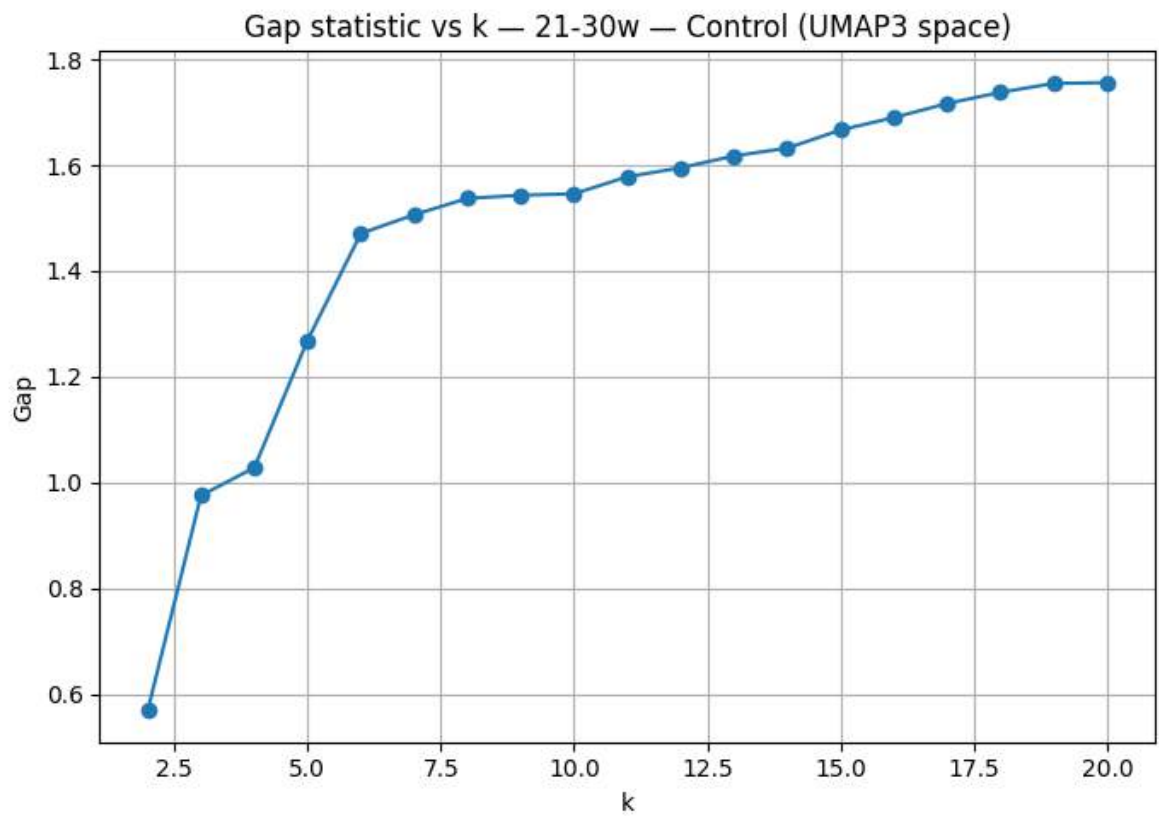


P2: KMeans in UMAP-5D (3D projection) — 11-20w — Pre-Eclampsia (k=20)

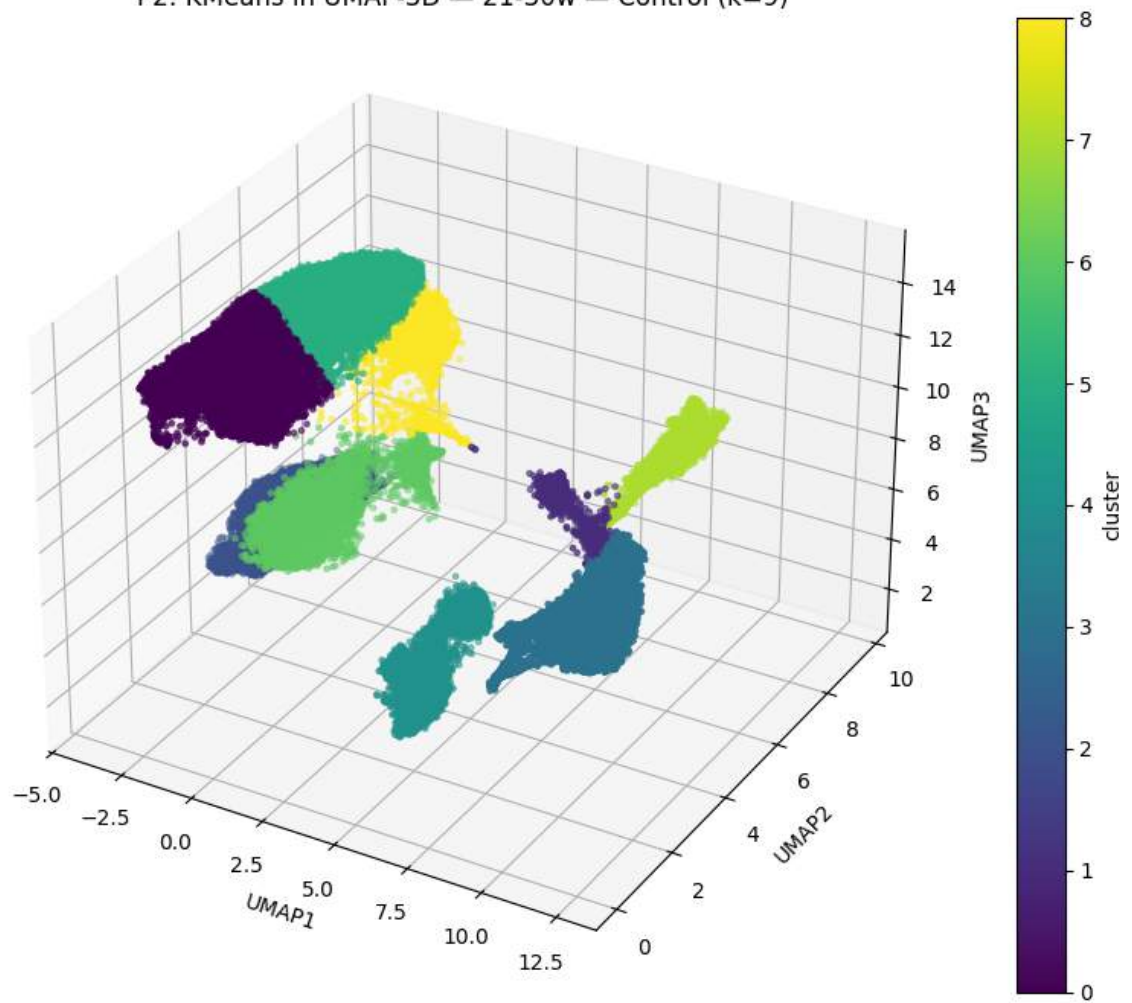


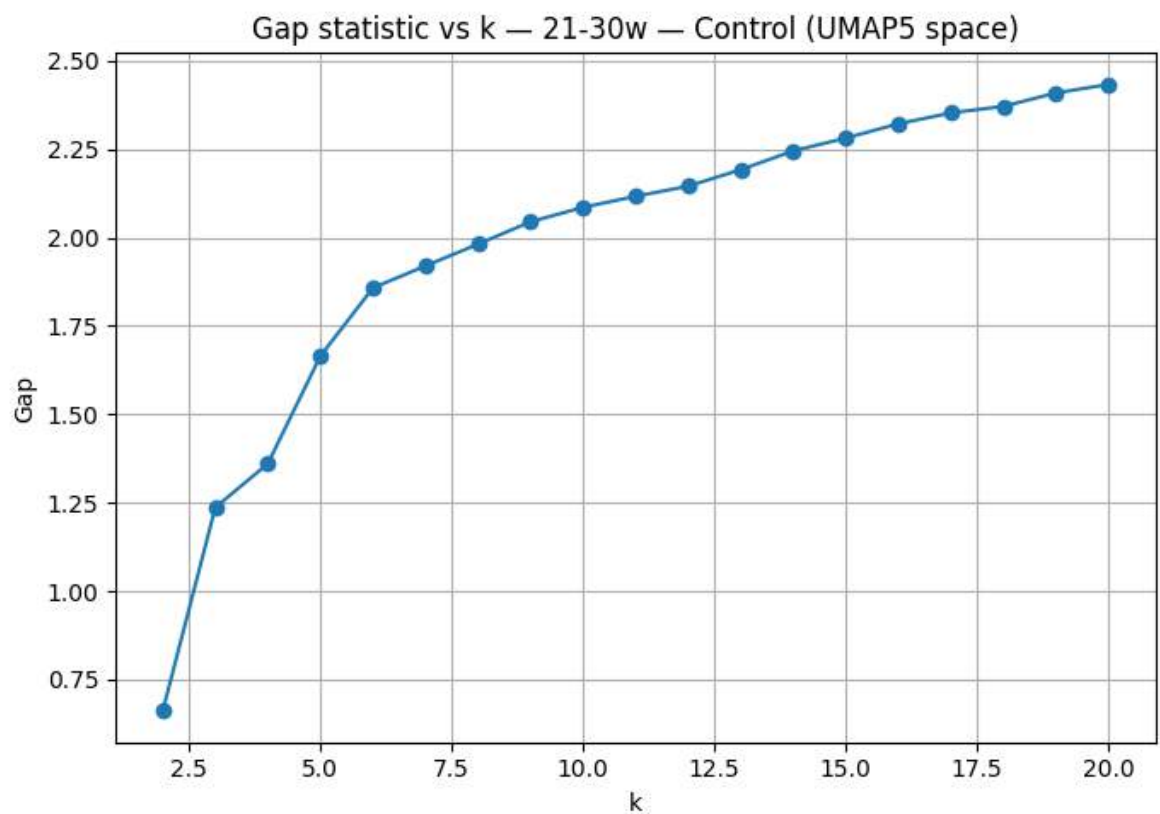
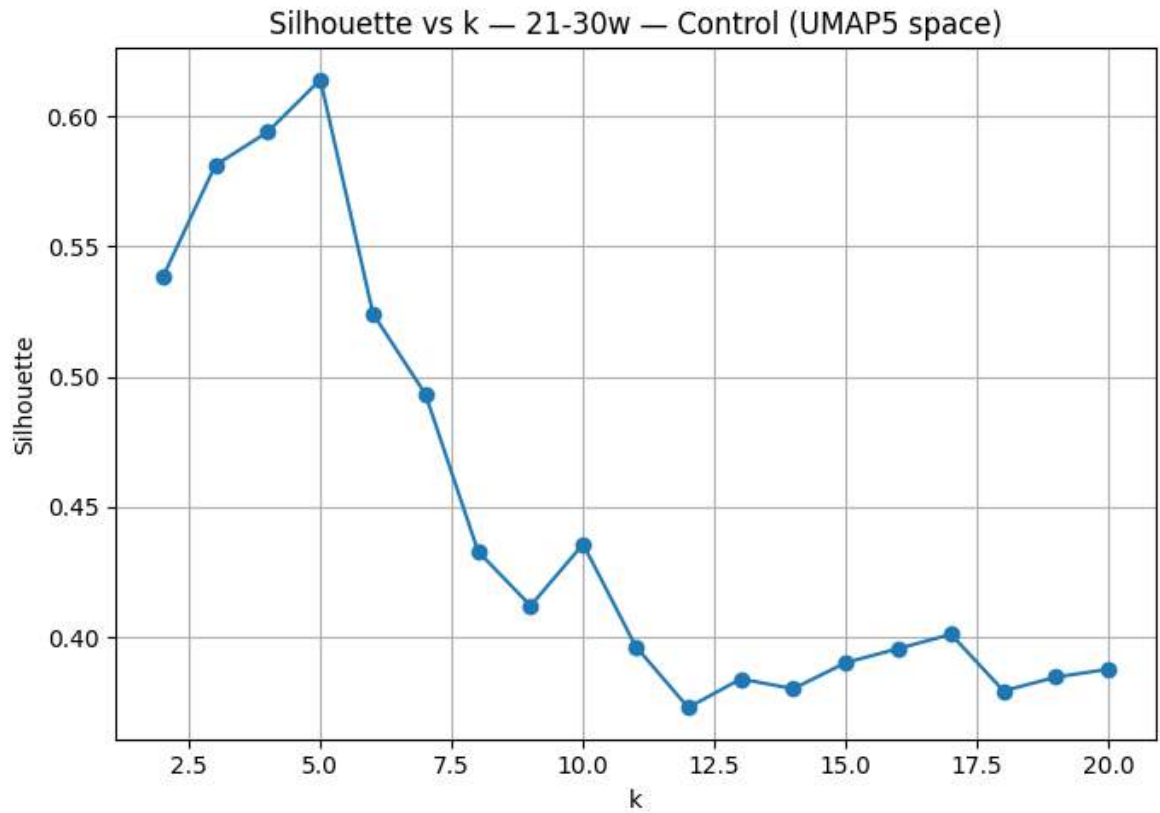




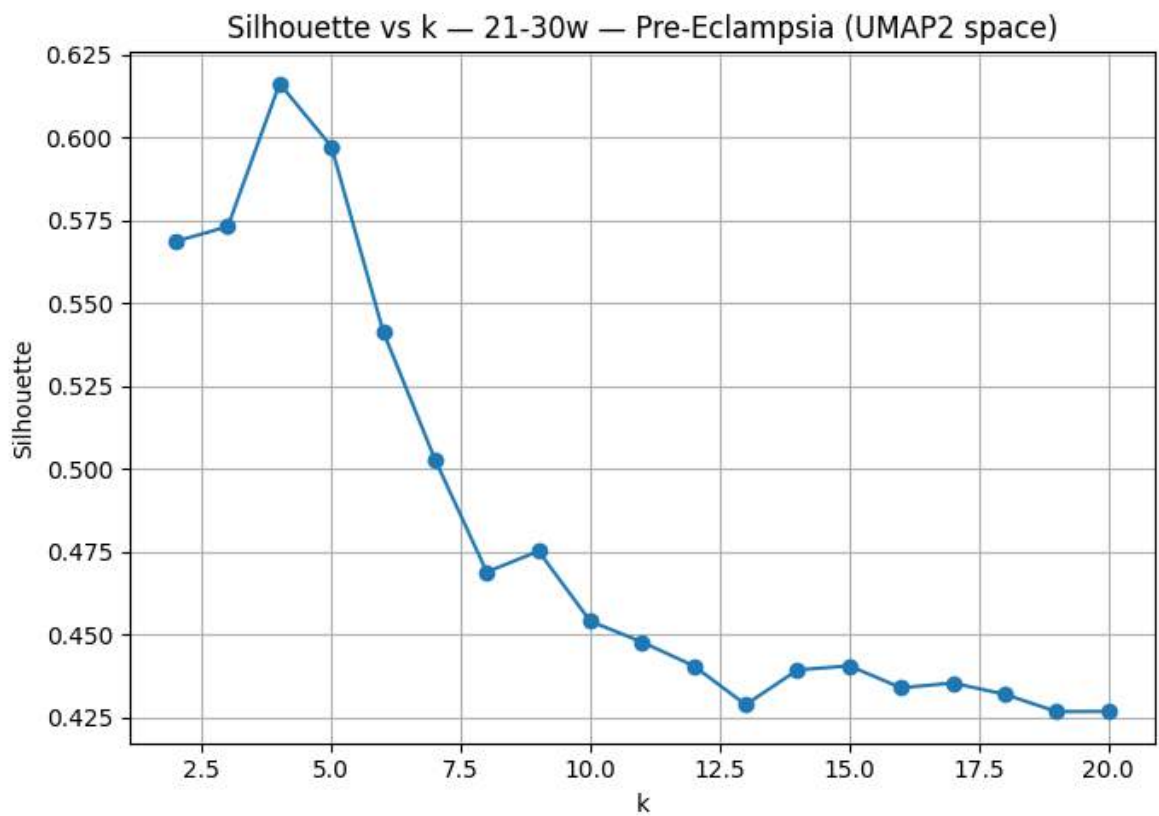
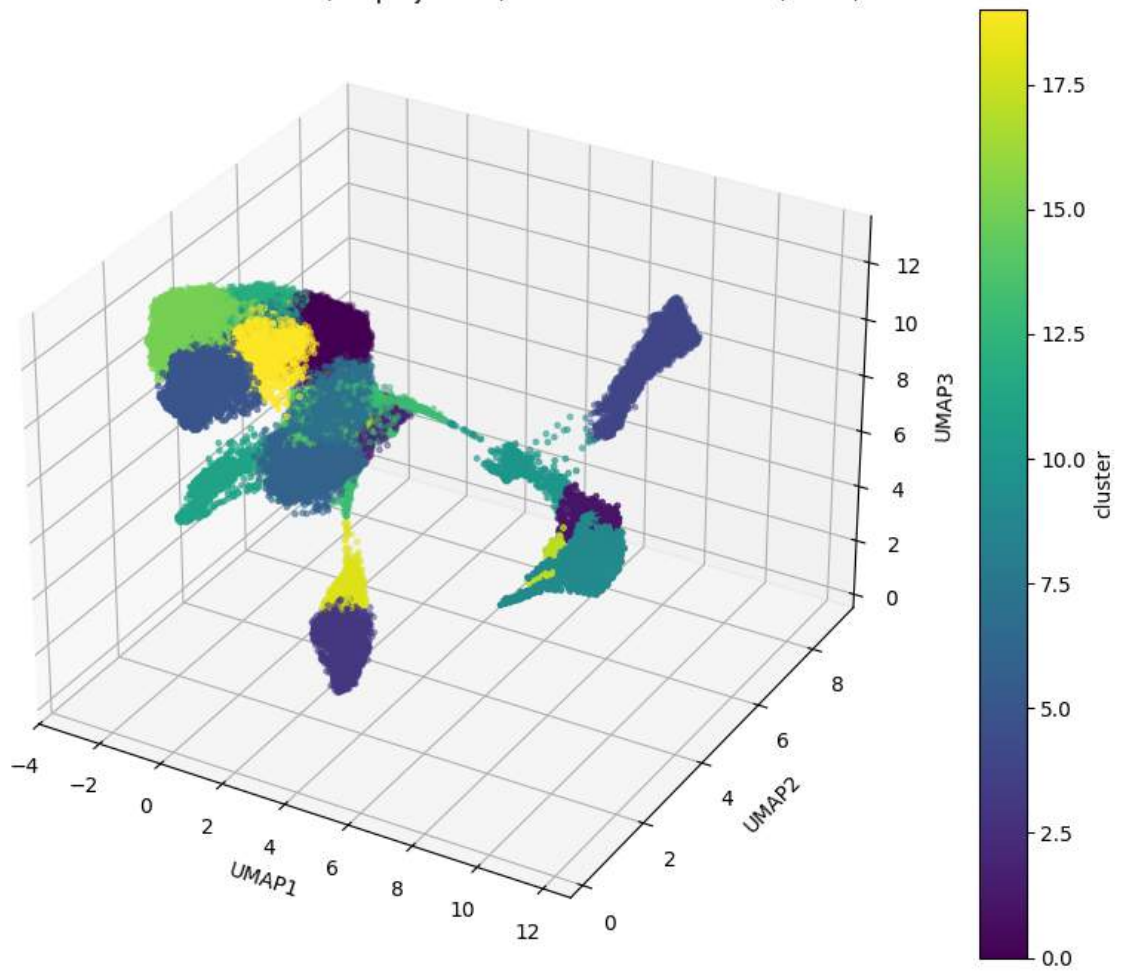


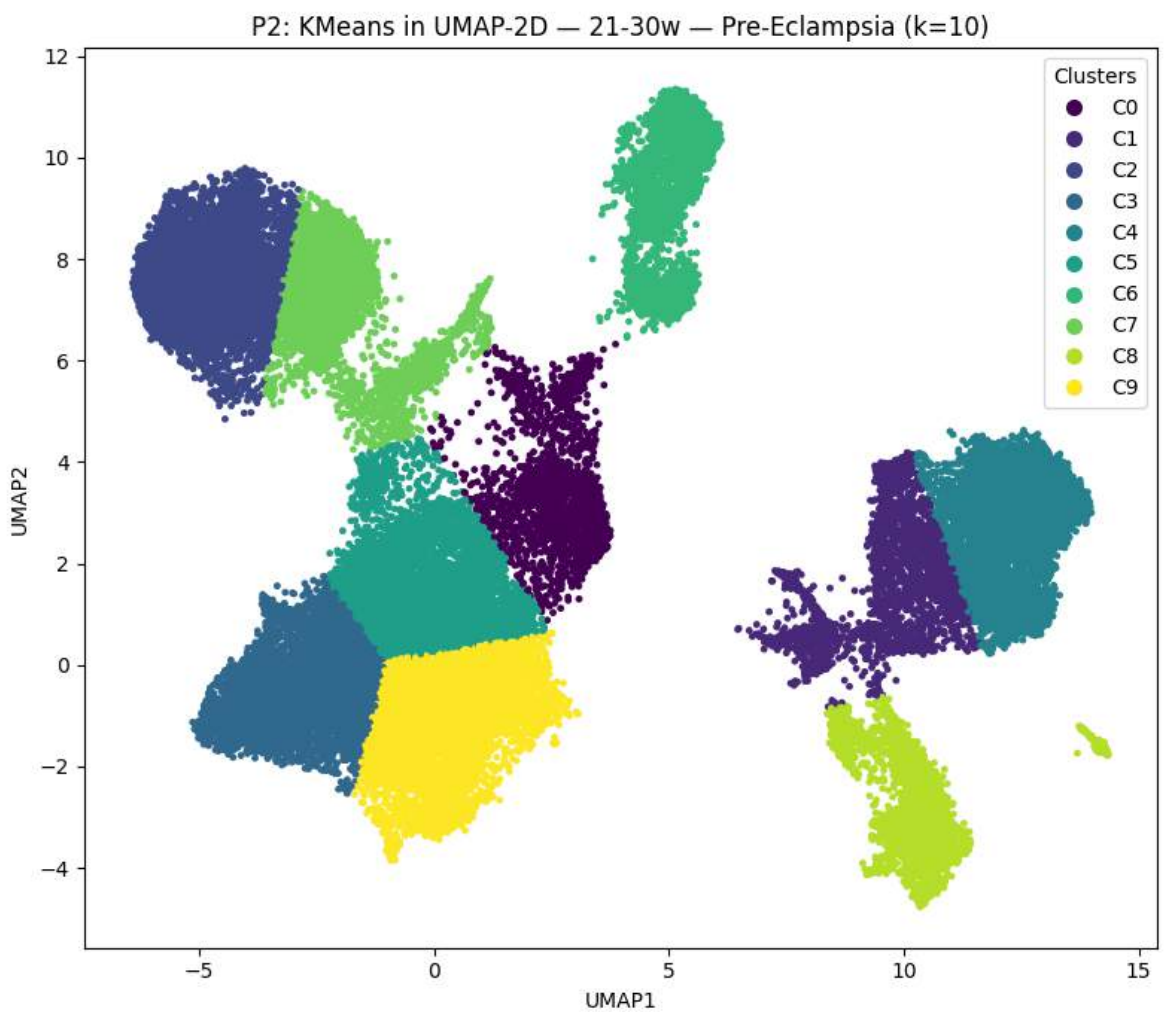
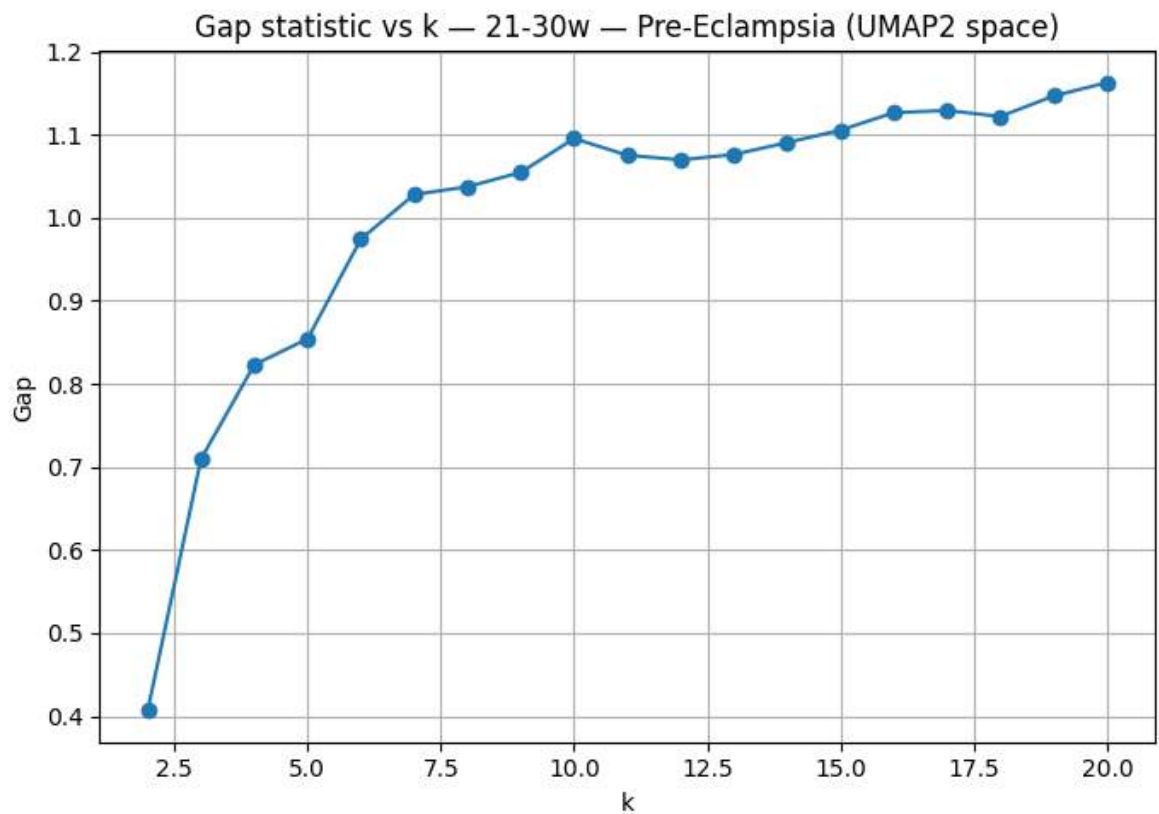
P2: KMeans in UMAP-3D — 21-30w — Control (k=9)

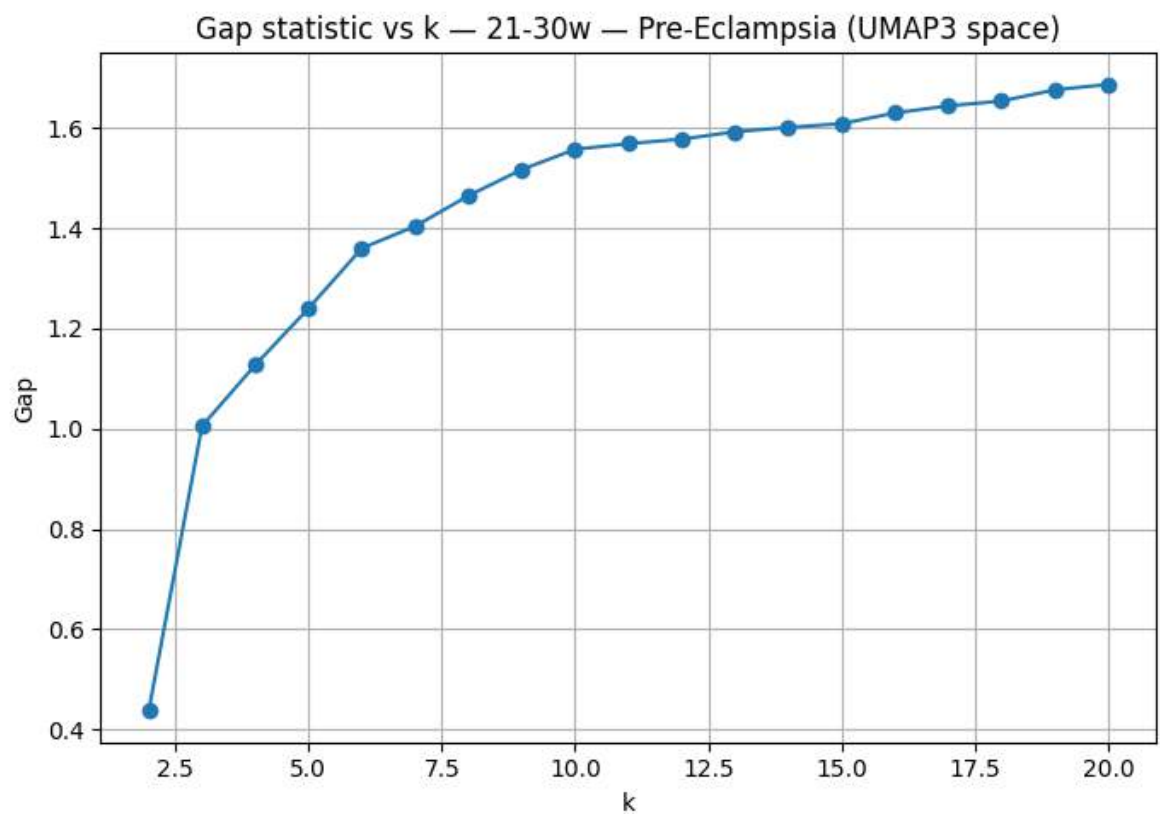
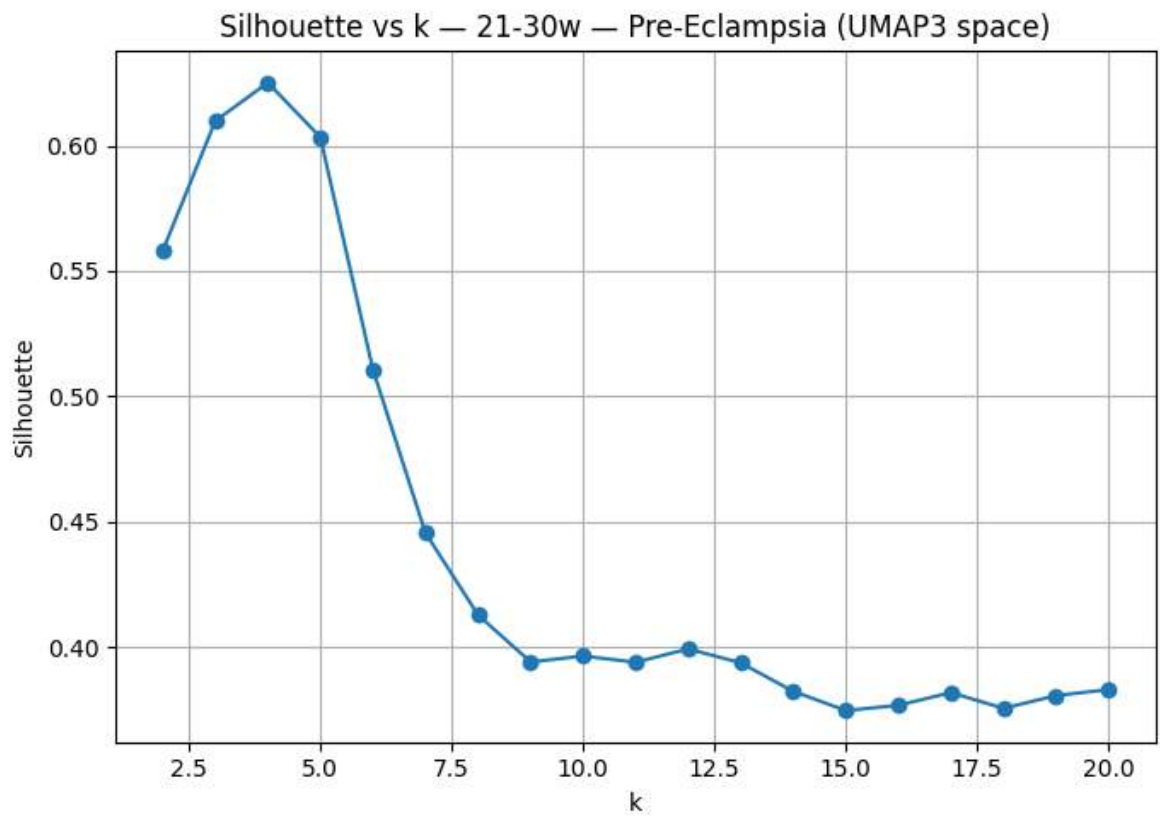




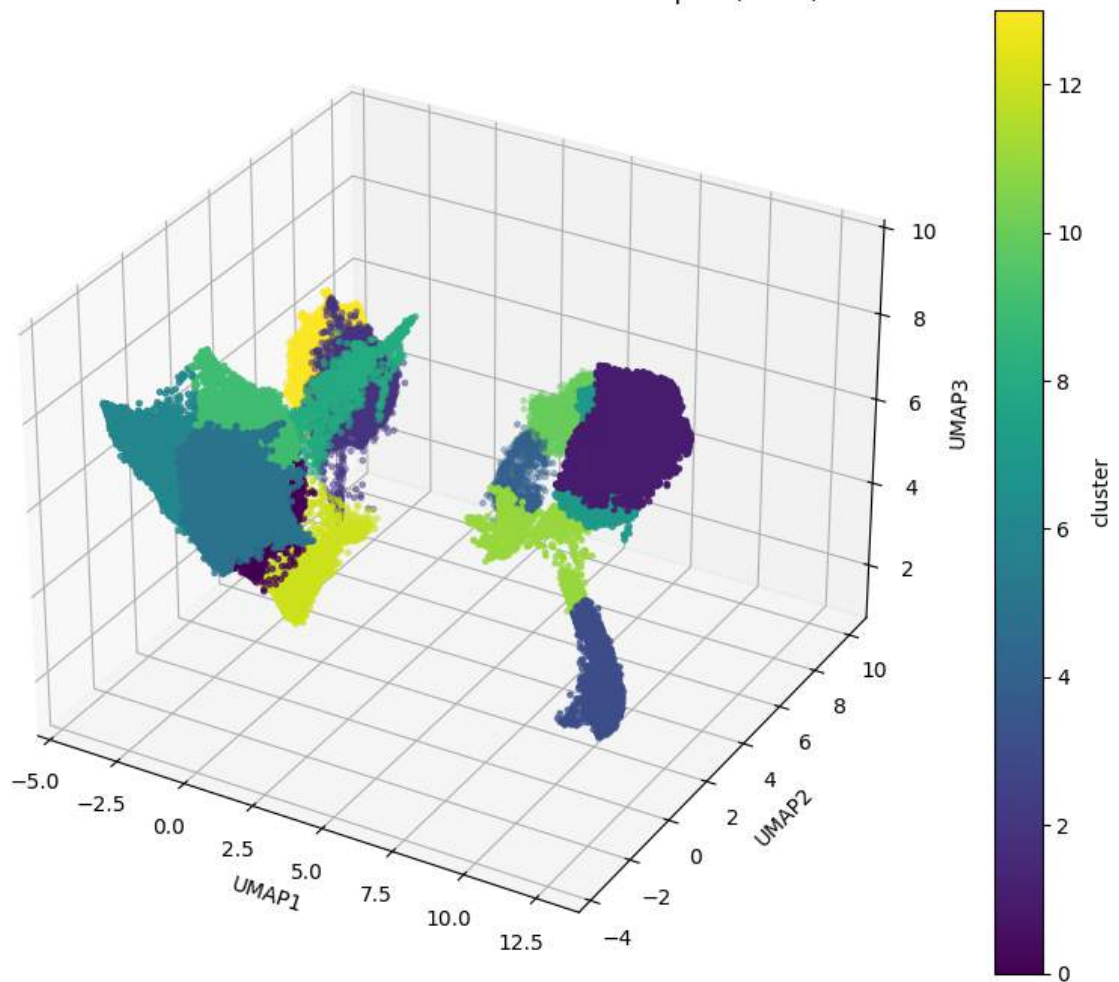
P2: KMeans in UMAP-5D (3D projection) — 21-30w — Control (k=20)



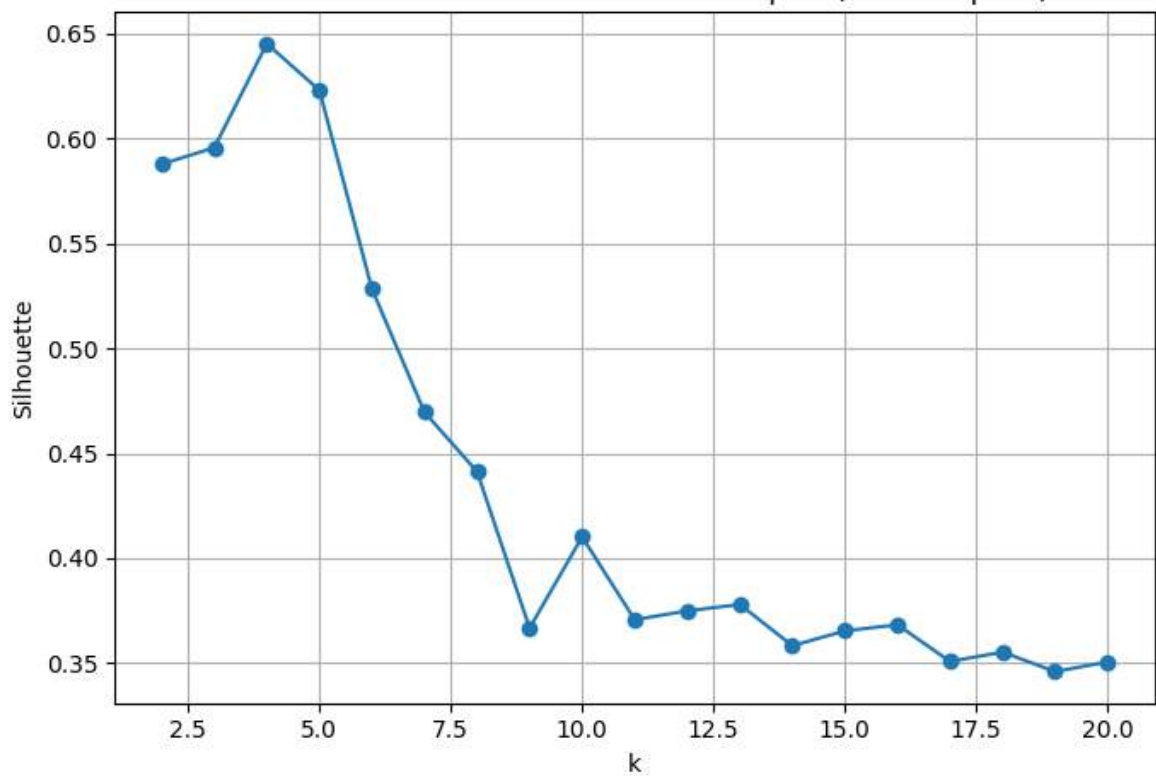


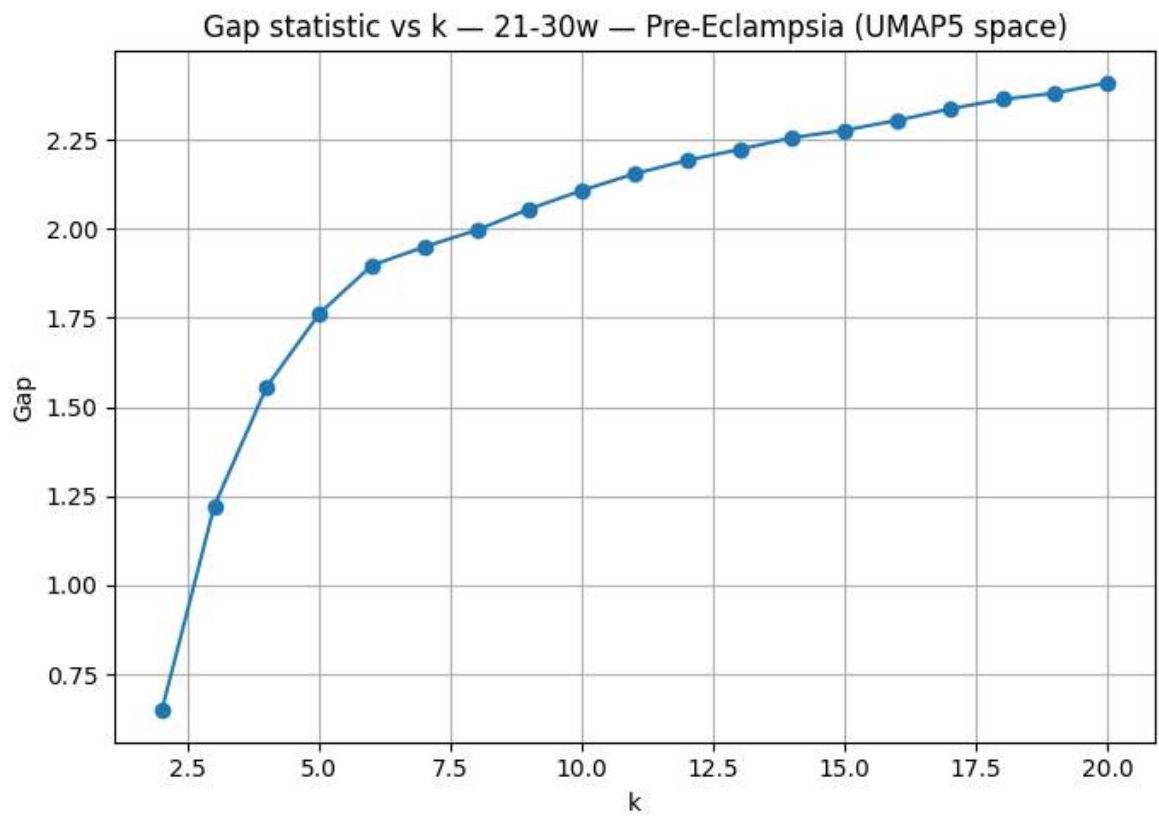


P2: KMeans in UMAP-3D — 21-30w — Pre-Eclampsia (k=14)

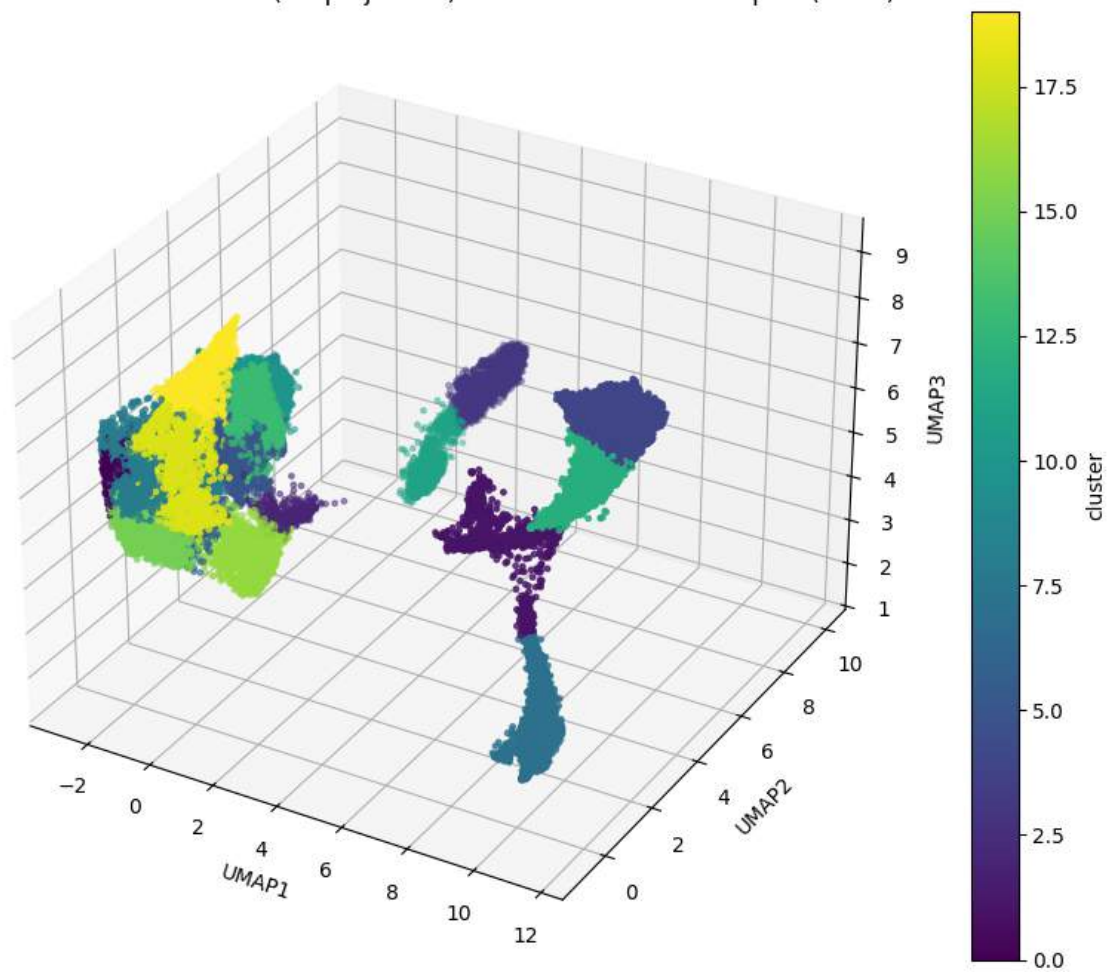


Silhouette vs k — 21-30w — Pre-Eclampsia (UMAP5 space)





P2: KMeans in UMAP-5D (3D projection) — 21-30w — Pre-Eclampsia (k=20)



Pipeline 2 complete. Summary saved to kmeans_p2_summary.csv

Out[]:

	interval	condition	pipeline	k_final	k_sil	k_gap	silhouette_at_k	
0	1-10w	0	Pipeline2(UMAP2)	3	4	3	0.517421	10w_con
1	1-10w	0	Pipeline2(UMAP3)	14	3	14	0.418949	10w_con
2	1-10w	0	Pipeline2(UMAP5)	20	3	20	0.375193	10w_con
3	1-10w	1	Pipeline2(UMAP2)	5	2	5	0.535680	10w_con
4	1-10w	1	Pipeline2(UMAP3)	17	2	17	0.424369	10w_con
5	1-10w	1	Pipeline2(UMAP5)	20	2	20	0.375717	10w_con
6	11-20w	0	Pipeline2(UMAP2)	4	2	4	0.563179	20w_con
7	11-20w	0	Pipeline2(UMAP3)	7	3	7	0.482647	20w_con
8	11-20w	0	Pipeline2(UMAP5)	20	3	20	0.364061	20w_con
9	11-20w	1	Pipeline2(UMAP2)	6	4	6	0.549438	20w_con
10	11-20w	1	Pipeline2(UMAP3)	17	5	17	0.397770	20w_con
11	11-20w	1	Pipeline2(UMAP5)	20	5	20	0.366248	20w_con
12	21-30w	0	Pipeline2(UMAP2)	3	5	3	0.521461	30w_con
13	21-30w	0	Pipeline2(UMAP3)	9	5	9	0.433434	30w_con
14	21-30w	0	Pipeline2(UMAP5)	20	5	20	0.391174	30w_con
15	21-30w	1	Pipeline2(UMAP2)	10	4	10	0.441152	30w_con
16	21-30w	1	Pipeline2(UMAP3)	14	4	14	0.374304	30w_con
17	21-30w	1	Pipeline2(UMAP5)	20	4	20	0.348739	30w_con

It is clear that umap embeds produce way clearer and higher quality clusters

HDBSCAN

Pipeline 1 (PCA based clustering) UMAP only for visualization

```
In [ ]: # Defining file path and excluding non marker columns

output_dir = "/content/drive/MyDrive/preeclampsia_results"
Path(output_dir).mkdir(parents=True, exist_ok=True)

# columns that are not markers
NON_MARKER = {
    "condition", "age", "age_label", "filename", "age_bin_10w",
    "_umap1", "_umap2", "PCA_1", "PCA_2",
    "KMeans_Labels", # explicit
}

# keep only numeric columns that are not in NON_MARKER
numeric_cols = [c for c in eda.columns if pd.api.types.is_numeric_dtype(eda[c])]
MARKERS = [c for c in numeric_cols if c not in NON_MARKER]

print(f"Markers (n={len(MARKERS)}): {MARKERS[:10]}{' ...' if len(MARKERS)>10 else ''}")
Markers (n=33): ['CD45', 'CD66', 'CD7', 'CD19', 'CD45RA', 'CD11b', 'CD4', 'CD8a', 'CD11c', 'CD123'] ...
```

```
In [ ]: # Find age bins that contain data
valid_bins = [b for b in eda['age_bin_10w'].cat.categories if (eda['age_bin_10w']
print("Age bins with data:", list(map(str, valid_bins)))

COND_NAME = {0:'Control', 1:'Pre-Eclampsia'}
RANDOM_STATE = 42

def silhouette_nonnoise(X, labels):
    # Define a silhouette helper that ignores noise (-1)
    mask = labels != -1
    if mask.sum() < 3: return np.nan
    labs = labels[mask]
    if len(np.unique(labs)) < 2: return np.nan
    return silhouette_score(X[mask], labs)

def plot_umap2(emb2, labels, title, out_png):
    # d plot 2D UMAP colored by cluster (noise in grey)
    plt.figure(figsize=(8,7))
    noise = labels == -1
    if (~noise).any():
        sc = plt.scatter(emb2[~noise,0], emb2[~noise,1], c=labels[~noise], s=6,
    else:
        sc = None
    if noise.any():
        plt.scatter(emb2[noise,0], emb2[noise,1], c="lightgrey", s=5, label="Noi
        plt.legend(frameon=True)
    plt.title(title); plt.xlabel("UMAP1"); plt.ylabel("UMAP2")
    if sc is not None: plt.colorbar(sc, label="Cluster ID")
    plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.show()
```

Age bins with data: ['1-10w', '11-20w', '21-30w']

PCA20 Standardization and noise reduction

```
In [ ]: for interval in valid_bins:
        for cond in [0,1]:
```

```

sub = eda[(eda['age_bin_10w']==interval) & (eda['condition']==cond)]
if sub.empty:
    continue
tag = f"{interval}_{cond}"
pfile = os.path.join(output_dir, f"pca20_{tag}.npz")
if os.path.exists(pfile):
    print("PCA20 cached:", tag);
    continue
X = sub[MARKERS].to_numpy(dtype="float32")
Xs = StandardScaler().fit_transform(X)
X_pca20 = PCA(n_components=20, random_state=RANDOM_STATE).fit_transform(
    np.save(pfile, X_pca20)
print("PCA20 saved:", tag, X_pca20.shape)

```

PCA20 cached: 1-10w_0

PCA20 cached: 1-10w_1

PCA20 cached: 11-20w_0

PCA20 cached: 11-20w_1

PCA20 cached: 21-30w_0

PCA20 cached: 21-30w_1

Pipeline 1 — HDBSCAN in PCA-20, visualize with UMAP-2D

```

In [ ]: # Cell 2 - HDBSCAN on PCA-20 (Pipeline 1)
p1_rows = []

# Pipeline 1 (PCA-20 space)
def grid_for_n(n, pct=(0.002, 0.005, 0.01)):
    """
    mcs grid = absolute sizes U percentage-of-n (both clamped to [5, 80] and ≤ r
    pct defaults: 0.2%, 0.5%, 1.0%
    """
    base_abs = [10, 20, 40, 60]
    base_from_pct = [int(np.ceil(p * n)) for p in pct]

    mcs = sorted({
        m for m in (base_abs + base_from_pct)
        if 5 <= m <= min(80, n)
    })

    minsamples = [1, 3, 5, 10]
    methods = ["eom", "leaf"]
    return mcs, minsamples, methods

for interval in valid_bins:
    for cond in [0,1]:
        sub = eda[(eda['age_bin_10w']==interval) & (eda['condition']==cond)]
        if sub.empty:
            continue
        tag = f"{interval}_{cond}"
        X_pca20 = np.load(os.path.join(output_dir, f"pca20_{tag}.npz"))
        n = len(X_pca20)
        if n < 50:
            print(f"[P1] {tag}: too few cells ({n})");
            continue

        best = None
        mcs_list, mns_list, meth_list = grid_for_n(n)
        for mcs in mcs_list:
            for mns in mns_list:

```

```

for meth in meth_list:
    model = hdbscan.HDBSCAN(
        min_cluster_size=mcs,
        min_samples=mns if mns is not None else None,
        cluster_selection_method=meth,
        core_dist_n_jobs=1
    ).fit(X_pca20)
    labels = model.labels_
    sil = silhouette_nonnoise(X_pca20, labels)
    noise_frac = (labels==-1).mean()
    mean_prob = float(np.mean(model.probabilities_))
    n_clusters = int(len(np.setdiff1d(np.unique(labels), [-1])))
    # composite: prefer higher silhouette, lower noise
    composite = (np.nan_to_num(sil, nan=-1.0)) - 0.5*noise_frac
    cand = dict(mcs=mcs, mns=mns, meth=meth, sil=sil, noise=noise,
                mean_prob=mean_prob, n_clusters=n_clusters, comp=composite)
    if (best is None) or (cand["composite"] > best["composite"]):
        best = cand

# refit best on all, just reuse last best params for clean object
model = hdbscan.HDBSCAN(
    min_cluster_size=best["mcs"],
    min_samples=best["mns"] if best["mns"] is not None else None,
    cluster_selection_method=best["meth"],
    core_dist_n_jobs=1
).fit(X_pca20)
labels = model.labels_

# save labels aligned to eda
pd.Series(labels, index=sub.index, name="hdbscan_pipeline1_pca20")\
    .to_csv(os.path.join(output_dir, f"labels_hdbscan_pipeline1__{tag}.csv"))

# UMAP-2D for visualization (fit on PCA-20)
emb2 = umap.UMAP(n_components=2, n_neighbors=30, min_dist=0.3, random_state=0)\
    .fit_transform(X_pca20)
np.save(os.path.join(output_dir, f"umap2_hdbscan_pipeline1__{tag}.npy"), emb2)

out_png = os.path.join(output_dir, f"umap2_plot_hdbscan_pipeline1__{tag}.png")
title = (f"HDBSCAN (Pipeline 1: PCA-20) - {interval} - {COND_NAME[cond]}")
        f"mcs={best['mcs']}, mns={best['mns']}, {best['meth']}, "
        f"clusters={best['n_clusters']}, noise={best['noise']:.2f}")
plot_umap2(emb2, labels, title, out_png)

p1_rows.append([
    str(interval), cond, n, best["mcs"], best["mns"], best["meth"],
    best["n_clusters"], best["noise"], best["mean_prob"], best["sil"], cond
])

p1_metrics = pd.DataFrame(p1_rows, columns=[
    "interval", "condition", "n_cells", "min_cluster_size", "min_samples", "selection",
    "n_clusters", "noise_frac", "mean_prob", "silhouette_nonnoise", "umap_plot_png"
])
p1_metrics.to_csv(os.path.join(output_dir, "hdbscan_pipeline1_metrics.csv"), index=False)
p1_metrics

```

[illegible]

[illegible]

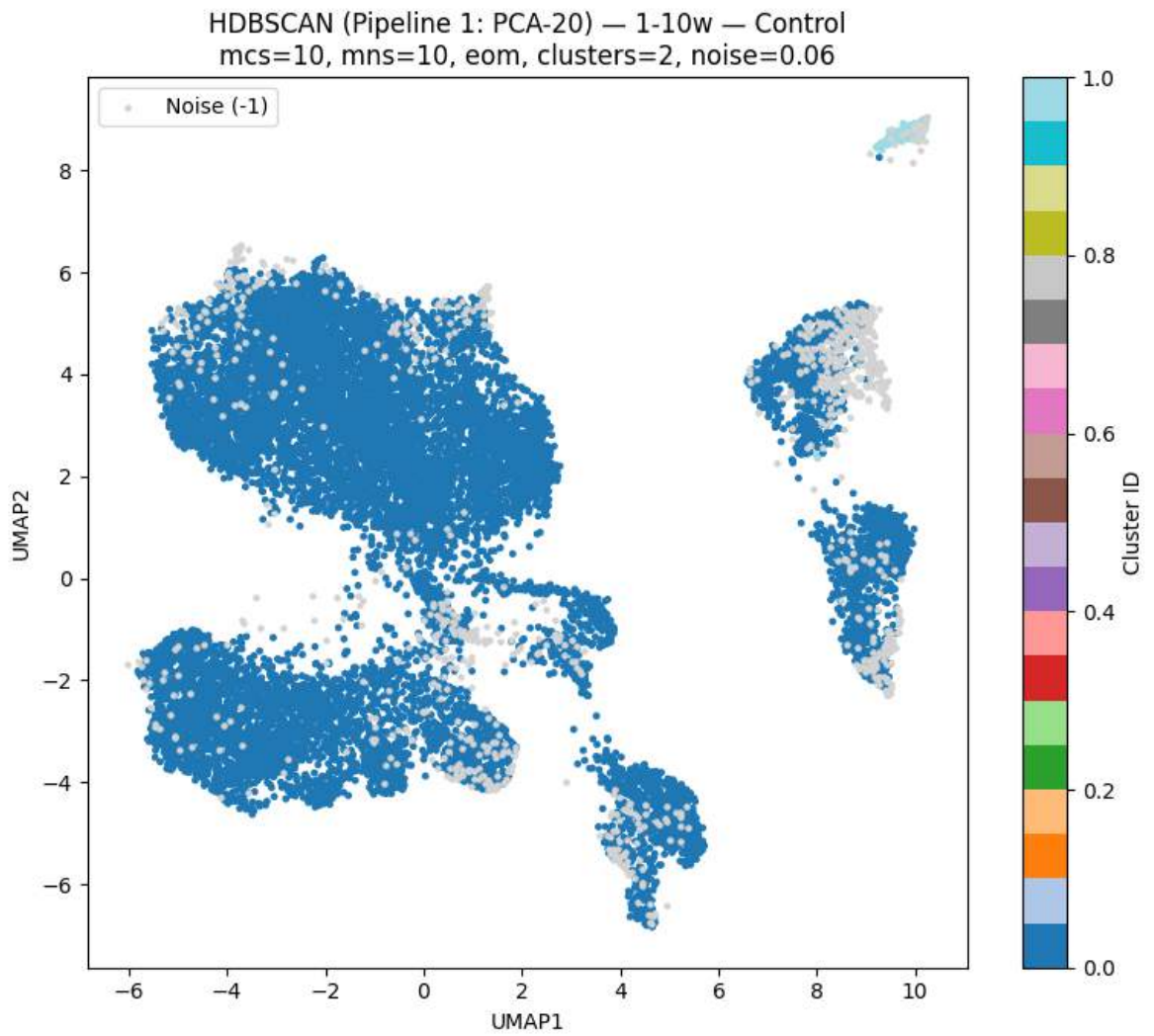
[illegible]

[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(
```



[illegible]

[illegible]

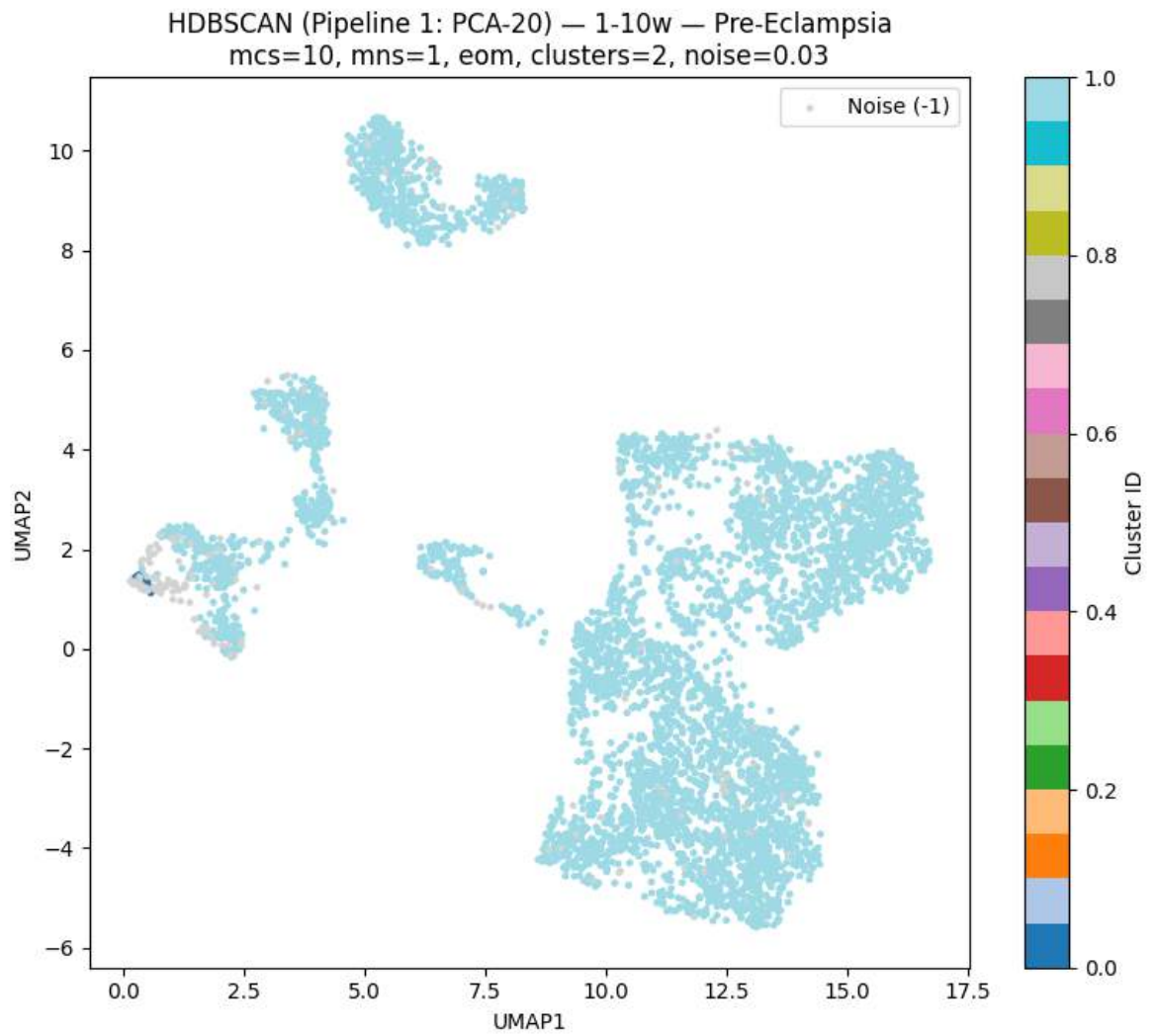
[illegible]

[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(
```



[illegible]

[illegible]

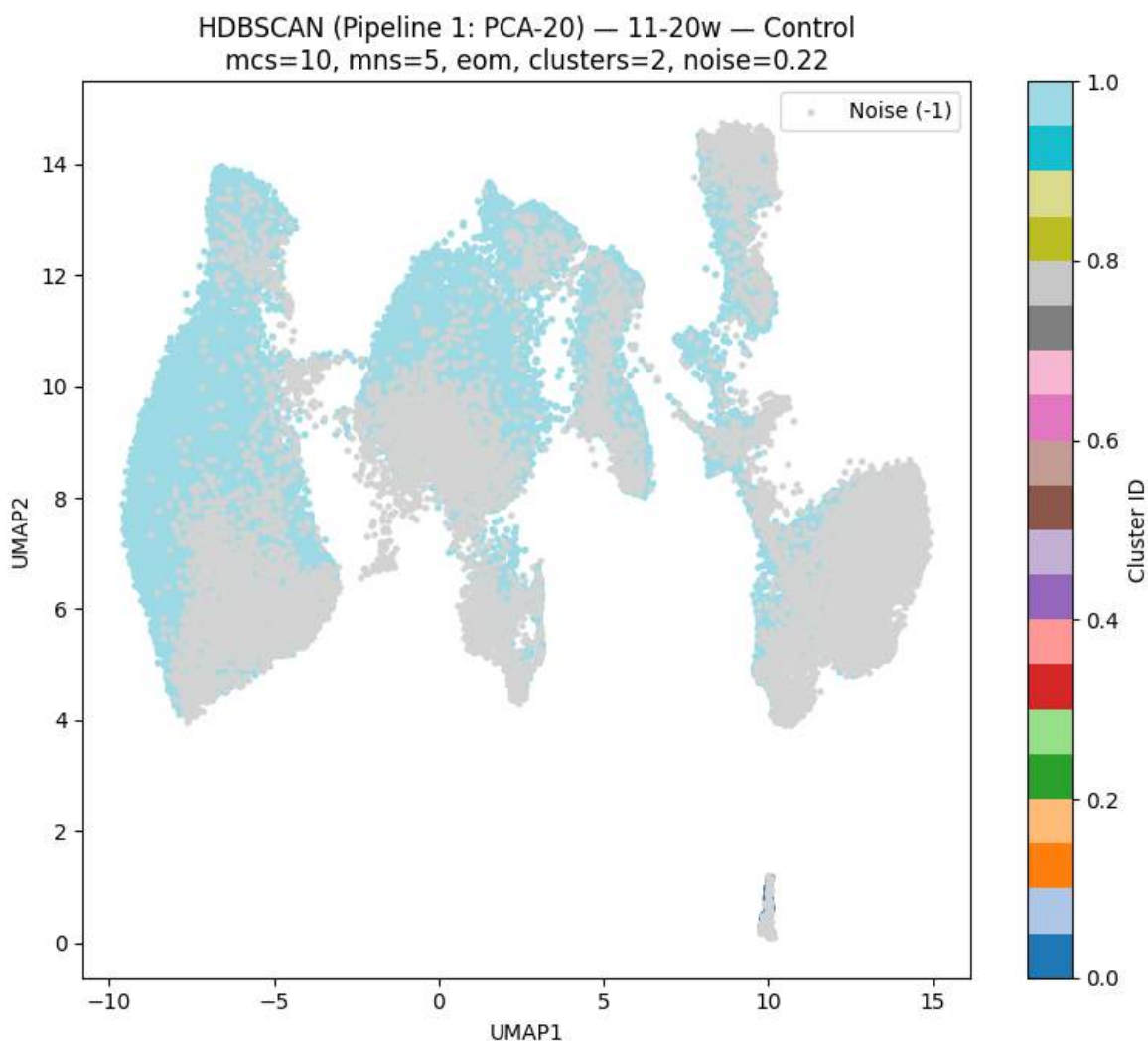
[illegible]

[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(

```



[illegible]

[illegible]

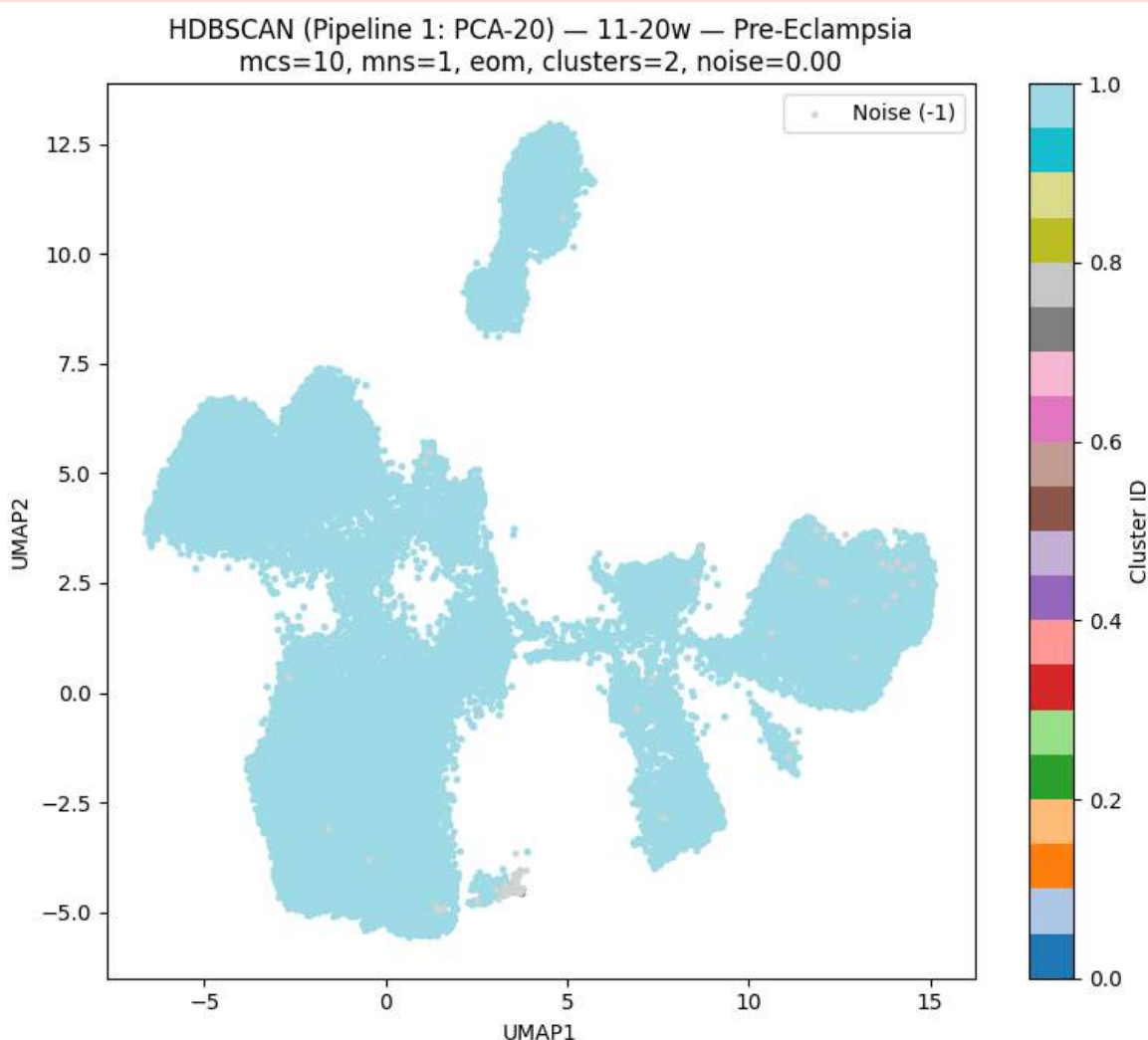
[illegible]

[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(

```



[illegible]

[illegible]

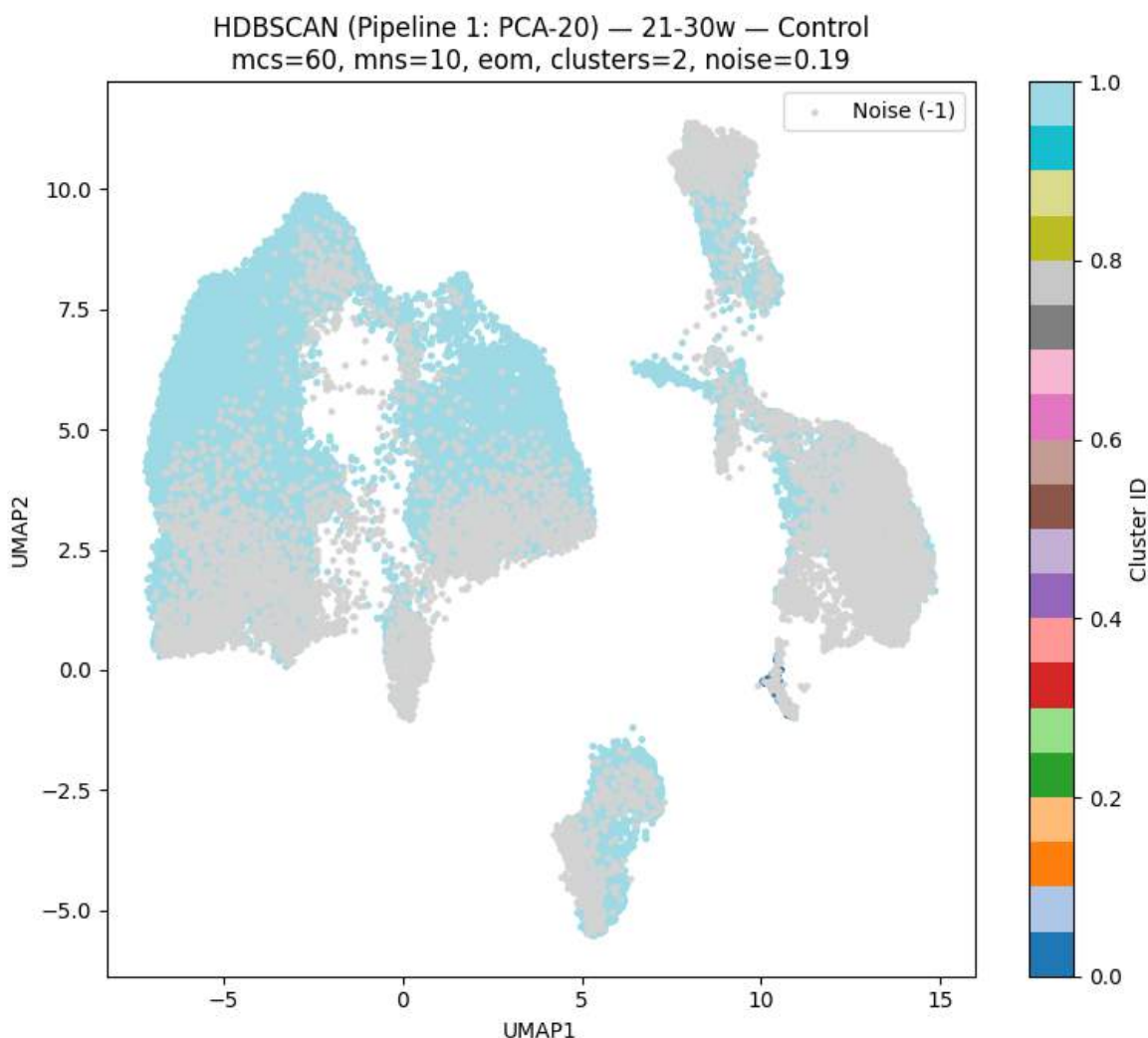
[illegible]

[illegible]


```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(

```



[illegible]

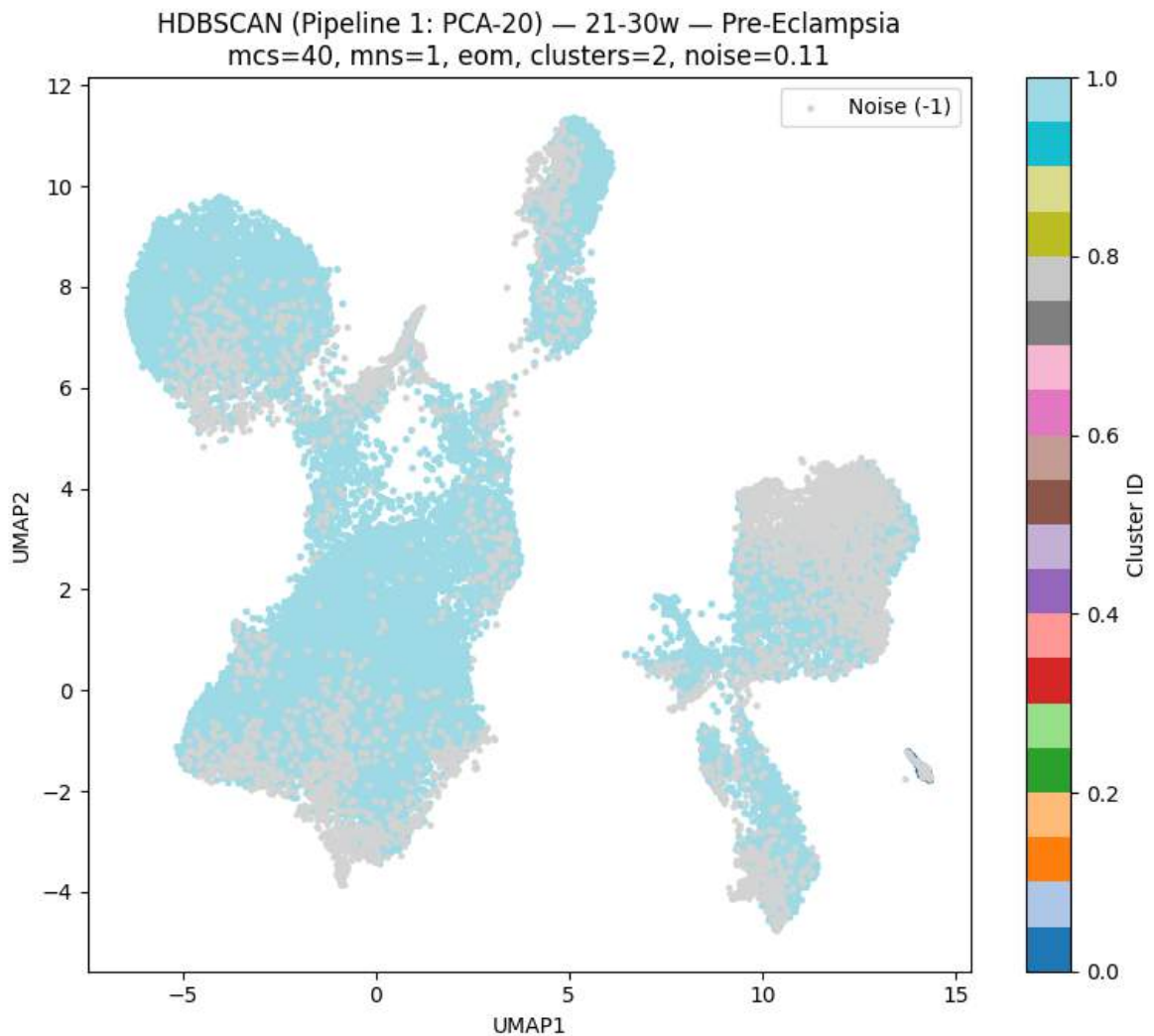
[illegible]

[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(
```

Out[]:

	interval	condition	n_cells	min_cluster_size	min_samples	selection_method	n_clusters
0	1-10w	0	15000	10	10	eom	
1	1-10w	1	5000	10	1	eom	
2	11-20w	0	55000	10	5	eom	
3	11-20w	1	65000	10	1	eom	
4	21-30w	0	50000	60	10	eom	
5	21-30w	1	40000	40	1	eom	

```

In [ ]: p1_path = os.path.join(output_dir, "hdbscan_pipeline1_metrics.csv")
if os.path.exists(p1_path):
    df_p1 = pd.read_csv(p1_path)
    df_p1["algorithm"] = "HDBSCAN"
    df_p1["pipeline"] = "PCA20 (P1, base)"
    df_p1 = df_p1.rename(columns={"umap_plot_png": "plot_png"})
    hdb_tables.append(df_p1)

```

HDBScan hyperparameter tuning PCA Clustering space and UMAP for visualization

```

In [ ]: # Load prior metrics (to compare composite = silhouette - 0.5*noise)
prev_metrics_path = os.path.join(output_dir, "hdbscan_pipeline1_metrics.csv")

```

```

prev_metrics = pd.read_csv(prev_metrics_path) if os.path.exists(prev_metrics_pat

def prev_composite(interval, cond):
    if prev_metrics.empty:
        return -np.inf
    m = prev_metrics[(prev_metrics["interval"].astype(str)==str(interval)) &
                     (prev_metrics["condition"].astype(int)==int(cond))]
    if m.empty:
        return -np.inf
    sil = float(m.iloc[-1]["silhouette_nonnoise"])
    noise = float(m.iloc[-1]["noise_frac"])
    return np.nan_to_num(sil, nan=-1.0) - 0.5*noise

# Extra grid (cap to n)
def extra_mcs_grid(n):
    base = [90, 100, 110, 120]
    return [m for m in base if m <= n]

minsamples_grid = [1, 3, 5, 10]
methods_grid    = ["eom", "leaf"]

```

The hyperparameter tuning sweeps through the adjusted parameters clusters and only saves clusters that show improved silhouette scores compared to the last

```

In [ ]: rows_hi = []

for interval in valid_bins:
    for cond in [0, 1]:
        # subset & PCA20
        sub = eda[(eda['age_bin_10w']==interval) & (eda['condition']==cond)]
        if sub.empty:
            continue
        tag = f"{interval}_{cond}"
        pca_path = os.path.join(output_dir, f"pca20_{tag}.npy")
        if not os.path.exists(pca_path):
            print(f"[P1-HI] Missing PCA20 for {tag}, skipping.")
            continue
        X_pca20 = np.load(pca_path)
        n = len(X_pca20)
        if n < 90:
            print(f"[P1-HI] {tag}: n={n} < 90, no high mcs to test.")
            continue

        best = None
        for mcs in extra_mcs_grid(n):
            for mns in minsamples_grid:
                for meth in methods_grid:
                    model = hdbscan.HDBSCAN(
                        min_cluster_size=mcs,
                        min_samples=mns if mns is not None else None,
                        cluster_selection_method=meth,
                        core_dist_n_jobs=1
                    ).fit(X_pca20)

                    labels = model.labels_
                    sil = silhouette_nonnoise(X_pca20, labels)
                    noise_frac = (labels == -1).mean()
                    mean_prob = float(np.mean(model.probabilities_))
                    n_clusters = int(len(np.setdiff1d(np.unique(labels), [-1])))

```

```

        composite = np.nan_to_num(sil, nan=-1.0) - 0.5*noise_frac

        cand = dict(mcs=mcs, mns=mns, meth=meth, sil=sil, noise=nois
                    mean_prob=mean_prob, n_clusters=n_clusters, comp
                    labels=labels)
        if (best is None) or (cand["composite"] > best["composite"]):
            best = cand

    if best is None:
        print(f"[P1-HI] {tag}: no candidate found.")
        continue

    # Compare to previous best; if improved, save under *_hi files
    prev_comp = prev_composite(interval, cond)
    if best["composite"] <= prev_comp:
        print(f"[P1-HI] {tag}: best(composite={best['composite']:.3f}) <= pr
    else:
        # Save improved labels
        labels = best["labels"]
        pd.Series(labels, index=sub.index, name="hdbscan_pipeline1_pca20_hi",
                  .to_csv(os.path.join(output_dir, f"labels_hdbscan_pipeline1_hi_{t

        # UMAP-2D for visualization (fresh viz for the HI model)
        emb2 = umap.UMAP(n_components=2, n_neighbors=30, min_dist=0.3, random_state=42)
        .fit_transform(X_pca20)
        np.save(os.path.join(output_dir, f"umap2_hdbscan_pipeline1_hi_{tag}

        out_png = os.path.join(output_dir, f"umap2_plot_hdbscan_pipeline1_hi
        title = (f"HDBSCAN (Pipeline 1: PCA-20, HI mcs) - {interval} - {COND
                  f"mcs={best['mcs']}, mns={best['mns']}, {best['meth']}, "
                  f"clusters={best['n_clusters']}, noise={best['noise']:.2f}")
        plot_umap2(emb2, labels, title, out_png)

        rows_hi.append([
            str(interval), cond, n, best["mcs"], best["mns"], best["meth"],
            best["n_clusters"], best["noise"], best["mean_prob"], best["sil"],
            best["composite"], os.path.basename(out_png)
        ])
    print(f"[P1-HI] {tag}: improved → saved *_hi outputs.")

```

[illegible]

[illegible]

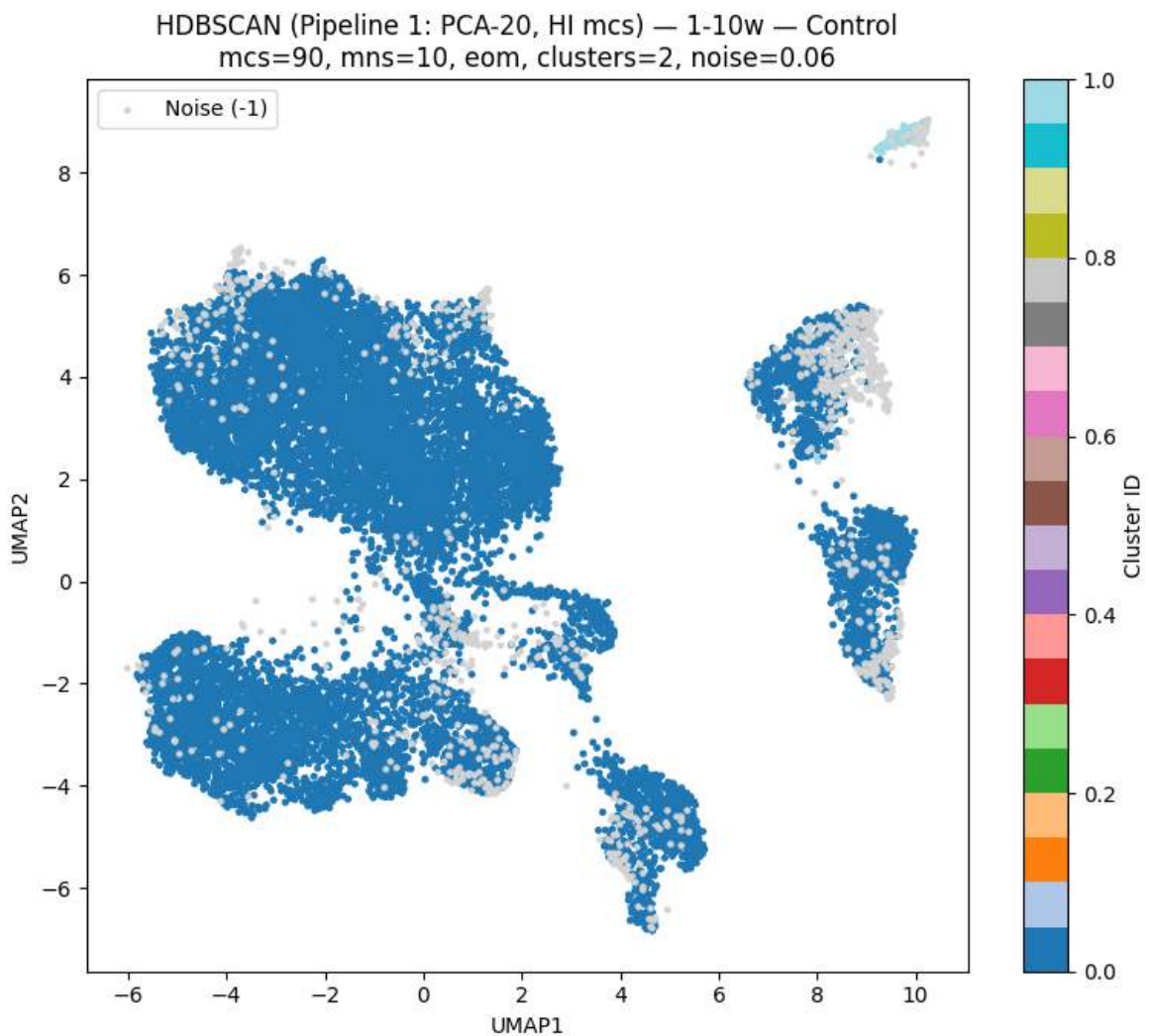
[illegible]

[illegible]


```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(

```



[P1-HI] 1-10w_0: improved → saved *_hi outputs.

[illegible]

[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
[P1-HI] 1-10w_1: best(composite=0.106) <= prev(0.459) – keeping previous.
```

[illegible]

[illegible]

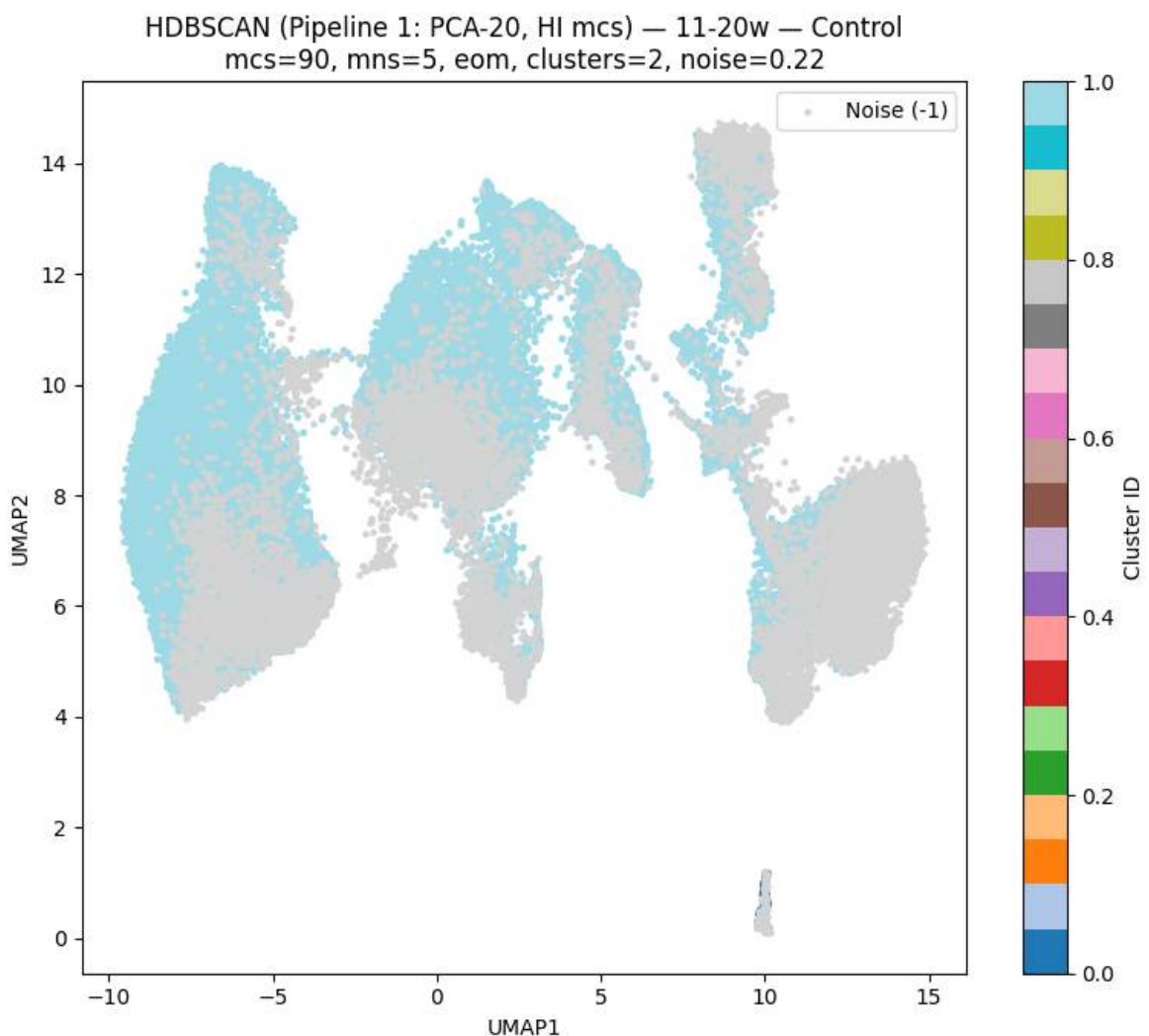
[illegible]

[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(

```



[P1-HI] 11-20w_0: improved → saved *_hi outputs.

[illegible]

[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
[P1-HI] 11-20w_1: best(composite=0.237) <= prev(0.567) – keeping previous.
```

[illegible]

[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
[P1-HI] 21-30w_0: best(composite=0.280) <= prev(0.280) – keeping previous.
```


[illegible]

[illegible]

[illegible]

[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
[P1-HI] 21-30w_1: best(composite=0.302) <= prev(0.302) - keeping previous.

```

```

In [ ]: # Save HI metrics (only rows that beat previous)
if rows_hi:
    hi_df = pd.DataFrame(rows_hi, columns=[
        "interval", "condition", "n_cells", "min_cluster_size", "min_samples", "selection_method",
        "n_clusters", "noise_frac", "mean_prob", "silhouette_nonnoise", "composite",
    ])
    hi_path = os.path.join(output_dir, "hdbscan_pipeline1_metrics_hi.csv")
    hi_df.to_csv(hi_path, index=False)
    print("Saved improved HI metrics →", hi_path)

    # also produce a merged view for convenience
    prev_metrics["composite"] = np.nan_to_num(prev_metrics["silhouette_nonnoise"])
    merged = prev_metrics.copy()
    for _, r in hi_df.iterrows():
        mask = (merged["interval"].astype(str) == r["interval"]) & (merged["condition"] == r["condition"])
        # append the HI row (don't overwrite the old one)
        merged = pd.concat([merged, pd.DataFrame([
            "interval": r["interval"],
            "condition": r["condition"],
            "n_cells": r["n_cells"],
            "min_cluster_size": r["min_cluster_size"],
            "min_samples": r["min_samples"],
            "selection_method": r["selection_method"],
            "n_clusters": r["n_clusters"],
            "noise_frac": r["noise_frac"],
            "mean_prob": r["mean_prob"],
            "silhouette_nonnoise": r["silhouette_nonnoise"],
            "umap_plot_png": r["umap_plot_png"],
            "composite": r["composite"]
        ])], ignore_index=True)

    merged_path = os.path.join(output_dir, "hdbscan_pipeline1_metrics_merged.csv")
    merged.to_csv(merged_path, index=False)
    print("Saved merged metrics (original + HI) →", merged_path)
else:
    print("No HI configuration beat the previous composite; no *_hi outputs written")

```

Saved improved HI metrics → /content/drive/MyDrive/preeclampsia_results/hdbscan_pipeline1_metrics_hi.csv

Saved merged metrics (original + HI) → /content/drive/MyDrive/preeclampsia_results/hdbscan_pipeline1_metrics_merged.csv

```
In [ ]: # Saving all plots and runs into CSVs and PNGs
p1_path = os.path.join(output_dir, "hdbscan_pipeline1_metrics_hi.csv")
if os.path.exists(p1_path):
    df_p1 = pd.read_csv(p1_path)
    df_p1["algorithm"] = "HDBSCAN"
    df_p1["pipeline"] = "PCA20 (P1, base)"
    df_p1 = df_p1.rename(columns={"umap_plot_png": "plot_png"})
    hdb_tables.append(df_p1)
```

Pipeline 2: HDBSCAN umap embed 2d, 3d, 5d, and UMAP for vizualization

```
In [ ]: # Fixing random seed, setting constants
RANDOM_STATE = 42
COND_NAME = {0:'Control', 1:'Pre-Eclampsia'}

def silhouette_nonnoise(X, labels):
    #Computing silhouette score excluding noise
    mask = (labels != -1)
    if mask.sum() < 3:
        return np.nan
    labs = labels[mask]
    if len(np.unique(labs)) < 2:
        return np.nan
    return silhouette_score(X[mask], labs)

def plot_umap2(emb2, labels, title, out_png):
    #Plotting and saving scatter plots 2D umap embeds
    plt.figure(figsize=(8,7))
    noise = (labels == -1)
    # clusters
    if (~noise).any():
        sc = plt.scatter(emb2[~noise,0], emb2[~noise,1], c=labels[~noise], s=6,
    else:
        sc = None
    # noise
    if noise.any():
        plt.scatter(emb2[noise,0], emb2[noise,1], c="lightgrey", s=5, label="Noi
        plt.legend(frameon=True)
    plt.title(title); plt.xlabel("UMAP1"); plt.ylabel("UMAP2")
    if sc is not None:
        plt.colorbar(sc, label="Cluster ID")
    plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.show(); plt.close()

def plot_umap3(emb3, labels, title, out_png):
    #Plotting and saving cluster plots 3D UMAP embeds
    from mpl_toolkits.mplot3d import Axes3D # noqa: F401
    fig = plt.figure(figsize=(9,7))
    ax = fig.add_subplot(111, projection='3d')
    noise = (labels == -1)
    p = ax.scatter(emb3[~noise,0], emb3[~noise,1], emb3[~noise,2],
        c=labels[~noise], s=6, cmap="tab20")
    if noise.any():
        ax.scatter(emb3[noise,0], emb3[noise,1], emb3[noise,2],
            c="lightgrey", s=5, label="Noise (-1)")
        ax.legend()
    ax.set_title(title); ax.set_xlabel("UMAP1"); ax.set_ylabel("UMAP2"); ax.set
    fig.colorbar(p, label="Cluster ID")
    plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.show(); plt.close()
```

```
def grid_for_n_umap(n, pct=(0.002, 0.005, 0.01)):
    """
    Reuse the same logic as Pipeline 1
    """
    base_abs = [10, 20, 40, 60]
    base_from_pct = [int(np.ceil(p * n)) for p in pct]
    mcs = sorted({
        m for m in (base_abs + base_from_pct)
        if 5 <= m <= min(80, n)
    })
    minsamples = [1, 3, 5, 10]
    methods = ["eom", "leaf"]
    return mcs, minsamples, methods
```

UMAP cache (2D/3D/5D) from PCA20

```
In [ ]: # Build & cache UMAP embeddings per (interval, condition, dim)
# Assumes: eda, output_dir, valid_bins, MARKERS (33 markers) already define

Path(output_dir).mkdir(parents=True, exist_ok=True)

# Ensure PCA20 cache exists (re-uses your P1 code paths)
for interval in valid_bins:
    for cond in [0,1]:
        sub = eda[(eda['age_bin_10w']==interval) & (eda['condition']==cond)]
        if sub.empty:
            continue
        tag = f"{interval}_{cond}"
        pfile = os.path.join(output_dir, f"pca20__{tag}.npz")
        if not os.path.exists(pfile):
            X = sub[MARKERS].to_numpy(dtype="float32")
            Xs = StandardScaler().fit_transform(X)
            X_pca20 = PCA(n_components=20, random_state=RANDOM_STATE).fit_transform(Xs)
            np.save(pfile, X_pca20)
            print("PCA20 saved:", tag, X_pca20.shape)
        else:
            print("PCA20 cached:", tag)

# Build UMAP embeddings and cache
for interval in valid_bins:
    for cond in [0,1]:
        tag = f"{interval}_{cond}"
        pfile = os.path.join(output_dir, f"pca20__{tag}.npz")
        if not os.path.exists(pfile):
            print("Missing PCA20 for", tag);
            continue
        X_pca20 = np.load(pfile)

        for d in (2,3,5):
            umap_path = os.path.join(output_dir, f"umap{d}_pipeline2__{tag}.npz")
            if os.path.exists(umap_path):
                print(f"UMAP{d} cached:", tag)
                continue
            reducer = umap.UMAP(n_components=d, n_neighbors=30, min_dist=0.3, random_state=RANDOM_STATE)
            emb = reducer.fit_transform(X_pca20)
            np.save(umap_path, emb)
            print(f"UMAP{d} saved:", tag, emb.shape)
```



```

PCA20 cached: 1-10w_0
PCA20 cached: 1-10w_1
PCA20 cached: 11-20w_0
PCA20 cached: 11-20w_1
PCA20 cached: 21-30w_0
PCA20 cached: 21-30w_1
UMAP2 cached: 1-10w_0
UMAP3 cached: 1-10w_0
UMAP5 cached: 1-10w_0
UMAP2 cached: 1-10w_1
UMAP3 cached: 1-10w_1
UMAP5 cached: 1-10w_1
UMAP2 cached: 11-20w_0
UMAP3 cached: 11-20w_0
UMAP5 cached: 11-20w_0
UMAP2 cached: 11-20w_1
UMAP3 cached: 11-20w_1
UMAP5 cached: 11-20w_1
UMAP2 cached: 21-30w_0
UMAP3 cached: 21-30w_0
UMAP5 cached: 21-30w_0
UMAP2 cached: 21-30w_1
UMAP3 cached: 21-30w_1
UMAP5 cached: 21-30w_1

```

```

In [ ]: # Pipeline 2 HDBSCAN sweep in UMAP (2D/3D/5D) and save everything
p2_rows = []

for interval in valid_bins:
    for cond in [0,1]:
        tag = f"{interval}_{cond}"

        for d in (2,3,5):
            umap_path = os.path.join(output_dir, f"umap{d}_pipeline2_{tag}.npy")
            if not os.path.exists(umap_path):
                print(f"[P2] Missing UMAP{d} for {tag}, skipping.")
                continue

            emb = np.load(umap_path)
            n = len(emb)
            if n < 50:
                print(f"[P2] {tag} dim={d}: too few cells ({n})")
                continue

            best = None
            mcs_list, mns_list, meth_list = grid_for_n_umap(n)
            for mcs in mcs_list:
                for mns in mns_list:
                    for meth in meth_list:
                        model = hdbscan.HDBSCAN(
                            min_cluster_size=mcs,
                            min_samples=mns if mns is not None else None,
                            cluster_selection_method=meth,
                            core_dist_n_jobs=1
                        ).fit(emb)

                        labels = model.labels_
                        sil = silhouette_nonnoise(emb, labels)
                        noise_frac = (labels == -1).mean()
                        mean_prob = float(np.mean(model.probabilities_))

```

```

n_clusters = int(len(np.setdiff1d(np.unique(labels), [-1
composite = np.nan_to_num(sil, nan=-1.0) - 0.5*noise_fr

cand = dict(dim=d, mcs=mcs, mns=mns, meth=meth, sil=sil,
            mean_prob=mean_prob, n_clusters=n_clusters,
            labels=labels)
if (best is None) or (cand["composite"] > best["composit
best = cand

# Refit best (already fit) - just use best["labels"]
labels = best["labels"].astype(int)

# Save labels aligned to eda subset
sub_idx = eda[(eda['age_bin_10w']==interval) & (eda['condition']==co
lab_name = f"hdbscan_pipeline2_umap{best['dim']}
labels_csv = os.path.join(output_dir, f"labels_hdbscan_pipeline2_uma
pd.Series(labels, index=sub_idx, name=lab_name).to_csv(labels_csv)

# Save plot
if best["dim"] == 2:
    out_png = os.path.join(output_dir, f"umap2_plot_hdbscan_pipeline
    title = (f"HDBSCAN (Pipeline 2: UMAP-2D) - {interval} - {COND_NA
            f"mcs={best['mcs']}, mns={best['mns']}, {best['meth']},
            f"clusters={best['n_clusters']}, noise={best['noise']}:.
    plot_umap2(emb, labels, title, out_png)
elif best["dim"] == 3:
    out_png = os.path.join(output_dir, f"umap3_plot_hdbscan_pipeline
    title = (f"HDBSCAN (Pipeline 2: UMAP-3D) - {interval} - {COND_NA
            f"mcs={best['mcs']}, mns={best['mns']}, {best['meth']},
            f"clusters={best['n_clusters']}, noise={best['noise']}:.
    plot_umap3(emb, labels, title, out_png)
else:
    # For 5D, project to 2D UMAP *for visualization only* (no reclu
    umap2_for_viz = umap.UMAP(n_components=2, n_neighbors=30, min_di
        .fit_transform(emb)
    out_png = os.path.join(output_dir, f"umap5_plot2Dviz_hdbscan_pip
    title = (f"HDBSCAN (Pipeline 2: UMAP-5D clustering; 2D viz) - {i
            f"mcs={best['mcs']}, mns={best['mns']}, {best['meth']},
            f"clusters={best['n_clusters']}, noise={best['noise']}:.
    plot_umap2(umap2_for_viz, labels, title, out_png)
    # also persist viz embedding
    np.save(os.path.join(output_dir, f"umap5_viz2d_{tag}.npy"), uma

# Record metrics row
p2_rows.append([
    str(interval), cond, n, best["dim"], best["mcs"], best["mns"], b
    best["n_clusters"], best["noise"], best["mean_prob"], best["sil"
])

# Save metrics table
p2_metrics = pd.DataFrame(p2_rows, columns=[
    "interval", "condition", "n_cells", "umap_dim", "min_cluster_size", "min_samples"
    "n_clusters", "noise_frac", "mean_prob", "silhouette_nonnoise", "plot_png", "labe
])
p2_metrics_path = os.path.join(output_dir, "hdbscan_pipeline2_metrics.csv")
p2_metrics.to_csv(p2_metrics_path, index=False)
p2_metrics

```

[illegible]

[illegible]

[illegible]

[illegible]

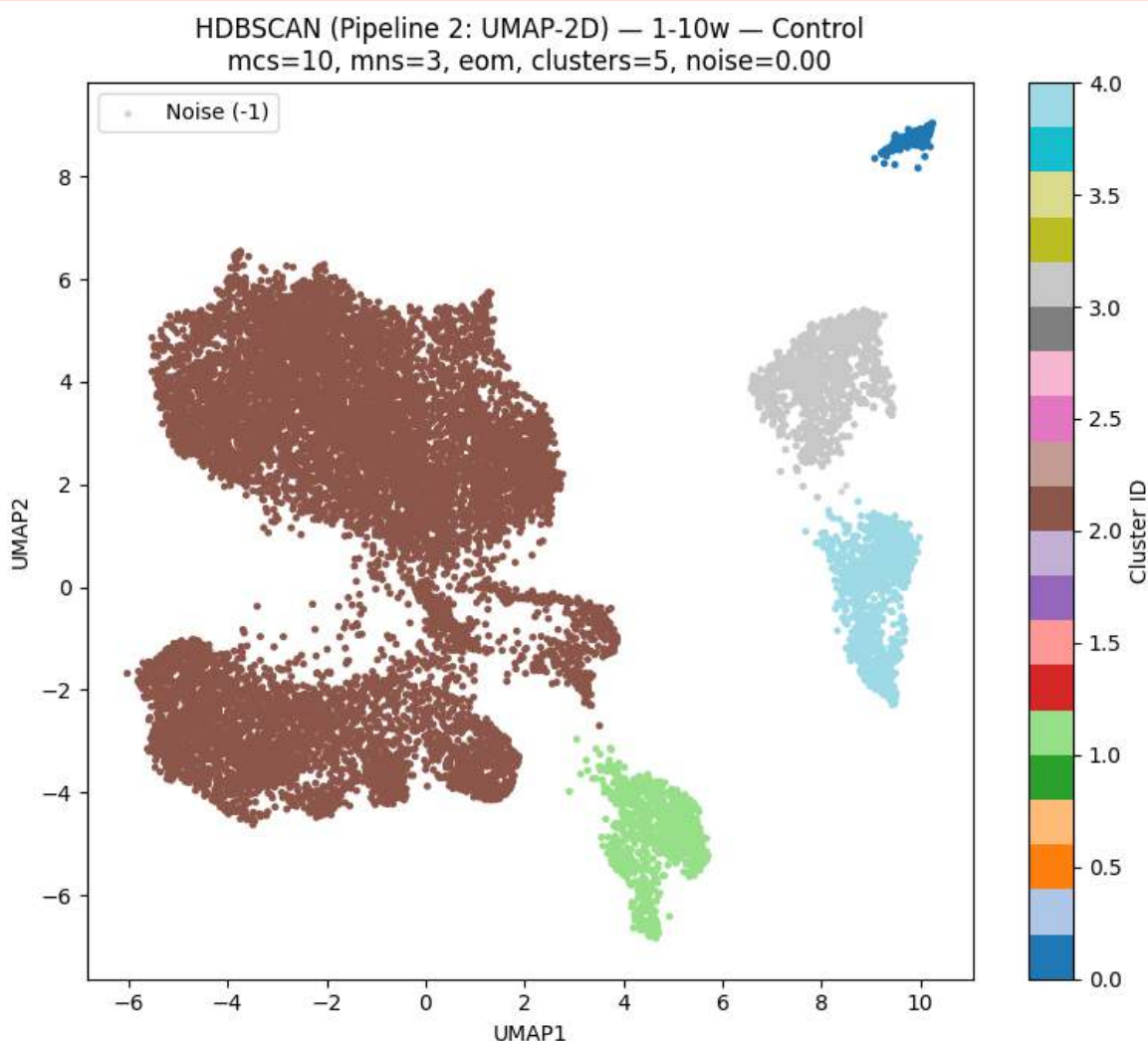
[illegible]

[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```



[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

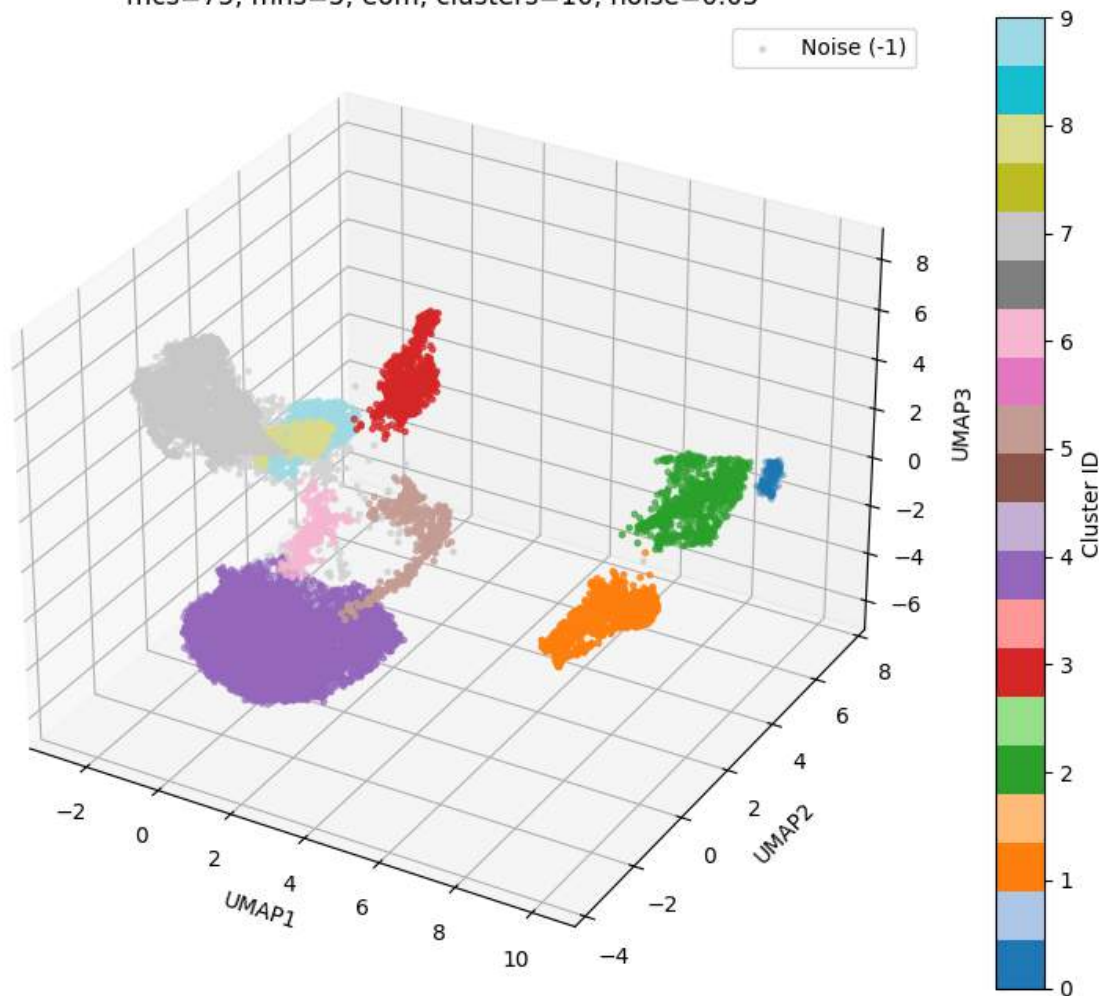
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN (Pipeline 2: UMAP-3D) — 1-10w — Control
 mcs=75, mns=5, eom, clusters=10, noise=0.05



[illegible]

[illegible]

[illegible]

[illegible]

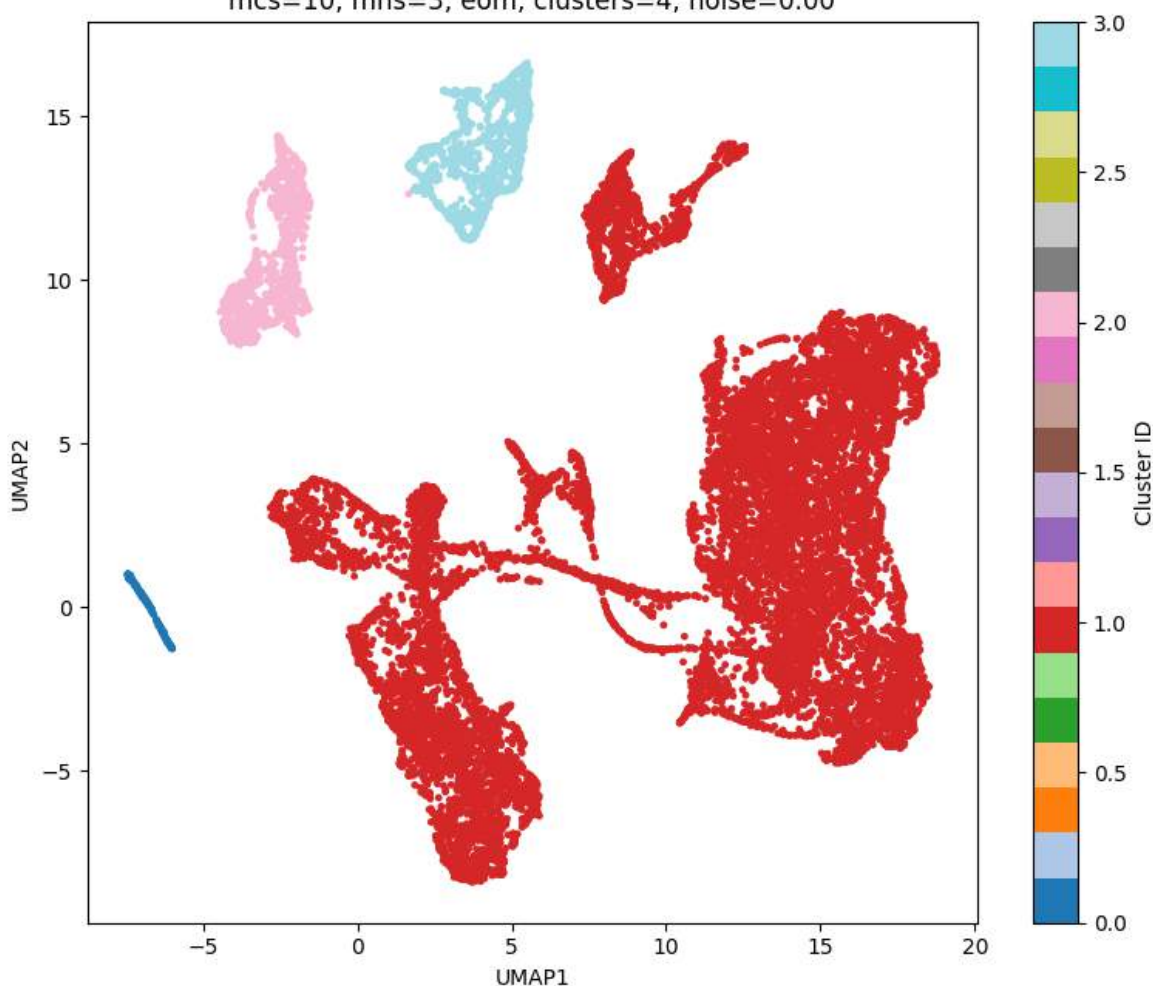
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(

```

HDBSCAN (Pipeline 2: UMAP-5D clustering; 2D viz) — 1-10w — Control
 mcs=10, mns=3, eom, clusters=4, noise=0.00



[illegible]

[illegible]

[illegible]

[illegible]

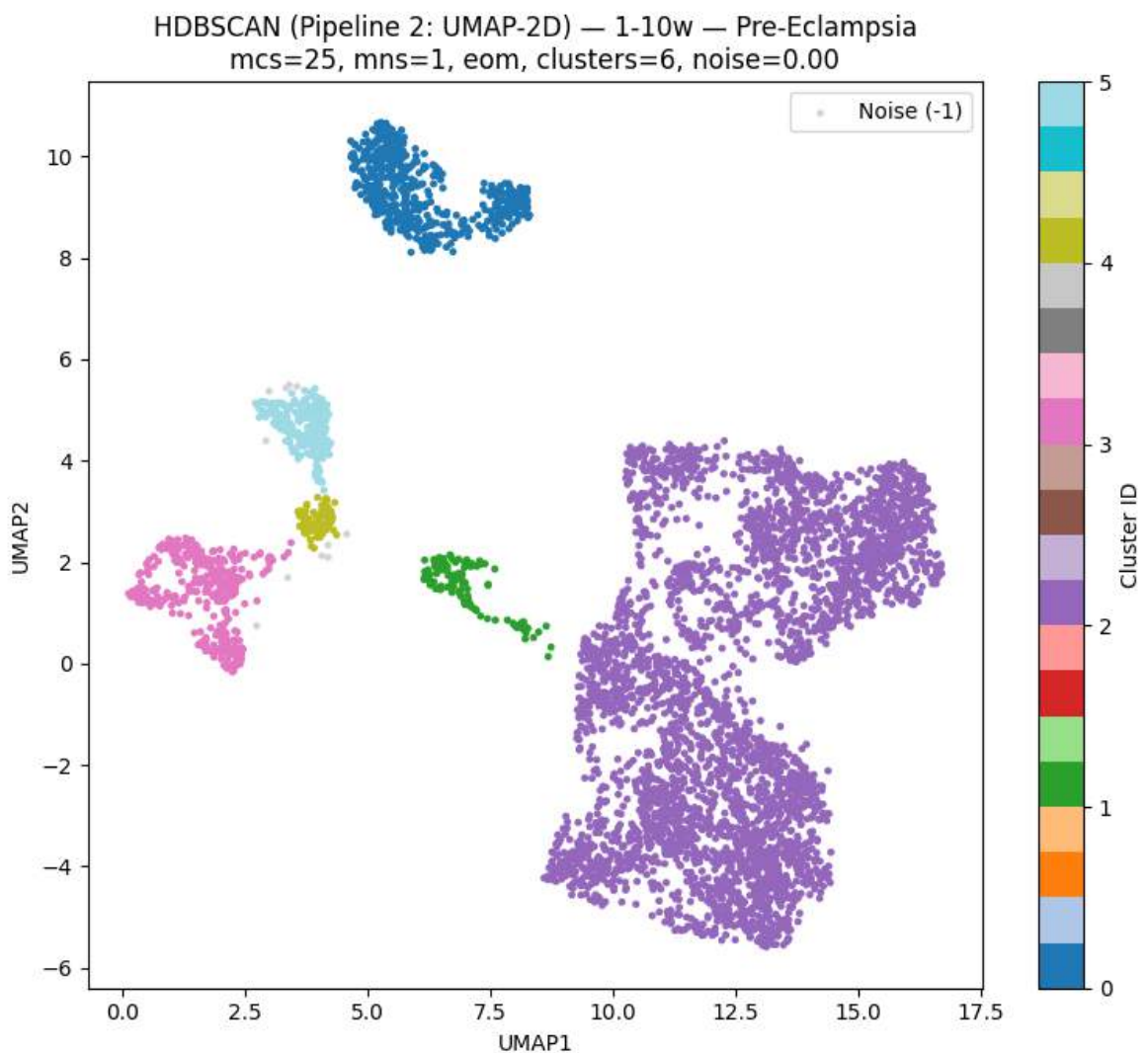
[illegible]

[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```



[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

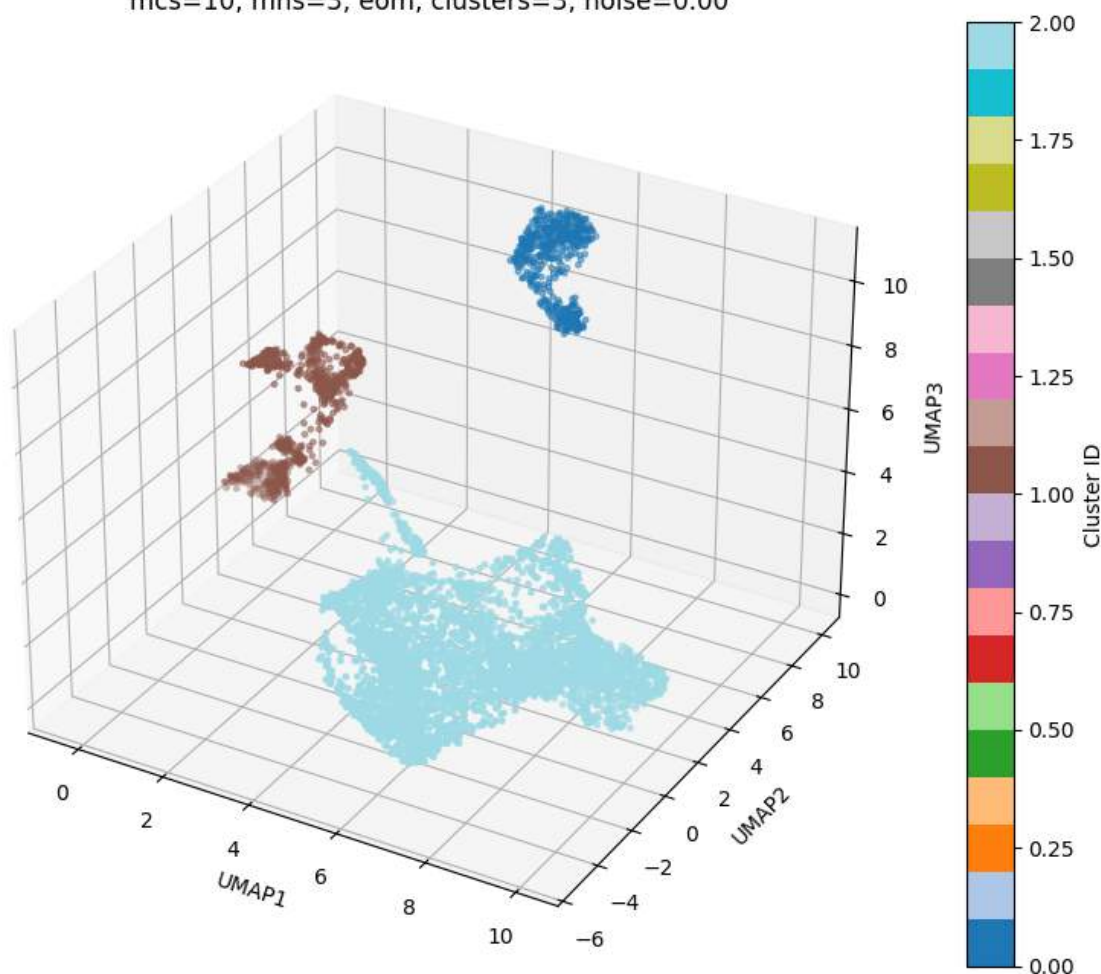
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN (Pipeline 2: UMAP-3D) — 1-10w — Pre-Eclampsia
mcs=10, mns=3, eom, clusters=3, noise=0.00



[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

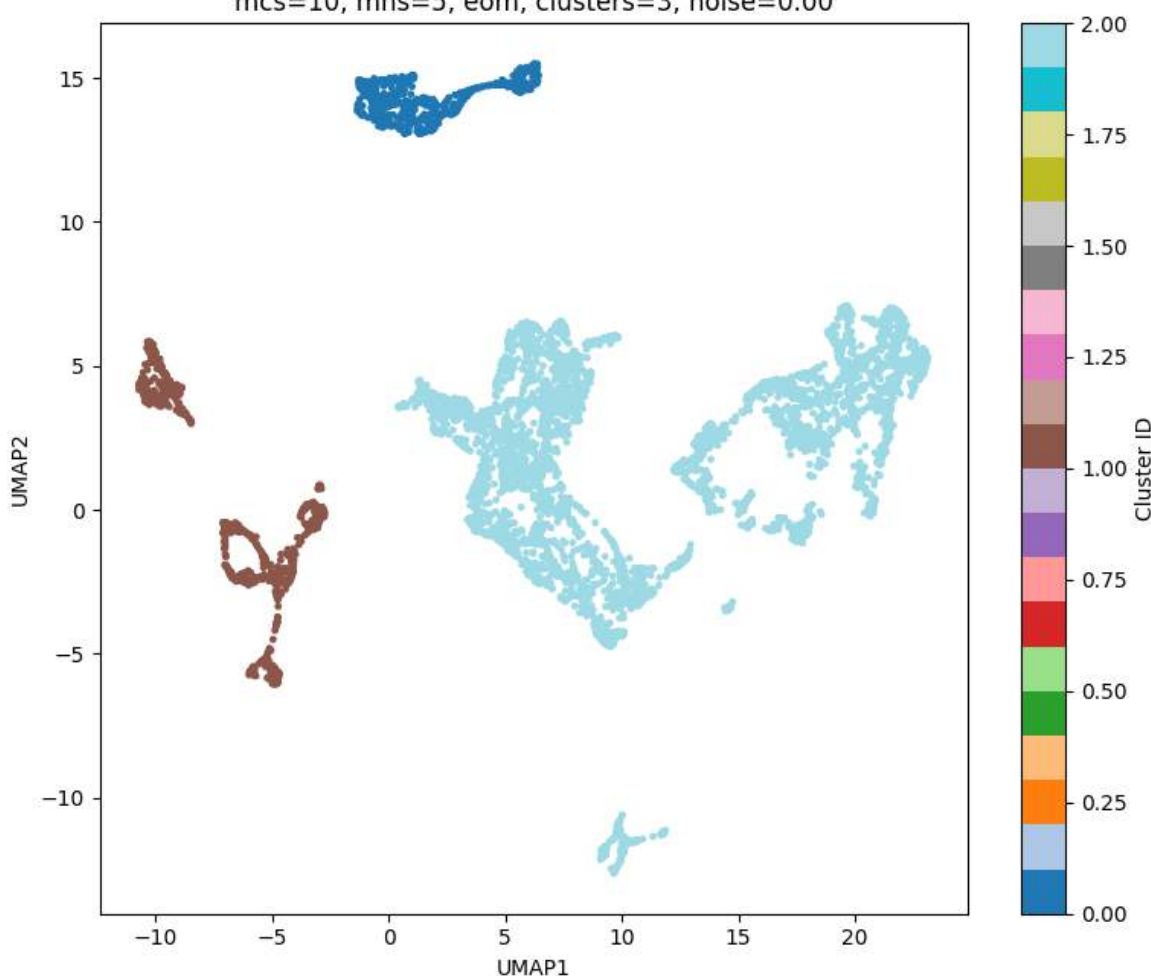
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(

```

HDBSCAN (Pipeline 2: UMAP-5D clustering; 2D viz) — 1-10w — Pre-Eclampsia
mcs=10, mns=5, eom, clusters=3, noise=0.00



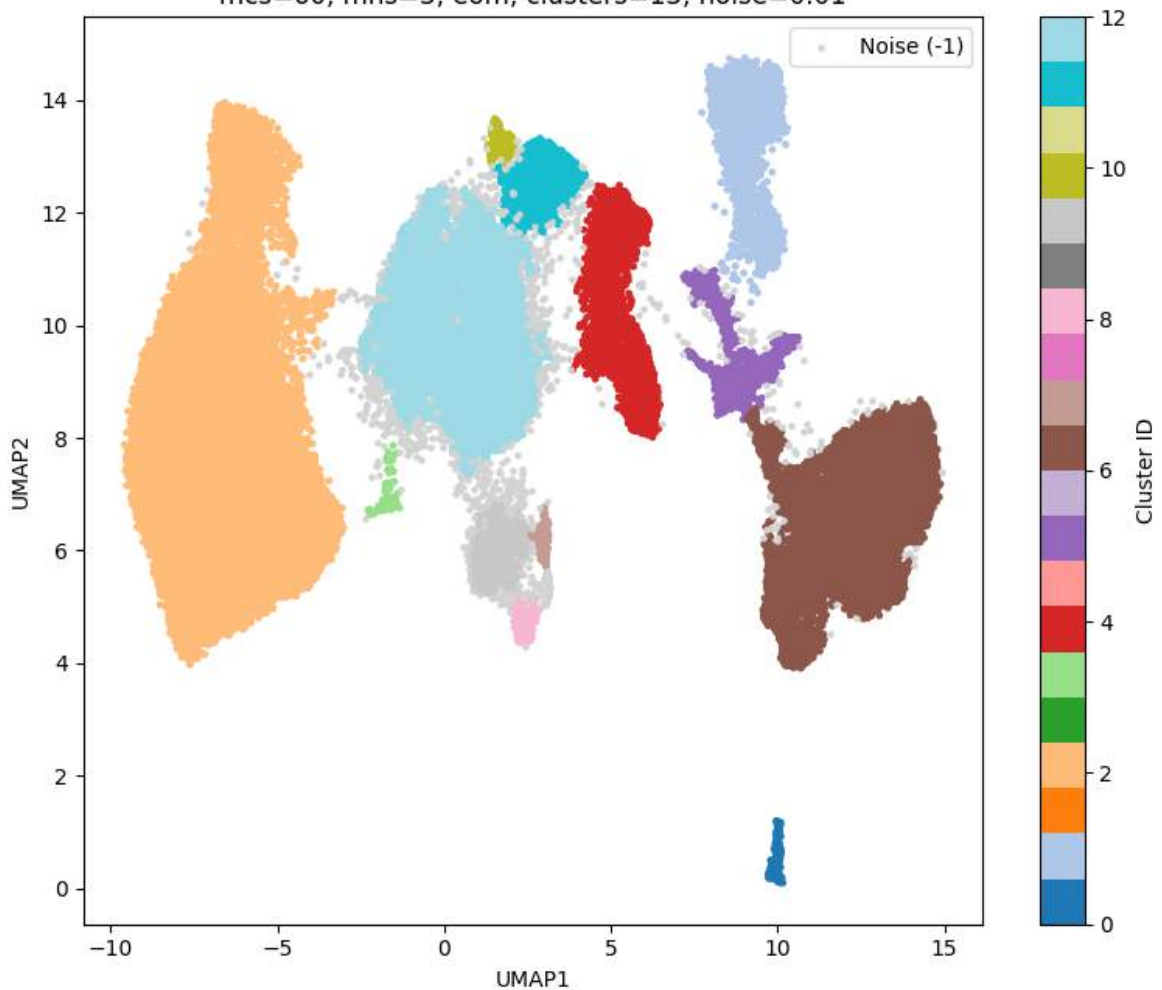
[illegible]

[illegible]

[illegible]

[illegible]

HDBSCAN (Pipeline 2: UMAP-2D) — 11-20w — Control
mcs=60, mns=3, eom, clusters=13, noise=0.01



[illegible]

[illegible]

[illegible]

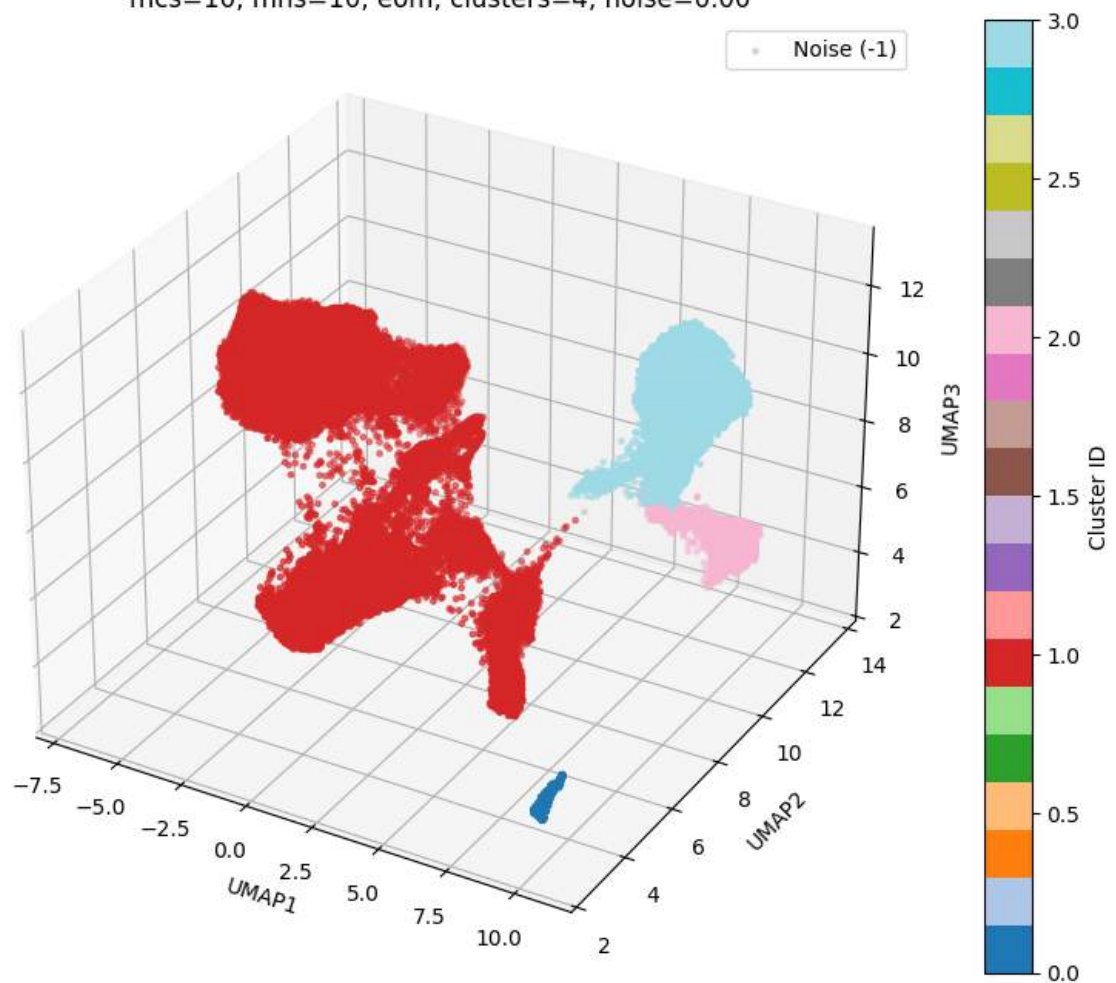
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN (Pipeline 2: UMAP-3D) — 11-20w — Control
 mcs=10, mns=10, eom, clusters=4, noise=0.00



[illegible]

[illegible]

[illegible]

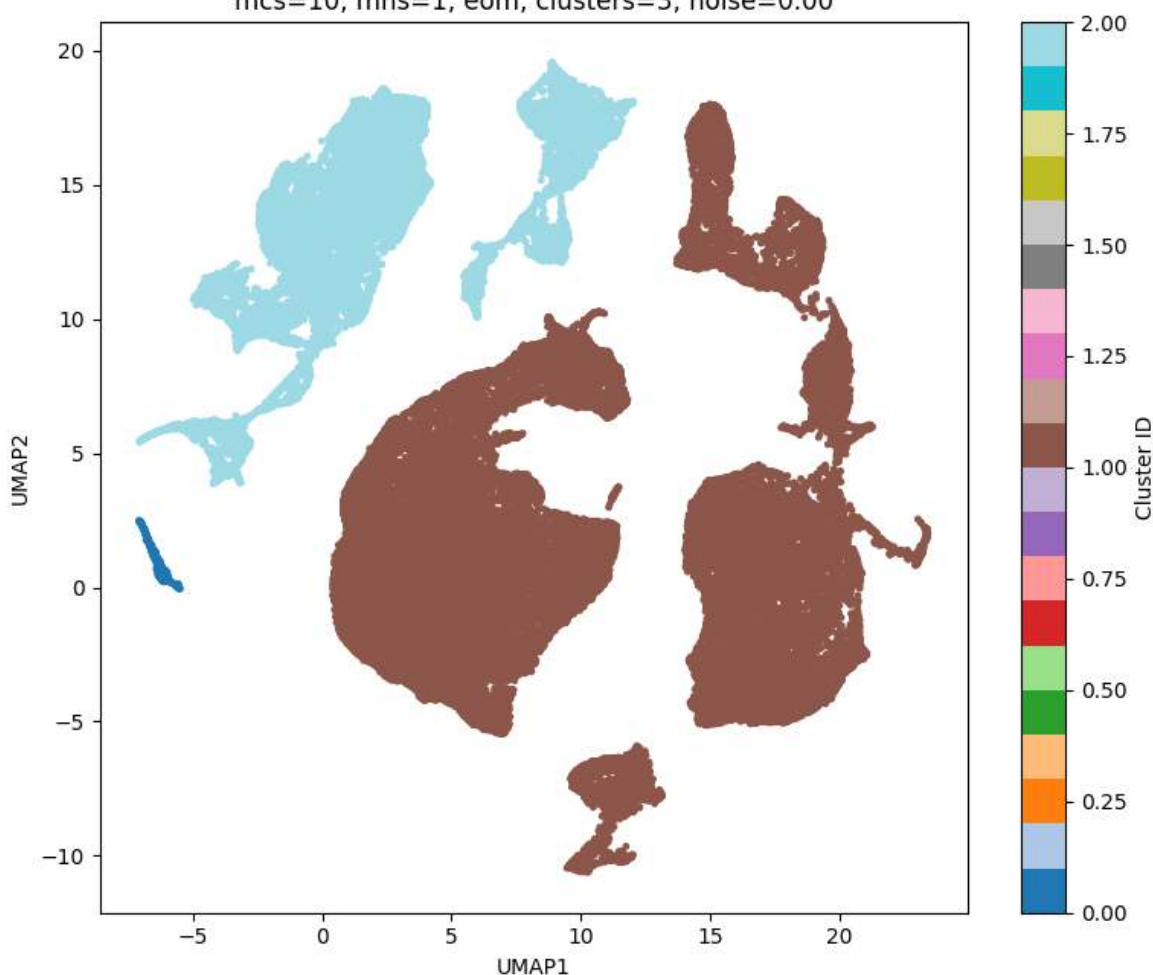
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(

```

HDBSCAN (Pipeline 2: UMAP-5D clustering; 2D viz) — 11-20w — Control
mcs=10, mns=1, eom, clusters=3, noise=0.00



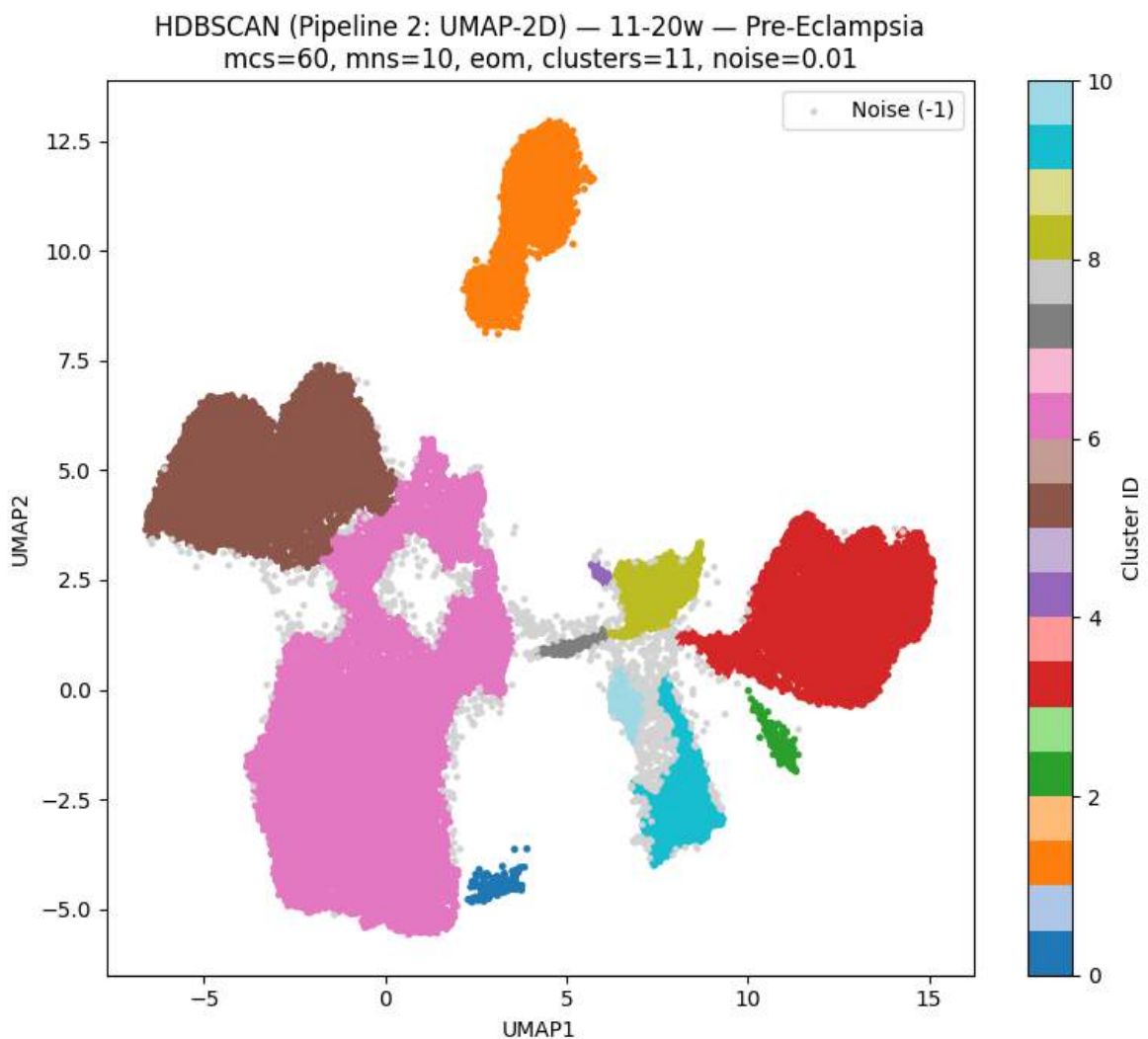
[illegible]

[illegible]

[illegible]

[illegible]


```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```



[illegible]

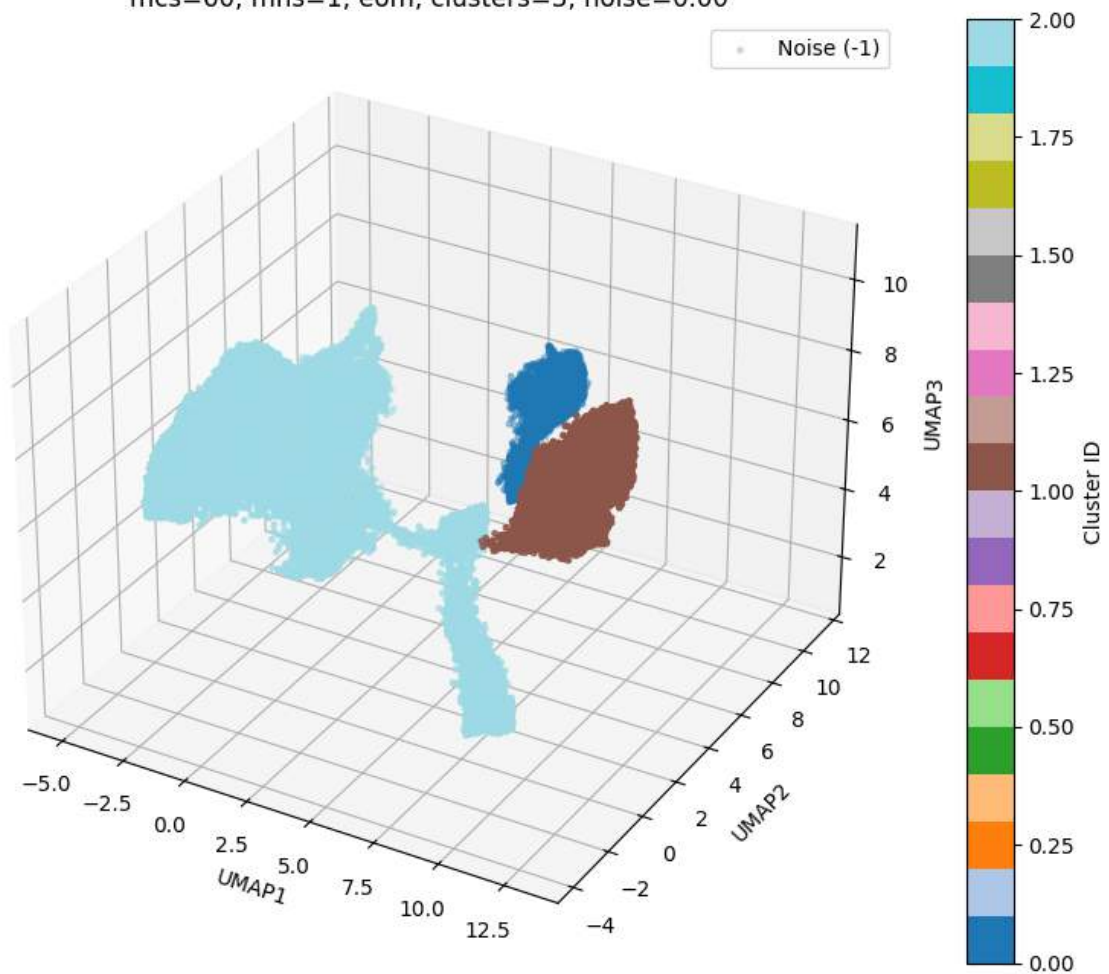
[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```

HDBSCAN (Pipeline 2: UMAP-3D) — 11-20w — Pre-Eclampsia
mcs=60, mns=1, eom, clusters=3, noise=0.00



[illegible]

[illegible]

[illegible]

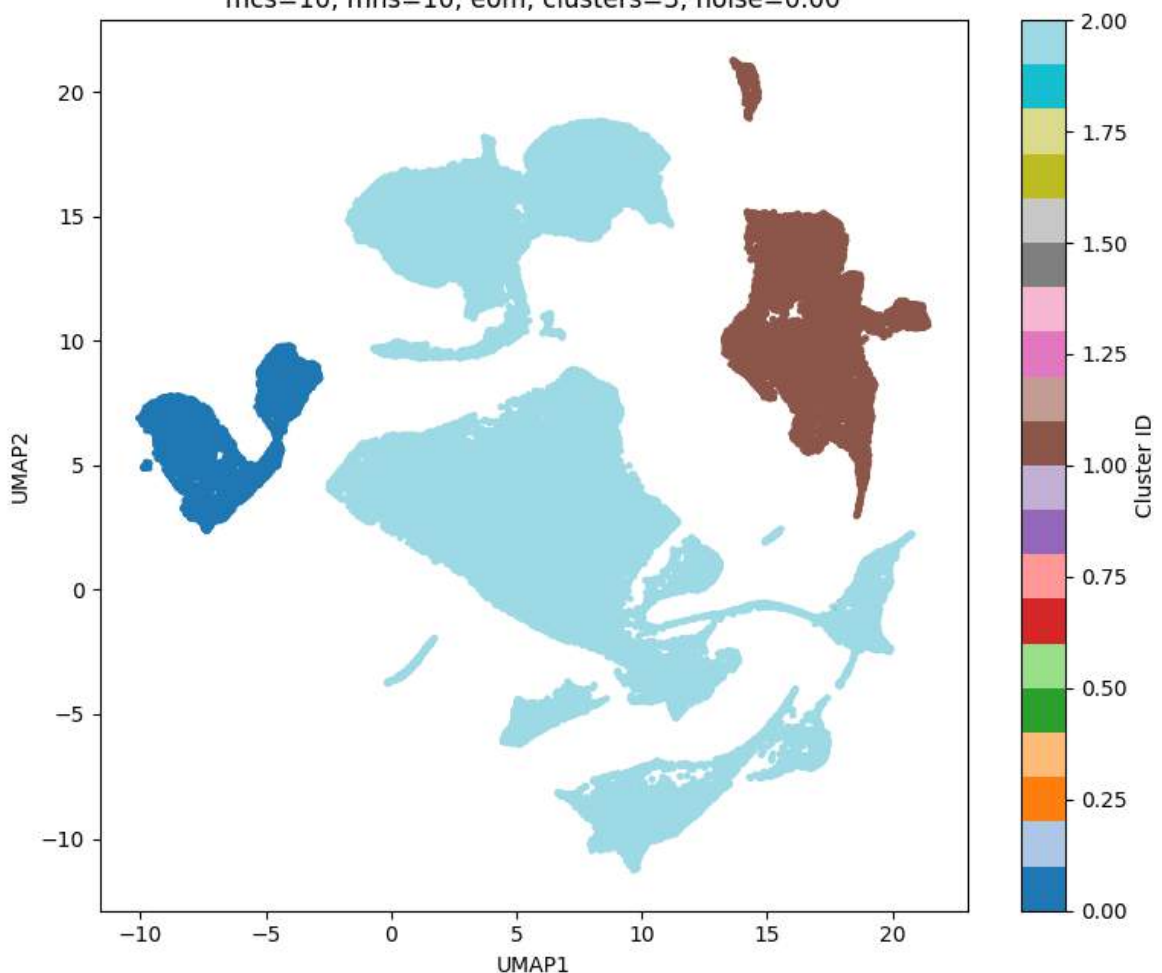
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(

```

HDBSCAN (Pipeline 2: UMAP-5D clustering; 2D viz) — 11-20w — Pre-Eclampsia
 mcs=10, mns=10, eom, clusters=3, noise=0.00



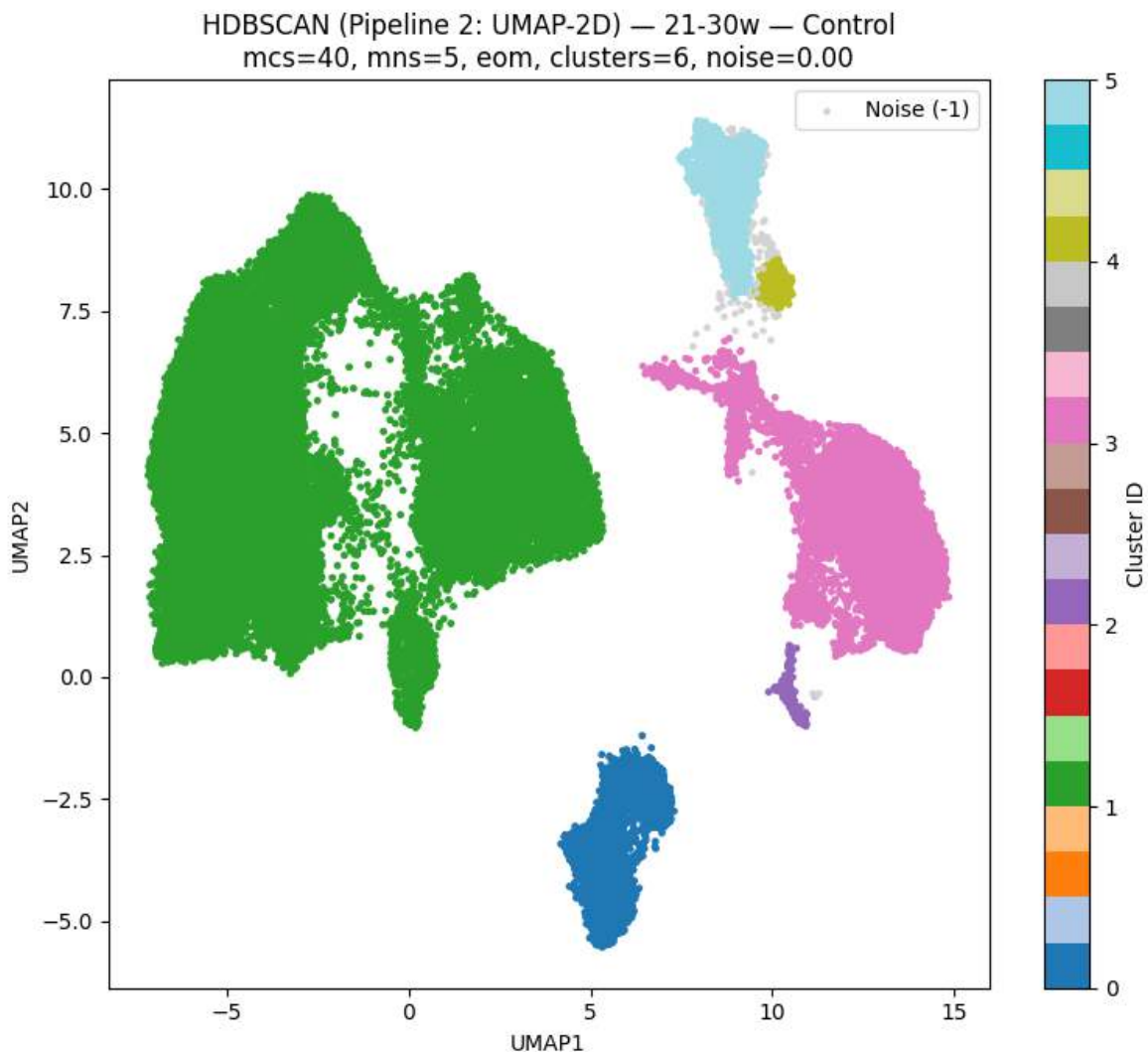
[illegible]

[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```



[illegible]

[illegible]

[illegible]

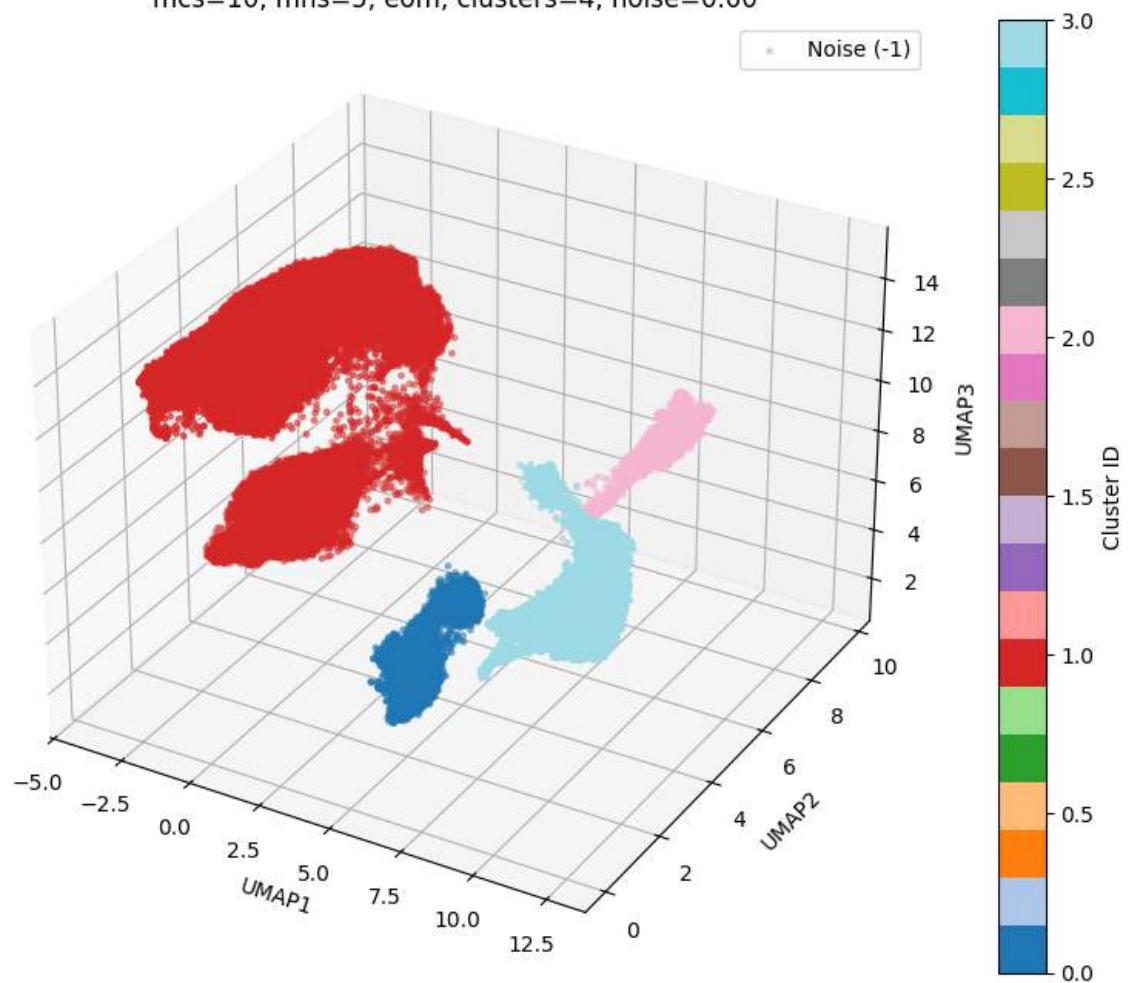
[illegible]


```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN (Pipeline 2: UMAP-3D) — 21-30w — Control
 mcs=10, mns=5, eom, clusters=4, noise=0.00



[illegible]

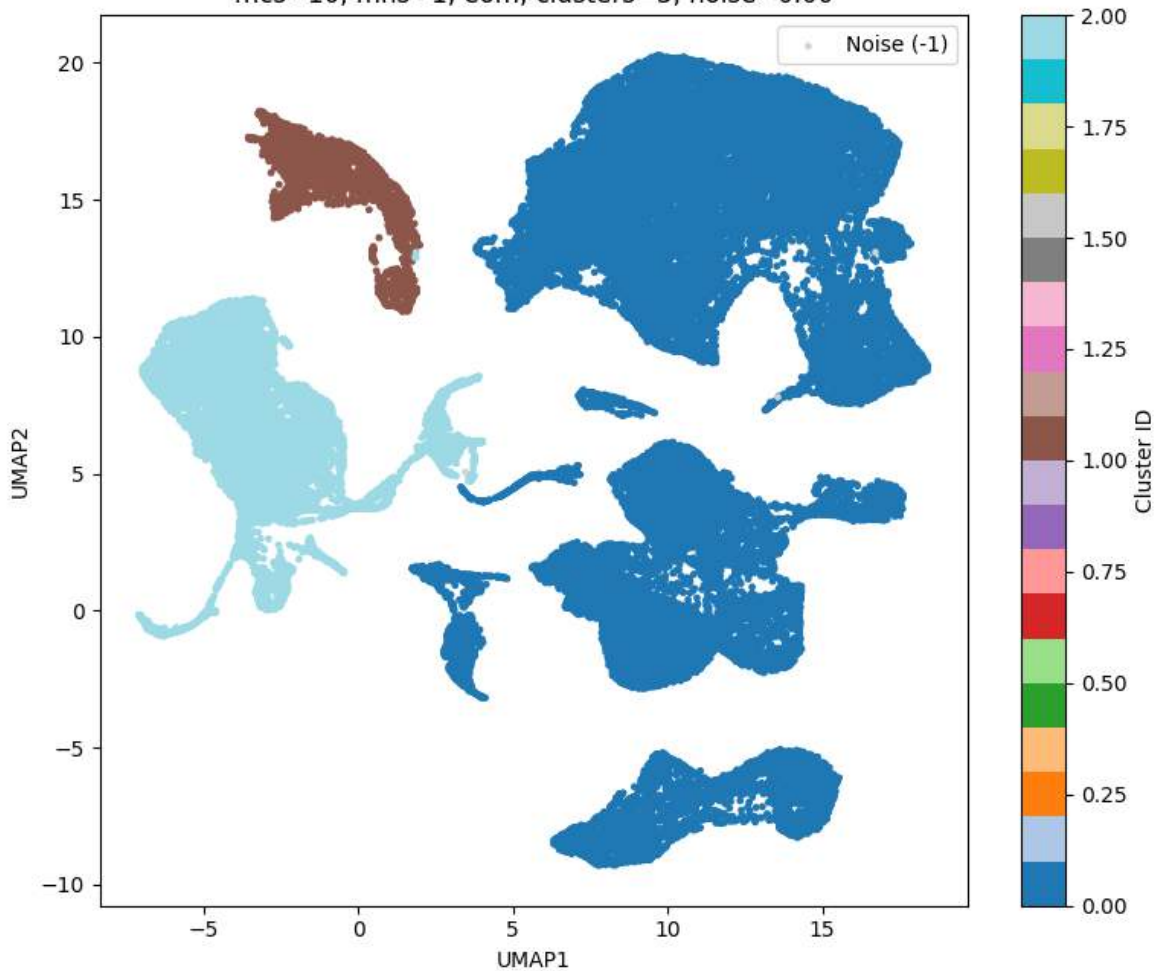
[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(
```

HDBSCAN (Pipeline 2: UMAP-5D clustering; 2D viz) — 21-30w — Control
mcs=10, mns=1, eom, clusters=3, noise=0.00



[illegible]

[illegible]

[illegible]

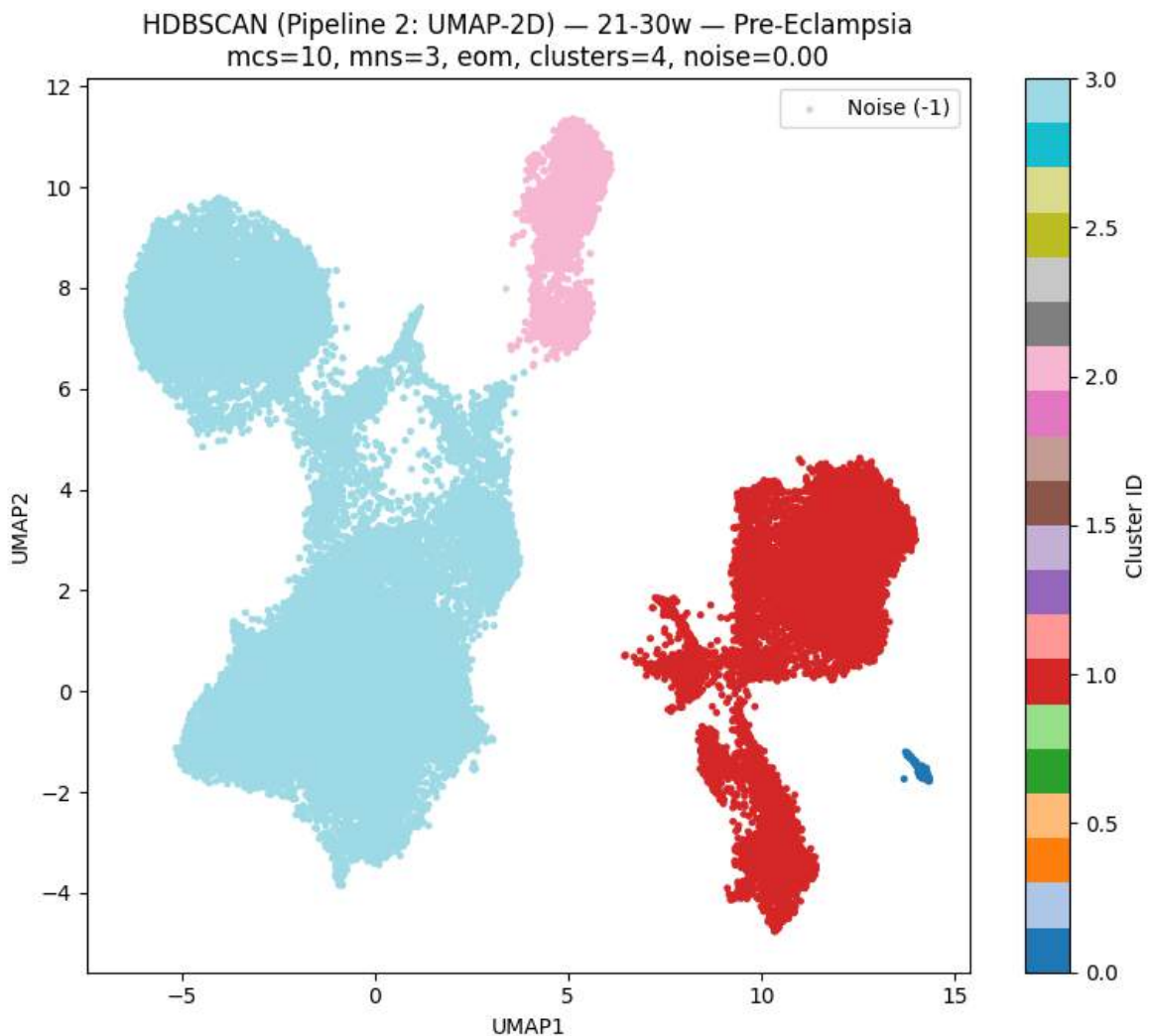
[illegible]

[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```



[illegible]

[illegible]

[illegible]

[illegible]

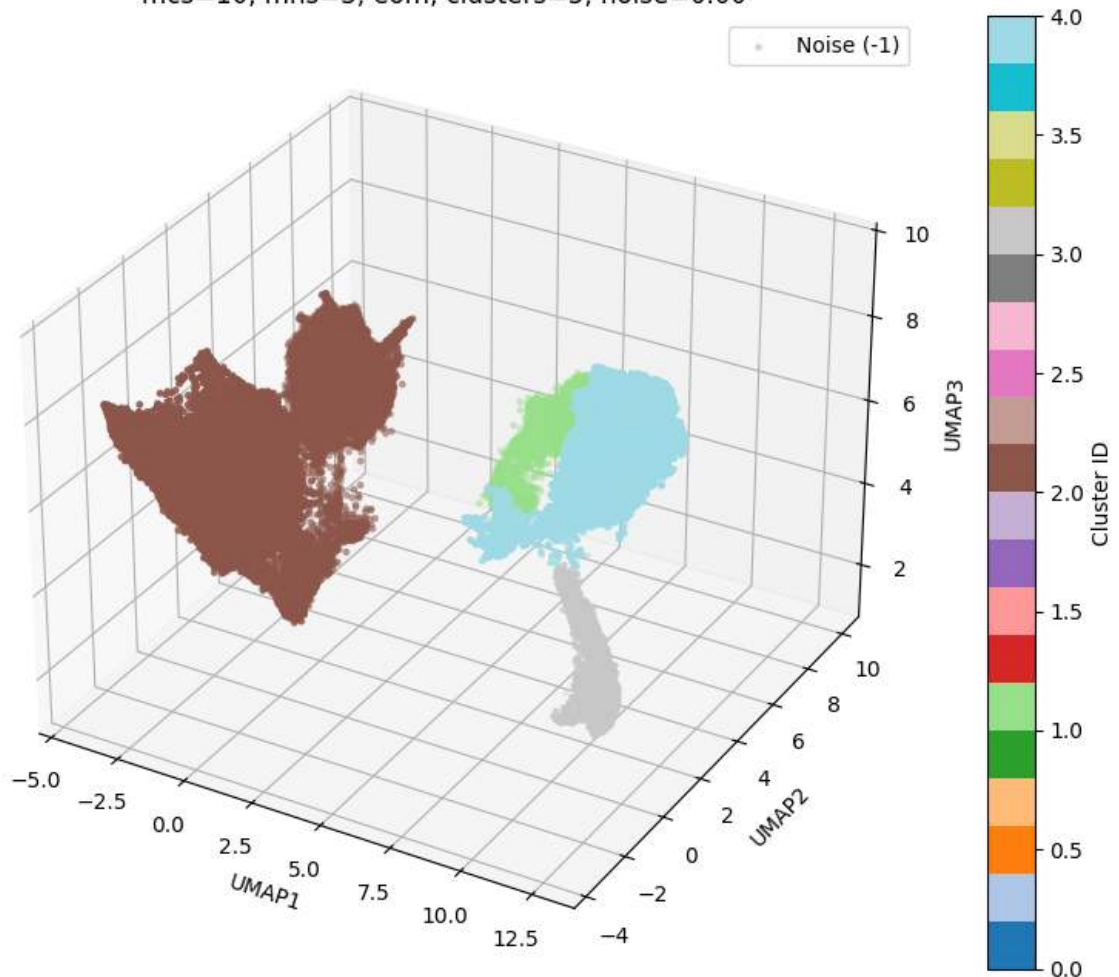
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN (Pipeline 2: UMAP-3D) — 21-30w — Pre-Eclampsia
 mcs=10, mns=3, eom, clusters=5, noise=0.00



[illegible]

[illegible]

[illegible]

[illegible]

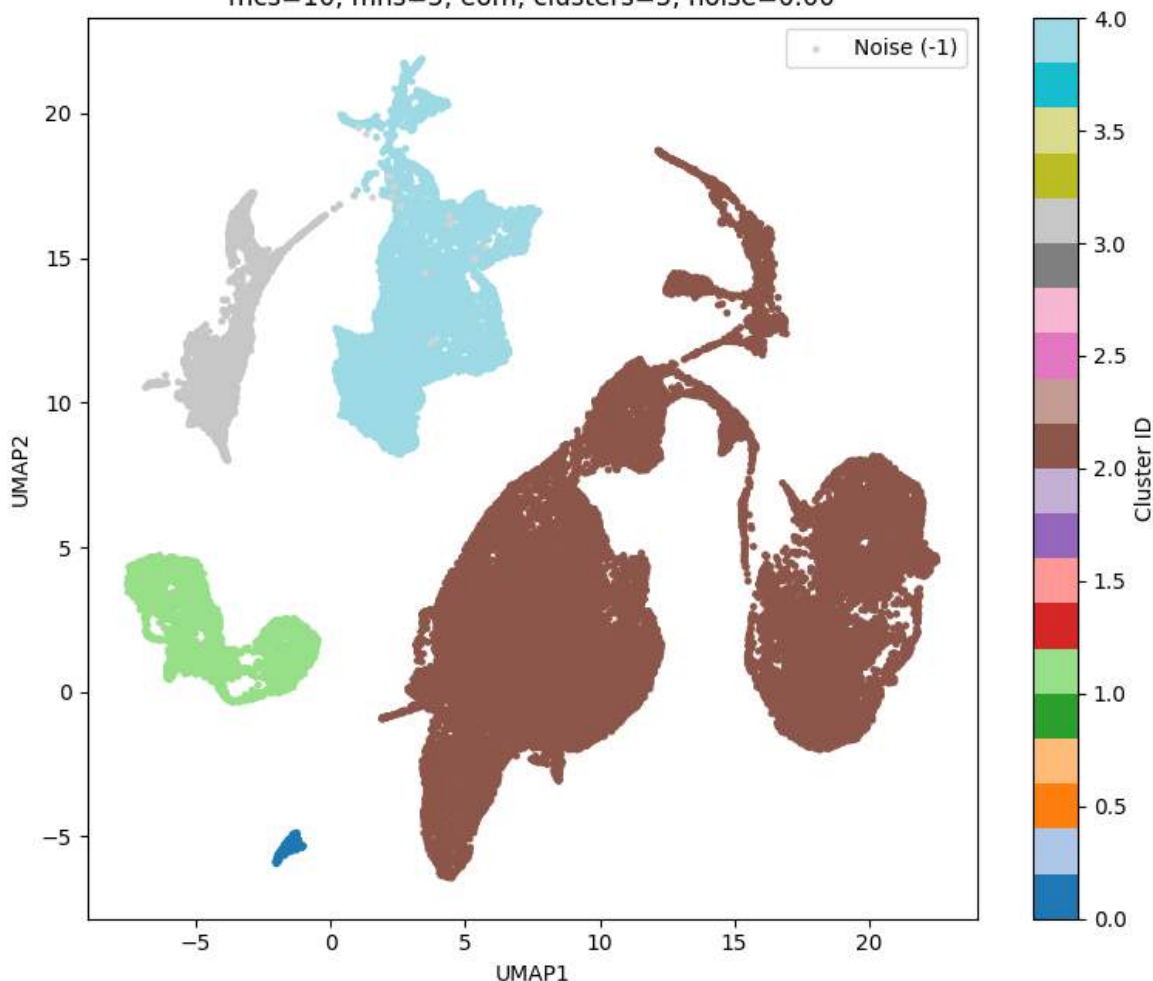
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(

```

HDBSCAN (Pipeline 2: UMAP-5D clustering; 2D viz) — 21-30w — Pre-Eclampsia
mcs=10, mns=5, eom, clusters=5, noise=0.00



Out[]:

	interval	condition	n_cells	umap_dim	min_cluster_size	min_samples	selection_me
0	1-10w	0	15000	2	10	3	
1	1-10w	0	15000	3	75	5	
2	1-10w	0	15000	5	10	3	
3	1-10w	1	5000	2	25	1	
4	1-10w	1	5000	3	10	3	
5	1-10w	1	5000	5	10	5	
6	11-20w	0	55000	2	60	3	
7	11-20w	0	55000	3	10	10	
8	11-20w	0	55000	5	10	1	
9	11-20w	1	65000	2	60	10	
10	11-20w	1	65000	3	60	1	
11	11-20w	1	65000	5	10	10	
12	21-30w	0	50000	2	40	5	
13	21-30w	0	50000	3	10	5	
14	21-30w	0	50000	5	10	1	
15	21-30w	1	40000	2	10	3	
16	21-30w	1	40000	3	10	3	
17	21-30w	1	40000	5	10	5	

Hyperparameter tuning Pipeline 2 HDBScan

```
In [ ]: # Paths
eda_path = "/content/drive/MyDrive/pre eclampsia/eda.csv" # - saved EDA
output_dir = "/content/drive/MyDrive/preeclampsia_results"
Path(output_dir).mkdir(parents=True, exist_ok=True)
```

Reloading EDA and defining age bin intervals

```
In [ ]: # reloading EDA and ensuring all is intact for clustering
eda_path = "/content/drive/MyDrive/pre eclampsia/eda.csv"

# Reload
eda = pd.read_csv(eda_path)

# Backup once (optional but recommended)
bak = eda_path.replace(".csv", ".bak.csv")
if not os.path.exists(bak):
    shutil.copyfile(eda_path, bak)
    print("Backup saved →", bak)

# Define EXACT labels used earlier (note the en-dash '-' and the '30+w' label)
```

```

labels = ["1-10w", "11-20w", "21-30w", "30+w"]

# Bin edges: [1,11), [11,21), [21,31), [31, inf)
bins = [1, 11, 21, 31, np.inf]

# Create/recreate categorical bins (left-closed, right-open)
eda["age_bin_10w"] = pd.cut(
    eda["age"],
    bins=bins,
    labels=labels,
    include_lowest=True,
    right=False
)

# Ensure categorical dtype with the full, ordered category list
eda["age_bin_10w"] = eda["age_bin_10w"].astype(pd.CategoricalDtype(categories=la

print("Counts per bin (including empty categories):")
print(eda["age_bin_10w"].value_counts(dropna=False).sort_index())

```

Backup saved → /content/drive/MyDrive/preeclampsia/eda.bak.csv

Counts per bin (including empty categories):

age_bin_10w

1-10w 20000

11-20w 120000

21-30w 90000

30+w 0

Name: count, dtype: int64

```

In [ ]: eda.to_csv(eda_path, index=False)
        print("eda.csv updated with age_bin_10w")

```

eda.csv updated with age_bin_10w

Setting constants

```

In [ ]: # Load EDA
        eda = pd.read_csv(eda_path)
        if "age_bin_10w" in eda.columns and not pd.api.types.is_categorical_dtype(eda["a
            eda["age_bin_10w"] = eda["age_bin_10w"].astype("category")

# Bins that actually have data
valid_bins = [b for b in eda["age_bin_10w"].cat.categories if (eda["age_bin_10w"
print("Bins:", list(map(str, valid_bins)))

COND_NAME      = {0:"Control", 1:"Pre-Eclampsia"}
RANDOM_STATE    = 42

```

Bins: ['11-20w', '1-10w', '21-30w']

/tmp/ipython-input-1899415307.py:3: DeprecationWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead

```

    if "age_bin_10w" in eda.columns and not pd.api.types.is_categorical_dtype(eda
["age_bin_10w"]):

```

```

In [ ]: # Defining functions for hyperparameter tuning 3d/5d
        def silhouette_nonnoise(X, labels):
            mask = (labels != -1)
            if mask.sum() < 3: return np.nan
            labs = labels[mask]
            if len(np.unique(labs)) < 2: return np.nan

```

```

    return silhouette_score(X[mask], labs)

def composite_score(sil, noise_frac):
    return np.nan_to_num(sil, nan=-1.0) - 0.5*float(noise_frac)

def plot_umap2(emb2, labels, title, out_png):
    plt.figure(figsize=(8,7))
    noise = (labels == -1)
    if (~noise).any():
        sc = plt.scatter(emb2[~noise,0], emb2[~noise,1], c=labels[~noise], s=6,
        else:
            sc = None
    if noise.any():
        plt.scatter(emb2[noise,0], emb2[noise,1], c="lightgrey", s=5, label="Noi
        plt.legend(frameon=True)
    plt.title(title); plt.xlabel("UMAP1"); plt.ylabel("UMAP2")
    if sc is not None: plt.colorbar(sc, label="Cluster ID")
    plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.show(); plt.close()

def plot_umap3(emb3, labels, title, out_png):
    from mpl_toolkits.mplot3d import Axes3D # noqa
    fig = plt.figure(figsize=(9,7))
    ax = fig.add_subplot(111, projection='3d')
    noise = (labels == -1)
    p = ax.scatter(emb3[~noise,0], emb3[~noise,1], emb3[~noise,2],
        c=labels[~noise], s=6, cmap="tab20")
    if noise.any():
        ax.scatter(emb3[noise,0], emb3[noise,1], emb3[noise,2], c="lightgrey", s
        ax.legend()
    ax.set_title(title); ax.set_xlabel("UMAP1"); ax.set_ylabel("UMAP2"); ax.set_
    fig.colorbar(p, label="Cluster ID")
    plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.show()

```

```

In [ ]: # checking to see if cached PCAs exist
eda = pd.read_csv(eda_path)
eda["age_bin_10w"] = eda["age_bin_10w"].astype("category")

NON_MARKER = {"condition", "age", "age_label", "filename", "age_bin_10w",
               "_umap1", "_umap2", "PCA_1", "PCA_2", "KMeans_Labels"}
numeric_cols = [c for c in eda.columns if pd.api.types.is_numeric_dtype(eda[c])]
MARKERS = [c for c in numeric_cols if c not in NON_MARKER]

valid_bins = [b for b in eda["age_bin_10w"].cat.categories if (eda["age_bin_10w"]
RANDOM_STATE = 42

for interval in valid_bins:
    for cond in [0,1]:
        sub = eda[(eda["age_bin_10w"]==interval) & (eda["condition"]==cond)]
        if sub.empty: continue
        tag = f"{interval}_{cond}"
        pfile = os.path.join(output_dir, f"pca20_{tag}.npz")
        if os.path.exists(pfile):
            print("PCA20 cached:", tag)
            continue
        X = sub[MARKERS].to_numpy(dtype="float32")
        Xs = StandardScaler().fit_transform(X)
        X_pca20 = PCA(n_components=20, random_state=RANDOM_STATE).fit_transform(
        np.save(pfile, X_pca20)
        print("PCA20 saved:", tag, X_pca20.shape)

```

PCA20 saved: 11-20w_0 (55000, 20)
 PCA20 saved: 11-20w_1 (65000, 20)
 PCA20 saved: 1-10w_0 (15000, 20)
 PCA20 saved: 1-10w_1 (5000, 20)
 PCA20 saved: 21-30w_0 (50000, 20)
 PCA20 saved: 21-30w_1 (40000, 20)

rebuiding umap

```
In [ ]: for interval in valid_bins:
        for cond in [0,1]:
            tag = f"{interval}_{cond}"
            pfile = os.path.join(output_dir, f"pca20_{tag}.npz")
            if not os.path.exists(pfile):
                print("Missing PCA20 for", tag, "> skip");
                continue
            X_pca20 = np.load(pfile)

            for d in (2,3,5):
                upath = os.path.join(output_dir, f"umap{d}_pipeline2_{tag}.npz")
                if os.path.exists(upath):
                    print(f"UMAP{d} cached:", tag);
                    continue
                reducer = umap.UMAP(n_components=d, n_neighbors=30, min_dist=0.3, ra
                emb = reducer.fit_transform(X_pca20)
                np.save(upath, emb)
                print(f"UMAP{d} saved:", tag, emb.shape)
```

/usr/local/lib/python3.12/dist-packages/umap/umap.py:1952: UserWarning: n_jobs v
 alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
 warn(

UMAP2 saved: 11-20w_0 (55000, 2)

/usr/local/lib/python3.12/dist-packages/umap/umap.py:1952: UserWarning: n_jobs v
 alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
 warn(

UMAP3 saved: 11-20w_0 (55000, 3)

/usr/local/lib/python3.12/dist-packages/umap/umap.py:1952: UserWarning: n_jobs v
 alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
 warn(

UMAP5 saved: 11-20w_0 (55000, 5)

/usr/local/lib/python3.12/dist-packages/umap/umap.py:1952: UserWarning: n_jobs v
 alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
 warn(

UMAP2 saved: 11-20w_1 (65000, 2)

/usr/local/lib/python3.12/dist-packages/umap/umap.py:1952: UserWarning: n_jobs v
 alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
 warn(

UMAP3 saved: 11-20w_1 (65000, 3)

/usr/local/lib/python3.12/dist-packages/umap/umap.py:1952: UserWarning: n_jobs v
 alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
 warn(

UMAP5 saved: 11-20w_1 (65000, 5)

/usr/local/lib/python3.12/dist-packages/umap/umap.py:1952: UserWarning: n_jobs v
 alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
 warn(

UMAP2 saved: 1-10w_0 (15000, 2)

```

/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
    warn(
UMAP3 saved: 1-10w_0 (15000, 3)
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
    warn(
UMAP5 saved: 1-10w_0 (15000, 5)
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
    warn(
UMAP2 saved: 1-10w_1 (5000, 2)
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
    warn(
UMAP3 saved: 1-10w_1 (5000, 3)
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
    warn(
UMAP5 saved: 1-10w_1 (5000, 5)
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
    warn(
UMAP2 saved: 21-30w_0 (50000, 2)
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
    warn(
UMAP3 saved: 21-30w_0 (50000, 3)
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
    warn(
UMAP5 saved: 21-30w_0 (50000, 5)
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
    warn(
UMAP2 saved: 21-30w_1 (40000, 2)
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
    warn(
UMAP3 saved: 21-30w_1 (40000, 3)
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
    warn(
UMAP5 saved: 21-30w_1 (40000, 5)

```

Defining tuning grid

```

In [ ]: def grid_for_n_umap(n):
        # Clamp mcs to n just in case, but your n is large
        mcs_list = [m for m in [10, 15, 20, 30] if m <= n]
        mns_list = [3, 5, 8, 10]
        methods = ["eom", "leaf"]
        return mcs_list, mns_list, methods

```

Running hyperparameter grid using umap embed


```

In [ ]: metrics_rows = []

for interval in valid_bins:
    for cond in [0,1]:
        tag = f"{interval}_{cond}"

        for d in (2,3,5):
            umap_path = os.path.join(output_dir, f"umap{d}_pipeline2_{tag}.npy")
            if not os.path.exists(umap_path):
                print(f"[P2] Missing UMAP{d} for {tag}, skipping.")
                continue

            emb = np.load(umap_path)
            n = len(emb)
            if n < 50:
                print(f"[P2] {tag} dim={d}: too few cells ({n})")
                continue

            best = None
            mcs_list, mns_list, meth_list = grid_for_n_umap(n)

            for mcs in mcs_list:
                for mns in mns_list:
                    for meth in meth_list:
                        model = hdbscan.HDBSCAN(
                            min_cluster_size=mcs,
                            min_samples=mns,
                            cluster_selection_method=meth,
                            core_dist_n_jobs=1
                        ).fit(emb)

                        labels = model.labels_.astype(int)
                        sil = silhouette_nonnoise(emb, labels)
                        noise_frac = (labels == -1).mean()
                        mean_prob = float(np.mean(model.proBABILITIES_))
                        n_clusters = int(len(np.setdiff1d(np.unique(labels), [-1]
                        comp = composite_score(sil, noise_frac)

                        cand = dict(dim=d, mcs=mcs, mns=mns, meth=meth, sil=sil,
                                noise=noise_frac, mean_prob=mean_prob,
                                n_clusters=n_clusters, composite=comp, label
                        if (best is None) or (cand["composite"] > best["composit
                            best = cand

            if best is None:
                print(f"[P2] {tag} dim={d}: no candidate found.")
                continue

            labels = best["labels"]
            sub_idx = eda[(eda['age_bin_10w']==interval) & (eda['condition']==cc

            # Save labels
            lab_name = f"hdbscan_pipeline2_umap{best['dim']}_tuned"
            labels_csv = os.path.join(output_dir, f"labels_{lab_name}_{tag}.csv")
            pd.Series(labels, index=sub_idx, name=lab_name).to_csv(labels_csv)

            # Save plot (5D gets a proper 3D viz through PCA on the 5D embedding)
            if best["dim"] == 2:
                out_png = os.path.join(output_dir, f"umap2_plot_{lab_name}_{tag}

```

```

        title = (f"HDBSCAN tuned (P2: UMAP-2D) - {interval} - {COND_NAME}
                f"mcs={best['mcs']}, mns={best['mns']}, {best['meth']},
                f"C={best['n_clusters']}, noise={best['noise']:.3f}, si
        plot_umap2(emb, labels, title, out_png)

    elif best["dim"] == 3:
        out_png = os.path.join(output_dir, f"umap3_plot_{lab_name}__{tag}
        title = (f"HDBSCAN tuned (P2: UMAP-3D) - {interval} - {COND_NAME}
                f"mcs={best['mcs']}, mns={best['mns']}, {best['meth']},
                f"C={best['n_clusters']}, noise={best['noise']:.3f}, si
        plot_umap3(emb, labels, title, out_png)

    else: # dim == 5 project UMAP-5D to 3D for visualization (PCA)
        emb3 = PCA(n_components=3, random_state=RANDOM_STATE).fit_transf
        np.save(os.path.join(output_dir, f"umap5_pca3viz_tuned__{tag}.np
        out_png = os.path.join(output_dir, f"umap5_pca3viz_plot_{lab_nam
        title = (f"HDBSCAN tuned (P2: UMAP-5D; 3D viz PCA) - {interval}
                f"mcs={best['mcs']}, mns={best['mns']}, {best['meth']},
                f"C={best['n_clusters']}, noise={best['noise']:.3f}, si
        plot_umap3(emb3, labels, title, out_png)

    # Metrics row
    metrics_rows.append([
        str(interval), cond, n, best["dim"], best["mcs"], best["mns"], b
        best["n_clusters"], best["noise"], best["mean_prob"], best["sil"
        best["composite"], out_png, labels_csv
    ])

# Save metrics
metrics_df = pd.DataFrame(metrics_rows, columns=[
    "interval", "condition", "n_cells", "umap_dim", "min_cluster_size", "min_samples"
    "n_clusters", "noise_frac", "mean_prob", "silhouette_nonnoise", "composite", "plo
])
out_metrics = os.path.join(output_dir, "hdbscan_pipeline2_metrics_TUNED.csv")
metrics_df.to_csv(out_metrics, index=False)
metrics_df

```

[illegible]

[illegible]

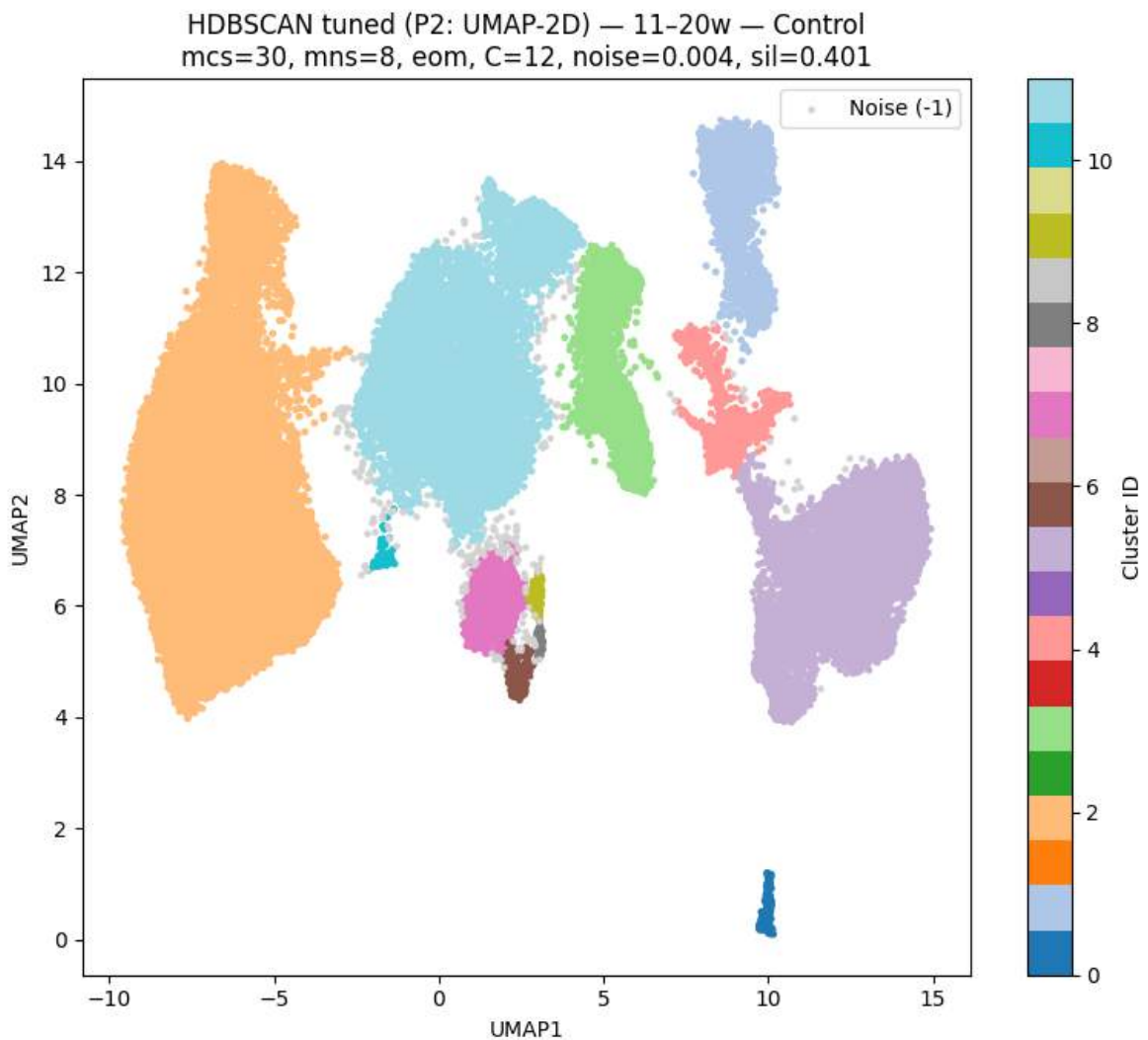
[illegible]

[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```



[illegible]

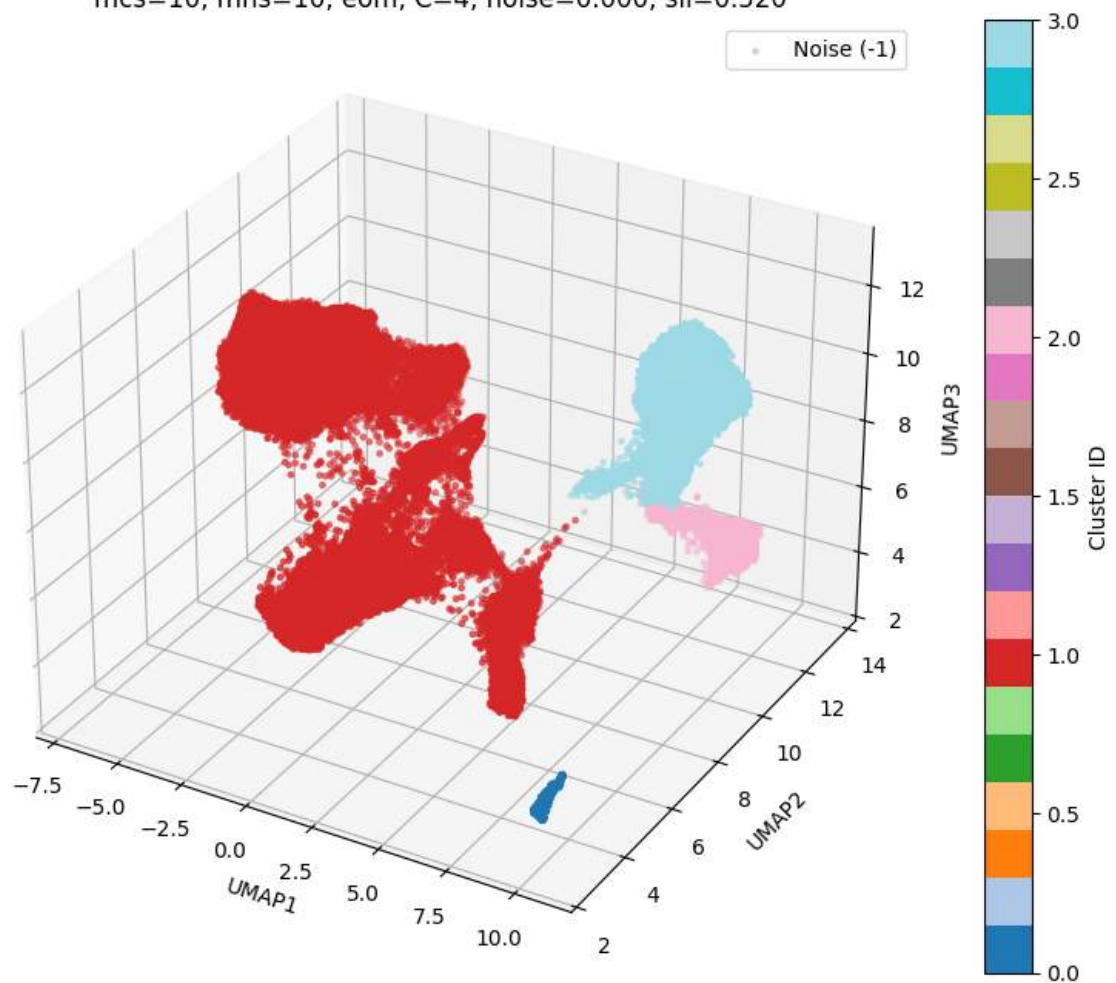
[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```

HDBSCAN tuned (P2: UMAP-3D) — 11-20w — Control
mcs=10, mns=10, eom, C=4, noise=0.000, sil=0.520



[illegible]

[illegible]

[illegible]

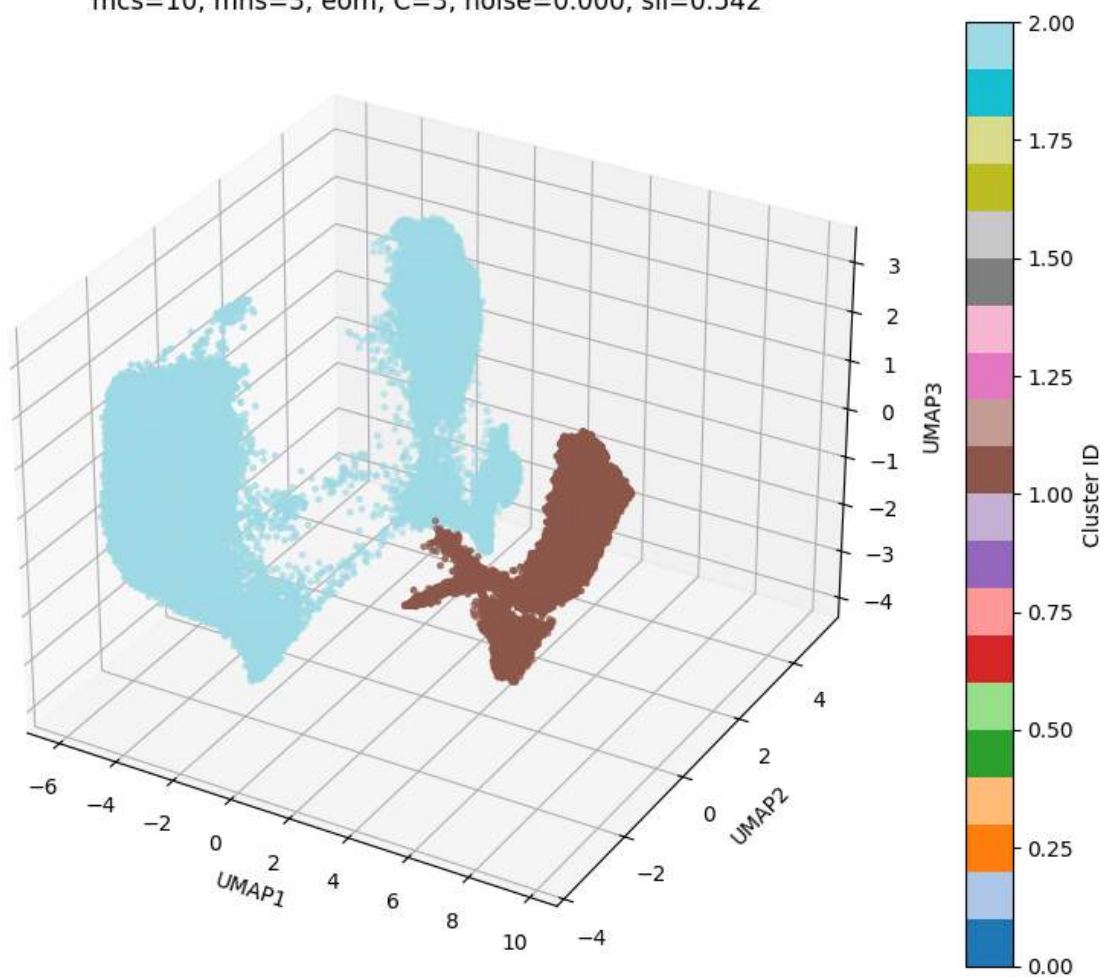
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN tuned (P2: UMAP-5D; 3D viz PCA) — 11-20w — Control
 mcs=10, mns=3, eom, C=3, noise=0.000, sil=0.542



[illegible]

[illegible]

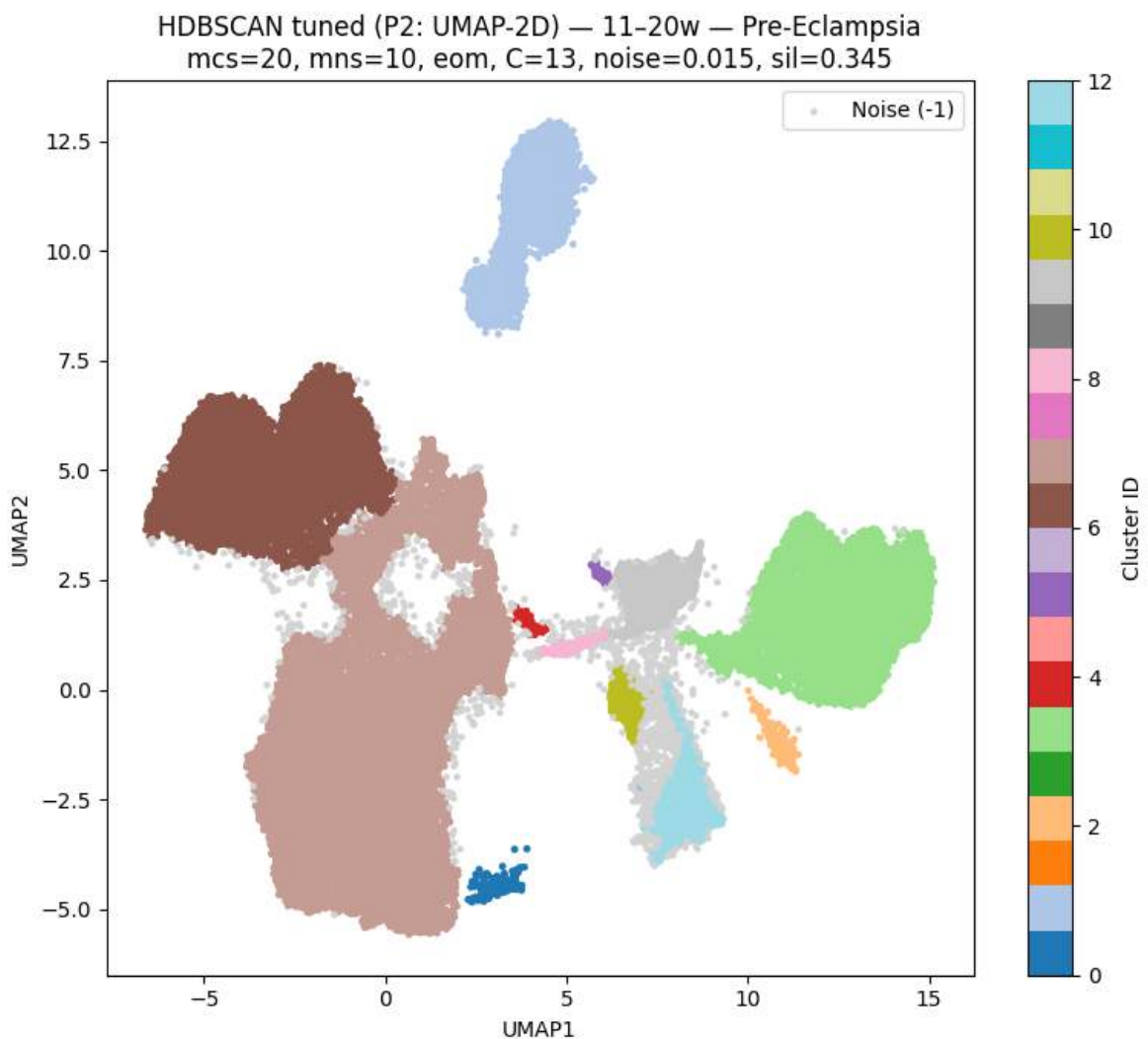
[illegible]

[illegible]


```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```



[illegible]

[illegible]

[illegible]

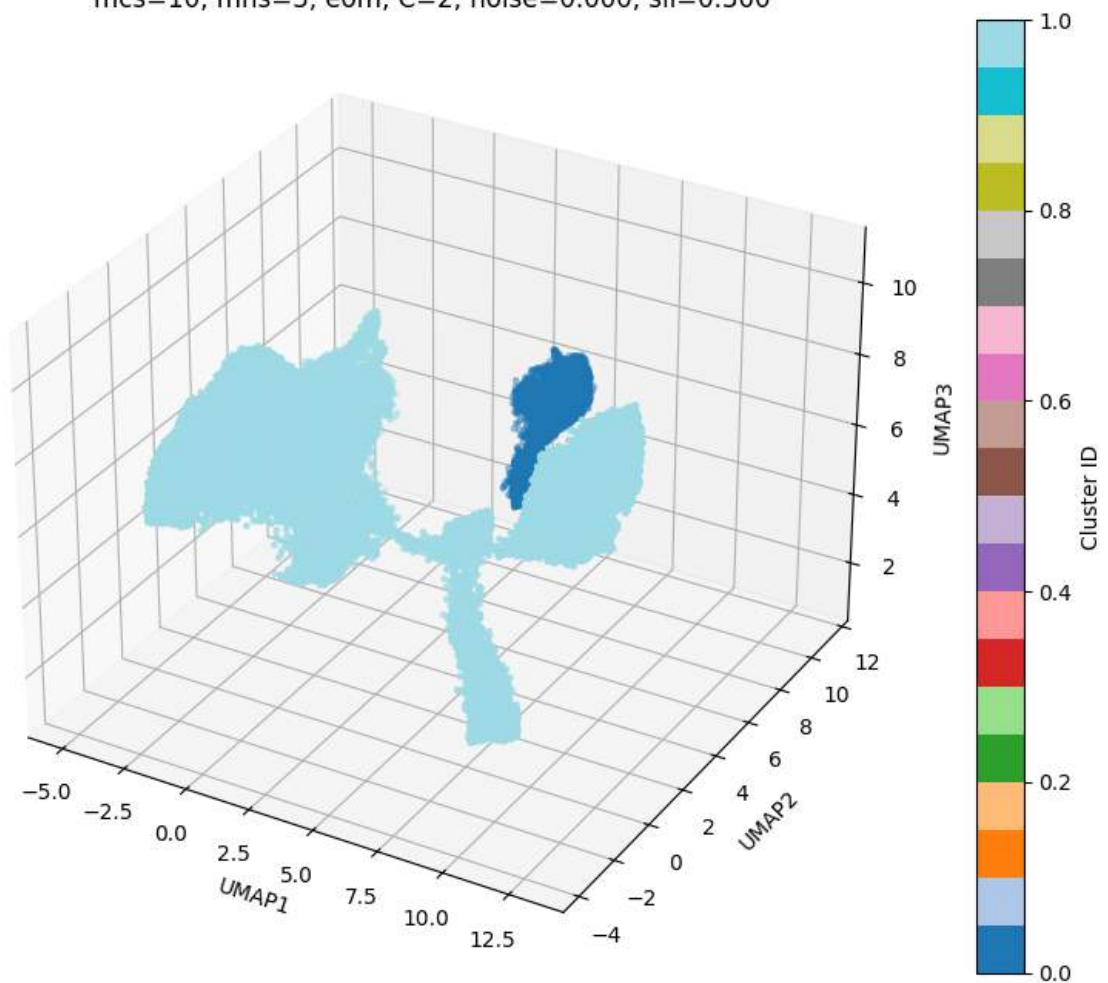
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN tuned (P2: UMAP-3D) — 11-20w — Pre-Eclampsia
 mcs=10, mns=3, eom, C=2, noise=0.000, sil=0.300



[illegible]

[illegible]

[illegible]

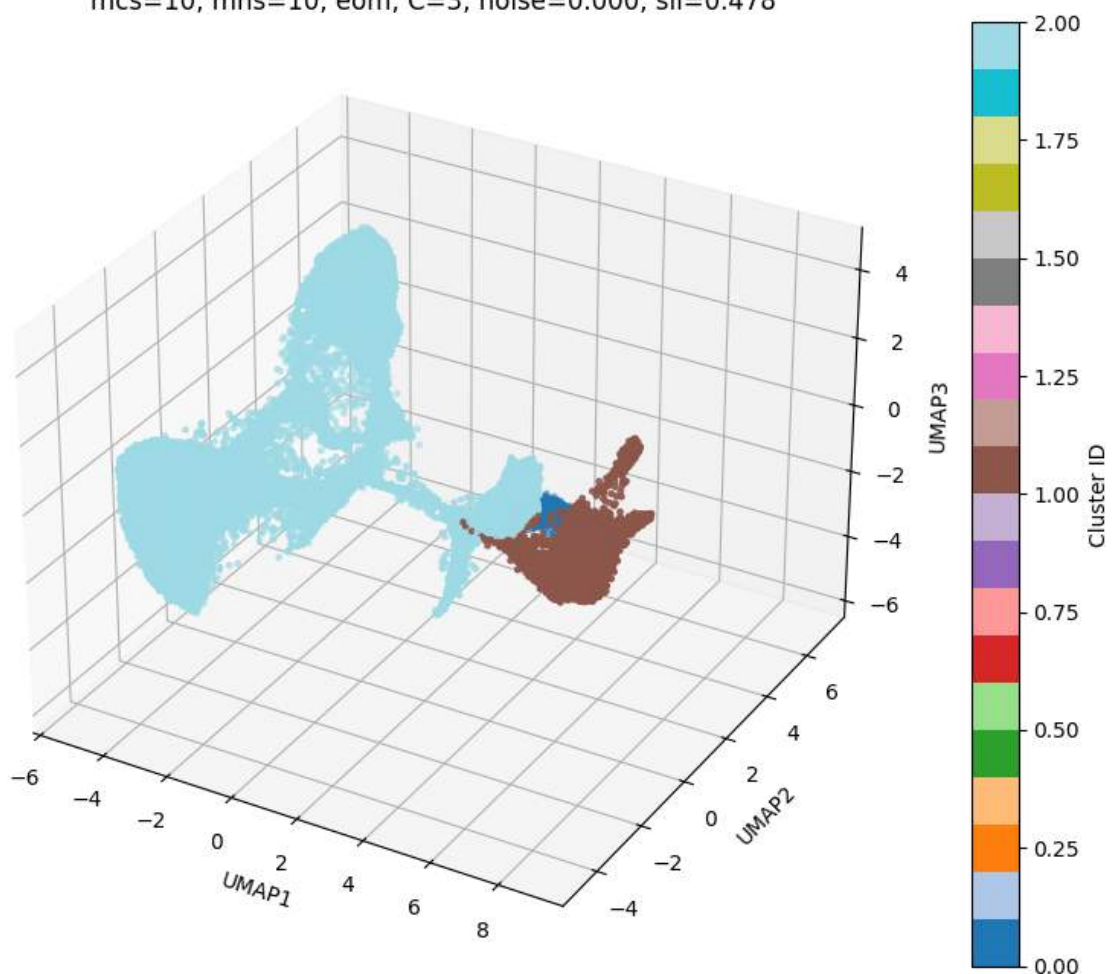
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN tuned (P2: UMAP-5D; 3D viz PCA) — 11-20w — Pre-Eclampsia
 mcs=10, mns=10, eom, C=3, noise=0.000, sil=0.478



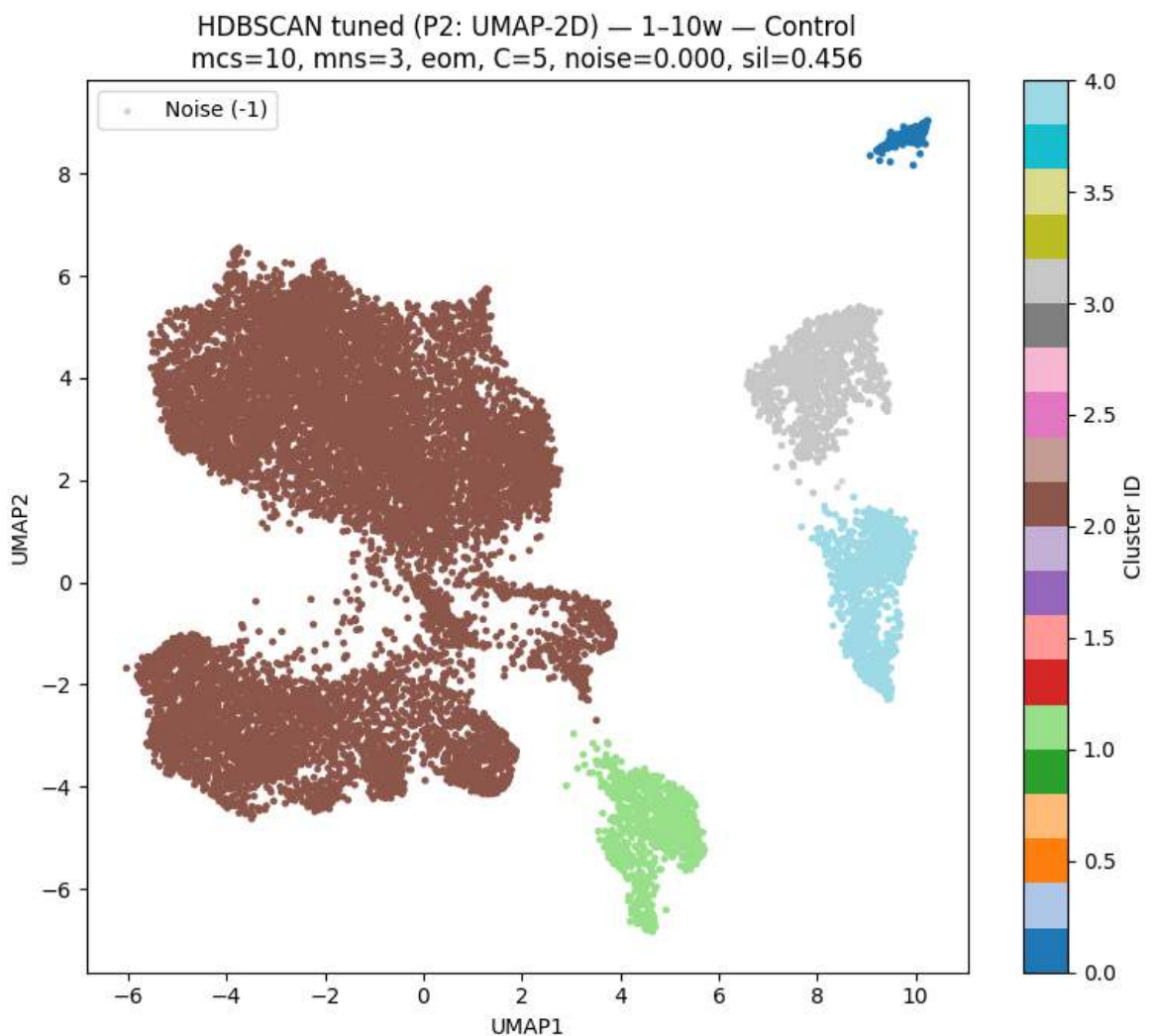
[illegible]

[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```



[illegible]

[illegible]

[illegible]

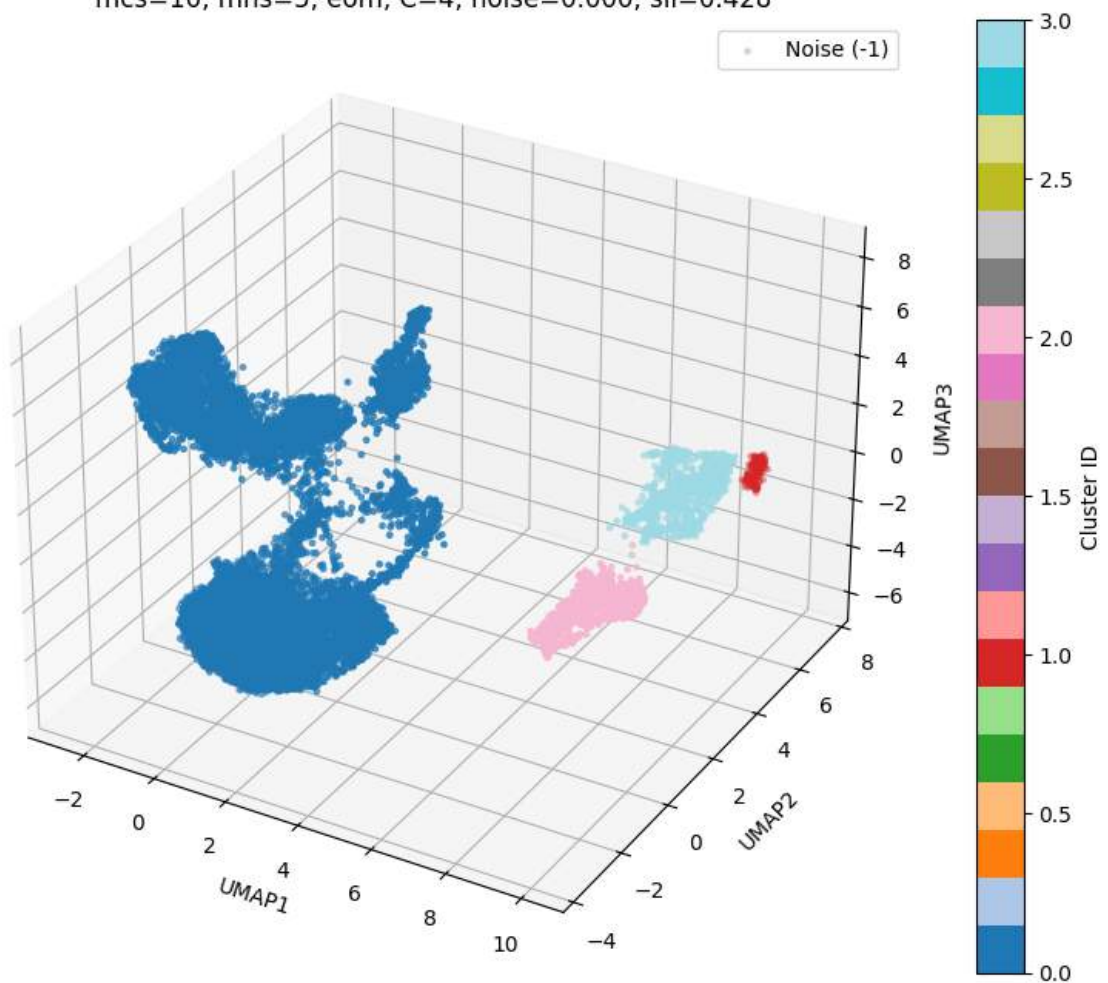
[illegible]


```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN tuned (P2: UMAP-3D) — 1-10w — Control
mcs=10, mns=5, eom, C=4, noise=0.000, sil=0.428



[illegible]

[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

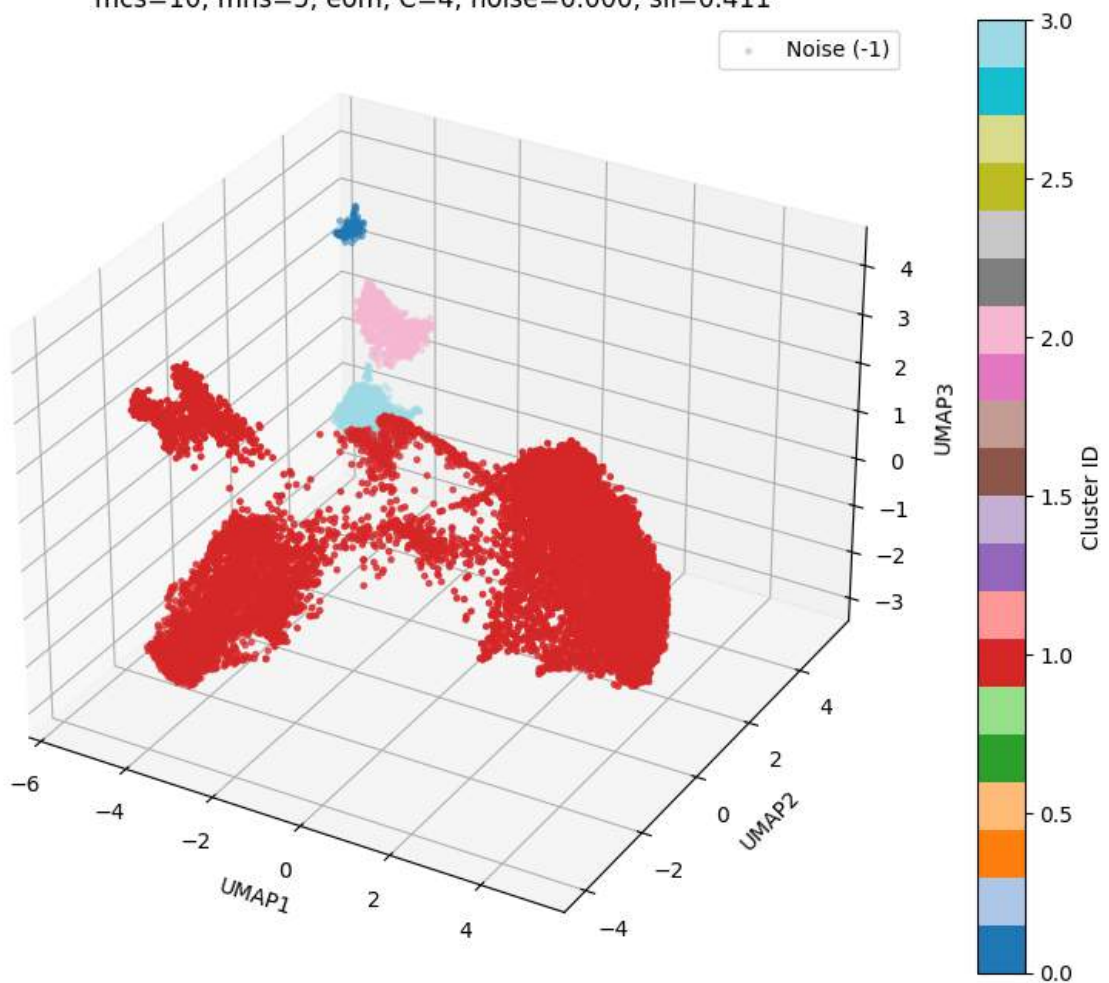
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN tuned (P2: UMAP-5D; 3D viz PCA) — 1-10w — Control
 mcs=10, mns=5, eom, C=4, noise=0.000, sil=0.411



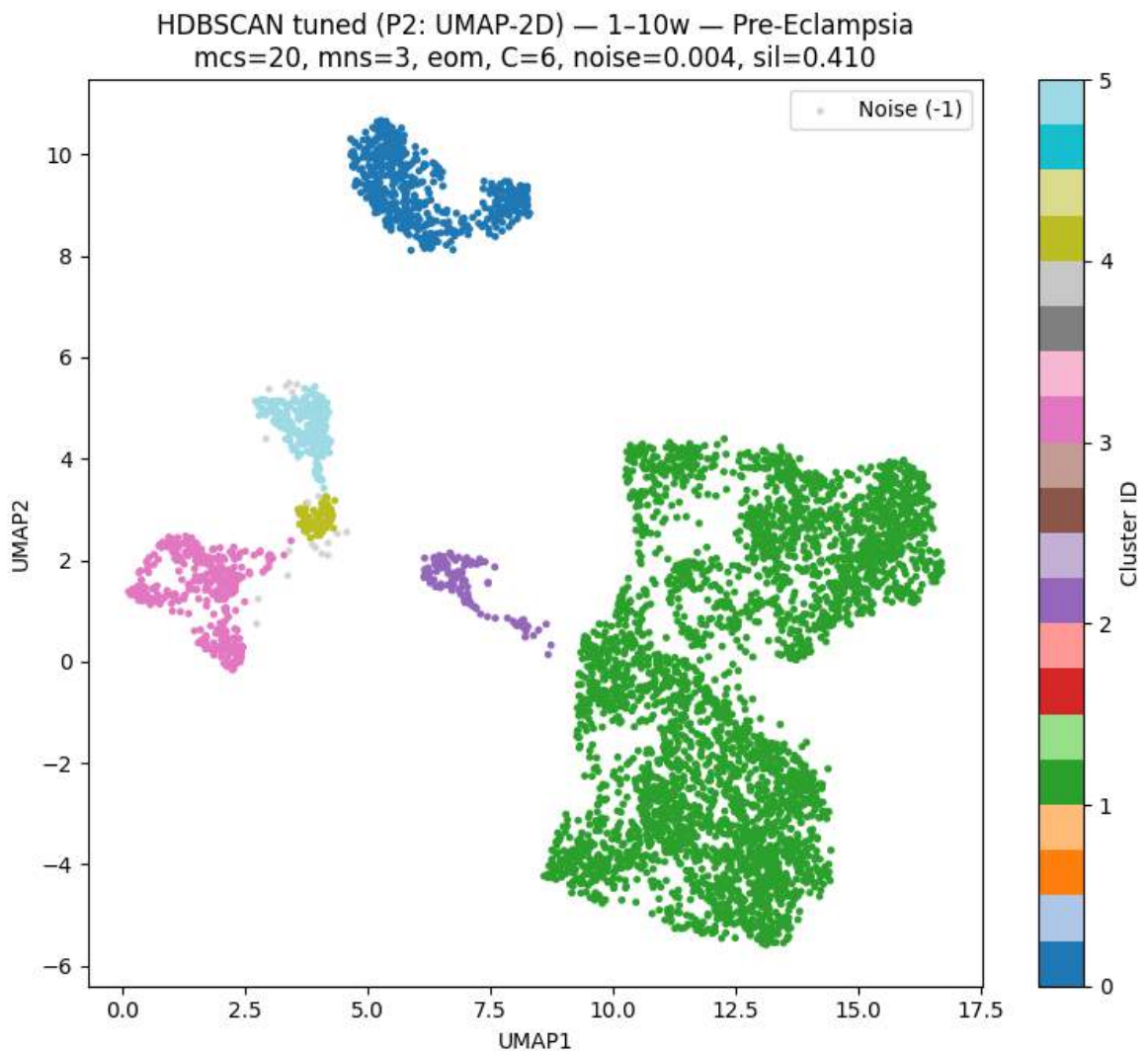
[illegible]

[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```



[illegible]

[illegible]

[illegible]

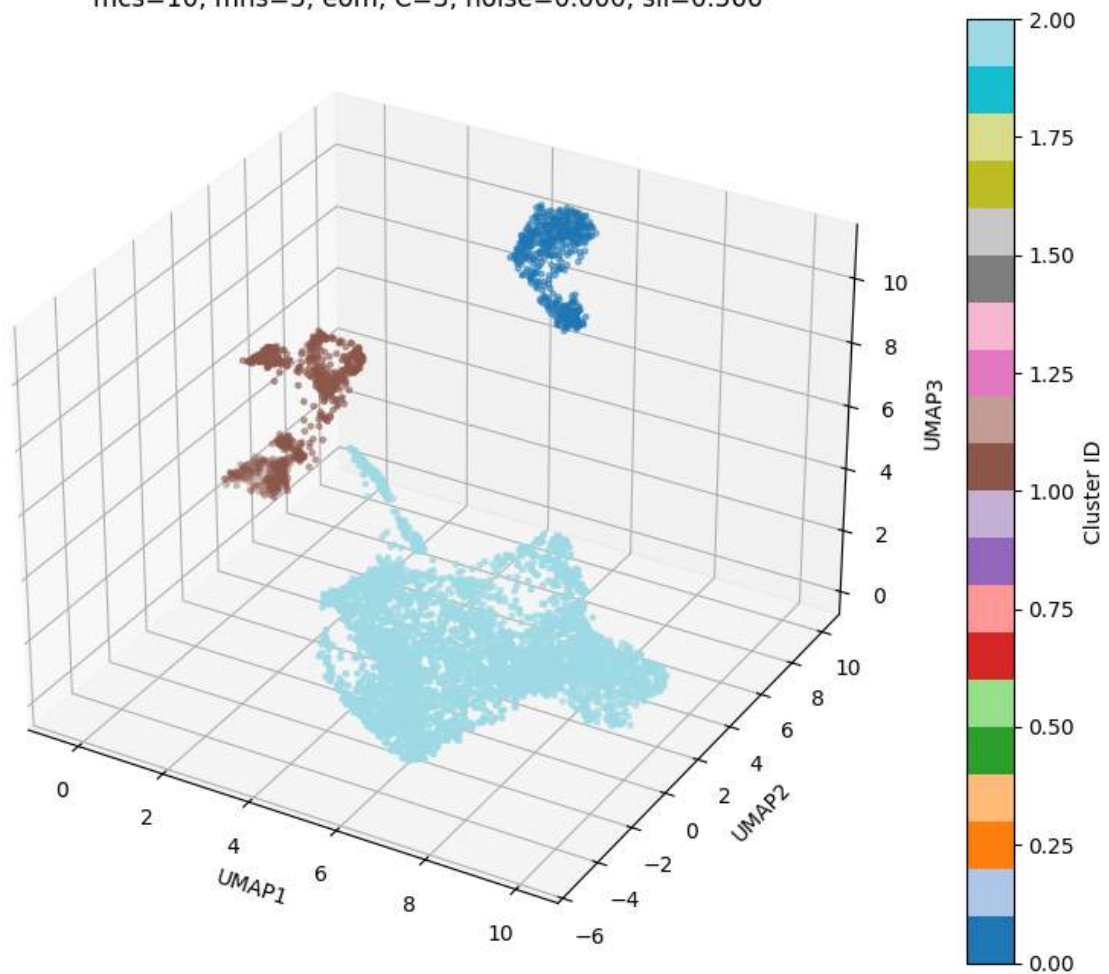
[illegible]


```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN tuned (P2: UMAP-3D) — 1-10w — Pre-Eclampsia
 mcs=10, mns=3, eom, C=3, noise=0.000, sil=0.566



[illegible]

[illegible]

[illegible]

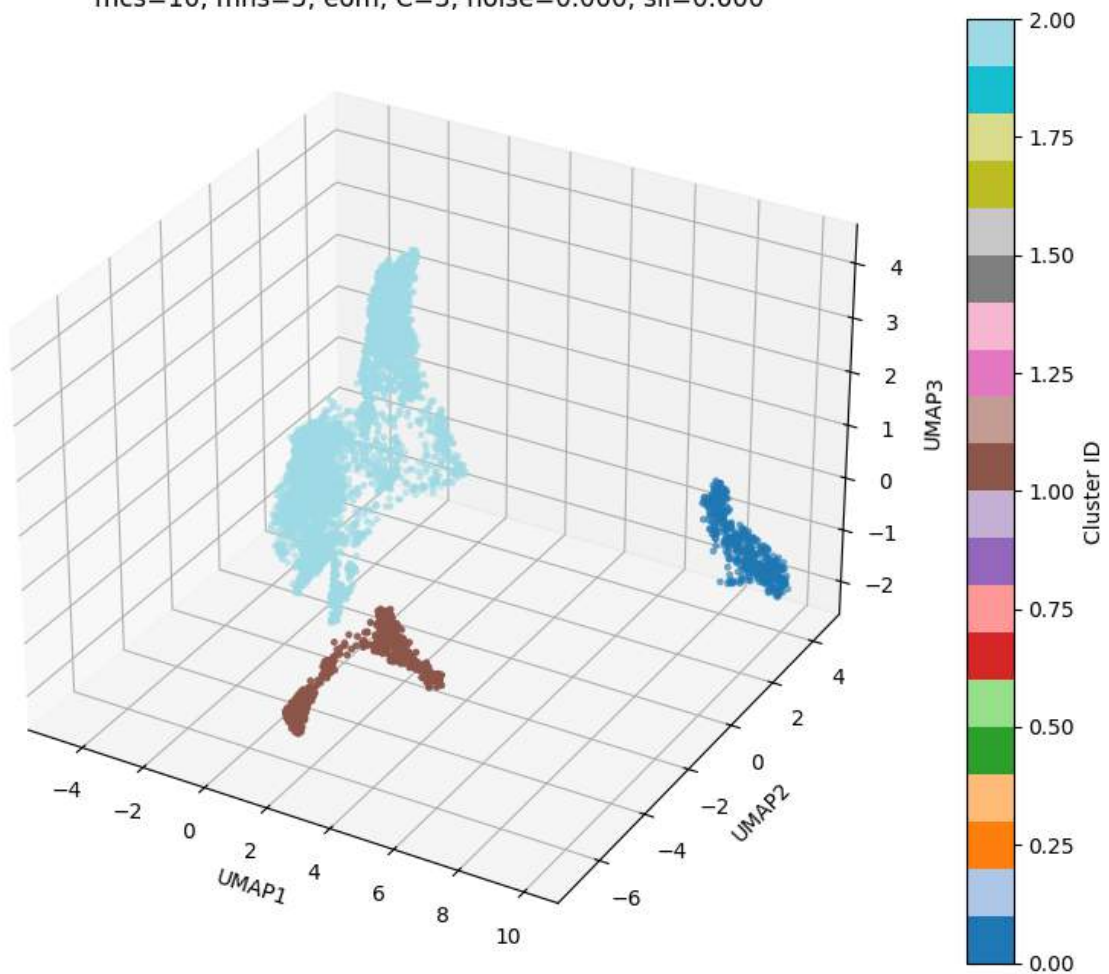
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN tuned (P2: UMAP-5D; 3D viz PCA) — 1-10w — Pre-Eclampsia
 mcs=10, mns=5, eom, C=3, noise=0.000, sil=0.600



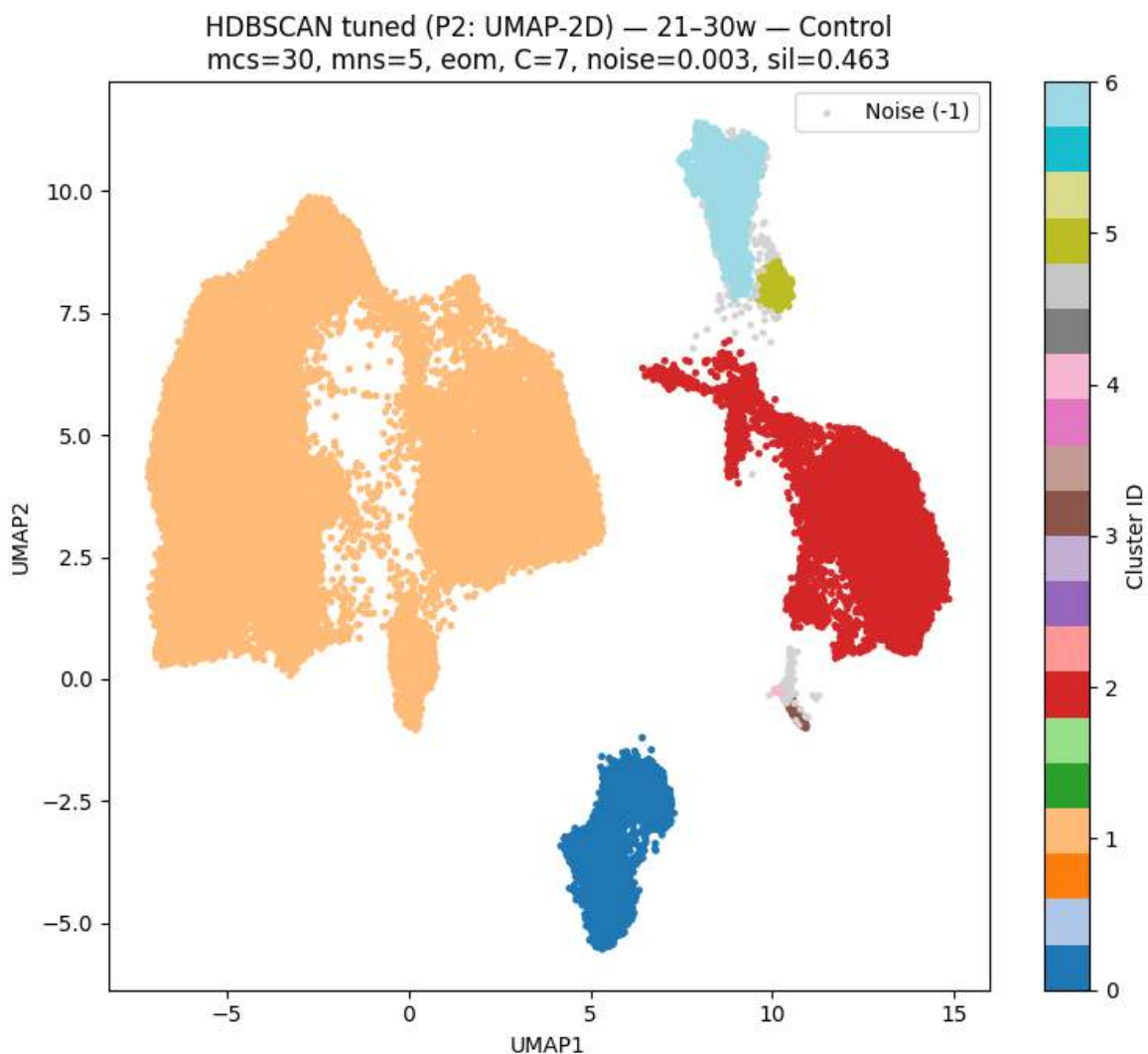
[illegible]

[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```



[illegible]

[illegible]

[illegible]

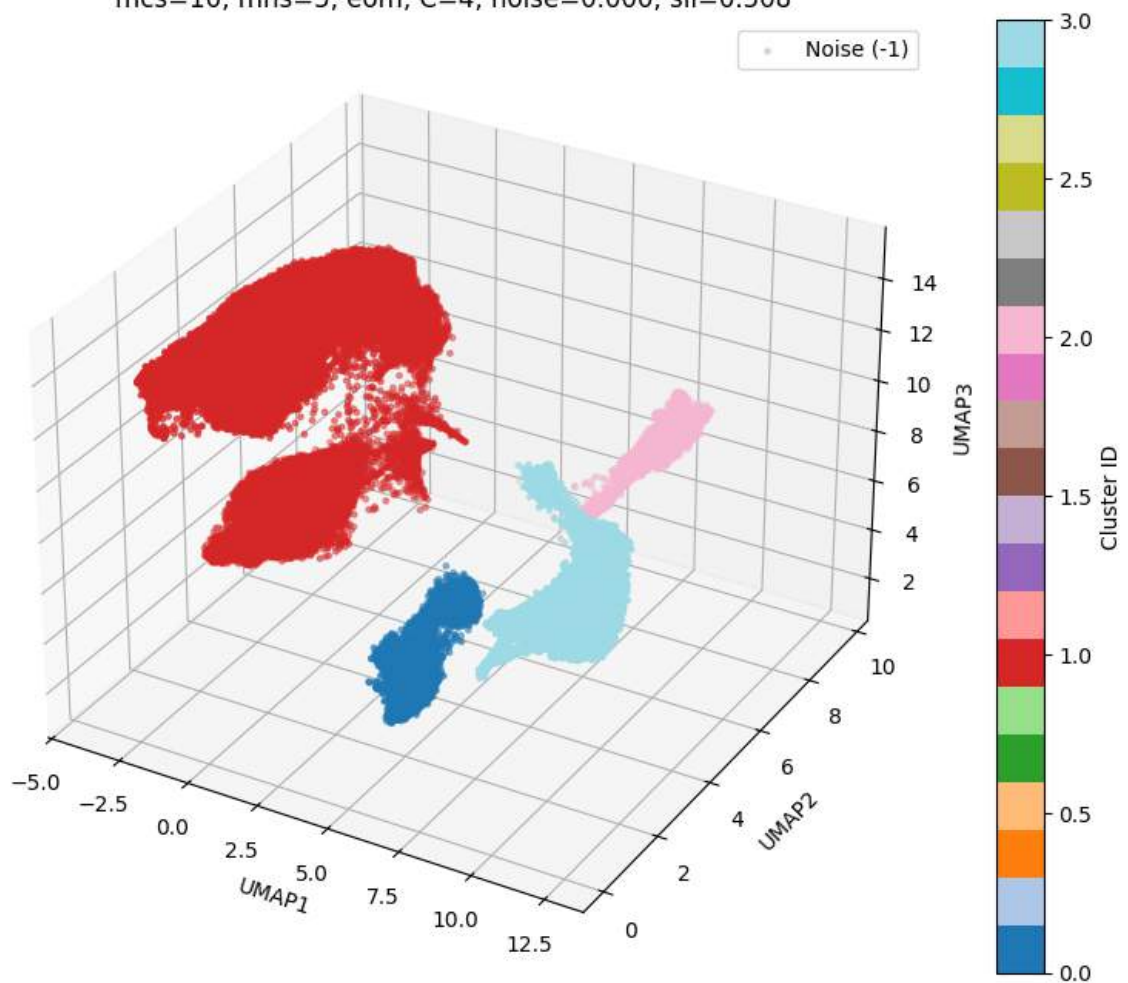
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN tuned (P2: UMAP-3D) — 21-30w — Control
 mcs=10, mns=5, eom, C=4, noise=0.000, sil=0.508



[illegible]

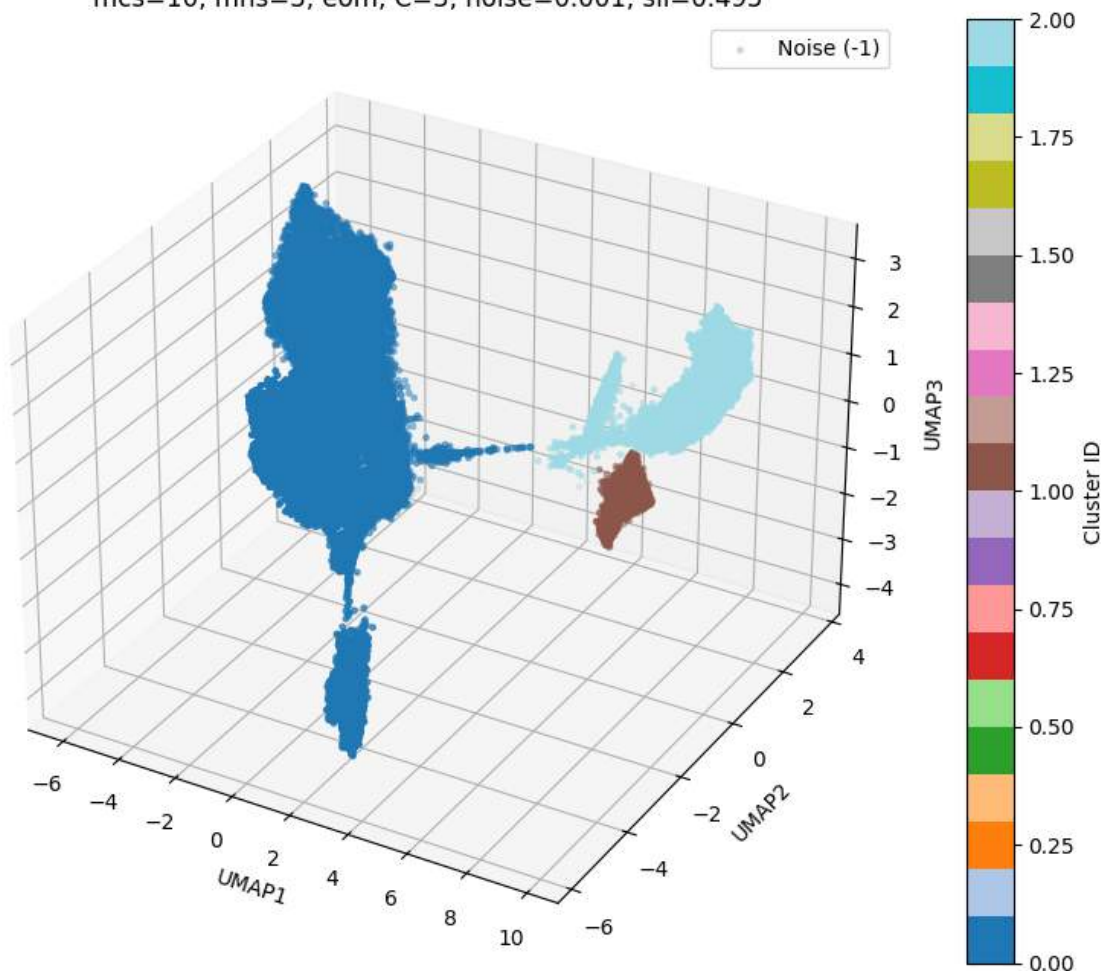
[illegible]

[illegible]

[illegible]


```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```

HDBSCAN tuned (P2: UMAP-5D; 3D viz PCA) — 21-30w — Control
mcs=10, mns=3, eom, C=3, noise=0.001, sil=0.495



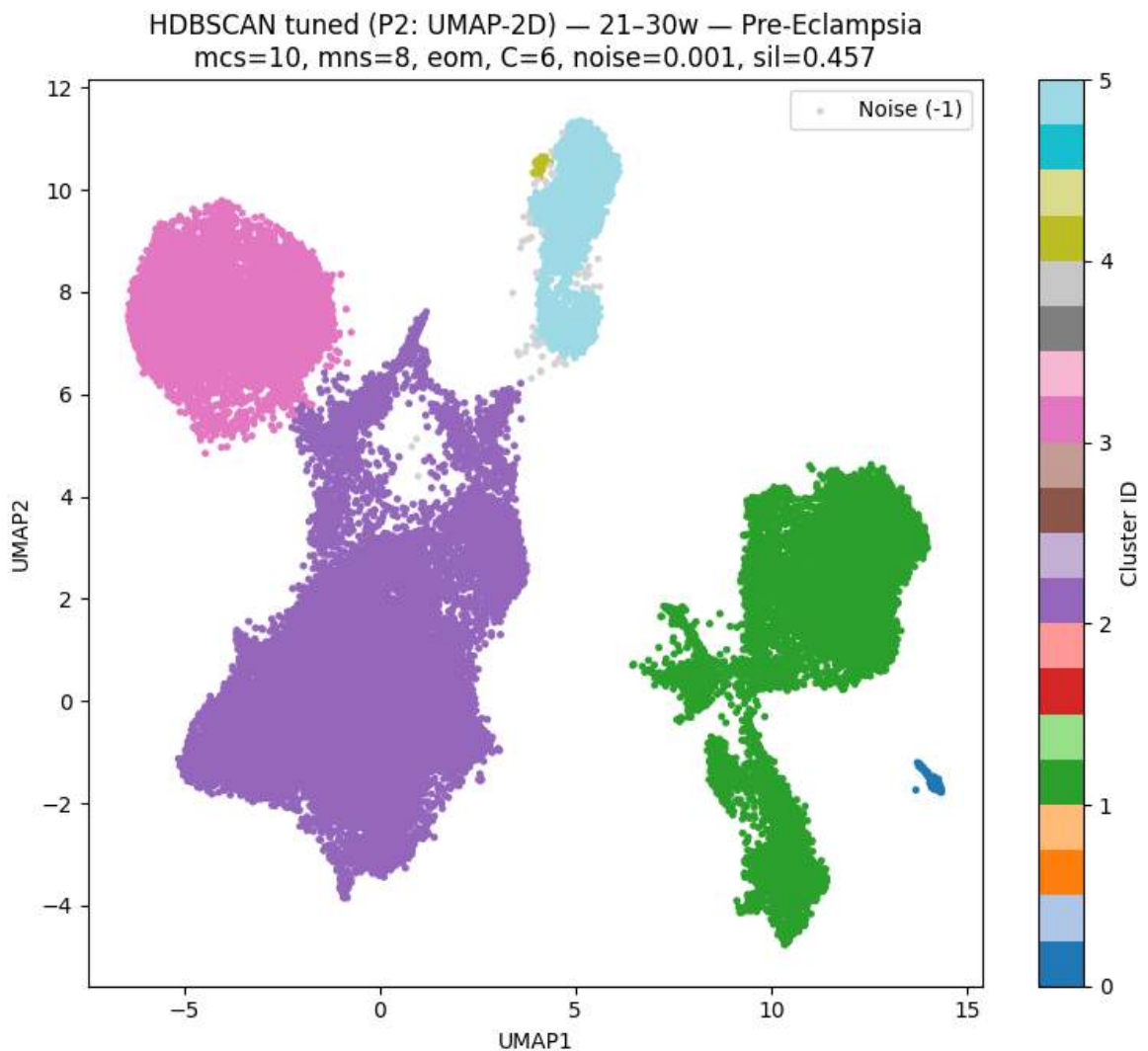
[illegible]

[illegible]

[illegible]

[illegible]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```



[illegible]

[illegible]

[illegible]

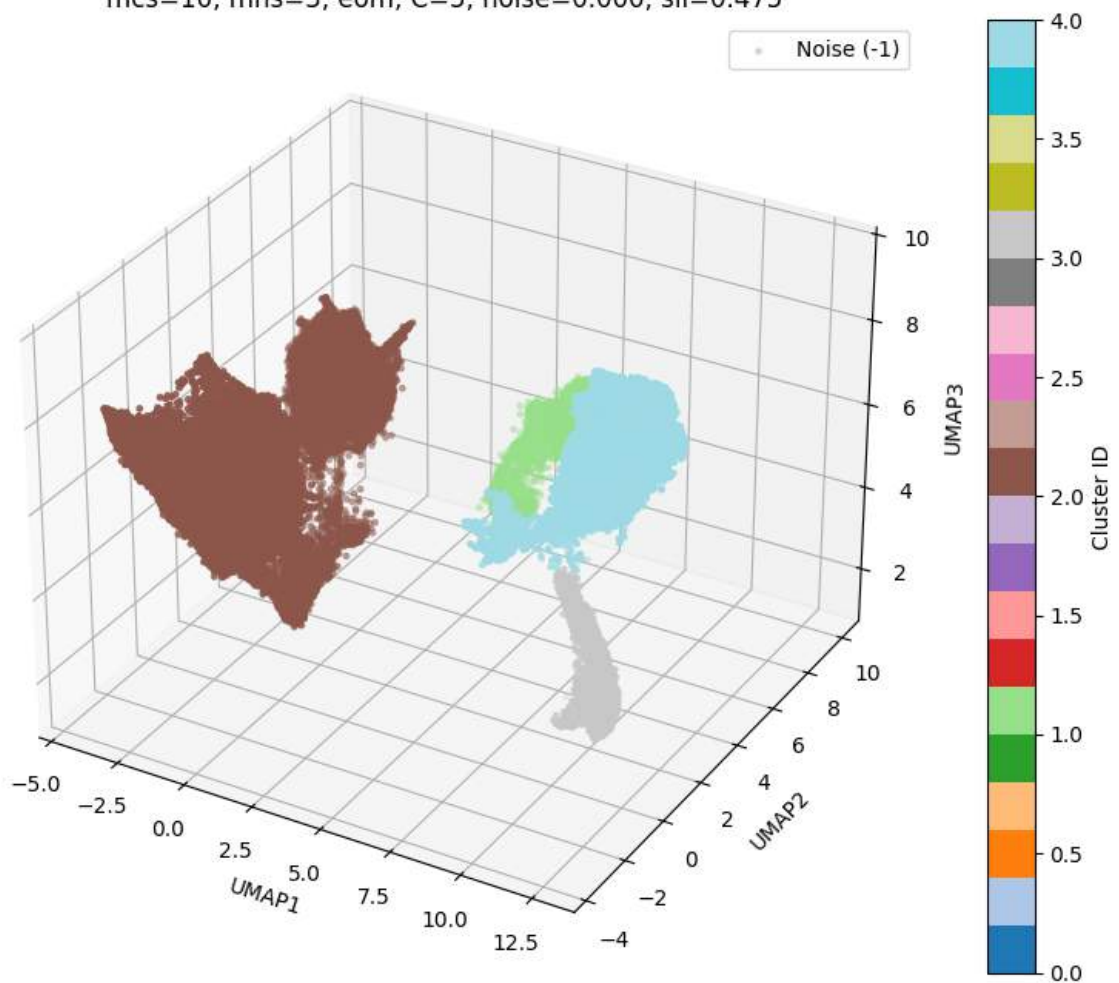
[illegible]

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

HDBSCAN tuned (P2: UMAP-3D) — 21-30w — Pre-Eclampsia
 mcs=10, mns=3, eom, C=5, noise=0.000, sil=0.475



[illegible]

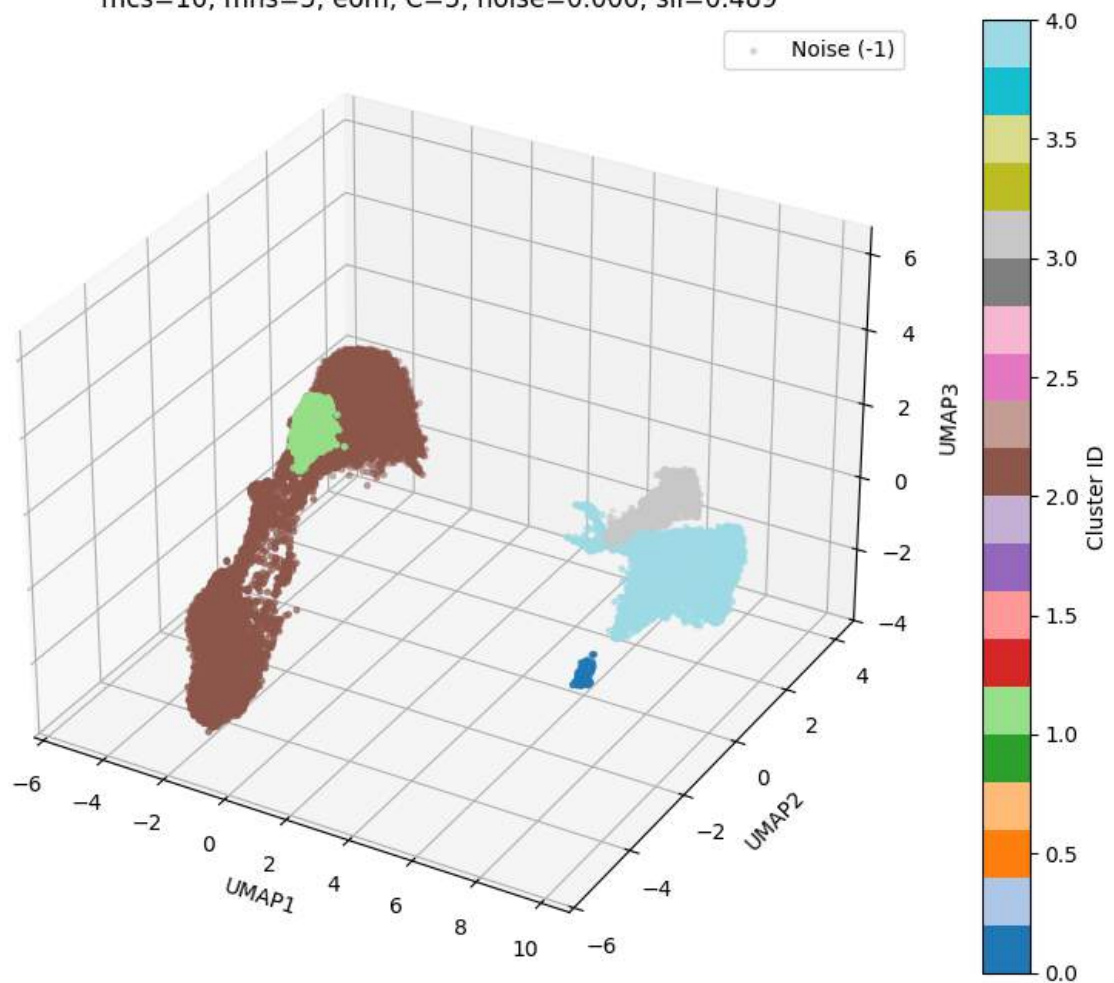
[illegible]

[illegible]

[illegible]


```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```

HDBSCAN tuned (P2: UMAP-5D; 3D viz PCA) — 21-30w — Pre-Eclampsia
mcs=10, mns=5, eom, C=5, noise=0.000, sil=0.489



Out[]:

	interval	condition	n_cells	umap_dim	min_cluster_size	min_samples	selection_me
0	11-20w	0	55000	2	30	8	
1	11-20w	0	55000	3	10	10	
2	11-20w	0	55000	5	10	3	
3	11-20w	1	65000	2	20	10	
4	11-20w	1	65000	3	10	3	
5	11-20w	1	65000	5	10	10	
6	1-10w	0	15000	2	10	3	
7	1-10w	0	15000	3	10	5	
8	1-10w	0	15000	5	10	5	
9	1-10w	1	5000	2	20	3	
10	1-10w	1	5000	3	10	3	
11	1-10w	1	5000	5	10	5	
12	21-30w	0	50000	2	30	5	
13	21-30w	0	50000	3	10	5	
14	21-30w	0	50000	5	10	3	
15	21-30w	1	40000	2	10	8	
16	21-30w	1	40000	3	10	3	
17	21-30w	1	40000	5	10	5	

Phenograph clustering

Defining path, searching for age interval bins and setting random seed

```
In [ ]: # Paths
eda_path = "/content/drive/MyDrive/pre eclampsia/eda.csv"
output_dir = "/content/drive/MyDrive/preeclampsia_results"
Path(output_dir).mkdir(parents=True, exist_ok=True)

# Load EDA
eda = pd.read_csv(eda_path)
if "age_bin_10w" in eda.columns and not pd.api.types.is_categorical_dtype(eda["age_bin_10w"]):
    # ensure exact category order to match your filenames
    cats = ["1-10w", "11-20w", "21-30w", "30+w"]
    eda["age_bin_10w"] = eda["age_bin_10w"].astype(pd.CategoricalDtype(categories=cats))

# Bins with data
valid_bins = [b for b in eda["age_bin_10w"].cat.categories if (eda["age_bin_10w"] == b).sum() > 0]
COND_NAME = {0: "Control", 1: "Pre-Eclampsia"}
RANDOM_STATE = 42

print("Bins with data:", list(map(str, valid_bins)))
```

Bins with data: ['1-10w', '11-20w', '21-30w']

```
/tmp/ipython-input-3560726876.py:8: DeprecationWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead
```

```
if "age_bin_10w" in eda.columns and not pd.api.types.is_categorical_dtype(eda["age_bin_10w"]):
```

Defining relevant function for Phenograph in pipeline 1

```
In [ ]: def sil_or_nan(X, labels):
        if len(np.unique(labels)) < 2 or X.shape[0] < 3:
            return np.nan
        return silhouette_score(X, labels)

def composite_score_pheno(sil, Q, Qmin=-0.1, Qmax=1.0):
    """
    Combine silhouette (primary) + Louvain modularity Q (secondary).
    Normalize Q roughly to [0,1] before mixing.
    """
    qn = (Q - Qmin) / (Qmax - Qmin)
    qn = float(np.clip(qn, 0.0, 1.0))
    sil = float(np.nan_to_num(sil, nan=-1.0))
    return 0.7*sil + 0.3*qn

def plot_umap2(emb2, labels, title, out_png):
    plt.figure(figsize=(8,7))
    sc = plt.scatter(emb2[:,0], emb2[:,1], c=labels, s=6, cmap="tab20")
    plt.title(title); plt.xlabel("UMAP1"); plt.ylabel("UMAP2")
    plt.colorbar(sc, label="Cluster ID")
    plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.show(); plt.close()

def plot_umap3(emb3, labels, title, out_png):
    from mpl_toolkits.mplot3d import Axes3D # noqa
    fig = plt.figure(figsize=(9,7))
    ax = fig.add_subplot(111, projection='3d')
    p = ax.scatter(emb3[:,0], emb3[:,1], emb3[:,2], c=labels, s=6, cmap="tab20")
    ax.set_title(title); ax.set_xlabel("UMAP1"); ax.set_ylabel("UMAP2"); ax.set_zlabel("UMAP3")
    fig.colorbar(p, label="Cluster ID")
    plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.show()
```

Pipeline 1: Phenograph clustering in PCA space, visualizing in UMAP

```
In [ ]: # PhenoGraph k grid
K_GRID_P1 = [15, 30, 50, 75]

p1_rows = []

for interval in valid_bins:
    for cond in [0,1]:
        tag = f"{interval}_{cond}"
        pca_path = os.path.join(output_dir, f"pca20_{tag}.npy")
        if not os.path.exists(pca_path):
            print("[P1] Missing PCA20 for", tag, "- skip");
            continue

        X_pca20 = np.load(pca_path)
        sub_idx = eda[(eda["age_bin_10w"]==interval) & (eda["condition"]==cond)]
        n = len(X_pca20)
        if n < 50:
```

```

print(f"[P1] {tag}: too few cells ({n})")
continue

best = None
for k in K_GRID_P1:
    try:
        communities, graph, Q = phenograph.cluster(X_pca20, k=k) # Louv
    except Exception as e:
        print(f"[P1] {tag} k={k} failed:", e)
        continue

    labels = np.asarray(communities, dtype=int)
    sil = sil_or_nan(X_pca20, labels)
    comp = composite_score_pheno(sil, Q)

    cand = dict(k=k, sil=sil, Q=Q, composite=comp, labels=labels)
    if (best is None) or (cand["composite"] > best["composite"]):
        best = cand

    # record per-k metrics
    p1_rows.append([
        str(interval), cond, n, k, sil, Q, comp,
        "", "" # plot_png, labels_csv (filled for best below)
    ])

if best is None:
    print(f"[P1] {tag}: no candidate found.")
    continue

# save BEST labels aligned to eda rows
labels = best["labels"]
lab_name = f"phenograph_pipeline1_pca20_k{best['k']}"
labels_csv = os.path.join(output_dir, f"labels_{lab_name}__{tag}.csv")
pd.Series(labels, index=sub_idx, name=lab_name).to_csv(labels_csv)

# UMAP-2D viz from PCA20 (viz only)
umap2 = umap.UMAP(n_components=2, n_neighbors=30, min_dist=0.3, random_s
emb2 = umap2.fit_transform(X_pca20)
np.save(os.path.join(output_dir, f"umap2_p1viz__{tag}.npy"), emb2)

out_png = os.path.join(output_dir, f"umap2_plot_{lab_name}__{tag}.png")
title = (f"PhenoGraph (Pipeline 1: PCA-20) - {interval} - {COND_NAME[c
        f"k={best['k']}, Q={best['Q']:.3f}, sil={best['sil']:.3f}")
plot_umap2(emb2, labels, title, out_png)

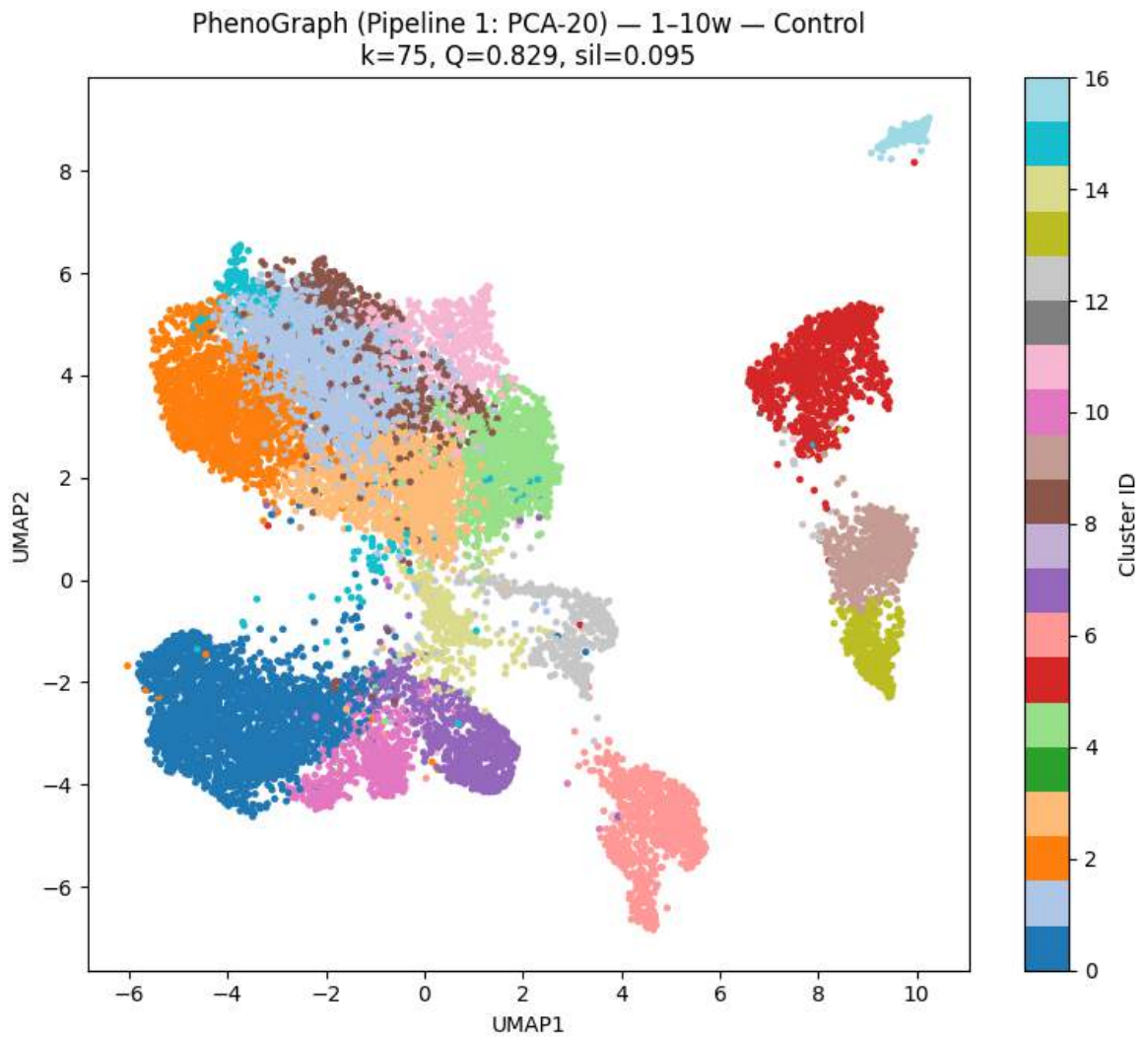
# back-fill plot/labels paths for the best k row we added above
for i in range(len(p1_rows)-1, -1, -1):
    if (p1_rows[i][0]==str(interval) and p1_rows[i][1]==cond and p1_rows
        p1_rows[i][7] = out_png
        p1_rows[i][8] = labels_csv
        break

# Save P1 metrics
p1_metrics = pd.DataFrame(p1_rows, columns=[
    "interval", "condition", "n_cells", "k", "silhouette", "modularity_Q", "composite"
])
p1_path = os.path.join(output_dir, "phenograph_pipeline1_metrics.csv")
p1_metrics.to_csv(p1_path, index=False)
p1_metrics

```

Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 2.4577341079711914 seconds
Jaccard graph constructed in 5.636429309844971 seconds
Wrote graph to binary file in 0.3554558753967285 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.860775$
Louvain completed 21 runs in 6.294033527374268 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 14.95700979232788 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 2.478530168533325 seconds
Jaccard graph constructed in 9.066539764404297 seconds
Wrote graph to binary file in 0.5430636405944824 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.839683$
After 11 runs, maximum modularity is $Q = 0.841977$
Louvain completed 31 runs in 12.038712739944458 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 24.359645128250122 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.7000505924224854 seconds
Jaccard graph constructed in 14.78075647354126 seconds
Wrote graph to binary file in 0.5365936756134033 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.83353$
After 5 runs, maximum modularity is $Q = 0.835297$
Louvain completed 25 runs in 17.015297412872314 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 34.225513219833374 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 2.8632354736328125 seconds
Jaccard graph constructed in 37.59316658973694 seconds
Wrote graph to binary file in 0.8929591178894043 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.8285$
Louvain completed 21 runs in 28.384441614151 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 70.08240079879761 seconds

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v  
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.  
warn(
```



```

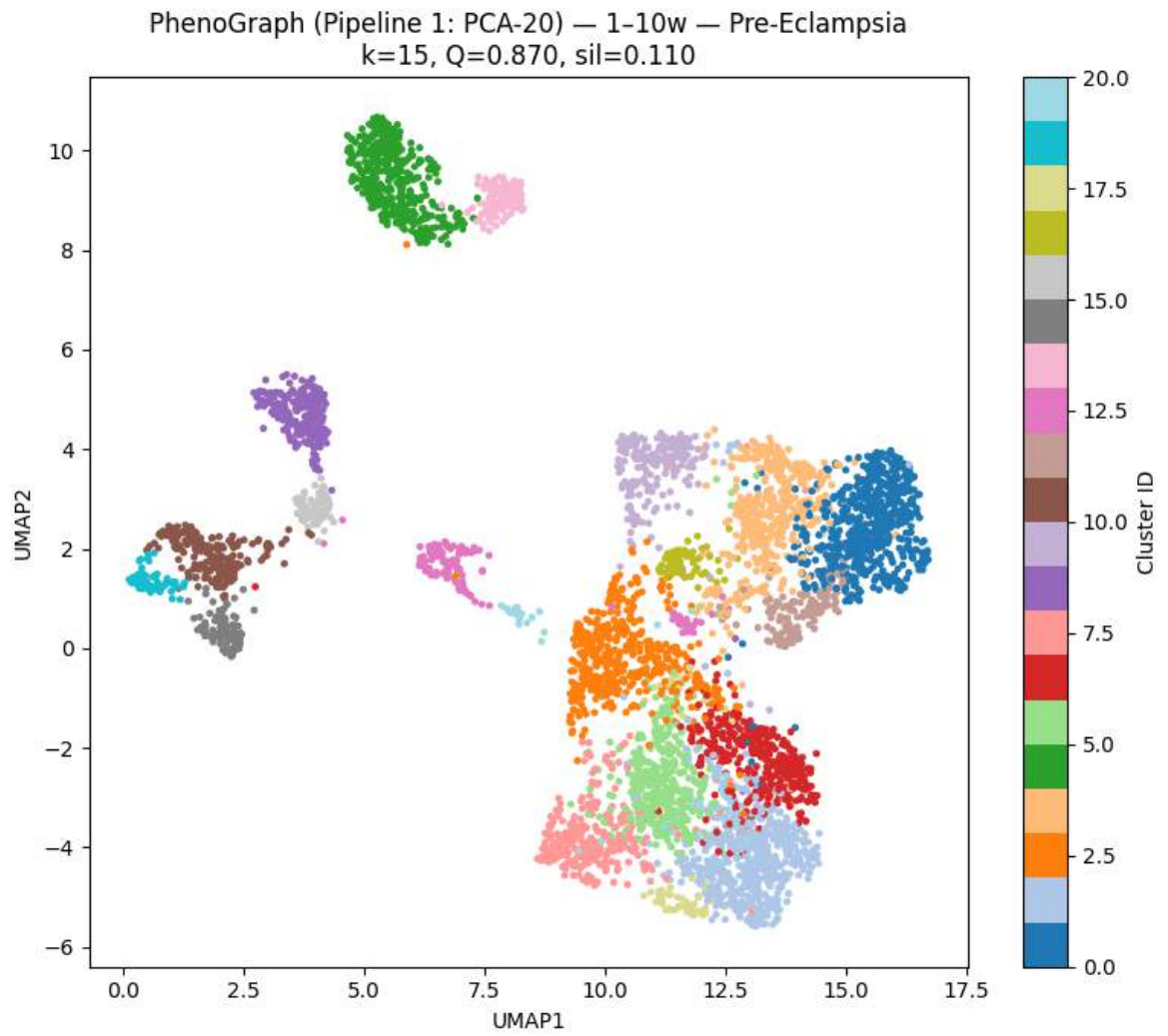
Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.19489359855651855 seconds
Jaccard graph constructed in 0.7978644371032715 seconds
Wrote graph to binary file in 0.07816123962402344 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.866019
After 2 runs, maximum modularity is Q = 0.868152
After 4 runs, maximum modularity is Q = 0.870026
Louvain completed 24 runs in 1.2471613883972168 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 2.5093252658843994 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.33467936515808105 seconds
Jaccard graph constructed in 3.2290725708007812 seconds
Wrote graph to binary file in 0.12001681327819824 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.843949
After 2 runs, maximum modularity is Q = 0.847329
After 10 runs, maximum modularity is Q = 0.848882
Louvain completed 30 runs in 2.1010994911193848 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 5.9178900718688965 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.27641773223876953 seconds
Jaccard graph constructed in 4.406474590301514 seconds
Wrote graph to binary file in 0.18783020973205566 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.831265
After 2 runs, maximum modularity is Q = 0.832603
After 17 runs, maximum modularity is Q = 0.833769
Louvain completed 37 runs in 4.791187047958374 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 9.860882997512817 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.49033522605895996 seconds
Jaccard graph constructed in 8.90018105506897 seconds
Wrote graph to binary file in 0.26665425300598145 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.817414
After 8 runs, maximum modularity is Q = 0.818432
Louvain completed 28 runs in 5.850321531295776 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 15.676376581192017 seconds

```

```

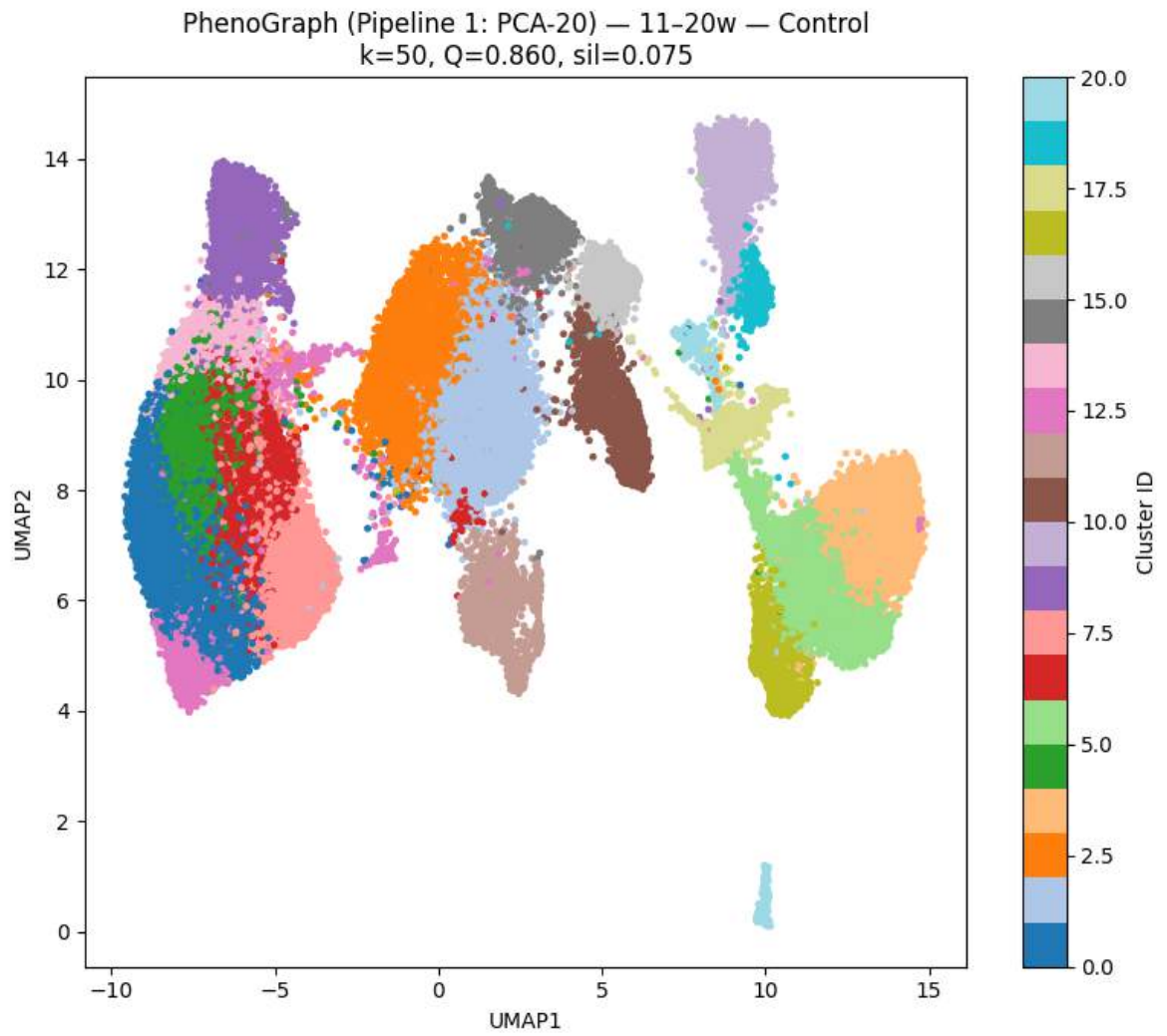
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
warn(

```

```
Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 20.417306184768677 seconds
Jaccard graph constructed in 9.361889123916626 seconds
Wrote graph to binary file in 0.5518670082092285 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.878304
After 8 runs, maximum modularity is Q = 0.879508
Louvain completed 28 runs in 40.526148557662964 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 71.1019937992096 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 20.914916038513184 seconds
Jaccard graph constructed in 25.362165689468384 seconds
Wrote graph to binary file in 1.2090742588043213 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.86539
After 19 runs, maximum modularity is Q = 0.867081
Louvain completed 39 runs in 135.50092029571533 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 183.404536485672 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 22.76852536201477 seconds
Jaccard graph constructed in 52.39689540863037 seconds
Wrote graph to binary file in 2.59897780418396 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.859272
After 16 runs, maximum modularity is Q = 0.860381
Louvain completed 36 runs in 162.1243782043457 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 240.38350248336792 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 26.082009315490723 seconds
Jaccard graph constructed in 111.58397006988525 seconds
Wrote graph to binary file in 3.361581325531006 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.853772
After 12 runs, maximum modularity is Q = 0.854943
Louvain completed 32 runs in 192.6594045162201 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 334.48915457725525 seconds

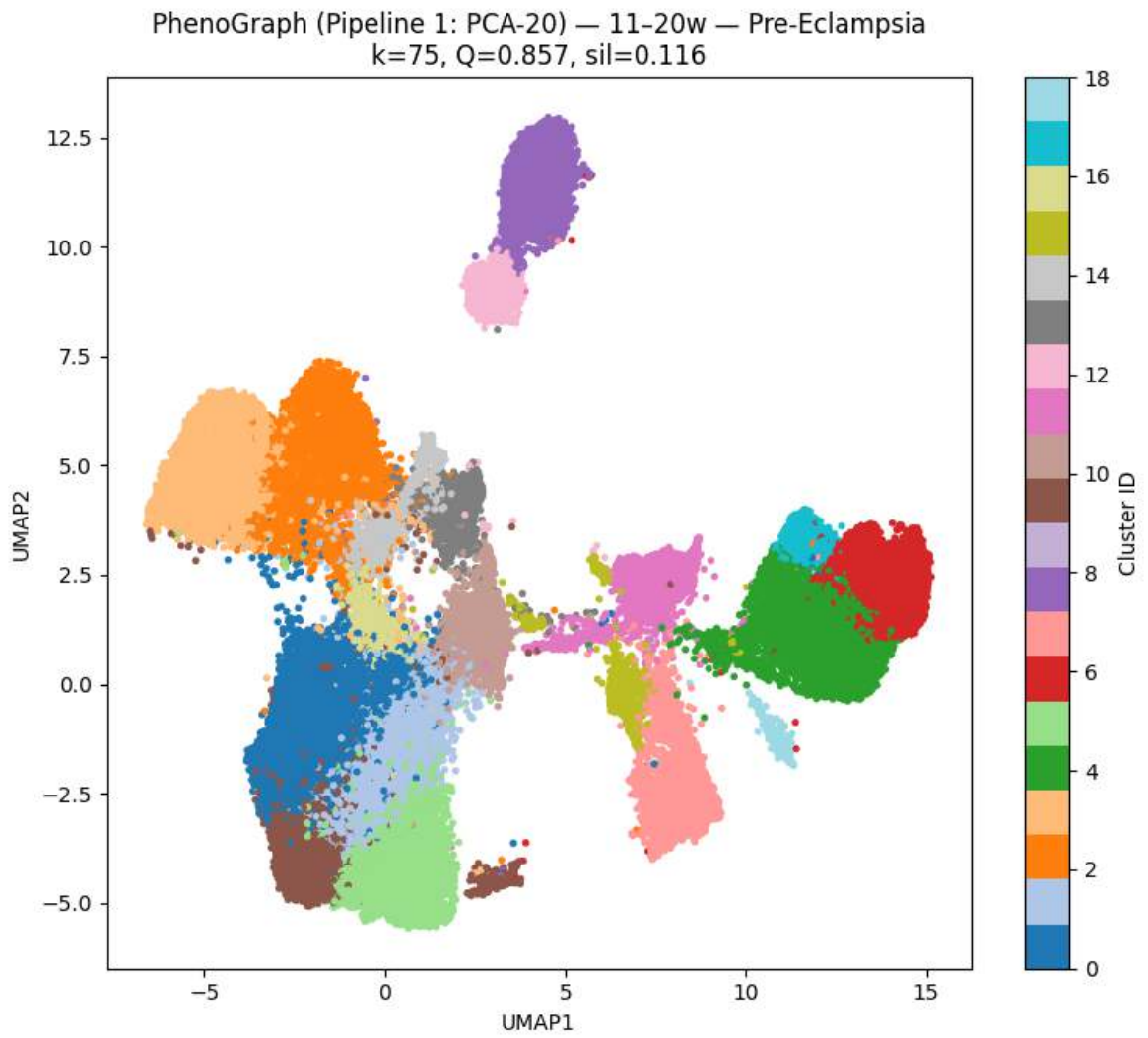
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
warn(
```



```

Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 29.11481738090515 seconds
Jaccard graph constructed in 11.32977819442749 seconds
Wrote graph to binary file in 0.6449935436248779 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.877864
After 8 runs, maximum modularity is Q = 0.878993
Louvain completed 28 runs in 60.041160345077515 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 101.45594477653503 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 29.454662561416626 seconds
Jaccard graph constructed in 27.27253794670105 seconds
Wrote graph to binary file in 1.8308613300323486 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.865818
After 2 runs, maximum modularity is Q = 0.866857
Louvain completed 22 runs in 114.8340585231781 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 173.84074449539185 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 30.29647135734558 seconds
Jaccard graph constructed in 63.72284984588623 seconds
Wrote graph to binary file in 2.902965784072876 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.859572
After 7 runs, maximum modularity is Q = 0.860617
After 8 runs, maximum modularity is Q = 0.861864
Louvain completed 28 runs in 198.73414731025696 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 296.34581184387207 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 31.24999237060547 seconds
Jaccard graph constructed in 124.91819548606873 seconds
Wrote graph to binary file in 4.987254858016968 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.85469
After 2 runs, maximum modularity is Q = 0.857188
Louvain completed 22 runs in 175.78902912139893 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 338.0486979484558 seconds
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
warn(

```

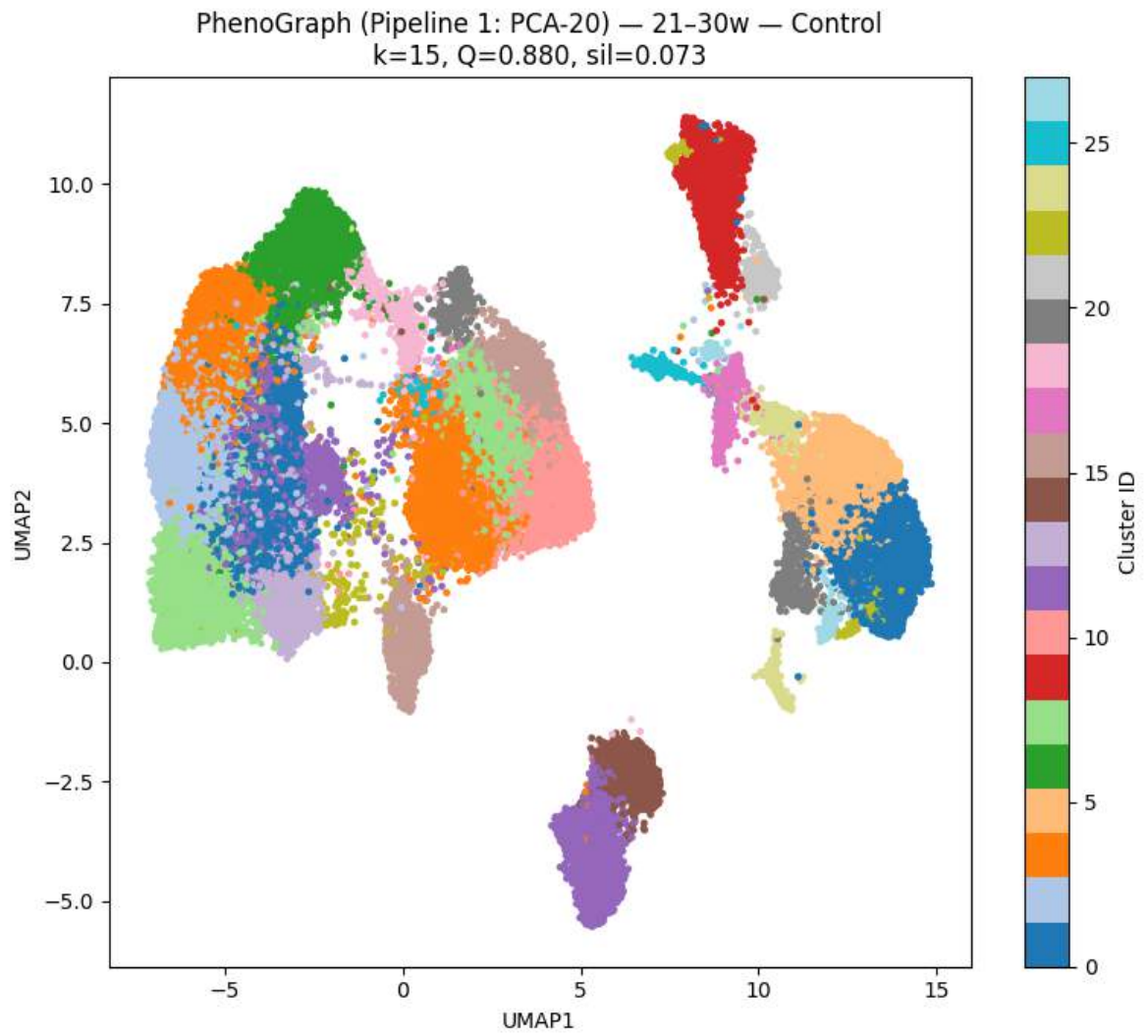


```

Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 17.153490781784058 seconds
Jaccard graph constructed in 9.476423025131226 seconds
Wrote graph to binary file in 0.5131828784942627 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.879215
After 3 runs, maximum modularity is Q = 0.880405
Louvain completed 23 runs in 30.25386381149292 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 57.634586334228516 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 17.52731156349182 seconds
Jaccard graph constructed in 23.184793710708618 seconds
Wrote graph to binary file in 1.1476893424987793 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.867527
Louvain completed 21 runs in 73.08880805969238 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 115.29065895080566 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 18.735613107681274 seconds
Jaccard graph constructed in 48.75269794464111 seconds
Wrote graph to binary file in 2.456914186477661 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.861105
After 3 runs, maximum modularity is Q = 0.862609
Louvain completed 23 runs in 100.21780943870544 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 170.69646167755127 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 18.698137044906616 seconds
Jaccard graph constructed in 100.62883973121643 seconds
Wrote graph to binary file in 2.698906898498535 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.856246
After 2 runs, maximum modularity is Q = 0.857309
Louvain completed 22 runs in 129.56333827972412 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 252.28792929649353 seconds

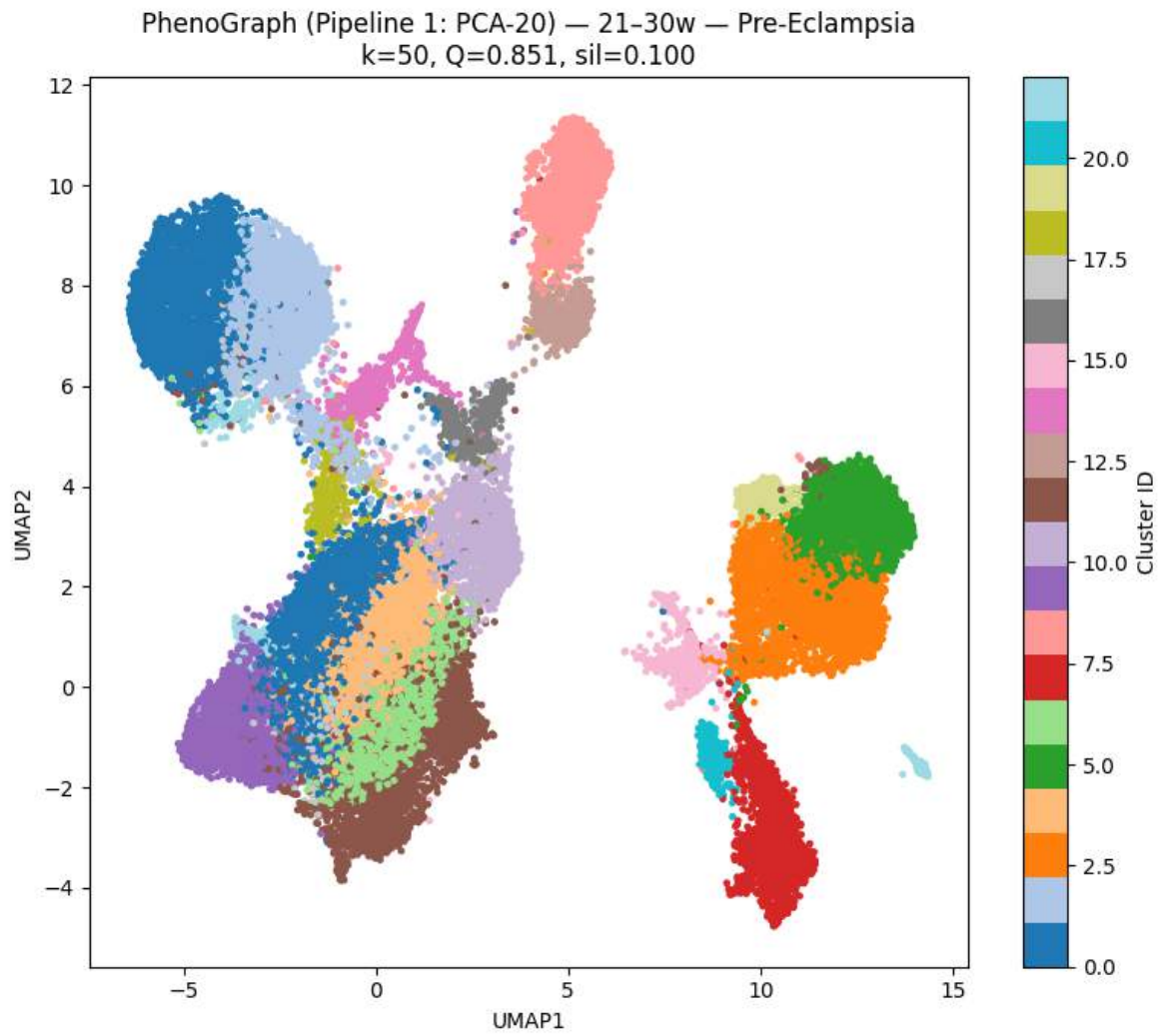
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
warn(

```




```
Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 11.725306510925293 seconds
Jaccard graph constructed in 5.548134088516235 seconds
Wrote graph to binary file in 0.4502108097076416 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.87304
After 5 runs, maximum modularity is Q = 0.874613
Louvain completed 25 runs in 23.610889673233032 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 41.568034172058105 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 11.925205707550049 seconds
Jaccard graph constructed in 17.351835250854492 seconds
Wrote graph to binary file in 0.865933895111084 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.856256
After 3 runs, maximum modularity is Q = 0.857793
Louvain completed 23 runs in 62.17176938056946 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 92.60756158828735 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 12.529904842376709 seconds
Jaccard graph constructed in 42.72072148323059 seconds
Wrote graph to binary file in 1.4505205154418945 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.850628
Louvain completed 21 runs in 63.892030000686646 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 121.07069206237793 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 12.820068597793579 seconds
Jaccard graph constructed in 77.07938766479492 seconds
Wrote graph to binary file in 3.480226755142212 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.844467
After 2 runs, maximum modularity is Q = 0.845832
Louvain completed 22 runs in 94.70589780807495 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 188.71069884300232 seconds

/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs v
alue 1 overridden to 1 by setting random_state. Use no seed for parallelism.
warn(
```



Out[]:

	interval	condition	n_cells	k	silhouette	modularity_Q	composite
0	1-10w	0	15000	15	0.065217	0.860775	0.307682
1	1-10w	0	15000	30	0.071296	0.841977	0.306810
2	1-10w	0	15000	50	0.082086	0.835297	0.312541
3	1-10w	0	15000	75	0.095394	0.828500	0.320003 /content/drive,
4	1-10w	1	5000	15	0.110356	0.870026	0.341802 /content/drive,
5	1-10w	1	5000	30	0.115062	0.848882	0.339329
6	1-10w	1	5000	50	0.110781	0.833769	0.332211
7	1-10w	1	5000	75	0.126400	0.818432	0.338962
8	11-20w	0	55000	15	0.063866	0.879508	0.311845
9	11-20w	0	55000	30	0.063538	0.867081	0.308226
10	11-20w	0	55000	50	0.075342	0.860381	0.314662 /content/drive,
11	11-20w	0	55000	75	0.070885	0.854943	0.310059
12	11-20w	1	65000	15	0.083420	0.878993	0.325392
13	11-20w	1	65000	30	0.106660	0.866857	0.338350
14	11-20w	1	65000	50	0.097862	0.861864	0.330830
15	11-20w	1	65000	75	0.116412	0.857188	0.342539 /content/drive,
16	21-30w	0	50000	15	0.072615	0.880405	0.318213 /content/drive,
17	21-30w	0	50000	30	0.076276	0.867527	0.317264
18	21-30w	0	50000	50	0.077927	0.862609	0.317078
19	21-30w	0	50000	75	0.078151	0.857309	0.315790
20	21-30w	1	40000	15	0.064024	0.874613	0.310621
21	21-30w	1	40000	30	0.095336	0.857793	0.327951
22	21-30w	1	40000	50	0.099878	0.850628	0.329177 /content/drive,
23	21-30w	1	40000	75	0.100299	0.845832	0.328164



Pipeline 2: Phenograph cluster embedded and visualized in UMAP

```
In [ ]: K_GRID_P2 = [15, 30, 50, 75]

p2_rows = []

for interval in valid_bins:
    for cond in [0,1]:
        tag = f"{interval}_{cond}"
        sub_idx = eda[(eda["age_bin_10w"]==interval) & (eda["condition"]==cond)]

        for d in (2,3,5):
```

```

upath = os.path.join(output_dir, f"umap{d}_pipeline2__{tag}.npy")
if not os.path.exists(upath):
    print(f"[P2] Missing UMAP{d} for {tag} - skip")
    continue

emb = np.load(upath)
n = len(emb)
if n < 50:
    print(f"[P2] {tag} dim={d}: too few cells ({n})")
    continue

best = None
for k in K_GRID_P2:
    try:
        communities, graph, Q = phenograph.cluster(emb, k=k)
    except Exception as e:
        print(f"[P2] {tag} dim={d} k={k} failed:", e)
        continue

    labels = np.asarray(communities, dtype=int)
    sil = sil_or_nan(emb, labels)
    comp = composite_score_pheno(sil, Q)

    cand = dict(dim=d, k=k, sil=sil, Q=Q, composite=comp, labels=lab
if (best is None) or (cand["composite"] > best["composite"]):
    best = cand

    # record per-k metrics
    p2_rows.append([
        str(interval), cond, n, d, k, sil, Q, comp,
        "", "" # plot_png, labels_csv (filled for best below)
    ])

if best is None:
    print(f"[P2] {tag} dim={d}: no candidate found.")
    continue

# Save BEST labels
labels = best["labels"]
lab_name = f"phenograph_pipeline2_umap{best['dim']}_k{best['k']}"
labels_csv = os.path.join(output_dir, f"labels_{lab_name}__{tag}.csv")
pd.Series(labels, index=sub_idx, name=lab_name).to_csv(labels_csv)

# Plots: 2D/3D directly; 5D - PCA to 3D for viz
if best["dim"] == 2:
    out_png = os.path.join(output_dir, f"umap2_plot_{lab_name}__{tag}
    title = (f"PhenoGraph (P2: UMAP-2D) - {interval} - {COND_NAME[
        f"k={best['k']}", Q={best['Q']:.3f}, sil={best['sil']}:
    plot_umap2(emb, labels, title, out_png)
elif best["dim"] == 3:
    out_png = os.path.join(output_dir, f"umap3_plot_{lab_name}__{tag}
    title = (f"PhenoGraph (P2: UMAP-3D) - {interval} - {COND_NAME[
        f"k={best['k']}", Q={best['Q']:.3f}, sil={best['sil']}:
    plot_umap3(emb, labels, title, out_png)
else: # 5D → PCA→3D for viz
    emb3 = PCA(n_components=3, random_state=RANDOM_STATE).fit_transf
    np.save(os.path.join(output_dir, f"umap5_pca3viz_pheno__{tag}.np
    out_png = os.path.join(output_dir, f"umap5_pca3viz_plot_{lab_nam
    title = (f"PhenoGraph (P2: UMAP-5D; 3D viz PCA) - {interval} -
        f"k={best['k']}", Q={best['Q']:.3f}, sil={best['sil']}:

```

```

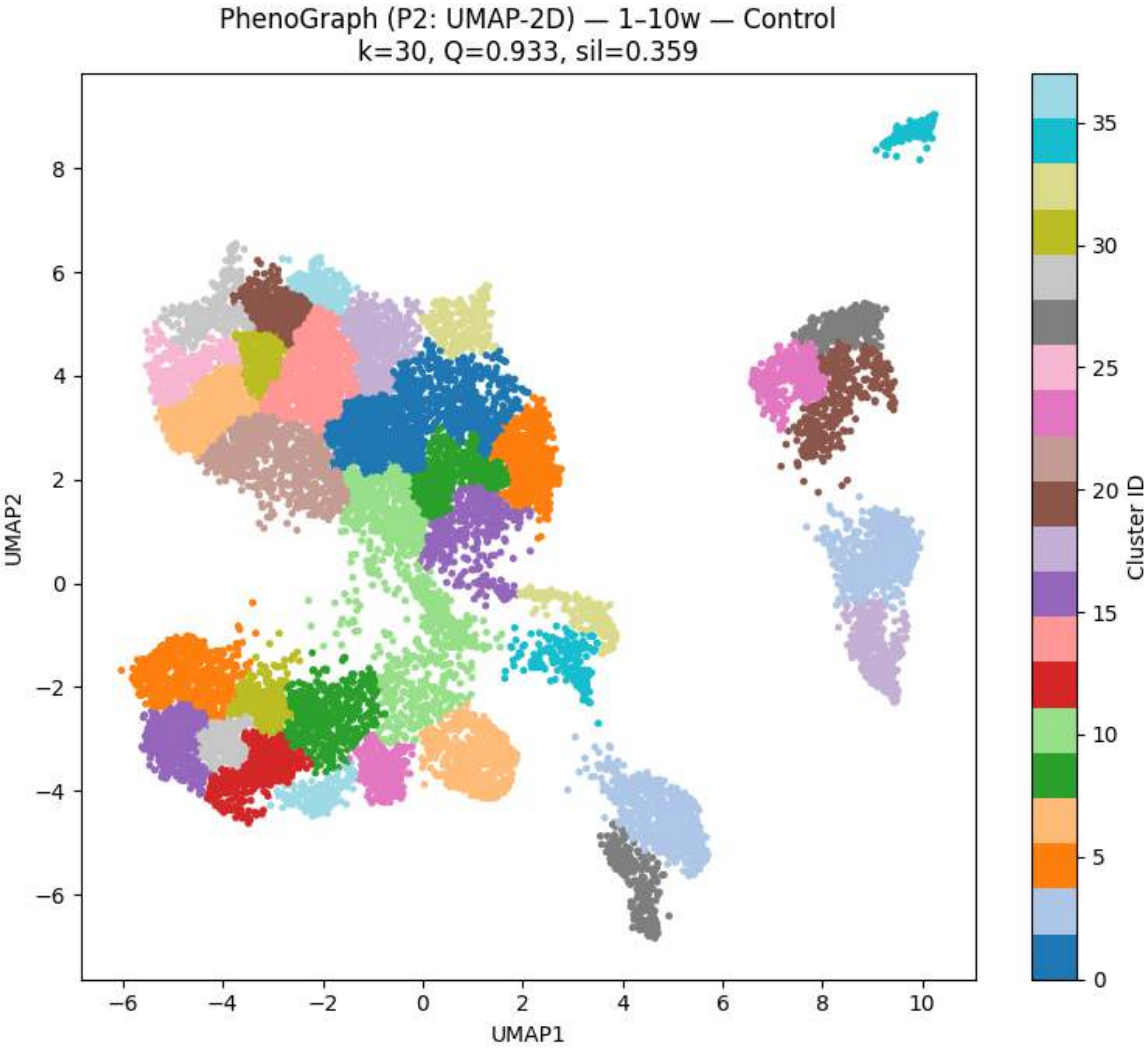
        plot_umap3(emb3, labels, title, out_png)

        # back-fill plot/labels paths for the best row
        for i in range(len(p2_rows)-1, -1, -1):
            if (p2_rows[i][0]==str(interval) and p2_rows[i][1]==cond and
                p2_rows[i][3]==best["dim"] and p2_rows[i][4]==best["k"]):
                p2_rows[i][8] = out_png
                p2_rows[i][9] = labels_csv
                break

    # Save P2 metrics
    p2_metrics = pd.DataFrame(p2_rows, columns=[
        "interval", "condition", "n_cells", "umap_dim", "k", "silhouette", "modularity_Q",
    ])
    p2_path = os.path.join(output_dir, "phenograph_pipeline2_metrics.csv")
    p2_metrics.to_csv(p2_path, index=False)
    p2_metrics

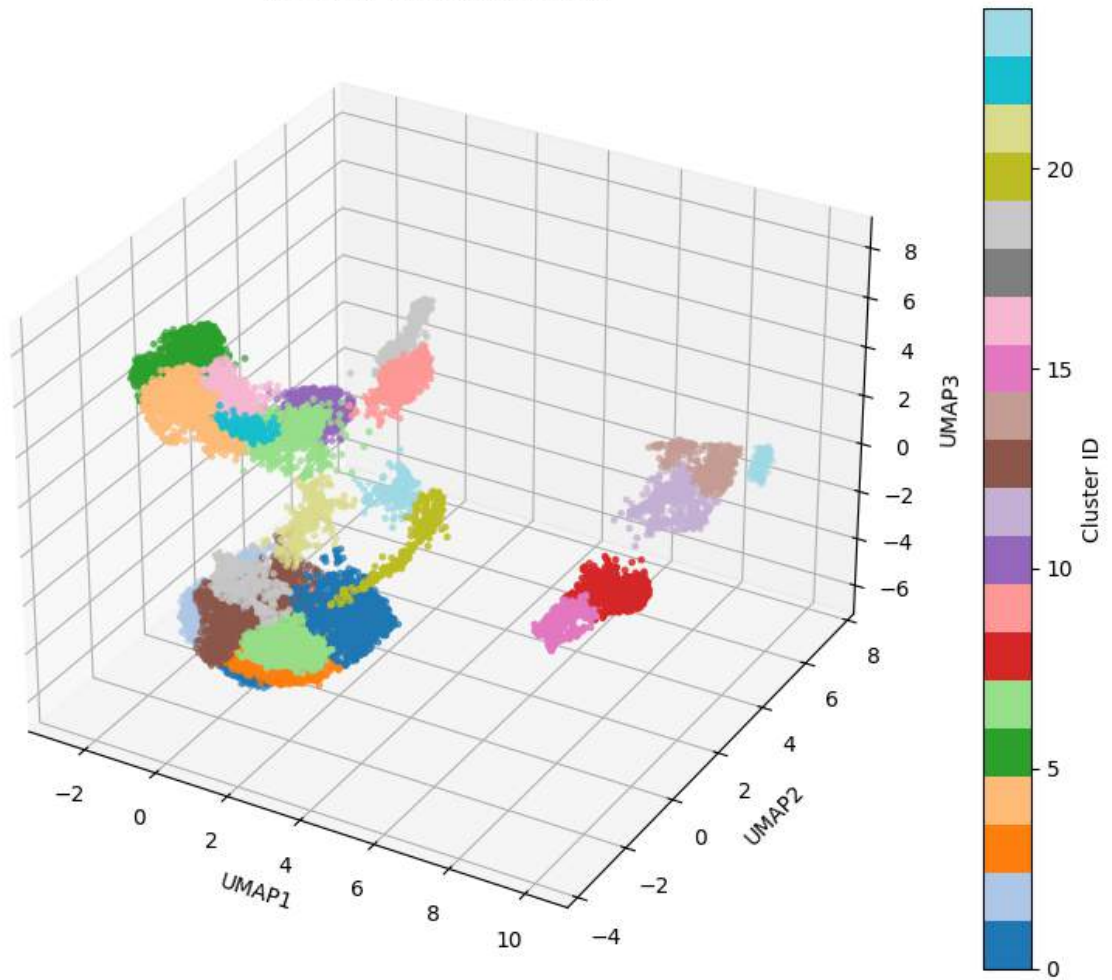
```

Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.0857236385345459 seconds
 Jaccard graph constructed in 5.918573379516602 seconds
 Wrote graph to binary file in 0.28778576850891113 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.954651$
 Louvain completed 21 runs in 3.327977418899536 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 9.858136177062988 seconds
 Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.9365622997283936 seconds
 Jaccard graph constructed in 10.778674840927124 seconds
 Wrote graph to binary file in 0.4461839199066162 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.933362$
 Louvain completed 21 runs in 7.140343189239502 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 19.568145990371704 seconds
 Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.2524552345275879 seconds
 Jaccard graph constructed in 16.197835445404053 seconds
 Wrote graph to binary file in 0.4151194095611572 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.918671$
 Louvain completed 21 runs in 6.282273769378662 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 23.435439825057983 seconds
 Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.2300090789794922 seconds
 Jaccard graph constructed in 30.844839572906494 seconds
 Wrote graph to binary file in 0.5864996910095215 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.907554$
 Louvain completed 21 runs in 8.516942977905273 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 40.50095748901367 seconds



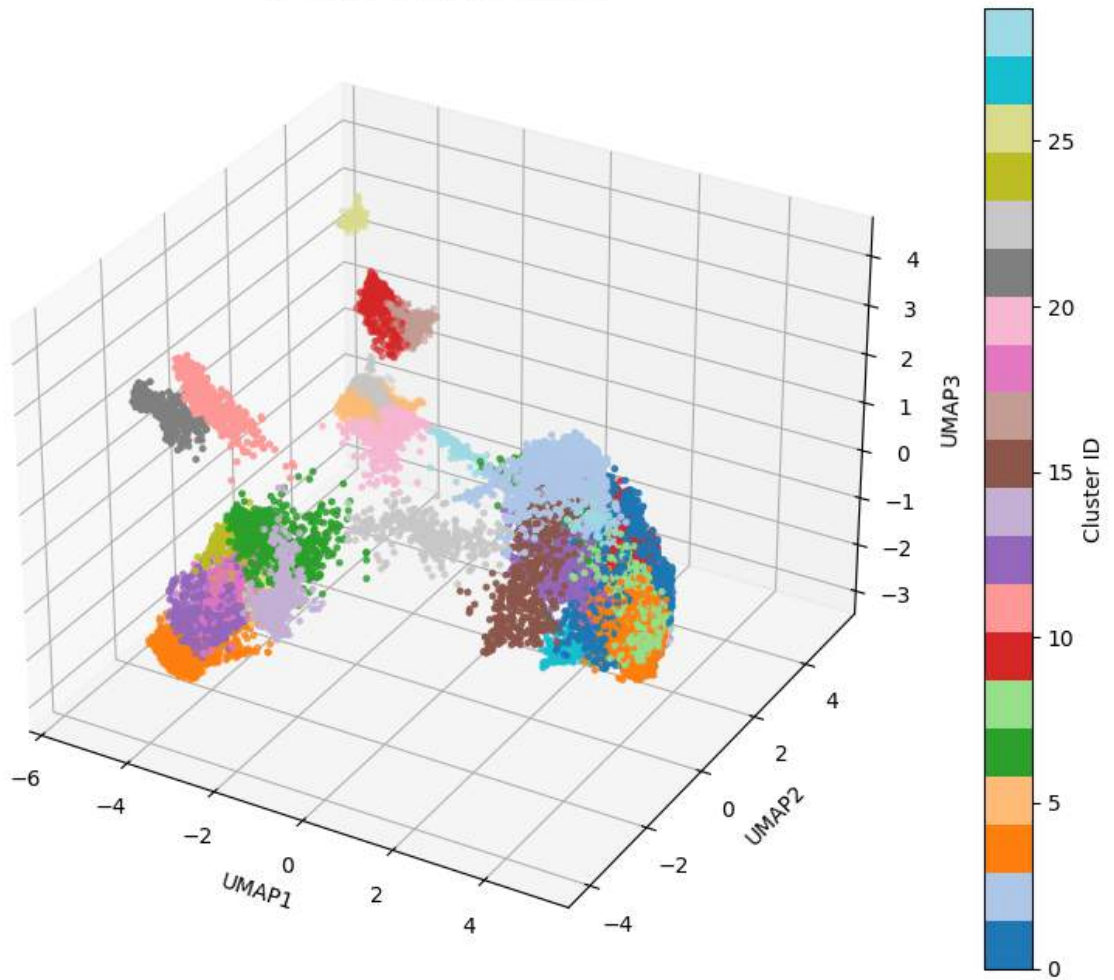
Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.12861871719360352 seconds
Jaccard graph constructed in 2.983523368835449 seconds
Wrote graph to binary file in 0.13943123817443848 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.938105$
Louvain completed 21 runs in 2.14382266998291 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 5.566559314727783 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.17662906646728516 seconds
Jaccard graph constructed in 6.168273210525513 seconds
Wrote graph to binary file in 0.28883814811706543 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.916962$
Louvain completed 21 runs in 4.083479404449463 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 10.995734453201294 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.2394254207611084 seconds
Jaccard graph constructed in 15.4584481716156 seconds
Wrote graph to binary file in 0.439861536026001 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.905065$
Louvain completed 21 runs in 7.174122333526611 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 23.60745930671692 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.2764286994934082 seconds
Jaccard graph constructed in 31.135936975479126 seconds
Wrote graph to binary file in 0.640667200088501 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.8947$
After 4 runs, maximum modularity is $Q = 0.896676$
Louvain completed 24 runs in 13.02083683013916 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 45.33040761947632 seconds

PhenoGraph (P2: UMAP-3D) — 1-10w — Control
k=75, Q=0.897, sil=0.333

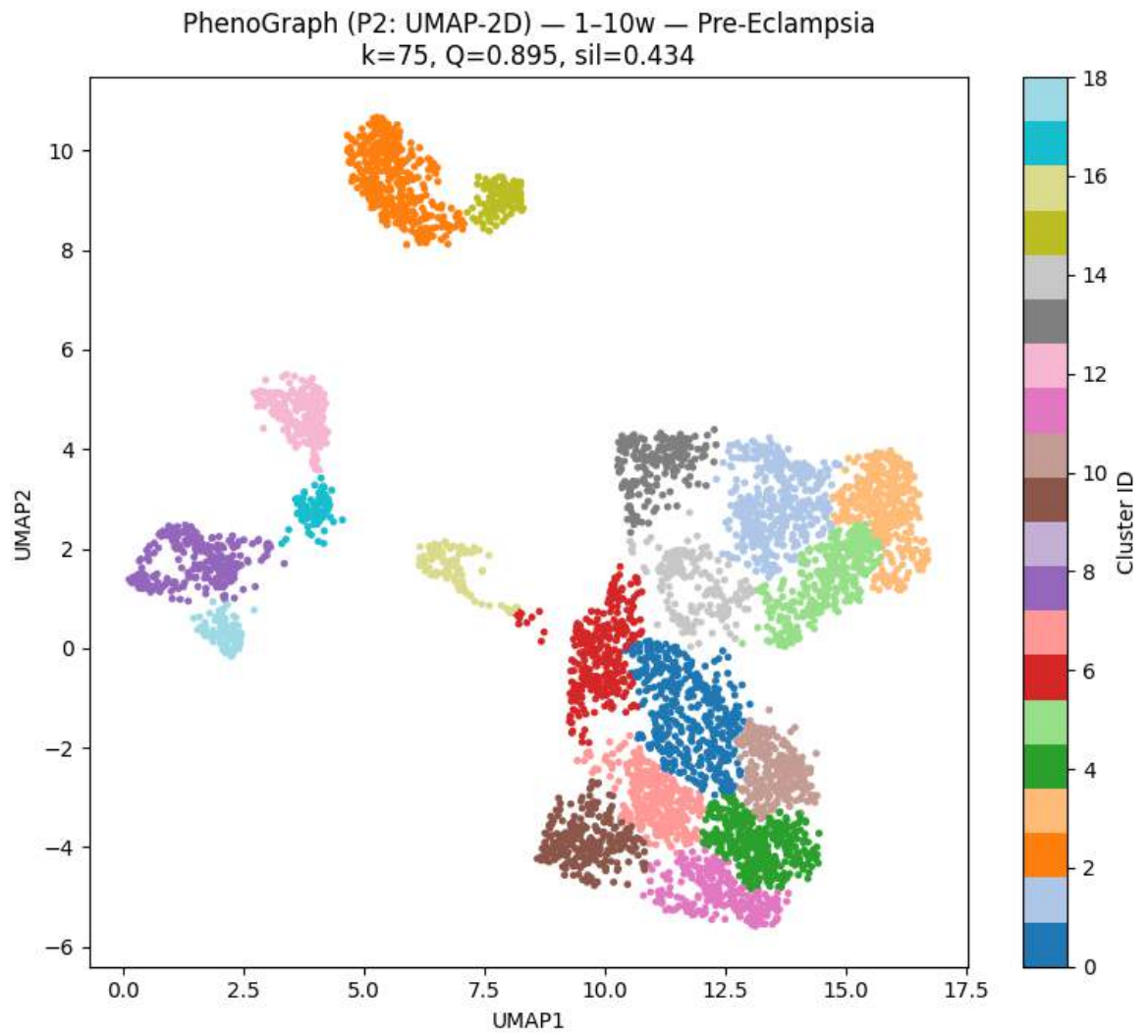


Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.2321932315826416 seconds
Jaccard graph constructed in 2.2276687622070312 seconds
Wrote graph to binary file in 0.14226293563842773 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.934931$
Louvain completed 21 runs in 6.203776836395264 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 9.087557554244995 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.28054308891296387 seconds
Jaccard graph constructed in 8.337928533554077 seconds
Wrote graph to binary file in 0.3240780830383301 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.915672$
After 8 runs, maximum modularity is $Q = 0.916739$
Louvain completed 28 runs in 5.605554580688477 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 14.760338068008423 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.37146973609924316 seconds
Jaccard graph constructed in 15.80234980583191 seconds
Wrote graph to binary file in 0.719287633895874 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.905605$
Louvain completed 21 runs in 7.589822292327881 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 24.72990655899048 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.4611482620239258 seconds
Jaccard graph constructed in 30.871861696243286 seconds
Wrote graph to binary file in 0.8018889427185059 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.897996$
Louvain completed 21 runs in 10.145070791244507 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 42.552079916000366 seconds

PhenoGraph (P2: UMAP-5D; 3D viz PCA) — 1-10w — Control
k=50, Q=0.906, sil=0.275

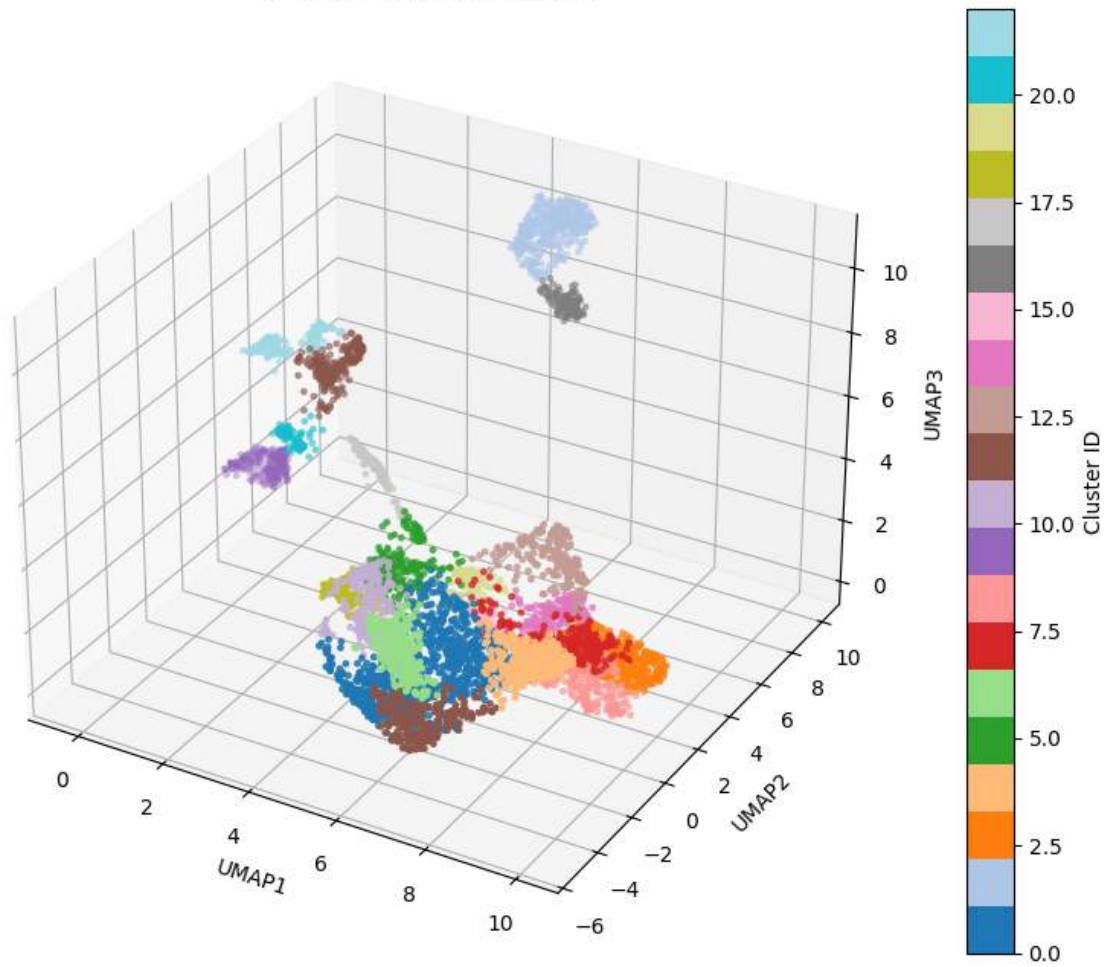


Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.042336463928222656 seconds
Jaccard graph constructed in 0.804069995880127 seconds
Wrote graph to binary file in 0.06404519081115723 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.942664$
After 8 runs, maximum modularity is $Q = 0.943691$
Louvain completed 28 runs in 0.807793140411377 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 1.8809504508972168 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.045433998107910156 seconds
Jaccard graph constructed in 2.1602790355682373 seconds
Wrote graph to binary file in 0.09897780418395996 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.919925$
After 5 runs, maximum modularity is $Q = 0.921112$
Louvain completed 25 runs in 1.0760385990142822 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 3.552825927734375 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.0670626163482666 seconds
Jaccard graph constructed in 5.682548999786377 seconds
Wrote graph to binary file in 0.24373531341552734 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.904969$
After 2 runs, maximum modularity is $Q = 0.906022$
Louvain completed 22 runs in 2.135037660598755 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 8.311368227005005 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.09566020965576172 seconds
Jaccard graph constructed in 9.127800464630127 seconds
Wrote graph to binary file in 0.36591219902038574 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.893736$
After 4 runs, maximum modularity is $Q = 0.895084$
Louvain completed 24 runs in 3.5438926219940186 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 13.339587211608887 seconds



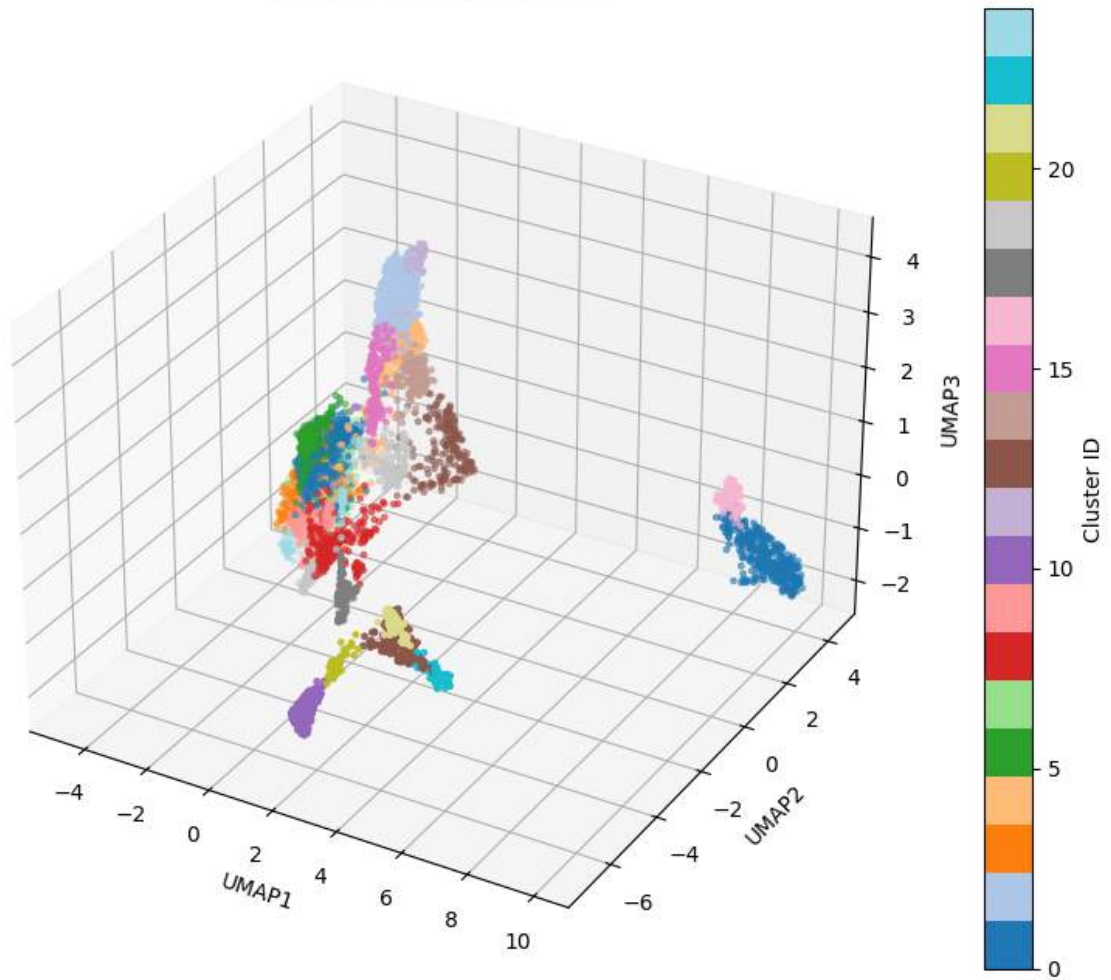
Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.04564213752746582 seconds
Jaccard graph constructed in 1.7962822914123535 seconds
Wrote graph to binary file in 0.0636141300201416 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.934203$
After 4 runs, maximum modularity is $Q = 0.935336$
Louvain completed 24 runs in 0.795250415802002 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 2.8639049530029297 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.05515480041503906 seconds
Jaccard graph constructed in 2.1715073585510254 seconds
Wrote graph to binary file in 0.10356664657592773 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.916613$
Louvain completed 21 runs in 0.9280097484588623 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 3.426386594772339 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.07000946998596191 seconds
Jaccard graph constructed in 6.6701130867004395 seconds
Wrote graph to binary file in 0.15716052055358887 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.903198$
After 4 runs, maximum modularity is $Q = 0.904445$
Louvain completed 24 runs in 1.5639948844909668 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 8.642413139343262 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.09061384201049805 seconds
Jaccard graph constructed in 9.935669660568237 seconds
Wrote graph to binary file in 0.4087255001068115 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.891792$
After 2 runs, maximum modularity is $Q = 0.893192$
After 3 runs, maximum modularity is $Q = 0.894759$
Louvain completed 23 runs in 3.165839433670044 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 13.816184282302856 seconds

PhenoGraph (P2: UMAP-3D) — 1-10w — Pre-Eclampsia
k=75, Q=0.895, sil=0.379

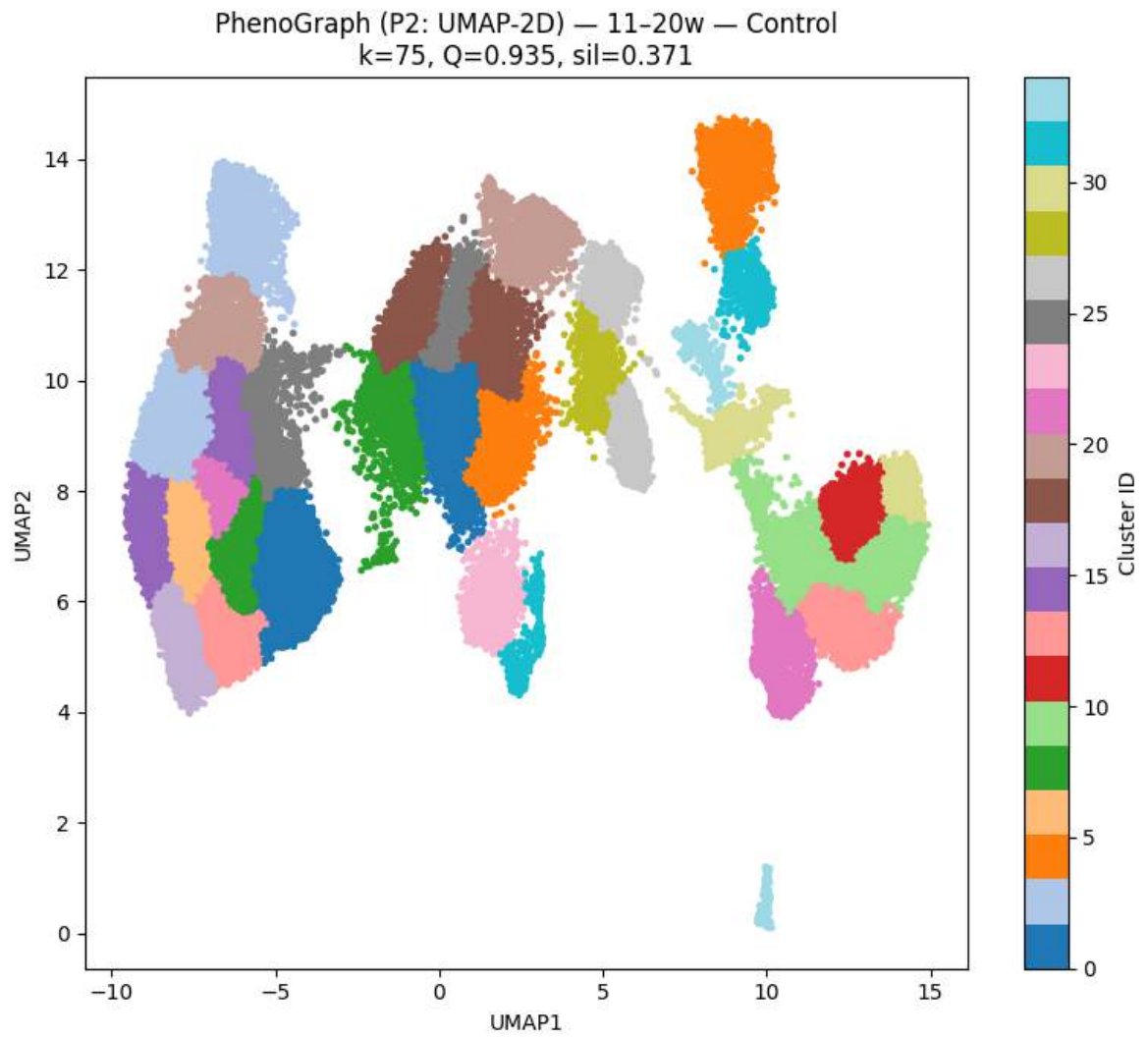


Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.08072376251220703 seconds
Jaccard graph constructed in 0.8670732975006104 seconds
Wrote graph to binary file in 0.0681617259979248 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.935327$
After 11 runs, maximum modularity is $Q = 0.936376$
Louvain completed 31 runs in 0.9650804996490479 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 2.143683910369873 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.08779096603393555 seconds
Jaccard graph constructed in 2.163046360015869 seconds
Wrote graph to binary file in 0.11830258369445801 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.917072$
After 2 runs, maximum modularity is $Q = 0.919707$
Louvain completed 22 runs in 1.1093873977661133 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 3.657668113708496 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.13077712059020996 seconds
Jaccard graph constructed in 6.835906744003296 seconds
Wrote graph to binary file in 0.1588432788848877 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.908586$
After 3 runs, maximum modularity is $Q = 0.909942$
Louvain completed 23 runs in 1.717193603515625 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 9.036579132080078 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.12836384773254395 seconds
Jaccard graph constructed in 11.242846012115479 seconds
Wrote graph to binary file in 0.22828412055969238 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.900523$
Louvain completed 21 runs in 2.2432072162628174 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 14.049453973770142 seconds

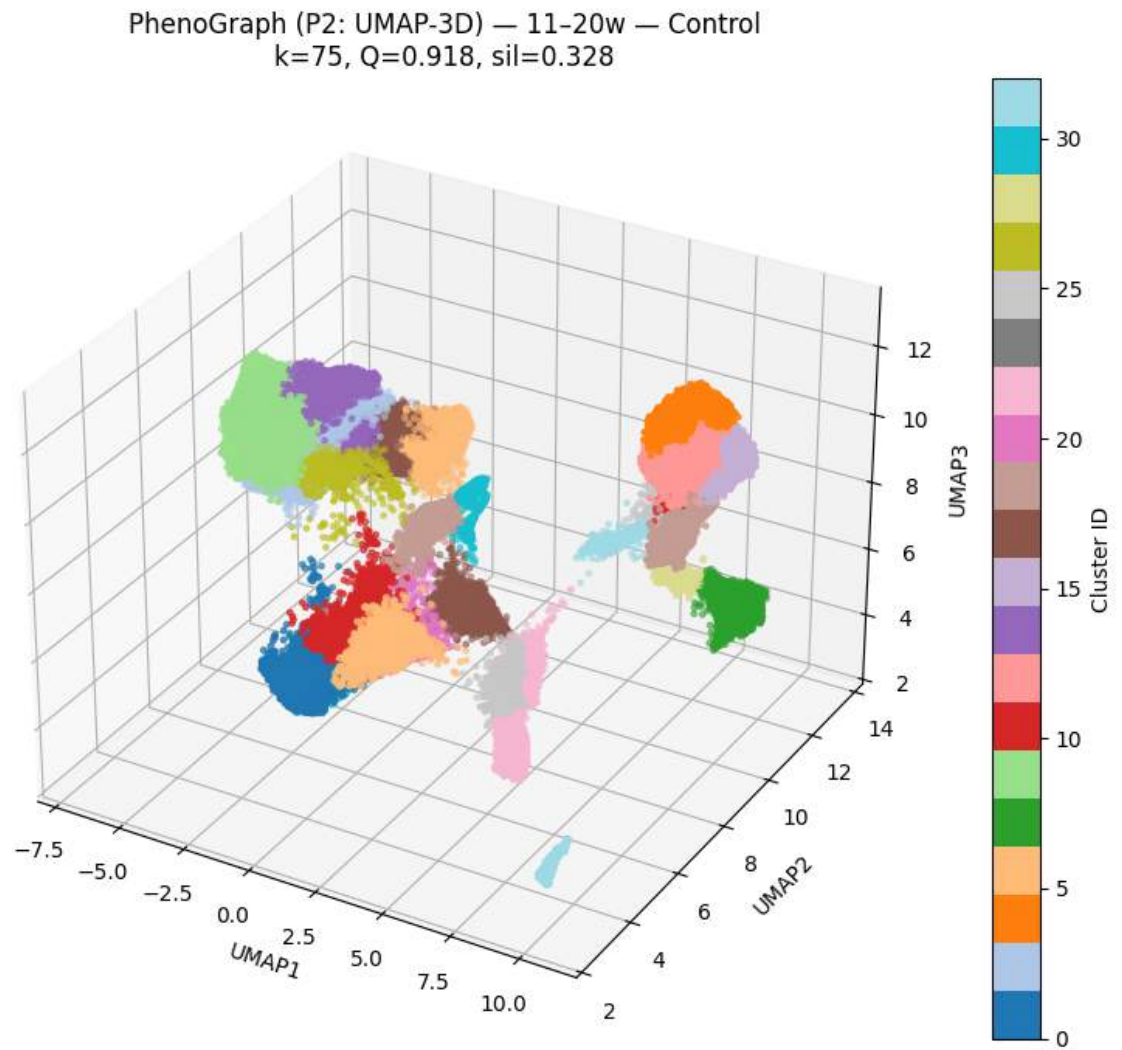
PhenoGraph (P2: UMAP-5D; 3D viz PCA) — 1-10w — Pre-Eclampsia
k=50, Q=0.910, sil=0.361



Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.30388832092285156 seconds
Jaccard graph constructed in 10.55767560005188 seconds
Wrote graph to binary file in 0.46034884452819824 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.969225$
Louvain completed 21 runs in 10.790173292160034 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 22.48080348968506 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.43401193618774414 seconds
Jaccard graph constructed in 26.362016677856445 seconds
Wrote graph to binary file in 1.027817726135254 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.95426$
Louvain completed 21 runs in 18.60772681236267 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 46.862545013427734 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.6106100082397461 seconds
Jaccard graph constructed in 57.00464916229248 seconds
Wrote graph to binary file in 1.4219090938568115 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.942432$
Louvain completed 21 runs in 29.608220100402832 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 89.12611269950867 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.939204216003418 seconds
Jaccard graph constructed in 115.97995924949646 seconds
Wrote graph to binary file in 2.7673351764678955 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.932403$
After 2 runs, maximum modularity is $Q = 0.933452$
After 19 runs, maximum modularity is $Q = 0.934597$
Louvain completed 39 runs in 87.01297998428345 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 207.48347115516663 seconds

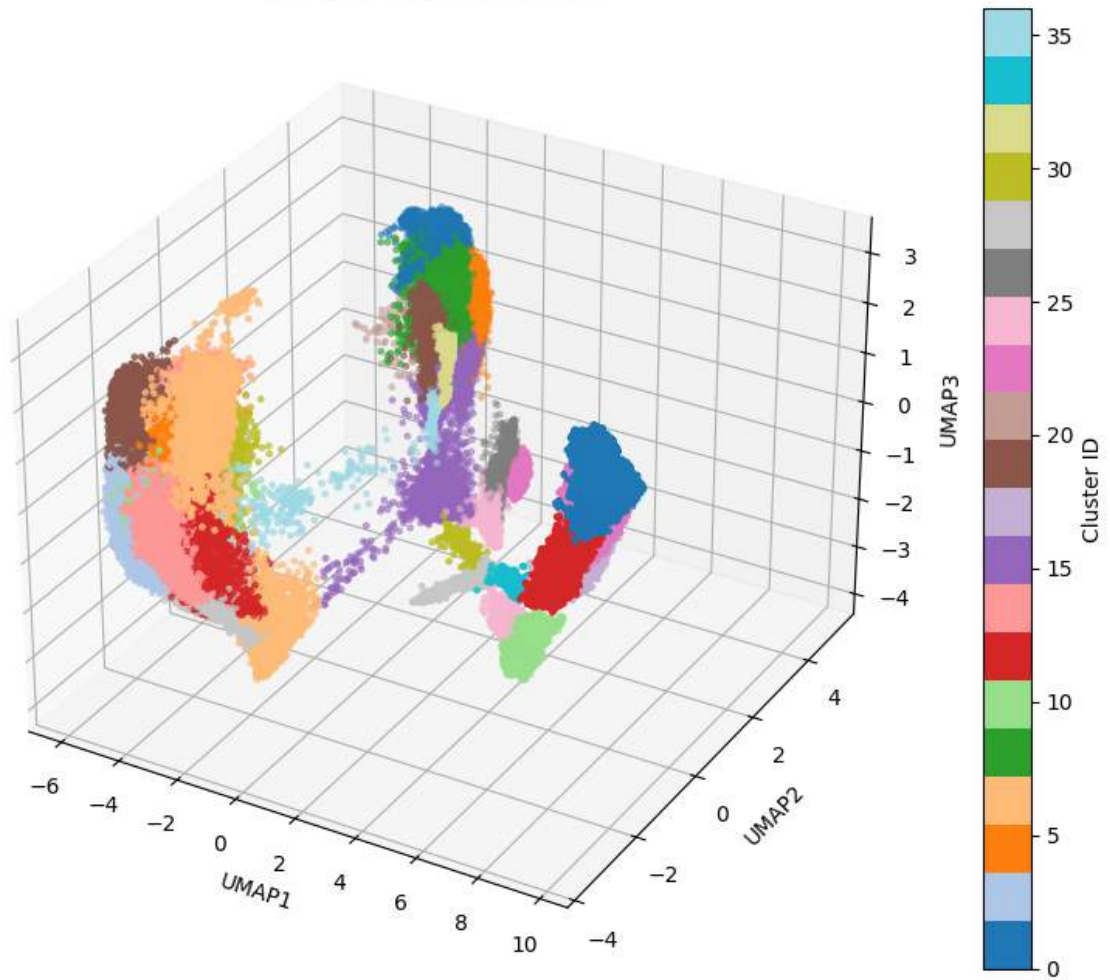


Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.5045514106750488 seconds
Jaccard graph constructed in 8.572937250137329 seconds
Wrote graph to binary file in 0.8044350147247314 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.950029$
Louvain completed 21 runs in 13.033194065093994 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 23.358282804489136 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.6920373439788818 seconds
Jaccard graph constructed in 26.701717376708984 seconds
Wrote graph to binary file in 1.5472571849822998 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.933266$
After 14 runs, maximum modularity is $Q = 0.934274$
Louvain completed 34 runs in 37.05558776855469 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 66.41417336463928 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.9408950805664062 seconds
Jaccard graph constructed in 56.35566282272339 seconds
Wrote graph to binary file in 1.646838903427124 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.923248$
After 6 runs, maximum modularity is $Q = 0.925111$
Louvain completed 26 runs in 49.576590061187744 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 109.09265613555908 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.2306272983551025 seconds
Jaccard graph constructed in 113.37866163253784 seconds
Wrote graph to binary file in 2.2070860862731934 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.915781$
After 5 runs, maximum modularity is $Q = 0.918473$
Louvain completed 25 runs in 77.1668632030487 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 194.83334589004517 seconds

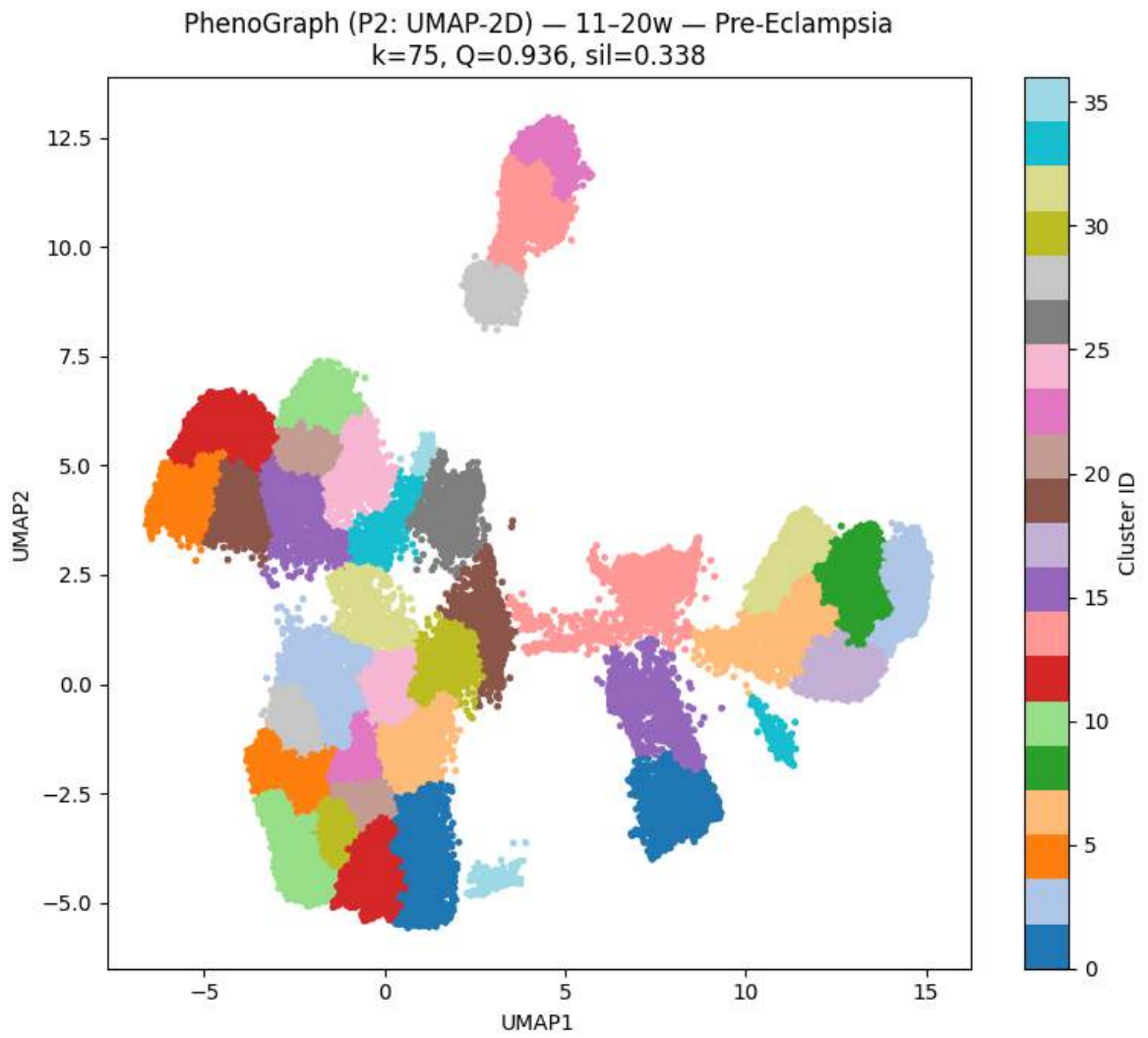


Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.194518804550171 seconds
Jaccard graph constructed in 10.069123268127441 seconds
Wrote graph to binary file in 0.5166373252868652 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.942542$
After 20 runs, maximum modularity is $Q = 0.943588$
Louvain completed 40 runs in 28.01643180847168 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 40.09514570236206 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.4899020195007324 seconds
Jaccard graph constructed in 27.421047687530518 seconds
Wrote graph to binary file in 0.9641668796539307 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.92951$
Louvain completed 21 runs in 27.13815951347351 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 57.40084362030029 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 2.844202756881714 seconds
Jaccard graph constructed in 54.471699476242065 seconds
Wrote graph to binary file in 2.3934314250946045 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.921772$
Louvain completed 21 runs in 44.41286635398865 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 104.6539056301117 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 2.3868913650512695 seconds
Jaccard graph constructed in 113.87963795661926 seconds
Wrote graph to binary file in 2.3156516551971436 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.91476$
After 2 runs, maximum modularity is $Q = 0.917124$
Louvain completed 22 runs in 74.78628158569336 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 194.09418392181396 seconds

PhenoGraph (P2: UMAP-5D; 3D viz PCA) — 11-20w — Control
k=50, Q=0.922, sil=0.286

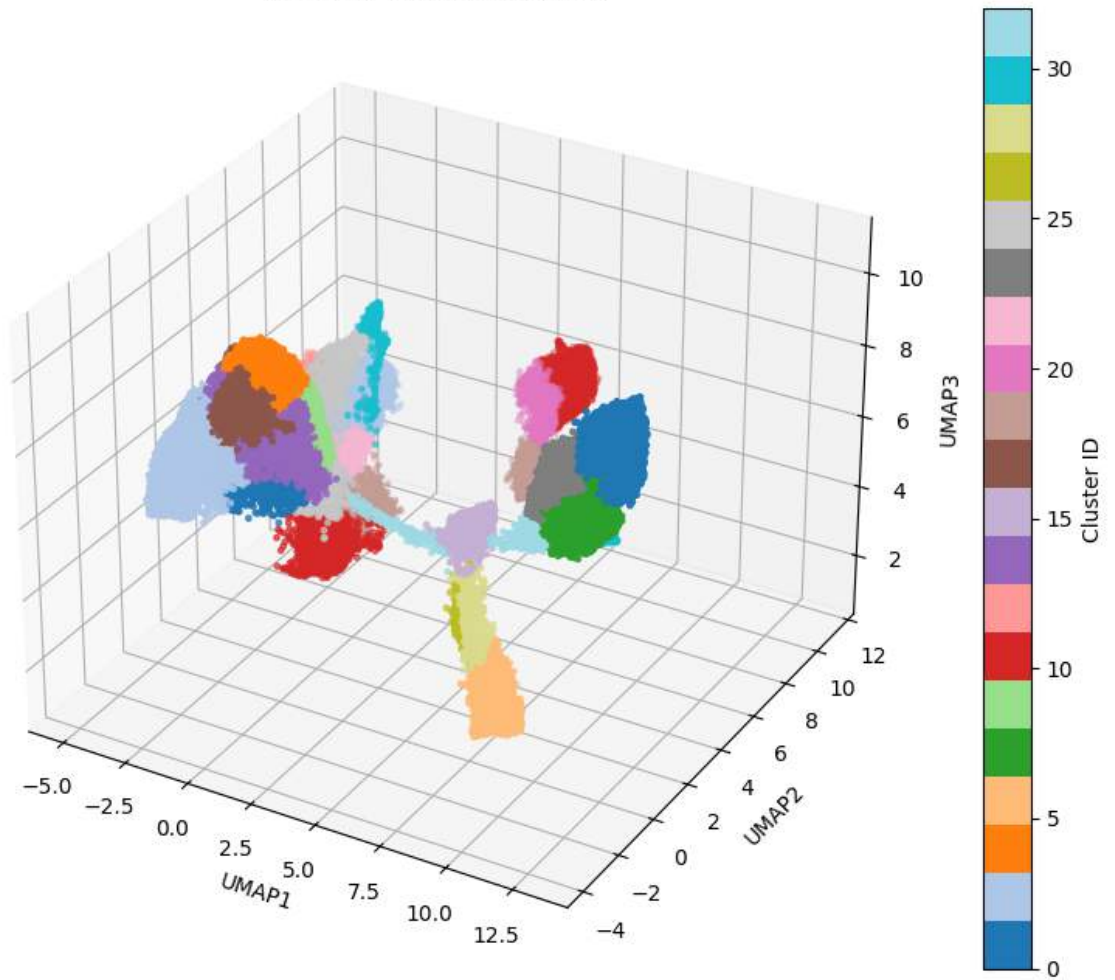


Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.5520713329315186 seconds
Jaccard graph constructed in 10.247398614883423 seconds
Wrote graph to binary file in 0.5236749649047852 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.971249$
Louvain completed 21 runs in 13.55150818824768 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 25.201714277267456 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.543565034866333 seconds
Jaccard graph constructed in 30.251997232437134 seconds
Wrote graph to binary file in 1.0271573066711426 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.955991$
Louvain completed 21 runs in 21.412699699401855 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 53.74493956565857 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 3.3102734088897705 seconds
Jaccard graph constructed in 103.21991968154907 seconds
Wrote graph to binary file in 1.6966862678527832 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.944351$
After 10 runs, maximum modularity is $Q = 0.94536$
Louvain completed 30 runs in 51.42619252204895 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 160.44779086112976 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.005422592163086 seconds
Jaccard graph constructed in 133.62590885162354 seconds
Wrote graph to binary file in 2.4806792736053467 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.935795$
Louvain completed 21 runs in 57.245609760284424 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 195.15918946266174 seconds



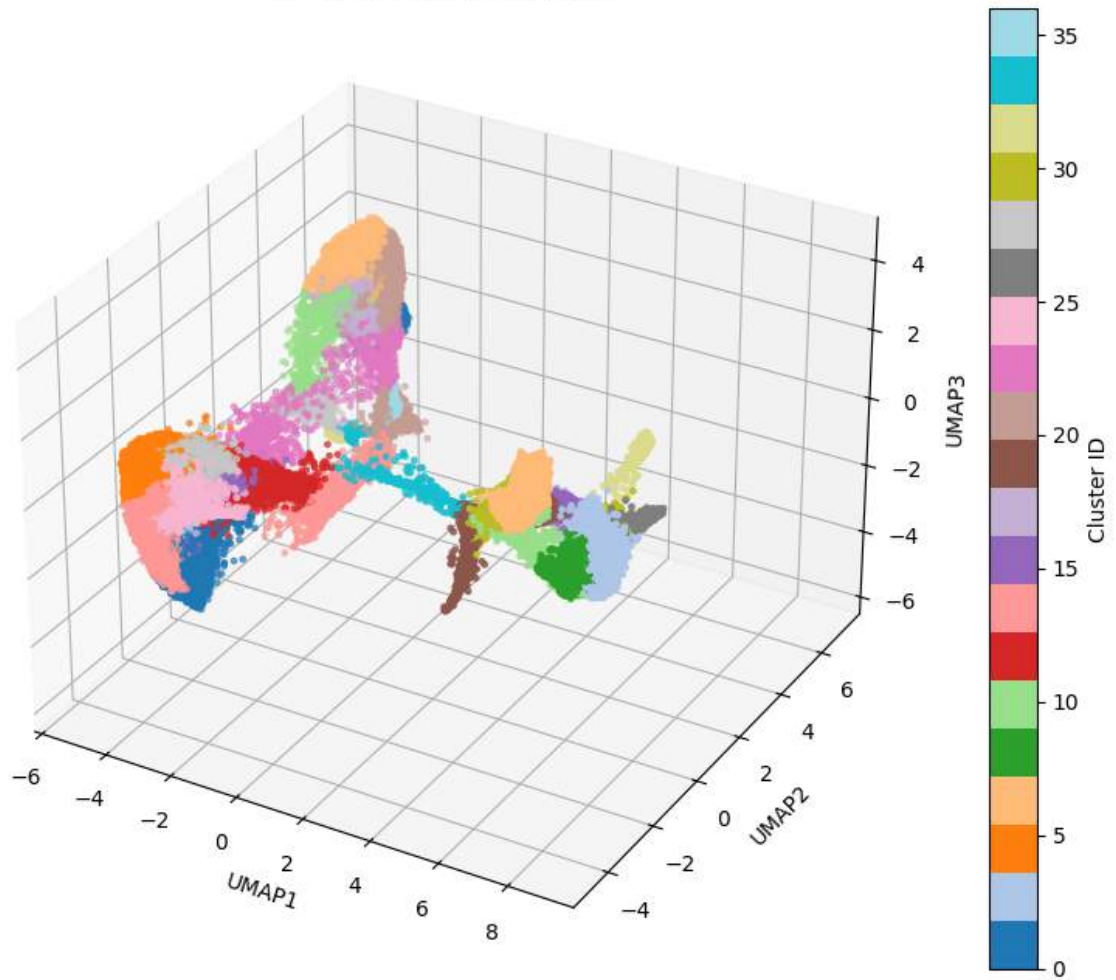
Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.6205911636352539 seconds
Jaccard graph constructed in 9.081222295761108 seconds
Wrote graph to binary file in 0.905571699142456 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.951206$
Louvain completed 21 runs in 16.58712387084961 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 27.51264977455139 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.3739285469055176 seconds
Jaccard graph constructed in 29.071558952331543 seconds
Wrote graph to binary file in 1.3584628105163574 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.934047$
After 11 runs, maximum modularity is $Q = 0.93507$
Louvain completed 31 runs in 42.49927377700806 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 74.81302380561829 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.1630985736846924 seconds
Jaccard graph constructed in 68.77799820899963 seconds
Wrote graph to binary file in 1.7814514636993408 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.92499$
Louvain completed 21 runs in 49.351067781448364 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 121.78013896942139 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.6681842803955078 seconds
Jaccard graph constructed in 132.73785495758057 seconds
Wrote graph to binary file in 2.6518306732177734 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.916984$
After 5 runs, maximum modularity is $Q = 0.918623$
Louvain completed 25 runs in 104.87120628356934 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 242.95914101600647 seconds

PhenoGraph (P2: UMAP-3D) — 11-20w — Pre-Eclampsia
k=75, Q=0.919, sil=0.334

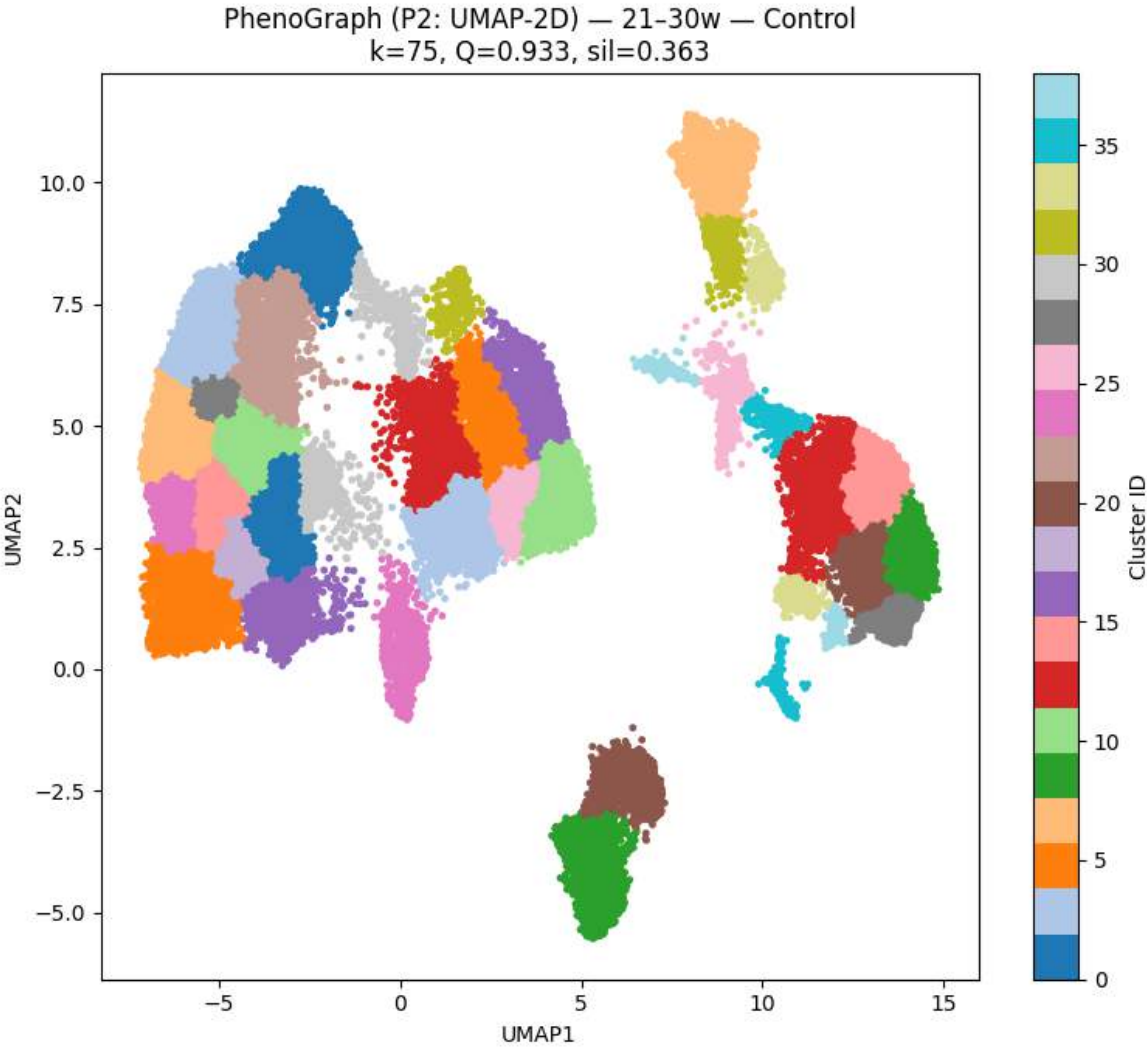


Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.5508151054382324 seconds
Jaccard graph constructed in 9.969399452209473 seconds
Wrote graph to binary file in 0.9604587554931641 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.942294$
Louvain completed 21 runs in 18.922974109649658 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 31.799071311950684 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 3.5107367038726807 seconds
Jaccard graph constructed in 29.378758907318115 seconds
Wrote graph to binary file in 1.1446993350982666 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.927688$
After 6 runs, maximum modularity is $Q = 0.928968$
Louvain completed 26 runs in 44.389915466308594 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 78.88566946983337 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 4.32801628112793 seconds
Jaccard graph constructed in 65.53401136398315 seconds
Wrote graph to binary file in 1.820542812347412 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.920468$
Louvain completed 21 runs in 61.280978202819824 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 133.6609387397766 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 3.258732795715332 seconds
Jaccard graph constructed in 133.60960054397583 seconds
Wrote graph to binary file in 2.6873321533203125 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.913512$
After 4 runs, maximum modularity is $Q = 0.914992$
Louvain completed 24 runs in 110.26674175262451 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 250.64656615257263 seconds

PhenoGraph (P2: UMAP-5D; 3D viz PCA) — 11–20w — Pre-Eclampsia
k=50, Q=0.920, sil=0.287

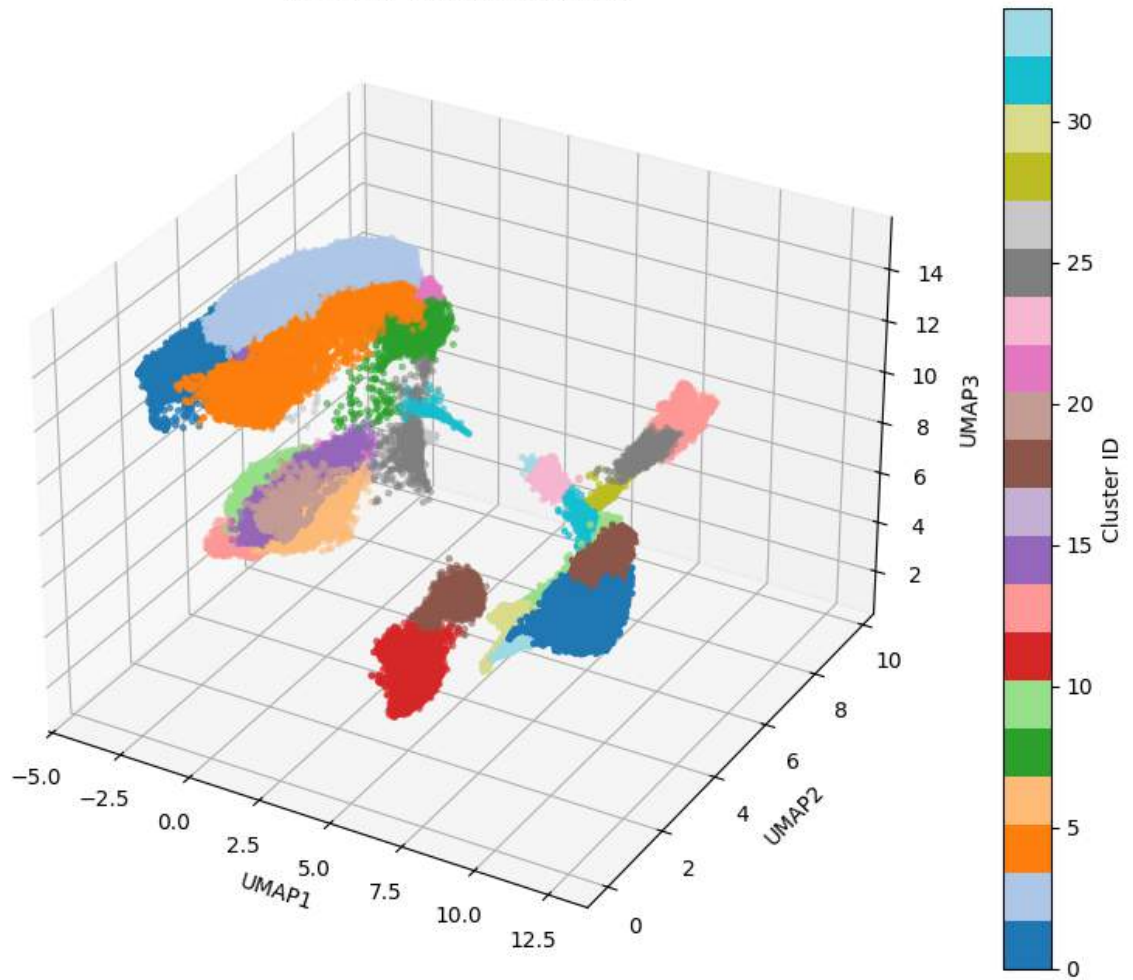


Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.38698267936706543 seconds
Jaccard graph constructed in 9.429227828979492 seconds
Wrote graph to binary file in 0.4316542148590088 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.969007$
Louvain completed 21 runs in 10.788639783859253 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 21.34224033355713 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.5537750720977783 seconds
Jaccard graph constructed in 22.219398498535156 seconds
Wrote graph to binary file in 0.7905900478363037 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.952023$
Louvain completed 21 runs in 15.946340560913086 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 39.85898184776306 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.5318789482116699 seconds
Jaccard graph constructed in 52.29702067375183 seconds
Wrote graph to binary file in 1.739326000213623 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.941198$
Louvain completed 21 runs in 28.002020120620728 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 83.09319424629211 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.0148968696594238 seconds
Jaccard graph constructed in 102.28388214111328 seconds
Wrote graph to binary file in 1.9019098281860352 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.931513$
After 16 runs, maximum modularity is $Q = 0.932687$
Louvain completed 36 runs in 71.11742305755615 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 176.99584293365479 seconds



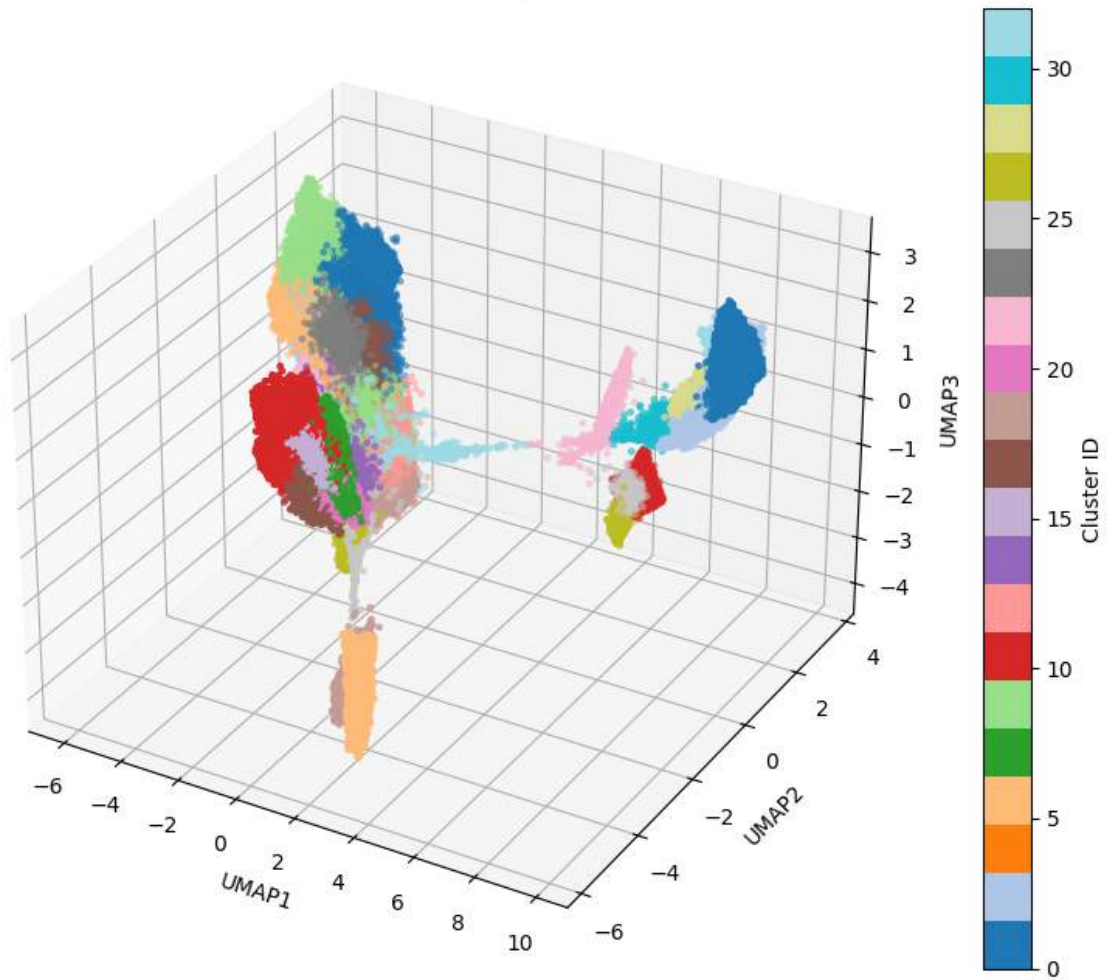
Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.68039870262146 seconds
Jaccard graph constructed in 8.594483613967896 seconds
Wrote graph to binary file in 0.4313781261444092 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.95033$
Louvain completed 21 runs in 11.850771427154541 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 21.83424210548401 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.6433522701263428 seconds
Jaccard graph constructed in 21.847865343093872 seconds
Wrote graph to binary file in 0.8115208148956299 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.934158$
After 6 runs, maximum modularity is $Q = 0.935404$
Louvain completed 26 runs in 23.963337659835815 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 47.636497259140015 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.8467509746551514 seconds
Jaccard graph constructed in 50.68617844581604 seconds
Wrote graph to binary file in 1.3695242404937744 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.924424$
Louvain completed 21 runs in 34.71543884277344 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 88.1140148639679 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.7423973083496094 seconds
Jaccard graph constructed in 100.74828124046326 seconds
Wrote graph to binary file in 3.384539842605591 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.917225$
After 5 runs, maximum modularity is $Q = 0.918275$
Louvain completed 25 runs in 70.29280424118042 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 176.87585282325745 seconds

PhenoGraph (P2: UMAP-3D) — 21-30w — Control
k=75, Q=0.918, sil=0.328

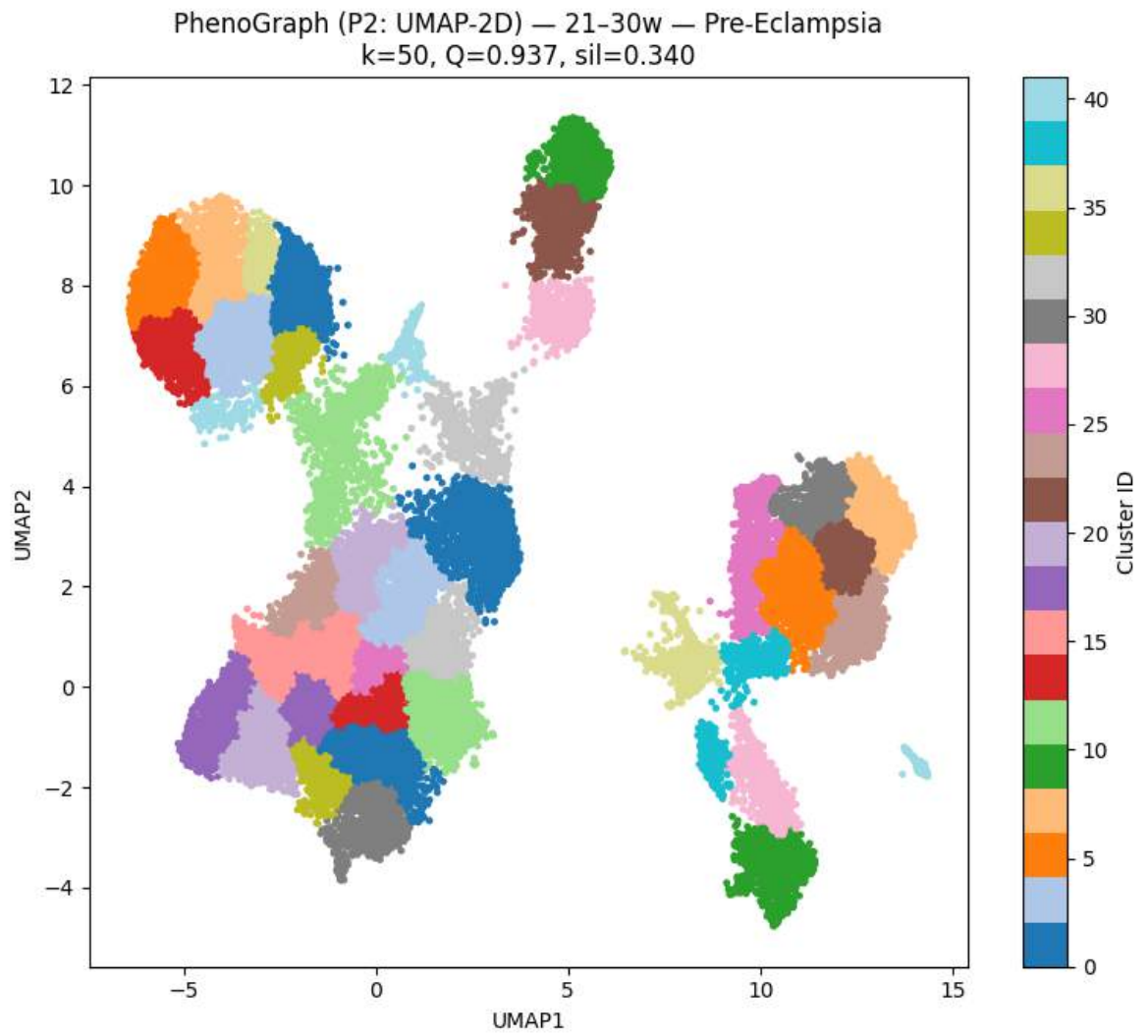


Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.9884350299835205 seconds
Jaccard graph constructed in 9.002669095993042 seconds
Wrote graph to binary file in 0.4428095817565918 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.941581$
Louvain completed 21 runs in 12.56697678565979 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 23.379966735839844 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.3143370151519775 seconds
Jaccard graph constructed in 23.926613807678223 seconds
Wrote graph to binary file in 0.8711996078491211 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.926637$
After 5 runs, maximum modularity is $Q = 0.927883$
Louvain completed 25 runs in 27.38376808166504 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 53.86960983276367 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.6784794330596924 seconds
Jaccard graph constructed in 50.700093269348145 seconds
Wrote graph to binary file in 1.406818151473999 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.918978$
After 19 runs, maximum modularity is $Q = 0.920104$
Louvain completed 39 runs in 73.1050980091095 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 127.46223759651184 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 2.1512227058410645 seconds
Jaccard graph constructed in 102.71904182434082 seconds
Wrote graph to binary file in 2.0973093509674072 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.914857$
Louvain completed 21 runs in 63.99302887916565 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 171.6791501045227 seconds

PhenoGraph (P2: UMAP-5D; 3D viz PCA) — 21-30w — Control
k=75, Q=0.915, sil=0.296

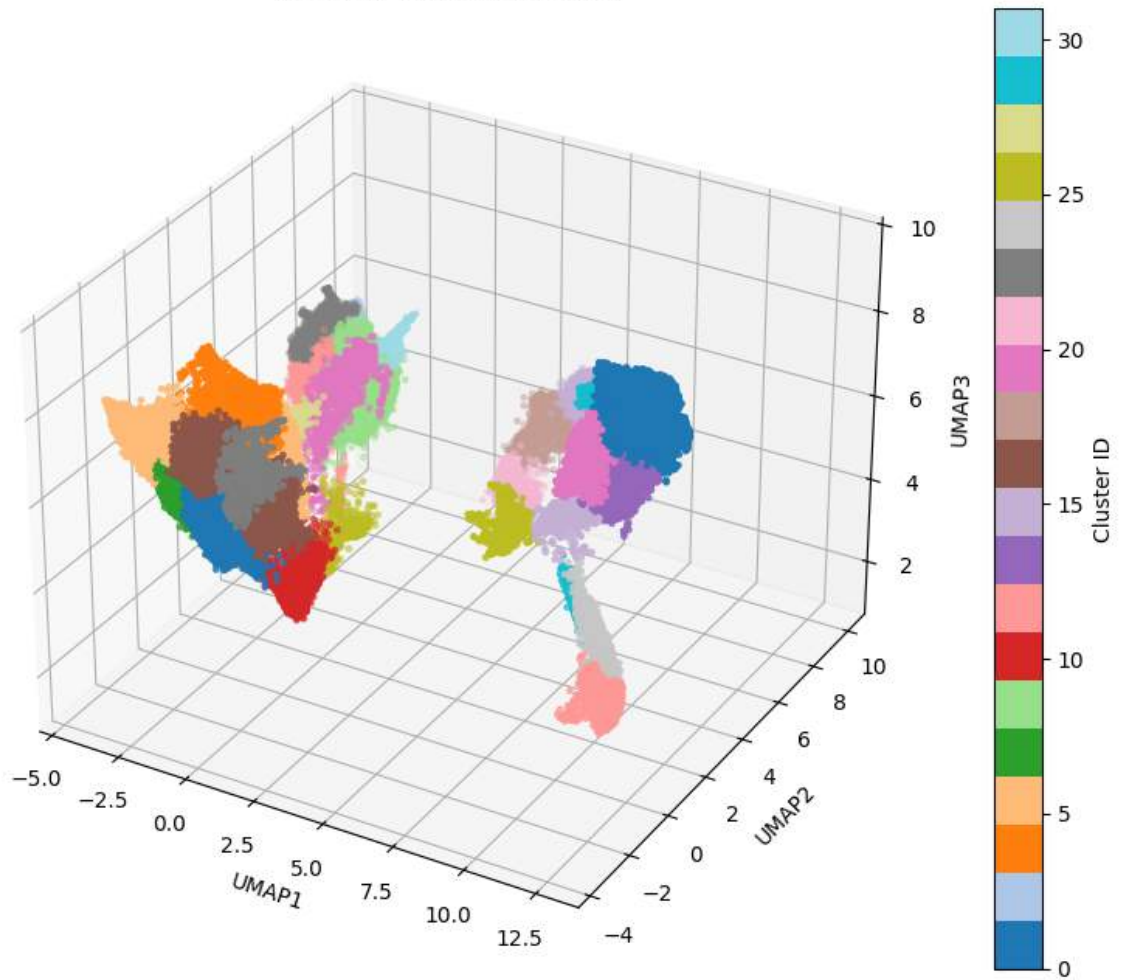


Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.22890043258666992 seconds
Jaccard graph constructed in 6.60007381439209 seconds
Wrote graph to binary file in 0.5516088008880615 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.966221$
Louvain completed 21 runs in 7.328965663909912 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 15.020009279251099 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.31279444694519043 seconds
Jaccard graph constructed in 20.302430152893066 seconds
Wrote graph to binary file in 0.6326568126678467 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.949396$
Louvain completed 21 runs in 12.894609212875366 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 34.477375745773315 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.4413108825683594 seconds
Jaccard graph constructed in 43.88787055015564 seconds
Wrote graph to binary file in 1.028573751449585 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.936858$
Louvain completed 21 runs in 19.992124795913696 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 65.79433369636536 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.5595052242279053 seconds
Jaccard graph constructed in 81.49974727630615 seconds
Wrote graph to binary file in 2.5807251930236816 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.926949$
Louvain completed 21 runs in 30.53014373779297 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 116.089022397995 seconds



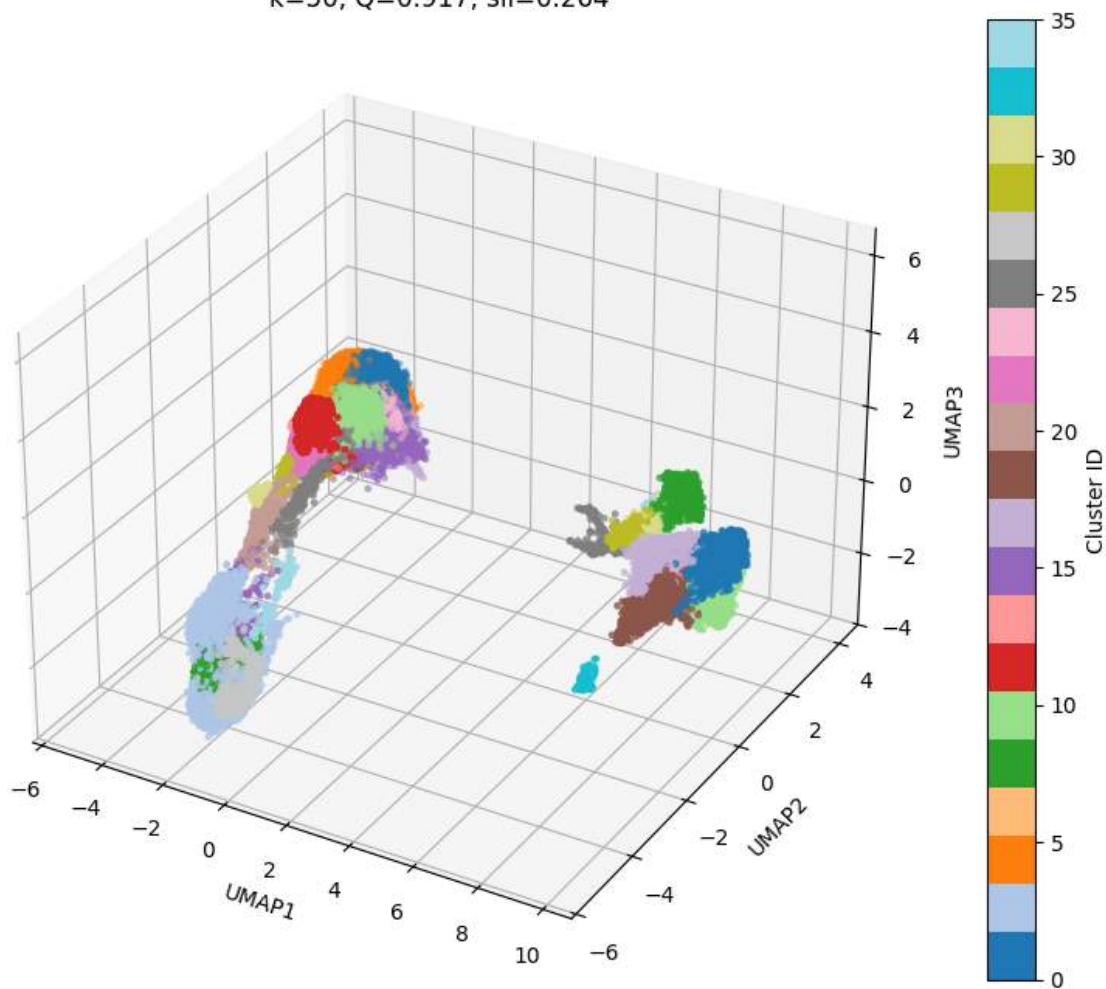
Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.3840479850769043 seconds
Jaccard graph constructed in 6.321303606033325 seconds
Wrote graph to binary file in 0.4664609432220459 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.947259$
Louvain completed 21 runs in 8.980515241622925 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 16.42342448234558 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.4831569194793701 seconds
Jaccard graph constructed in 17.954944133758545 seconds
Wrote graph to binary file in 0.6462545394897461 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.92949$
After 14 runs, maximum modularity is $Q = 0.930619$
Louvain completed 34 runs in 23.93451476097107 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 43.36597752571106 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.6558337211608887 seconds
Jaccard graph constructed in 39.965131521224976 seconds
Wrote graph to binary file in 1.8179571628570557 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.918432$
After 4 runs, maximum modularity is $Q = 0.919501$
After 18 runs, maximum modularity is $Q = 0.920834$
Louvain completed 38 runs in 46.52628040313721 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 89.45271921157837 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.855665922164917 seconds
Jaccard graph constructed in 81.90394425392151 seconds
Wrote graph to binary file in 1.6251897811889648 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.911077$
After 5 runs, maximum modularity is $Q = 0.912331$
Louvain completed 25 runs in 50.27504825592041 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 135.1870744228363 seconds

PhenoGraph (P2: UMAP-3D) — 21-30w — Pre-Eclampsia
k=75, Q=0.912, sil=0.310



Finding 15 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.7465865612030029 seconds
Jaccard graph constructed in 7.435351610183716 seconds
Wrote graph to binary file in 0.3614039421081543 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.941103$
Louvain completed 21 runs in 10.144482851028442 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 19.08319878578186 seconds
Finding 30 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.0297002792358398 seconds
Jaccard graph constructed in 19.09278655052185 seconds
Wrote graph to binary file in 1.0746006965637207 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.925796$
Louvain completed 21 runs in 17.962279319763184 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 39.52839970588684 seconds
Finding 50 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.3124110698699951 seconds
Jaccard graph constructed in 42.61692667007446 seconds
Wrote graph to binary file in 1.1272501945495605 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.916543$
Louvain completed 21 runs in 28.838755130767822 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 74.33293557167053 seconds
Finding 75 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.6508300304412842 seconds
Jaccard graph constructed in 81.08863496780396 seconds
Wrote graph to binary file in 1.682194709777832 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.909521$
After 6 runs, maximum modularity is $Q = 0.910658$
Louvain completed 26 runs in 50.169018030166626 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 135.1273889541626 seconds

PhenoGraph (P2: UMAP-5D; 3D viz PCA) — 21–30w — Pre-Eclampsia
 $k=50$, $Q=0.917$, $sil=0.264$



Out[]:

	interval	condition	n_cells	umap_dim	k	silhouette	modularity_Q	composite
0	1–10w	0	15000	2	15	0.323643	0.954651	0.514182
1	1–10w	0	15000	2	30	0.358728	0.933362	0.532935
2	1–10w	0	15000	2	50	0.354911	0.918671	0.526257
3	1–10w	0	15000	2	75	0.358083	0.907554	0.525446
4	1–10w	0	15000	3	15	0.272343	0.938105	0.473760
...	...	...	...	...	...	...	...	...
67	21–30w	1	40000	3	75	0.309795	0.912331	0.492947
68	21–30w	1	40000	5	15	0.218981	0.941103	0.437224
69	21–30w	1	40000	5	30	0.257662	0.925796	0.460126
70	21–30w	1	40000	5	50	0.264265	0.916543	0.462224
71	21–30w	1	40000	5	75	0.264243	0.910658	0.460604

72 rows × 10 columns



Hyperparameter tuning with pipeline 2: 2d, 3d, 5d embed

```

In [ ]: # helpers
if 'sil_or_nan' not in globals():
    def sil_or_nan(X, labels):
        if len(np.unique(labels)) < 2 or X.shape[0] < 3:
            return np.nan
        return silhouette_score(X, labels)

if 'composite_score_pheno' not in globals():
    def composite_score_pheno(sil, Q, Qmin=-0.1, Qmax=1.0):
        qn = (Q - Qmin) / (Qmax - Qmin)
        qn = float(np.clip(qn, 0.0, 1.0))
        sil = float(np.nan_to_num(sil, nan=-1.0))
        return 0.7*sil + 0.3*qn

if 'plot_umap2' not in globals():
    def plot_umap2(emb2, labels, title, out_png):
        plt.figure(figsize=(8,7))
        sc = plt.scatter(emb2[:,0], emb2[:,1], c=labels, s=6, cmap="tab20")
        plt.title(title); plt.xlabel("UMAP1"); plt.ylabel("UMAP2")
        plt.colorbar(sc, label="Cluster ID")
        plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.show(); plt.close

if 'plot_umap3' not in globals():
    def plot_umap3(emb3, labels, title, out_png):
        from mpl_toolkits.mplot3d import Axes3D # noqa
        fig = plt.figure(figsize=(9,7))
        ax = fig.add_subplot(111, projection='3d')
        p = ax.scatter(emb3[:,0], emb3[:,1], emb3[:,2], c=labels, s=6, cmap="tab20")
        ax.set_title(title); ax.set_xlabel("UMAP1"); ax.set_ylabel("UMAP2"); ax.set_zlabel("UMAP3")
        fig.colorbar(p, label="Cluster ID")
        plt.tight_layout(); plt.savefig(out_png, dpi=150); plt.show(); plt.close

# sweep config
K_GRID_P2_HI = [80, 100, 120] # assuming you meant 100 (not 10)

metrics_rows = []

for interval in valid_bins:
    for cond in [0,1]:
        tag = f"{interval}_{cond}"
        sub_idx = eda[(eda["age_bin_10w"]==interval) & (eda["condition"]==cond)]

        for d in (2,3,5):
            upath = os.path.join(output_dir, f"umap{d}_pipeline2_{tag}.npy")
            if not os.path.exists(upath):
                print(f"[P2-Pheno HI] Missing UMAP{d} for {tag} - skip")
                continue

            emb = np.load(upath)
            n = len(emb)
            if n < 50:
                print(f"[P2-Pheno HI] {tag} dim={d}: too few cells ({n})")
                continue

            best = None
            for k in K_GRID_P2_HI:
                if k >= n: # phenograph needs k < n
                    continue
                try:

```

```

        communities, graph, Q = phenograph.cluster(emb, k=k)
    except Exception as e:
        print(f"[P2-Pheno HI] {tag} dim={d} k={k} failed:", e)
        continue

    labels = np.asarray(communities, dtype=int)
    sil = sil_or_nan(emb, labels)
    comp = composite_score_pheno(sil, Q)

    metrics_rows.append([
        str(interval), cond, n, d, k, sil, Q, comp, "", ""
    ])

    cand = dict(dim=d, k=k, sil=sil, Q=Q, composite=comp, labels=labels)
    if (best is None) or (cand["composite"] > best["composite"]):
        best = cand

if best is None:
    print(f"[P2-Pheno HI] {tag} dim={d}: no candidate found.")
    continue

# Save BEST labels (aligned to eda)
labels = best["labels"]
lab_name = f"phenograph_pipeline2_umap{best['dim']}_k{best['k']}_H
labels_csv = os.path.join(output_dir, f"labels_{lab_name}__{tag}.csv")
pd.Series(labels, index=sub_idx, name=lab_name).to_csv(labels_csv)

# Plots: 2D/3D direct; 5D → PCA → 3D for viz
if best["dim"] == 2:
    out_png = os.path.join(output_dir, f"umap2_plot_{lab_name}__{tag}")
    title = (f"PhenoGraph tuned (P2: UMAP-2D) – {interval} – {COND}
            f"k={best['k']}, Q={best['Q']:.3f}, sil={best['sil']}:")
    plot_umap2(emb, labels, title, out_png)
elif best["dim"] == 3:
    out_png = os.path.join(output_dir, f"umap3_plot_{lab_name}__{tag}")
    title = (f"PhenoGraph tuned (P2: UMAP-3D) – {interval} – {COND}
            f"k={best['k']}, Q={best['Q']:.3f}, sil={best['sil']}:")
    plot_umap3(emb, labels, title, out_png)
else: # 5D → PCA to 3D for viz only
    emb3 = PCA(n_components=3, random_state=RANDOM_STATE).fit_transform(emb)
    np.save(os.path.join(output_dir, f"umap5_pca3viz_pheno_HI__{tag}.npy"), emb3)
    out_png = os.path.join(output_dir, f"umap5_pca3viz_plot_{lab_name}__{tag}.png")
    title = (f"PhenoGraph tuned (P2: UMAP-5D; 3D viz PCA) – {interval} – {COND}
            f"k={best['k']}, Q={best['Q']:.3f}, sil={best['sil']}:")
    plot_umap3(emb3, labels, title, out_png)

# Fill plot/labels path for the last matching metrics row (best k)
for i in range(len(metrics_rows)-1, -1, -1):
    if (metrics_rows[i][0]==str(interval) and metrics_rows[i][1]==cond
        and metrics_rows[i][3]==best["dim"] and metrics_rows[i][4]==best["k"]
        and metrics_rows[i][8]=="" and metrics_rows[i][9]==""):
        metrics_rows[i][8] = out_png
        metrics_rows[i][9] = labels_csv
        break

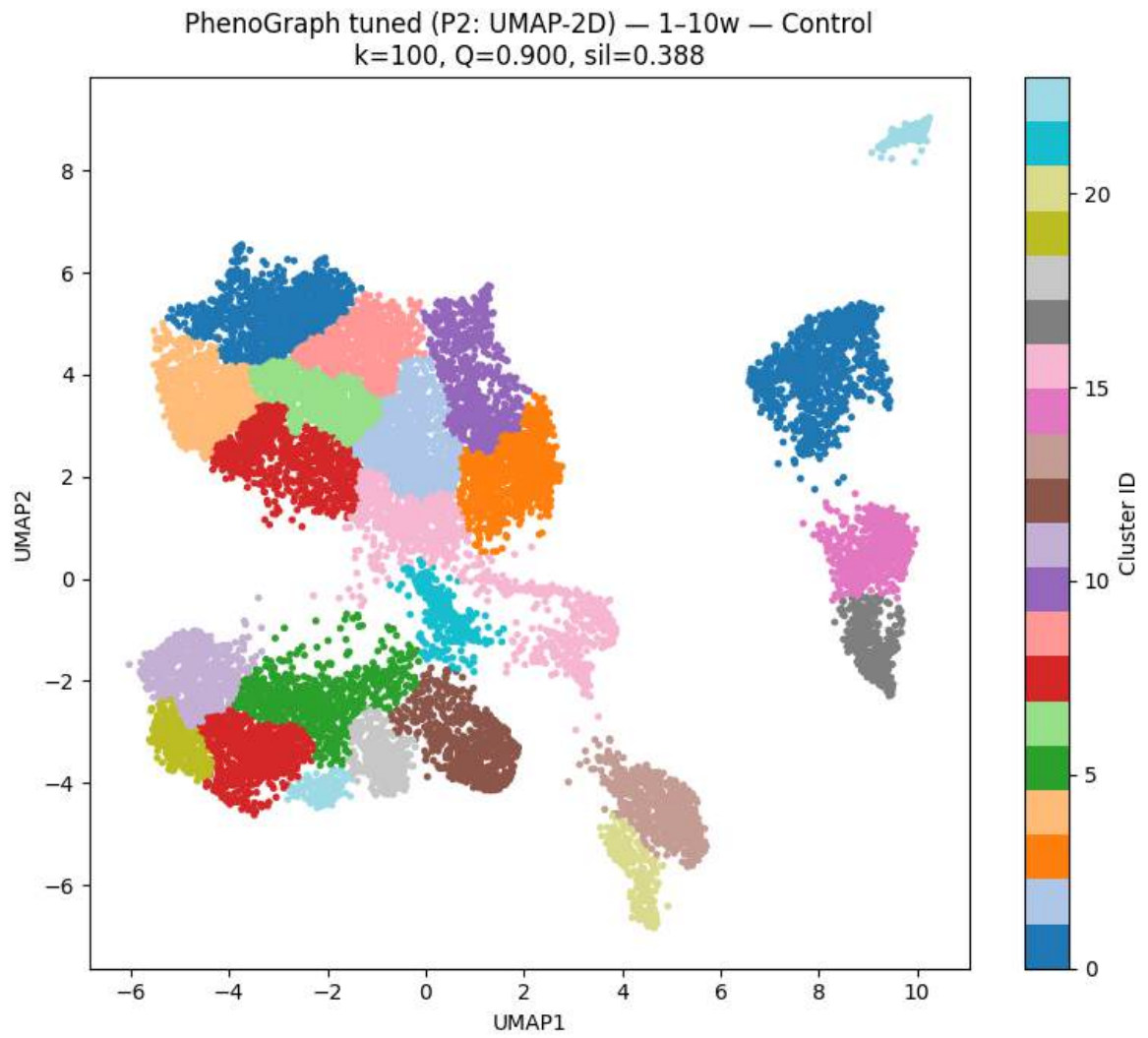
# Save metrics table (HI sweep only)
p2_hi_metrics = pd.DataFrame(metrics_rows, columns=[
    "interval", "condition", "n_cells", "umap_dim", "k", "silhouette", "modularity_Q",
])
hi_path = os.path.join(output_dir, "phenograph_pipeline2_metrics_HI.csv")

```



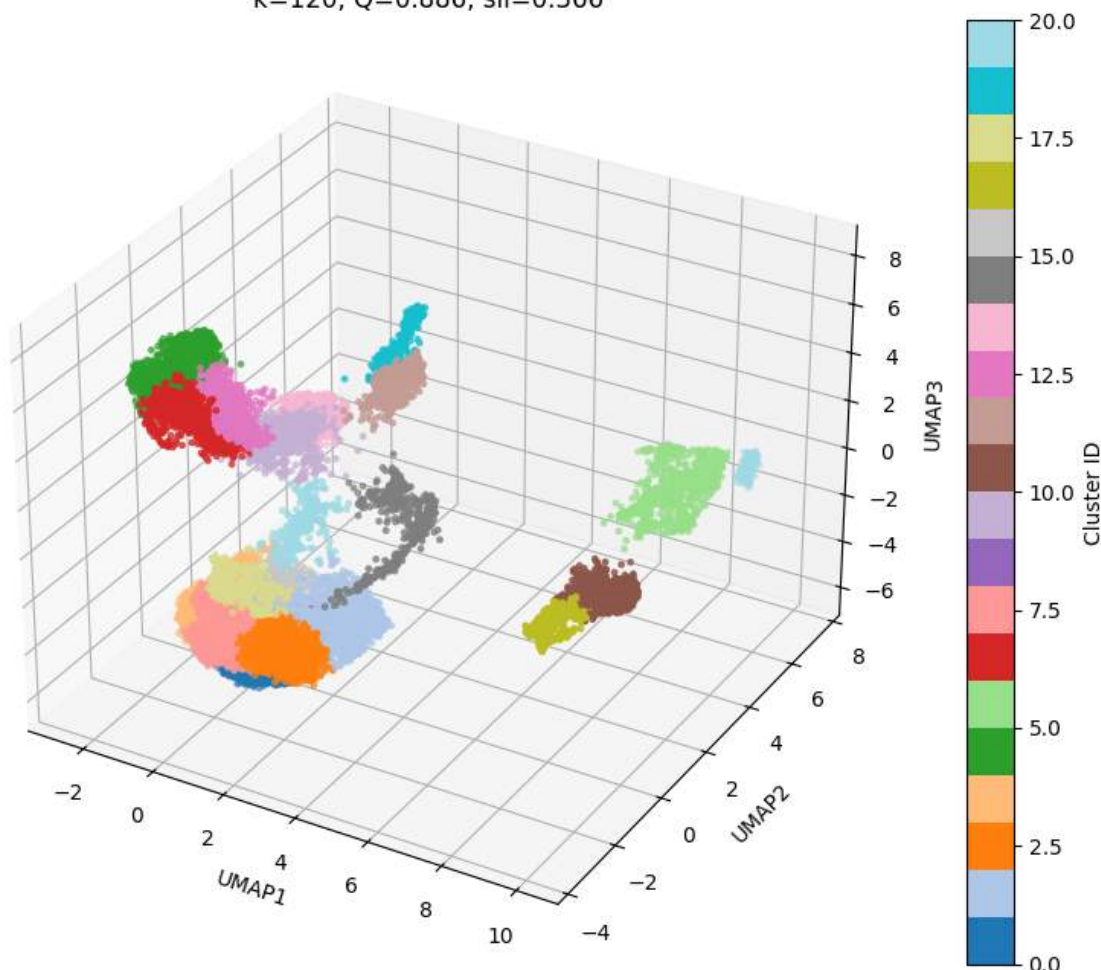
```
p2_hi_metrics.to_csv(hi_path, index=False)
p2_hi_metrics
```

```
Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.5899536609649658 seconds
Jaccard graph constructed in 56.654866218566895 seconds
Wrote graph to binary file in 0.6557438373565674 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.904904
After 5 runs, maximum modularity is Q = 0.906349
Louvain completed 25 runs in 11.717837572097778 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 69.88822746276855 seconds
Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.48054075241088867 seconds
Jaccard graph constructed in 57.43522095680237 seconds
Wrote graph to binary file in 0.7909152507781982 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.899593
Louvain completed 21 runs in 12.230415344238281 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 71.2388026714325 seconds
Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.41320347785949707 seconds
Jaccard graph constructed in 79.7306272983551 seconds
Wrote graph to binary file in 1.3023576736450195 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is Q = 0.893384
After 5 runs, maximum modularity is Q = 0.894703
Louvain completed 25 runs in 17.352773904800415 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 99.18062472343445 seconds
```



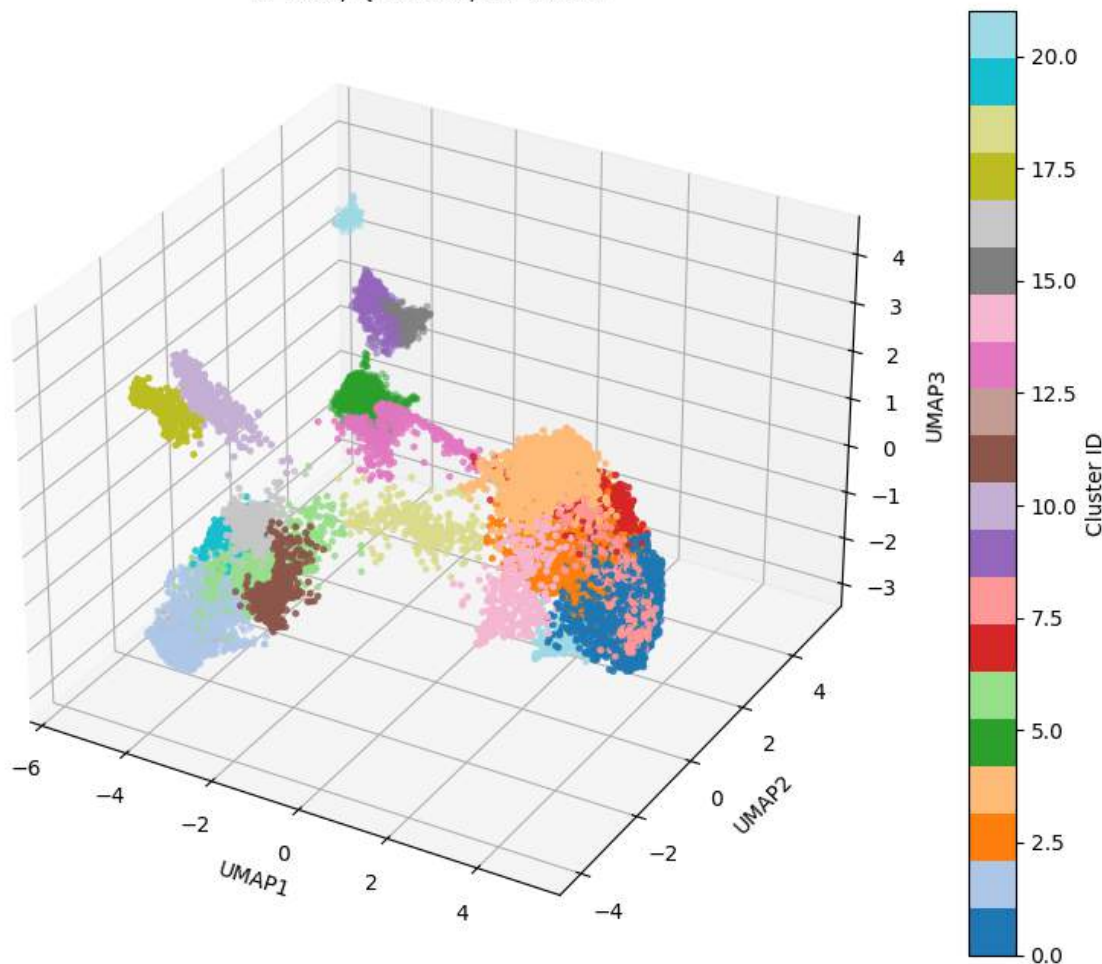
Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.29480481147766113 seconds
 Jaccard graph constructed in 39.5627338886261 seconds
 Wrote graph to binary file in 0.6457810401916504 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.893092$
 After 8 runs, maximum modularity is $Q = 0.894286$
 Louvain completed 28 runs in 15.88611102104187 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 56.66210699081421 seconds
 Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.3370859622955322 seconds
 Jaccard graph constructed in 57.03735709190369 seconds
 Wrote graph to binary file in 0.8114674091339111 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.888974$
 After 20 runs, maximum modularity is $Q = 0.890404$
 Louvain completed 40 runs in 26.97830367088318 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 85.47802567481995 seconds
 Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.42449378967285156 seconds
 Jaccard graph constructed in 81.33421206474304 seconds
 Wrote graph to binary file in 0.9628715515136719 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.885645$
 Louvain completed 21 runs in 20.1605544090271 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 103.2361114025116 seconds

PhenoGraph tuned (P2: UMAP-3D) — 1-10w — Control
 $k=120$, $Q=0.886$, $sil=0.366$



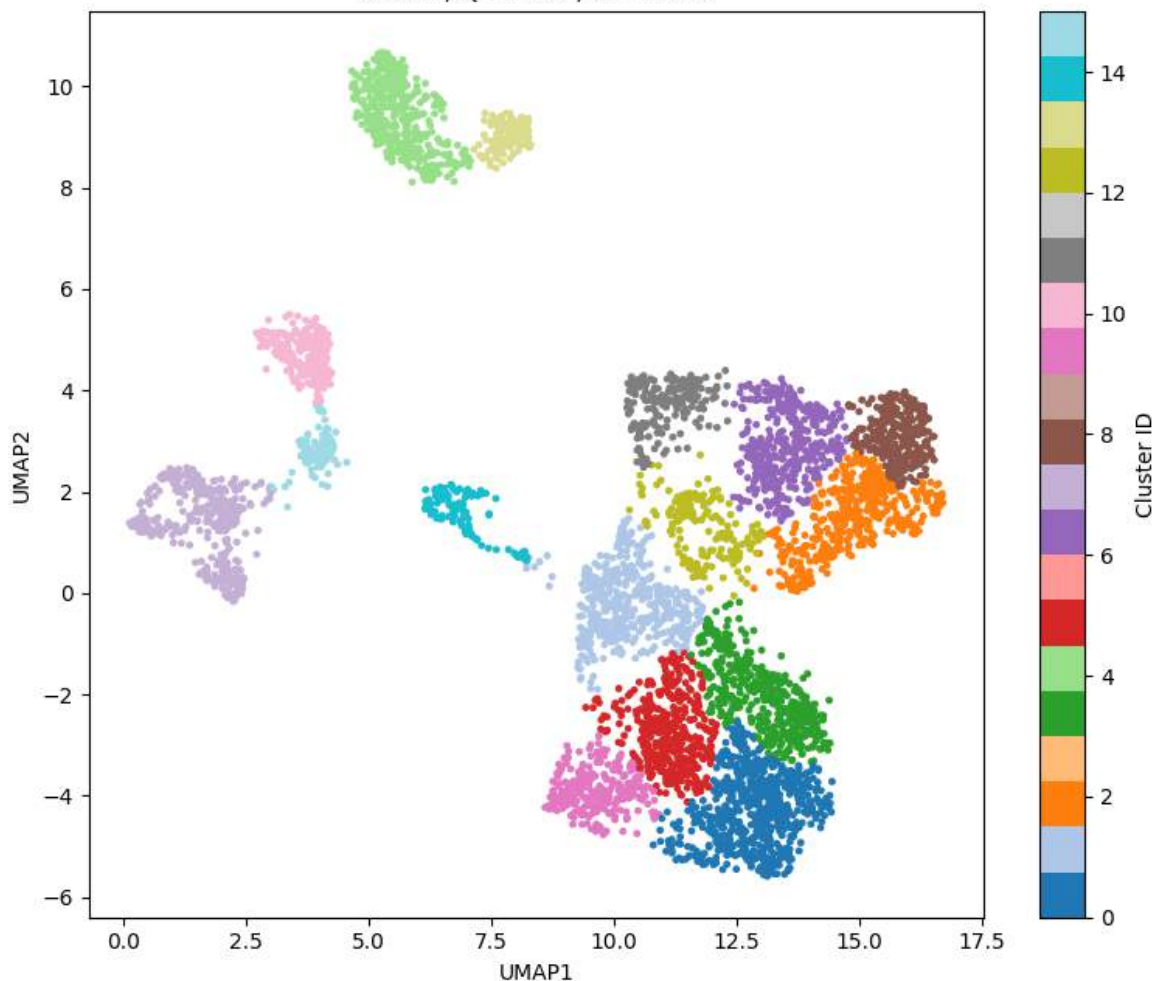
Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.4551231861114502 seconds
 Jaccard graph constructed in 38.12514877319336 seconds
 Wrote graph to binary file in 0.6848361492156982 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.895674$
 After 6 runs, maximum modularity is $Q = 0.897096$
 Louvain completed 26 runs in 14.256024837493896 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 53.904757499694824 seconds
 Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.5327968597412109 seconds
 Jaccard graph constructed in 54.78483819961548 seconds
 Wrote graph to binary file in 1.280853509902954 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.892533$
 Louvain completed 21 runs in 12.814197301864624 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 69.79962396621704 seconds
 Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.593036413192749 seconds
 Jaccard graph constructed in 77.82380390167236 seconds
 Wrote graph to binary file in 0.9961912631988525 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.88828$
 After 6 runs, maximum modularity is $Q = 0.88976$
 Louvain completed 26 runs in 18.126380443572998 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 97.88890385627747 seconds

PhenoGraph tuned (P2: UMAP-5D; 3D viz PCA) — 1-10w — Control
 $k=120$, $Q=0.890$, $sil=0.318$



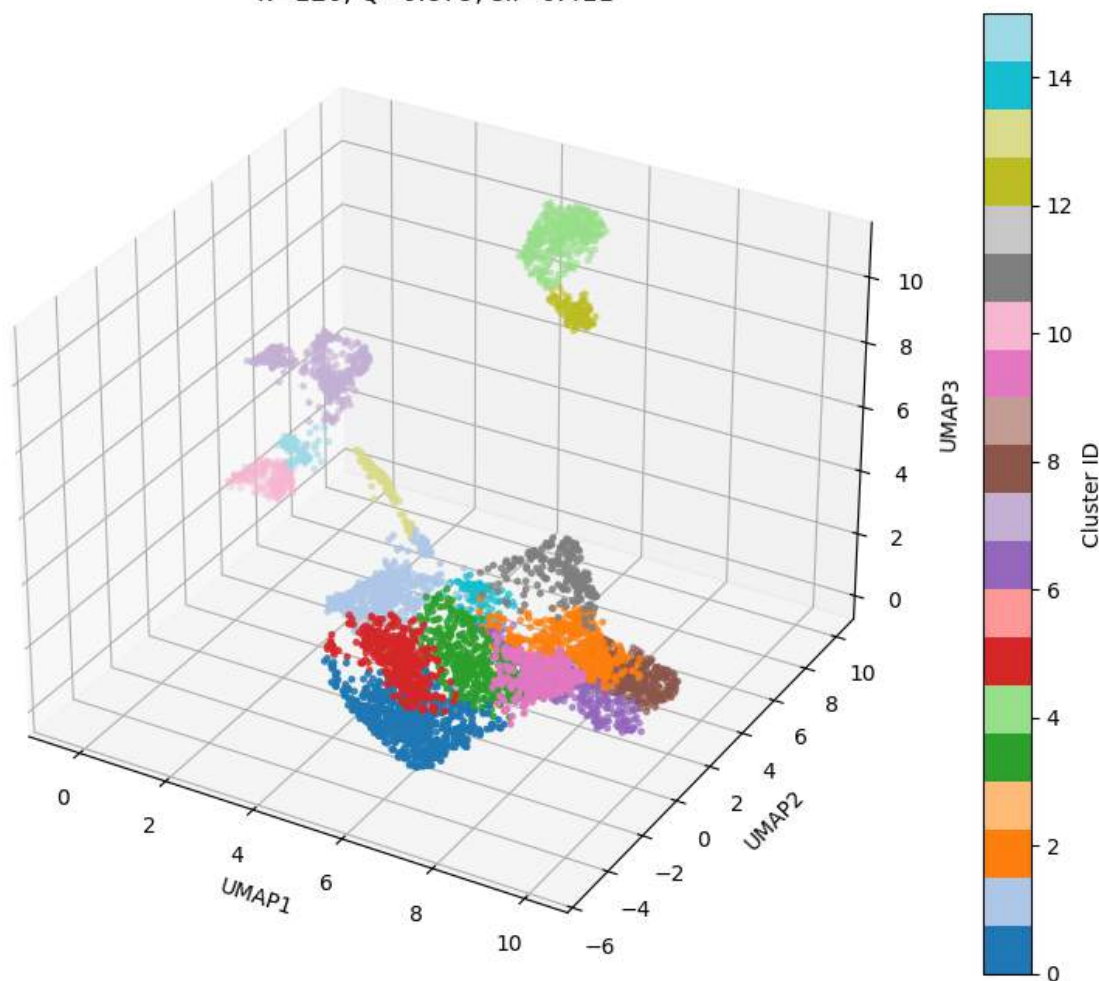
Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.10540080070495605 seconds
 Jaccard graph constructed in 12.913098573684692 seconds
 Wrote graph to binary file in 0.36884236335754395 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.891624$
 After 2 runs, maximum modularity is $Q = 0.892948$
 Louvain completed 22 runs in 2.313417911529541 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 15.90874981880188 seconds
 Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.0955667495727539 seconds
 Jaccard graph constructed in 19.037564516067505 seconds
 Wrote graph to binary file in 0.26856017112731934 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.885353$
 Louvain completed 21 runs in 3.087334394454956 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 22.763197898864746 seconds
 Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.18383312225341797 seconds
 Jaccard graph constructed in 26.812366247177124 seconds
 Wrote graph to binary file in 0.3202660083770752 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.877324$
 Louvain completed 21 runs in 3.953068494796753 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 31.583303928375244 seconds

PhenoGraph tuned (P2: UMAP-2D) — 1-10w — Pre-Eclampsia
 $k=120$, $Q=0.877$, $sil=0.459$



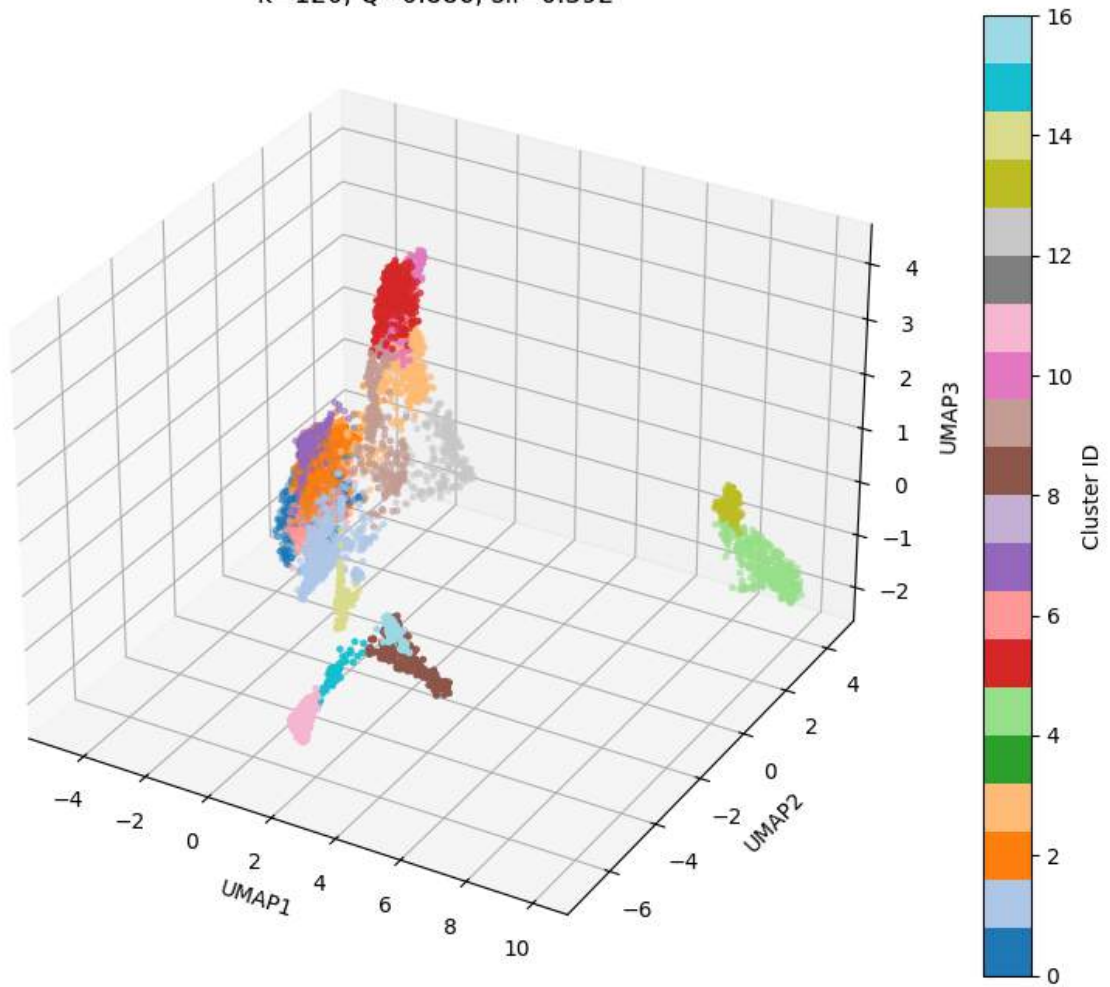
Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.10299944877624512 seconds
Jaccard graph constructed in 12.176169395446777 seconds
Wrote graph to binary file in 0.3841128349304199 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.891456$
Louvain completed 21 runs in 3.1442651748657227 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 16.024455547332764 seconds
Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.11995077133178711 seconds
Jaccard graph constructed in 18.96436309814453 seconds
Wrote graph to binary file in 0.28047823905944824 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.884298$
After 2 runs, maximum modularity is $Q = 0.885822$
Louvain completed 22 runs in 3.605628490447998 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 23.174992561340332 seconds
Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.18984436988830566 seconds
Jaccard graph constructed in 26.601105213165283 seconds
Wrote graph to binary file in 0.31958556175231934 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.877835$
After 11 runs, maximum modularity is $Q = 0.879349$
Louvain completed 31 runs in 6.15482234954834 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 33.579211950302124 seconds

PhenoGraph tuned (P2: UMAP-3D) — 1-10w — Pre-Eclampsia
 $k=120$, $Q=0.879$, $sil=0.411$

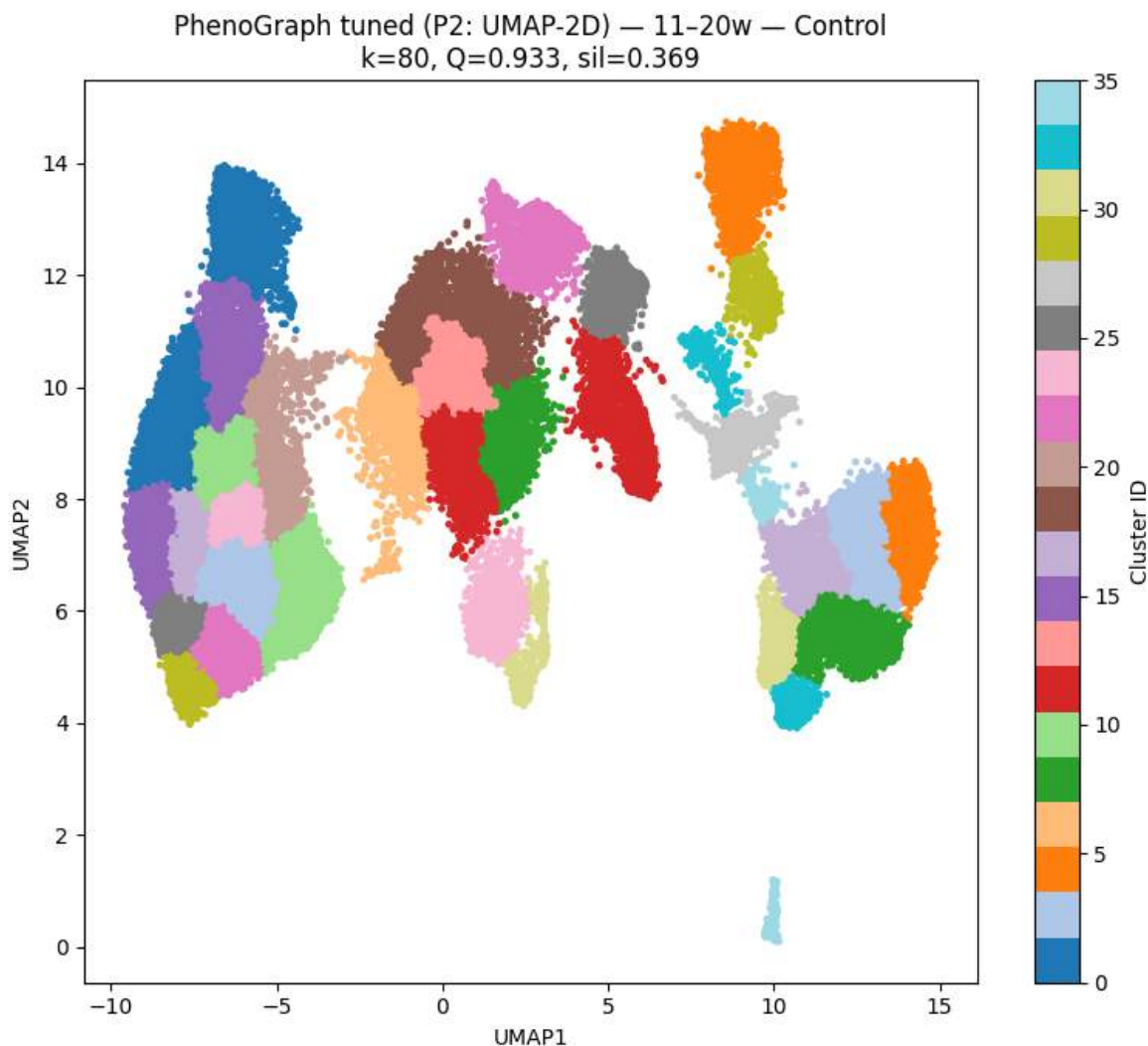


Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.14897942543029785 seconds
 Jaccard graph constructed in 12.902084112167358 seconds
 Wrote graph to binary file in 0.3995072841644287 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.898129$
 After 5 runs, maximum modularity is $Q = 0.899225$
 Louvain completed 25 runs in 3.5974810123443604 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 17.265822887420654 seconds
 Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.14778971672058105 seconds
 Jaccard graph constructed in 18.565330266952515 seconds
 Wrote graph to binary file in 0.2780928611755371 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.892829$
 Louvain completed 21 runs in 2.5130116939544678 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 21.715619802474976 seconds
 Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.275402307510376 seconds
 Jaccard graph constructed in 26.725098371505737 seconds
 Wrote graph to binary file in 0.33141422271728516 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.886067$
 Louvain completed 21 runs in 3.1272857189178467 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 30.746273517608643 seconds

PhenoGraph tuned (P2: UMAP-5D; 3D viz PCA) — 1-10w — Pre-Eclampsia
k=120, Q=0.886, sil=0.392

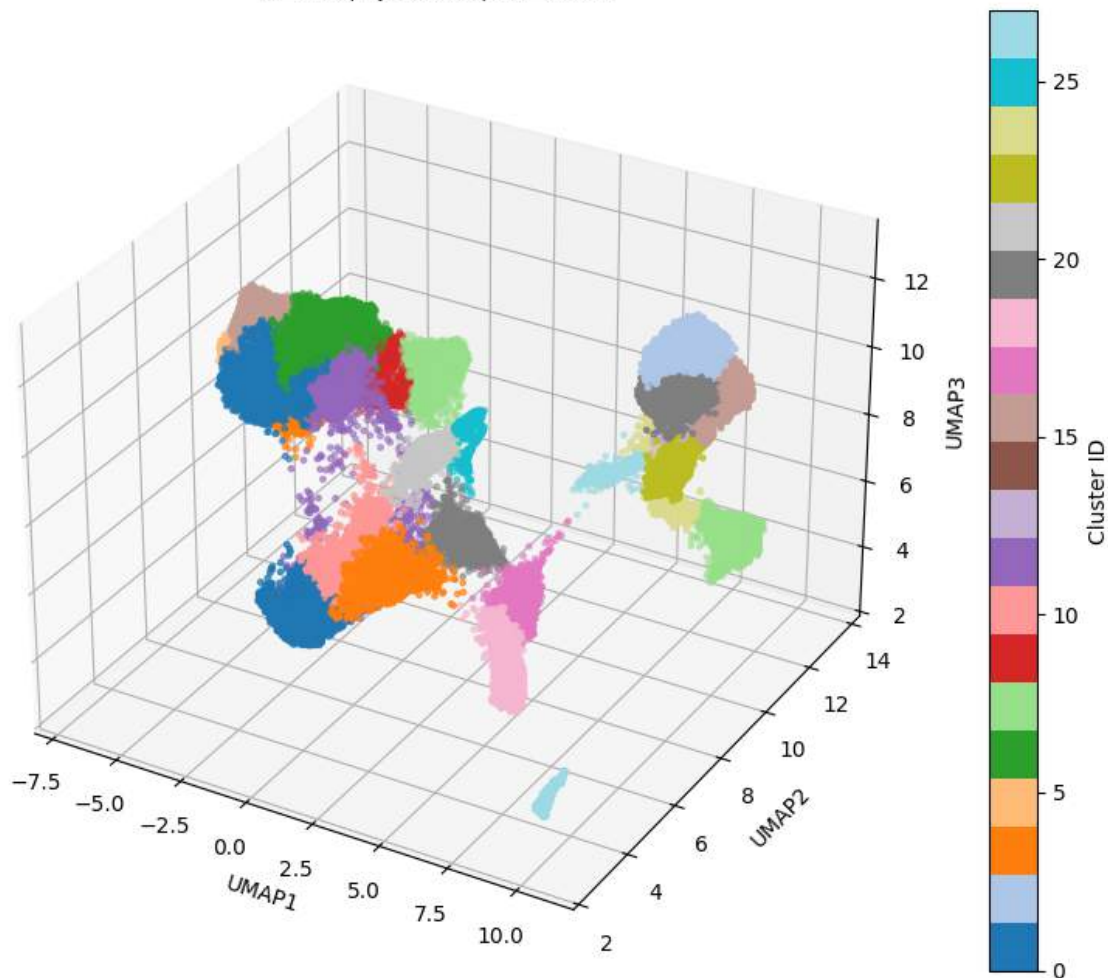


Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.1767127513885498 seconds
Jaccard graph constructed in 142.46805262565613 seconds
Wrote graph to binary file in 2.2592639923095703 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.93178$
After 15 runs, maximum modularity is $Q = 0.932783$
Louvain completed 35 runs in 85.70280599594116 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 232.4992711544037 seconds
Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.0172159671783447 seconds
Jaccard graph constructed in 211.17093753814697 seconds
Wrote graph to binary file in 2.9018566608428955 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.926986$
After 2 runs, maximum modularity is $Q = 0.928399$
Louvain completed 22 runs in 67.34201192855835 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 283.3714294433594 seconds
Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.209791898727417 seconds
Jaccard graph constructed in 289.565012216568 seconds
Wrote graph to binary file in 5.246692895889282 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.923134$
After 3 runs, maximum modularity is $Q = 0.924225$
Louvain completed 23 runs in 88.06163787841797 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 385.2835388183594 seconds



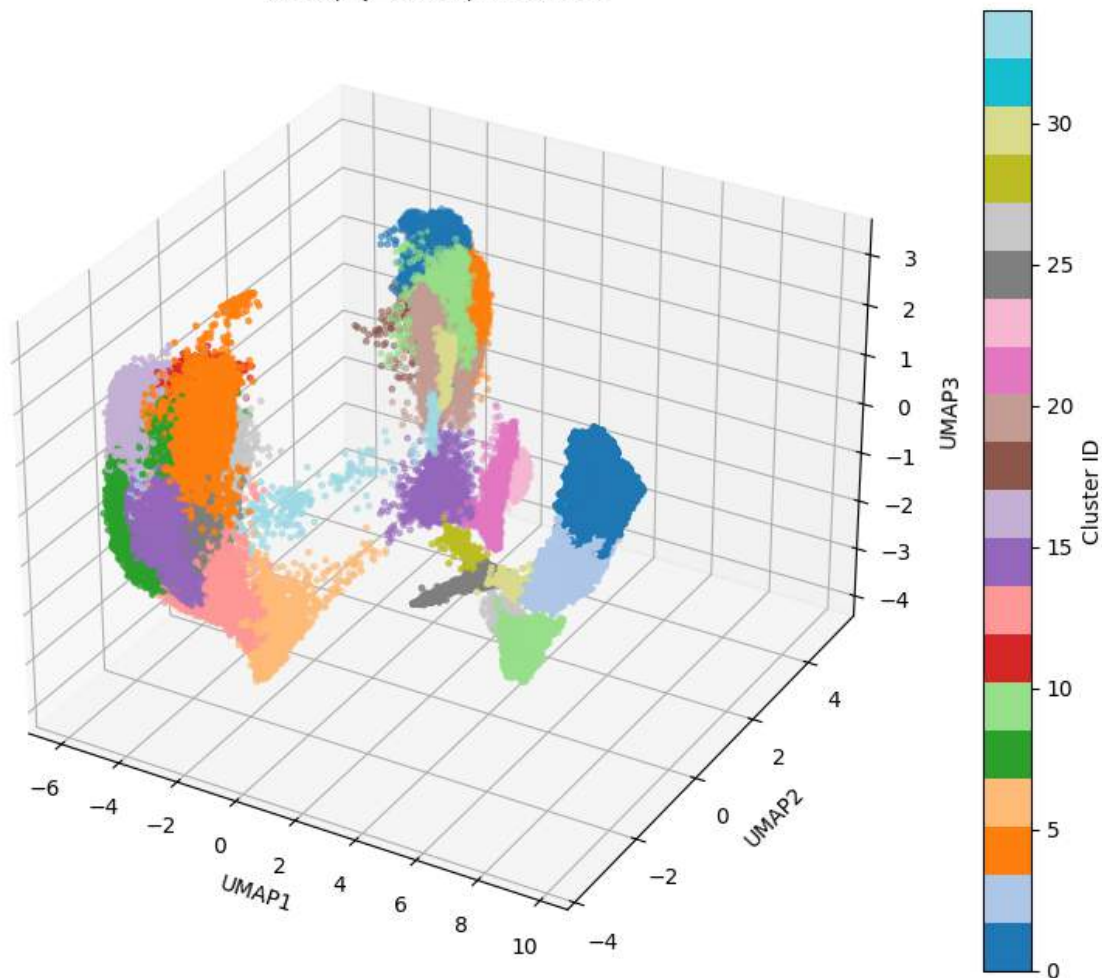
Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 1.3511431217193604 seconds
 Jaccard graph constructed in 140.1334433555603 seconds
 Wrote graph to binary file in 2.390990972518921 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is Q = 0.915138
 After 4 runs, maximum modularity is Q = 0.916236
 Louvain completed 24 runs in 77.52475476264954 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 222.09095573425293 seconds
 Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 1.5073015689849854 seconds
 Jaccard graph constructed in 208.79099583625793 seconds
 Wrote graph to binary file in 2.94399356842041 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is Q = 0.912021
 Louvain completed 21 runs in 95.28827023506165 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 309.4135932922363 seconds
 Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 2.901097297668457 seconds
 Jaccard graph constructed in 283.4177348613739 seconds
 Wrote graph to binary file in 5.23621678352356 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is Q = 0.909825
 Louvain completed 21 runs in 121.71966695785522 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 414.3698744773865 seconds

PhenoGraph tuned (P2: UMAP-3D) — 11-20w — Control
 $k=100$, $Q=0.912$, $sil=0.337$

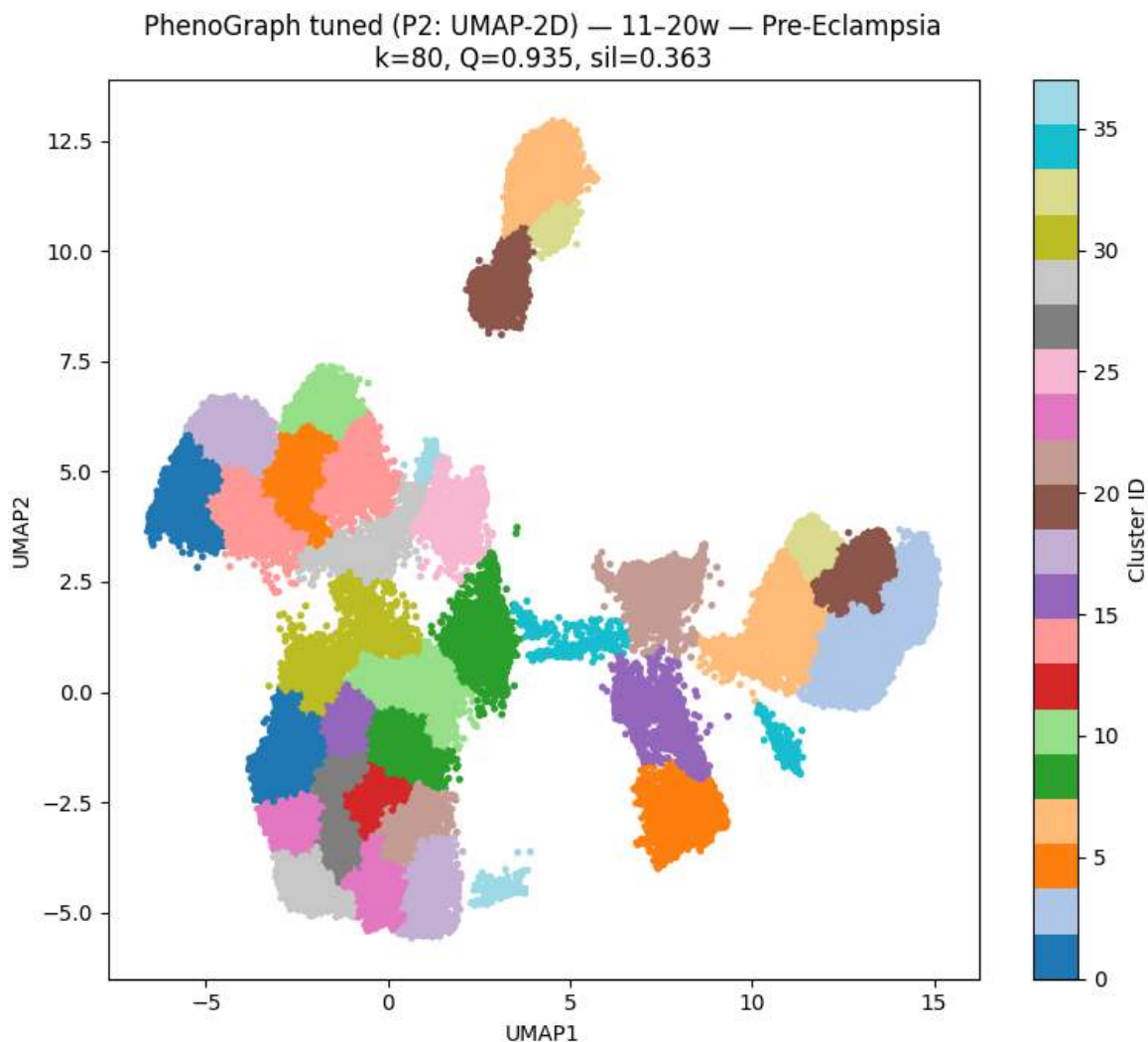


Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 6.423569917678833 seconds
 Jaccard graph constructed in 163.31687808036804 seconds
 Wrote graph to binary file in 2.48531436920166 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.916684$
 Louvain completed 21 runs in 78.76989793777466 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 251.8383538722992 seconds
 Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 2.7469632625579834 seconds
 Jaccard graph constructed in 206.81621170043945 seconds
 Wrote graph to binary file in 3.113595962524414 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.913116$
 Louvain completed 21 runs in 106.0077383518219 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 319.81260323524475 seconds
 Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 2.9850099086761475 seconds
 Jaccard graph constructed in 287.16913747787476 seconds
 Wrote graph to binary file in 3.810793876647949 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.910808$
 After 19 runs, maximum modularity is $Q = 0.911971$
 Louvain completed 39 runs in 217.16253304481506 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 512.4358265399933 seconds

PhenoGraph tuned (P2: UMAP-5D; 3D viz PCA) — 11-20w — Control
 $k=80$, $Q=0.917$, $sil=0.313$

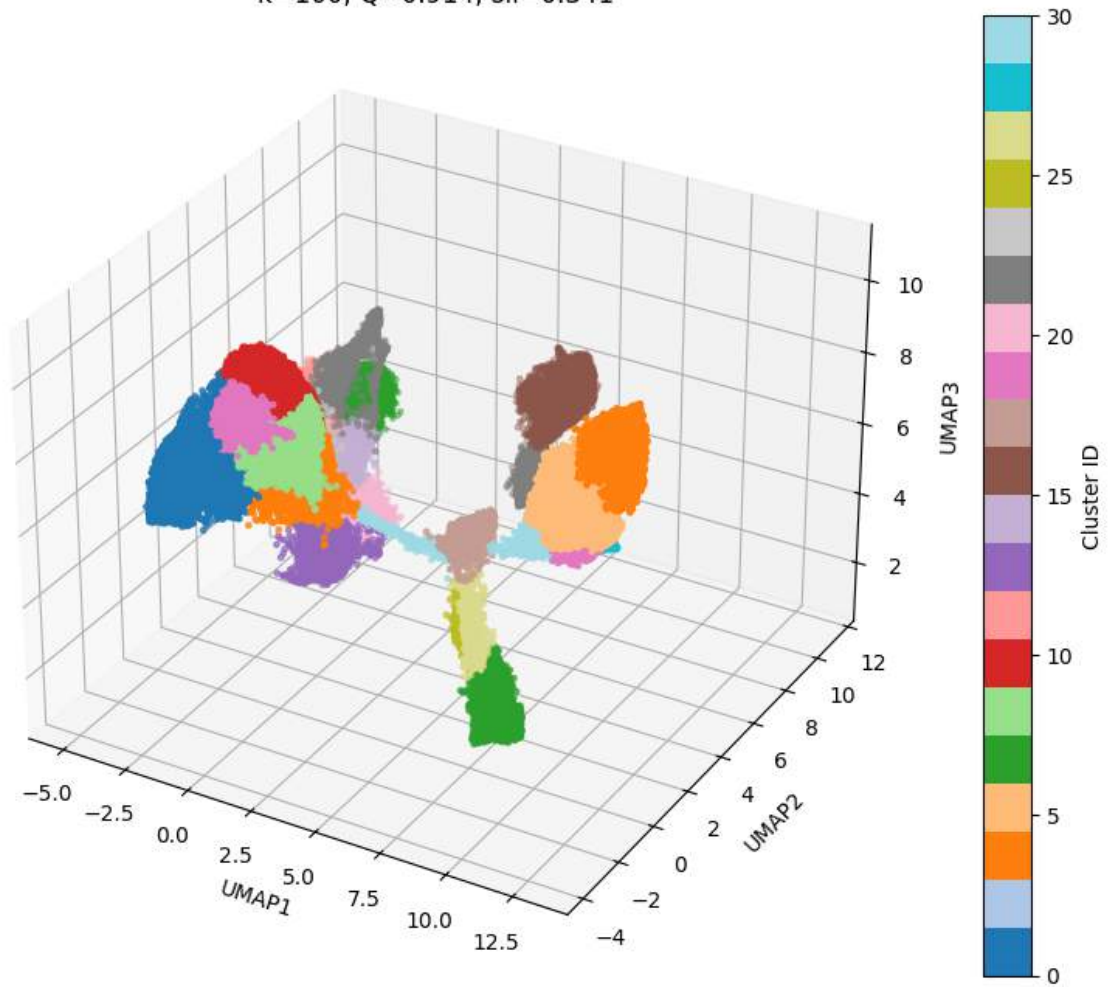


Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 1.7326080799102783 seconds
 Jaccard graph constructed in 170.90297198295593 seconds
 Wrote graph to binary file in 2.744663715362549 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.934046$
 After 20 runs, maximum modularity is $Q = 0.935157$
 Louvain completed 40 runs in 118.03657984733582 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 294.3375360965729 seconds
 Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 1.282806396484375 seconds
 Jaccard graph constructed in 247.65699076652527 seconds
 Wrote graph to binary file in 4.546693325042725 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.930046$
 Louvain completed 21 runs in 85.38459825515747 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 340.0627393722534 seconds
 Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 2.4960215091705322 seconds
 Jaccard graph constructed in 343.779727935791 seconds
 Wrote graph to binary file in 5.625703811645508 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.925683$
 Louvain completed 21 runs in 92.3410210609436 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 445.641104221344 seconds



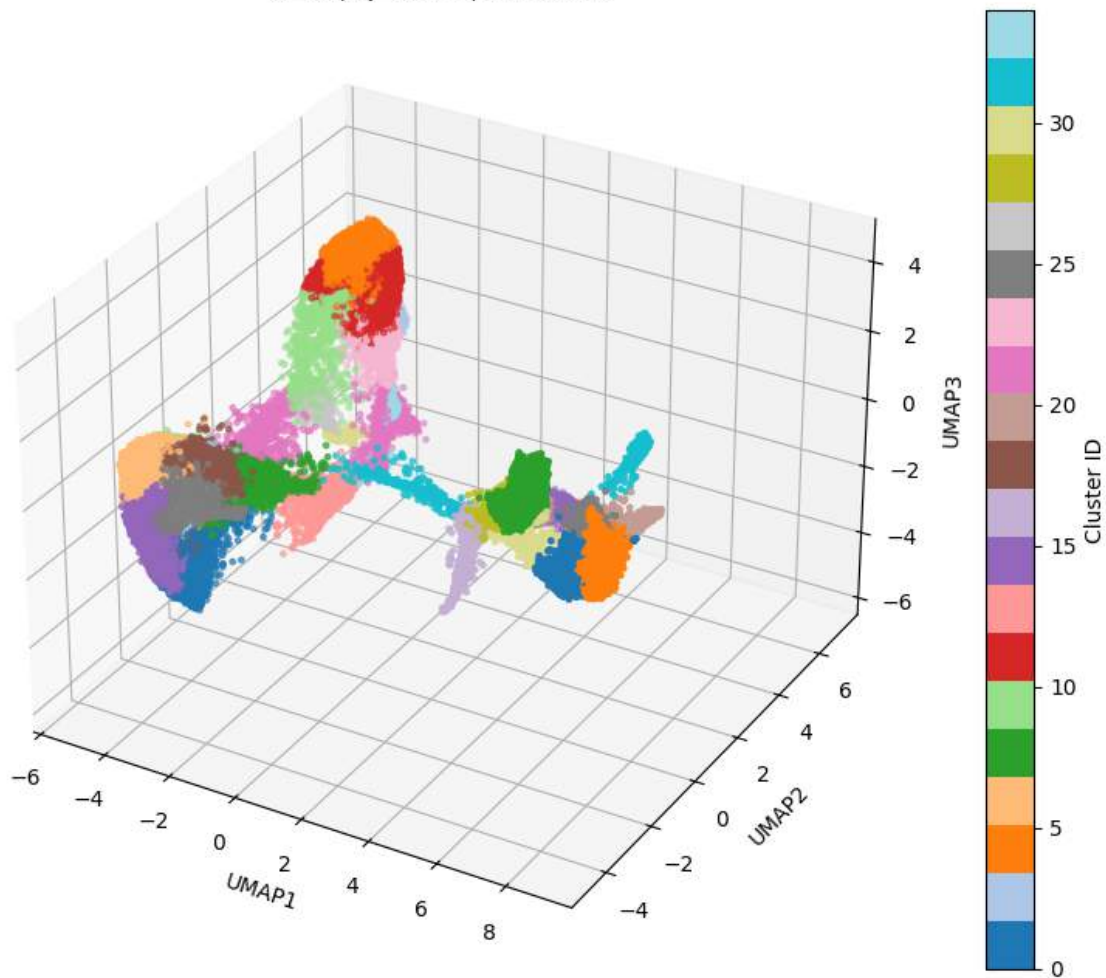
Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 2.385453462600708 seconds
 Jaccard graph constructed in 165.1489782333374 seconds
 Wrote graph to binary file in 4.375205039978027 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is Q = 0.917325
 Louvain completed 21 runs in 92.17990732192993 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 265.05629444122314 seconds
 Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 1.9558680057525635 seconds
 Jaccard graph constructed in 260.1485273838043 seconds
 Wrote graph to binary file in 4.013524770736694 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is Q = 0.912439
 After 3 runs, maximum modularity is Q = 0.913568
 Louvain completed 23 runs in 136.94790410995483 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 404.2919273376465 seconds
 Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 3.5336079597473145 seconds
 Jaccard graph constructed in 352.8401629924774 seconds
 Wrote graph to binary file in 4.658632755279541 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is Q = 0.9117
 Louvain completed 21 runs in 140.27329778671265 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 502.84379148483276 seconds

PhenoGraph tuned (P2: UMAP-3D) — 11-20w — Pre-Eclampsia
k=100, Q=0.914, sil=0.341

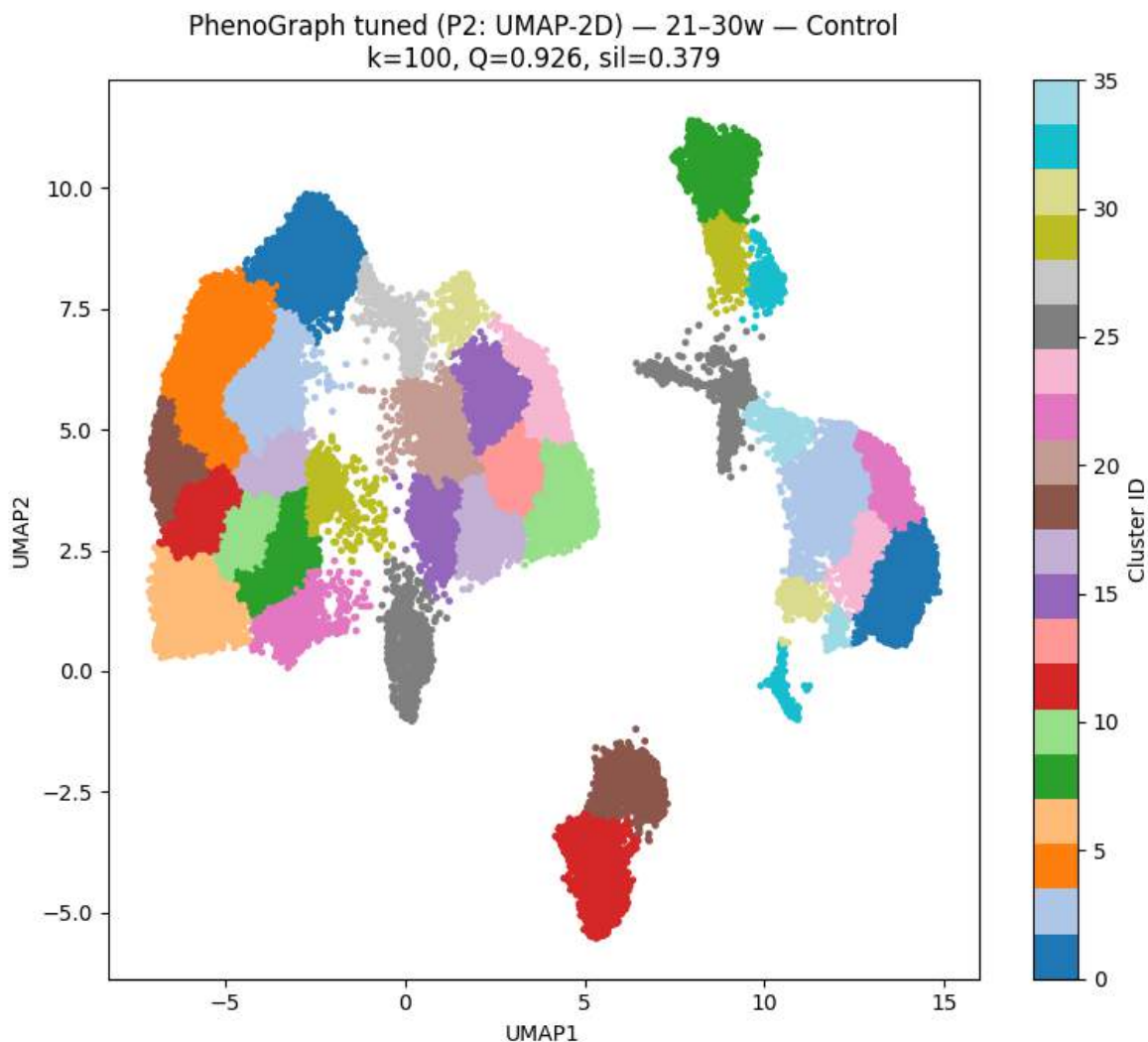


Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 4.162856340408325 seconds
Jaccard graph constructed in 172.89700865745544 seconds
Wrote graph to binary file in 3.0249881744384766 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.913112$
After 3 runs, maximum modularity is $Q = 0.915074$
Louvain completed 23 runs in 116.9862208366394 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 298.0786452293396 seconds
Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 4.600757598876953 seconds
Jaccard graph constructed in 255.1728286743164 seconds
Wrote graph to binary file in 4.7846550941467285 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.912341$
Louvain completed 21 runs in 150.72172665596008 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 416.55774760246277 seconds
Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 6.7239158153533936 seconds
Jaccard graph constructed in 349.76835656166077 seconds
Wrote graph to binary file in 6.079084396362305 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.909648$
After 3 runs, maximum modularity is $Q = 0.911285$
Louvain completed 23 runs in 190.18268847465515 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 554.1229033470154 seconds

PhenoGraph tuned (P2: UMAP-5D; 3D viz PCA) — 11-20w — Pre-Eclampsia
 $k=80$, $Q=0.915$, $sil=0.284$

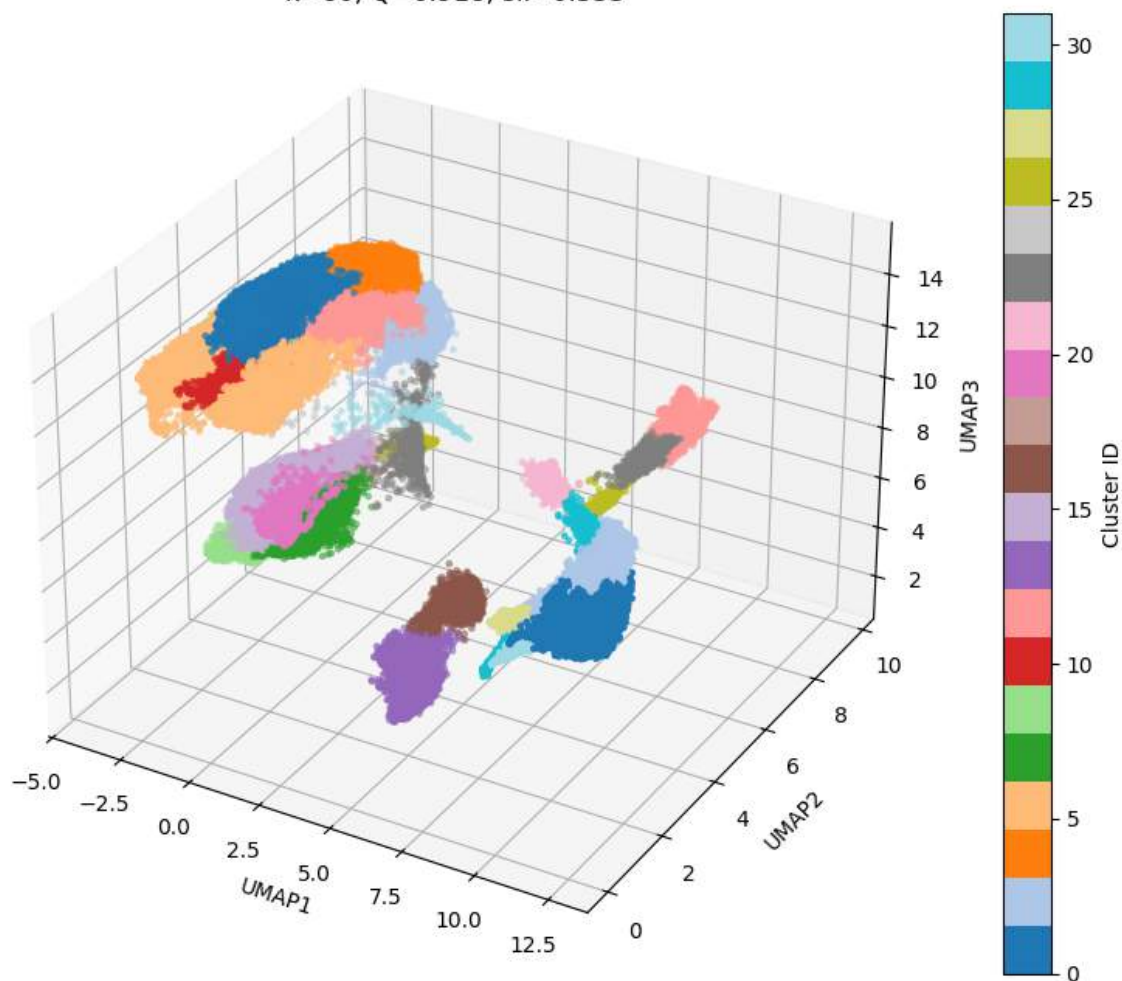


Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 0.8293545246124268 seconds
 Jaccard graph constructed in 136.87858271598816 seconds
 Wrote graph to binary file in 2.203493118286133 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.930592$
 Louvain completed 21 runs in 46.48412847518921 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 187.1946222782135 seconds
 Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 1.057882308959961 seconds
 Jaccard graph constructed in 191.26708126068115 seconds
 Wrote graph to binary file in 2.5211703777313232 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.925208$
 After 6 runs, maximum modularity is $Q = 0.926284$
 Louvain completed 26 runs in 72.91448903083801 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 268.54842495918274 seconds
 Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 1.0748341083526611 seconds
 Jaccard graph constructed in 261.2863140106201 seconds
 Wrote graph to binary file in 3.0627686977386475 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.921768$
 Louvain completed 21 runs in 67.9498393535614 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 334.3175096511841 seconds



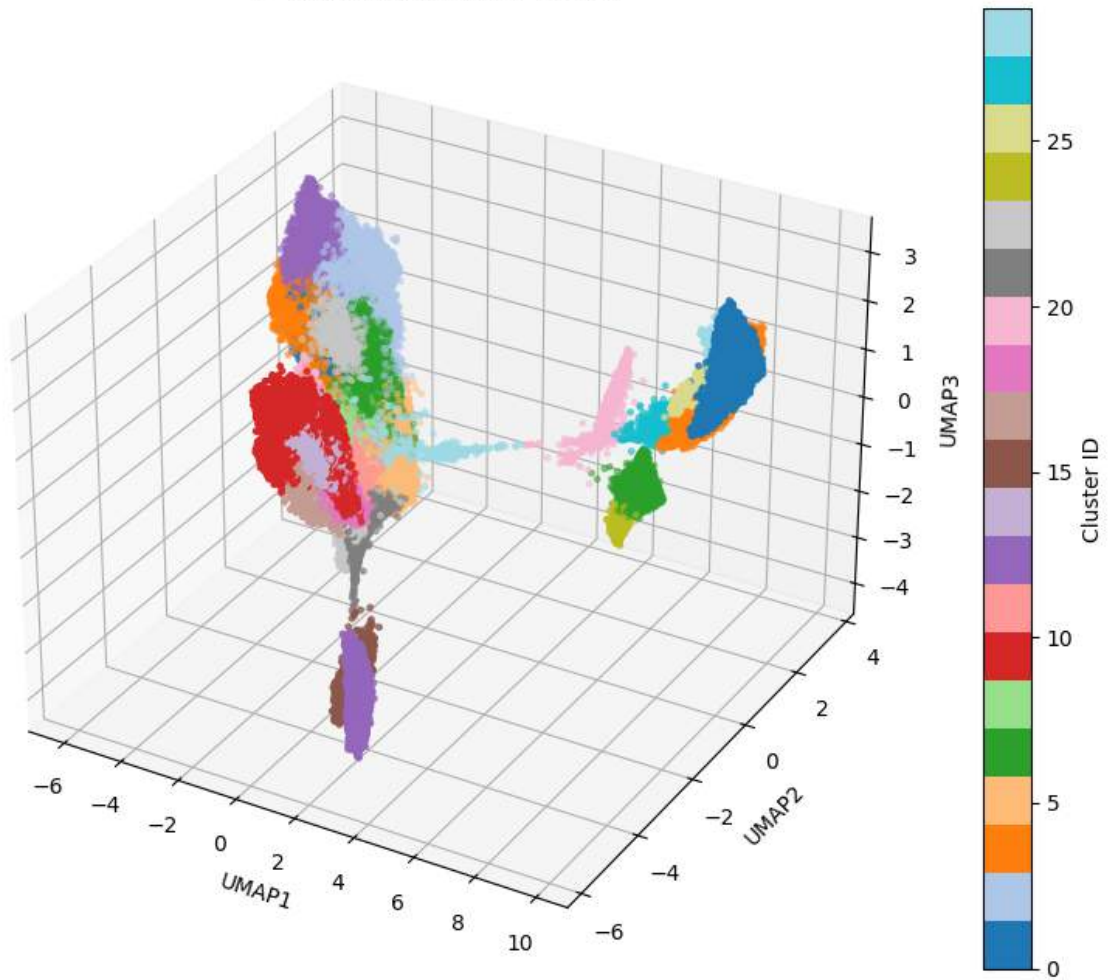
Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 1.294168472290039 seconds
 Jaccard graph constructed in 129.1274151802063 seconds
 Wrote graph to binary file in 2.1600759029388428 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.918377$
 Louvain completed 21 runs in 61.96269083023071 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 195.29214429855347 seconds
 Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 1.3986611366271973 seconds
 Jaccard graph constructed in 186.07723808288574 seconds
 Wrote graph to binary file in 3.4225685596466064 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.913749$
 Louvain completed 21 runs in 79.4472451210022 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 271.3554618358612 seconds
 Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 1.7225430011749268 seconds
 Jaccard graph constructed in 263.86975383758545 seconds
 Wrote graph to binary file in 4.068781852722168 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.9096$
 After 2 runs, maximum modularity is $Q = 0.911456$
 Louvain completed 22 runs in 98.06776595115662 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 368.90172123908997 seconds

PhenoGraph tuned (P2: UMAP-3D) — 21-30w — Control
 $k=80$, $Q=0.918$, $sil=0.353$

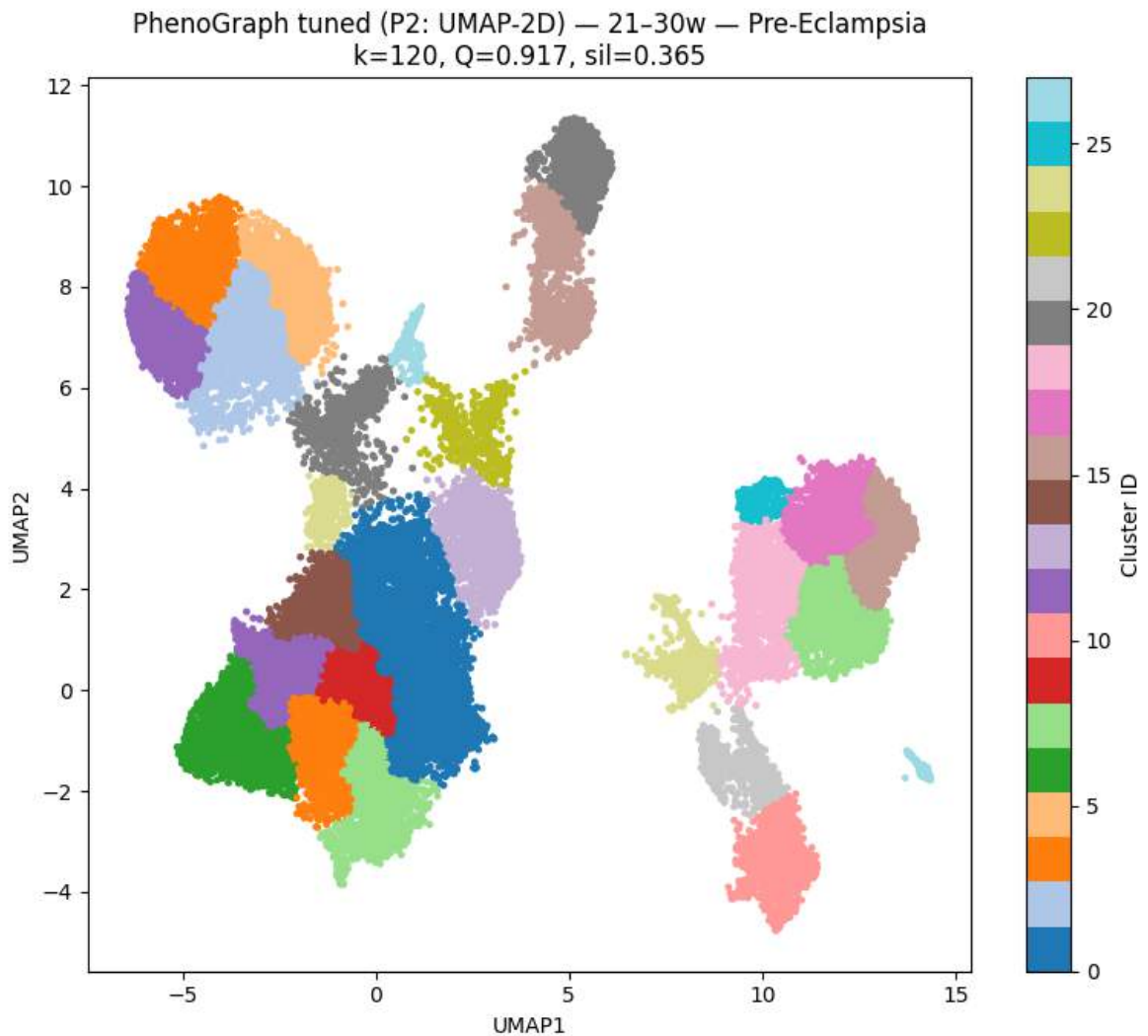


Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 2.1196954250335693 seconds
 Jaccard graph constructed in 125.99928021430969 seconds
 Wrote graph to binary file in 2.2087697982788086 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.914084$
 After 9 runs, maximum modularity is $Q = 0.915335$
 Louvain completed 29 runs in 90.45610237121582 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 221.46142196655273 seconds
 Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 3.998262882232666 seconds
 Jaccard graph constructed in 186.17840909957886 seconds
 Wrote graph to binary file in 2.8879668712615967 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.912818$
 Louvain completed 21 runs in 83.37931489944458 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 277.2561311721802 seconds
 Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
 Neighbors computed in 2.7295825481414795 seconds
 Jaccard graph constructed in 256.4590380191803 seconds
 Wrote graph to binary file in 3.9662435054779053 seconds
 Running Louvain modularity optimization
 After 1 runs, maximum modularity is $Q = 0.911409$
 Louvain completed 21 runs in 95.15346670150757 seconds
 Sorting communities by size, please wait ...
 PhenoGraph completed in 359.3791494369507 seconds

PhenoGraph tuned (P2: UMAP-5D; 3D viz PCA) — 21-30w — Control
k=120, Q=0.911, sil=0.330



Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.7445361614227295 seconds
Jaccard graph constructed in 103.57435536384583 seconds
Wrote graph to binary file in 1.7577593326568604 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.925116$
After 9 runs, maximum modularity is $Q = 0.926355$
Louvain completed 29 runs in 45.22431397438049 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 151.99092745780945 seconds
Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 0.766826868057251 seconds
Jaccard graph constructed in 151.24197268486023 seconds
Wrote graph to binary file in 2.075916290283203 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.919907$
Louvain completed 21 runs in 44.60724711418152 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 199.4109697341919 seconds
Finding 120 nearest neighbors using minkowski metric and 'auto' algorithm
Neighbors computed in 1.2362353801727295 seconds
Jaccard graph constructed in 217.33736753463745 seconds
Wrote graph to binary file in 2.608494997024536 seconds
Running Louvain modularity optimization
After 1 runs, maximum modularity is $Q = 0.914973$
After 5 runs, maximum modularity is $Q = 0.916517$
Louvain completed 25 runs in 70.25891304016113 seconds
Sorting communities by size, please wait ...
PhenoGraph completed in 292.43560910224915 seconds



Finding 80 nearest neighbors using minkowski metric and 'auto' algorithm

Neighbors computed in 0.9889395236968994 seconds

Jaccard graph constructed in 104.61696720123291 seconds

Wrote graph to binary file in 1.7993323802947998 seconds

Running Louvain modularity optimization

After 1 runs, maximum modularity is Q = 0.909612

After 4 runs, maximum modularity is Q = 0.911501

Louvain completed 24 runs in 53.352036237716675 seconds

Sorting communities by size, please wait ...

PhenoGraph completed in 161.39341521263123 seconds

Finding 100 nearest neighbors using minkowski metric and 'auto' algorithm

Neighbors computed in 1.1251580715179443 seconds

Creating a storage inventory for all cluster runs across algos

```
In [ ]: import os
import pandas as pd
```

```
In [ ]: # --- Adjust these if your paths are different ---
output_dir = "/content/drive/MyDrive/preeclampsia_results"
eda_path = "/content/drive/MyDrive/pre eclampsia/eda.csv"

eda = pd.read_csv(eda_path)

# Make sure age_bin_10w is a categorical with the labels you used
if "age_bin_10w" in eda.columns and not pd.api.types.is_categorical_dtype(eda["a
cats = ["1-10w", "11-20w", "21-30w", "30+w"]
eda["age_bin_10w"] = eda["age_bin_10w"].astype(
```



```

        pd.CategoricalDtype(categories=cats, ordered=True)
    )

    # Map (interval, condition) -> n_cells so we can reuse everywhere
    cell_count_map = (
        eda
        .groupby(["age_bin_10w", "condition"])
        .size()
        .rename("n_cells")
        .reset_index()
    )
    cell_count_map["interval"] = cell_count_map["age_bin_10w"].astype(str)
    cell_count_map["condition"] = cell_count_map["condition"].astype(int)
    cell_count_map = cell_count_map[["interval", "condition", "n_cells"]]

    def add_n_cells(df, interval_col="interval", cond_col="condition"):
        """Left-join n_cells from eda onto df by (interval, condition)."""
        m = cell_count_map.copy()
        # normalise types
        df[interval_col] = df[interval_col].astype(str)
        df[cond_col] = df[cond_col].astype(int)
        return df.merge(
            m,
            how="left",
            left_on=[interval_col, cond_col],
            right_on=["interval", "condition"],
            suffixes=("", "_from_map")
        )

    def abs_path_or_none(rel):
        if isinstance(rel, str) and rel.strip():
            return os.path.join(output_dir, rel)
        return ""

    def safe_read_csv(path, **kwargs):
        """Read CSV if exists, otherwise return None and print a message."""
        if os.path.exists(path):
            df = pd.read_csv(path, **kwargs)
            df = df.reset_index(drop=True)
            return df
        else:
            print(f" File not found: {path}")
            return None

```

```

/tmp/ipython-input-3047874919.py:9: DeprecationWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead
    if "age_bin_10w" in eda.columns and not pd.api.types.is_categorical_dtype(eda["age_bin_10w"]):
/tmp/ipython-input-3047874919.py:18: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
    .groupby(["age_bin_10w", "condition"])

```

In []: # --- Build clean n_cells map from EDA ---

```

def n_cells_map_from_eda(eda):
    """
    Returns a Series indexed by (interval, condition) with counts of cells.
    interval == age_bin_10w; condition == 0/1

```

```

"""
s = (
    eda
    .groupby(["age_bin_10w", "condition"], observed=False)
    .size()
    .rename("n_cells")
)
return s

n_cells_map = n_cells_map_from_eda(eda)

def fill_n_cells(df):
    """
    Add n_cells_from_map column using (interval, condition) key.
    If df already has an n_cells column, keep it but also add the mapped one.
    """
    df = df.copy()
    key = list(zip(df["interval"].astype(str), df["condition"].astype(int)))
    df["n_cells_from_map"] = [
        n_cells_map.get((interval, cond), np.nan) for interval, cond in key
    ]
    return df

```

For reproducibility and also to recover and lay foundation for the following steps which include evaluation, marker enrichment, etc., I created an inventory of cluster run information and other relevant metadata

K means inventory

```

In [ ]: # Putting both k means pipelines into 1 csv for analysis
# Path ---
output_dir = "/content/drive/MyDrive/preeclampsia_results"
eda_path = "/content/drive/MyDrive/pre eclampsia/eda.csv"

eda = pd.read_csv(eda_path)

# Ensuring age bin has correct labels and data type
if "age_bin_10w" in eda.columns and not pd.api.types.is_categorical_dtype(eda["age_bin_10w"]):
    cats = ["1-10w", "11-20w", "21-30w", "30+w"]
    eda["age_bin_10w"] = eda["age_bin_10w"].astype(
        pd.CategoricalDtype(categories=cats, ordered=True)
    )

# Map (interval, condition)
cell_count_map = (
    eda
    .groupby(["age_bin_10w", "condition"])
    .size()
    .rename("n_cells")
    .reset_index()
)
cell_count_map["interval"] = cell_count_map["age_bin_10w"].astype(str)
cell_count_map["condition"] = cell_count_map["condition"].astype(int)
cell_count_map = cell_count_map[["interval", "condition", "n_cells"]]

def add_n_cells(df, interval_col="interval", cond_col="condition"):
    """Left-join n_cells from eda onto df by (interval, condition)."""
    m = cell_count_map.copy()

```

```

# normalise types
df[interval_col] = df[interval_col].astype(str)
df[cond_col]      = df[cond_col].astype(int)
return df.merge(
    m,
    how="left",
    left_on=[interval_col, cond_col],
    right_on=["interval", "condition"],
    suffixes=("", "_from_map")
)

def abs_path_or_none(rel):
    if isinstance(rel, str) and rel.strip():
        return os.path.join(output_dir, rel)
    return ""

def safe_read_csv(path, **kwargs):
    """Read CSV if exists, otherwise return None and print a message."""
    if os.path.exists(path):
        df = pd.read_csv(path, **kwargs)
        df = df.reset_index(drop=True)
        return df
    else:
        print(f" File not found: {path}")
        return None

# KMEANS INVENTORY

kmeans_tables = []

# Pipeline 1 summary (PCA20 → KMeans, UMAP2 viz)
p1_path = os.path.join(output_dir, "kmeans_p1_summary.csv")
p1 = safe_read_csv(p1_path)
if p1 is not None:
    p1["algorithm"] = "KMeans"
    p1["pipeline"] = "PCA20→KMeans (P1)"
    # add full paths for plots/CSVs
    for col in ["silhouette_csv", "gap_csv", "silhouette_png", "gap_png", "umap_plot":
        if col in p1.columns:
            p1[col] = p1[col].apply(abs_path_or_none)
    # add n_cells
    p1 = add_n_cells(p1, "interval", "condition")
    kmeans_tables.append(p1)

# Pipeline 2 summary (UMAP2/3/5 → KMeans)
p2_path = os.path.join(output_dir, "kmeans_p2_summary.csv")
p2 = safe_read_csv(p2_path)
if p2 is not None:
    p2["algorithm"] = "KMeans"
    # pipeline column already encodes UMAP dimension
    # add full paths for plots/CSVs
    for col in ["silhouette_csv", "gap_csv", "silhouette_png", "gap_png", "umap_plot":
        if col in p2.columns:
            p2[col] = p2[col].apply(abs_path_or_none)
    # add n_cells
    p2 = add_n_cells(p2, "interval", "condition")
    kmeans_tables.append(p2)

if kmeans_tables:

```

```

kmeans_inventory = pd.concat(kmeans_tables, ignore_index=True, sort=False)
kmeans_inventory_path = os.path.join(output_dir, "kmeans_inventory.csv")
kmeans_inventory.to_csv(kmeans_inventory_path, index=False)
print(" KMeans inventory saved to:", kmeans_inventory_path)
else:
    print("No KMeans summary tables found; KMeans inventory not created.")

```

/tmp/ipython-input-3761972848.py:9: DeprecationWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead

```

if "age_bin_10w" in eda.columns and not pd.api.types.is_categorical_dtype(eda["age_bin_10w"]):

```

/tmp/ipython-input-3761972848.py:18: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```

.groupby(["age_bin_10w", "condition"])

```

KMeans inventory saved to: /content/drive/MyDrive/preeclampsia_results/kmeans_inventory.csv

```

In [ ]: # Displaying inventory
output_dir = "/content/drive/MyDrive/preeclampsia_results"
kmeans_inv_path = os.path.join(output_dir, "kmeans_inventory.csv")

# Load and inspect
kmeans_inv = pd.read_csv(kmeans_inv_path)

print("KMeans inventory shape:", kmeans_inv.shape)
print("KMeans inventory columns:", list(kmeans_inv.columns))

# Optional: peek at the first few rows
kmeans_inv.head()

```

KMeans inventory shape: (24, 14)

KMeans inventory columns: ['interval', 'condition', 'pipeline', 'k_final', 'k_sil', 'k_gap', 'silhouette_at_k', 'silhouette_csv', 'gap_csv', 'silhouette_png', 'gap_png', 'umap_plot_png', 'algorithm', 'n_cells']

```

Out[ ]:

```

	interval	condition	pipeline	k_final	k_sil	k_gap	silhouette_at_k
0	1-10w	0	PCA20→KMeans (P1)	20	2	20	-0.087348 /content/dr
1	1-10w	1	PCA20→KMeans (P1)	20	2	20	0.134068 /content/dr
2	11-20w	0	PCA20→KMeans (P1)	20	2	20	-0.098687 /content/dr
3	11-20w	1	PCA20→KMeans (P1)	17	2	17	-0.056533 /content/dr
4	21-30w	0	PCA20→KMeans (P1)	20	2	20	-0.173056 /content/dr

Trying to map cluster csv labels

```

In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

```

```

# Search the folder for all KMeans Label files
all_files = os.listdir(output_dir)

kmeans_label_files = [
    f for f in all_files
    if f.startswith("labels_kmeans_") and f.endswith(".csv")
]

print(f"FOUND {len(kmeans_label_files)} KMeans label files:\n")

for f in kmeans_label_files:
    print(" →", os.path.join(output_dir, f))

# Parse interval, condition, pipeline, umap_dim from filenames
rows = []

for fname in kmeans_label_files:
    fpath = os.path.join(output_dir, fname)
    raw = fname

    # Patterns:
    # P1: labels_kmeans_pipeline1__1-10w_0.csv
    # P2: labels_kmeans_pipeline2_umap3__11-20w_1.csv

    pipeline = None
    umap_dim = None

    if "pipeline1" in raw.lower():
        pipeline = "pipeline1"
    elif "pipeline2" in raw.lower():
        pipeline = "pipeline2"
        # extract UMAP dimension
        m = re.search(r"umap(\d)", raw.lower())
        if m:
            umap_dim = int(m.group(1))

    # interval and condition
    m2 = re.search(r"__(.*)(\d)\.csv", raw)
    if not m2:
        print("⚠ Could not parse interval/condition:", raw)
        continue

    interval = m2.group(1)
    condition = int(m2.group(2))

    rows.append({
        "filename": raw,
        "interval": interval,
        "condition": condition,
        "pipeline": pipeline,
        "umap_dim": umap_dim,
        "path": fpath
    })

# Create table
kmeans_label_lookup = pd.DataFrame(rows)

print("\n\n===== KMEANS LABEL LOOKUP (clean table) =====")

```

```
print(kmeans_label_lookup.shape)
display(kmeans_label_lookup)
```

FOUND 24 KMeans label files:

```
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_0.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_1.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__11-20w_0.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__11-20w_1.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__21-30w_0.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__21-30w_1.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__1-10w_0.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__1-10w_0.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__1-10w_0.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__1-10w_1.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__1-10w_1.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__1-10w_1.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__11-20w_0.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__11-20w_0.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__11-20w_0.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__11-20w_1.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__11-20w_1.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__11-20w_1.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__21-30w_0.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__21-30w_0.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__21-30w_0.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__21-30w_1.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__21-30w_1.csv
→ /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__21-30w_1.csv
```

```
===== KMEANS LABEL LOOKUP (clean table) =====
(24, 6)
```

	filename	interval	condition	pipeline	umap_dim	
0	labels_kmeans_pipeline1__1-10w_0.csv	1-10w	0	pipeline1	NaN	/content,
1	labels_kmeans_pipeline1__1-10w_1.csv	1-10w	1	pipeline1	NaN	/content,
2	labels_kmeans_pipeline1__11-20w_0.csv	11-20w	0	pipeline1	NaN	/content,
3	labels_kmeans_pipeline1__11-20w_1.csv	11-20w	1	pipeline1	NaN	/content,
4	labels_kmeans_pipeline1__21-30w_0.csv	21-30w	0	pipeline1	NaN	/content,
5	labels_kmeans_pipeline1__21-30w_1.csv	21-30w	1	pipeline1	NaN	/content,
6	labels_kmeans_pipeline2_umap2__1-10w_0.csv	1-10w	0	pipeline2	2.0	/content,
7	labels_kmeans_pipeline2_umap3__1-10w_0.csv	1-10w	0	pipeline2	3.0	/content,
8	labels_kmeans_pipeline2_umap5__1-10w_0.csv	1-10w	0	pipeline2	5.0	/content,
9	labels_kmeans_pipeline2_umap2__1-10w_1.csv	1-10w	1	pipeline2	2.0	/content,
10	labels_kmeans_pipeline2_umap3__1-10w_1.csv	1-10w	1	pipeline2	3.0	/content,
11	labels_kmeans_pipeline2_umap5__1-10w_1.csv	1-10w	1	pipeline2	5.0	/content,
12	labels_kmeans_pipeline2_umap2__11-20w_0.csv	11-20w	0	pipeline2	2.0	/content,
13	labels_kmeans_pipeline2_umap3__11-20w_0.csv	11-20w	0	pipeline2	3.0	/content,
14	labels_kmeans_pipeline2_umap5__11-20w_0.csv	11-20w	0	pipeline2	5.0	/content,
15	labels_kmeans_pipeline2_umap2__11-20w_1.csv	11-20w	1	pipeline2	2.0	/content,
16	labels_kmeans_pipeline2_umap3__11-20w_1.csv	11-20w	1	pipeline2	3.0	/content,
17	labels_kmeans_pipeline2_umap5__11-20w_1.csv	11-20w	1	pipeline2	5.0	/content,
18	labels_kmeans_pipeline2_umap2__21-30w_0.csv	21-30w	0	pipeline2	2.0	/content,
19	labels_kmeans_pipeline2_umap3__21-30w_0.csv	21-30w	0	pipeline2	3.0	/content,
20	labels_kmeans_pipeline2_umap5__21-30w_0.csv	21-30w	0	pipeline2	5.0	/content,

	filename	interval	condition	pipeline	umap_dim	
21	labels_kmeans_pipeline2_umap2_21-30w_1.csv	21-30w	1	pipeline2	2.0	/content,
22	labels_kmeans_pipeline2_umap3_21-30w_1.csv	21-30w	1	pipeline2	3.0	/content,
23	labels_kmeans_pipeline2_umap5_21-30w_1.csv	21-30w	1	pipeline2	5.0	/content,

```
In [ ]: # Save
save_path = os.path.join(output_dir, "kmeans_label_lookup.csv")
kmeans_label_lookup.to_csv(save_path, index=False)
print("\nSaved lookup table →", save_path)
```

Saved lookup table → /content/drive/MyDrive/preeclampsia_results/kmeans_label_lookup.csv

Trying to map key columns to kmeans inventory for easy lookup

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

# Helper: print unique column values safely

def show_uniques(df, name, cols):
    print(f"\n===== {name} =====\n")
    for col in cols:
        if col in df.columns:
            print(f"--- {col} ---")
            print(df[col].unique())
            print()
        else:
            print(f"--- {col} NOT FOUND ---\n")

# Load files
inv_path = os.path.join(output_dir, "kmeans_inventory.csv")
lookup_path = os.path.join(output_dir, "kmeans_label_lookup.csv")

inv = pd.read_csv(inv_path)
lookup = pd.read_csv(lookup_path)

print("Files loaded.")
print("Inventory shape:", inv.shape)
print("Lookup shape:", lookup.shape)

# Example: check uniques
show_uniques(inv, "Inventory", inv.columns)
show_uniques(lookup, "Lookup", lookup.columns)
```

Files loaded.

Inventory shape: (24, 14)

Lookup shape: (24, 6)

===== Inventory =====

--- interval ---

['1-10w' '11-20w' '21-30w']

--- condition ---

[0 1]

--- pipeline ---

['PCA20→KMeans (P1)' 'Pipeline2(UMAP2)' 'Pipeline2(UMAP3)'
'Pipeline2(UMAP5)']

--- k_final ---

[20 17 3 14 5 4 7 6 9 10]

--- k_sil ---

[2 4 3 5]

--- k_gap ---

[20 17 3 14 5 4 7 6 9 10]

--- silhouette_at_k ---

-0.08734806	0.13406774	-0.09868692	-0.05653308	-0.1730557	-0.05144696
0.5174209	0.41894928	0.37519312	0.5356804	0.42436874	0.37571728
0.56317914	0.48264682	0.36406147	0.5494382	0.39776972	0.3662484
0.521461	0.4334338	0.39117423	0.44115204	0.37430397	0.34873936]

--- silhouette_csv ---

['/content/drive/MyDrive/preeclampsia_results/p1_1-10w_cond0_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p1_1-10w_cond1_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p1_11-20w_cond0_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p1_11-20w_cond1_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p1_21-30w_cond0_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p1_21-30w_cond1_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_1-10w_cond0_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_1-10w_cond0_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_1-10w_cond0_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_1-10w_cond1_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_1-10w_cond1_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_1-10w_cond1_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_11-20w_cond0_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_11-20w_cond0_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_11-20w_cond0_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_11-20w_cond1_silhouette.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_11-20w_cond1_silhouette.csv']

```

'/content/drive/MyDrive/preeclampsia_results/p2_umap5_11-20w_cond1_silhouette.cs
v'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_21-30w_cond0_silhouette.cs
v'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_21-30w_cond0_silhouette.cs
v'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_21-30w_cond0_silhouette.cs
v'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_21-30w_cond1_silhouette.cs
v'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_21-30w_cond1_silhouette.cs
v'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_21-30w_cond1_silhouette.cs
v']

```

```

--- gap_csv ---

```

```

['/content/drive/MyDrive/preeclampsia_results/p1_1-10w_cond0_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p1_1-10w_cond1_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p1_11-20w_cond0_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p1_11-20w_cond1_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p1_21-30w_cond0_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p1_21-30w_cond1_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_1-10w_cond0_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_1-10w_cond0_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_1-10w_cond0_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_1-10w_cond1_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_1-10w_cond1_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_1-10w_cond1_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_11-20w_cond0_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_11-20w_cond0_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_11-20w_cond0_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_11-20w_cond1_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_11-20w_cond1_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_11-20w_cond1_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_21-30w_cond0_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_21-30w_cond0_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_21-30w_cond0_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_21-30w_cond1_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_21-30w_cond1_gap.csv'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_21-30w_cond1_gap.csv']

```

```

--- silhouette_png ---

```

```

['/content/drive/MyDrive/preeclampsia_results/p1_1-10w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p1_1-10w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p1_11-20w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p1_11-20w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p1_21-30w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p1_21-30w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_1-10w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_1-10w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_1-10w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_1-10w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_1-10w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_1-10w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_11-20w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_11-20w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_11-20w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_11-20w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_11-20w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_11-20w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_21-30w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_21-30w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_21-30w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_21-30w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_21-30w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_21-30w_cond1_silhouette.png']

```

```

'/content/drive/MyDrive/preeclampsia_results/p2_umap2_11-20w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_11-20w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_11-20w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_11-20w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_11-20w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_11-20w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_21-30w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_21-30w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_21-30w_cond0_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_21-30w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_21-30w_cond1_silhouette.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_21-30w_cond1_silhouette.png'
g']

```

--- gap_png ---

```

['/content/drive/MyDrive/preeclampsia_results/p1_1-10w_cond0_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p1_1-10w_cond1_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p1_11-20w_cond0_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p1_11-20w_cond1_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p1_21-30w_cond0_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p1_21-30w_cond1_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_1-10w_cond0_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_1-10w_cond0_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_1-10w_cond0_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_1-10w_cond1_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_1-10w_cond1_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_1-10w_cond1_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_11-20w_cond0_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_11-20w_cond0_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_11-20w_cond0_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_11-20w_cond1_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_11-20w_cond1_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_11-20w_cond1_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_21-30w_cond0_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_21-30w_cond0_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_21-30w_cond0_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap2_21-30w_cond1_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap3_21-30w_cond1_gap.png'
'/content/drive/MyDrive/preeclampsia_results/p2_umap5_21-30w_cond1_gap.png']

```

--- umap_plot_png ---

```

['/content/drive/MyDrive/preeclampsia_results/umap2_plot_kmeans_pipeline1__1-10w_0.png'
'/content/drive/MyDrive/preeclampsia_results/umap2_plot_kmeans_pipeline1__1-10w_1.png'
'/content/drive/MyDrive/preeclampsia_results/umap2_plot_kmeans_pipeline1__11-20w_0.png'
'/content/drive/MyDrive/preeclampsia_results/umap2_plot_kmeans_pipeline1__11-20w_1.png'

```

```

'/content/drive/MyDrive/preeclampsia_results/umap2_plot_kmeans_pipeline1__21-30w_0.png'
'/content/drive/MyDrive/preeclampsia_results/umap2_plot_kmeans_pipeline1__21-30w_1.png'
'/content/drive/MyDrive/preeclampsia_results/umap2_plot_kmeans_pipeline2__1-10w_0.png'
'/content/drive/MyDrive/preeclampsia_results/umap3_plot_kmeans_pipeline2__1-10w_0.png'
'/content/drive/MyDrive/preeclampsia_results/umap5_plot3D_kmeans_pipeline2__1-10w_0.png'
'/content/drive/MyDrive/preeclampsia_results/umap2_plot_kmeans_pipeline2__1-10w_1.png'
'/content/drive/MyDrive/preeclampsia_results/umap3_plot_kmeans_pipeline2__1-10w_1.png'
'/content/drive/MyDrive/preeclampsia_results/umap5_plot3D_kmeans_pipeline2__1-10w_1.png'
'/content/drive/MyDrive/preeclampsia_results/umap2_plot_kmeans_pipeline2__11-20w_0.png'
'/content/drive/MyDrive/preeclampsia_results/umap3_plot_kmeans_pipeline2__11-20w_0.png'
'/content/drive/MyDrive/preeclampsia_results/umap5_plot3D_kmeans_pipeline2__11-20w_0.png'
'/content/drive/MyDrive/preeclampsia_results/umap2_plot_kmeans_pipeline2__11-20w_1.png'
'/content/drive/MyDrive/preeclampsia_results/umap3_plot_kmeans_pipeline2__11-20w_1.png'
'/content/drive/MyDrive/preeclampsia_results/umap5_plot3D_kmeans_pipeline2__11-20w_1.png'
'/content/drive/MyDrive/preeclampsia_results/umap2_plot_kmeans_pipeline2__21-30w_0.png'
'/content/drive/MyDrive/preeclampsia_results/umap3_plot_kmeans_pipeline2__21-30w_0.png'
'/content/drive/MyDrive/preeclampsia_results/umap5_plot3D_kmeans_pipeline2__21-30w_0.png'
'/content/drive/MyDrive/preeclampsia_results/umap2_plot_kmeans_pipeline2__21-30w_1.png'
'/content/drive/MyDrive/preeclampsia_results/umap3_plot_kmeans_pipeline2__21-30w_1.png'
'/content/drive/MyDrive/preeclampsia_results/umap5_plot3D_kmeans_pipeline2__21-30w_1.png']

```

```

--- algorithm ---
['KMeans']

```

```

--- n_cells ---
[nan]

```

```

===== Lookup =====

```

```

--- filename ---
['labels_kmeans_pipeline1__1-10w_0.csv'
'labels_kmeans_pipeline1__1-10w_1.csv'
'labels_kmeans_pipeline1__11-20w_0.csv'
'labels_kmeans_pipeline1__11-20w_1.csv'
'labels_kmeans_pipeline1__21-30w_0.csv'
'labels_kmeans_pipeline1__21-30w_1.csv'
'labels_kmeans_pipeline2_umap2__1-10w_0.csv'
'labels_kmeans_pipeline2_umap3__1-10w_0.csv'
'labels_kmeans_pipeline2_umap5__1-10w_0.csv']

```

```

'labels_kmeans_pipeline2_umap2__1-10w_1.csv'
'labels_kmeans_pipeline2_umap3__1-10w_1.csv'
'labels_kmeans_pipeline2_umap5__1-10w_1.csv'
'labels_kmeans_pipeline2_umap2__11-20w_0.csv'
'labels_kmeans_pipeline2_umap3__11-20w_0.csv'
'labels_kmeans_pipeline2_umap5__11-20w_0.csv'
'labels_kmeans_pipeline2_umap2__11-20w_1.csv'
'labels_kmeans_pipeline2_umap3__11-20w_1.csv'
'labels_kmeans_pipeline2_umap5__11-20w_1.csv'
'labels_kmeans_pipeline2_umap2__21-30w_0.csv'
'labels_kmeans_pipeline2_umap3__21-30w_0.csv'
'labels_kmeans_pipeline2_umap5__21-30w_0.csv'
'labels_kmeans_pipeline2_umap2__21-30w_1.csv'
'labels_kmeans_pipeline2_umap3__21-30w_1.csv'
'labels_kmeans_pipeline2_umap5__21-30w_1.csv']

--- interval ---
['1-10w' '11-20w' '21-30w']

--- condition ---
[0 1]

--- pipeline ---
['pipeline1' 'pipeline2']

--- umap_dim ---
[nan 2. 3. 5.]

--- path ---
['/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__11-20w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__11-20w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__21-30w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__21-30w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__1-10w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__1-10w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__1-10w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__1-10w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__1-10w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__1-10w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__11-20w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__11-20w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__11-20w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__11-20w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__11-20w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__11-20w_1.csv']

```

```

0w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__11-2
0w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__11-2
0w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__21-3
0w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__21-3
0w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__21-3
0w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__21-3
0w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__21-3
0w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__21-3
0w_1.csv']

```

The below cleans and standardizes a K-means inventory file, links each entry to the correct label CSV using a lookup table, checks whether those label files actually exist on disk, reports any missing ones, and saves the updated inventory back to disk.

```

In [ ]: # path
output_dir = "/content/drive/MyDrive/preeclampsia_results"

# Helper: print unique column values safely
def show_uniques(df, name, cols):
    print(f"\n===== {name} =====\n")
    for col in cols:
        if col in df.columns:
            print(f"--- {col} ---")
            print(df[col].unique())
            print()
        else:
            print(f"--- {col} NOT FOUND ---\n")

# Loading two CSVs (inventory and Label Lookup

inv_path = os.path.join(output_dir, "kmeans_inventory.csv")
lookup_path = os.path.join(output_dir, "kmeans_label_lookup.csv")

inv = pd.read_csv(inv_path)
lookup = pd.read_csv(lookup_path)

print("Files loaded.")
print("Inventory shape:", inv.shape)
print("Lookup shape:", lookup.shape)

show_uniques(inv, "Inventory (original)",
              ["interval", "condition", "pipeline"])
show_uniques(lookup, "Lookup (original)",
              ["interval", "condition", "pipeline", "umap_dim"])

# 1 & 2. Standardise 'pipeline' and create 'umap_dim' in inventory

# Keep a copy of the original pipeline strings for parsing UMAP dim
orig_pipeline = inv["pipeline"].astype(str)

```



```

# Extract UMAP dimension from strings like 'Pipeline2(UMAP2)'
# If no UMAP is present (e.g. 'PCA20 - KMeans (P1)'), this will be NaN.
inv["umap_dim"] = (
    orig_pipeline
    .str.extract(r"UMAP(\d+)", expand=False) # captures the digits after 'UMAP'
    .astype(float)                          # convert to numeric (float)
)

# Standardise pipeline names to 'pipeline1' and 'pipeline2'
inv["pipeline"] = np.where(
    orig_pipeline.str.contains("PCA20", na=False), # pipeline 1 condition
    "pipeline1",                                  # then value
    "pipeline2"                                   # else value
)

show_uniques(inv, "Inventory (after pipeline + umap_dim)",
              ["pipeline", "umap_dim", "interval", "condition"])

# 3. Map Lookup 'path' - inventory 'label_csv'
#     via interval, condition, pipeline, umap_dim

# We'll merge on the shared keys
merge_keys = ["interval", "condition", "pipeline", "umap_dim"]

# Take only the needed columns from Lookup
lkup_small = lookup[merge_keys + ["path"]].copy()

# Perform a left merge so we keep all rows from inv
inv = inv.merge(
    lkup_small,
    on=merge_keys,
    how="left"
)

# Rename 'path' from Lookup to 'label_csv' in inventory
inv = inv.rename(columns={"path": "label_csv"})

show_uniques(inv, "Inventory (with label_csv)",
              ["pipeline", "umap_dim", "label_csv"])

# Confirm if each label_csv path exists

def safe_exists(p):
    if isinstance(p, str) and p != "":
        return os.path.exists(p)
    return False

inv["label_exists"] = inv["label_csv"].apply(safe_exists)

print("\nLabel CSV existence summary:")
print(inv["label_exists"].value_counts())

# print rows where label files are missing
missing = inv[~inv["label_exists"]]
if not missing.empty:
    print("\nRows with missing label CSV files:")
    print(missing[["interval", "condition", "pipeline", "umap_dim", "label_csv"]])

# Save updated inventory back to disk

```

```
updated_inv_path = os.path.join(output_dir, "kmeans_inventory.csv")
inv.to_csv(updated_inv_path, index=False)
print("\nUpdated inventory saved to:", updated_inv_path)
```

```

Files loaded.
Inventory shape: (24, 14)
Lookup shape: (24, 6)

===== Inventory (original) =====

--- interval ---
['1-10w' '11-20w' '21-30w']

--- condition ---
[0 1]

--- pipeline ---
['PCA20→KMeans (P1)' 'Pipeline2(UMAP2)' 'Pipeline2(UMAP3)'
 'Pipeline2(UMAP5)']

===== Lookup (original) =====

--- interval ---
['1-10w' '11-20w' '21-30w']

--- condition ---
[0 1]

--- pipeline ---
['pipeline1' 'pipeline2']

--- umap_dim ---
[nan  2.  3.  5.]

===== Inventory (after pipeline + umap_dim) =====

--- pipeline ---
['pipeline1' 'pipeline2']

--- umap_dim ---
[nan  2.  3.  5.]

--- interval ---
['1-10w' '11-20w' '21-30w']

--- condition ---
[0 1]

===== Inventory (with label_csv) =====

--- pipeline ---
['pipeline1' 'pipeline2']

--- umap_dim ---
[nan  2.  3.  5.]

--- label_csv ---
['/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_0.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_1.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_2.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_3.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_4.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_5.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_6.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_7.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_8.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_9.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_10.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_11.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_12.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_13.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_14.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_15.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_16.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_17.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_18.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_19.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_20.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_21.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_22.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_23.csv'
 '/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__1-10w_24.csv']

```

```

'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__11-20w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__11-20w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__21-30w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline1__21-30w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__1-10w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__1-10w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__1-10w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__1-10w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__1-10w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__1-10w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__11-20w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__11-20w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__11-20w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__11-20w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__11-20w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__11-20w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__21-30w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__21-30w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__21-30w_0.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap2__21-30w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap3__21-30w_1.csv'
'/content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_umap5__21-30w_1.csv']

```

Label CSV existence summary:

label_exists

True 24

Name: count, dtype: int64

Updated inventory saved to: /content/drive/MyDrive/preeclampsia_results/kmeans_inventory.csv

```

In [ ]: # Checking to see if changes were made
output_dir = "/content/drive/MyDrive/preeclampsia_results"
updated_inv_path = os.path.join(output_dir, "kmeans_inventory.csv")

# Load and display

```

```
updated_inv = pd.read_csv(updated_inv_path)
updated_inv
```

Out []:

	interval	condition	pipeline	k_final	k_sil	k_gap	silhouette_at_k	
0	1-10w	0	pipeline1	20	2	20	-0.087348	/content/drive/M
1	1-10w	1	pipeline1	20	2	20	0.134068	/content/drive/M
2	11-20w	0	pipeline1	20	2	20	-0.098687	/content/drive/M
3	11-20w	1	pipeline1	17	2	17	-0.056533	/content/drive/M
4	21-30w	0	pipeline1	20	2	20	-0.173056	/content/drive/M
5	21-30w	1	pipeline1	20	2	20	-0.051447	/content/drive/M
6	1-10w	0	pipeline2	3	4	3	0.517421	/content/drive/M
7	1-10w	0	pipeline2	14	3	14	0.418949	/content/drive/M
8	1-10w	0	pipeline2	20	3	20	0.375193	/content/drive/M
9	1-10w	1	pipeline2	5	2	5	0.535680	/content/drive/M
10	1-10w	1	pipeline2	17	2	17	0.424369	/content/drive/M
11	1-10w	1	pipeline2	20	2	20	0.375717	/content/drive/M
12	11-20w	0	pipeline2	4	2	4	0.563179	/content/drive/M
13	11-20w	0	pipeline2	7	3	7	0.482647	/content/drive/M
14	11-20w	0	pipeline2	20	3	20	0.364061	/content/drive/M
15	11-20w	1	pipeline2	6	4	6	0.549438	/content/drive/M
16	11-20w	1	pipeline2	17	5	17	0.397770	/content/drive/M
17	11-20w	1	pipeline2	20	5	20	0.366248	/content/drive/M
18	21-30w	0	pipeline2	3	5	3	0.521461	/content/drive/M
19	21-30w	0	pipeline2	9	5	9	0.433434	/content/drive/M
20	21-30w	0	pipeline2	20	5	20	0.391174	/content/drive/M
21	21-30w	1	pipeline2	10	4	10	0.441152	/content/drive/M
22	21-30w	1	pipeline2	14	4	14	0.374304	/content/drive/M
23	21-30w	1	pipeline2	20	4	20	0.348739	/content/drive/M



HDBSCAN inventory creation

In []:

```
db_tables = []

# List of HDBSCAN metric files and labels for the pipeline column
hdb_files = [
    ("hdbscan_pipeline1_metrics.csv", "HDBSCAN", "PCA20 (P1, base)"),
    ("hdbscan_pipeline1_metrics_hi.csv", "HDBSCAN", "PCA20 (P1, HI-only i"),
    ("hdbscan_pipeline1_metrics_merged.csv", "HDBSCAN", "PCA20 (P1, merged ba"),
    ("hdbscan_pipeline2_metrics.csv", "HDBSCAN", "UMAP (P2, base)"),
```

```

    ("hdbscan_pipeline2_metrics_TUNED.csv",      "HDBSCAN", "UMAP (P2, tuned)"),
]

for fname, algo_name, pipeline_name in hdb_files:
    fpath = os.path.join(output_dir, fname)
    if not os.path.exists(fpath):
        print("HDBSCAN metrics missing ->", fname)
        continue

    df = pd.read_csv(fpath)
    if df.empty:
        continue

    # Normalise some expected column names so we can concatenate safely
    df = df.loc[:, ~df.columns.duplicated()]
    df["interval"] = df["interval"].astype(str)
    df["condition"] = df["condition"].astype(int)
    df["algorithm"] = algo_name
    df["pipeline"] = pipeline_name

    hdb_tables.append(df)

if not hdb_tables:
    print("No HDBSCAN metrics tables found; HDBSCAN inventory not created.")
else:
    hdb_inventory = pd.concat(hdb_tables, ignore_index=True)

    # Ensure we have n_cells from EDA where missing
    hdb_inventory = fill_n_cells(hdb_inventory)

    # Unify plot columns: for P1 we had 'umap_plot_png'; for P2 we had 'plot_png'
    if "plot_png" not in hdb_inventory.columns:
        hdb_inventory["plot_png"] = pd.NA

    if "umap_plot_png" in hdb_inventory.columns:
        mask_p1 = hdb_inventory["pipeline"].str.contains("PCA20", na=False)
        hdb_inventory.loc[mask_p1, "plot_png"] = (
            hdb_inventory.loc[mask_p1, "plot_png"]
            .fillna(hdb_inventory.loc[mask_p1, "umap_plot_png"])
        )

    # Add labels_csv for Pipeline 1 runs (base + HI + merged) if missing
    if "labels_csv" not in hdb_inventory.columns:
        hdb_inventory["labels_csv"] = pd.NA

    mask_p1_any = hdb_inventory["pipeline"].str.contains("PCA20", na=False)
    for idx, row in hdb_inventory[mask_p1_any].iterrows():
        if pd.isna(row["labels_csv"]) or row["labels_csv"] == "":
            tag = f"{row['interval']}_{int(row['condition'])}"

            # HI vs base: use the naming pattern you used in the HDBSCAN code
            if "HI" in row["pipeline"]:
                fname = f"labels_hdbscan_pipeline1_hi_{tag}.csv"
            else:
                fname = f"labels_hdbscan_pipeline1_{tag}.csv"

            hdb_inventory.at[idx, "labels_csv"] = os.path.join(output_dir, fname)

    # compute composite if present or easy to derive
    if "composite" not in hdb_inventory.columns:

```

```

if {"silhouette_nonnoise", "noise_frac"}.issubset(hdb_inventory.columns)
    comp = (
        hdb_inventory["silhouette_nonnoise"].fillna(-1.0)
        - 0.5 * hdb_inventory["noise_frac"].astype(float)
    )
    hdb_inventory["composite"] = comp

# Save final inventory
hdb_inventory_path = os.path.join(output_dir, "hdbscan_inventory.csv")
hdb_inventory.to_csv(hdb_inventory_path, index=False)
print("HDBSCAN inventory saved →", hdb_inventory_path)

hdb_inventory.head()

```

HDBSCAN inventory saved → /content/drive/MyDrive/preeclampsia_results/hdbscan_inventory.csv

Out[]:

	interval	condition	n_cells	min_cluster_size	min_samples	selection_method	n_clusters
--	----------	-----------	---------	------------------	-------------	------------------	------------

0	1-10w	0	15000	10	10	eom	
---	-------	---	-------	----	----	-----	--

1	1-10w	1	5000	10	1	eom	
---	-------	---	------	----	---	-----	--

2	11-20w	0	55000	10	5	eom	
---	--------	---	-------	----	---	-----	--

3	11-20w	1	65000	10	1	eom	
---	--------	---	-------	----	---	-----	--

4	21-30w	0	50000	60	10	eom	
---	--------	---	-------	----	----	-----	--



Phenograph cluster runs inventory creation

```

In [ ]: # Load EDA and prepare interval/condition counts
eda_path = "/content/drive/MyDrive/preeclampsia/eda.csv"
output_dir = "/content/drive/MyDrive/preeclampsia_results"

eda = pd.read_csv(eda_path)

# Make sure age_bin_10w is a categorical with your usual labels
if "age_bin_10w" in eda.columns and not pd.api.types.is_categorical_dtype(eda["age_bin_10w"]):
    cats = ["1-10w", "11-20w", "21-30w", "30+w"]
    eda["age_bin_10w"] = eda["age_bin_10w"].astype(
        pd.CategoricalDtype(categories=cats, ordered=True)
    )

# This gives a MultiIndex series indexed by (interval, condition)
n_cells_series = (
    eda.groupby(["age_bin_10w", "condition"])
        .size()
        .rename("n_cells")
)

```



```

)

# Turn that into a plain Python dict: {(interval_str, cond_int): n_cells}
n_cells_map = {
    (str(interval), int(cond)): int(n)
    for (interval, cond), n in n_cells_series.items()
}

def add_n_cells_from_map(df):
    """
    Add a column n_cells_from_map (and fill n_cells when missing)
    using (interval, condition) → n_cells mapping from EDA.
    """
    # Ensure types are consistent
    df["interval"] = df["interval"].astype(str)
    df["condition"] = df["condition"].astype(int)

    mapped = []
    for _, row in df[["interval", "condition"]].iterrows():
        key = (row["interval"], row["condition"])
        mapped.append(n_cells_map.get(key, np.nan))

    df["n_cells_from_map"] = mapped

    if "n_cells" in df.columns:
        df["n_cells"] = df["n_cells"].fillna(df["n_cells_from_map"])
    else:
        df["n_cells"] = df["n_cells_from_map"]

    return df

```

/tmp/ipython-input-1903743919.py:8: DeprecationWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead

if "age_bin_10w" in eda.columns and not pd.api.types.is_categorical_dtype(eda["age_bin_10w"]):

/tmp/ipython-input-1903743919.py:16: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

eda.groupby(["age_bin_10w", "condition"])

In []: pheno_tables = []

```

# Pipeline 1: PCA-20
p1_metrics_path = os.path.join(output_dir, "phenograph_pipeline1_metrics.csv")
if os.path.exists(p1_metrics_path):
    p1 = pd.read_csv(p1_metrics_path)

    # Ensure expected columns exist
    for col in ["plot_png", "labels_csv"]:
        if col not in p1.columns:
            p1[col] = np.nan

    # These runs are in PCA20 space (no UMAP dimension)
    p1["umap_dim"] = np.nan
    p1["algorithm"] = "PhenoGraph"
    p1["pipeline"] = "PCA20 (P1)"

    pheno_tables.append(p1)

```

```

else:
    print("phenograph_pipeline1_metrics.csv not found - skipping P1.")

# Pipeline 2: UMAP base
p2_metrics_path = os.path.join(output_dir, "phenograph_pipeline2_metrics.csv")
if os.path.exists(p2_metrics_path):
    p2 = pd.read_csv(p2_metrics_path)

    for col in ["plot_png", "labels_csv"]:
        if col not in p2.columns:
            p2[col] = np.nan

    # These already have a 'umap_dim' column (2, 3, or 5)
    p2["algorithm"] = "PhenoGraph"
    p2["pipeline"] = "UMAP (P2, base)"

    pheno_tables.append(p2)
else:
    print("phenograph_pipeline2_metrics.csv not found - skipping P2 base.")

# Pipeline 2: UMAP HI (tuned high-k sweep)
p2_hi_metrics_path = os.path.join(output_dir, "phenograph_pipeline2_metrics_HI.csv")
if os.path.exists(p2_hi_metrics_path):
    p2_hi = pd.read_csv(p2_hi_metrics_path)

    for col in ["plot_png", "labels_csv"]:
        if col not in p2_hi.columns:
            p2_hi[col] = np.nan

    p2_hi["algorithm"] = "PhenoGraph"
    p2_hi["pipeline"] = "UMAP (P2, HI)"

    pheno_tables.append(p2_hi)
else:
    print("phenograph_pipeline2_metrics_HI.csv not found - skipping P2 HI.")

# Combine & add n_cells_from_map
if pheno_tables:
    pheno_inventory = pd.concat(pheno_tables, ignore_index=True)

    # Make sure core columns exist (if any missing, create as NaN)
    needed_cols = [
        "interval", "condition", "n_cells", "k", "umap_dim",
        "silhouette", "modularity_Q", "composite",
        "plot_png", "labels_csv", "algorithm", "pipeline",
    ]
    for col in needed_cols:
        if col not in pheno_inventory.columns:
            pheno_inventory[col] = np.nan

    # Add n_cells_from_map (and backfill n_cells if NaN)
    pheno_inventory = add_n_cells_from_map(pheno_inventory)

    # Save final inventory
    pheno_inventory_path = os.path.join(output_dir, "phenograph_inventory.csv")
    pheno_inventory.to_csv(pheno_inventory_path, index=False)
    print("PhenoGraph inventory saved →", pheno_inventory_path)
else:
    print("No PhenoGraph metrics tables found; PhenoGraph inventory not created.")

```

phenograph_pipeline2_metrics_HI.csv not found - skipping P2 HI.
 PhenoGraph inventory saved → /content/drive/MyDrive/preeclampsia_results/phenograph_inventory.csv

Loading available PhenoGraph metrics CSVs for multiple pipelines, normalizing columns and tag with algorithm/pipeline, combine into one inventory, adding n_cells and a composite score (if missing), save it to disk, and show the head

```
In [ ]: pheno_tables = []

pheno_files = [
    ("phenograph_pipeline1_metrics.csv", "PhenoGraph", "PCA20 (P1)"),
    ("phenograph_pipeline2_metrics.csv", "PhenoGraph", "UMAP (P2, base)"),
    ("phenograph_pipeline2_metrics_HI.csv", "PhenoGraph", "UMAP (P2, HI)"),
]

for fname, algo_name, pipeline_name in pheno_files:
    fpath = os.path.join(output_dir, fname)
    if not os.path.exists(fpath):
        print("PhenoGraph metrics missing →", fname)
        continue

    df = pd.read_csv(fpath)
    if df.empty:
        continue

    df = df.loc[:, ~df.columns.duplicated()]
    df["interval"] = df["interval"].astype(str)
    df["condition"] = df["condition"].astype(int)
    df["algorithm"] = algo_name
    df["pipeline"] = pipeline_name

    pheno_tables.append(df)

if not pheno_tables:
    print("No PhenoGraph metrics tables found; PhenoGraph inventory not created.")
else:
    pheno_inventory = pd.concat(pheno_tables, ignore_index=True)

    # Add n_cells for context
    pheno_inventory = fill_n_cells(pheno_inventory)

    # make sure we have a composite score
    if "composite" not in pheno_inventory.columns:
        if {"silhouette", "modularity_Q"}.issubset(pheno_inventory.columns):
            # same weighting you used before
            sil = pheno_inventory["silhouette"].fillna(-1.0).astype(float)
            Q = pheno_inventory["modularity_Q"].astype(float)
            qn = (Q - (-0.1)) / (1.0 - (-0.1))
            qn = qn.clip(0.0, 1.0)
            pheno_inventory["composite"] = 0.7 * sil + 0.3 * qn

    pheno_inventory_path = os.path.join(output_dir, "phenograph_inventory.csv")
    pheno_inventory.to_csv(pheno_inventory_path, index=False)
    print("PhenoGraph inventory saved →", pheno_inventory_path)

pheno_inventory.head()
```

PhenoGraph metrics missing → phenograph_pipeline2_metrics_HI.csv

PhenoGraph inventory saved → /content/drive/MyDrive/preeclampsia_results/phenograph_inventory.csv

Out []:

	interval	condition	n_cells	k	silhouette	modularity_Q	composite	
0	1–10w	0	15000	15	0.065217	0.860775	0.307682	
1	1–10w	0	15000	30	0.071296	0.841977	0.306810	
2	1–10w	0	15000	50	0.082086	0.835297	0.312541	
3	1–10w	0	15000	75	0.095394	0.828500	0.320003	/content/drive/MyDrive/preeclampsia_results/phenograph_inventory.csv
4	1–10w	1	5000	15	0.110356	0.870026	0.341802	/content/drive/MyDrive/preeclampsia_results/phenograph_inventory.csv

I couldn't find some file labels for saved cluster png so I had to take extra measures below

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"
inv_path = os.path.join(output_dir, "phenograph_inventory.csv")

# Load PhenoGraph inventory
ph_inv = pd.read_csv(inv_path)
print("PhenoGraph inventory:", ph_inv.shape)
print("Columns:", list(ph_inv.columns))

# Make sure these exist; if not, create
if "umap_dim" not in ph_inv.columns:
    ph_inv["umap_dim"] = np.nan
if "k" not in ph_inv.columns:
    raise KeyError("Inventory must have a 'k' column for PhenoGraph runs.")

# 2. Find all PhenoGraph label files
# (labels_phenograph_*.csv)

label_files = []
for fname in os.listdir(output_dir):
    if fname.startswith("labels_phenograph_") and fname.endswith(".csv"):
        label_files.append(os.path.join(output_dir, fname))

label_files = sorted(label_files)
print(f"Found {len(label_files)} PhenoGraph label files:")
for p in label_files[:10]:
    print("→", os.path.basename(p))
if len(label_files) > 10:
    print("...")

# 3. Parse filenames into a small lookup table
# We want: interval, condition, pipeline_group (P1/P2),
#          umap_dim (NaN for PCA20), k, labels_csv
```

```

def parse_pheno_label_path(path):
    """
    Parse a path like:
    labels_phenograph_pipeline1_pca20_k75__1-10w_0.csv
    labels_phenograph_pipeline2_umap3_k75__21-30w_1.csv
    labels_phenograph_pipeline2_umap2_k100_HI__1-10w_0.csv
    into metadata.
    """
    fname = os.path.basename(path)

    m = re.match(r"labels_phenograph_(.+)__([^_]+)(\d+)\.csv", fname)
    if not m:
        # Unexpected pattern → skip
        return None

    body = m.group(1) # e.g. "pipeline2_umap3_k75", "pipeline2_umap2_k100_H
    interval = m.group(2) # e.g. "1-10w"
    cond = int(m.group(3))

    # Pipeline group: P1 (PCA20) vs P2 (UMAP)
    if "pipeline1" in body:
        pipeline_group = "P1"
        umap_dim = np.nan
    elif "pipeline2" in body:
        pipeline_group = "P2"
        # infer UMAP dim
        if "umap2" in body:
            umap_dim = 2
        elif "umap3" in body:
            umap_dim = 3
        elif "umap5" in body:
            umap_dim = 5
        else:
            umap_dim = np.nan
    else:
        pipeline_group = "UNKNOWN"
        umap_dim = np.nan

    # Extract k (first "kNN" we find)
    k_match = re.search(r"k(\d+)", body)
    if k_match:
        k = int(k_match.group(1))
    else:
        k = np.nan

    return {
        "interval": interval,
        "condition": cond,
        "pipeline_group": pipeline_group,
        "umap_dim": umap_dim,
        "k": k,
        "labels_csv_file": path,
    }

label_meta = []
for path in label_files:
    info = parse_pheno_label_path(path)
    if info is not None:

```

```

        label_meta.append(info)

label_df = pd.DataFrame(label_meta)
print("\nParsed label metadata:", label_df.shape)
display(label_df.head(10))

# Build a matching key in BOTH tables
# (interval, condition, pipeline_group, umap_dim, k)

def norm_interval(s):
    """Normalise interval text (dash variants, whitespace)."""
    if pd.isna(s):
        return ""
    s = str(s).strip()
    # make both '1-10w' and '1-10w' the same
    s = s.replace("-", "-")
    return s

# derive pipeline_group in inventory from 'pipeline' text
def pipeline_to_group(p):
    p = str(p)
    if "PCA20" in p or "(P1" in p:
        return "P1"
    elif "P2" in p or "UMAP" in p:
        return "P2"
    else:
        return "UNKNOWN"

ph_inv["interval_norm"] = ph_inv["interval"].apply(norm_interval)
ph_inv["condition_int"] = ph_inv["condition"].astype(int)
ph_inv["pipeline_group"] = ph_inv["pipeline"].apply(pipeline_to_group)
ph_inv["umap_dim_int"] = ph_inv["umap_dim"].fillna(-1).astype(int)
ph_inv["k_int"] = ph_inv["k"].astype(int)

label_df["interval_norm"] = label_df["interval"].apply(norm_interval)
label_df["condition_int"] = label_df["condition"].astype(int)
label_df["umap_dim_int"] = label_df["umap_dim"].fillna(-1).astype(int)
label_df["k_int"] = label_df["k"].astype(int)

ph_inv["match_key"] = list(zip(
    ph_inv["interval_norm"],
    ph_inv["condition_int"],
    ph_inv["pipeline_group"],
    ph_inv["umap_dim_int"],
    ph_inv["k_int"],
))
label_df["match_key"] = list(zip(
    label_df["interval_norm"],
    label_df["condition_int"],
    label_df["pipeline_group"],
    label_df["umap_dim_int"],
    label_df["k_int"],
))

# Create a Lookup: one Label per match_key
label_lookup = label_df[["match_key", "labels_csv_file"]].copy()
# Drop duplicates if any (keep first)
label_lookup = label_lookup.drop_duplicates(subset=["match_key"])

```

```

print("\nLabel lookup:", label_lookup.shape)
display(label_lookup.head(10))

# 5. Merge labels into inventory
ph_inv = ph_inv.merge(label_lookup, on="match_key", how="left")

# Make/overwrite final labels_csv column
if "labels_csv" in ph_inv.columns:
    ph_inv["labels_csv"] = ph_inv["labels_csv"].where(
        ph_inv["labels_csv"].notna(), ph_inv["labels_csv_file"]
    )
else:
    ph_inv["labels_csv"] = ph_inv["labels_csv_file"]

# sanity check
ph_inv["labels_found"] = ph_inv["labels_csv"].notna()

print("\nAfter attaching labels_csv:")
display(ph_inv[[
    "interval", "condition", "pipeline", "umap_dim", "k", "labels_csv", "labels_
]].head(30))

print("Total rows with labels:", ph_inv["labels_found"].sum(), "/", len(ph_inv))

# Clean up helper columns and SAVE

ph_inv_clean = ph_inv.drop(columns=[
    "interval_norm", "condition_int", "pipeline_group",
    "umap_dim_int", "k_int", "match_key", "labels_csv_file"
], errors="ignore")

# keep labels_found if you like; otherwise drop it too
# ph_inv_clean = ph_inv_clean.drop(columns=["labels_found"], errors="ignore")

ph_inv_clean.to_csv(inv_path, index=False)
print("\nUpdated PhenoGraph inventory saved →", inv_path)

```

PhenoGraph inventory: (96, 13)

Columns: ['interval', 'condition', 'n_cells', 'k', 'silhouette', 'modularity_Q', 'composite', 'plot_png', 'labels_csv', 'umap_dim', 'algorithm', 'pipeline', 'n_cells_from_map']

Found 40 PhenoGraph label files:

- labels_phenograph_pipeline1_pca20_k15__1-10w_1.csv
- labels_phenograph_pipeline1_pca20_k15__21-30w_0.csv
- labels_phenograph_pipeline1_pca20_k50__11-20w_0.csv
- labels_phenograph_pipeline1_pca20_k50__21-30w_1.csv
- labels_phenograph_pipeline1_pca20_k75__11-20w_1.csv
- labels_phenograph_pipeline1_pca20_k75__1-10w_0.csv
- labels_phenograph_pipeline2_umap2_k100_HI__1-10w_0.csv
- labels_phenograph_pipeline2_umap2_k100_HI__21-30w_0.csv
- labels_phenograph_pipeline2_umap2_k120_HI__1-10w_1.csv
- labels_phenograph_pipeline2_umap2_k120_HI__21-30w_1.csv
- ...

Parsed label metadata: (40, 6)

	interval	condition	pipeline_group	umap_dim	k	lab
0	1–10w	1	P1	NaN	15	/content/drive/MyDrive/preeclampsia
1	21–30w	0	P1	NaN	15	/content/drive/MyDrive/preeclampsia
2	11–20w	0	P1	NaN	50	/content/drive/MyDrive/preeclampsia
3	21–30w	1	P1	NaN	50	/content/drive/MyDrive/preeclampsia
4	11–20w	1	P1	NaN	75	/content/drive/MyDrive/preeclampsia
5	1–10w	0	P1	NaN	75	/content/drive/MyDrive/preeclampsia
6	1–10w	0	P2	2.0	100	/content/drive/MyDrive/preeclampsia
7	21–30w	0	P2	2.0	100	/content/drive/MyDrive/preeclampsia
8	1–10w	1	P2	2.0	120	/content/drive/MyDrive/preeclampsia
9	21–30w	1	P2	2.0	120	/content/drive/MyDrive/preeclampsia



Label lookup: (40, 2)

	match_key	labels_csv_file
0	(1-10w, 1, P1, -1, 15)	/content/drive/MyDrive/preeclampsia_results/la...
1	(21-30w, 0, P1, -1, 15)	/content/drive/MyDrive/preeclampsia_results/la...
2	(11-20w, 0, P1, -1, 50)	/content/drive/MyDrive/preeclampsia_results/la...
3	(21-30w, 1, P1, -1, 50)	/content/drive/MyDrive/preeclampsia_results/la...
4	(11-20w, 1, P1, -1, 75)	/content/drive/MyDrive/preeclampsia_results/la...
5	(1-10w, 0, P1, -1, 75)	/content/drive/MyDrive/preeclampsia_results/la...
6	(1-10w, 0, P2, 2, 100)	/content/drive/MyDrive/preeclampsia_results/la...
7	(21-30w, 0, P2, 2, 100)	/content/drive/MyDrive/preeclampsia_results/la...
8	(1-10w, 1, P2, 2, 120)	/content/drive/MyDrive/preeclampsia_results/la...
9	(21-30w, 1, P2, 2, 120)	/content/drive/MyDrive/preeclampsia_results/la...

After attaching labels_csv:

	interval	condition	pipeline	umap_dim	k	labels
0	1–10w	0	PCA20 (P1)	NaN	15	
1	1–10w	0	PCA20 (P1)	NaN	30	
2	1–10w	0	PCA20 (P1)	NaN	50	
3	1–10w	0	PCA20 (P1)	NaN	75	/content/drive/MyDrive/preeclampsia_results
4	1–10w	1	PCA20 (P1)	NaN	15	/content/drive/MyDrive/preeclampsia_results
5	1–10w	1	PCA20 (P1)	NaN	30	
6	1–10w	1	PCA20 (P1)	NaN	50	
7	1–10w	1	PCA20 (P1)	NaN	75	
8	11–20w	0	PCA20 (P1)	NaN	15	
9	11–20w	0	PCA20 (P1)	NaN	30	
10	11–20w	0	PCA20 (P1)	NaN	50	/content/drive/MyDrive/preeclampsia_results
11	11–20w	0	PCA20 (P1)	NaN	75	
12	11–20w	1	PCA20 (P1)	NaN	15	
13	11–20w	1	PCA20 (P1)	NaN	30	
14	11–20w	1	PCA20 (P1)	NaN	50	
15	11–20w	1	PCA20 (P1)	NaN	75	/content/drive/MyDrive/preeclampsia_results
16	21–30w	0	PCA20 (P1)	NaN	15	/content/drive/MyDrive/preeclampsia_results
17	21–30w	0	PCA20 (P1)	NaN	30	
18	21–30w	0	PCA20 (P1)	NaN	50	
19	21–30w	0	PCA20 (P1)	NaN	75	
20	21–30w	1	PCA20 (P1)	NaN	15	

	interval	condition	pipeline	umap_dim	k	labels
21	21–30w	1	PCA20 (P1)	NaN	30	
22	21–30w	1	PCA20 (P1)	NaN	50	/content/drive/MyDrive/preeclampsia_results
23	21–30w	1	PCA20 (P1)	NaN	75	
24	1–10w	0	UMAP (P2, base)	2.0	15	
25	1–10w	0	UMAP (P2, base)	2.0	30	/content/drive/MyDrive/preeclampsia_results
26	1–10w	0	UMAP (P2, base)	2.0	50	
27	1–10w	0	UMAP (P2, base)	2.0	75	
28	1–10w	0	UMAP (P2, base)	3.0	15	
29	1–10w	0	UMAP (P2, base)	3.0	30	

Total rows with labels: 24 / 96

Updated PhenoGraph inventory saved → /content/drive/MyDrive/preeclampsia_results/phenograph_inventory.csv

I then later realized that the reason why some on my hyperparameter tuning cluster pngs were missing was because they did not perform better than the initial run and therefore were not saved

Evaluation using SH, DB, CH (Internal Metrics)

K means

```
In [ ]: # uploading kmeans inventory
output_dir = "/content/drive/MyDrive/preeclampsia_results"

inv_path = os.path.join(output_dir, "kmeans_inventory.csv")
kminv = pd.read_csv(inv_path)

print("KMeans inventory:", kminv.shape)
display(kminv.head())
```

KMeans inventory: (24, 17)

	interval	condition	pipeline	k_final	k_sil	k_gap	silhouette_at_k	
0	1-10w	0	pipeline1	20	2	20	-0.087348	/content/drive/MyDr
1	1-10w	1	pipeline1	20	2	20	0.134068	/content/drive/MyDr
2	11-20w	0	pipeline1	20	2	20	-0.098687	/content/drive/MyDr
3	11-20w	1	pipeline1	17	2	17	-0.056533	/content/drive/MyDr
4	21-30w	0	pipeline1	20	2	20	-0.173056	/content/drive/MyDr

```
In [ ]: # Helper: Load Labels Series from CSV (single column)
def load_labels_series(csv_path):
    df = pd.read_csv(csv_path)
    s = df.iloc[:, -1]
    return s.to_numpy()

# Helper: infer UMAP dim from PNG filename if needed
def infer_umap_dim_from_png(path_str):
    base = os.path.basename(str(path_str))
    if "umap2" in base:
        return 2
    if "umap3" in base:
        return 3
    if "umap5" in base:
        return 5
    return np.nan

# Core: compute SH, DBI, CH (for non-noise KMeans all points are used)
def compute_cluster_metrics(X, labels):
    X = np.asarray(X)
    labels = np.asarray(labels)

    n = len(labels)
    uniq = np.unique(labels)

    if n < 3 or len(uniq) < 2:
        return np.nan, np.nan, np.nan

    try:
        sil = silhouette_score(X, labels)
    except Exception:
        sil = np.nan

    try:
        db = davies_bouldin_score(X, labels)
    except Exception:
        db = np.nan

    try:
        ch = calinski_harabasz_score(X, labels)
    except Exception:
        ch = np.nan

    return float(sil), float(db), float(ch)
```

Computing classical cluster evaluation SH, DB, CH

```

In [ ]: # Computing evaluation metrics
metrics_rows = []

for idx, r in kminv.iterrows():
    interval = r["interval"]
    cond = int(r["condition"])
    pipeline = r["pipeline"] # "pipeline1" or "pipeline2"
    algo = r["algorithm"] # "KMeans"
    k_final = int(r["k_final"])
    umap_png = r["umap_plot_png"]
    umap_dim = r.get("umap_dim", np.nan)
    label_csv = r.get("label_csv", np.nan)

    tag = f"{interval}_{cond}"
    print(f"\n[Run {idx}] {interval}, cond={cond}, pipeline={pipeline}, k={k_fin

# Decide which embedding & labels to use based on new 'pipeline' + 'umap_dim'
if pipeline == "pipeline1":
    # PCA20 pipeline
    space = "pca20"
    umap_dim = np.nan # override to be explicit
    X_path = os.path.join(output_dir, f"pca20_{tag}.npy")

    if pd.notna(label_csv):
        labels_path = label_csv
    else:
        labels_path = os.path.join(
            output_dir, f"labels_kmeans_pipeline1_{tag}.csv"
        )

elif pipeline == "pipeline2":
    # UMAP pipeline - must have a valid umap_dim
    if np.isnan(umap_dim):
        print(" -> umap_dim is NaN for pipeline2, skipping.")
        continue

    d = int(umap_dim)
    space = f"umap{d}"
    X_path = os.path.join(output_dir, f"umap{d}_pipeline2_{tag}.npy")

    if pd.notna(label_csv):
        labels_path = label_csv
    else:
        labels_path = os.path.join(
            output_dir, f"labels_kmeans_pipeline2_umap{d}_{tag}.csv"
        )

else:
    print(f" -> Unknown pipeline '{pipeline}', skipping.")
    continue

# Sanity: check files exist
if not os.path.exists(X_path):
    print(" -> missing embedding:", X_path)
    continue
if not os.path.exists(labels_path):
    print(" -> missing labels:", labels_path)
    continue

```

```

# Load and check lengths
X = np.load(X_path)
labels = load_labels_series(labels_path)

if len(X) != len(labels):
    print(f" -> length mismatch X({len(X)}) vs labels({len(labels)}), skipping")
    continue

n_cells = len(labels)
n_clusters = len(np.unique(labels))

sil, db, ch = compute_cluster_metrics(X, labels)
print(f" n={n_cells}, clusters={n_clusters}, SH={sil:.3f}, DB={db:.3f}, CH=

metrics_rows.append({
    "interval": interval,
    "condition": cond,
    "algorithm": algo,
    "pipeline": pipeline,
    "space": space,
    "umap_dim": umap_dim,
    "k_final": k_final,
    "n_cells": n_cells,
    "n_clusters": n_clusters,
    "silhouette": sil,
    "davies_bouldin": db,
    "calinski_harabasz": ch,
    "umap_plot_png": umap_png,
    "label_csv": labels_path,
})

kmeans_eval = pd.DataFrame(metrics_rows)
eval_path = os.path.join(output_dir, "kmeans_eval_metrics.csv")
kmeans_eval.to_csv(eval_path, index=False)

print("\nSaved KMeans eval metrics ->", eval_path)
display(kmeans_eval.head(10))

```

[Run 0] 1-10w, cond=0, pipeline=pipeline1, k=20
n=15000, clusters=20, SH=0.155, DB=1.686, CH=1114.1

[Run 1] 1-10w, cond=1, pipeline=pipeline1, k=20
n=5000, clusters=20, SH=0.134, DB=1.898, CH=386.9

[Run 2] 11-20w, cond=0, pipeline=pipeline1, k=20
n=55000, clusters=20, SH=0.131, DB=1.867, CH=4148.6

[Run 3] 11-20w, cond=1, pipeline=pipeline1, k=17
n=65000, clusters=17, SH=0.147, DB=1.769, CH=5741.4

[Run 4] 21-30w, cond=0, pipeline=pipeline1, k=20
n=50000, clusters=20, SH=0.130, DB=1.896, CH=3965.7

[Run 5] 21-30w, cond=1, pipeline=pipeline1, k=20
n=40000, clusters=20, SH=0.129, DB=1.798, CH=3090.8

[Run 6] 1-10w, cond=0, pipeline=pipeline2, k=3
n=15000, clusters=3, SH=0.516, DB=0.757, CH=17262.9

[Run 7] 1-10w, cond=0, pipeline=pipeline2, k=14
n=15000, clusters=14, SH=0.420, DB=0.795, CH=28589.4

[Run 8] 1-10w, cond=0, pipeline=pipeline2, k=20
n=15000, clusters=20, SH=0.376, DB=0.984, CH=20404.5

[Run 9] 1-10w, cond=1, pipeline=pipeline2, k=5
n=5000, clusters=5, SH=0.536, DB=0.648, CH=11532.6

[Run 10] 1-10w, cond=1, pipeline=pipeline2, k=17
n=5000, clusters=17, SH=0.424, DB=0.800, CH=11073.0

[Run 11] 1-10w, cond=1, pipeline=pipeline2, k=20
n=5000, clusters=20, SH=0.376, DB=0.988, CH=9798.2

[Run 12] 11-20w, cond=0, pipeline=pipeline2, k=4
n=55000, clusters=4, SH=0.561, DB=0.662, CH=159847.3

[Run 13] 11-20w, cond=0, pipeline=pipeline2, k=7
n=55000, clusters=7, SH=0.479, DB=0.876, CH=106088.0

[Run 14] 11-20w, cond=0, pipeline=pipeline2, k=20
n=55000, clusters=20, SH=0.372, DB=0.924, CH=97557.6

[Run 15] 11-20w, cond=1, pipeline=pipeline2, k=6
n=65000, clusters=6, SH=0.549, DB=0.660, CH=157656.2

[Run 16] 11-20w, cond=1, pipeline=pipeline2, k=17
n=65000, clusters=17, SH=0.403, DB=0.882, CH=129431.2

[Run 17] 11-20w, cond=1, pipeline=pipeline2, k=20
n=65000, clusters=20, SH=0.367, DB=0.969, CH=113716.8

[Run 18] 21-30w, cond=0, pipeline=pipeline2, k=3
n=50000, clusters=3, SH=0.522, DB=0.785, CH=86935.5

[Run 19] 21-30w, cond=0, pipeline=pipeline2, k=9
n=50000, clusters=9, SH=0.429, DB=0.844, CH=108301.4

[Run 20] 21-30w, cond=0, pipeline=pipeline2, k=20
n=50000, clusters=20, SH=0.390, DB=0.900, CH=89717.2

[Run 21] 21-30w, cond=1, pipeline=pipeline2, k=10
n=40000, clusters=10, SH=0.447, DB=0.779, CH=107452.4

[Run 22] 21-30w, cond=1, pipeline=pipeline2, k=14
n=40000, clusters=14, SH=0.374, DB=0.951, CH=85754.8

[Run 23] 21-30w, cond=1, pipeline=pipeline2, k=20
n=40000, clusters=20, SH=0.349, DB=1.028, CH=76019.9

Saved KMeans eval metrics → /content/drive/MyDrive/preeclampsia_results/kmeans_eval_metrics.csv

	interval	condition	algorithm	pipeline	space	umap_dim	k_final	n_cells	n_clusters
0	1-10w	0	KMeans	pipeline1	pca20	NaN	20	15000	20
1	1-10w	1	KMeans	pipeline1	pca20	NaN	20	5000	20
2	11-20w	0	KMeans	pipeline1	pca20	NaN	20	55000	20
3	11-20w	1	KMeans	pipeline1	pca20	NaN	17	65000	17
4	21-30w	0	KMeans	pipeline1	pca20	NaN	20	50000	20
5	21-30w	1	KMeans	pipeline1	pca20	NaN	20	40000	20
6	1-10w	0	KMeans	pipeline2	umap2	2.0	3	15000	3
7	1-10w	0	KMeans	pipeline2	umap3	3.0	14	15000	14
8	1-10w	0	KMeans	pipeline2	umap5	5.0	20	15000	20
9	1-10w	1	KMeans	pipeline2	umap2	2.0	5	5000	5

Computing composite score and adding best per interval

Calculating composition score by : $(sh_z - db_z + 0.5 \cdot ch_z)$

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"
eval_path = os.path.join(output_dir, "kmeans_eval_metrics.csv")

km = pd.read_csv(eval_path)
print("KMeans eval metrics:", km.shape)

# Keep only rows with all three metrics present
needed = ["silhouette", "davies_bouldin", "calinski_harabasz"]
km = km.dropna(subset=needed)
print("After dropping missing SH/DB/CH:", km.shape)

def _z(x: pd.Series) -> pd.Series:
    x = x.astype(float)
    m = x.mean()
    s = x.std(ddof=0)
    if s == 0 or np.isnan(s):
        return x * 0.0 # all zeros
    return (x - m) / s
```

```

# 1) z-scores for SH, DB, CH (CH log-transformed)
km["sh_z"] = _z(km["silhouette"])

km["db_z_raw"] = km["davies_bouldin"].astype(float)
km["db_z"] = _z(km["db_z_raw"])

km["ch_log"] = np.log1p(km["calinski_harabasz"].astype(float))
km["ch_z"] = _z(km["ch_log"])

# noise term is 0 for KMeans, but keep for consistency
km["noise_frac"] = 0.0
km["noise_z"] = 0.0

# 2) Composite evaluation score (purely metric-based)
km["composite_eval"] = (
    km["sh_z"]          # silhouette ↑
    - km["db_z"]         # DBI ↓
    + 0.5 * km["ch_z"]   # CH ↑
    - 0.5 * km["noise_z"] # noise ↓ (0 for KMeans)
)

kmeans_scored_path = os.path.join(output_dir, "kmeans_eval_scored.csv")
km.to_csv(kmeans_scored_path, index=False)
print("Saved KMeans eval-scored table →", kmeans_scored_path)
display(km)

```

KMeans eval metrics: (24, 14)

After dropping missing SH/DB/CH: (24, 14)

Saved KMeans eval-scored table → /content/drive/MyDrive/preeclampsia_results/kmeans_eval_scored.csv

	interval	condition	algorithm	pipeline	space	umap_dim	k_final	n_cells	n_cluste
0	1-10w	0	KMeans	pipeline1	pca20	NaN	20	15000	2
1	1-10w	1	KMeans	pipeline1	pca20	NaN	20	5000	2
2	11-20w	0	KMeans	pipeline1	pca20	NaN	20	55000	2
3	11-20w	1	KMeans	pipeline1	pca20	NaN	17	65000	2
4	21-30w	0	KMeans	pipeline1	pca20	NaN	20	50000	2
5	21-30w	1	KMeans	pipeline1	pca20	NaN	20	40000	2
6	1-10w	0	KMeans	pipeline2	umap2	2.0	3	15000	2
7	1-10w	0	KMeans	pipeline2	umap3	3.0	14	15000	2
8	1-10w	0	KMeans	pipeline2	umap5	5.0	20	15000	2
9	1-10w	1	KMeans	pipeline2	umap2	2.0	5	5000	2
10	1-10w	1	KMeans	pipeline2	umap3	3.0	17	5000	2
11	1-10w	1	KMeans	pipeline2	umap5	5.0	20	5000	2
12	11-20w	0	KMeans	pipeline2	umap2	2.0	4	55000	2
13	11-20w	0	KMeans	pipeline2	umap3	3.0	7	55000	2
14	11-20w	0	KMeans	pipeline2	umap5	5.0	20	55000	2
15	11-20w	1	KMeans	pipeline2	umap2	2.0	6	65000	2
16	11-20w	1	KMeans	pipeline2	umap3	3.0	17	65000	2
17	11-20w	1	KMeans	pipeline2	umap5	5.0	20	65000	2
18	21-30w	0	KMeans	pipeline2	umap2	2.0	3	50000	2
19	21-30w	0	KMeans	pipeline2	umap3	3.0	9	50000	2
20	21-30w	0	KMeans	pipeline2	umap5	5.0	20	50000	2
21	21-30w	1	KMeans	pipeline2	umap2	2.0	10	40000	2
22	21-30w	1	KMeans	pipeline2	umap3	3.0	14	40000	2
23	21-30w	1	KMeans	pipeline2	umap5	5.0	20	40000	2

24 rows × 22 columns



Selecting the best per interval for each pipeline

```
In [ ]: # Best per (interval, condition) across both pipelines (6 rows total)
best_rows = []

for interval in sorted(km["interval"].unique()):
    for cond in sorted(km["condition"].unique()):
        sub = km[(km["interval"] == interval) & (km["condition"] == cond)]
        if sub.empty:
            continue
```

```

# sort by composite_eval desc
sub_sorted = sub.sort_values(by="composite_eval", ascending=False)

best = sub_sorted.iloc[0]
best_rows.append(best)

kmeans_best_per_interval = pd.DataFrame(best_rows).reset_index(drop=True)

best_path = os.path.join(output_dir, "kmeans_best_per_interval_across_pipelines.
kmeans_best_per_interval.to_csv(best_path, index=False)

print("Saved KMeans BEST per (interval, condition) →", best_path)
display(kmeans_best_per_interval[[
    "interval", "condition", "pipeline", "space", "umap_dim",
    "k_final", "n_cells", "n_clusters",
    "silhouette", "davies_bouldin", "calinski_harabasz",
    "composite_eval", "umap_plot_png", "umap_dim", "label_csv"
]])

```

Saved KMeans BEST per (interval, condition) → /content/drive/MyDrive/preeclampsia_results/kmeans_best_per_interval_across_pipelines.csv

	interval	condition	pipeline	space	umap_dim	k_final	n_cells	n_clusters	silhouette
0	1-10w	0	pipeline2	umap2	2.0	3	15000	3	0.516154
1	1-10w	1	pipeline2	umap2	2.0	5	5000	5	0.535680
2	11-20w	0	pipeline2	umap2	2.0	4	55000	4	0.561167
3	11-20w	1	pipeline2	umap2	2.0	6	65000	6	0.548862
4	21-30w	0	pipeline2	umap2	2.0	3	50000	3	0.521679
5	21-30w	1	pipeline2	umap2	2.0	10	40000	10	0.447130

Selecting the best performing per interval for pipeline 1

```

In [ ]: # Load the evaluation-scored KMeans table
scored_path = os.path.join(output_dir, "kmeans_eval_scored.csv")
km = pd.read_csv(scored_path)

print("Loaded KMeans scored table:", km.shape)

# Filter: keep only Pipeline 1 rows
km_p1 = km[km["pipeline"] == "pipeline1"]
print("Pipeline 1 KMeans rows:", km_p1.shape)

# Step 2 – Best for each (interval, condition)
best_rows = []

for interval in sorted(km_p1["interval"].unique()):
    for cond in sorted(km_p1["condition"].unique()):
        sub = km_p1[(km_p1["interval"] == interval) &
                    (km_p1["condition"] == cond)]

        if sub.empty:
            continue

```

```

# Sort by composite_eval descending
sub_sorted = sub.sort_values(by="composite_eval", ascending=False)
best = sub_sorted.iloc[0]
best_rows.append(best)

km_best_p1 = pd.DataFrame(best_rows).reset_index(drop=True)

# Save & display in clean table form
p1_best_path = os.path.join(output_dir, "kmeans_best_pipeline1_only.csv")
km_best_p1.to_csv(p1_best_path, index=False)

print("\nSaved best Pipeline-1 KMeans runs →", p1_best_path)

display(
    km_best_p1[
        [
            "interval", "condition", "pipeline", "space", "k_final",
            "n_cells", "n_clusters",
            "silhouette", "davies_bouldin", "calinski_harabasz",
            "composite_eval",
            "umap_plot_png", "umap_dim", "label_csv"
        ]
    ]
)

```

Loaded KMeans scored table: (24, 22)

Pipeline 1 KMeans rows: (6, 22)

Saved best Pipeline-1 KMeans runs → /content/drive/MyDrive/preeclampsia_results/kmeans_best_pipeline1_only.csv

	interval	condition	pipeline	space	k_final	n_cells	n_clusters	silhouette	davies_bou
0	1-10w	0	pipeline1	pca20	20	15000	20	0.154929	1.68
1	1-10w	1	pipeline1	pca20	20	5000	20	0.134068	1.89
2	11-20w	0	pipeline1	pca20	20	55000	20	0.130542	1.86
3	11-20w	1	pipeline1	pca20	17	65000	17	0.147183	1.76
4	21-30w	0	pipeline1	pca20	20	50000	20	0.130226	1.89
5	21-30w	1	pipeline1	pca20	20	40000	20	0.128672	1.79

```

In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

all_metrics_path = os.path.join(output_dir, "kmeans_eval_metrics.csv")
eval_scored_path = os.path.join(output_dir, "kmeans_eval_scored.csv")

print("All-metrics path:", all_metrics_path)
print("Eval-scored path:", eval_scored_path)

km_all = pd.read_csv(eval_scored_path)
print("kmeans_all_metrics_SH_DB_CH shape:", km_all.shape)
display(km_all.head())

```

All-metrics path: /content/drive/MyDrive/preeclampsia_results/kmeans_all_metrics_SH_DB_CH.csv

Eval-scored path: /content/drive/MyDrive/preeclampsia_results/kmeans_eval_scored.csv

kmeans_all_metrics_SH_DB_CH shape: (24, 20)

	interval	condition	pipeline	algorithm	space	umap_dim	k	n_cells	n_clusters	sil
0	1-10w	0	pipeline1	kmeans	pca20	NaN	20	15000	20	0
1	1-10w	1	pipeline1	kmeans	pca20	NaN	20	5000	20	0
2	11-20w	0	pipeline1	kmeans	pca20	NaN	20	55000	20	0
3	11-20w	1	pipeline1	kmeans	pca20	NaN	17	65000	17	0
4	21-30w	0	pipeline1	kmeans	pca20	NaN	20	50000	20	0

```
In [ ]: # --- Build composite_eval if missing ---
def _z(x: pd.Series) -> pd.Series:
    x = x.astype(float)
    m = x.mean()
    s = x.std(ddof=0)
    if s == 0 or np.isnan(s):
        return x*0.0
    return (x - m) / s

km = km_all.copy()

needed = ["silhouette", "davies_bouldin", "calinski_harabasz"]
km = km.dropna(subset=needed)
print("After dropping rows missing SH/DB/CH:", km.shape)

km["sh_z"] = _z(km["silhouette"])
km["db_z_raw"] = km["davies_bouldin"].astype(float)
km["db_z"] = _z(km["db_z_raw"])
km["ch_log"] = np.log1p(km["calinski_harabasz"].astype(float))
km["ch_z"] = _z(km["ch_log"])

# KMeans has no noise, but we keep column for compatibility
if "noise_frac" in km.columns:
    km["noise_frac_filled"] = km["noise_frac"].fillna(0.0)
else:
    km["noise_frac_filled"] = 0.0
km["noise_z"] = _z(km["noise_frac_filled"])

km["composite_eval"] = (
    km["sh_z"]
    - km["db_z"]
    + 0.5 * km["ch_z"]
    - 0.5 * km["noise_z"]
)

km.to_csv(eval_scored_path, index=False)
print("Saved eval-scored KMeans ->", eval_scored_path)
display(km.head())
```

After dropping rows missing SH/DB/CH: (24, 20)

Saved eval-scored KMeans → /content/drive/MyDrive/preeclampsia_results/kmeans_eval_scored.csv

	interval	condition	pipeline	algorithm	space	umap_dim	k	n_cells	n_clusters	sill
0	1-10w	0	pipeline1	kmeans	pca20	NaN	20	15000	20	0
1	1-10w	1	pipeline1	kmeans	pca20	NaN	20	5000	20	0
2	11-20w	0	pipeline1	kmeans	pca20	NaN	20	55000	20	0
3	11-20w	1	pipeline1	kmeans	pca20	NaN	17	65000	17	0
4	21-30w	0	pipeline1	kmeans	pca20	NaN	20	50000	20	0

Plotting the best performing cluster runs of pipeline 1 to the best performing cluster runs of pipeline 2

```
In [ ]: # Simplify pipeline labels to P1 vs P2
def simplify_pipeline(p):
    p = str(p)
    if "pipeline1" in p.lower() or "pca" in p or "(P1" in p:
        return "P1"
    if "pipeline2" in p.lower() or "umap" in p or "(P2" in p:
        return "P2"
    return p

km["pipeline_simple"] = km["pipeline"].apply(simplify_pipeline)

print(km["pipeline_simple"].value_counts())

metrics = ["silhouette", "davies_bouldin", "calinski_harabasz", "composite_eval"]
titles = {
    "silhouette": "Silhouette",
    "davies_bouldin": "Davies-Bouldin",
    "calinski_harabasz": "Calinski-Harabasz",
    "composite_eval": "Composite evaluation score"
}

for metric in metrics:
    plt.figure(figsize=(8,5))

    data_p1 = km.loc[km["pipeline_simple"]=="P1", metric].dropna()
    data_p2 = km.loc[km["pipeline_simple"]=="P2", metric].dropna()

    plt.violinplot(
        [data_p1, data_p2],
        positions=[1,2],
        showmeans=True,
        showextrema=True
    )
    plt.xticks([1,2], ["Pipeline 1 (PCA20→KMeans)", "Pipeline 2 (UMAP→KMeans)"],
    plt.ylabel(metric)
    plt.title(f"KMeans - {titles.get(metric, metric)}")
    plt.tight_layout()
    plt.show()
```

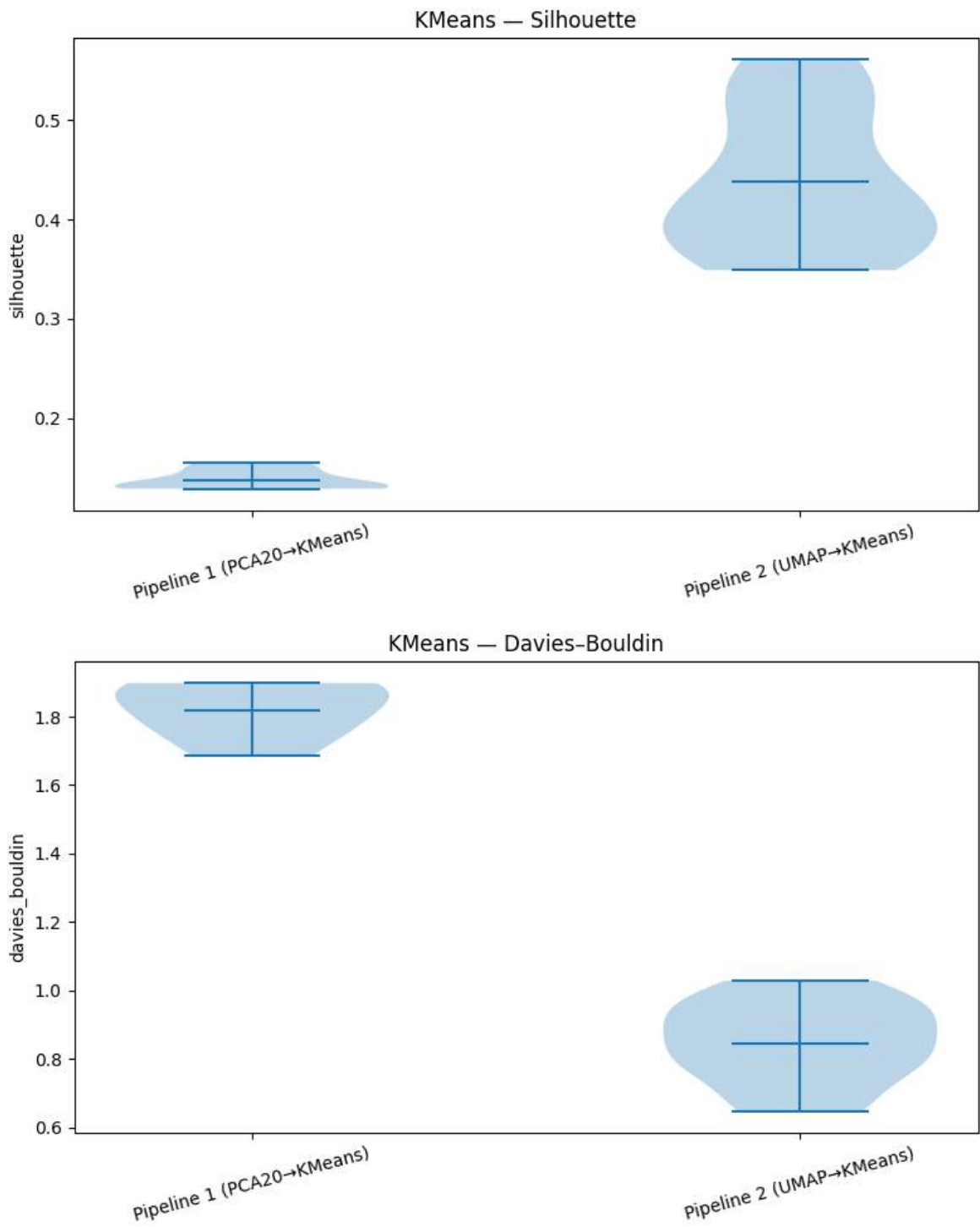


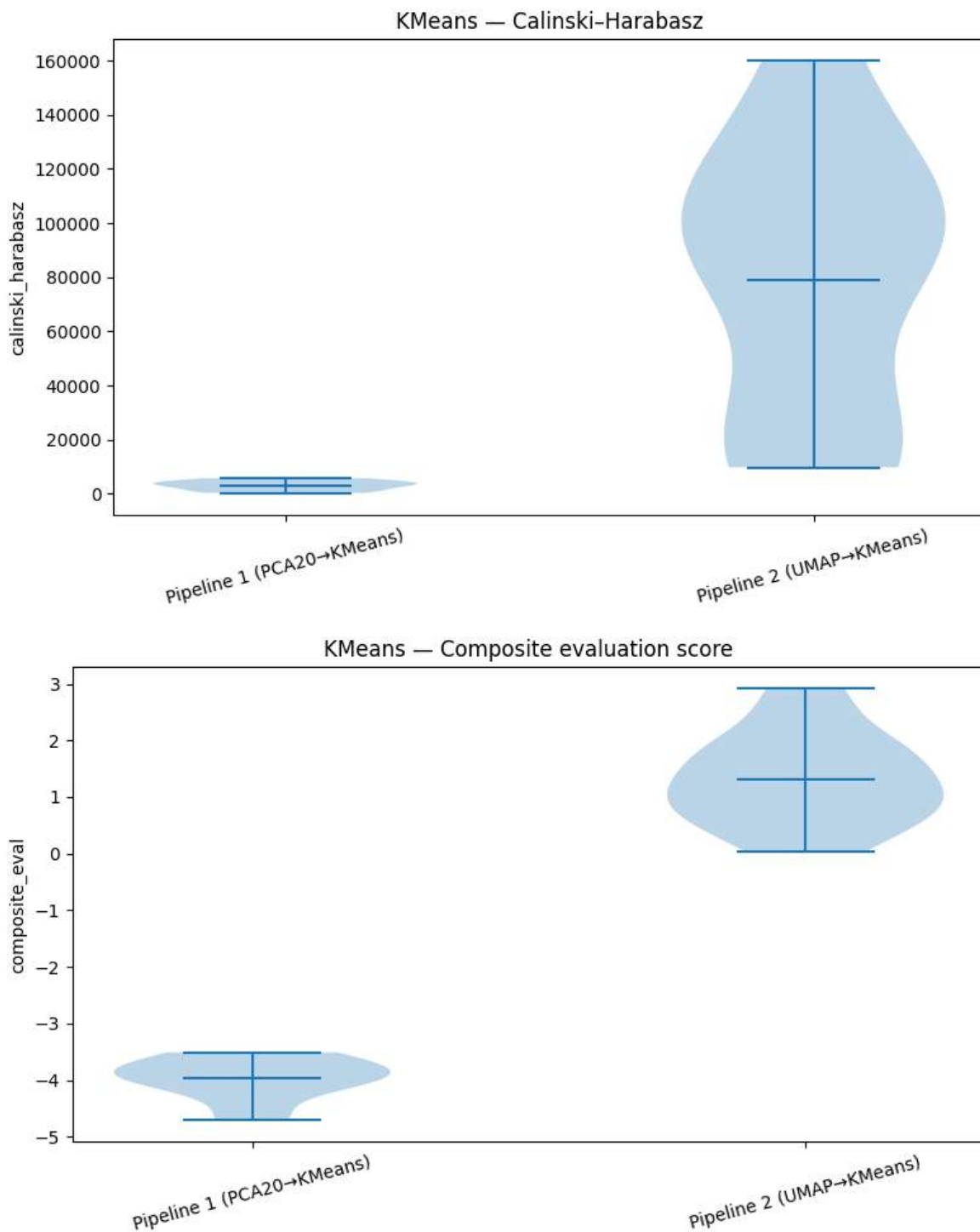
```
pipeline_simple
```

```
P2    18
```

```
P1     6
```

```
Name: count, dtype: int64
```





In []:

```

In [ ]: base = "/content/drive/MyDrive/preeclampsia_results"

files = {
    "Best pipeline 1 only": os.path.join(base, "kmeans_best_pipeline1_only.csv")
    "Best across pipelines": os.path.join(base, "kmeans_best_per_interval_across")
}

tables = {name: pd.read_csv(path) for name, path in files.items()}
for name, df in tables.items():
    print(name, df.shape)
    display(df.head())

```

Best pipeline 1 only (6, 22)

	interval	condition	algorithm	pipeline	space	umap_dim	k_final	n_cells	n_clusters
0	1-10w	0	KMeans	pipeline1	pca20	NaN	20	15000	20
1	1-10w	1	KMeans	pipeline1	pca20	NaN	20	5000	20
2	11-20w	0	KMeans	pipeline1	pca20	NaN	20	55000	20
3	11-20w	1	KMeans	pipeline1	pca20	NaN	17	65000	17
4	21-30w	0	KMeans	pipeline1	pca20	NaN	20	50000	20

5 rows × 22 columns

Best across pipelines (6, 22)

	interval	condition	algorithm	pipeline	space	umap_dim	k_final	n_cells	n_clusters
0	1-10w	0	KMeans	pipeline2	umap2	2.0	3	15000	3
1	1-10w	1	KMeans	pipeline2	umap2	2.0	5	5000	5
2	11-20w	0	KMeans	pipeline2	umap2	2.0	4	55000	4
3	11-20w	1	KMeans	pipeline2	umap2	2.0	6	65000	6
4	21-30w	0	KMeans	pipeline2	umap2	2.0	3	50000	3

5 rows × 22 columns

Comparing best of pipeline 1 with Best across pipelines

```
In [ ]: best_p1_path = os.path.join(output_dir, "kmeans_best_pipeline1_only.csv")
best_across_path = os.path.join(output_dir, "kmeans_best_per_interval_across_pip

best_p1 = pd.read_csv(best_p1_path)
best_across = pd.read_csv(best_across_path)

print("Best P1:", best_p1.shape)
display(best_p1.head())
print("Best across pipelines:", best_across.shape)
display(best_across.head())
```

Best P1: (6, 22)

	interval	condition	algorithm	pipeline	space	umap_dim	k_final	n_cells	n_clusters
0	1-10w	0	KMeans	pipeline1	pca20	NaN	20	15000	20
1	1-10w	1	KMeans	pipeline1	pca20	NaN	20	5000	20
2	11-20w	0	KMeans	pipeline1	pca20	NaN	20	55000	20
3	11-20w	1	KMeans	pipeline1	pca20	NaN	17	65000	17
4	21-30w	0	KMeans	pipeline1	pca20	NaN	20	50000	20

5 rows × 22 columns

Best across pipelines: (6, 22)

	interval	condition	algorithm	pipeline	space	umap_dim	k_final	n_cells	n_clusters
0	1-10w	0	KMeans	pipeline2	umap2	2.0	3	15000	3
1	1-10w	1	KMeans	pipeline2	umap2	2.0	5	5000	5
2	11-20w	0	KMeans	pipeline2	umap2	2.0	4	55000	4
3	11-20w	1	KMeans	pipeline2	umap2	2.0	6	65000	6
4	21-30w	0	KMeans	pipeline2	umap2	2.0	3	50000	3

5 rows × 22 columns

```
In [ ]: for df in [best_p1, best_across]:
        if "composite_eval" not in df.columns:
            df["sh_z"] = _z(df["silhouette"])
            df["db_z_raw"] = df["davies_bouldin"].astype(float)
            df["db_z"] = _z(df["db_z_raw"])
            df["ch_log"] = np.log1p(df["calinski_harabasz"].astype(float))
            df["ch_z"] = _z(df["ch_log"])
            df["noise_frac_filled"] = 0.0
            df["noise_z"] = _z(df["noise_frac_filled"])
            df["composite_eval"] = (
                df["sh_z"] - df["db_z"] + 0.5*df["ch_z"] - 0.5*df["noise_z"]
            )

        # (Optional) save updates back
        best_p1.to_csv(best_p1_path, index=False)
        best_across.to_csv(best_across_path, index=False)
```

```
In [ ]: COND_NAME = {0: "Control", 1: "PE"}

def make_label(row):
    return f"{row['interval']} | {COND_NAME.get(int(row['condition']), row['cond

# Make sure we have the same 6 (interval, condition) combos in both tables
cols_key = ["interval", "condition"]

metrics_for_bars = ["silhouette", "davies_bouldin", "calinski_harabasz", "compos

for metric in metrics_for_bars:
```

```

# merge on interval + condition so rows align
merged = best_p1[cols_key + [metric]].merge(
    best_across[cols_key + [metric]],
    on=cols_key,
    suffixes=("_p1", "_across")
)

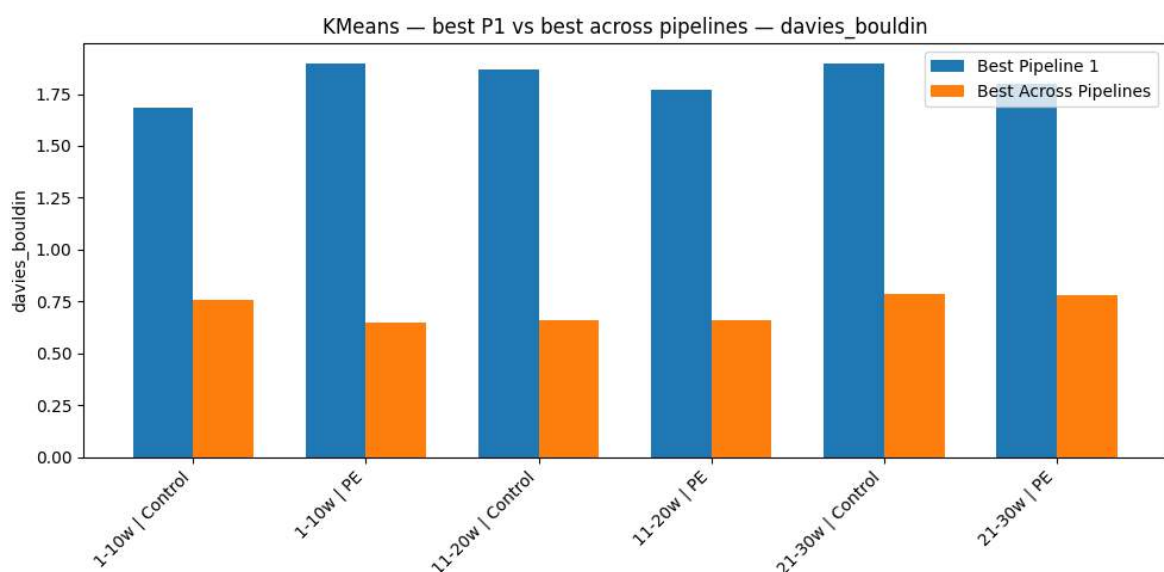
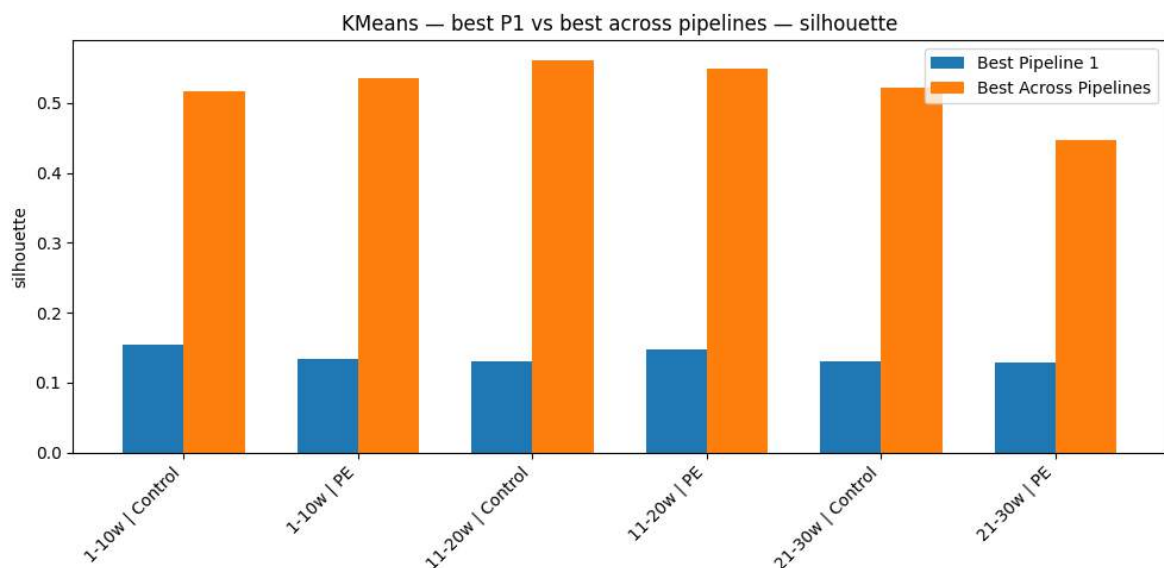
merged = merged.sort_values(["interval", "condition"]).reset_index(drop=True)
labels = merged.apply(make_label, axis=1)

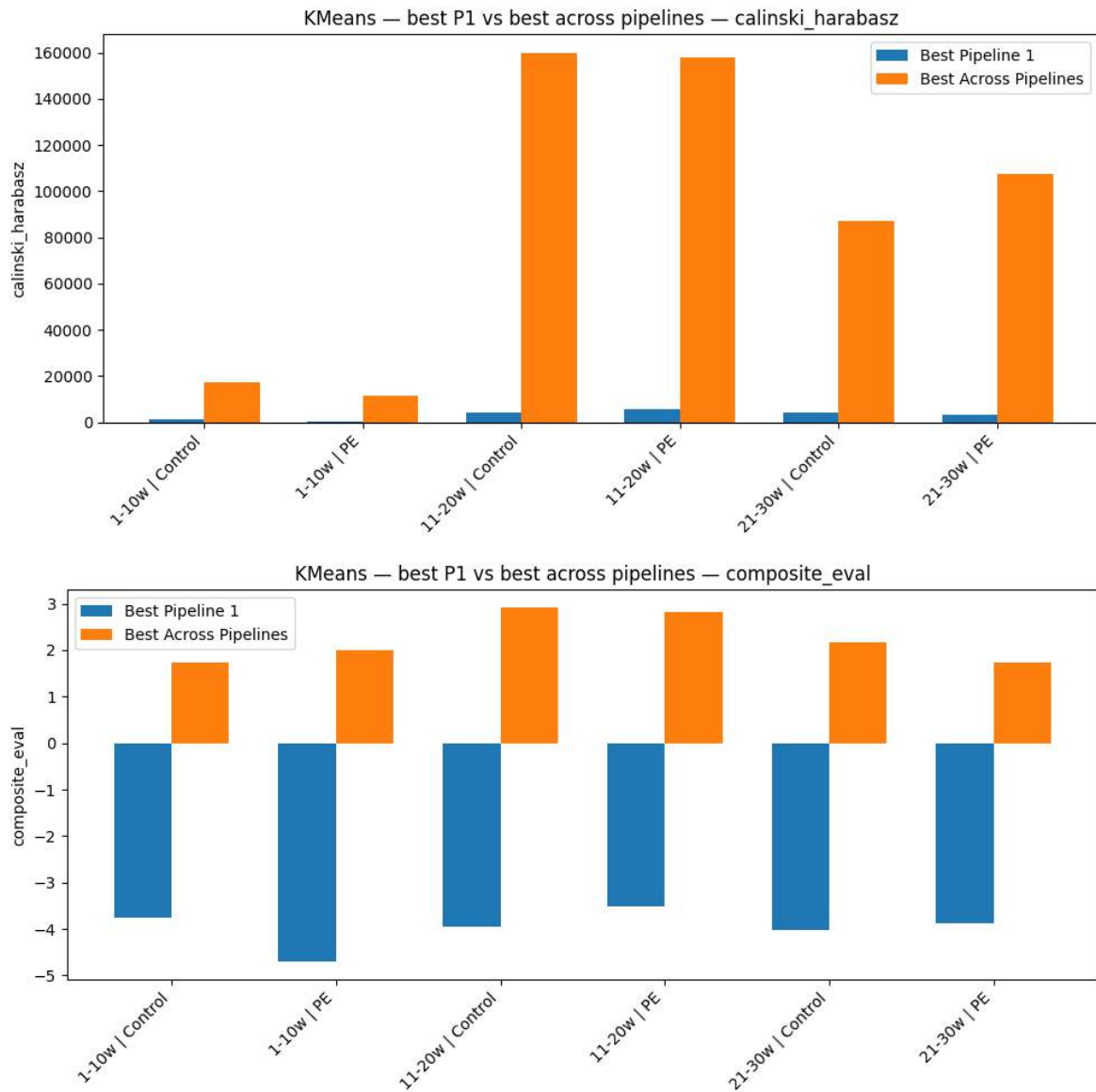
x = np.arange(len(merged))
width = 0.35

plt.figure(figsize=(10,5))
plt.bar(x - width/2, merged[f"{metric}_p1"], width, label="Best Pipeline 1")
plt.bar(x + width/2, merged[f"{metric}_across"], width, label="Best Across Pipelines")

plt.xticks(x, labels, rotation=45, ha="right")
plt.ylabel(metric)
plt.title(f"KMeans — best P1 vs best across pipelines — {metric}")
plt.legend()
plt.tight_layout()
plt.show()

```





Heatmap of the above metrics

```
In [ ]: COND_NAME = {0: "Control", 1: "PE"}

def make_metric_heatmaps(df, title_prefix):
    df = df.copy()

    # Make sure these columns are the right types
    df["interval"] = df["interval"].astype(str)
    df["condition"] = df["condition"].astype(int)

    # Row Label = interval + condition name
    df["row_label"] = df.apply(
        lambda r: f"{r['interval']} | {COND_NAME.get(r['condition'], r['condition'])}"
    )

    # Column Label (run id). If you have 'space', include it, else just pipeline
    if "space" in df.columns:
        df["run_label"] = df["pipeline"].astype(str) + " / " + df["space"].astype(str)
    else:
        df["run_label"] = df["pipeline"].astype(str)

    # Sort rows nicely
    df = df.sort_values(["interval", "condition"])
```

```

# Metrics to plot (only those that exist in this df)
possible_metrics = ["silhouette", "davies_bouldin", "calinski_harabasz", "co
metrics = [m for m in possible_metrics if m in df.columns]
print(f"{title_prefix}: plotting metrics →", metrics)

for metric in metrics:
    sub = df[["row_label", "run_label", metric]].dropna(subset=[metric])
    if sub.empty:
        print(f" Skipping {metric}, no data")
        continue

    # Pivot: rows = interval+condition, columns = run, values = metric
    piv = sub.pivot_table(
        index="row_label", columns="run_label", values=metric, aggfunc="firs
    )

    plt.figure(figsize=(1.5 * len(piv.columns) + 3, 0.4 * len(piv.index) + 3)
    im = plt.imshow(piv.values, aspect="auto", cmap="viridis")
    cbar = plt.colorbar(im)

    # Helpful colorbar labels
    if metric == "silhouette":
        cbar.set_label("Silhouette (-1 to 1, higher better)")
    elif metric == "davies_bouldin":
        cbar.set_label("Davies-Bouldin (lower better)")
    elif metric == "calinski_harabasz":
        cbar.set_label("Calinski-Harabasz (higher better)")
    elif metric == "composite_eval":
        cbar.set_label("Composite eval score (higher better)")
    else:
        cbar.set_label(metric)

    plt.xticks(range(len(piv.columns)), piv.columns, rotation=45, ha="right")
    plt.yticks(range(len(piv.index)), piv.index)
    plt.title(f"{title_prefix} - {metric}")
    plt.tight_layout()
    plt.show()

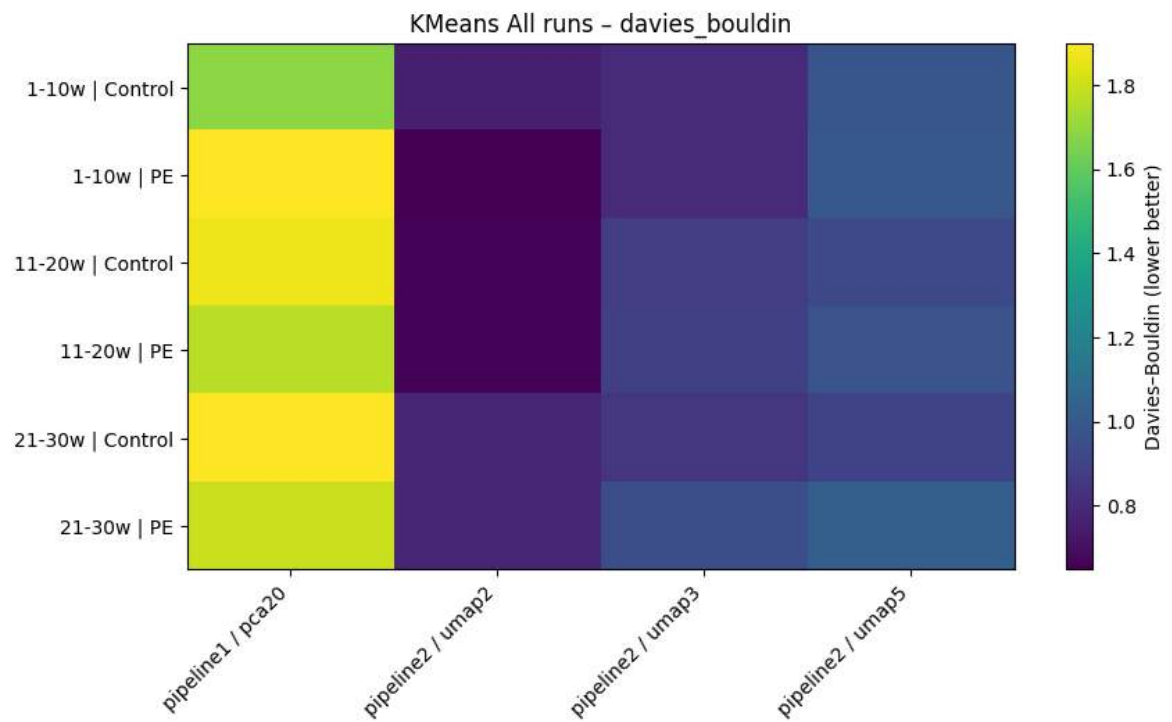
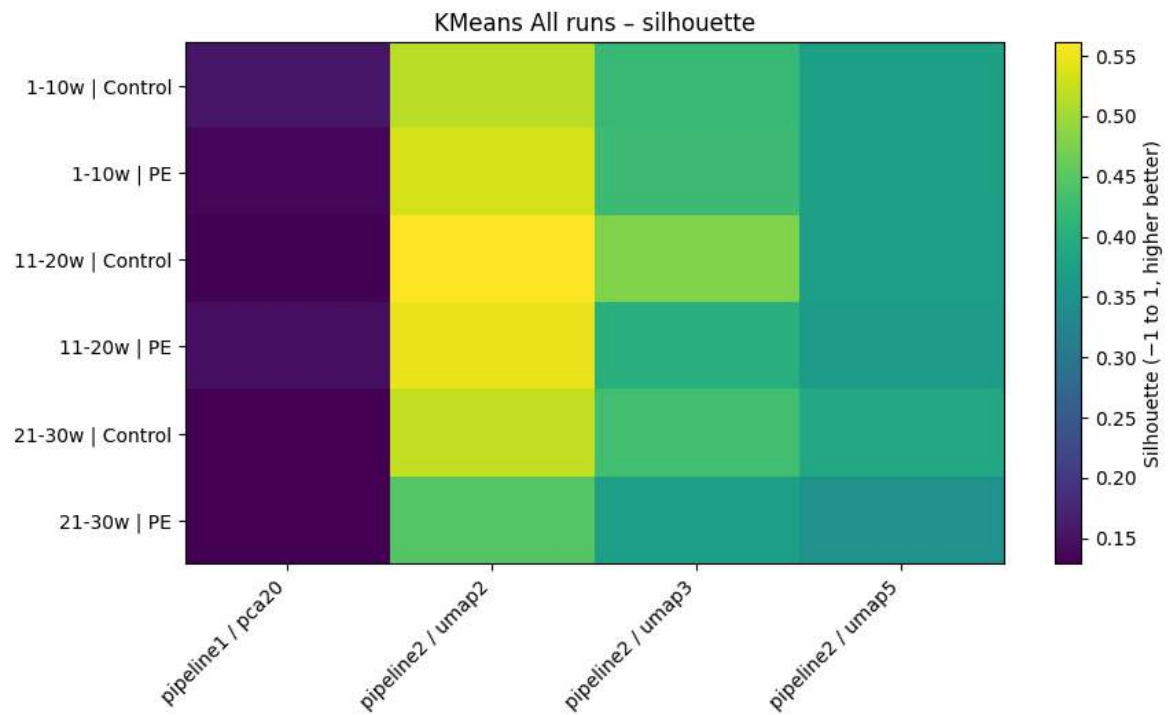
```

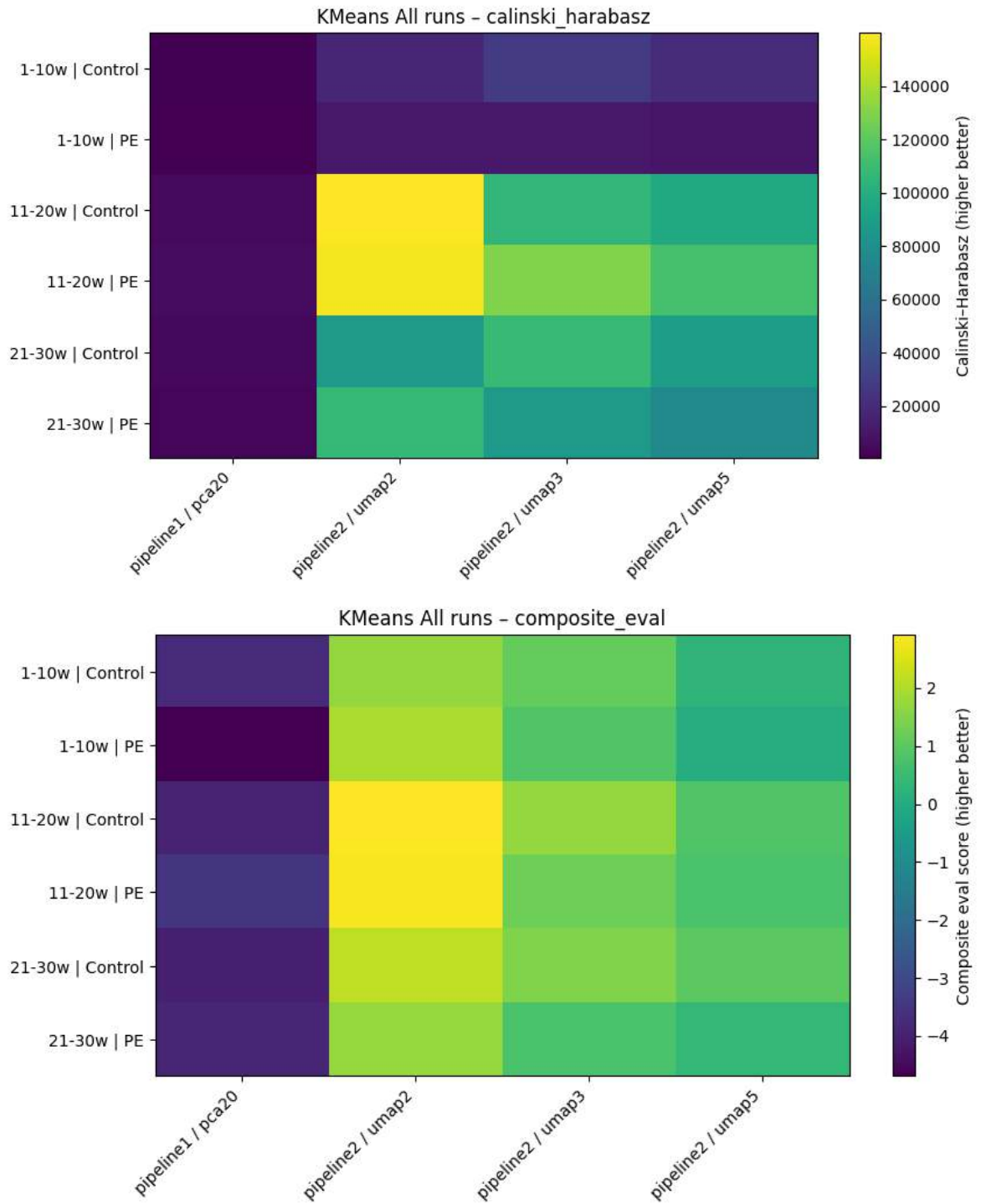
```

In [ ]: for name, df in tables.items():
        make_metric_heatmaps(df, f"KMeans {name}")

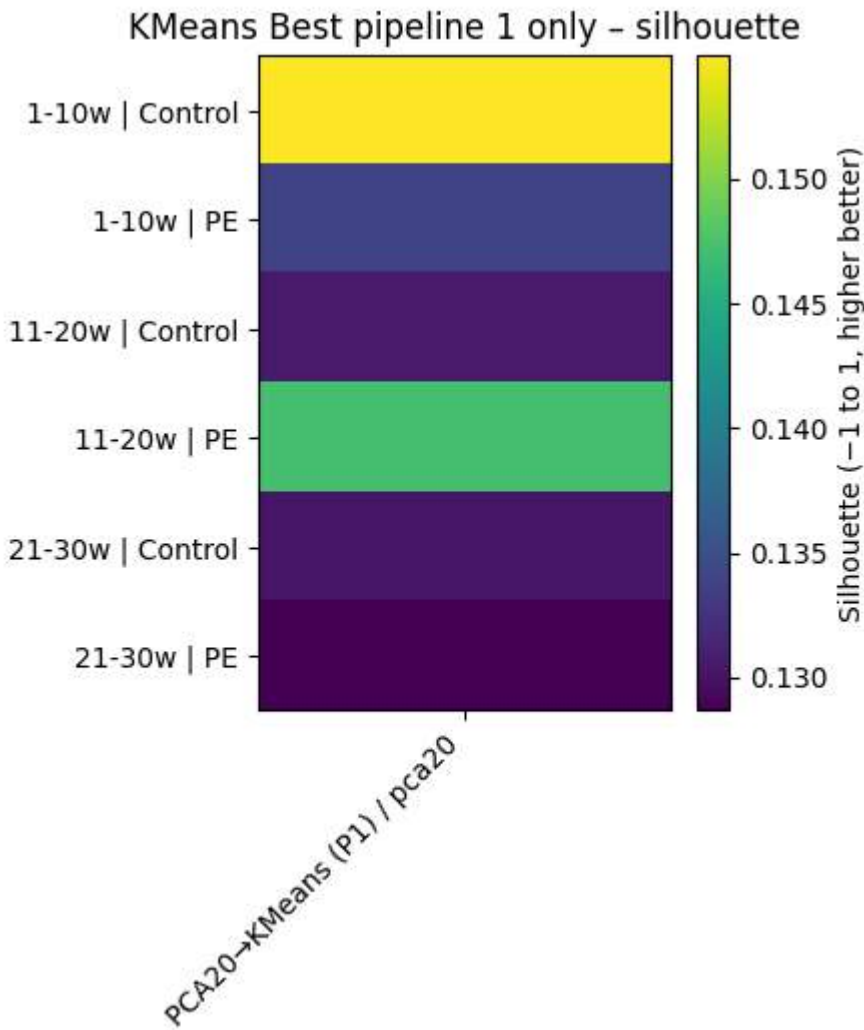
```

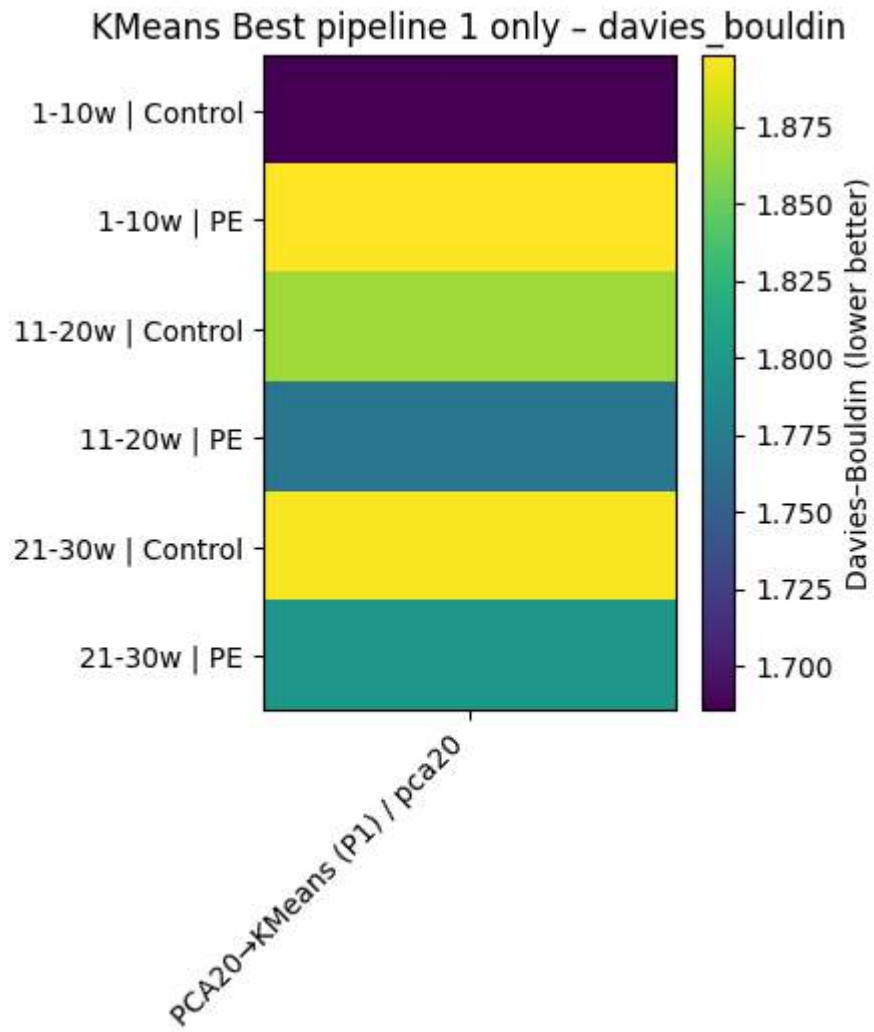
KMeans All runs: plotting metrics → ['silhouette', 'davies_bouldin', 'calinski_harabasz', 'composite_eval']

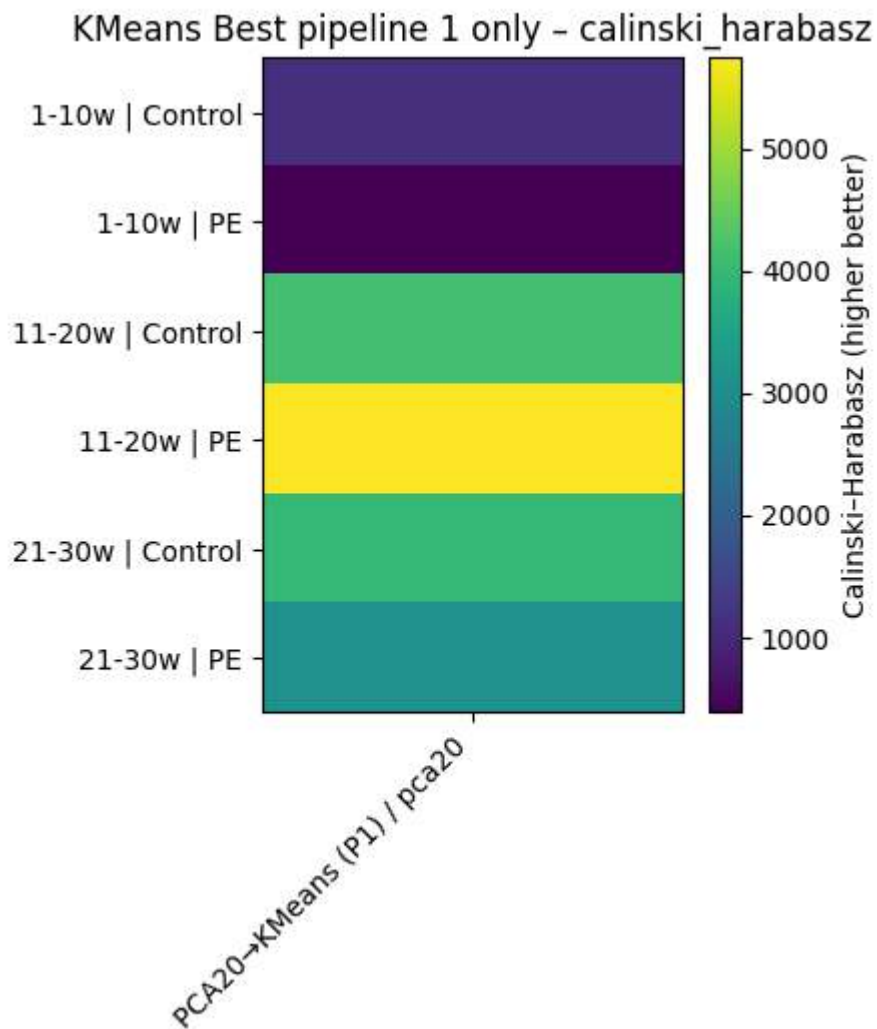


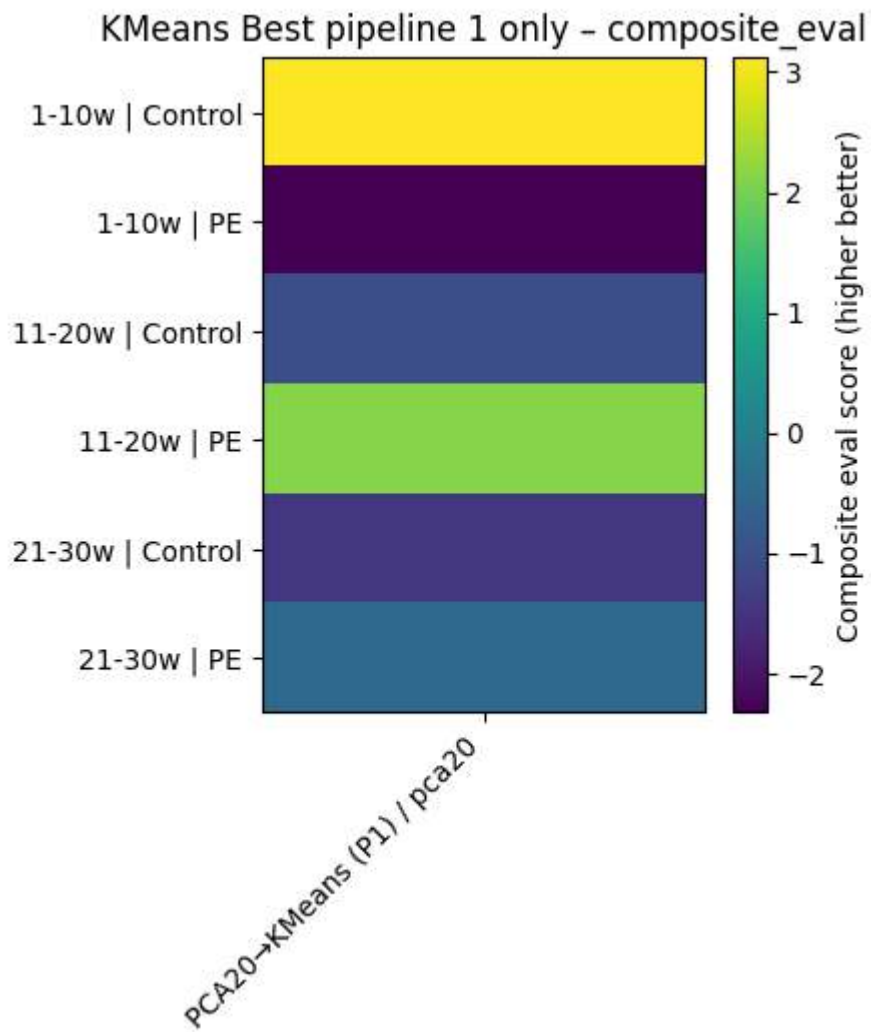


KMeans Best pipeline 1 only: plotting metrics → ['silhouette', 'davies_bouldin', 'calinski_harabasz', 'composite_eval']

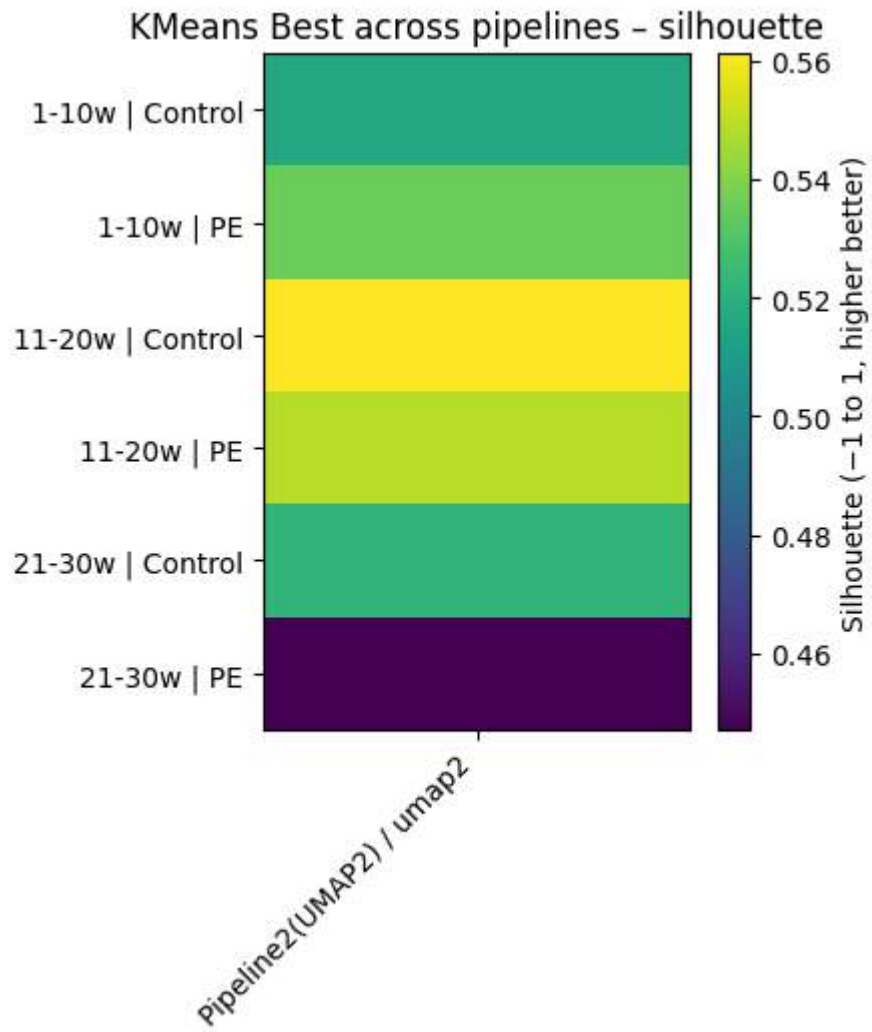


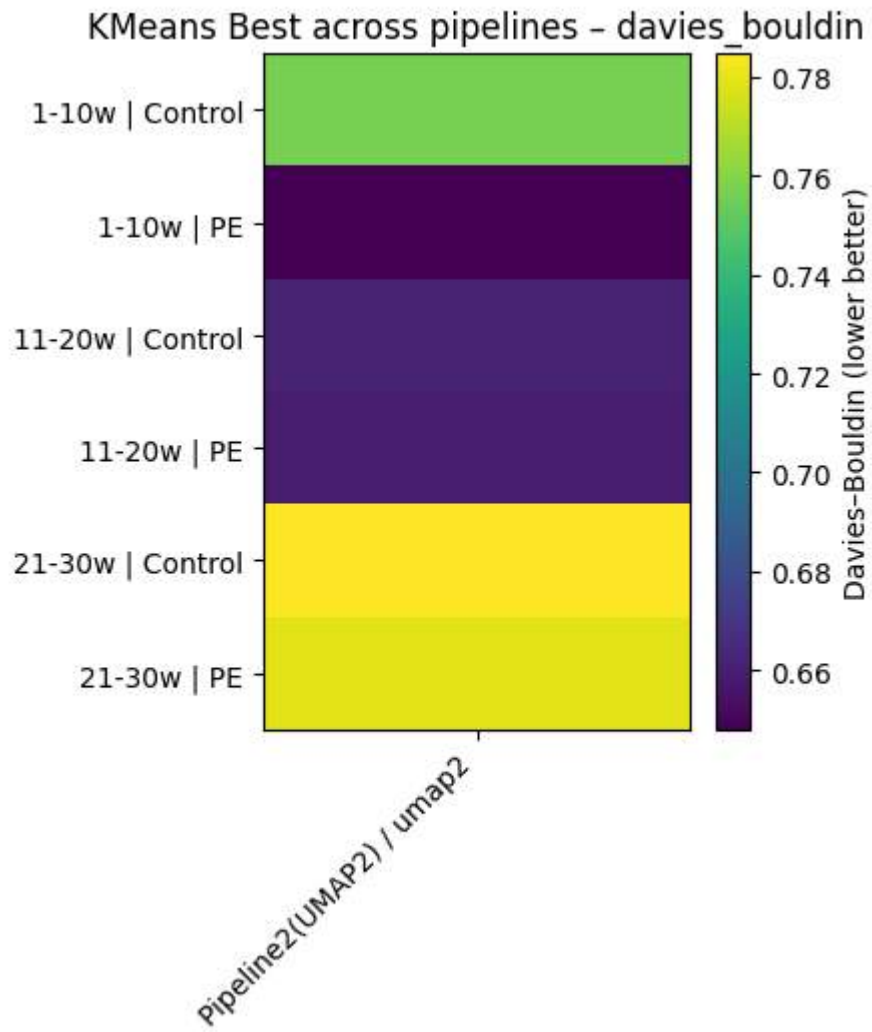


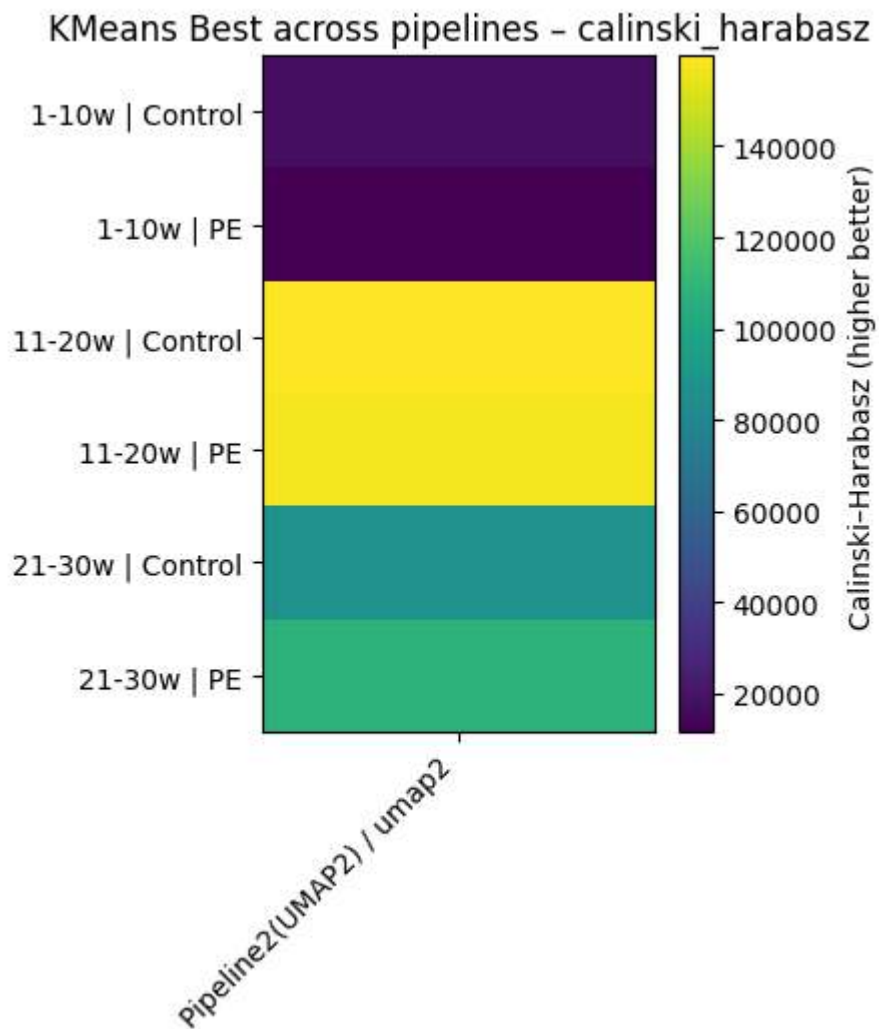


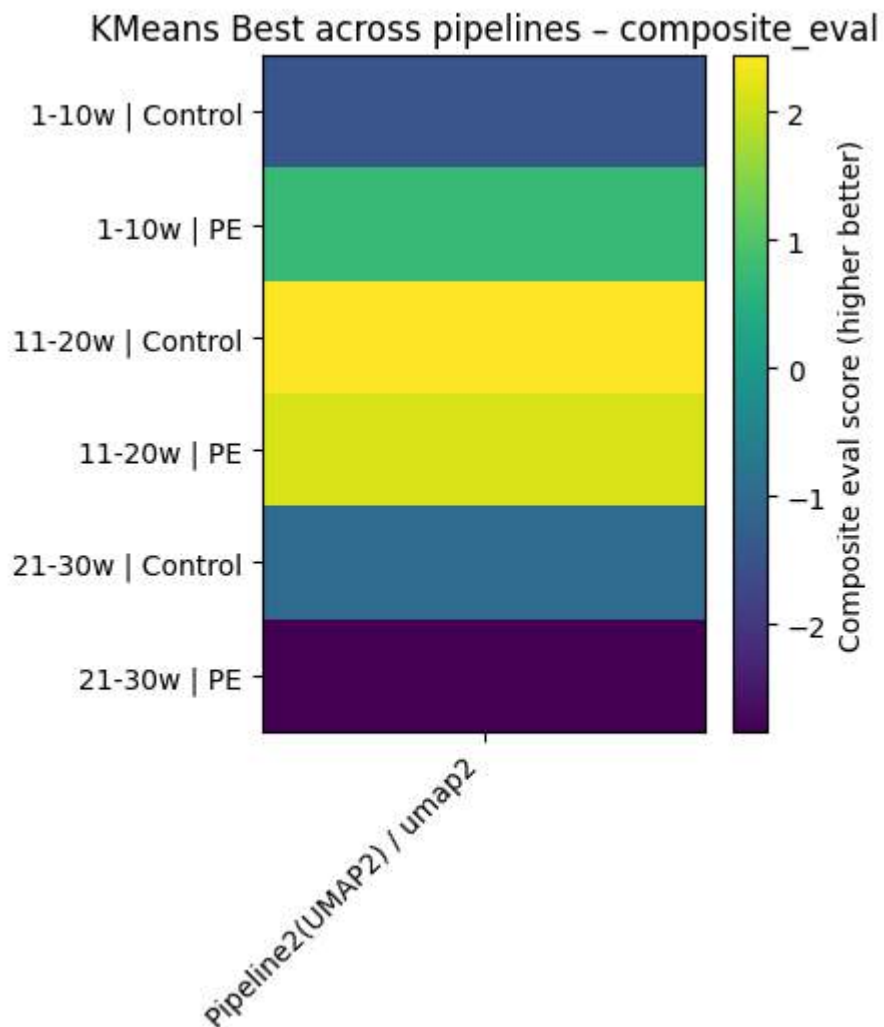


KMeans Best across pipelines: plotting metrics → ['silhouette', 'davies_bouldin', 'calinski_harabasz', 'composite_eval']









HDBscan evaluation of runs

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

# Path to hdb cluster inventory
hdb_inv_path = os.path.join(output_dir, "hdbscan_inventory.csv")

hdb_inv = pd.read_csv(hdb_inv_path)
print("HDB inventory:", hdb_inv.shape)
hdb_inv.head()
```

HDB inventory: (156, 18)

Out[]:

	interval	condition	n_cells	min_cluster_size	min_samples	selection_method	n_clusters
--	----------	-----------	---------	------------------	-------------	------------------	------------

0	1-10w	0	15000	10	10	eom	
---	-------	---	-------	----	----	-----	--

1	1-10w	1	5000	10	1	eom	
---	-------	---	------	----	---	-----	--

2	11-20w	0	55000	10	5	eom	
---	--------	---	-------	----	---	-----	--

3	11-20w	1	65000	10	1	eom	
---	--------	---	-------	----	---	-----	--

4	21-30w	0	50000	60	10	eom	
---	--------	---	-------	----	----	-----	--

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

# Load HDBSCAN inventory
hdb_inv_path = os.path.join(output_dir, "hdbscan_inventory.csv")
hdb_inv = pd.read_csv(hdb_inv_path)
print("HDB inventory:", hdb_inv.shape)
display(hdb_inv.head())

# Case sensitive filtering to find the saved inventory
hdb = hdb_inv[hdb_inv["algorithm"].str.lower() == "hdbscan"].copy()
print("After filtering for HDBSCAN:", hdb.shape)

# Load labels from CSV saved as a Series
def load_labels_series(csv_path: str) -> np.ndarray:
    df = pd.read_csv(csv_path)
    s = df.iloc[:, -1]
    return s.to_numpy()

# infer embedding path from pipeline / umap_dim
def get_embedding_path(row):
    interval = str(row["interval"])
    cond = int(row["condition"])
    tag = f"{interval}_{cond}"

    pipeline = str(row["pipeline"])
    umap_dim = row.get("umap_dim", np.nan)

    # Pipeline 1 - PCA20 space
    if "PCA20" in pipeline:
        return os.path.join(output_dir, f"pca20_{tag}.npy"), "pca20", np.nan

    # Pipeline 2 - UMAP space
    # Prefer explicit umap_dim if present
    if not pd.isna(umap_dim):
        d = int(umap_dim)
    else:
```

```

    # fallback: try to infer from plot filename
    fname = str(row.get("plot_png", "")) or str(row.get("umap_plot_png", ""))
    if "umap2" in fname:
        d = 2
    elif "umap3" in fname:
        d = 3
    elif "umap5" in fname:
        d = 5
    else:
        return None, None, np.nan

    emb_path = os.path.join(output_dir, f"umap{d}_pipeline2__{tag}.npy")
    return emb_path, f"umap{d}", d

# compute metrics on NON-NOISE only
def compute_cluster_metrics_nonnoise(X, labels):
    X = np.asarray(X)
    labels = np.asarray(labels)

    n = len(labels)
    if n == 0:
        return np.nan, np.nan, np.nan, np.nan, 0

    noise_mask = (labels == -1)
    noise_frac = float(noise_mask.mean())

    keep = ~noise_mask
    if keep.sum() < 3:
        return np.nan, np.nan, np.nan, noise_frac, 0

    X_nn = X[keep]
    labels_nn = labels[keep]
    uniq = np.unique(labels_nn)
    n_clusters = len(uniq)

    if n_clusters < 2:
        return np.nan, np.nan, np.nan, noise_frac, n_clusters

    try:
        sil = float(silhouette_score(X_nn, labels_nn))
    except Exception:
        sil = np.nan
    try:
        db = float(davies_bouldin_score(X_nn, labels_nn))
    except Exception:
        db = np.nan
    try:
        ch = float(calinski_harabasz_score(X_nn, labels_nn))
    except Exception:
        ch = np.nan

    return sil, db, ch, noise_frac, n_clusters

metrics_rows = []

for i, r in hdb.iterrows():
    interval = r["interval"]
    cond = int(r["condition"])
    pipeline = r["pipeline"]

```

```

algo      = r["algorithm"]

labels_csv = r.get("labels_csv", np.nan)
if not isinstance(labels_csv, str) or not os.path.exists(labels_csv):
    print(f"[HDB metrics] No labels file found for {interval}_{cond} ({pipeline})")
    continue

emb_path, space, umap_dim = get_embedding_path(r)
if emb_path is None or not os.path.exists(emb_path):
    print(f"[HDB metrics] Missing embedding for {interval}_{cond} ({pipeline})")
    continue

X = np.load(emb_path)
labels = load_labels_series(labels_csv)

if len(X) != len(labels):
    print(f"[HDB metrics] Length mismatch for {interval}_{cond} ({pipeline})")
    print(f"X={len(X)}, labels={len(labels)}, skipping.")
    continue

sil, db, ch, noise_frac_calc, n_clusters_nn = compute_cluster_metrics_nonnoi

metrics_rows.append({
    "interval": interval,
    "condition": cond,
    "pipeline": pipeline,
    "algorithm": algo,
    "space": space,
    "umap_dim": umap_dim,
    "min_cluster_size": r.get("min_cluster_size", np.nan),
    "min_samples": r.get("min_samples", np.nan),
    "selection_method": r.get("selection_method", np.nan),
    "n_cells": len(labels),
    "n_clusters_nonnoise": n_clusters_nn,
    "noise_frac": noise_frac_calc,
    "silhouette": sil,
    "davies_bouldin": db,
    "calinski_harabasz": ch,
    # keep the paths from the inventory so you can find plots easily
    "plot_png": r.get("plot_png", r.get("umap_plot_png", np.nan)),
    "labels_csv": labels_csv,
})

hdb_metrics = pd.DataFrame(metrics_rows)
print("HDBSCAN metrics table:", hdb_metrics.shape)
display(hdb_metrics.head())

hdb_metrics_path = os.path.join(output_dir, "hdbscan_all_metrics_SH_DB_CH.csv")
hdb_metrics.to_csv(hdb_metrics_path, index=False)
print("Saved HDBSCAN metrics →", hdb_metrics_path)

```

HDB inventory: (156, 18)

	interval	condition	n_cells	min_cluster_size	min_samples	selection_method	n_clusters
0	1-10w	0	15000	10	10	eom	2
1	1-10w	1	5000	10	1	eom	2
2	11-20w	0	55000	10	5	eom	2
3	11-20w	1	65000	10	1	eom	2
4	21-30w	0	50000	60	10	eom	2

◀  ▶

After filtering for HDBSCAN: (156, 18)

[HDB metrics] No labels file found for 1-10w_1 (PCA20 (P1, merged base+HI)), skipping.

[HDB metrics] No labels file found for 11-20w_1 (PCA20 (P1, merged base+HI)), skipping.

[HDB metrics] No labels file found for 21-30w_0 (PCA20 (P1, merged base+HI)), skipping.

[HDB metrics] No labels file found for 21-30w_1 (PCA20 (P1, merged base+HI)), skipping.

[HDB metrics] No labels file found for 1-10w_1 (PCA20 (P1, merged base+HI)), skipping.

[HDB metrics] No labels file found for 11-20w_1 (PCA20 (P1, merged base+HI)), skipping.

[HDB metrics] No labels file found for 21-30w_0 (PCA20 (P1, merged base+HI)), skipping.

[HDB metrics] No labels file found for 21-30w_1 (PCA20 (P1, merged base+HI)), skipping.

[HDB metrics] No labels file found for 1-10w_1 (PCA20 (P1, merged base+HI)), skipping.

[HDB metrics] No labels file found for 11-20w_1 (PCA20 (P1, merged base+HI)), skipping.

[HDB metrics] No labels file found for 21-30w_0 (PCA20 (P1, merged base+HI)), skipping.

[HDB metrics] No labels file found for 21-30w_1 (PCA20 (P1, merged base+HI)), skipping.

HDBSCAN metrics table: (144, 17)

	interval	condition	pipeline	algorithm	space	umap_dim	min_cluster_size	min_samp
0	1-10w	0	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
1	1-10w	1	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
2	11-20w	0	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
3	11-20w	1	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
4	21-30w	0	PCA20 (P1, base)	HDBSCAN	pca20	NaN	60	

◀  ▶

Saved HDBSCAN metrics → /content/drive/MyDrive/preeclampsia_results/hdbscan_all_metrics_SH_DB_CH.csv

Composite Score for SH, DB, CH only on non-noise points

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"
hdb_metrics_path = os.path.join(output_dir, "hdbscan_all_metrics_SH_DB_CH.csv")

hdbm = pd.read_csv(hdb_metrics_path)
print("HDBSCAN metrics shape before dropna:", hdbm.shape)

needed = ["silhouette", "davies_bouldin", "calinski_harabasz"]
hdbm = hdbm.dropna(subset=needed)
print("After dropping missing SH/DB/CH:", hdbm.shape)

def _z(x: pd.Series) -> pd.Series:
    x = x.astype(float)
    m = x.mean()
    s = x.std(ddof=0)
    if s == 0 or np.isnan(s):
        return x * 0.0
    return (x - m) / s

# z-scores
hdbm["sh_z"] = _z(hdbm["silhouette"])

hdbm["db_z_raw"] = hdbm["davies_bouldin"].astype(float)
hdbm["db_z"] = _z(hdbm["db_z_raw"])

hdbm["ch_log"] = np.log1p(hdbm["calinski_harabasz"].astype(float))
hdbm["ch_z"] = _z(hdbm["ch_log"])

if "noise_frac" in hdbm.columns:
    hdbm["noise_frac_filled"] = hdbm["noise_frac"].fillna(0.0)
else:
    hdbm["noise_frac_filled"] = 0.0
hdbm["noise_z"] = _z(hdbm["noise_frac_filled"])
```

```
# composite evaluation score - higher is better
hdbm["composite_eval"] = (
    hdbm["sh_z"]          # SH ↑
    - hdbm["db_z"]        # DBI ↓
    + 0.5 * hdbm["ch_z"]  # CH ↑
    - 0.5 * hdbm["noise_z"] # noise ↓
)

hdb_eval_path = os.path.join(output_dir, "hdbscan_eval_scored.csv")
hdbm.to_csv(hdb_eval_path, index=False)
print("Saved HDBSCAN eval-scored table →", hdb_eval_path)
display(hdbm)
```

HDBSCAN metrics shape before dropna: (144, 17)

After dropping missing SH/DB/CH: (144, 17)

Saved HDBSCAN eval-scored table → /content/drive/MyDrive/preeclampsia_results/hdbscan_eval_scored.csv

	interval	condition	pipeline	algorithm	space	umap_dim	min_cluster_size	min_sa
0	1-10w	0	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
1	1-10w	1	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
2	11-20w	0	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
3	11-20w	1	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
4	21-30w	0	PCA20 (P1, base)	HDBSCAN	pca20	NaN	60	
...	...	...	...	...	...	...	...	...
139	21-30w	0	UMAP (P2, tuned)	HDBSCAN	umap3	3.0	10	
140	21-30w	0	UMAP (P2, tuned)	HDBSCAN	umap5	5.0	10	
141	21-30w	1	UMAP (P2, tuned)	HDBSCAN	umap2	2.0	10	
142	21-30w	1	UMAP (P2, tuned)	HDBSCAN	umap3	3.0	10	
143	21-30w	1	UMAP (P2, tuned)	HDBSCAN	umap5	5.0	10	

144 rows × 25 columns



Best across pipelines HDBSCAN

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

# --- Load the master evaluation table ---
hdb_path = os.path.join(output_dir, "hdbscan_eval_scored.csv")
hdb = pd.read_csv(hdb_path)

print("Loaded HDBSCAN eval table:", hdb.shape)
display(hdb.head())
```

Loaded HDBSCAN eval table: (144, 25)

	interval	condition	pipeline	algorithm	space	umap_dim	min_cluster_size	min_samp
0	1-10w	0	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
1	1-10w	1	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
2	11-20w	0	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
3	11-20w	1	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
4	21-30w	0	PCA20 (P1, base)	HDBSCAN	pca20	NaN	60	

5 rows × 25 columns



```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

eval_path = os.path.join(output_dir, "hdbscan_eval_scored.csv")
hdb = pd.read_csv(eval_path)
print("Eval-scored HDB table:", hdb.shape)

# If multiple algorithms in the file, keep only HDBSCAN
if "algorithm" in hdb.columns:
    hdb = hdb[hdb["algorithm"].str.lower() == "hdbscan"].copy()
    print("After filtering HDBSCAN only:", hdb.shape)

# Require composite_eval >= 1 as you requested
hdb = hdb[hdb["composite_eval"] >= 1].copy()
print("After composite_eval >= 1 filter:", hdb.shape)
```

Eval-scored HDB table: (144, 25)
 After filtering HDBSCAN only: (144, 25)
 After composite_eval >= 1 filter: (66, 25)

```
In [ ]: def normalise_interval(x):
    # convert to string, strip spaces, normalise dashes
    s = str(x).strip()
    # replace en-dash and em-dash with simple hyphen
    s = s.replace("-", "-").replace("-", "-")
    return s

hdb["interval_norm"] = hdb["interval"].map(normalise_interval)
hdb["condition_norm"] = hdb["condition"].astype(int)
```

```
In [ ]: hdb_sorted = hdb.sort_values(
    by=["interval_norm", "condition_norm", "silhouette", "composite_eval"],
    ascending=[True, True, False, False]
).copy()
```

```

hdb_sorted["rank_within_interval_condition"] = (
    hdb_sorted
    .groupby(["interval_norm", "condition_norm"])
    .cumcount() + 1
)

print("With ranking column:")
display(hdb_sorted[[
    "interval", "condition", "interval_norm", "condition_norm",
    "pipeline", "space" if "space" in hdb_sorted.columns else "umap_dim",
    "silhouette", "composite_eval", "rank_within_interval_condition", "labels_csv"
]].sort_values(["interval_norm", "condition_norm", "rank_within_interval_conditi

```

With ranking column:

	interval	condition	interval_norm	condition_norm	pipeline	space	silhouette	cor
12	1-10w	0	1-10w	0	UMAP (P2, base)	umap2	0.455572	
36	1-10w	0	1-10w	0	UMAP (P2, tuned)	umap2	0.455572	
60	1-10w	0	1-10w	0	UMAP (P2, base)	umap2	0.455572	
84	1-10w	0	1-10w	0	UMAP (P2, tuned)	umap2	0.455572	
108	1-10w	0	1-10w	0	UMAP (P2, base)	umap2	0.455572	
...	...	...	...	...	...	...	...	...
46	21-30w	1	21-30w	1	UMAP (P2, tuned)	umap3	0.475247	
76	21-30w	1	21-30w	1	UMAP (P2, base)	umap3	0.475247	
94	21-30w	1	21-30w	1	UMAP (P2, tuned)	umap3	0.475247	
124	21-30w	1	21-30w	1	UMAP (P2, base)	umap3	0.475247	
142	21-30w	1	21-30w	1	UMAP (P2, tuned)	umap3	0.475247	

66 rows × 10 columns



Shortlisting to the best per interval/ Condition

```
In [ ]: hdb_best6 = (
    hdb_sorted[hdb_sorted["rank_within_interval_condition"] == 1]
    .copy()
    .reset_index(drop=True)
)

print("Best 1 per (interval, condition):", hdb_best6.shape)
display(hdb_best6[[
    "interval", "condition", "pipeline",
    "space" if "space" in hdb_best6.columns else "umap_dim",
    "silhouette", "composite_eval", "rank_within_interval_condition",
    "plot_png", "labels_csv"
]])
```

Best 1 per (interval, condition): (6, 28)

	interval	condition	pipeline	space	silhouette	composite_eval	rank_within_interval_c
0	1-10w	0	UMAP (P2, umap2 base)		0.455572	1.507343	
1	1-10w	1	UMAP (P2, umap5 base)		0.599960	3.088461	
2	11-20w	0	UMAP (P2, umap5 tuned)		0.541894	2.311847	
3	11-20w	1	UMAP (P2, umap5 base)		0.477691	1.268321	
4	21-30w	0	UMAP (P2, umap3 tuned)		0.507583	2.432742	
5	21-30w	1	UMAP (P2, umap5 base)		0.489319	1.736101	

```
In [ ]: across_path = os.path.join(output_dir, "hdbscan_best_per_interval_across_pipelin
hdb_best6.to_csv(across_path, index=False)
print("Overwrote:", across_path)
```

Overwrote: /content/drive/MyDrive/preeclampsia_results/hdbscan_best_per_interval_across_pipelines.csv

Best pipeline 1 hdb run

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

# Load the eval-scored HDBSCAN table
eval_path = os.path.join(output_dir, "hdbscan_eval_scored.csv")
print("Loading:", eval_path)
hdb = pd.read_csv(eval_path)
```

```

print("Original eval shape:", hdb.shape)

# Keep only HDBSCAN rows (if multiple algos are present)
if "algorithm" in hdb.columns:
    hdb = hdb[hdb["algorithm"].str.lower() == "hdbscan"].copy()
    print("After filtering algorithm == 'hdbscan':", hdb.shape)

# Keep only Pipeline 1 runs (PCA20 / P1)
# Adjust the condition if your pipeline labels differ, but this should
# catch things like "PCA20 (P1, base)" / "PCA20 (P1, merged base+HI)"
mask_p1 = (
    hdb["pipeline"].astype(str).str.contains("PCA20", case=False)
    | hdb["pipeline"].astype(str).str.contains("P1", case=False)
)
hdb_p1 = hdb[mask_p1].copy()
print("Pipeline 1 only:", hdb_p1.shape)

# Decide which score column to use for tie-breaking
if "composite_eval" in hdb_p1.columns:
    score_col = "composite_eval"
elif "composite" in hdb_p1.columns:
    score_col = "composite"
else:
    raise KeyError("Neither 'composite_eval' nor 'composite' found in HDB table.")

print(f"Using '{score_col}' for ranking within P1.")

# Sort within each (interval, condition):
#   - primary: silhouette (desc)
#   - secondary: composite_eval / composite (desc)
hdb_p1_sorted = hdb_p1.sort_values(
    by=["interval", "condition", "silhouette", score_col],
    ascending=[True, True, False, False]
).copy()

# Rank: 1 = best in that (interval, condition) in Pipeline 1
hdb_p1_sorted["rank_within_interval_condition"] = (
    hdb_p1_sorted.groupby(["interval", "condition"])
    .cumcount() + 1
)

print("With ranking column (Pipeline 1):")
display(hdb_p1_sorted[[
    "interval", "condition", "pipeline",
    "silhouette", score_col, "rank_within_interval_condition"
]])

# Keep only rank == 1 - exactly one run per (interval, condition)
hdb_p1_best6 = (
    hdb_p1_sorted[hdb_p1_sorted["rank_within_interval_condition"] == 1]
    .copy()
    .reset_index(drop=True)
)

print("\nBest Pipeline 1 HDBSCAN runs (1 per interval x condition):", hdb_p1_best6)
display(hdb_p1_best6)

# 8) Save as the "best pipeline 1 only" table
best_p1_path = os.path.join(output_dir, "hdbscan_best_pipeline1_only.csv")

```



```
hdb_p1_best6.to_csv(best_p1_path, index=False)  
print("\nSaved best Pipeline 1 HDBSCAN runs →", best_p1_path)
```

Loading: /content/drive/MyDrive/preeclampsia_results/hdbscan_eval_scored.csv
Original eval shape: (144, 25)
After filtering algorithm == 'hdbscan': (144, 25)
Pipeline 1 only: (36, 25)
Using 'composite_eval' for ranking within P1.
With ranking column (Pipeline 1):

	interval	condition	pipeline	silhouette	composite_eval	rank_within_interval_c
0	1-10w	0	PCA20 (P1, base)	0.388371	-3.591601	
6	1-10w	0	PCA20 (P1, HI-only improvements)	0.388371	-3.591601	
8	1-10w	0	PCA20 (P1, merged base+HI)	0.388371	-3.591601	
10	1-10w	0	PCA20 (P1, merged base+HI)	0.388371	-3.591601	
48	1-10w	0	PCA20 (P1, base)	0.388371	-3.591601	
54	1-10w	0	PCA20 (P1, HI-only improvements)	0.388371	-3.591601	
56	1-10w	0	PCA20 (P1, merged base+HI)	0.388371	-3.591601	
58	1-10w	0	PCA20 (P1, merged base+HI)	0.388371	-3.591601	
96	1-10w	0	PCA20 (P1, base)	0.388371	-3.591601	
102	1-10w	0	PCA20 (P1, HI-only improvements)	0.388371	-3.591601	
104	1-10w	0	PCA20 (P1, merged base+HI)	0.388371	-3.591601	
106	1-10w	0	PCA20 (P1, merged base+HI)	0.388371	-3.591601	
1	1-10w	1	PCA20 (P1, base)	0.474656	-1.900871	
49	1-10w	1	PCA20 (P1, base)	0.474656	-1.900871	
97	1-10w	1	PCA20 (P1, base)	0.474656	-1.900871	
2	11-20w	0	PCA20 (P1, base)	0.367416	-4.808738	
7	11-20w	0	PCA20 (P1, HI-only improvements)	0.367416	-4.808738	

	interval	condition	pipeline	silhouette	composite_eval	rank_within_interval_c
9	11-20w	0	PCA20 (P1, merged base+HI)	0.367416	-4.808738	
11	11-20w	0	PCA20 (P1, merged base+HI)	0.367416	-4.808738	
50	11-20w	0	PCA20 (P1, base)	0.367416	-4.808738	
55	11-20w	0	PCA20 (P1, HI-only improvements)	0.367416	-4.808738	
57	11-20w	0	PCA20 (P1, merged base+HI)	0.367416	-4.808738	
59	11-20w	0	PCA20 (P1, merged base+HI)	0.367416	-4.808738	
98	11-20w	0	PCA20 (P1, base)	0.367416	-4.808738	
103	11-20w	0	PCA20 (P1, HI-only improvements)	0.367416	-4.808738	
105	11-20w	0	PCA20 (P1, merged base+HI)	0.367416	-4.808738	
107	11-20w	0	PCA20 (P1, merged base+HI)	0.367416	-4.808738	
3	11-20w	1	PCA20 (P1, base)	0.567595	0.642299	
51	11-20w	1	PCA20 (P1, base)	0.567595	0.642299	
99	11-20w	1	PCA20 (P1, base)	0.567595	0.642299	
4	21-30w	0	PCA20 (P1, base)	0.375740	-4.592081	
52	21-30w	0	PCA20 (P1, base)	0.375740	-4.592081	
100	21-30w	0	PCA20 (P1, base)	0.375740	-4.592081	
5	21-30w	1	PCA20 (P1, base)	0.355354	-4.435635	
53	21-30w	1	PCA20 (P1, base)	0.355354	-4.435635	

	interval	condition	pipeline	silhouette	composite_eval	rank_within_interval_c
101	21-30w	1	PCA20 (P1, base)	0.355354	-4.435635	

Best Pipeline 1 HDBSCAN runs (1 per interval × condition): (6, 26)

	interval	condition	pipeline	algorithm	space	umap_dim	min_cluster_size	min_samp
0	1-10w	0	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
1	1-10w	1	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
2	11-20w	0	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
3	11-20w	1	PCA20 (P1, base)	HDBSCAN	pca20	NaN	10	
4	21-30w	0	PCA20 (P1, base)	HDBSCAN	pca20	NaN	60	
5	21-30w	1	PCA20 (P1, base)	HDBSCAN	pca20	NaN	40	

6 rows × 26 columns

Saved best Pipeline 1 HDBSCAN runs → /content/drive/MyDrive/preeclampsia_results/hdbscan_best_pipeline1_only.csv

visualization of evaluation metrics for hdbscan

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"
eval_path = os.path.join(output_dir, "hdbscan_eval_scored.csv")

hdb = pd.read_csv(eval_path)

# Filter to HDBSCAN runs only
hdb = hdb[hdb["algorithm"].str.lower() == "hdbscan"].copy()

# Simplify pipeline
def simplify(p):
    p = str(p)
    if "PCA20" in p or "P1" in p:
        return "Pipeline 1"
    if "UMAP" in p or "P2" in p:
        return "Pipeline 2"
    return "Other"

hdb["pipeline_simple"] = hdb["pipeline"].apply(simplify)
```

```

metrics = ["silhouette", "davies_bouldin", "calinski_harabasz", "composite_eval"]

plt.figure(figsize=(12, 8))

for i, metric in enumerate(metrics, 1):
    plt.subplot(2, 2, i)
    sns.violinplot(
        data=hdb,
        x="pipeline_simple",
        y=metric,
        palette=["#1f77b4", "#ff7f0e"]
    )
    sns.stripplot(
        data=hdb,
        x="pipeline_simple",
        y=metric,
        color="black",
        size=2,
        jitter=0.15,
        alpha=0.5
    )
    plt.title(metric.replace("_", " ").title())

plt.tight_layout()
plt.show()

```

/tmp/ipython-input-3271589215.py:26: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.violinplot(
/tmp/ipython-input-3271589215.py:26: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

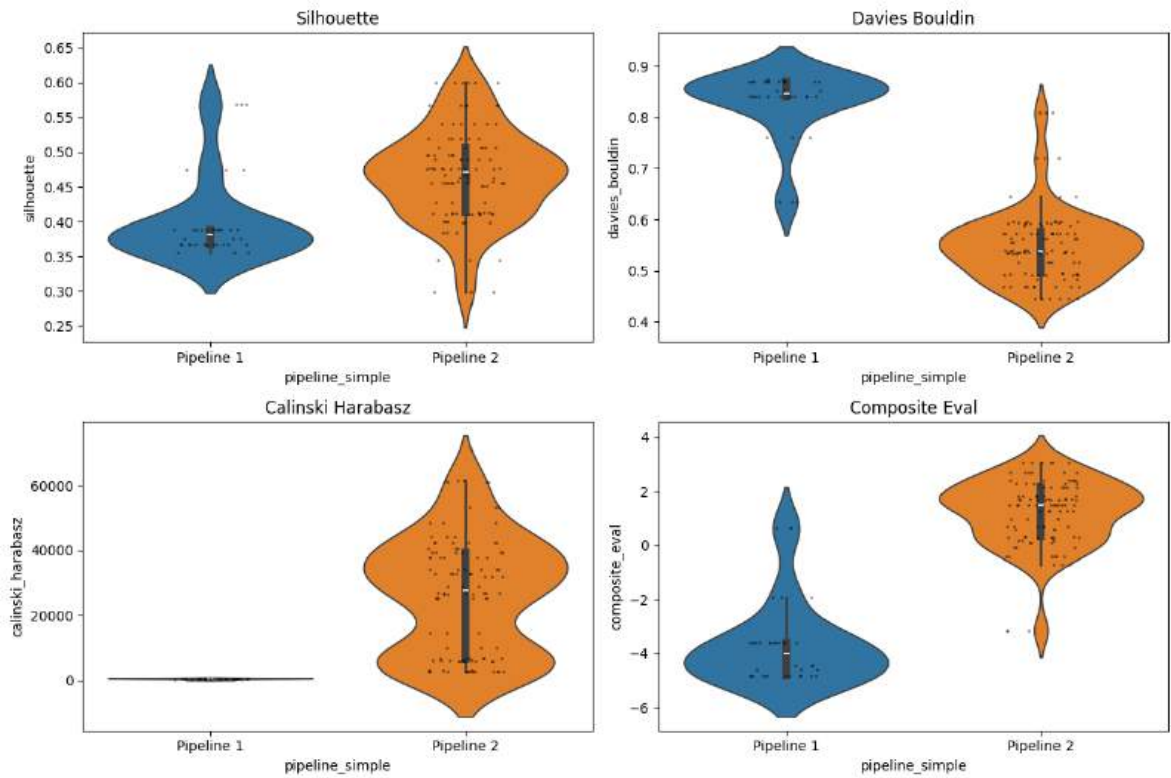
```
sns.violinplot(
/tmp/ipython-input-3271589215.py:26: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.violinplot(
/tmp/ipython-input-3271589215.py:26: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.violinplot(
```



Bar plot view

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

# Load both tables

p1_path = os.path.join(output_dir, "hdbscan_best_pipeline1_only.csv")
across_path = os.path.join(output_dir, "hdbscan_best_per_interval_across_pipeline1.csv")

p1 = pd.read_csv(p1_path)
across = pd.read_csv(across_path)

print("Loaded P1:", p1.shape)
print("Loaded Across:", across.shape)

# Build readable labels
COND_NAME = {0: "Control", 1: "PE"}
p1["interval_cond"] = p1.apply(lambda r: f"{r['interval']} | {COND_NAME[int(r['condition'])]}", axis=1)
across["interval_cond"] = across.apply(lambda r: f"{r['interval']} | {COND_NAME[int(r['condition'])]}", axis=1)

# Ensure ordering is consistent
p1 = p1.sort_values(["interval", "condition"])
across = across.sort_values(["interval", "condition"])

metrics = ["silhouette", "davies_bouldin", "calinski_harabasz", "composite_eval"]

# Plot side-by-side grouped bars
for metric in metrics:

    if metric not in p1.columns or metric not in across.columns:
        print(f"Skipping {metric} - not found.")
        continue

    fig, ax = plt.subplots(figsize=(12,5))
```

```

labels = p1["interval_cond"].tolist() # same intervals for both
x = np.arange(len(labels))           # the label locations
width = 0.35                         # width of each bar

# Pipeline 1 bars (left bars)
ax.bar(x - width/2, p1[metric], width, label="Pipeline 1 (PCA20)")

# Best across pipelines bars (right bars)
ax.bar(x + width/2, across[metric], width, label="Best Across Pipelines")

# Labeling
ax.set_title(f"HDBSCAN - {metric.replace('_', ' ').title()} - P1 vs Across Pipelines")
ax.set_ylabel(metric.replace("_", " ").title())
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=45, ha="right")

ax.legend()

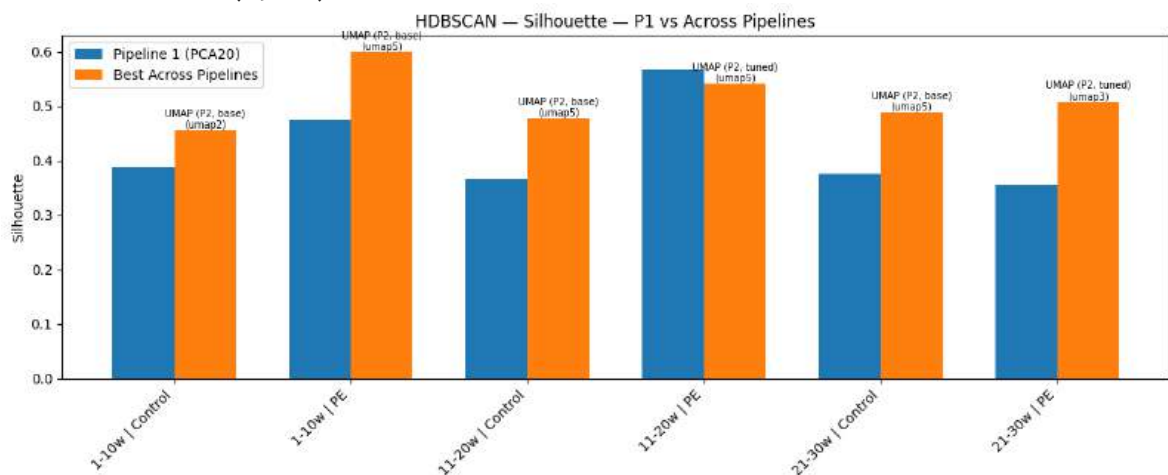
# annotate across-pipeline bars with actual pipeline/space
for i, (pipe, space, val) in enumerate(zip(across["pipeline"],
                                           across.get("space", [""] * len(across[metric]))):
    label = pipe
    if isinstance(space, str) and len(space) > 0:
        label += f"\n({space})"
    ax.text(i + width/2, val, label,
            ha="center", va="bottom", fontsize=7)

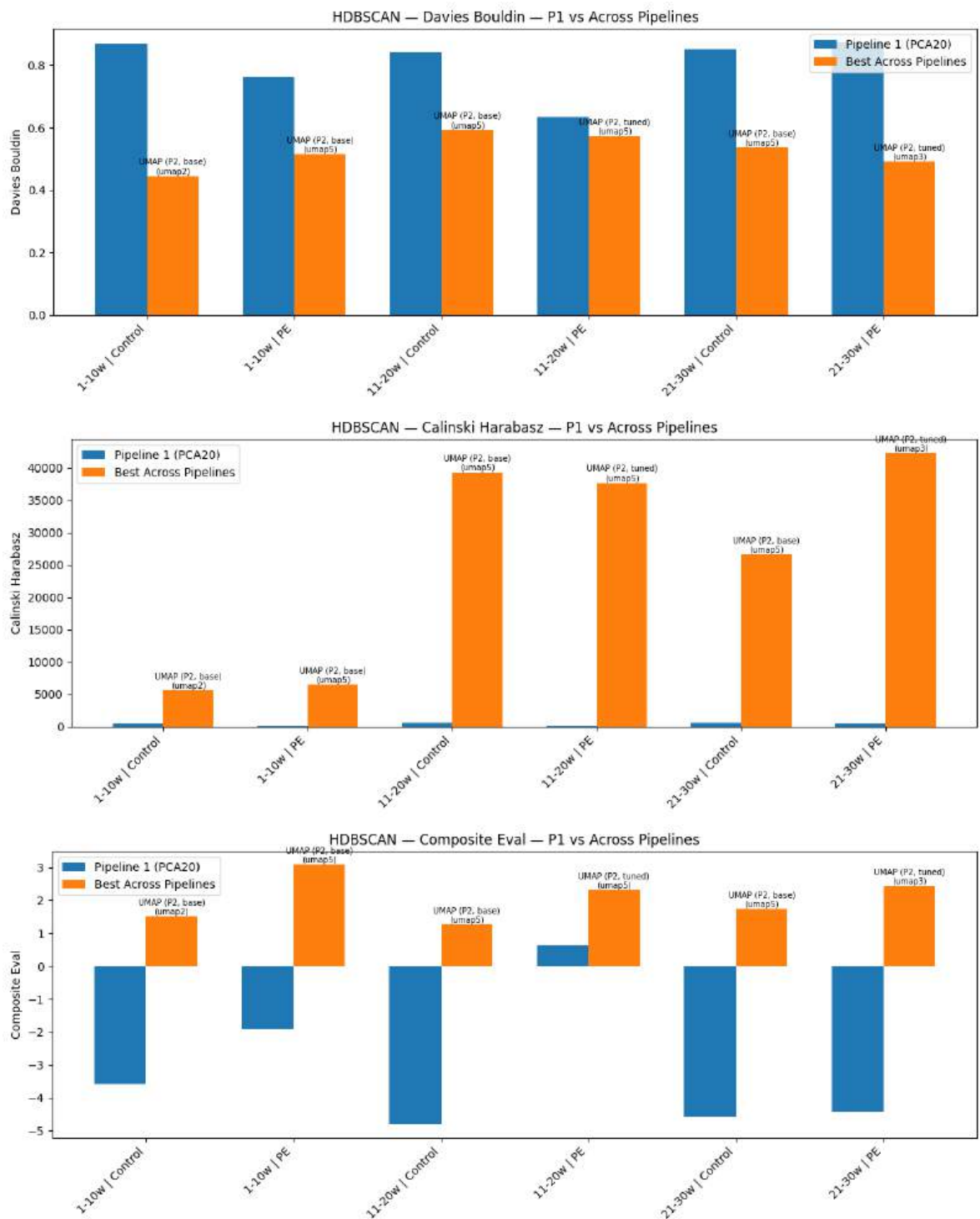
plt.tight_layout()
plt.show()

```

Loaded P1: (6, 26)

Loaded Across: (6, 28)





Evaluation of phenograph's performance (SH, CH, DB)

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

# Load inventory

inv_path = os.path.join(output_dir, "phenograph_inventory.csv")
print("Loading:", inv_path)

ph = pd.read_csv(inv_path)
print("Inventory shape:", ph.shape)
display(ph.head())

# Filter to only actual PhenoGraph algorithm rows
```

```
ph = ph[ph["algorithm"].str.lower() == "phenograph"].copy()
print("After filtering PhenoGraph:", ph.shape)
```

Loading: /content/drive/MyDrive/preeclampsia_results/phenograph_inventory.csv
Inventory shape: (96, 13)

	interval	condition	n_cells	k	silhouette	modularity_Q	composite
0	1-10w	0	15000	15	0.065217	0.860775	0.307682
1	1-10w	0	15000	30	0.071296	0.841977	0.306810
2	1-10w	0	15000	50	0.082086	0.835297	0.312541
3	1-10w	0	15000	75	0.095394	0.828500	0.320003 /content/drive/My
4	1-10w	1	5000	15	0.110356	0.870026	0.341802 /content/drive/My

After filtering PhenoGraph: (96, 13)

```
In [ ]: def load_labels(csv_path):
    df = pd.read_csv(csv_path)
    labels = df.iloc[:, -1].to_numpy()
    return labels

# Helper: infer embedding path
def get_embedding_path(row):
    interval = str(row["interval"])
    cond = int(row["condition"])
    tag = f"{interval}_{cond}"

    pipeline = str(row["pipeline"])
    umap_dim = row.get("umap_dim", np.nan)

    # Pipeline 1 → PCA20
    if "PCA20" in pipeline:
        return os.path.join(output_dir, f"pca20_{tag}.npy"), "pca20", np.nan

    # Pipeline 2 → UMAP
    if not pd.isna(umap_dim):
        d = int(umap_dim)
    else:
        # fallback from plot name
        fname = str(row.get("plot_png", ""))
        if "umap2" in fname: d = 2
        elif "umap3" in fname: d = 3
        elif "umap5" in fname: d = 5
        else:
            return None, None, np.nan

    return os.path.join(output_dir, f"umap{d}_pipeline2_{tag}.npy"), f"umap{d}"

# Helper: compute metrics
def compute_metrics(X, labels):
```

```

X = np.asarray(X)
labels = np.asarray(labels)

if len(np.unique(labels)) < 2:
    return np.nan, np.nan, np.nan, len(np.unique(labels))

try:
    sil = float(silhouette_score(X, labels))
except:
    sil = np.nan

try:
    db = float(davies_bouldin_score(X, labels))
except:
    db = np.nan

try:
    ch = float(calinski_harabasz_score(X, labels))
except:
    ch = np.nan

return sil, db, ch, len(np.unique(labels))

# Compute all metrics
rows = []

for i, r in ph.iterrows():
    interval = r["interval"]
    cond = int(r["condition"])
    pipeline = r["pipeline"]
    algo = r["algorithm"]

    labels_csv = r.get("labels_csv", None)
    if not isinstance(labels_csv, str) or not os.path.exists(labels_csv):
        print(f"[PhenoGraph] Missing labels for {interval}_{cond} ({pipeline})")
        continue

    emb_path, space, dim = get_embedding_path(r)
    if emb_path is None or not os.path.exists(emb_path):
        print(f"[PhenoGraph] Missing embedding for {interval}_{cond} ({pipeline})")
        continue

    X = np.load(emb_path)
    labels = load_labels(labels_csv)

    if len(X) != len(labels):
        print(f"[PhenoGraph] Length mismatch for {interval}_{cond}: X={len(X)},")
        continue

    sil, db, ch, n_clusters = compute_metrics(X, labels)

    rows.append({
        "interval": interval,
        "condition": cond,
        "pipeline": pipeline,
        "space": space,
        "umap_dim": dim,
        "k": r.get("k", np.nan),
        "n_cells": len(labels),
        "n_clusters": n_clusters,

```

```

        "silhouette": sil,
        "davies_bouldin": db,
        "calinski_harabasz": ch,
        "modularity_Q": r.get("modularity_Q", np.nan),
        "composite_old": r.get("composite", np.nan),
        "plot_png": r.get("plot_png", np.nan),
        "labels_csv": labels_csv
    })

ph_eval = pd.DataFrame(rows)
print("PhenoGraph evaluation table:", ph_eval.shape)
display(ph_eval.head())

# Compute composite_eval (same as KMeans / HDBSCAN)
def _z(x):
    x = x.astype(float)
    m, s = x.mean(), x.std(ddof=0)
    if s == 0 or np.isnan(s):
        return x*0.0
    return (x - m)/s

ph_eval["sh_z"] = _z(ph_eval["silhouette"])
ph_eval["db_z"] = _z(ph_eval["davies_bouldin"])
ph_eval["ch_log"] = np.log1p(ph_eval["calinski_harabasz"])
ph_eval["ch_z"] = _z(ph_eval["ch_log"])
ph_eval["noise_z"] = 0 # PhenoGraph has no noise

ph_eval["composite_eval"] = (
    ph_eval["sh_z"]
    - ph_eval["db_z"]
    + 0.5 * ph_eval["ch_z"]
)

# Save

eval_path = os.path.join(output_dir, "phenograph_eval_scored.csv")
ph_eval.to_csv(eval_path, index=False)

print("\nSaved PhenoGraph scored table →", eval_path)

```

593/662

[PhenoGraph] Missing labels for 21-30w_0 (UMAP (P2, base))
 [PhenoGraph] Missing labels for 21-30w_0 (UMAP (P2, base))
 [PhenoGraph] Missing labels for 21-30w_0 (UMAP (P2, base))
 [PhenoGraph] Missing labels for 21-30w_1 (UMAP (P2, base))
 [PhenoGraph] Missing labels for 21-30w_1 (UMAP (P2, base))
 [PhenoGraph] Missing labels for 21-30w_1 (UMAP (P2, base))
 [PhenoGraph] Missing labels for 21-30w_1 (UMAP (P2, base))
 [PhenoGraph] Missing labels for 21-30w_1 (UMAP (P2, base))
 [PhenoGraph] Missing labels for 21-30w_1 (UMAP (P2, base))
 [PhenoGraph] Missing labels for 21-30w_1 (UMAP (P2, base))
 [PhenoGraph] Missing labels for 21-30w_1 (UMAP (P2, base))
 PhenoGraph evaluation table: (24, 15)

	interval	condition	pipeline	space	umap_dim	k	n_cells	n_clusters	silhouette	da
0	1-10w	0	PCA20 (P1)	pca20	NaN	75	15000	17	0.095394	
1	1-10w	1	PCA20 (P1)	pca20	NaN	15	5000	21	0.110356	
2	11-20w	0	PCA20 (P1)	pca20	NaN	50	55000	21	0.075342	
3	11-20w	1	PCA20 (P1)	pca20	NaN	75	65000	19	0.116412	
4	21-30w	0	PCA20 (P1)	pca20	NaN	15	50000	28	0.072615	

Saved PhenoGraph scored table → /content/drive/MyDrive/preeclampsia_results/pheno
 graph_eval_scored.csv

```
In [ ]: # Load full scored table
eval_path = os.path.join(output_dir, "phenograph_eval_scored.csv")
print("Loading:", eval_path)

ph = pd.read_csv(eval_path)
print("Eval scored table:", ph.shape)
display(ph.head())

# Filter to PhenoGraph only

if "algorithm" in ph.columns:
    ph = ph[ph["algorithm"].str.lower() == "phenograph"].copy()

print("After filtering phenograph:", ph.shape)

# Keep only rows that have composite_eval
ph = ph.dropna(subset=["silhouette", "composite_eval"])
print("After dropping missing metrics:", ph.shape)

# Sort within each interval/condition
# Ranking order:
# 1) silhouette (descending)
# 2) composite_eval (descending)

ph_sorted = ph.sort_values(
    by=["interval", "condition", "silhouette", "composite_eval"],
```

```

    ascending=[True, True, False, False]
).copy()

# Assign rank within group

ph_sorted["rank_within_interval_condition"] = (
    ph_sorted.groupby(["interval", "condition"])
        .cumcount() + 1
)

print("With ranks:")
display(ph_sorted[
    ["interval", "condition", "pipeline", "space",
     "silhouette", "composite_eval", "rank_within_interval_condition"]
])

# 6. Keep only rank == 1
ph_best6 = ph_sorted[ph_sorted["rank_within_interval_condition"] == 1] \
    .copy() \
    .reset_index(drop=True)

print("\nBest 1 per (interval, condition):", ph_best6.shape)
display(ph_best6)

# Save → overwrite file
save_path = os.path.join(output_dir, "phenograph_best_per_interval_across_pipeli
ph_best6.to_csv(save_path, index=False)

print("\nSaved Best-Across-Pipelines →", save_path)

```

Loading: /content/drive/MyDrive/preeclampsia_results/phenograph_eval_scored.csv
 Eval scored table: (24, 21)

	interval	condition	pipeline	space	umap_dim	k	n_cells	n_clusters	silhouette	da
0	1–10w	0	PCA20 (P1)	pca20	NaN	75	15000	17	0.095394	
1	1–10w	1	PCA20 (P1)	pca20	NaN	15	5000	21	0.110356	
2	11–20w	0	PCA20 (P1)	pca20	NaN	50	55000	21	0.075342	
3	11–20w	1	PCA20 (P1)	pca20	NaN	75	65000	19	0.116412	
4	21–30w	0	PCA20 (P1)	pca20	NaN	15	50000	28	0.072615	

5 rows × 21 columns



After filtering phenograph: (24, 21)

After dropping missing metrics: (24, 21)

With ranks:

	interval	condition	pipeline	space	silhouette	composite_eval	rank_within_interval
12	11–20w	0	UMAP (P2, base)	umap2	0.371096	2.307720	
13	11–20w	0	UMAP (P2, base)	umap3	0.328375	1.456089	
14	11–20w	0	UMAP (P2, base)	umap5	0.285824	0.646571	
2	11–20w	0	PCA20 (P1)	pca20	0.075342	-4.217364	
15	11–20w	1	UMAP (P2, base)	umap2	0.337713	1.865419	
16	11–20w	1	UMAP (P2, base)	umap3	0.334445	1.655044	
17	11–20w	1	UMAP (P2, base)	umap5	0.287421	0.819183	
3	11–20w	1	PCA20 (P1)	pca20	0.116412	-3.681553	
6	1–10w	0	UMAP (P2, base)	umap2	0.358728	1.611654	
7	1–10w	0	UMAP (P2, base)	umap3	0.333491	1.127841	
8	1–10w	0	UMAP (P2, base)	umap5	0.275099	0.045359	
0	1–10w	0	PCA20 (P1)	pca20	0.095394	-3.861070	
9	1–10w	1	UMAP (P2, base)	umap2	0.433922	2.400763	
10	1–10w	1	UMAP (P2, base)	umap3	0.379174	1.409856	
11	1–10w	1	UMAP (P2, base)	umap5	0.361457	1.023708	
1	1–10w	1	PCA20 (P1)	pca20	0.110356	-4.274932	

	interval	condition	pipeline	space	silhouette	composite_eval	rank_within_interval_
18	21–30w	0	UMAP (P2, base)	umap2	0.363016	2.286687	
19	21–30w	0	UMAP (P2, base)	umap3	0.328226	1.462250	
20	21–30w	0	UMAP (P2, base)	umap5	0.295979	0.821629	
4	21–30w	0	PCA20 (P1)	pca20	0.072615	-4.290626	
21	21–30w	1	UMAP (P2, base)	umap2	0.339850	1.865267	
22	21–30w	1	UMAP (P2, base)	umap3	0.309795	1.145754	
23	21–30w	1	UMAP (P2, base)	umap5	0.264265	0.310080	
5	21–30w	1	PCA20 (P1)	pca20	0.099878	-3.935332	

Best 1 per (interval, condition): (6, 22)

	interval	condition	pipeline	space	umap_dim	k	n_cells	n_clusters	silhouette	da
0	11–20w	0	UMAP (P2, base)	umap2	2.0	75	55000	35	0.371096	
1	11–20w	1	UMAP (P2, base)	umap2	2.0	75	65000	37	0.337713	
2	1–10w	0	UMAP (P2, base)	umap2	2.0	30	15000	38	0.358728	
3	1–10w	1	UMAP (P2, base)	umap2	2.0	75	5000	19	0.433922	
4	21–30w	0	UMAP (P2, base)	umap2	2.0	75	50000	39	0.363016	
5	21–30w	1	UMAP (P2, base)	umap2	2.0	50	40000	42	0.339850	

6 rows × 22 columns

◀ Saved Best-Across-Pipelines → /content/drive/MyDrive/preeclampsia_results/phenograph_best_per_interval_across_pipelines.csv ▶

```
In [ ]: # Load PhenoGraph scored table
eval_path = os.path.join(output_dir, "phenograph_eval_scored.csv")
print("Loading:", eval_path)
ph = pd.read_csv(eval_path)
print("Columns in phenograph_eval_scored.csv:", list(ph.columns))
print("Shape:", ph.shape)

# Make sure we have an 'algorithm' column
if "algorithm" not in ph.columns:
    ph["algorithm"] = "PhenoGraph"

# Make sure we have a 'pipeline' column
# If your inventory used e.g. 'pipeline' already, this will just pass.
# If not, you can adapt this to your actual column name.
if "pipeline" not in ph.columns:
    raise KeyError(
        "No 'pipeline' column found in phenograph_eval_scored.csv. "
        "Check the printed column names above and adjust the code "
        "to use the correct column (e.g. 'space' or similar).")

# Filter PhenoGraph + Pipeline 1 (anything containing 'P1')
ph_p1 = ph[
    (ph["algorithm"].str.lower() == "phenograph") &
    (ph["pipeline"].astype(str).str.contains("P1"))
].copy()

print("Pipeline 1 rows (PhenoGraph):", ph_p1.shape)
```

```

# Drop rows missing metrics
ph_p1 = ph_p1.dropna(subset=["silhouette", "composite_eval"])

# Sort inside interval x condition
ph_p1_sorted = ph_p1.sort_values(
    by=["interval", "condition", "silhouette", "composite_eval"],
    ascending=[True, True, False, False]
).copy()

# Rank within each (interval, condition)
ph_p1_sorted["rank_within_interval_condition"] = (
    ph_p1_sorted.groupby(["interval", "condition"])
        .cumcount() + 1
)

# Keep rank 1 only - best P1 run for each interval+condition
ph_p1_best = (
    ph_p1_sorted[ph_p1_sorted["rank_within_interval_condition"] == 1]
    .reset_index(drop=True)
)

print("Best pipeline 1 (one per interval x condition):", ph_p1_best.shape)
display(ph_p1_best)

# Save
save_path = os.path.join(output_dir, "phenograph_best_pipeline1_only.csv")
ph_p1_best.to_csv(save_path, index=False)
print("Saved →", save_path)

```

Loading: /content/drive/MyDrive/preeclampsia_results/phenograph_eval_scored.csv
Columns in phenograph_eval_scored.csv: ['interval', 'condition', 'pipeline', 'space', 'umap_dim', 'k', 'n_cells', 'n_clusters', 'silhouette', 'davies_bouldin', 'calinski_harabasz', 'modularity_Q', 'composite_old', 'plot_png', 'labels_csv', 'sh_z', 'db_z', 'ch_log', 'ch_z', 'noise_z', 'composite_eval']
Shape: (24, 21)
Pipeline 1 rows (PhenoGraph): (6, 22)
Best pipeline 1 (one per interval x condition): (6, 23)

	interval	condition	pipeline	space	umap_dim	k	n_cells	n_clusters	silhouette	da
0	11–20w	0	PCA20 (P1)	pca20	NaN	50	55000	21	0.075342	
1	11–20w	1	PCA20 (P1)	pca20	NaN	75	65000	19	0.116412	
2	1–10w	0	PCA20 (P1)	pca20	NaN	75	15000	17	0.095394	
3	1–10w	1	PCA20 (P1)	pca20	NaN	15	5000	21	0.110356	
4	21–30w	0	PCA20 (P1)	pca20	NaN	15	50000	28	0.072615	
5	21–30w	1	PCA20 (P1)	pca20	NaN	50	40000	23	0.099878	

6 rows × 23 columns

◀  ▶
 Saved → /content/drive/MyDrive/preeclampsia_results/phenograph_best_pipeline1_only.csv

violin plot best performing phenograph clusters per pipeline

```
In [ ]: # Load full PhenoGraph eval-scored table
eval_path = os.path.join(output_dir, "phenograph_eval_scored.csv")
ph = pd.read_csv(eval_path)
print("PhenoGraph eval-scored:", ph.shape)
display(ph.head())

# If there's no algorithm column, just assume everything is PhenoGraph
if "algorithm" in ph.columns:
    ph = ph[ph["algorithm"].str.lower() == "phenograph"].copy()
    print("After filtering algorithm == phenograph:", ph.shape)

# Normalise pipeline labels to simple P1 / P2
def simplify_pipeline(name: str) -> str:
    s = str(name)
    if "P1" in s or "PCA20" in s:
        return "P1"
    if "P2" in s or "UMAP" in s:
        return "P2"
    return "Other"

ph["pipeline_simple"] = ph["pipeline"].apply(simplify_pipeline)

print(ph["pipeline_simple"].value_counts())

metrics = ["silhouette", "davies_bouldin", "calinski_harabasz", "composite_eval"]

# Violin plots: P1 vs P2 for each metric
sns.set(style="whitegrid")

for metric in metrics:
    if metric not in ph.columns:
        print(f"Skipping {metric} – not found in phenograph_eval_scored.csv")
```

```
continue

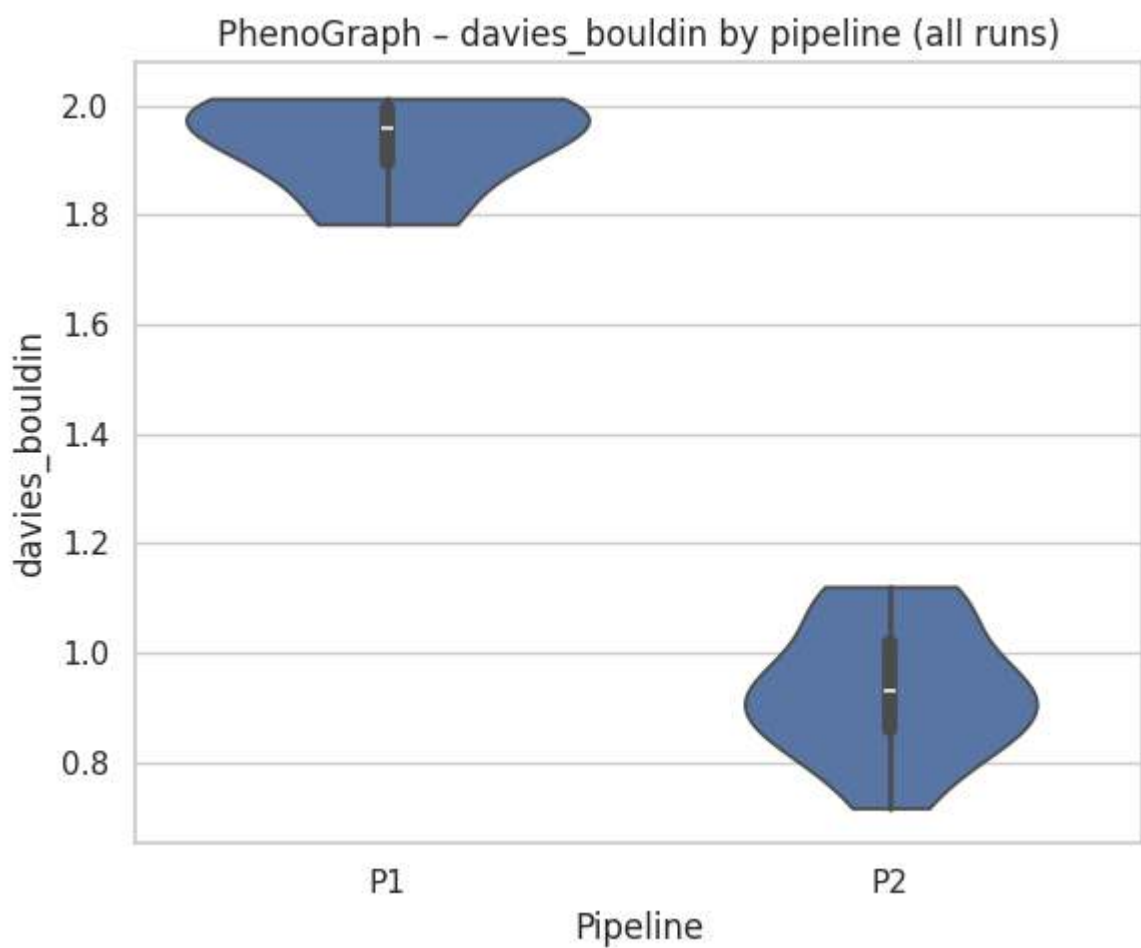
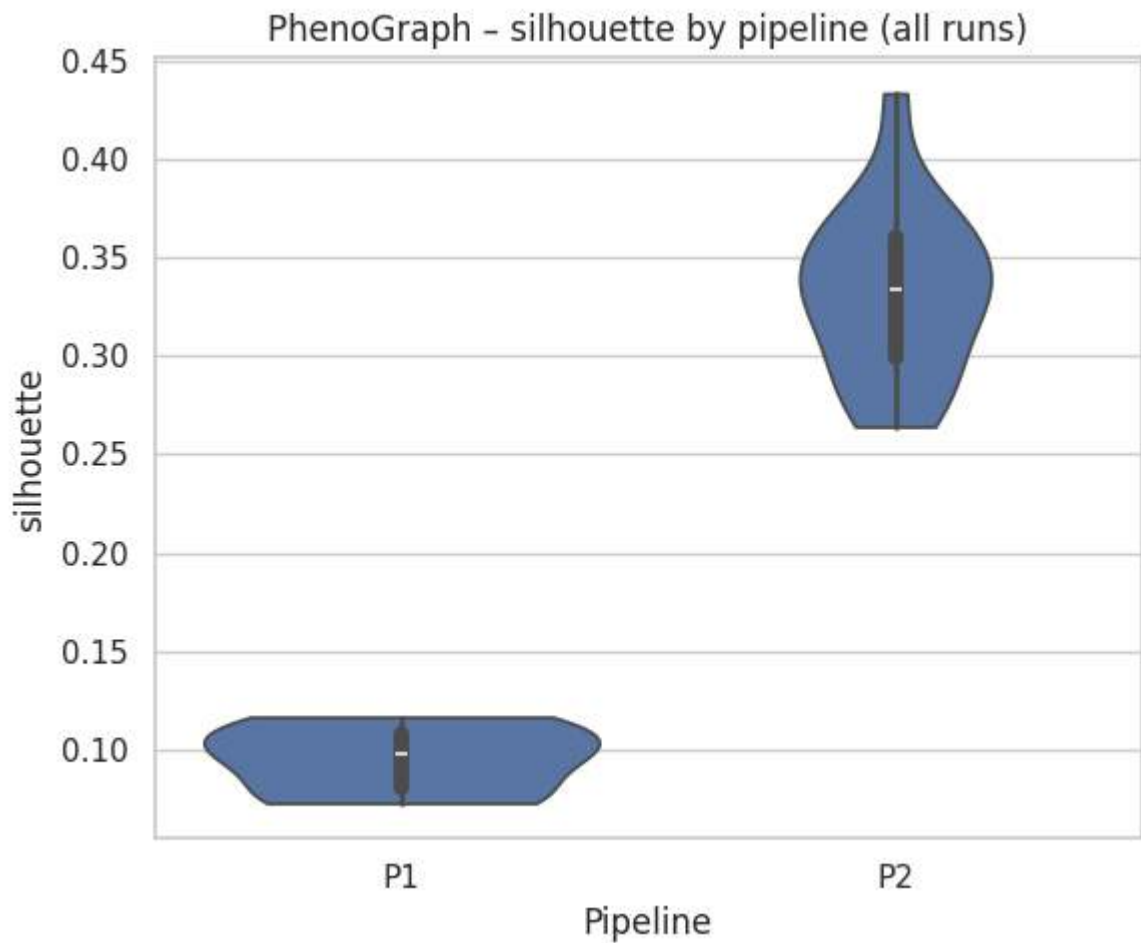
plt.figure(figsize=(6, 5))
sns.violinplot(
    data=ph,
    x="pipeline_simple",
    y=metric,
    order=["P1", "P2"],
    cut=0
)
plt.title(f"PhenoGraph - {metric} by pipeline (all runs)")
plt.xlabel("Pipeline")
plt.ylabel(metric)
plt.tight_layout()
plt.show()
```

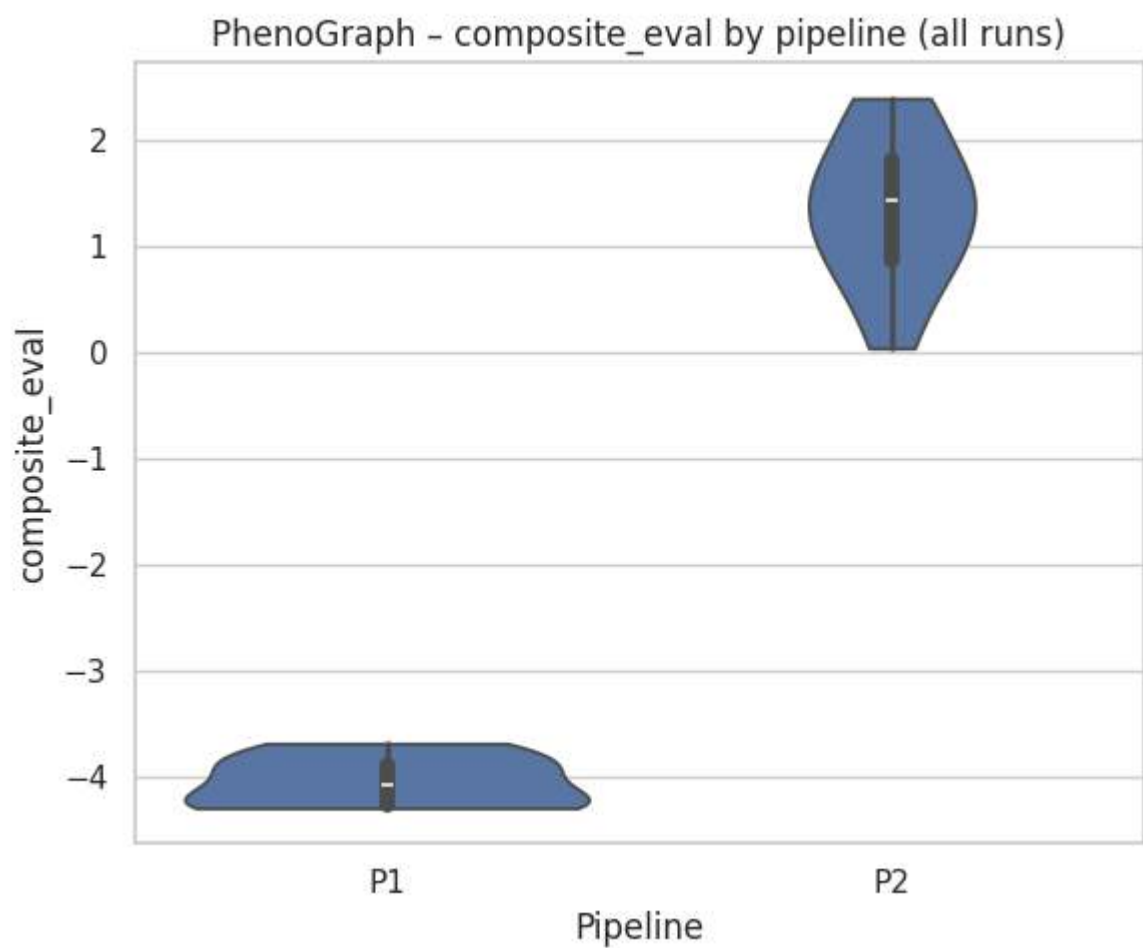
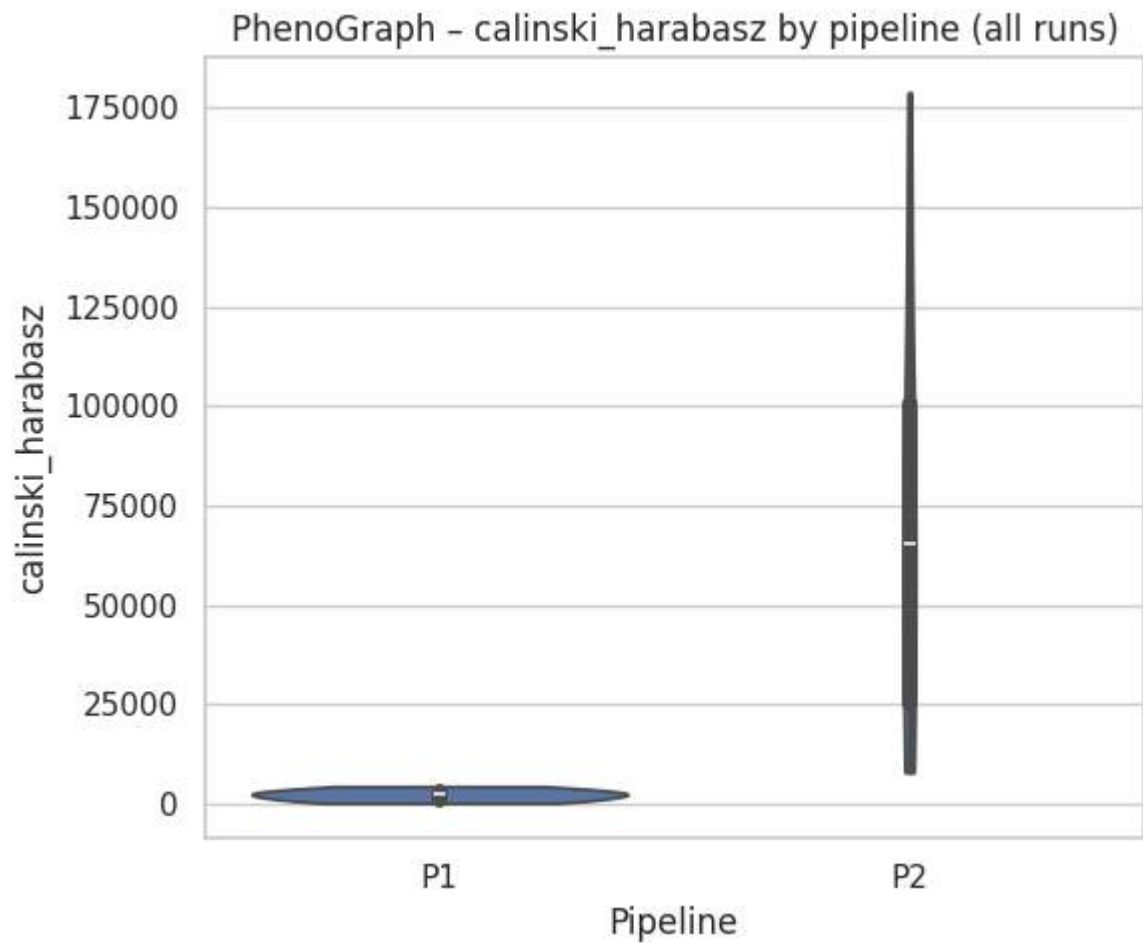
PhenoGraph eval-scored: (24, 21)

	interval	condition	pipeline	space	umap_dim	k	n_cells	n_clusters	silhouette	day
0	1-10w	0	PCA20 (P1)	pca20	NaN	75	15000	17	0.095394	
1	1-10w	1	PCA20 (P1)	pca20	NaN	15	5000	21	0.110356	
2	11-20w	0	PCA20 (P1)	pca20	NaN	50	55000	21	0.075342	
3	11-20w	1	PCA20 (P1)	pca20	NaN	75	65000	19	0.116412	
4	21-30w	0	PCA20 (P1)	pca20	NaN	15	50000	28	0.072615	

5 rows × 21 columns

```
◀  ▶
pipeline_simple
P2    18
P1     6
Name: count, dtype: int64
```





Visualizing same with bar plot

```

In [ ]: # Load best tables
p1_best_path = os.path.join(output_dir, "phenograph_best_pipeline1_only.csv")
across_best_path = os.path.join(output_dir, "phenograph_best_per_interval_across")

ph_p1_best = pd.read_csv(p1_best_path)
ph_across_best = pd.read_csv(across_best_path)

print("P1 best:", ph_p1_best.shape)
print("Across-pipelines best:", ph_across_best.shape)

display(ph_p1_best.head())
display(ph_across_best.head())

# Make a common key for x-axis: interval + condition
def make_key(df):
    return df["interval"].astype(str) + " | cond=" + df["condition"].astype(str)

ph_p1_best["interval_cond"] = make_key(ph_p1_best)
ph_across_best["interval_cond"] = make_key(ph_across_best)

ph_p1_best["which"] = "P1 best"
ph_across_best["which"] = "Across best"

# Combine
ph_best = pd.concat([ph_p1_best, ph_across_best], ignore_index=True)

metrics = ["silhouette", "davies_bouldin", "calinski_harabasz", "composite_eval"]

# ---- Side-by-side bar plots: P1-best vs Across-best ----
sns.set(style="whitegrid")

for metric in metrics:
    if metric not in ph_best.columns:
        print(f"Skipping {metric} - not in phenograph_best_*.csv")
        continue

    plt.figure(figsize=(8, 5))

    # We'll pivot: rows = interval_cond, columns = which, values = metric
    pivot = ph_best.pivot_table(
        index="interval_cond",
        columns="which",
        values=metric
    )

    # Sort x-axis by interval for nicer ordering
    pivot = pivot.sort_index()

    x = np.arange(len(pivot.index))
    width = 0.35

    plt.bar(x - width/2, pivot["P1 best"], width, label="P1 best")
    plt.bar(x + width/2, pivot["Across best"], width, label="Best across pipelines")

    plt.xticks(x, pivot.index, rotation=45, ha="right")
    plt.ylabel(metric)
    plt.title(f"PhenoGraph - {metric}: P1 best vs Across-pipelines best")
    plt.legend()

```

```
plt.tight_layout()  
plt.show()
```

P1 best: (6, 23)

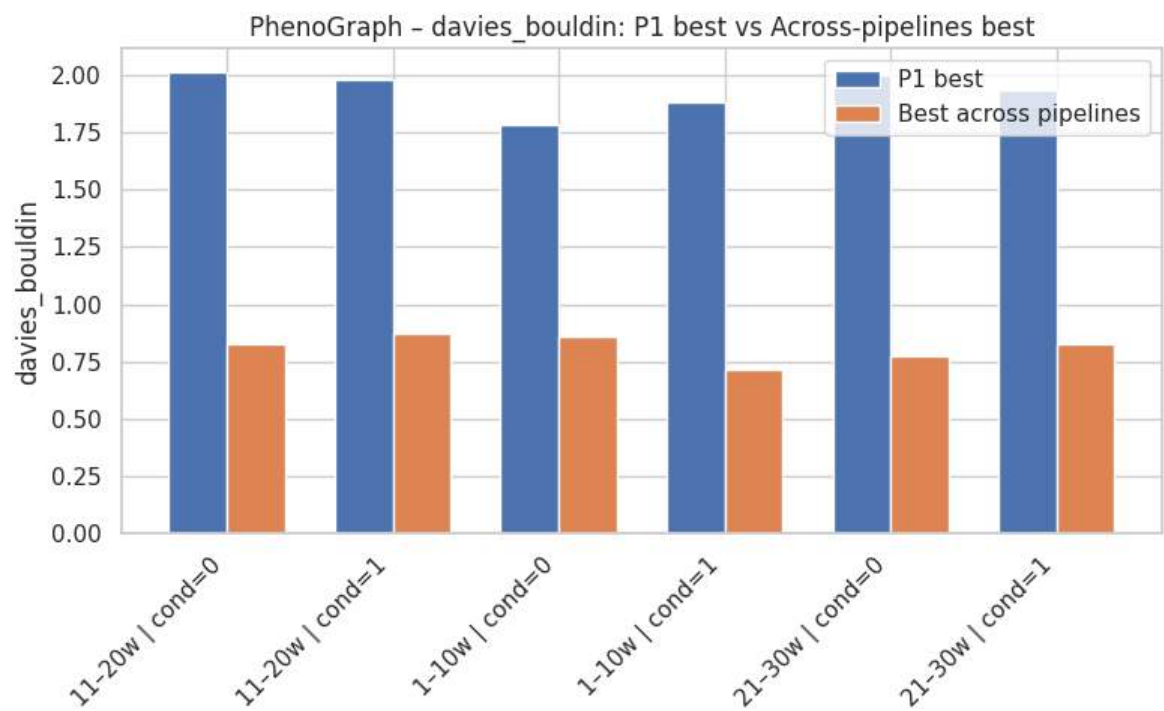
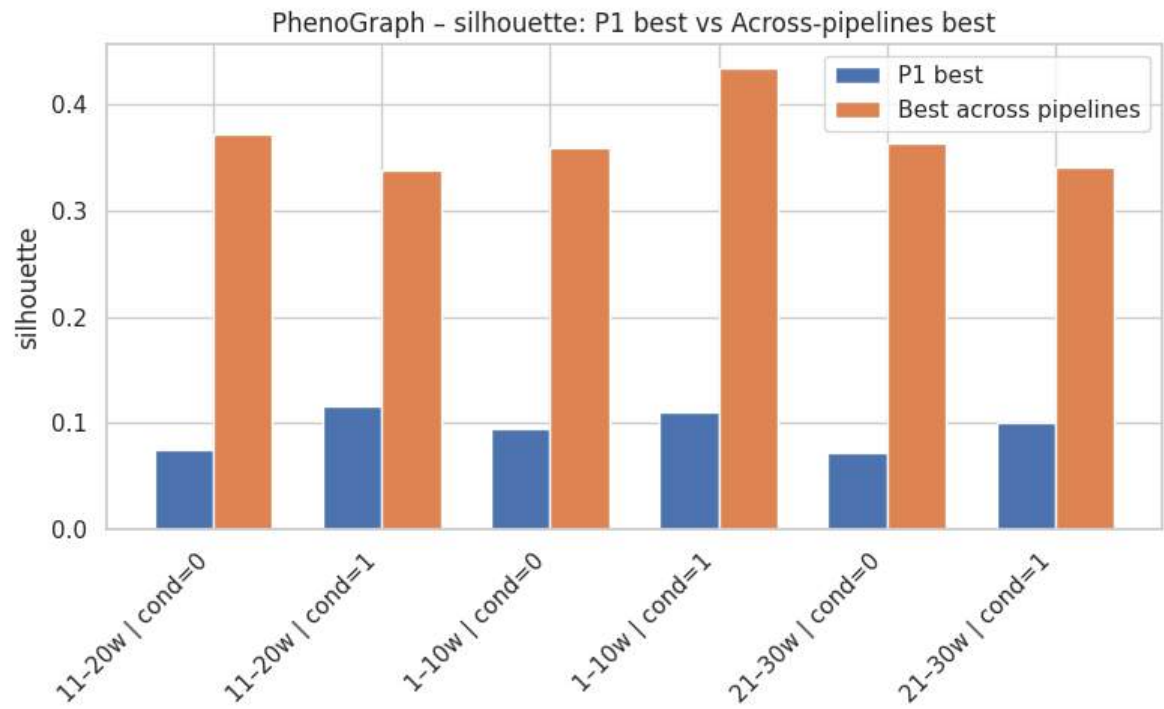
Across-pipelines best: (6, 22)

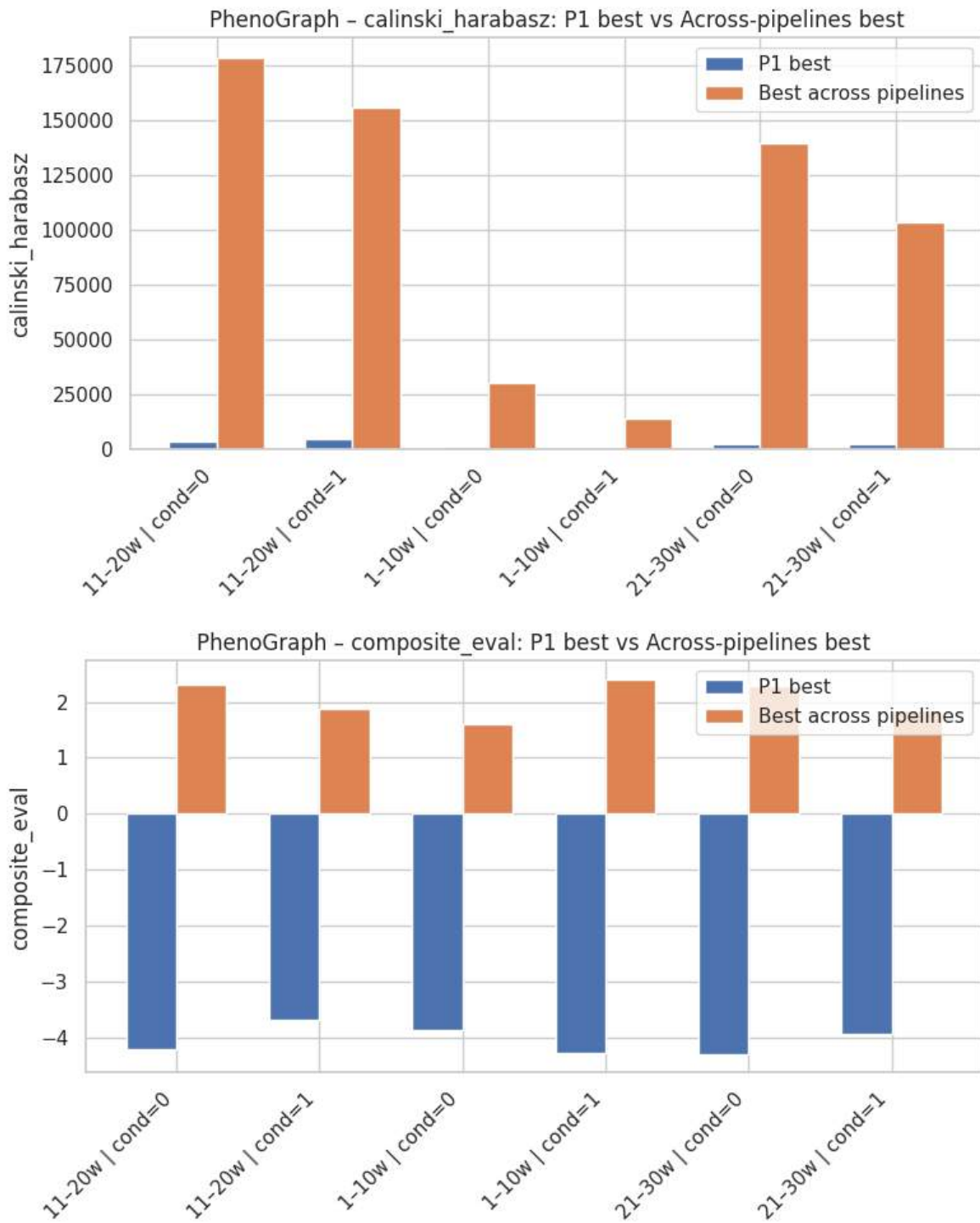
	interval	condition	pipeline	space	umap_dim	k	n_cells	n_clusters	silhouette	da
0	11–20w	0	PCA20 (P1)	pca20	NaN	50	55000	21	0.075342	
1	11–20w	1	PCA20 (P1)	pca20	NaN	75	65000	19	0.116412	
2	1–10w	0	PCA20 (P1)	pca20	NaN	75	15000	17	0.095394	
3	1–10w	1	PCA20 (P1)	pca20	NaN	15	5000	21	0.110356	
4	21–30w	0	PCA20 (P1)	pca20	NaN	15	50000	28	0.072615	

5 rows × 23 columns

	interval	condition	pipeline	space	umap_dim	k	n_cells	n_clusters	silhouette	da
0	11–20w	0	UMAP (P2, base)	umap2	2.0	75	55000	35	0.371096	
1	11–20w	1	UMAP (P2, base)	umap2	2.0	75	65000	37	0.337713	
2	1–10w	0	UMAP (P2, base)	umap2	2.0	30	15000	38	0.358728	
3	1–10w	1	UMAP (P2, base)	umap2	2.0	75	5000	19	0.433922	
4	21–30w	0	UMAP (P2, base)	umap2	2.0	75	50000	39	0.363016	

5 rows × 22 columns





Computing the best algorithms across board

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

files = {
    "kmeans": "kmeans_best_per_interval_across_pipelines.csv",
    "hdbscan": "hdbscan_best_per_interval_across_pipelines.csv",
    "phenograph": "phenograph_best_per_interval_across_pipelines.csv",
}

# Load and tag each algorithm's best table

dfs = []
for algo, fname in files.items():
```

```

path = os.path.join(output_dir, fname)
df = pd.read_csv(path)
df["algorithm"] = algo
dfs.append(df)

all_best = pd.concat(dfs, ignore_index=True)
print("Combined best table:", all_best.shape)

# Normalise metric column names if needed
metric_candidates = ["composite_eval", "composite_score", "composite_old"]
metric_col = None
for m in metric_candidates:
    if m in all_best.columns:
        metric_col = m
        break

if metric_col is None:
    raise ValueError("Could not find composite metric column in CSVs.")

print("Using composite column:", metric_col)

# Compute best-performing algorithm per metric
# Group by algorithm and take mean across all (interval, condition) winners
mean_scores = (
    all_best.groupby("algorithm")[["silhouette", metric_col]]
    .mean()
    .sort_values(by=metric_col, ascending=False)
)

print("\n=== Mean Scores Across All Conditions ===")
display(mean_scores)

# Identify global best-run for each metric

best_silhouette_row = all_best.loc[all_best["silhouette"].idxmax()]
best_composite_row = all_best.loc[all_best[metric_col].idxmax()]

print("\n=== Best Silhouette Overall ===")
display(best_silhouette_row[["algorithm", "interval", "condition", "silhouette"]])

print("\n=== Best Composite Score Overall ===")
display(best_composite_row[["algorithm", "interval", "condition", metric_col]])

```

Combined best table: (18, 43)

Using composite column: composite_eval

=== Mean Scores Across All Conditions ===

	silhouette	composite_eval
algorithm		
kmeans	0.521777	2.227450
hdbscan	0.512003	2.057469
phenograph	0.367387	2.056252

=== Best Silhouette Overall ===

7

algorithm hdbscan**interval** 1-10w**condition** 1**silhouette** 0.59996**dtype:** object

=== Best Composite Score Overall ===

7

algorithm hdbscan**interval** 1-10w**condition** 1**composite_eval** 3.088461**dtype:** object**Visualization of across algorithm performance**

Composite score ranking

```

In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

files = {
    "kmeans": "kmeans_best_per_interval_across_pipelines.csv",
    "hdbscan": "hdbscan_best_per_interval_across_pipelines.csv",
    "phenograph": "phenograph_best_per_interval_across_pipelines.csv",
}

# Load and add algorithm name
dfs = []
for algo, fname in files.items():
    path = os.path.join(output_dir, fname)
    df = pd.read_csv(path)
    df["algorithm"] = algo
    dfs.append(df)

all_best = pd.concat(dfs, ignore_index=True)
print("Combined best table:", all_best.shape)

# Identify composite metric column
metric_candidates = ["composite_eval", "composite_score", "composite_old"]
metric_col = next((m for m in metric_candidates if m in all_best.columns), None)

if metric_col is None:
    raise ValueError("No composite metric column found.")

print("Using composite metric:", metric_col)

# Build ranking table per interval x condition

```



```

ranking_rows = []

# unique groups
groups = all_best[["interval", "condition"]].drop_duplicates()

for _, row in groups.iterrows():
    interval = row["interval"]
    condition = row["condition"]

    subset = all_best[(all_best["interval"] == interval) &
                      (all_best["condition"] == condition)]

    if subset.empty:
        continue

    # Rank silhouette (higher = better)
    sub_sil = subset.sort_values(by="silhouette", ascending=False)
    sub_sil["silhouette_rank"] = range(1, len(sub_sil) + 1)

    # Rank composite
    sub_comp = subset.sort_values(by="metric_col", ascending=False)
    sub_comp["composite_rank"] = range(1, len(sub_comp) + 1)

    # Merge ranks back together
    merged = subset.merge(
        sub_sil[["algorithm", "silhouette_rank"]],
        on="algorithm",
        how="left"
    ).merge(
        sub_comp[["algorithm", "composite_rank"]],
        on="algorithm",
        how="left"
    )

    # Build final rows
    for _, r in merged.iterrows():
        ranking_rows.append({
            "interval": interval,
            "condition": condition,
            "algorithm": r["algorithm"],
            "silhouette": r["silhouette"],
            "metric_col": r["metric_col"],
            "silhouette_rank": int(r["silhouette_rank"]),
            "composite_rank": int(r["composite_rank"]),
        })

ranking_table = pd.DataFrame(ranking_rows)

print("\n=== Ranking Table ===")
display(ranking_table)

# Save
rank_path = os.path.join(output_dir, "algorithm_ranking_table.csv")
ranking_table.to_csv(rank_path, index=False)
print("Saved ranking table →", rank_path)

```

Combined best table: (18, 43)

Using composite metric: composite_eval

=== Ranking Table ===

	interval	condition	algorithm	silhouette	composite_eval	silhouette_rank	composi
0	1-10w	0	kmeans	0.516154	1.722881	1	
1	1-10w	0	hdbscan	0.455572	1.507343	2	
2	1-10w	1	kmeans	0.535680	1.990356	2	
3	1-10w	1	hdbscan	0.599960	3.088461	1	
4	11-20w	0	kmeans	0.561161	2.914948	1	
5	11-20w	1	kmeans	0.548862	2.829592	1	
6	11-20w	1	hdbscan	0.477691	1.268321	2	
7	21-30w	0	kmeans	0.521675	2.175900	1	
8	21-30w	1	kmeans	0.447130	1.731020	2	
9	21-30w	1	hdbscan	0.489319	1.736101	1	
10	11-20w	0	hdbscan	0.541894	2.311847	1	
11	11-20w	0	phenograph	0.371096	2.307720	2	
12	21-30w	0	hdbscan	0.507583	2.432742	1	
13	21-30w	0	phenograph	0.363016	2.286687	2	
14	11-20w	1	phenograph	0.337713	1.865419	1	
15	1-10w	0	phenograph	0.358728	1.611654	1	
16	1-10w	1	phenograph	0.433922	2.400763	1	
17	21-30w	1	phenograph	0.339850	1.865267	1	

◀ Saved ranking table → /content/drive/MyDrive/preeclampsia_results/algorithm_ranking_table.csv ▶

Visualization of algo ranking

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

files = {
    "kmeans": "kmeans_best_per_interval_across_pipelines.csv",
    "hdbscan": "hdbscan_best_per_interval_across_pipelines.csv",
    "phenograph": "phenograph_best_per_interval_across_pipelines.csv",
}

# Load and combine
all_rows = []

for algo, fname in files.items():
    path = os.path.join(output_dir, fname)
    df = pd.read_csv(path)
```

```

# Normalise interval dashes just in case
df["interval"] = (
    df["interval"].astype(str)
    .str.replace("-", "-")
    .str.replace("_", "-")
)

df["algorithm"] = algo
all_rows.append(df)

all_best = pd.concat(all_rows, ignore_index=True)
print("Combined best shape:", all_best.shape)

# Build ranking table for silhouette & composite_eval

metrics = ["silhouette", "composite_eval"]
rank_records = []

for (interval, cond), sub in all_best.groupby(["interval", "condition"]):
    for metric in metrics:
        # if the metric column is missing, skip
        if metric not in sub.columns:
            print(f"Warning: metric '{metric}' missing for group {interval}, {cond}")
            continue

        # Rank: higher metric = better (rank 1)
        ranks = sub[metric].rank(ascending=False, method="min")

        for algo, score, rnk in zip(sub["algorithm"], sub[metric], ranks):
            rank_records.append(
                {
                    "interval": interval,
                    "condition": cond,
                    "algorithm": algo,
                    "metric": metric,
                    "score": score,
                    "rank": int(rnk),
                }
            )

rank_df = pd.DataFrame(rank_records)
rank_path = os.path.join(output_dir, "cluster_algorithm_ranks_by_interval_condition.csv")
rank_df.to_csv(rank_path, index=False)
print("Saved ranking table →", rank_path)
display(rank_df.head())

# Heatmap plotting helper
def plot_rank_heatmap(rank_df, metric):
    # Filter to one metric and pivot: rows=algorithm, cols=interval+condition
    sub = rank_df[rank_df["metric"] == metric].copy()
    sub["interval_cond"] = sub["interval"].astype(str) + "_c" + sub["condition"]

    heat = sub.pivot(index="algorithm", columns="interval_cond", values="rank")

    plt.figure(figsize=(8, 4))
    im = plt.imshow(heat, aspect="auto")

    # Colorbar: Lower rank = better
    cbar = plt.colorbar(im)
    cbar.set_label("Rank (1 = best)")

```

```

plt.xticks(
    ticks=np.arange(len(heat.columns)),
    labels=heat.columns,
    rotation=45,
    ha="right"
)
plt.yticks(
    ticks=np.arange(len(heat.index)),
    labels=heat.index
)

plt.title(f"Algorithm rank by interval / condition - {metric}")
plt.xlabel("Interval_condition")
plt.ylabel("Algorithm")

# Overlay rank numbers in each cell
for i, algo in enumerate(heat.index):
    for j, col in enumerate(heat.columns):
        val = heat.loc[algo, col]
        if not np.isnan(val):
            plt.text(j, i, int(val), ha="center", va="center")

plt.tight_layout()
plt.show()

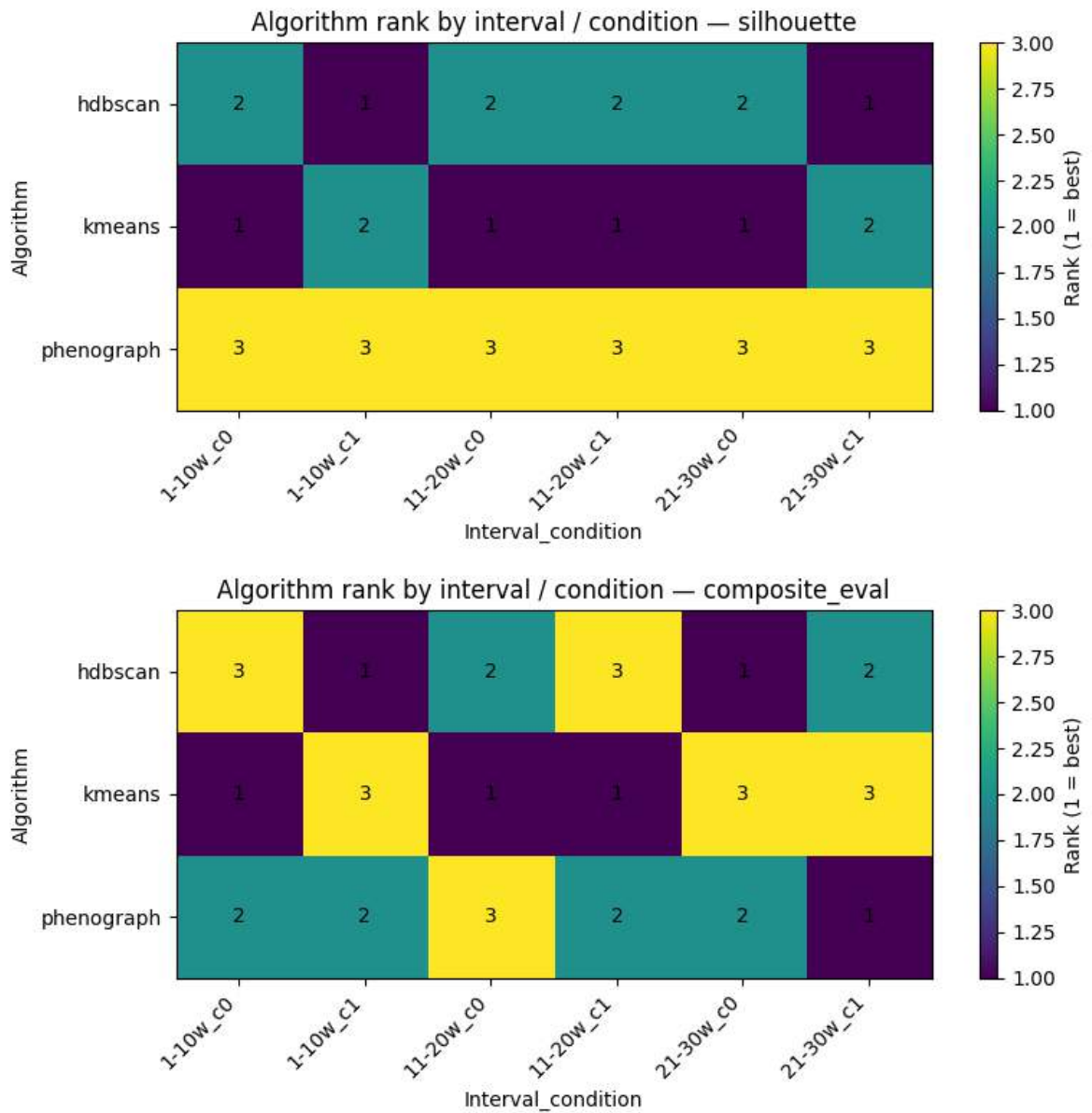
# Plot heatmaps for both metrics
plot_rank_heatmap(rank_df, "silhouette")
plot_rank_heatmap(rank_df, "composite_eval")

```

Combined best shape: (18, 43)

Saved ranking table → /content/drive/MyDrive/preeclampsia_results/cluster_algorithm_ranks_by_interval_condition.csv

	interval	condition	algorithm	metric	score	rank
0	1-10w	0	kmeans	silhouette	0.516154	1
1	1-10w	0	hdbscan	silhouette	0.455572	2
2	1-10w	0	phenograph	silhouette	0.358728	3
3	1-10w	0	kmeans	composite_eval	1.722881	1
4	1-10w	0	hdbscan	composite_eval	1.507343	3



Performing AR/AMI across algorithms I need to ensure all cluster file paths are mapped

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"
path = os.path.join(output_dir, "phenograph_best_per_interval_across_pipelines.csv")

# Load file
df = pd.read_csv(path)

print("Loaded:", df.shape)

# Unique values in label_csv
print("\n=== Unique values in label_csv ===")
unique_vals = df["labels_csv"].unique()
for v in unique_vals:
    print(" ", v)

print("\nTotal unique label_csv values:", len(unique_vals))
# Check for NaNs / missing paths
missing = df[df["labels_csv"].isna() | (df["labels_csv"] == "")]
print("\n=== Rows with missing label_csv (NaN or empty) ===")
print("Count:", missing.shape[0])
```

```
if not missing.empty:
    display(missing)

# Check how many are real paths on disk
def exists_or_nan(x):
    if isinstance(x, str) and x.strip() != "":
        return os.path.exists(x)
    return False

df["label_exists"] = df["labels_csv"].apply(exists_or_nan)

print("\n=== Existence summary ===")
print(df["label_exists"].value_counts())

missing_files = df.loc[~df["label_exists"], ["interval", "condition", "pipeline"]]
if not missing_files.empty:
    print("\nRows where label file does NOT exist:")
    display(missing_files)
```

Loaded: (6, 22)

=== Unique values in label_csv ===
nan

Total unique label_csv values: 1

=== Rows with missing label_csv (NaN or empty) ===
Count: 6

	interval	condition	pipeline	space	umap_dim	k	n_cells	n_clusters	silhouette	dataset
0	11–20w	0	UMAP (P2, base)	umap2	2.0	75	55000	35	0.371096	
1	11–20w	1	UMAP (P2, base)	umap2	2.0	75	65000	37	0.337713	
2	1–10w	0	UMAP (P2, base)	umap2	2.0	30	15000	38	0.358728	
3	1–10w	1	UMAP (P2, base)	umap2	2.0	75	5000	19	0.433922	
4	21–30w	0	UMAP (P2, base)	umap2	2.0	75	50000	39	0.363016	
5	21–30w	1	UMAP (P2, base)	umap2	2.0	50	40000	42	0.339850	

6 rows × 22 columns



```

=== Existence summary ===
label_exists
False      6
Name: count, dtype: int64

```

Rows where label file does NOT exist:

	interval	condition	pipeline	labels_csv
0	11–20w	0	UMAP (P2, base)	NaN
1	11–20w	1	UMAP (P2, base)	NaN
2	1–10w	0	UMAP (P2, base)	NaN
3	1–10w	1	UMAP (P2, base)	NaN
4	21–30w	0	UMAP (P2, base)	NaN
5	21–30w	1	UMAP (P2, base)	NaN

```

In [ ]: search_root = "/content/drive/MyDrive/preeclampsia_results"

matches = []

for root, dirs, files in os.walk(search_root):
    for fname in files:
        fname_lower = fname.lower()
        if "phenograph" in fname_lower and fname_lower.endswith(".csv"):
            full_path = os.path.join(root, fname)
            matches.append(full_path)

print("Found", len(matches), "Phenograph label CSV files:\n")
for m in matches:
    print(m)

```


Found 53 Phenograph label CSV files:

/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline1_pca20_k75__1-10w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline1_pca20_k15__1-10w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline1_pca20_k50__11-20w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline1_pca20_k75__11-20w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline1_pca20_k15__21-30w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline1_pca20_k50__21-30w_1.csv
/content/drive/MyDrive/preeclampsia_results/phenograph_pipeline1_metrics.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap2_k30__1-10w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap3_k75__1-10w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap5_k50__1-10w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap2_k75__1-10w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap3_k75__1-10w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap5_k50__1-10w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap2_k75__11-20w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap3_k75__11-20w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap5_k50__11-20w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap2_k75__11-20w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap3_k75__11-20w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap5_k50__11-20w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap2_k75__21-30w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap3_k75__21-30w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap5_k75__21-30w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap2_k50__21-30w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap3_k75__21-30w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap5_k50__21-30w_1.csv
/content/drive/MyDrive/preeclampsia_results/phenograph_pipeline2_metrics.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap2_k100_HI__1-10w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap3_k120_HI__1-10w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap5_k120_HI__1-10w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap2_k120_HI__1-10w_1.csv

```

/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap3_k12
0_HI__1-10w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap5_k12
0_HI__1-10w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap2_k80
_HI__11-20w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap3_k10
0_HI__11-20w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap5_k80
_HI__11-20w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap2_k80
_HI__11-20w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap3_k10
0_HI__11-20w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap5_k80
_HI__11-20w_1.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap2_k10
0_HI__21-30w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap3_k80
_HI__21-30w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap5_k12
0_HI__21-30w_0.csv
/content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2_umap2_k12
0_HI__21-30w_1.csv
/content/drive/MyDrive/preeclampsia_results/phenograph_all_metrics_SH_DB_CH.csv
/content/drive/MyDrive/preeclampsia_results/phenograph_top3_per_pipeline_SH_DB_C
H.csv
/content/drive/MyDrive/preeclampsia_results/phenograph_top3_across_pipelines_SH_D
B_CH.csv
/content/drive/MyDrive/preeclampsia_results/phenograph_ARI_AMI_vs_best_kmeans_hdb
scan.csv
/content/drive/MyDrive/preeclampsia_results/phenograph_vs_kmeans_ARI_AMI_summary.
csv
/content/drive/MyDrive/preeclampsia_results/table_best24_phenograph_pipeline2_ove
rview.csv
/content/drive/MyDrive/preeclampsia_results/phenograph_inventory.csv
/content/drive/MyDrive/preeclampsia_results/phenograph_best_per_pipeline_inventor
y.csv
/content/drive/MyDrive/preeclampsia_results/phenograph_eval_scored.csv
/content/drive/MyDrive/preeclampsia_results/phenograph_best_per_interval_across_p
ipelines.csv
/content/drive/MyDrive/preeclampsia_results/phenograph_best_pipeline1_only.csv

```

Condordance evaluation will be done on only best performing clusters per algorithm

```

In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"

# Load best table

best_path = os.path.join(output_dir, "phenograph_best_per_interval_across_pipeli
best = pd.read_csv(best_path)
print("Loaded best:", best.shape)

# Normalize: interval dashes, pipeline to pipeline1/pipeline2
best["interval_norm"] = best["interval"].astype(str).str.replace("-", "-").str.r

def simplify_pipeline(p):
    p = str(p).lower()
    if "p1" in p or "pipeline1" in p or "pca" in p:
        return "pipeline1"

```

```

    if "p2" in p or "pipeline2" in p or "umap" in p:
        return "pipeline2"
    return None

best["pipeline_norm"] = best["pipeline"].apply(simplify_pipeline)

# normalize space (pca20, umap2, umap3, umap5)
best["space_norm"] = best["space"].astype(str).str.lower().str.strip()

# numeric condition, k
best["cond_int"] = best["condition"].astype(int)
best["k_int"] = best["k"].astype(int)

# Collect all phenograph label files
files = glob.glob(os.path.join(output_dir, "labels_phenograph_*.csv"))
print("Found label files:", len(files))

# Parse filenames - build lookup table
pattern = re.compile(
    r"labels_phenograph_"
    r"(pipeline\d+)"_          # pipeline1 / pipeline2
    r"(pca20|umap\d+)"_       # space
    r"k(\d+)"                 # k
    r"(?:_[A-Za-z0-9]+)?"
    r"_"
    r"([^\_]+)"_              # interval
    r"([01])"                 # condition
    r"\.csv$"
)

lookup_rows = []

for f in files:
    name = os.path.basename(f)
    name2 = name.replace("-", "-").replace("-", "-") # normalize for match

    m = pattern.match(name2)
    if not m:
        print("Skipping unmatched filename:", name)
        continue

    pipeline_id, space, k_str, interval_str, cond_str = m.groups()

    lookup_rows.append({
        "pipeline_norm": pipeline_id.lower(),
        "space_norm": space.lower(),
        "k_int": int(k_str),
        "interval_norm": interval_str,
        "cond_int": int(cond_str),
        "labels_csv_file": f
    })

lookup = pd.DataFrame(lookup_rows)
print("Lookup shape:", lookup.shape)
display(lookup.head())

# Merge simple and exact key match
merge_keys = ["pipeline_norm", "space_norm", "k_int", "interval_norm", "cond_int"]

```

```

merged = best.merge(
    lookup[merge_keys + ["labels_csv_file"]],
    on=merge_keys,
    how="left"
)

# place into labels_csv
merged["labels_csv"] = merged["labels_csv_file"]

# Show missing
missing = merged[merged["labels_csv"].isna()]
print("\nMissing after mapping:", missing.shape[0])
if not missing.empty:
    display(missing[["interval", "condition", "pipeline", "space", "k"]])

# Save output (overwrite or new file)
out_path = os.path.join(output_dir, "phenograph_best_per_interval_across_pipelines.csv")
merged.to_csv(out_path, index=False)
print("\nSAVED updated file →", out_path)

```

Loaded best: (6, 30)

Found label files: 40

Lookup shape: (40, 6)

	pipeline_norm	space_norm	k_int	interval_norm	cond_int	
0	pipeline1	pca20	75	1-10w	0	/content/drive/MyDrive/preeclampsia_results/phenograph_best_per_interval_across_pipelines.csv
1	pipeline1	pca20	15	1-10w	1	/content/drive/MyDrive/preeclampsia_results/phenograph_best_per_interval_across_pipelines.csv
2	pipeline1	pca20	50	11-20w	0	/content/drive/MyDrive/preeclampsia_results/phenograph_best_per_interval_across_pipelines.csv
3	pipeline1	pca20	75	11-20w	1	/content/drive/MyDrive/preeclampsia_results/phenograph_best_per_interval_across_pipelines.csv
4	pipeline1	pca20	15	21-30w	0	/content/drive/MyDrive/preeclampsia_results/phenograph_best_per_interval_across_pipelines.csv

Missing after mapping: 0

SAVED updated file → /content/drive/MyDrive/preeclampsia_results/phenograph_best_per_interval_across_pipelines.csv

Compute ARI/AMI Between Algorithms (best 6 each)

```

In [ ]: # Fetching the best performing tables per algorithms

output_dir = "/content/drive/MyDrive/preeclampsia_results"

files = {
    "kmeans": "kmeans_best_per_interval_across_pipelines.csv",
    "hdbscan": "hdbscan_best_per_interval_across_pipelines.csv",
    "phenograph": "phenograph_best_per_interval_across_pipelines.csv",
}

# Helpers
def normalise_interval(x):
    # Make all dash types consistent so tables line up
    return str(x).replace("-", "-").replace("-", "-").strip()

def load_labels_series(path):

```

```

"""Load labels from a CSV and return a 1D Series of cluster labels."""
df = pd.read_csv(path)
# Heuristics: prefer common label column names; fallback to first column
for col in ["cluster", "label", "labels", "cluster_id", "cluster_label"]:
    if col in df.columns:
        return df[col].astype(int)
# If only one column, use it
if df.shape[1] == 1:
    return df.iloc[:, 0].astype(int)
# If more columns, just take the last one as a generic labels column
return df.iloc[:, -1].astype(int)

# Load the "best per interval" tables
tables = {}
for algo, fname in files.items():
    path = os.path.join(output_dir, fname)
    df = pd.read_csv(path)
    print(f"{algo}: loaded {fname} with shape {df.shape}")
    tables[algo] = df

# Map algo - which column has the path to labels
# NOTE: kmeans uses 'label_csv'; others use 'labels_csv'
label_col_map = {
    "kmeans": "label_csv", # singular
    "hdbscan": "labels_csv",
    "phenograph": "labels_csv",
}

# Build mapping: (algo) - {(interval_norm, cond_int): labels_path}
paths = {}

for algo, df in tables.items():
    label_col = label_col_map[algo]
    if label_col not in df.columns:
        raise ValueError(f"Expected column '{label_col}' in {files[algo]} for {a

    df = df.copy()
    df["interval_norm"] = df["interval"].apply(normalise_interval)
    df["cond_int"] = df["condition"].astype(int)

    algo_map = {}
    for _, row in df.iterrows():
        key = (row["interval_norm"], row["cond_int"])
        path = row[label_col]
        if isinstance(path, str) and path.strip():
            algo_map[key] = path
    paths[algo] = algo_map

# Check what keys we have for each algo
for algo, mapping in paths.items():
    print(f"{algo}: {len(mapping)} (interval, condition) pairs with label paths"

# Only compute ARI/AMI where all three algorithms have labels
common_keys = set(paths["kmeans"].keys()) & set(paths["hdbscan"].keys()) & set(p
print("\nCommon (interval, condition) keys across all algorithms:", len(common_k

# Compute ARI / AMI per interval & condition
rows = []

```

```

for interval_norm, cond_int in sorted(common_keys):
    kmeans_path = paths["kmeans"][interval_norm, cond_int]
    hdbscan_path = paths["hdbscan"][interval_norm, cond_int]
    pheno_path = paths["phenograph"][interval_norm, cond_int]

    # Check files exist
    missing_any = False
    for algo_name, pth in [("kmeans", kmeans_path),
                           ("hdbscan", hdbscan_path),
                           ("phenograph", pheno_path)]:
        if not os.path.exists(pth):
            print(f"Missing label file for {algo_name} at {interval_norm}, cond={cond_int}")
            missing_any = True
    if missing_any:
        continue

    # Load labels
    labels_km = load_labels_series(kmeans_path)
    labels_hd = load_labels_series(hdbscan_path)
    labels_ph = load_labels_series(pheno_path)

    # Ensure same number of cells
    n_km, n_hd, n_ph = len(labels_km), len(labels_hd), len(labels_ph)
    if not (n_km == n_hd == n_ph):
        print(f"Length mismatch at {interval_norm}, cond={cond_int}: "
              f"kmeans={n_km}, hdbscan={n_hd}, phenograph={n_ph} -> skipping")
        continue

    n_cells = n_km

    # Compute ARI/AMI between each pair
    ari_km_hd = adjusted_rand_score(labels_km, labels_hd)
    ami_km_hd = adjusted_mutual_info_score(labels_km, labels_hd)

    ari_km_ph = adjusted_rand_score(labels_km, labels_ph)
    ami_km_ph = adjusted_mutual_info_score(labels_km, labels_ph)

    ari_hd_ph = adjusted_rand_score(labels_hd, labels_ph)
    ami_hd_ph = adjusted_mutual_info_score(labels_hd, labels_ph)

    rows.append({
        "interval": interval_norm,
        "condition": cond_int,
        "n_cells": n_cells,

        "ARI_kmeans_hdbscan": ari_km_hd,
        "AMI_kmeans_hdbscan": ami_km_hd,

        "ARI_kmeans_phenograph": ari_km_ph,
        "AMI_kmeans_phenograph": ami_km_ph,

        "ARI_hdbscan_phenograph": ari_hd_ph,
        "AMI_hdbscan_phenograph": ami_hd_ph,

        "kmeans_labels_csv": kmeans_path,
        "hdbscan_labels_csv": hdbscan_path,
        "phenograph_labels_csv": pheno_path,
    })

ari_ami_df = pd.DataFrame(rows)

```

```
print("\nARI/AMI table shape:", ari_ami_df.shape)

# Save and display
ari_ami_path = os.path.join(output_dir, "cluster_algorithm_ARI_AMI_by_interval_c
ari_ami_df.to_csv(ari_ami_path, index=False)
print("Saved ARI/AMI comparison →", ari_ami_path)

display(ari_ami_df)
```

kmeans: loaded kmeans_best_per_interval_across_pipelines.csv with shape (6, 22)
hdbscan: loaded hdbscan_best_per_interval_across_pipelines.csv with shape (6, 28)
phenograph: loaded phenograph_best_per_interval_across_pipelines.csv with shape (6, 32)

kmeans: 6 (interval, condition) pairs with label paths
hdbscan: 6 (interval, condition) pairs with label paths
phenograph: 6 (interval, condition) pairs with label paths

Common (interval, condition) keys across all algorithms: 6

ARI/AMI table shape: (6, 12)

Saved ARI/AMI comparison → /content/drive/MyDrive/preeclampsia_results/cluster_algorithm_ARI_AMI_by_interval_condition.csv

	interval	condition	n_cells	ARI_kmeans_hdbscan	AMI_kmeans_hdbscan	ARI_kmeans_p
0	1-10w	0	15000	0.352328	0.534728	
1	1-10w	1	5000	0.316338	0.598655	
2	11-20w	0	55000	0.374530	0.499428	
3	11-20w	1	65000	0.282804	0.558850	
4	21-30w	0	50000	0.463028	0.603806	
5	21-30w	1	40000	0.209503	0.577850	

Visualization is algorithmic concordance of clusters

```
In [ ]: # File path
output_dir = "/content/drive/MyDrive/preeclampsia_results"

metrics_path = os.path.join(output_dir, "cluster_algorithm_ARI_AMI_by_interval_c
df = pd.read_csv(metrics_path)

print("Loaded ARI/AMI table:", df.shape)
print("Columns:", df.columns.tolist())

# Make sure interval is ordered nicely
if "interval" in df.columns:
    # Custom ordering
    interval_order = ["1-10w", "11-20w", "21-30w"]
    df["interval"] = df["interval"].astype(str).str.replace("-", "-").str.replac
    df["interval"] = pd.Categorical(df["interval"], categories=interval_order, c

# Combine interval and condition into a single x-axis label
df["group_label"] = df.apply(
    lambda r: f"{r['interval']}, cond={int(r['condition'])}", axis=1
```



```

)

# Reshape ARI and AMI columns into Long (tidy) format
ari_cols = [c for c in df.columns if c.startswith("ARI_")]
ami_cols = [c for c in df.columns if c.startswith("AMI_")]

if not ari_cols and not ami_cols:
    raise ValueError("No columns starting with 'ARI_' or 'AMI_' found. Check you

print("ARI columns:", ari_cols)
print("AMI columns:", ami_cols)

# Long format for ARI
ari_long = df.melt(
    id_vars=["interval", "condition", "group_label"],
    value_vars=ari_cols,
    var_name="pair",
    value_name="ARI"
)
ari_long["algo_pair"] = ari_long["pair"].str.replace("^ARI_", "", regex=True)

# Long format for AMI
ami_long = df.melt(
    id_vars=["interval", "condition", "group_label"],
    value_vars=ami_cols,
    var_name="pair",
    value_name="AMI"
)
ami_long["algo_pair"] = ami_long["pair"].str.replace("^AMI_", "", regex=True)

# Helper to make a grounded grouped bar plot for a given metric
def plot_grouped_bars(long_df, value_col, title, ylim=(0, 1.05)):
    """
    long_df: dataframe with columns [group_label, algo_pair, <value_col>]
    value_col: 'ARI' or 'AMI'
    """

    # Drop NaNs just in case
    long_df = long_df.dropna(subset=[value_col])

    groups = long_df["group_label"].unique().tolist()
    algo_pairs = long_df["algo_pair"].unique().tolist()

    n_groups = len(groups)
    n_pairs = len(algo_pairs)

    x = np.arange(n_groups)
    width = 0.8 / max(n_pairs, 1) # total bar group width ~0.8

    plt.figure(figsize=(max(8, n_groups * 1.2), 5))

    for i, pair in enumerate(algo_pairs):
        subset = long_df[long_df["algo_pair"] == pair]
        # Ensure order matches x positions
        subset = subset.set_index("group_label").reindex(groups)

        bar_positions = x + (i - (n_pairs - 1) / 2) * width
        plt.bar(bar_positions, subset[value_col].values, width=width, label=pair)

    plt.axhline(0, linestyle="--", linewidth=0.7)
    plt.xticks(x, groups, rotation=30, ha="right")

```

```

plt.ylim(ylim)
plt.ylabel(value_col)
plt.title(title)
plt.legend(title="Algorithm pair")
plt.tight_layout()
plt.show()

# Plot ARI and AMI as grounded grouped bar plots
if ari_cols:
    plot_grouped_bars(
        ari_long,
        value_col="ARI",
        title="Cluster algorithm similarity by interval & condition (ARI)",
        ylim=(0, 1.05)
    )

if ami_cols:
    plot_grouped_bars(
        ami_long,
        value_col="AMI",
        title="Cluster algorithm similarity by interval & condition (AMI)",
        ylim=(0, 1.05)
    )

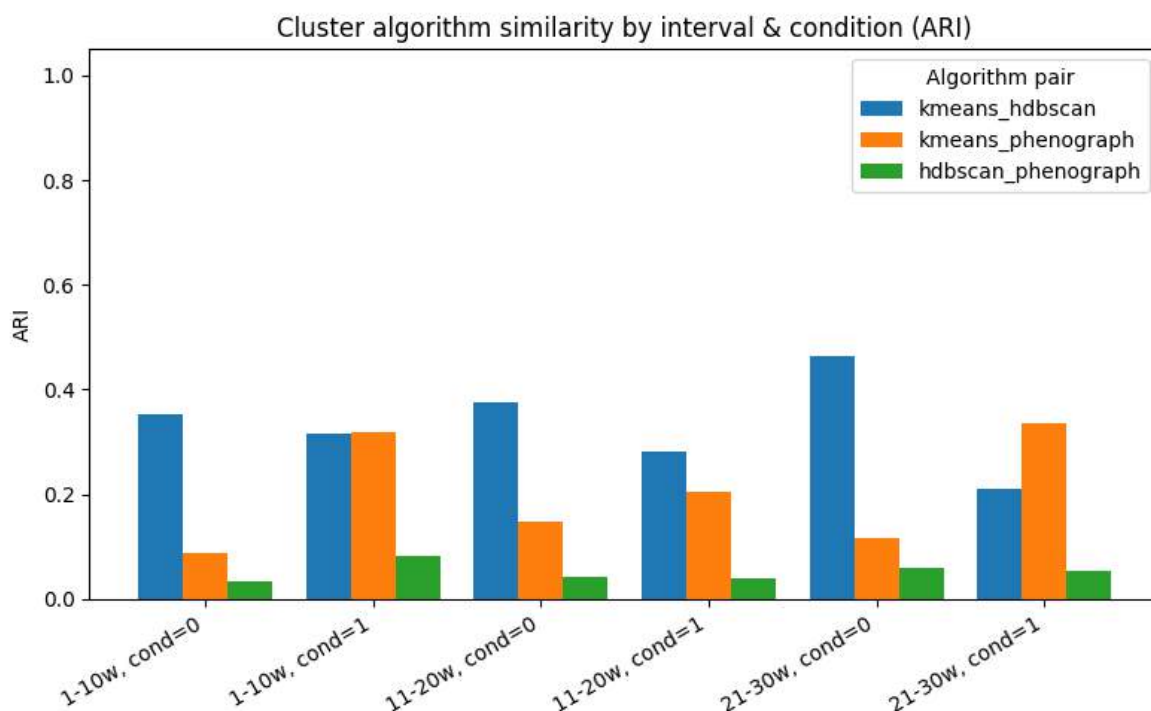
```

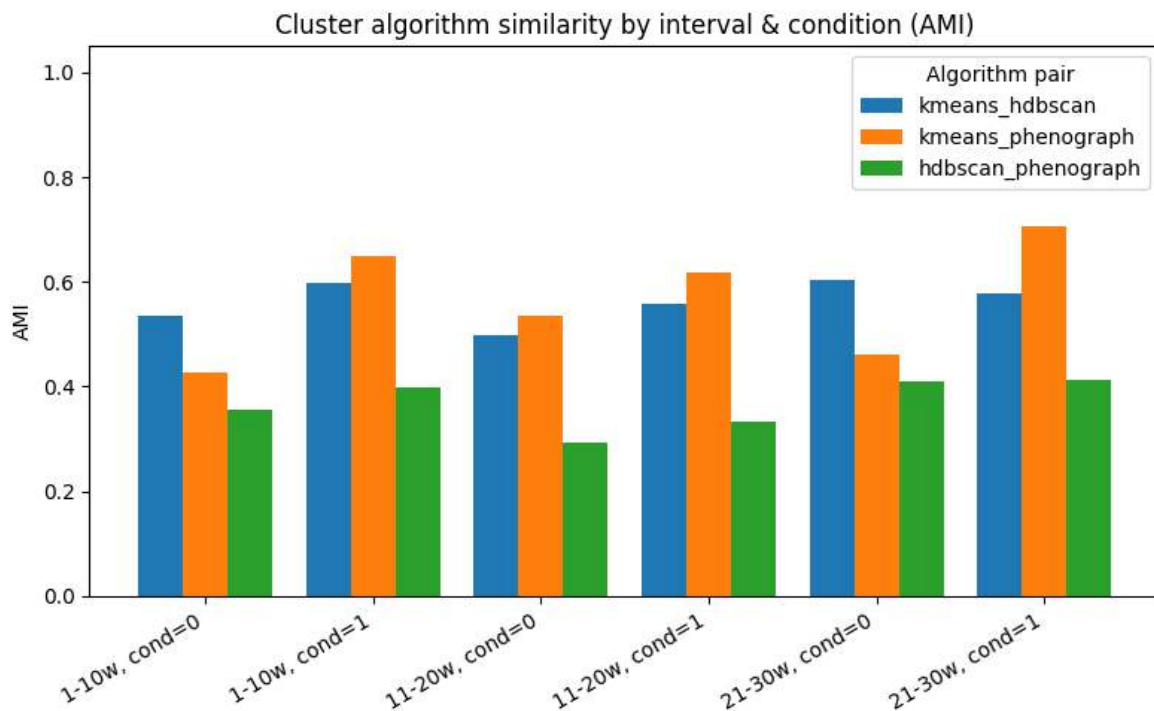
Loaded ARI/AMI table: (6, 12)

Columns: ['interval', 'condition', 'n_cells', 'ARI_kmeans_hdbscan', 'AMI_kmeans_hdbscan', 'ARI_kmeans_phenograph', 'AMI_kmeans_phenograph', 'ARI_hdbscan_phenograph', 'AMI_hdbscan_phenograph', 'kmeans_labels_csv', 'hdbscan_labels_csv', 'phenograph_labels_csv']

ARI columns: ['ARI_kmeans_hdbscan', 'ARI_kmeans_phenograph', 'ARI_hdbscan_phenograph']

AMI columns: ['AMI_kmeans_hdbscan', 'AMI_kmeans_phenograph', 'AMI_hdbscan_phenograph']





Marker Enrichment Analysis

Mean Log2 Fold Change and Mann Whitney's cluster enrichment evaluation

```
In [ ]: # Paths and files
output_dir = "/content/drive/MyDrive/preeclampsia_results"
eda_path = "/content/drive/MyDrive/pre eclampsia/eda.csv"

best_files = {
    "KMeans": "kmeans_best_per_interval_across_pipelines.csv",
    "HDBSCAN": "hdbscan_best_per_interval_across_pipelines.csv",
    "PhenoGraph": "phenograph_best_per_interval_across_pipelines.csv",
}

# Load EDA & define marker columns
eda = pd.read_csv(eda_path)

# Restore age_bin_10w as categorical if present
if "age_bin_10w" in eda.columns and not pd.api.types.is_categorical_dtype(eda["age_bin_10w"]):
    eda["age_bin_10w"] = eda["age_bin_10w"].astype("category")

# Non-marker columns
NON_MARKER = {
    "condition", "age", "age_label", "filename", "age_bin_10w",
    "_umap1", "_umap2", "PCA_1", "PCA_2",
    "KMeans_Labels",
}

numeric_cols = [c for c in eda.columns if pd.api.types.is_numeric_dtype(eda[c])]
MARKERS = [c for c in numeric_cols if c not in NON_MARKER]
print(f"Markers (n={len(MARKERS)}): {MARKERS}")

COND_NAME = {0: "Control", 1: "PE"}

# Load the three "best-per-interval" tables & stack
best_runs_list = []
```

```

for algo_name, fname in best_files.items():
    path = os.path.join(output_dir, fname)
    if not os.path.exists(path):
        print(f"WARNING: file not found → {path}")
        continue

    df = pd.read_csv(path)
    df["algorithm"] = algo_name # e.g. "KMeans", "HDBSCAN", "PhenoGraph"

    # Normalise label column name: KMeans file uses 'label_csv'
    if "labels_csv" not in df.columns and "label_csv" in df.columns:
        df = df.rename(columns={"label_csv": "labels_csv"})

    best_runs_list.append(df)

best_runs = pd.concat(best_runs_list, ignore_index=True)
print("Combined best-runs table:", best_runs.shape)

# Helper functions
def load_labels_series(labels_path):
    """
    Load a labels CSV saved with:
    - index = cell index in eda
    - one column = cluster labels
    Returns: pd.Series with numeric labels, indexed by cell index.
    """
    df = pd.read_csv(labels_path, index_col=0)
    if df.shape[1] == 1:
        s = df.iloc[:, 0]
    else:
        # Prefer numeric column if multiple
        num = df.select_dtypes(include=[np.number])
        if num.shape[1] > 0:
            s = num.iloc[:, 0]
        else:
            s = df.iloc[:, 0]
    s = pd.to_numeric(s, errors="coerce")
    return s

def compute_cluster_marker_stats(X_cluster, X_rest, eps=1e-3):
    """
    Compute mean/median/log2FC for each marker.
    """
    cl_mean = X_cluster.mean(axis=0)
    cl_med = np.median(X_cluster, axis=0)
    rs_mean = X_rest.mean(axis=0)
    rs_med = np.median(X_rest, axis=0)

    mean_diff = cl_mean - rs_mean
    median_diff = cl_med - rs_med
    log2FC_mean = np.log2(cl_mean + eps) - np.log2(rs_mean + eps)

    return {
        "cluster_mean": cl_mean,
        "cluster_median": cl_med,
        "rest_mean": rs_mean,
        "rest_median": rs_med,
        "mean_diff": mean_diff,
        "median_diff": median_diff,
        "log2FC_mean": log2FC_mean,
    }

```

```

    }

def benjamini_hochberg(pvals):
    """
    Benjamini-Hochberg FDR correction.
    pvals: 1D array-like (may contain NaN).
    returns: array of q-values (same shape, NaN where p is NaN).
    """
    p = np.array(pvals, dtype=float)
    q = np.full_like(p, np.nan)

    mask = np.isfinite(p)
    if not mask.any():
        return q

    p_valid = p[mask]
    n = p_valid.size
    order = np.argsort(p_valid)
    ranked_p = p_valid[order]

    # BH step-up
    bh = ranked_p * n / (np.arange(1, n + 1))
    # enforce monotonicity
    bh_rev = np.minimum.accumulate(bh[::-1])[::-1]

    q_valid = np.empty_like(p_valid)
    q_valid[order] = bh_rev

    q[mask] = q_valid
    return q

```

Markers (n=33): ['CD45', 'CD66', 'CD7', 'CD19', 'CD45RA', 'CD11b', 'CD4', 'CD8a', 'CD11c', 'CD123', 'CREB', 'STAT5', 'p38', 'TCRgd', 'STAT1', 'STAT3', 'S6', 'CD3', 'MAPKAPK2', 'Tbet', 'cPARP', 'Ikb', 'CD16', 'NFkB', 'GATA3', 'pSTAT6', 'CD25', 'CD3', 'pERK', 'RORgd', 'HLADR', 'CD14', 'CD56']

Combined best-runs table: (18, 43)

/tmp/ipython-input-2436697379.py:19: DeprecationWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead

if "age_bin_10w" in eda.columns and not pd.api.types.is_categorical_dtype(eda["age_bin_10w"]):

```

In [ ]: # Enrichment over all best runs
        # (cluster vs rest, per marker, per algorithm/interval/condition)

        enrich_rows = []
        min_cluster_size = 100

        for _, r in best_runs.iterrows():
            interval = str(r["interval"])
            cond = r.get("condition", np.nan)
            algo = str(r["algorithm"])
            pipeline = str(r.get("pipeline", ""))
            space = str(r.get("space", ""))
            dim = r.get("umap_dim", np.nan)

            if pd.isna(cond):
                print(f"Skipping row with NaN condition: {interval}, algo={algo}")
                continue

```

```

cond_int = int(cond)
cond_name = COND_NAME.get(cond_int, "Unknown")

# interval format in EDA uses en dash, so normalise
interval_eda = interval.replace("-", "-")

# Labels path - we now assume labels_csv exists and is correct
labels_path = r.get("labels_csv", np.nan)
if not isinstance(labels_path, str) or not os.path.exists(labels_path):
    print(f"Skipping: cannot find labels_csv for {algo} {interval} cond={cond}")
    print(" labels_csv =", labels_path)
    continue

print(f"\nRun: {interval} | {cond_name} | {algo} | {pipeline} | space={space}")
print(" labels:", labels_path)

# Load labels
labels = load_labels_series(labels_path)

# Subset EDA rows for this interval+condition
if "age_bin_10w" in eda.columns:
    sub_mask = (eda["age_bin_10w"].astype(str) == interval_eda) & (eda["cond"] == cond_name)
else:
    sub_mask = (eda["condition"] == cond_int)

sub_idx = eda.index[sub_mask]
if len(sub_idx) == 0:
    print(" - no EDA rows for this (interval,condition), skipping.")
    continue

# Align labels to subset
labs_sub = labels.reindex(sub_idx).dropna()
if labs_sub.empty:
    print(" - no label overlap with EDA subset, skipping.")
    continue

X_sub = eda.loc[labs_sub.index, MARKERS].to_numpy(dtype="float32")
labels_arr = labs_sub.to_numpy().astype(int)

unique_clusters = np.unique(labels_arr)
print(" clusters found:", unique_clusters)

for cid in unique_clusters:
    if cid == -1 and algo.lower() == "hdbscan":
        # skip HDBSCAN noise
        continue

    mask_c = labels_arr == cid
    n_cluster = int(mask_c.sum())
    n_rest = int((~mask_c).sum())

    if n_cluster < min_cluster_size:
        # skip tiny clusters
        continue

    X_c = X_sub[mask_c]
    X_r = X_sub[~mask_c]

    stats = compute_cluster_marker_stats(X_c, X_r)

```

```

for j, marker in enumerate(MARKERS):
    # Mann-Whitney U: cluster vs rest
    try:
        stat, pval = mannwhitneyu(
            X_c[:, j], X_r[:, j],
            alternative="two-sided"
        )
    except ValueError:
        stat, pval = np.nan, np.nan

    enrich_rows.append({
        "interval": interval,
        "condition": cond_int,
        "condition_name": cond_name,
        "algorithm": algo,
        "pipeline": pipeline,
        "space": space,
        "umap_dim": dim,
        "cluster_id": int(cid),
        "labels_csv": labels_path,

        "marker": marker,
        "n_cells_cluster": n_cluster,
        "n_cells_rest": n_rest,

        "cluster_mean": float(stats["cluster_mean"][j]),
        "cluster_median": float(stats["cluster_median"][j]),
        "rest_mean": float(stats["rest_mean"][j]),
        "rest_median": float(stats["rest_median"][j]),
        "mean_diff": float(stats["mean_diff"][j]),
        "median_diff": float(stats["median_diff"][j]),
        "log2FC_mean": float(stats["log2FC_mean"][j]),

        "mw_stat": float(stat),
        "mw_pvalue": float(pval),
    })

print("\nFinished computing raw enrichment rows.")
enrich_df = pd.DataFrame(enrich_rows)
print("Raw enrichment table shape:", enrich_df.shape)

# 5. Per-cluster Benjamini-Hochberg (q-values)
# i.e. correct over markers *within each cluster*

enrich_df["mw_qvalue"] = np.nan

group_cols = ["algorithm", "interval", "condition", "cluster_id"]

for key, sub in enrich_df.groupby(group_cols):
    idx = sub.index
    qvals = benjamini_hochberg(sub["mw_pvalue"].values)
    enrich_df.loc[idx, "mw_qvalue"] = qvals

# Optional: mark significant (e.g. q < 0.05)
enrich_df["significant_q05"] = enrich_df["mw_qvalue"] < 0.05

print("Enrichment table with per-cluster FDR:", enrich_df.shape)

# Save tidy enrichment table for plotting
out_path = os.path.join(

```



```
        output_dir,  
        "cluster_marker_enrichment_log2FC_MW_best_per_interval_per_algo.csv"  
    )  
    enrich_df.to_csv(out_path, index=False)  
    print("Saved enrichment table →", out_path)  
  
    # Quick peek  
    display(enrich_df.head())
```

```
Run: 1-10w | Control | KMeans | pipeline2 | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_uma
p2__1-10w_0.csv
  clusters found: [0 1 2]

Run: 1-10w | PE | KMeans | pipeline2 | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_uma
p2__1-10w_1.csv
  clusters found: [0 1 2 3 4]

Run: 11-20w | Control | KMeans | pipeline2 | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_uma
p2__11-20w_0.csv
  clusters found: [0 1 2 3]

Run: 11-20w | PE | KMeans | pipeline2 | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_uma
p2__11-20w_1.csv
  clusters found: [0 1 2 3 4 5]

Run: 21-30w | Control | KMeans | pipeline2 | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_uma
p2__21-30w_0.csv
  clusters found: [0 1 2]

Run: 21-30w | PE | KMeans | pipeline2 | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_kmeans_pipeline2_uma
p2__21-30w_1.csv
  clusters found: [0 1 2 3 4 5 6 7 8 9]

Run: 1-10w | Control | HDBSCAN | UMAP (P2, base) | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_hdbscan_pipeline2_um
ap2__1-10w_0.csv
  clusters found: [-1 0 1 2 3 4]

Run: 1-10w | PE | HDBSCAN | UMAP (P2, base) | space=umap5, dim=5.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_hdbscan_pipeline2_um
ap5__1-10w_1.csv
  clusters found: [0 1 2]

Run: 11-20w | Control | HDBSCAN | UMAP (P2, tuned) | space=umap5, dim=5.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_hdbscan_pipeline2_um
ap5_tuned__11-20w_0.csv
  clusters found: [0 1 2]

Run: 11-20w | PE | HDBSCAN | UMAP (P2, base) | space=umap5, dim=5.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_hdbscan_pipeline2_um
ap5__11-20w_1.csv
  clusters found: [0 1 2]

Run: 21-30w | Control | HDBSCAN | UMAP (P2, tuned) | space=umap3, dim=3.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_hdbscan_pipeline2_um
ap3_tuned__21-30w_0.csv
  clusters found: [-1 0 1 2 3]

Run: 21-30w | PE | HDBSCAN | UMAP (P2, base) | space=umap5, dim=5.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_hdbscan_pipeline2_um
ap5__21-30w_1.csv
  clusters found: [-1 0 1 2 3 4]
```

```
Run: 11-20w | Control | PhenoGraph | UMAP (P2, base) | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2
_umap2_k75__11-20w_0.csv
  clusters found: [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
21 22 23
24 25 26 27 28 29 30 31 32 33 34]
```

```
Run: 11-20w | PE | PhenoGraph | UMAP (P2, base) | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2
_umap2_k75__11-20w_1.csv
  clusters found: [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
21 22 23
24 25 26 27 28 29 30 31 32 33 34 35 36]
```

```
Run: 1-10w | Control | PhenoGraph | UMAP (P2, base) | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2
_umap2_k30__1-10w_0.csv
  clusters found: [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
21 22 23
24 25 26 27 28 29 30 31 32 33 34 35 36 37]
```

```
Run: 1-10w | PE | PhenoGraph | UMAP (P2, base) | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2
_umap2_k75__1-10w_1.csv
  clusters found: [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18]
```

```
Run: 21-30w | Control | PhenoGraph | UMAP (P2, base) | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2
_umap2_k75__21-30w_0.csv
  clusters found: [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
21 22 23
24 25 26 27 28 29 30 31 32 33 34 35 36 37 38]
```

```
Run: 21-30w | PE | PhenoGraph | UMAP (P2, base) | space=umap2, dim=2.0
  labels: /content/drive/MyDrive/preeclampsia_results/labels_phenograph_pipeline2
_umap2_k50__21-30w_1.csv
  clusters found: [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
21 22 23
24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41]
```

Finished computing raw enrichment rows.

Raw enrichment table shape: (8646, 21)

Enrichment table with per-cluster FDR: (8646, 23)

Saved enrichment table → /content/drive/MyDrive/preeclampsia_results/cluster_mark
er_enrichment_log2FC_MW_best_per_interval_per_algo.csv

	interval	condition	condition_name	algorithm	pipeline	space	umap_dim	cluster_i
0	1-10w	0	Control	KMeans	pipeline2	umap2	2.0	
1	1-10w	0	Control	KMeans	pipeline2	umap2	2.0	
2	1-10w	0	Control	KMeans	pipeline2	umap2	2.0	
3	1-10w	0	Control	KMeans	pipeline2	umap2	2.0	
4	1-10w	0	Control	KMeans	pipeline2	umap2	2.0	

5 rows × 23 columns



Visualization of enriched clusters

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"
        enrich_path = f"{output_dir}/cluster_marker_enrichment_log2FC_MW_best_per_interv

        enrich_df = pd.read_csv(enrich_path)
        print("Enrichment table:", enrich_df.shape)
```

Enrichment table: (8646, 23)

```
In [ ]: # Treating intervals as strings - Important
        enrich_df["interval"] = enrich_df["interval"].astype(str)

        # Loop over all (algorithm, interval, condition) combinations that exist
        for (algo, interval, cond), df_plot in enrich_df.groupby(["algorithm", "interval", "condition"])
            # Optional: skip if no data (shouldn't happen with groupby, but just in case)
            if df_plot.empty:
                continue

            # Pivot: rows = cluster_id, columns = markers, values = log2FC_mean
            heat = df_plot.pivot_table(
                index="cluster_id",
                columns="marker",
                values="log2FC_mean"
            )

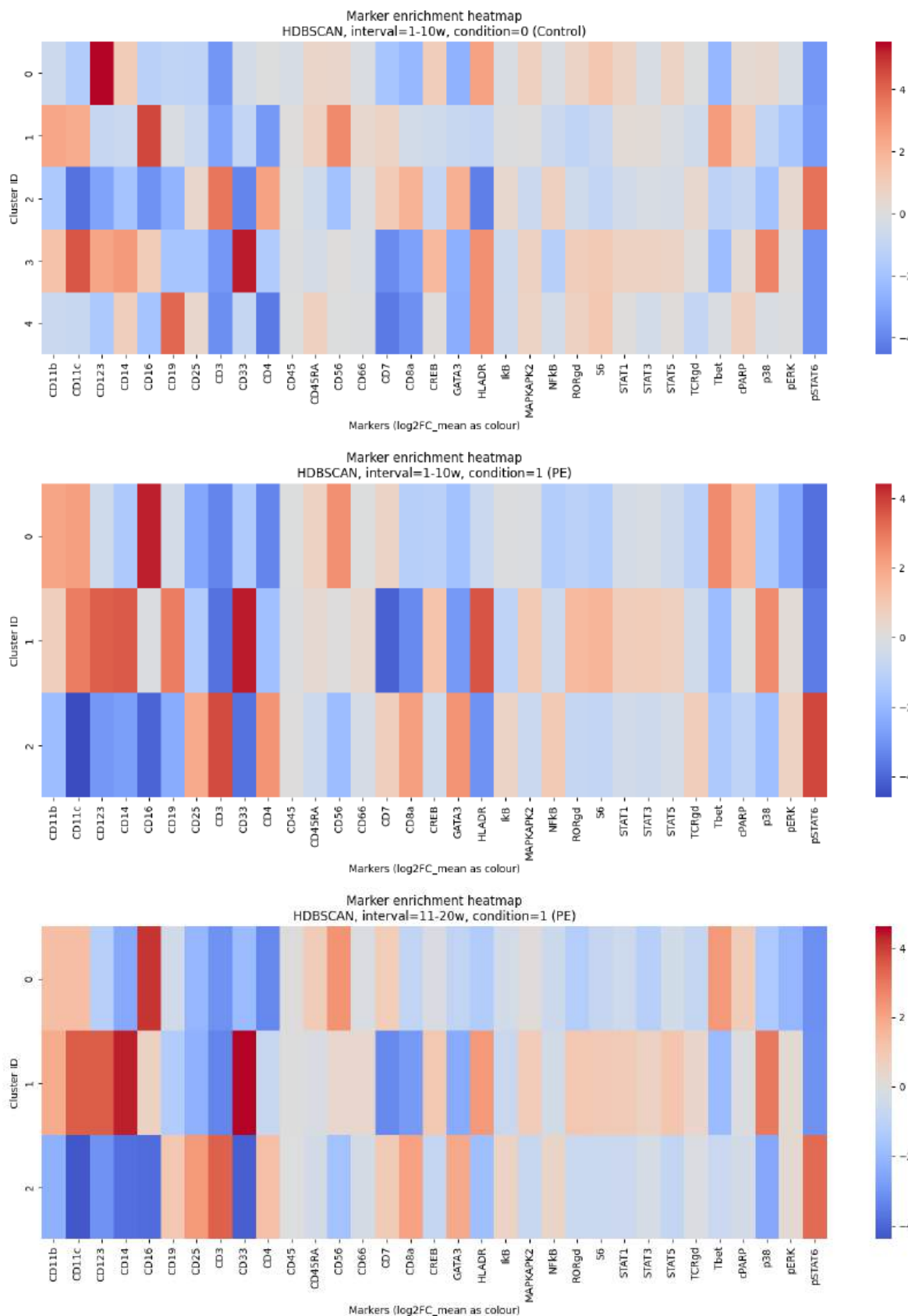
            if heat.empty:
                continue

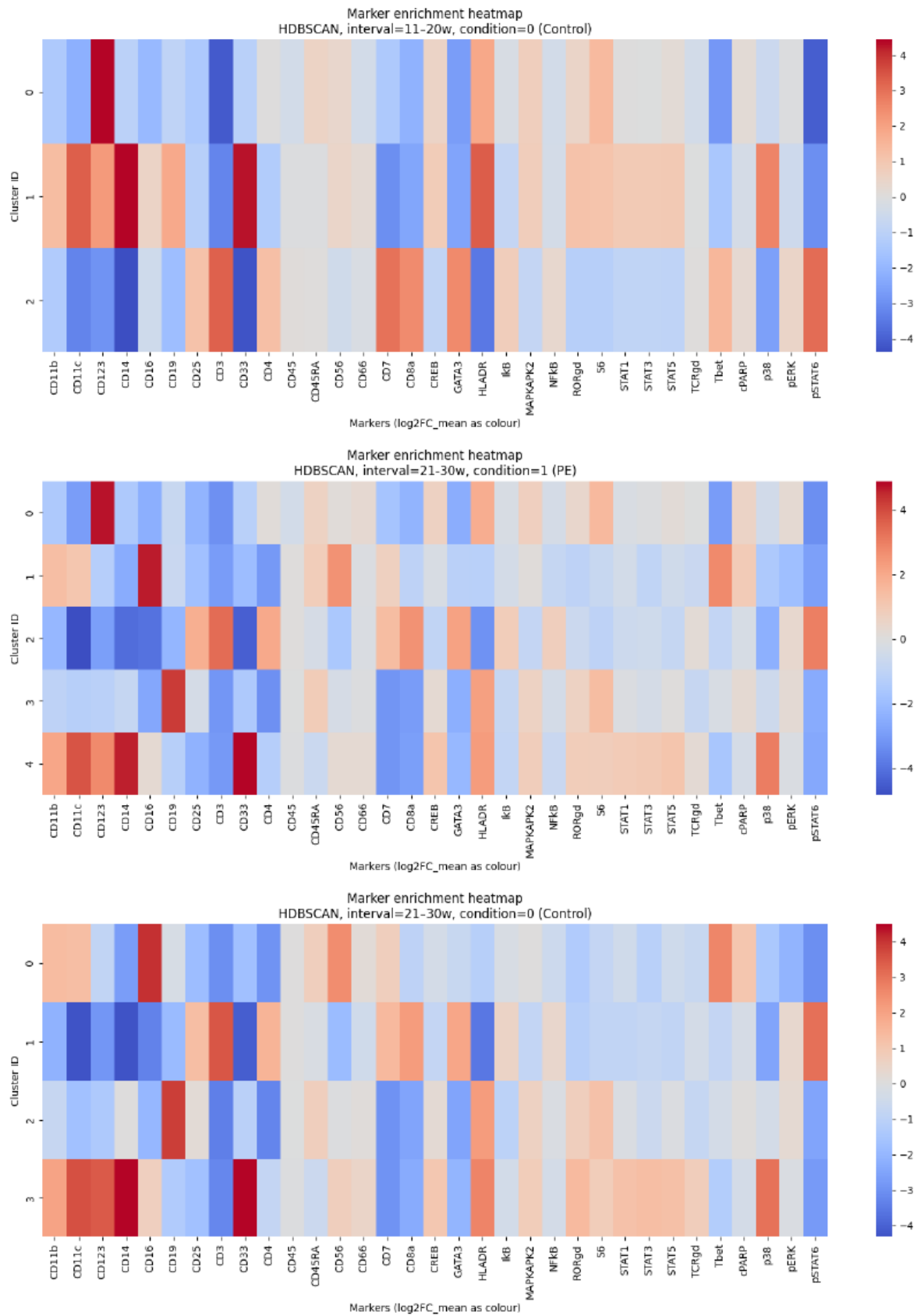
            plt.figure(figsize=(14, 6))
            sns.heatmap(heat, cmap="coolwarm", center=0)

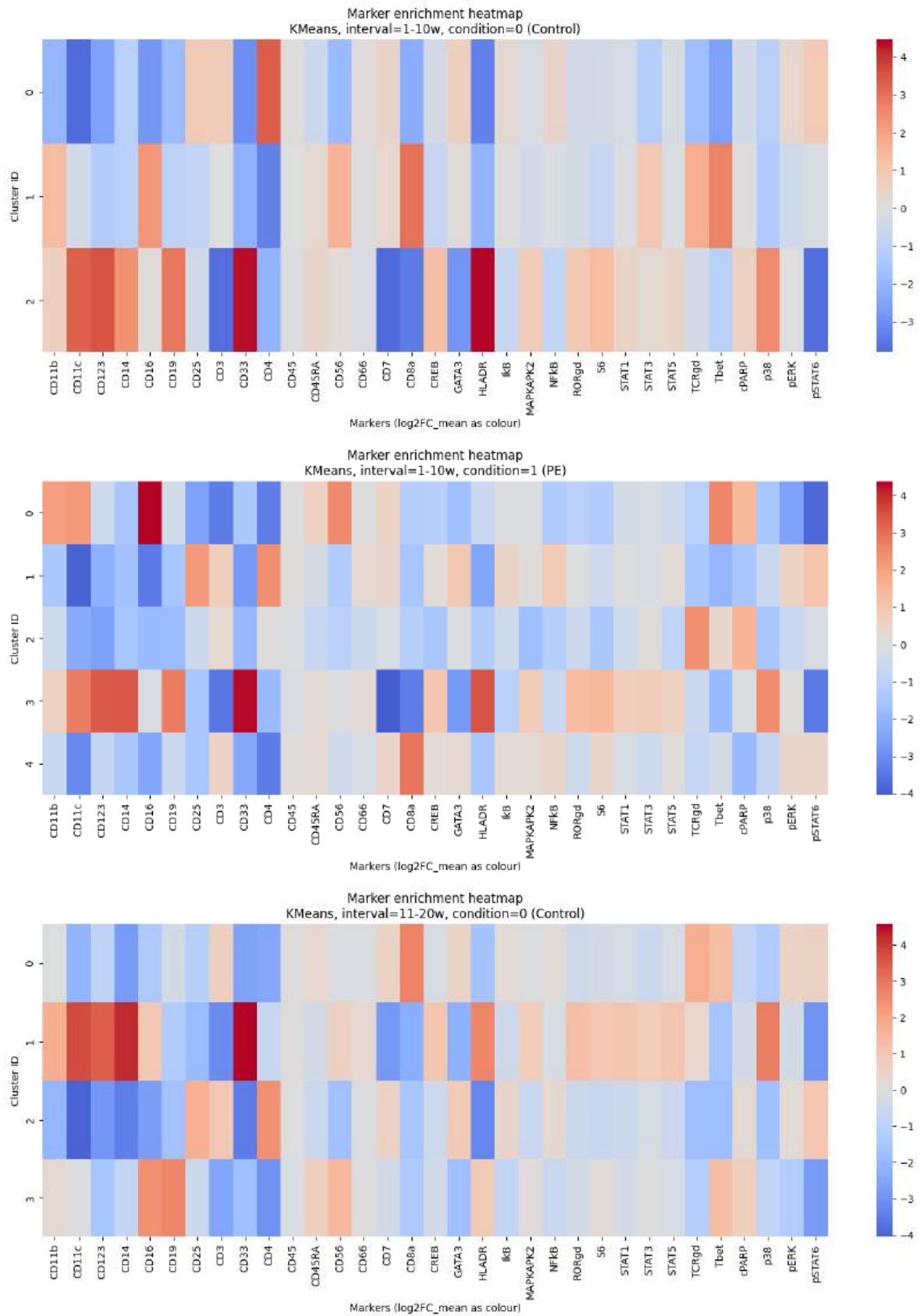
            # Condition name if you have it; otherwise just show the numeric
            if "condition_name" in df_plot.columns:
                cond_label = df_plot["condition_name"].iloc[0]
                cond_title = f"{cond} ({cond_label})"
            else:
                cond_title = str(cond)

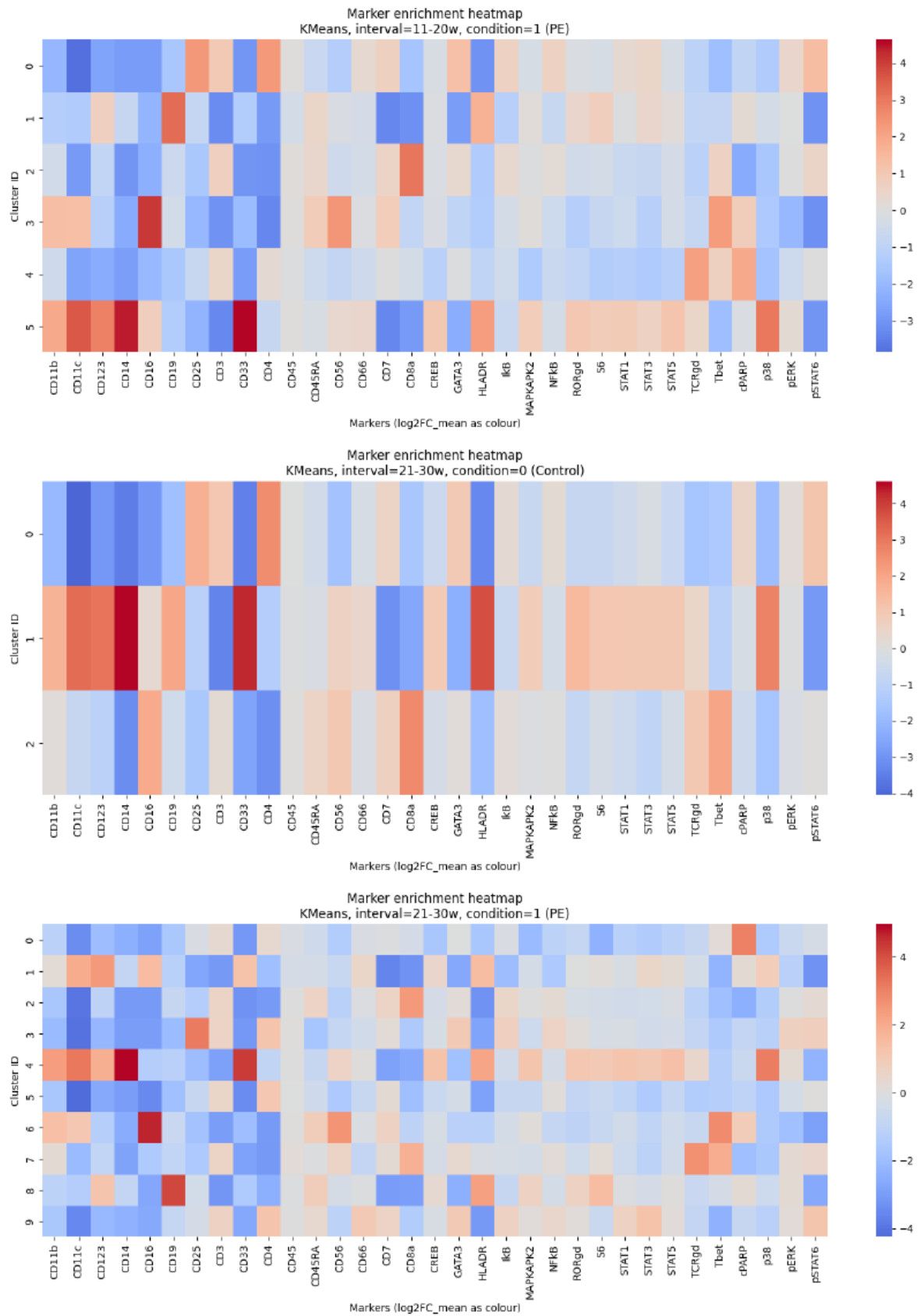
            plt.title(f"Marker enrichment heatmap\n{algo}, interval={interval}, condition={cond}")
            plt.xlabel("Markers (log2FC_mean as colour)")
            plt.ylabel("Cluster ID")
```

```
plt.tight_layout()  
plt.show()
```

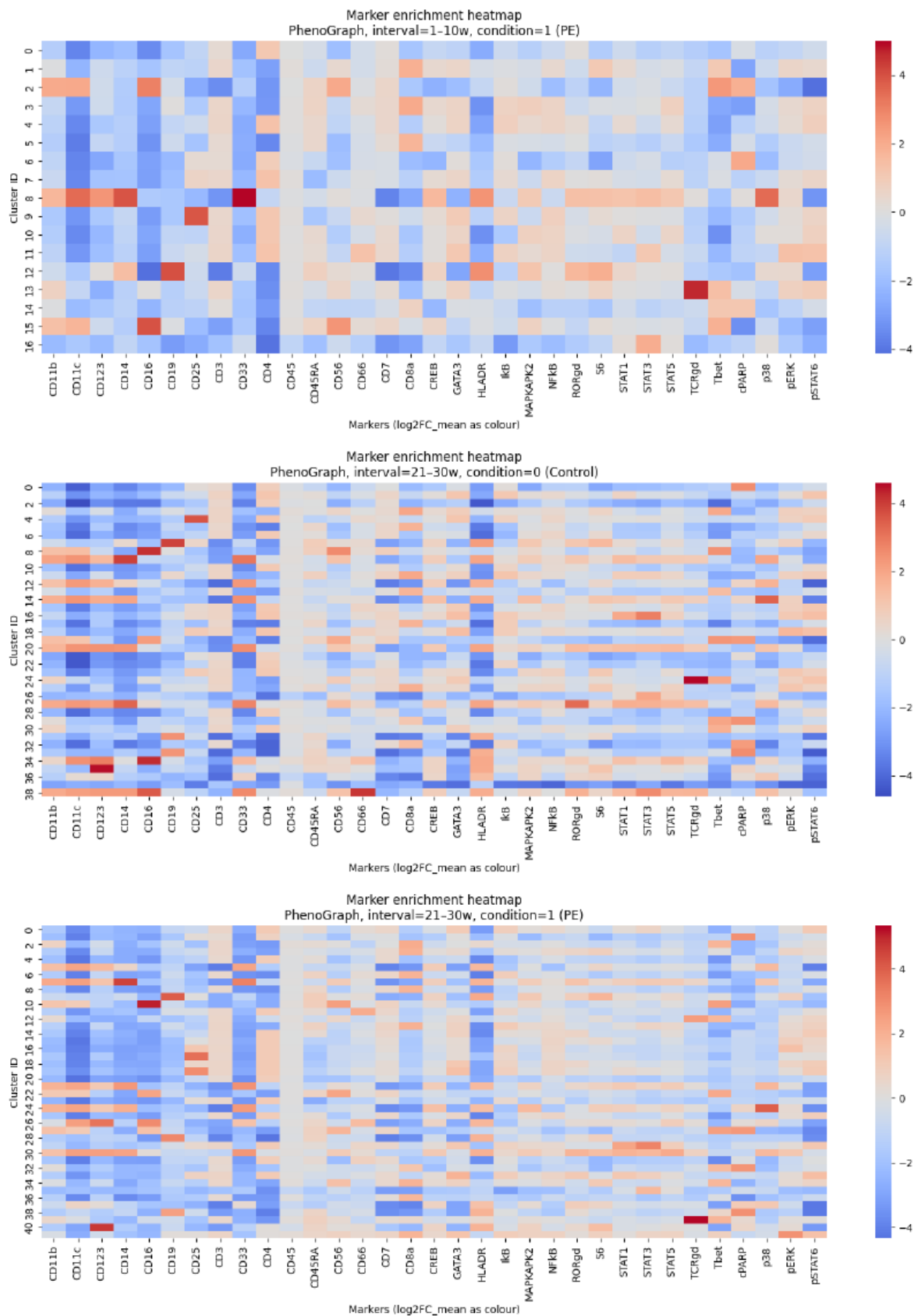












Lollipop chart view of enriched clusters

```
In [ ]: def subset_run(df, algo, interval, cond):
    """
    Return subset of enrichment table for one algorithm / interval / condition.
    """
    return df[
        (df["algorithm"] == algo) &
        (df["interval"] == interval) &
```

```
(df["condition"] == cond)
].copy()
```

```
In [ ]: def plot_lollipop_for_cluster(df_run, cluster_id, top_n=None):
        """
        Lollipop plot of log2FC per marker for one cluster.
        Optionally choose only top_n markers by |log2FC|.
        """
        df_c = df_run[df_run["cluster_id"] == cluster_id].copy()
        if df_c.empty:
            print(f"No rows for cluster {cluster_id}")
            return

        # sort by effect size
        df_c["abs_log2FC"] = df_c["log2FC_mean"].abs()
        df_c = df_c.sort_values("abs_log2FC", ascending=False)

        if top_n is not None:
            df_c = df_c.head(top_n)

        algo = df_c["algorithm"].iloc[0]
        interval = df_c["interval"].iloc[0]
        cond = df_c["condition"].iloc[0]

        plt.figure(figsize=(10, 6))

        y = np.arange(len(df_c))
        plt.hlines(y=y, xmin=0, xmax=df_c["log2FC_mean"], alpha=0.5)
        plt.scatter(df_c["log2FC_mean"], y, c=np.where(df_c["mw_qvalue"] < 0.05, "ta

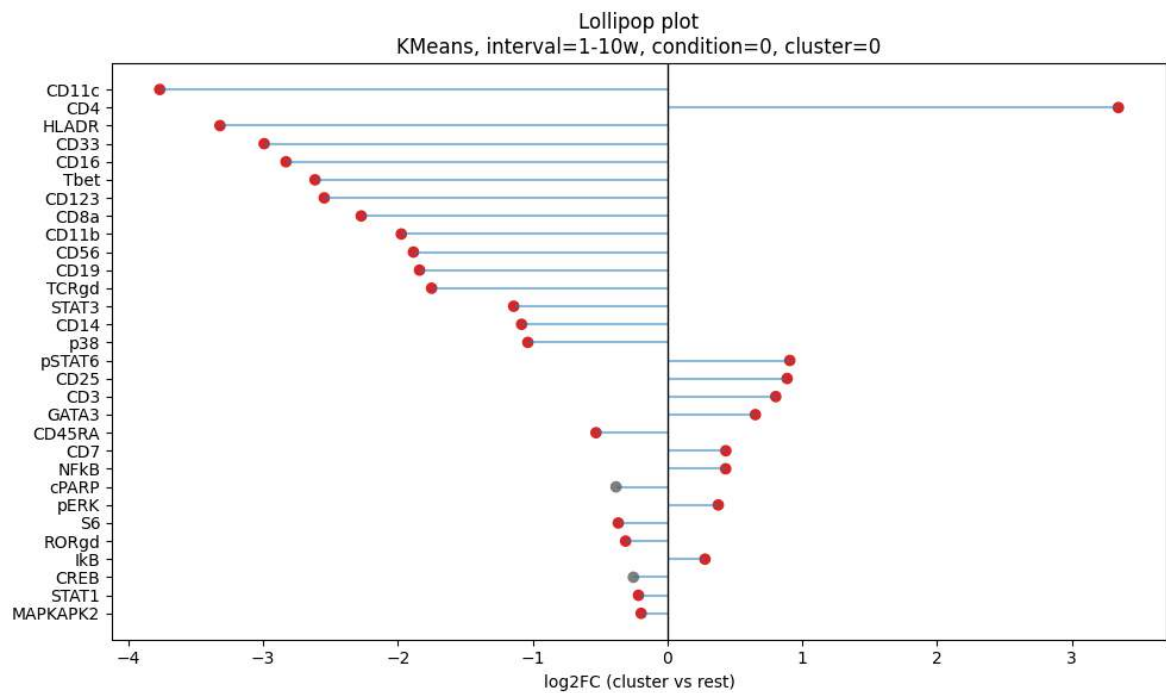
        plt.yticks(y, df_c["marker"])
        plt.axvline(0, color="black", linewidth=1)

        plt.xlabel("log2FC (cluster vs rest)")
        plt.title(f"Lollipop plot\n{algo}, interval={interval}, condition={cond}, cl
        plt.gca().invert_yaxis()
        plt.tight_layout()
        plt.show()

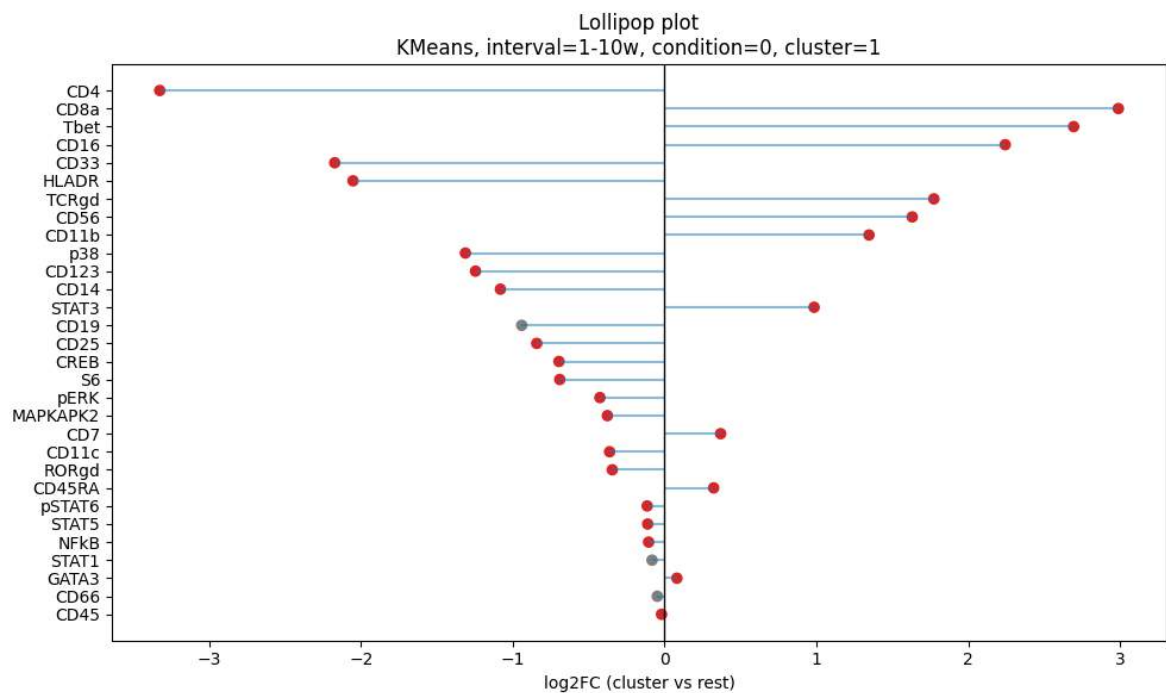
        # example loop over clusters for one run
        algo = "KMeans"
        interval = "1-10w"
        cond = 0

        df_run = subset_run(enrich_df, algo, interval, cond)
        for cid in sorted(df_run["cluster_id"].unique()):
            print(f"Lollipop → {algo}, interval={interval}, condition={cond}, cluster={c
            plot_lollipop_for_cluster(df_run, cid, top_n=30)
```

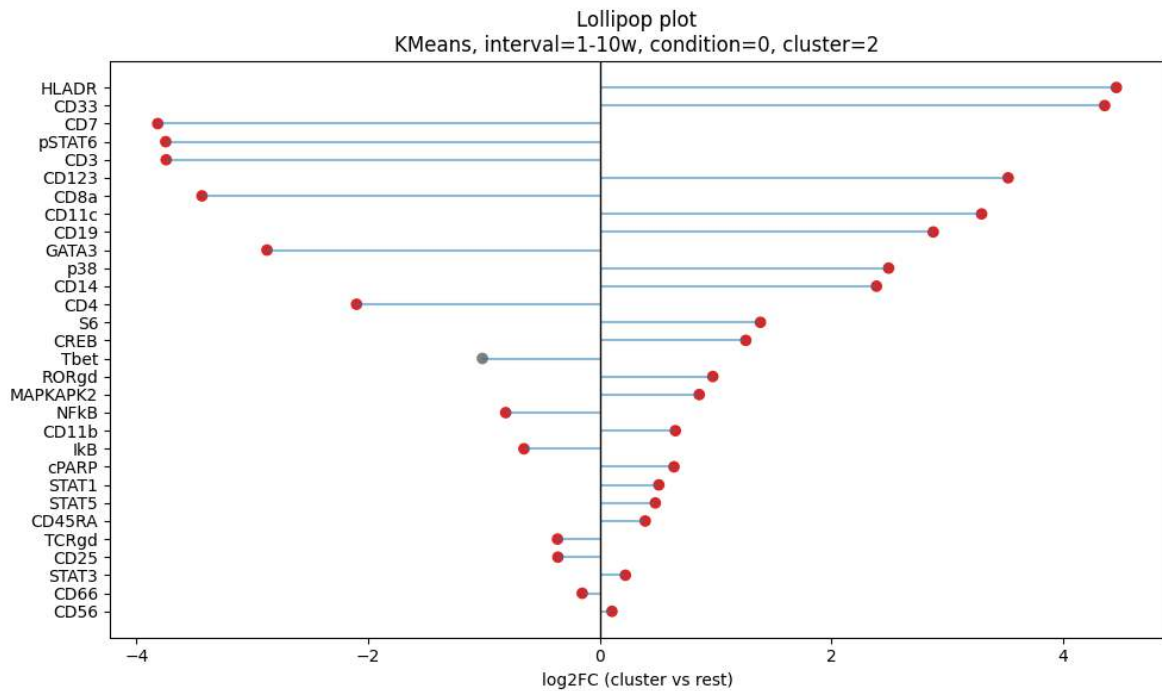
Lollipop → KMeans, interval=1-10w, condition=0, cluster=0



Lollipop → KMeans, interval=1-10w, condition=0, cluster=1



Lollipop → KMeans, interval=1-10w, condition=0, cluster=2



Cluster annotation

```
In [ ]: output_dir = "/content/drive/MyDrive/preeclampsia_results"
        enrich_path = os.path.join(
            output_dir,
            "cluster_marker_enrichment_log2FC_MW_best_per_interval_per_algo.csv"
        )

        enrich_df = pd.read_csv(enrich_path)
        print("Enrichment table:", enrich_df.shape)
        display(enrich_df.head())
```

Enrichment table: (8646, 23)

	interval	condition	condition_name	algorithm	pipeline	space	umap_dim	cluster_i
0	1-10w	0	Control	KMeans	pipeline2	umap2	2.0	
1	1-10w	0	Control	KMeans	pipeline2	umap2	2.0	
2	1-10w	0	Control	KMeans	pipeline2	umap2	2.0	
3	1-10w	0	Control	KMeans	pipeline2	umap2	2.0	
4	1-10w	0	Control	KMeans	pipeline2	umap2	2.0	

5 rows × 23 columns



I meticulously annotated all cell type combinations in weighted manner. As shown below

```
In [ ]: # Normalise interval dashes right after loading
        enrich_df["interval"] = (
            enrich_df["interval"]
            .astype(str)
            .str.replace("-", "-", regex=False)
```

```
.str.replace("-", "-", regex=False)
)
```

```
In [ ]: celltype_defs = [
    {"cell_type": "All Leukocytes", "primary": "CD45", "second
    {"cell_type": "All Myeloid Cells", "primary": "CD45", "second
    {"cell_type": "Classical Monocyte", "primary": "CD45", "second
    {"cell_type": "Inflammatory Monocyte", "primary": "CD45", "second
    {"cell_type": "Myeloid Dendritic Cell (mDC)", "primary": "CD45", "second
    {"cell_type": "Plasmacytoid DC (pDC)", "primary": "CD45", "second
    {"cell_type": "Neutrophil", "primary": "CD45", "second
    {"cell_type": "NK Cell", "primary": "CD45", "second
    {"cell_type": "CD56-Bright NK Cell", "primary": "CD45", "second
    {"cell_type": "NKT-like Cell", "primary": "CD45", "second
    {"cell_type": "Total T Cells", "primary": "CD45", "second
    {"cell_type": "Helper T Cell (Th)", "primary": "CD45", "second
    {"cell_type": "Cytotoxic T Cell (CTL)", "primary": "CD45", "second
    {"cell_type": "Gamma-Delta T Cell", "primary": "CD45", "second
    {"cell_type": "Naive T Cell", "primary": "CD45", "second
    {"cell_type": "Activated T Cell", "primary": "CD45", "second
    {"cell_type": "Th1 Cell", "primary": "CD45", "second
    {"cell_type": "Th2 Cell", "primary": "CD45", "second
    {"cell_type": "Th17 Cell", "primary": "CD45", "second
    {"cell_type": "Total B Cells", "primary": "CD45", "second
    {"cell_type": "Naive B Cell", "primary": "CD45", "second
    {"cell_type": "Activated B Cell", "primary": "CD45", "second
    {"cell_type": "Apoptotic Cell", "primary": "CD45", "second
    {"cell_type": "Proliferating Cell", "primary": "CD45", "second
    {"cell_type": "Stressed Cell", "primary": "CD45", "second
    {"cell_type": "IL-4/IL-13 Responsive", "primary": "CD45", "second
    {"cell_type": "NF-κB Activated", "primary": "CD45", "second
    {"cell_type": "IFN-Responsive", "primary": "CD45", "second
    {"cell_type": "IL-6 Responsive", "primary": "CD45", "second
    {"cell_type": "CREB Active", "primary": "CD45", "second
]

celltype_df = pd.DataFrame(celltype_defs)
display(celltype_df.head())
```

	cell_type	primary	secondary	tertiary	quaternary
0	All Leukocytes	CD45	None	None	None
1	All Myeloid Cells	CD45	CD33	None	None
2	Classical Monocyte	CD45	CD33	CD14	HLADR
3	Inflammatory Monocyte	CD45	CD33	CD14	CD11b
4	Myeloid Dendritic Cell (mDC)	CD45	CD33	CD11c	HLADR

```
In [ ]: # Helper: score cell types per cluster

# Role weights: you can tweak these
ROLE_WEIGHTS = {
    "primary": 4.0,
    "secondary": 3.0,
    "tertiary": 2.0,
    "quaternary": 1.0,
```



```

}

# thresholds
LOG2FC_MIN = 0.25 # minimal up-regulation to count as "enriched"
Q_MAX      = 0.05 # FDR threshold

def score_celltype_for_cluster(cluster_markers_df):
    """
    cluster_markers_df: subset of enrich_df for a single (algo, interval, condit
                        each row = one marker.
    Returns list of dicts: one row per (cell_type, cluster) with a score.
    """
    results = []

    # quick mapping from marker -> row (assume unique marker names per cluster)
    marker_map = {
        m: row for m, row in cluster_markers_df.set_index("marker").iterrows()
    }

    # cluster-level metadata (reuse across cell types)
    meta_cols = ["interval", "condition", "condition_name",
                 "algorithm", "pipeline", "space", "umap_dim",
                 "cluster_id", "labels_csv"]
    meta = {c: cluster_markers_df.iloc[0][c] for c in meta_cols if c in cluster_

    # some clusters may have n_cells_cluster in each row
    if "n_cells_cluster" in cluster_markers_df.columns:
        meta["n_cells_cluster"] = int(cluster_markers_df["n_cells_cluster"].iloc

    for _, ct in celltype_df.iterrows():
        ct_name = ct["cell_type"]
        score = 0.0
        n_markers = 0
        n_sig = 0
        max_log2fc = -np.inf

        for role in ["primary", "secondary", "tertiary", "quaternary"]:
            marker = ct[role]
            if marker is None or (isinstance(marker, float) and pd.isna(marker)):
                continue

            if marker not in marker_map:
                continue

            row = marker_map[marker]
            log2fc = row.get("log2FC_mean", np.nan)
            qval = row.get("mw_qvalue", np.nan)

            if pd.isna(log2fc):
                continue

            n_markers += 1
            max_log2fc = max(max_log2fc, log2fc)

            # only count if up-regulated and significant
            if (log2fc >= LOG2FC_MIN) and (not pd.isna(qval)) and (qval <= Q_MAX
                w = ROLE_WEIGHTS[role]
                score += w * log2fc
                n_sig += 1

```

```

    if n_markers == 0:
        continue # no markers present for this cell type in this cluster

    if score <= 0:
        # if you want *all* cell-type hints, remove this check
        continue

    result_row = {
        **meta,
        "cell_type": ct_name,
        "ct_score": score,
        "ct_n_markers": n_markers,
        "ct_n_sig_markers": n_sig,
        "ct_max_log2fc": max_log2fc,
    }
    results.append(result_row)

return results

```

```

In [ ]: # Build cluster × cell-type annotation table

cluster_keys = ["algorithm", "interval", "condition", "cluster_id"]

annot_rows = []

# group enrichment rows by cluster (algo, interval, condition, cluster_id)
for cluster_id, df_cluster in enrich_df.groupby(cluster_keys):
    rows = score_celltype_for_cluster(df_cluster)
    annot_rows.extend(rows)

cluster_ann = pd.DataFrame(annot_rows)

print("Cluster-level cell-type annotations:", cluster_ann.shape)
display(cluster_ann.head())

# Save for later use
ann_path = os.path.join(output_dir, "cluster_celltype_annotations_per_interval_p
cluster_ann.to_csv(ann_path, index=False)
print("Saved cluster annotations →", ann_path)

```

Cluster-level cell-type annotations: (3756, 15)

	interval	condition	condition_name	algorithm	pipeline	space	umap_dim	cluster_ic
0	1-10w	0	Control	HDBSCAN	UMAP (P2, base)	umap2	2.0	(
1	1-10w	0	Control	HDBSCAN	UMAP (P2, base)	umap2	2.0	(
2	1-10w	0	Control	HDBSCAN	UMAP (P2, base)	umap2	2.0	(
3	1-10w	0	Control	HDBSCAN	UMAP (P2, base)	umap2	2.0	(
4	1-10w	0	Control	HDBSCAN	UMAP (P2, base)	umap2	2.0	(

◀  ▶

Saved cluster annotations → /content/drive/MyDrive/preeclampsia_results/cluster_c
elltype_annotations_per_interval_per_algo.csv

Cell type pattern and enrichment PE vs Control

```
In [ ]: # VISUAL 1: Cluster × Cell-type Heatmap
# per (algorithm, interval, condition)

def plot_cluster_celltype_heatmap(cluster_ann,
                                   algo="KMeans",
                                   interval="1-10w",
                                   cond=0,
                                   vmin=None, vmax=None):
    """
    Shows all clusters for a given (algo, interval, condition)
    rows = clusters, cols = cell types, values = ct_score
    """
    df = cluster_ann[
        (cluster_ann["algorithm"] == algo) &
        (cluster_ann["interval"] == interval) &
        (cluster_ann["condition"] == cond)
    ]

    if df.empty:
        print("No annotation rows for", algo, interval, cond)
        return

    heat = df.pivot_table(
        index="cluster_id",
        columns="cell_type",
        values="ct_score",
        aggfunc="max"
    ).fillna(0.0)

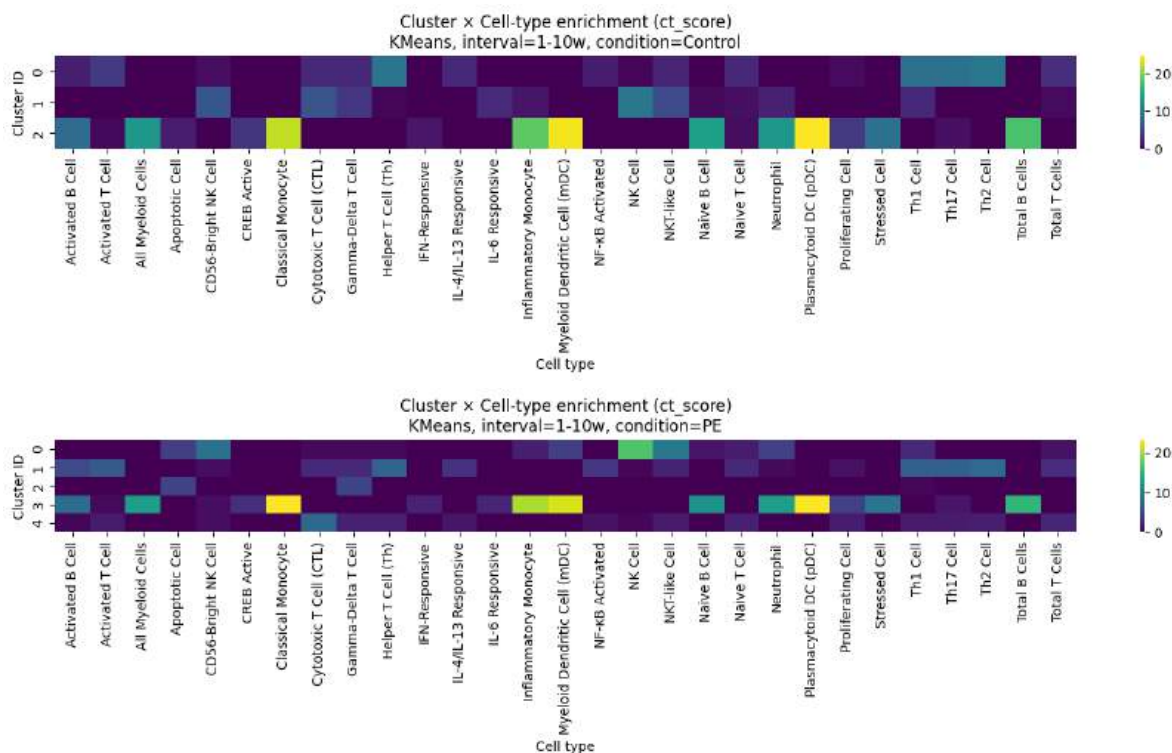
    plt.figure(figsize=(14, max(4, 0.4 * heat.shape[0])))
    sns.heatmap(
        heat,
```

```

        cmap="viridis",
        vmin=vmin,
        vmax=vmax
    )
    cond_name = df["condition_name"].iloc[0] if "condition_name" in df.columns else "Control"
    plt.title(f"Cluster x Cell-type enrichment (ct_score)\n{algo}, interval={interval}")
    plt.xlabel("Cell type")
    plt.ylabel("Cluster ID")
    plt.tight_layout()
    plt.show()

# USAGE:
plot_cluster_celltype_heatmap(cluster_ann, algo="KMeans", interval="1-10w", condition="Control")
plot_cluster_celltype_heatmap(cluster_ann, algo="KMeans", interval="1-10w", condition="PE")

```



To understand the distribution of cells within the cluster, below is an aggregated cell type score bar plot

```

In [ ]: # Cell-type x Condition BarPlot
        # per (algorithm, interval)

def plot_celltype_condition_bar(cluster_ann,
                                algo="KMeans",
                                interval="1-10w",
                                min_score=0.0,
                                top_n=20):
    """
    For a given algorithm + interval, compare PE vs Control:
    x = cell types, y = summed ct_score across clusters
    hue = condition (Control vs PE)
    """
    df = cluster_ann[
        (cluster_ann["algorithm"] == algo) &
        (cluster_ann["interval"] == interval)
    ]

```

```

if df.empty:
    print("No annotation rows for", algo, interval)
    return

# aggregate scores by (condition, cell_type)
agg = (
    df.groupby(["condition", "condition_name", "cell_type"], dropna=False)["ct_score"]
      .sum()
      .reset_index()
)

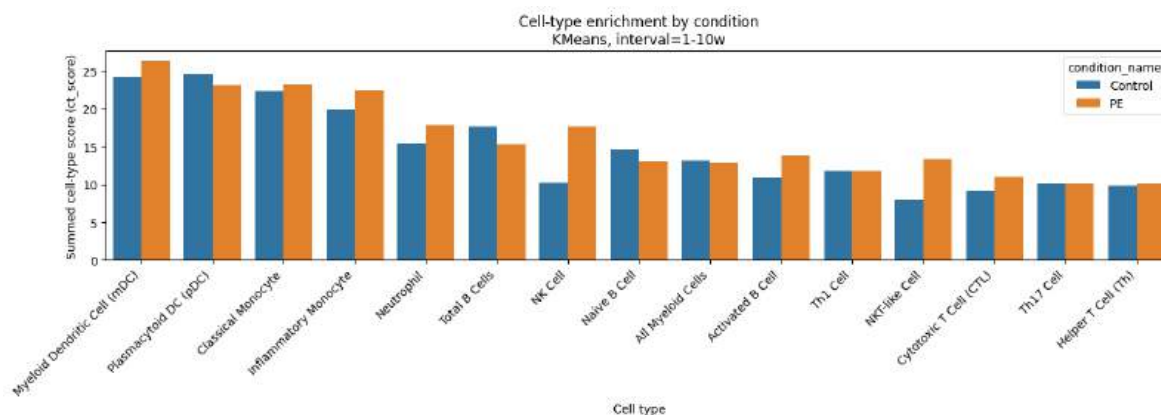
# filter weakly-scored types if desired
agg = agg[agg["ct_score"] >= min_score]

# keep only top N by overall score (sum across conditions)
agg["total_score"] = agg.groupby("cell_type")["ct_score"].transform("sum")
agg = agg.sort_values("total_score", ascending=False)
top_ct = agg["cell_type"].unique()[:top_n]
agg = agg[agg["cell_type"].isin(top_ct)]

plt.figure(figsize=(14, 5))
sns.barplot(
    data=agg,
    x="cell_type",
    y="ct_score",
    hue="condition_name"
    if "condition_name" in agg.columns
    else "condition"
)
plt.xticks(rotation=45, ha="right")
plt.ylabel("Summed cell-type score (ct_score)")
plt.xlabel("Cell type")
plt.title(f"Cell-type enrichment by condition\n{algo}, interval={interval}")
plt.tight_layout()
plt.show()

# EXAMPLE USAGE:
plot_celltype_condition_bar(cluster_ann, algo="KMeans", interval="1-10w", min_sc
plot_celltype_condition_bar(cluster_ann, algo="Phenograph", interval="11-20w")

```



No annotation rows for Phenograph 11-20w

Heatmap and Bar Plots Cell type cluster annotation

```

In [ ]: algos = sorted(cluster_ann["algorithm"].unique())
        intervals = sorted(cluster_ann["interval"].unique())

        for algo in algos:

```

```

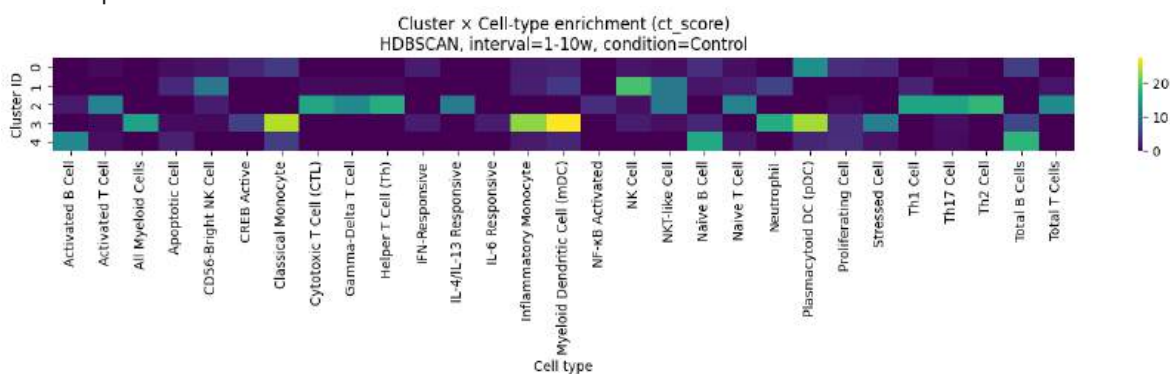
for interval in intervals:
    print(f"\n=== {algo} | interval {interval} ===")

    # Heatmaps for Control (0) and PE (1)
    for cond in sorted(cluster_ann["condition"].dropna().unique()):
        print(f"Heatmap: condition={cond}")
        plot_cluster_celltype_heatmap(cluster_ann,
                                       algo=algo,
                                       interval=interval,
                                       cond=cond)

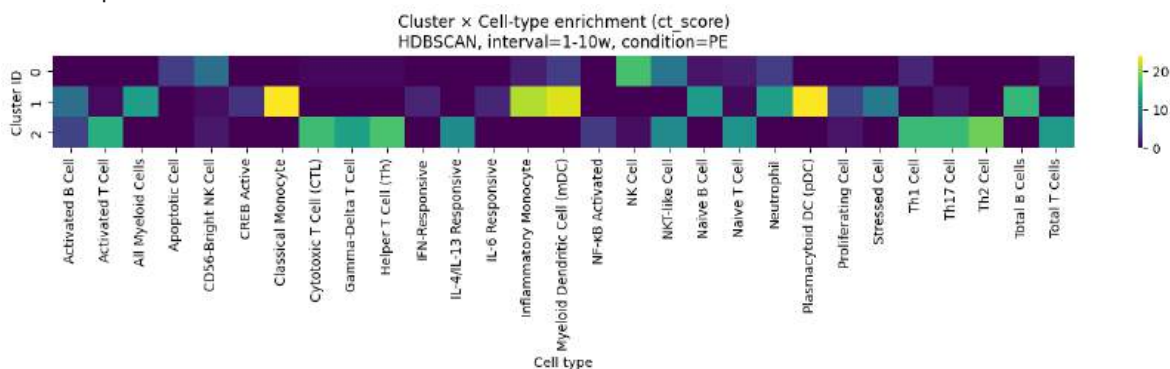
    # Bar comparison PE vs Control
    print("Barplot: PE vs Control")
    plot_celltype_condition_bar(cluster_ann,
                               algo=algo,
                               interval=interval,
                               min_score=0.0,
                               top_n=15)

```

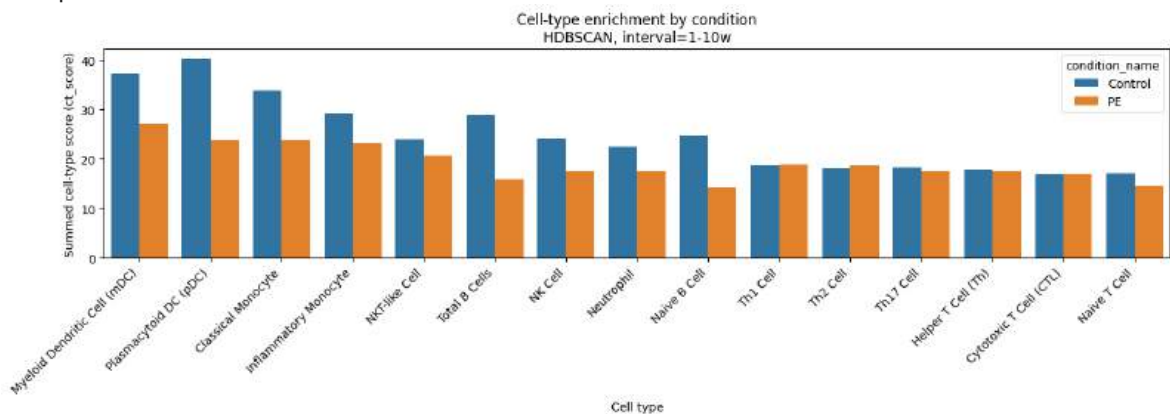
=== HDBSCAN | interval 1-10w ===
Heatmap: condition=0



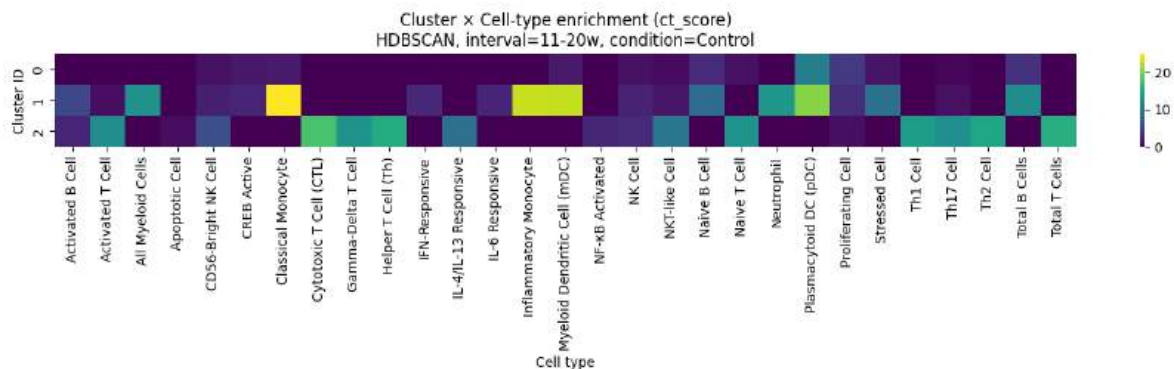
Heatmap: condition=1



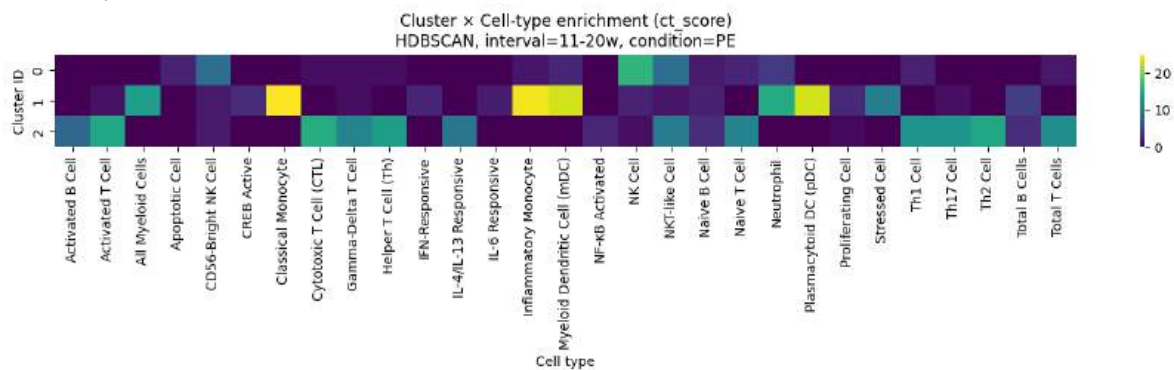
Barplot: PE vs Control



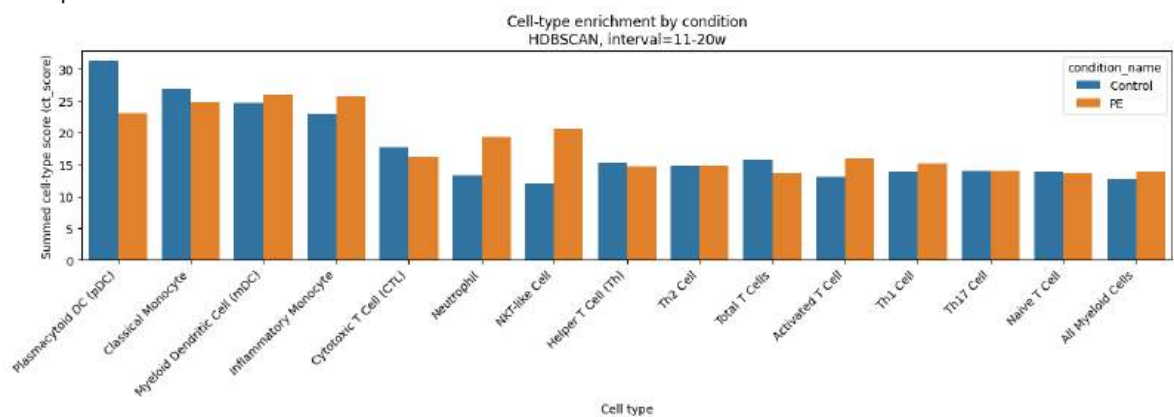
=== HDBSCAN | interval 11-20w ===
Heatmap: condition=0



Heatmap: condition=1

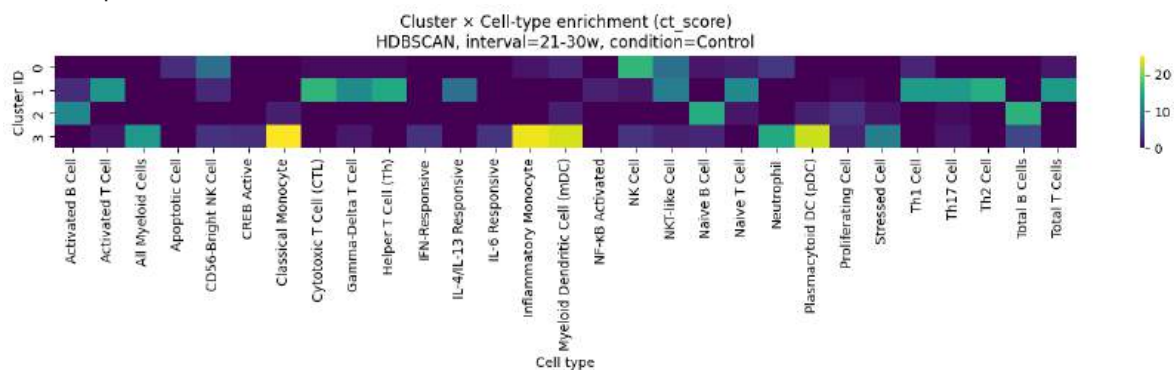


Barplot: PE vs Control

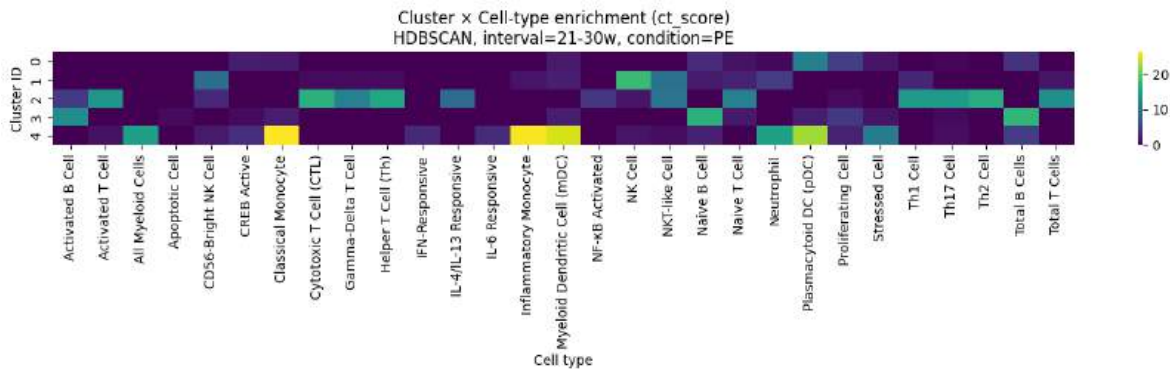


=== HDBSCAN | interval 21-30w ===

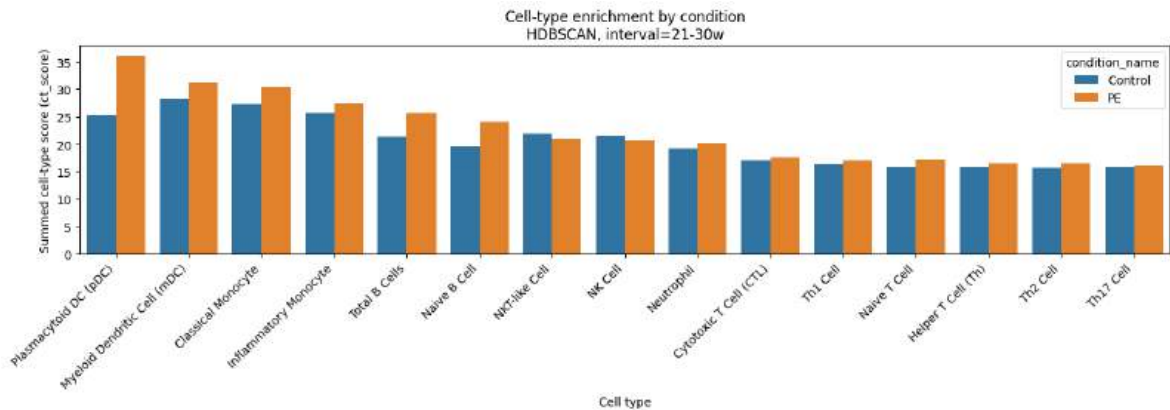
Heatmap: condition=0



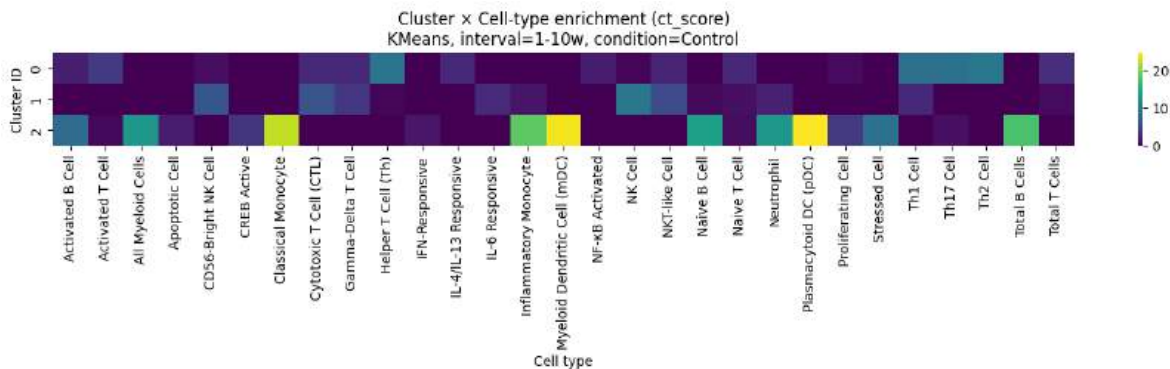
Heatmap: condition=1



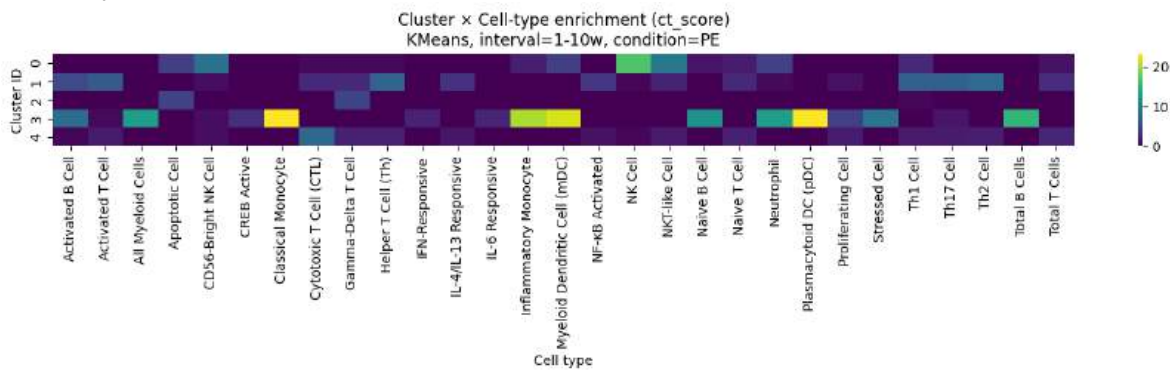
Barplot: PE vs Control



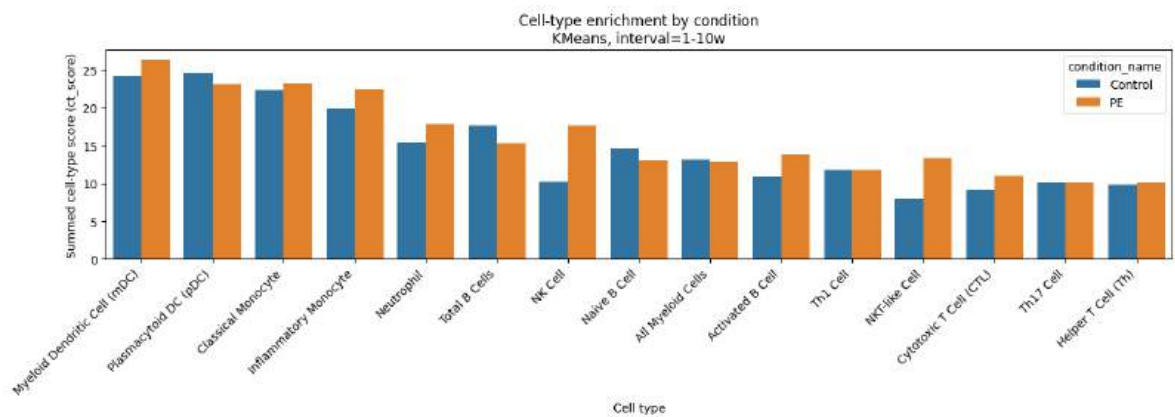
=== KMeans | interval 1-10w ===
Heatmap: condition=0



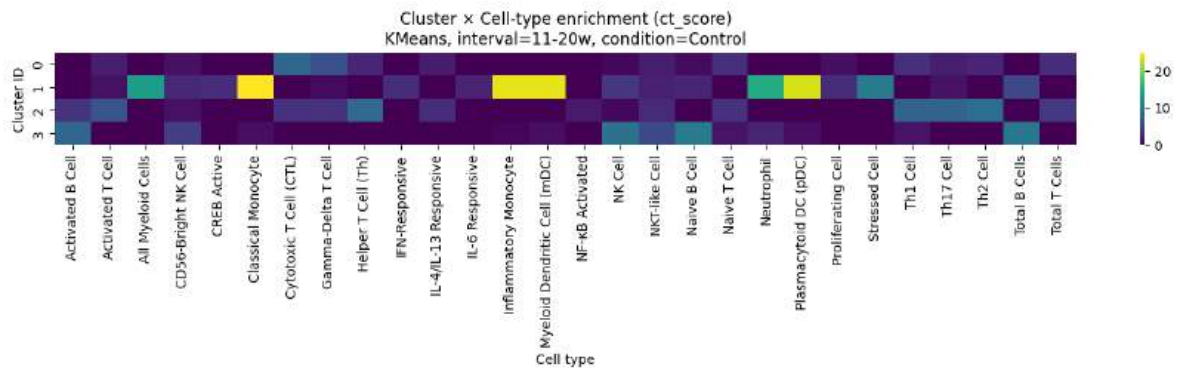
Heatmap: condition=1



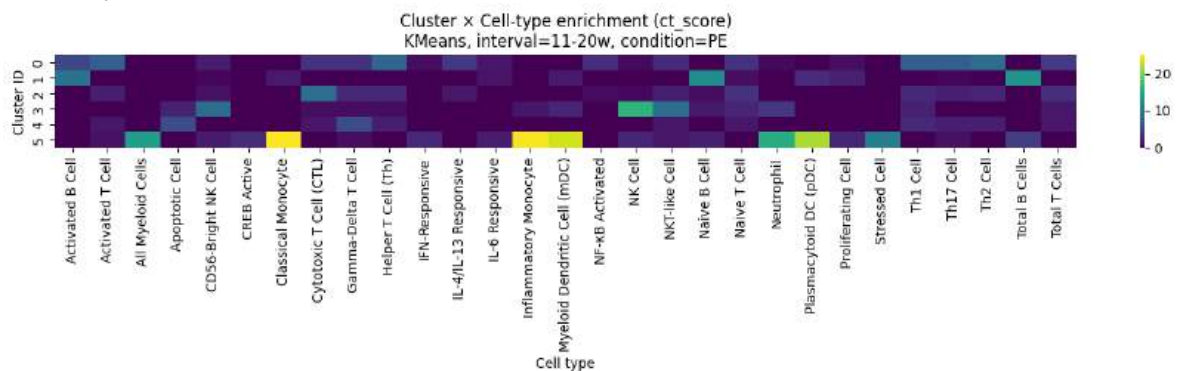
Barplot: PE vs Control



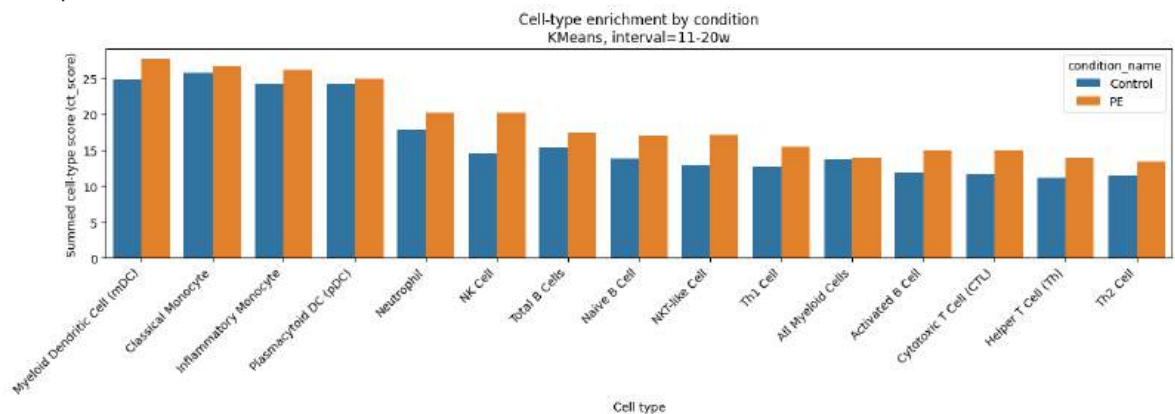
=== KMeans | interval 11-20w ===
Heatmap: condition=0



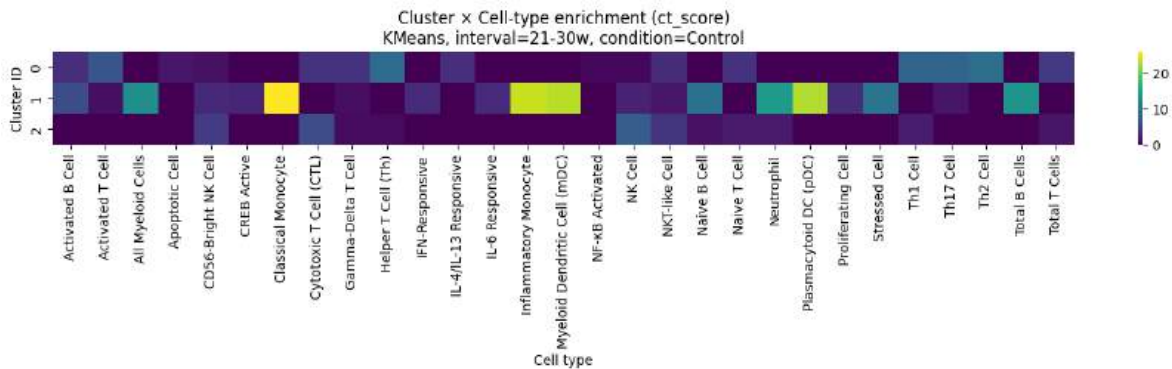
Heatmap: condition=1



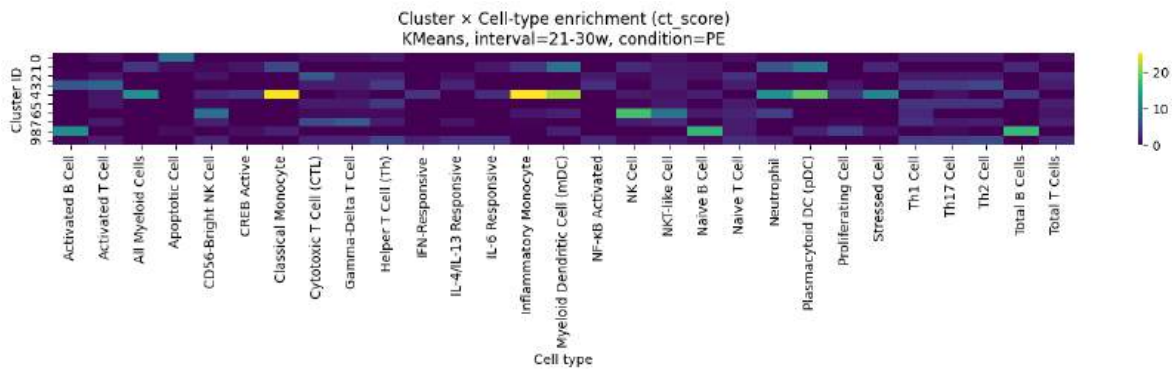
Barplot: PE vs Control



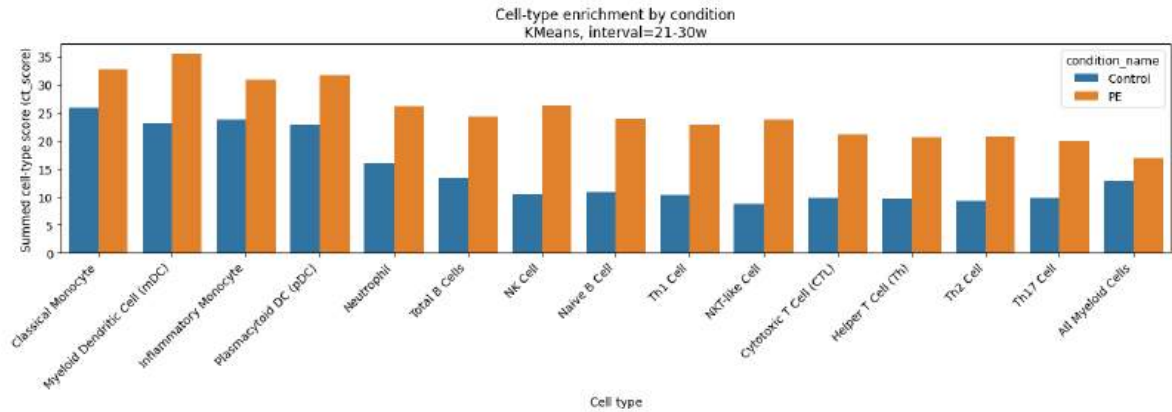
=== KMeans | interval 21-30w ===
Heatmap: condition=0



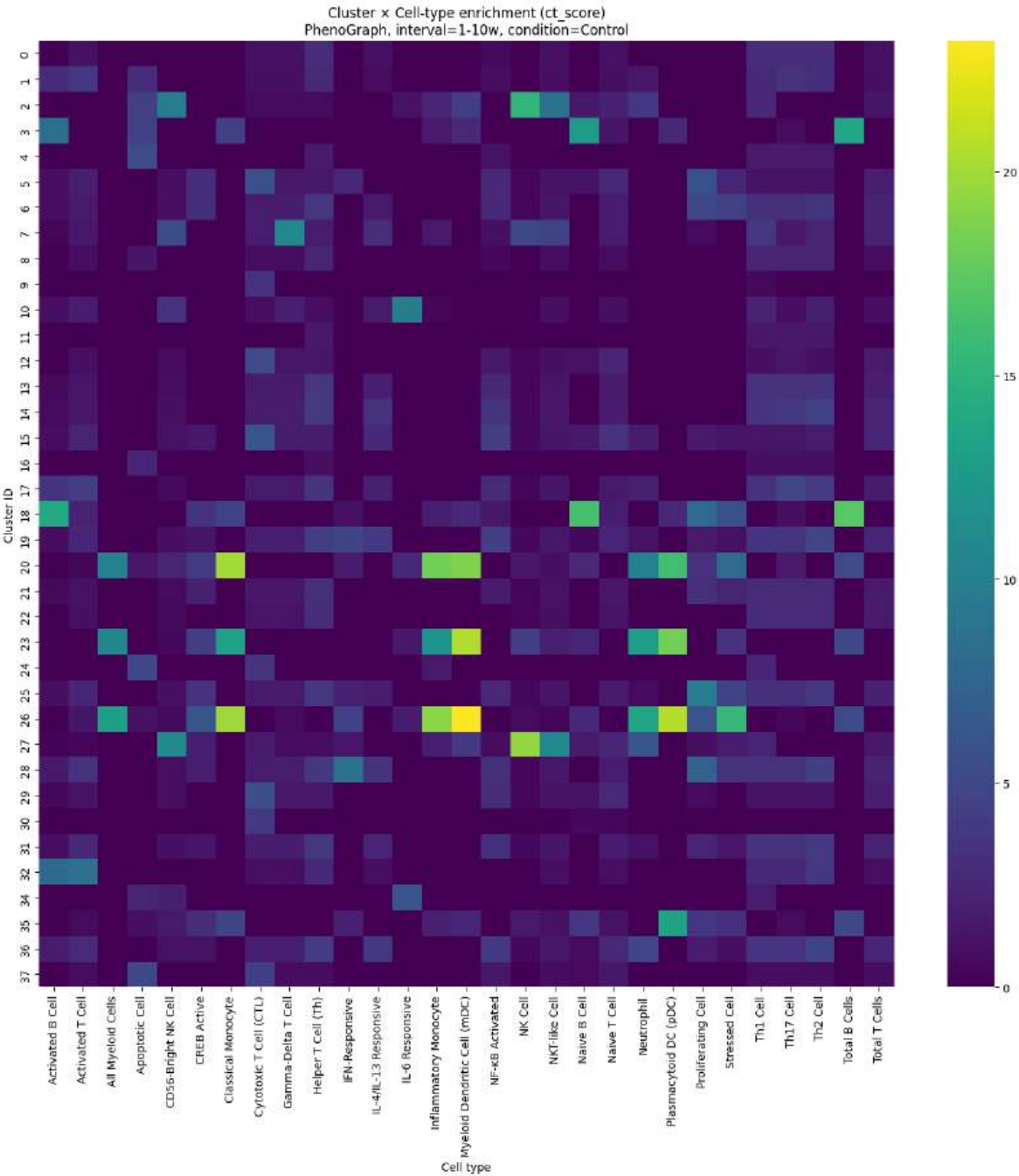
Heatmap: condition=1



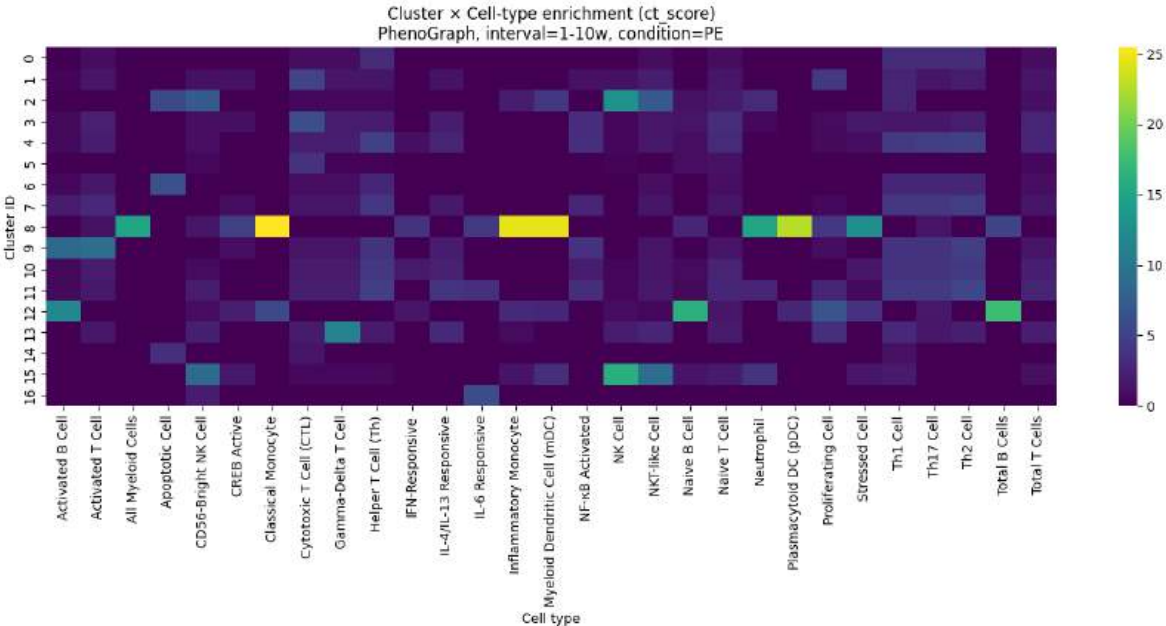
Barplot: PE vs Control



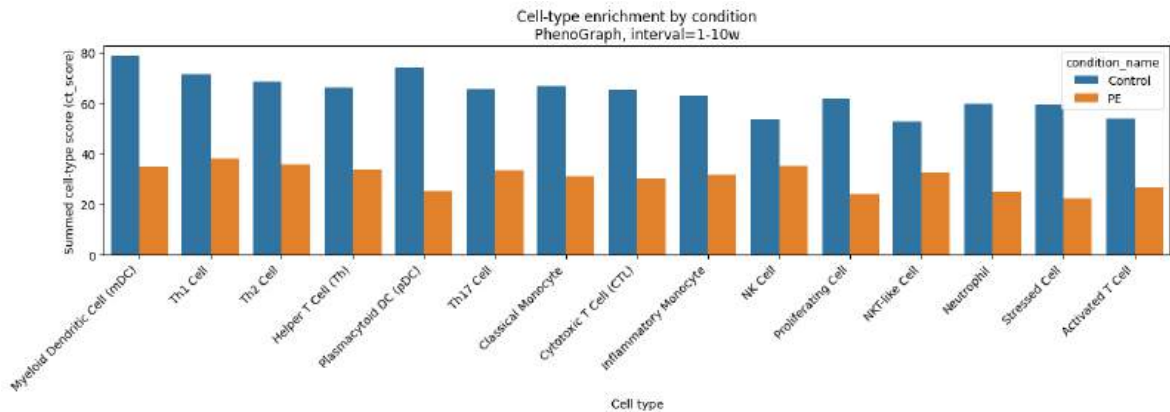
=== PhenoGraph | interval 1-10w ===
Heatmap: condition=0



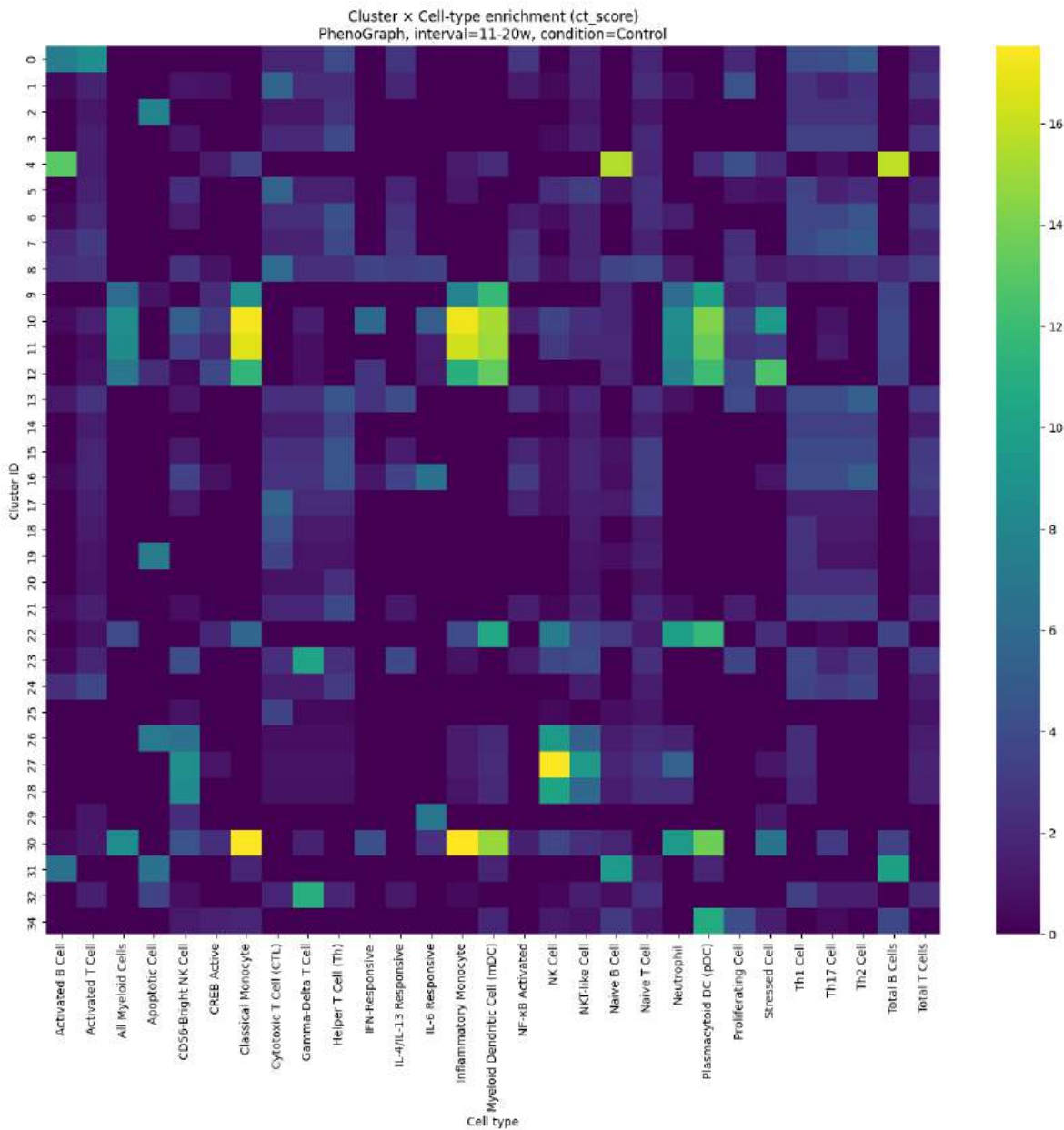
Heatmap: condition=1



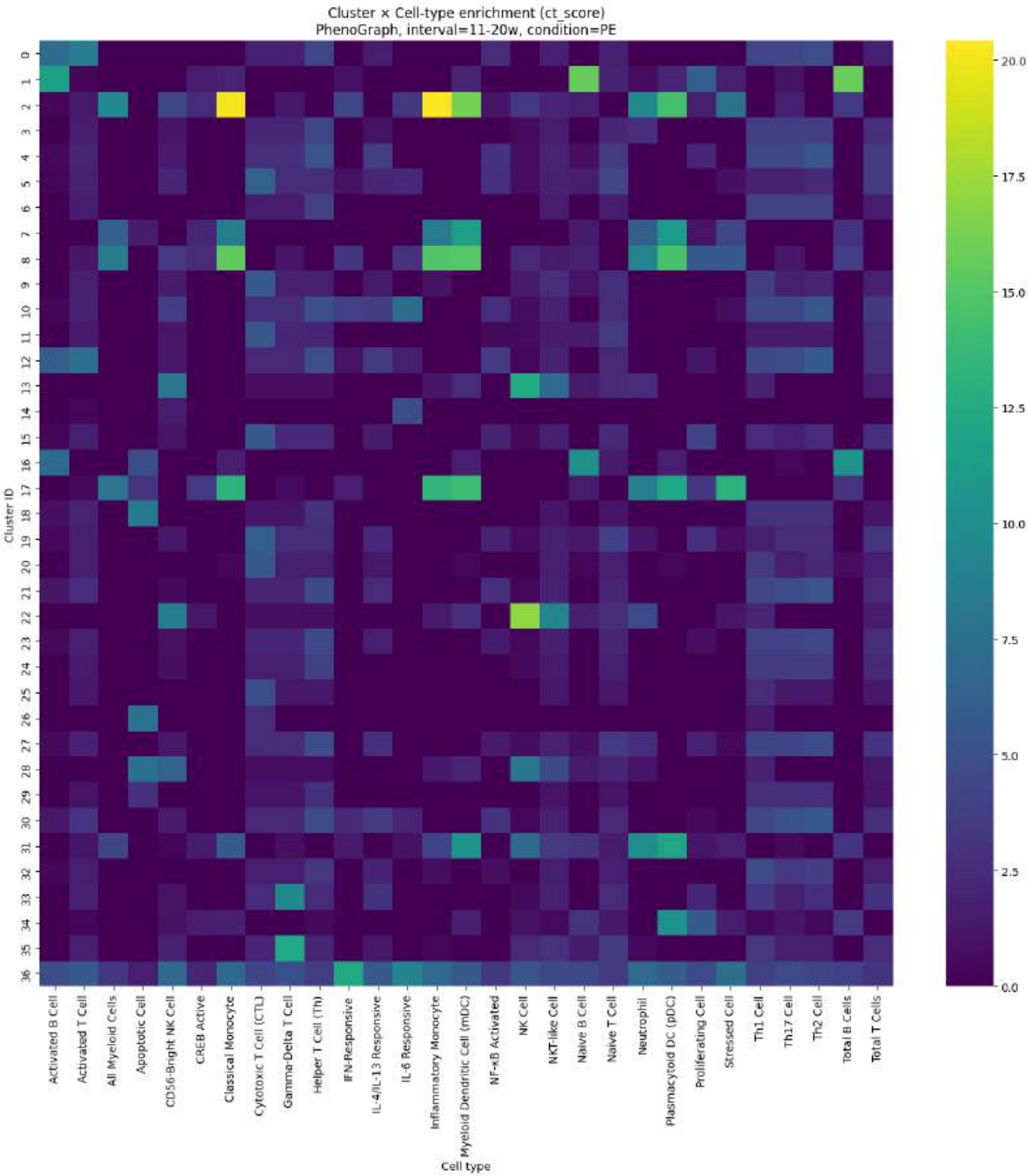
Barplot: PE vs Control



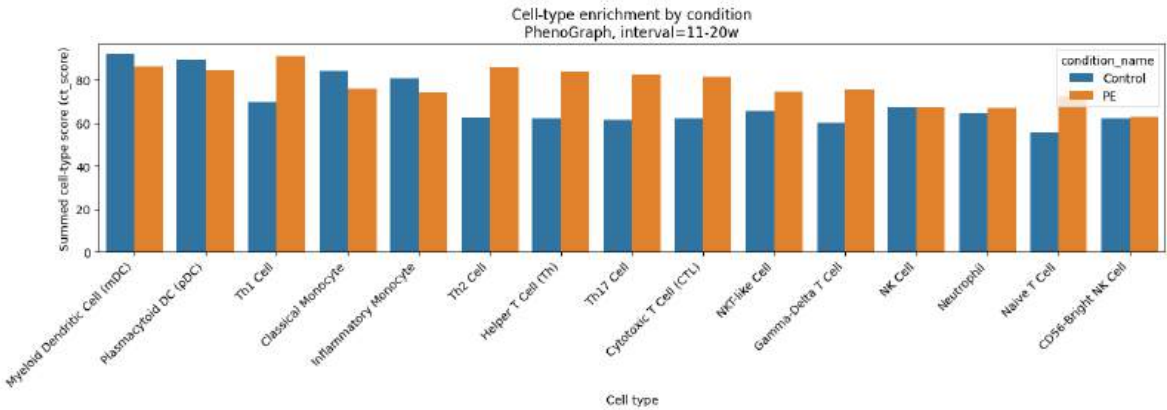
=== PhenoGraph | interval 11-20w ===
Heatmap: condition=0



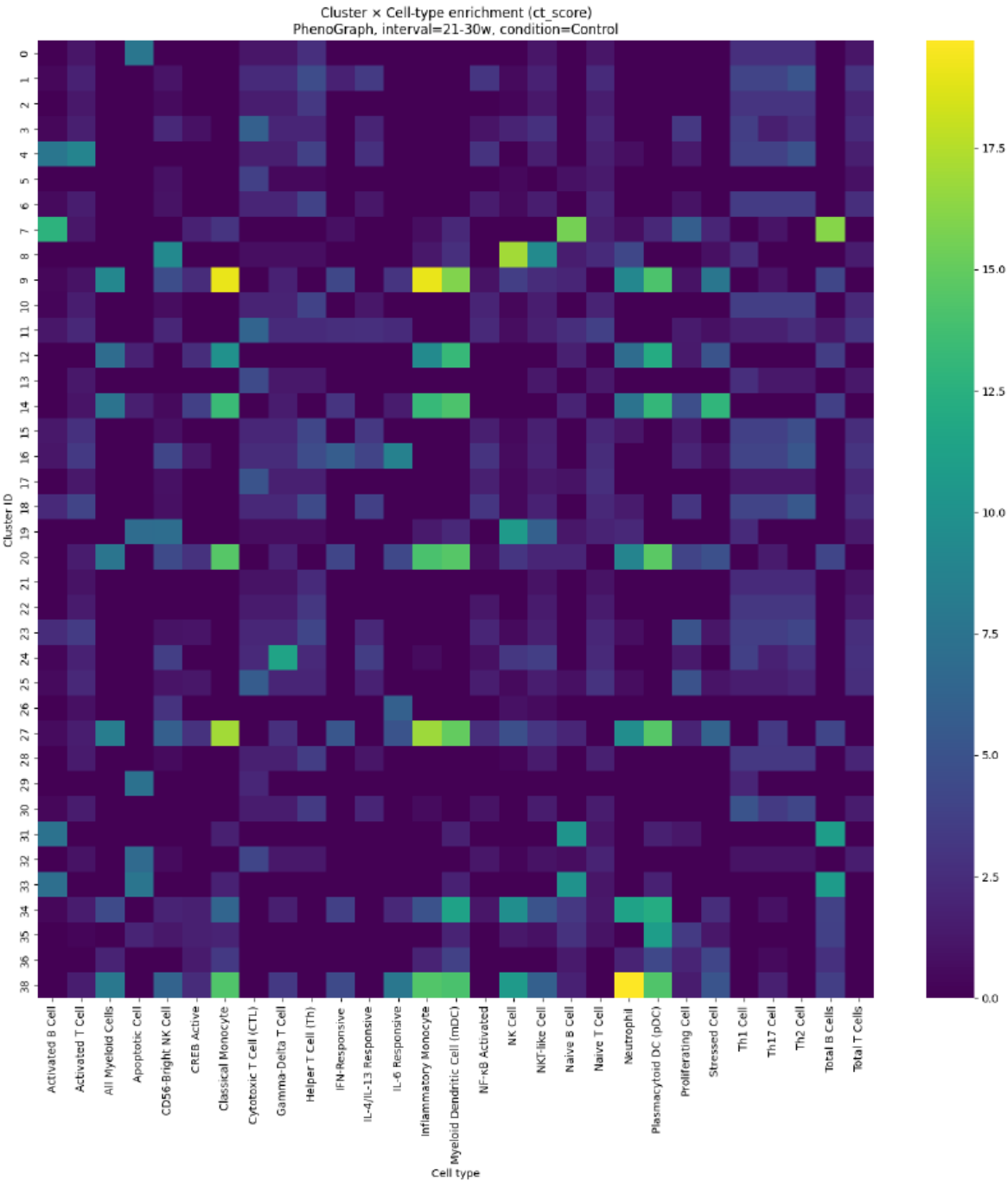
Heatmap: condition=1



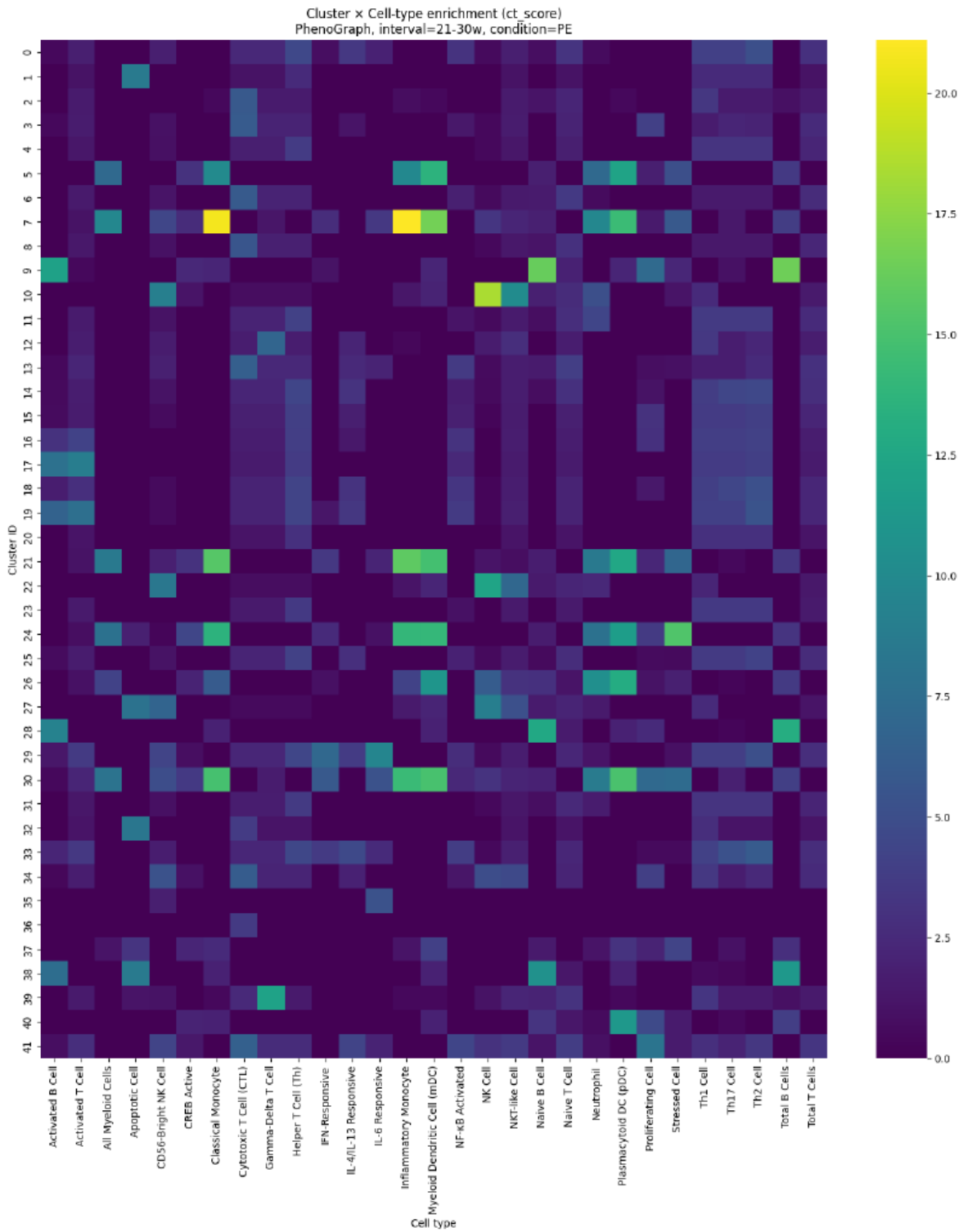
Barplot: PE vs Control



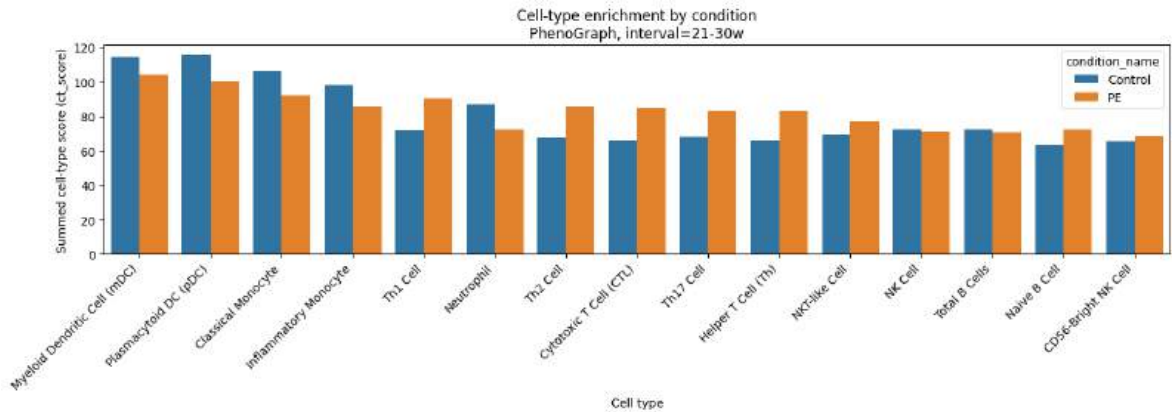
=== PhenoGraph | interval 21-30w ===
Heatmap: condition=0



Heatmap: condition=1



Barplot: PE vs Control



Minor Digression. EDA table summary before clustering for reporting purposes

```
In [ ]: eda_path = "/content/drive/MyDrive/pre eclampsia/eda.csv"
eda = pd.read_csv(eda_path)

print("EDA shape:", eda.shape)
```

EDA shape: (230000, 38)

```
In [ ]: # Normalize interval formatting to avoid 1-10w vs 1-10w mismatch
eda["age_bin_10w"] = (
    eda["age_bin_10w"]
    .astype(str)
    .str.replace("-", "-")
    .str.replace("-", "-")
)
```

```
In [ ]: # Total cells
total_cells = len(eda)

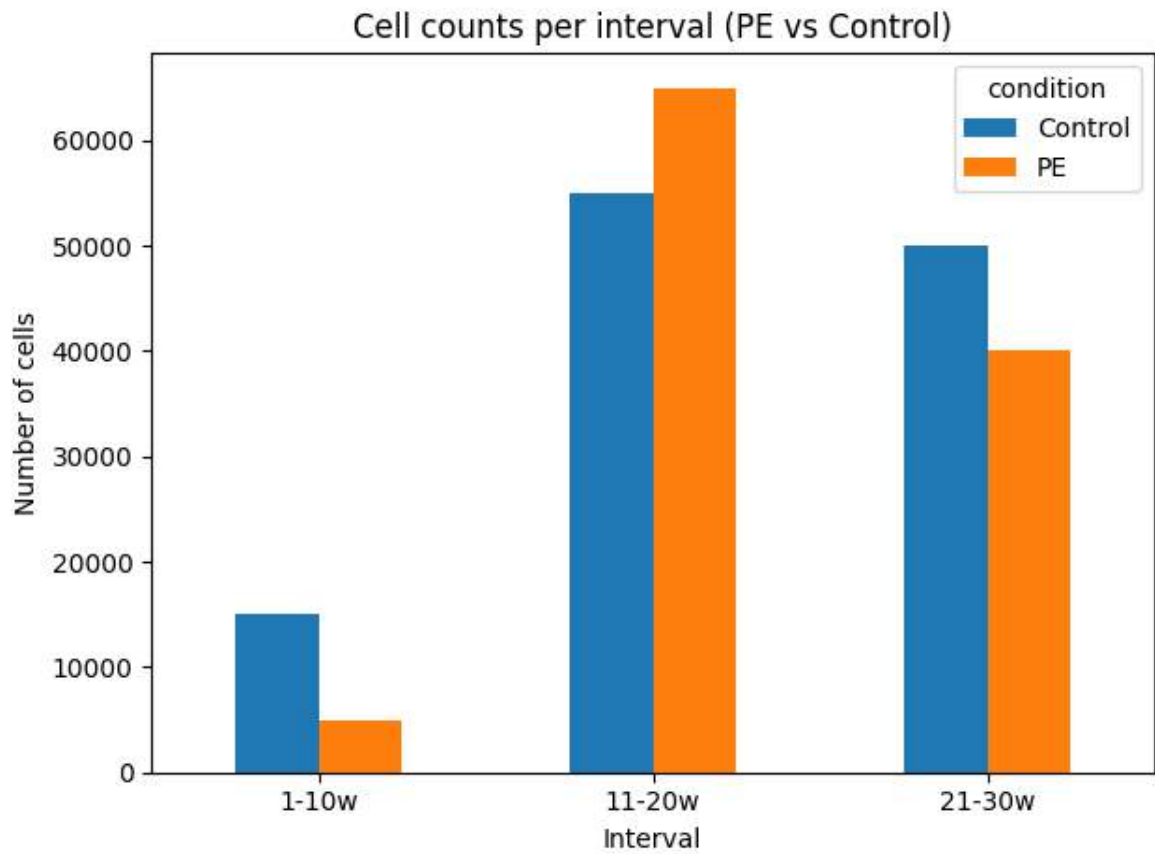
# Cells per interval
cells_per_interval = eda["age_bin_10w"].value_counts().sort_index()

# Cells per condition overall
cells_per_condition = eda["condition"].value_counts().rename({0: "Control", 1: "PE"})

# Cells per interval x condition
cells_interval_condition = (
    eda.groupby(["age_bin_10w", "condition"])
    .size()
    .unstack(fill_value=0)
    .rename(columns={0: "Control", 1: "PE"})
)
```

```
In [ ]: plt.figure(figsize=(8,5))
cells_interval_condition.plot(kind="bar", stacked=False)
plt.title("Cell counts per interval (PE vs Control)")
plt.ylabel("Number of cells")
plt.xlabel("Interval")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

<Figure size 800x500 with 0 Axes>



```
In [ ]: plt.figure(figsize=(5,5))
cells_per_condition.plot(kind="bar")
plt.title("Total PE vs Control cell counts")
plt.ylabel("Number of cells")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

